Starting with the CP

1. Ranges of input And Output

Data type	Range of input/output values
int	-2.15 x 10^9 to 2.15 x 10^9
unsigned int	0 to 4.29 x 10^9
long	-2.15 x 10^9 to 2.15 x 10^9
unsigned long	0 to 4.29 x 10^9
long long	-9.22 x 10^18 to 9.22 x 10^18
unsigned long long	0 to 18.44 x 10^18
float	±1.18 x 10^-38 to ±3.40 x 10^38
double	±2.23 x 10^-308 to ±1.79 x 10^308

Data type	Range of input/output values
char	-128 to 127
bool	**true** or **false**

Note that these ranges are based on typical implementations and may vary depending on the specific platform being used. It's always important to consult the documentation or specifications for the specific data types being used to get more accurate and detailed information about their ranges.

Mark as Read

Report An Issue

2. C++ Tricks For Competitive Programming

In competitive programming, it is important to read input as fast as possible so we save valuable time.

You must have seen various problem statements saying: "***Warning:*** *Large I/O data, be careful with certain languages (though most should be OK if the algorithm is well designed)*". The key for such problems is to use Faster I/O techniques.

It is often recommended to use scanf/printf instead of cin/cout for fast input and output. However, you can still use cin/cout and achieve the same speed as scanf/printf by including the following two lines in your main() function:

```
ios_base::sync_with_stdio(false);
```

It toggles on or off the synchronization of all the C++ standard streams with their corresponding standard C streams if it is called before the program performs its first input or output operation. Adding ios_base::sync_with_stdio (false); (which is true by default) before any I/O operation avoids this synchronization. It is a static member of the function of std::ios_base.

```
cin.tie(NULL);
```

tie() is a method that simply guarantees the flushing of std::cout before std::cin accepts an input. This is useful for interactive console programs which require the console to be updated constantly but also slows down the program for large I/O. The NULL part just returns a NULL pointer.

Moreover, you can include the standard template library (STL) with a single include:

```
    #include <bits/stdc++.h>
```

So your template for competitive programming could look like this:

```
#include <bits/stdc++.h>
using namespace std;

int main()
{
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    return 0;
}
```

It is recommended to use cout << "\n"; instead of cout << endl;. endl is slower because it forces a flushing stream, which is usually unnecessary (See this for details). (You'd need to flush if you were writing, say, an interactive progress bar, but not when writing a million lines of data.) Write '\n instead of endl.

Normal I/O: The code below uses cin and cout. The solution gets accepted with a runtime of 2.17 seconds.

C++

```
// A normal IO example code
#include <bits/stdc++.h>
using namespace std;
int main()
{
        int n, k, t;
        int cnt = 0;
        cin >> n >> k;
        for (int i=0; i<n; i++)
        {
                cin >> t;
                if (t % k == 0)
                        cnt++;
        }
        cout << cnt << "\n";
        return 0;
}
```

Fast I/O However, we can do better and reduce the runtime a lot by adding two lines. The program below gets accepted with a runtime of 0.41 seconds.

C++

```
// A fast IO program
#include <bits/stdc++.h>
using namespace std;

int main()
{
        // added the two lines below
        ios_base::sync_with_stdio(false);
        cin.tie(NULL);

        int n, k, t;
        int cnt = 0;
        cin >> n >> k;
        for (int i=0; i<n; i++)
        {
                cin >> t;
                if (t % k == 0)
                        cnt++;
        }
        cout << cnt << "\n";
        return 0;
}
```

3. Using Typedef and Macros in C++

Shortcode is ideal in competitive programming because programs should be written as fast as possible. Because of this, competitive programmers often define shorter names for datatypes and other parts of the code.
We here discuss the method of code shortening in C++ specifically.

Type names

Using the command *typedef* it is possible to give a shorter name to a datatype.
For example, the name long long is long, so we can define a shorter name ll:
typedef long long ll;
After this, the code

C++

```
long long a = 123456789;
long long b = 987654321;
cout << a * b << "\n";
```

can be shorted as follows:

C++
```
ll a = 123456789;
ll b = 987654321;
cout << a * b << "\n";
```

The command typedef can also be used with more complex types. For example, the following code gives the name vi for a vector of integers and the name pi for a pair that contains two integers,

C++
```
typedef vector<int> vi;
typedef pair<int, int> pi;
```

Macros

Another way to shorten code is to define **macros**. A macro means that certain strings in the code will be changed before the compilation. In C++, macros are defined using the #define keyword.

For example, we can define the following macros:

#define F first

#define S second

#define PB push_back

#define MP make_pair

After this, the code

v.push_back(make_pair(y1, x1)) ;

v.push_back(make_pair(y2, x2));

int d = v[i].first+v[i].second;

can be shorted as follows

v.PB(MP(y1, x1));

v.PB(MP(y2, x2));

int d = v[i].F+v[i].S;

A macro can also have parameters which makes it possible to shorten loops and others structures. For example, we can define the following macro:

#define REP(i, a, b) for (int i=a; i<=b; i++)

After this, the code

for (int i=1; i<=n; i++){

search(i);

}

can be shortened as follows:
REP(i, 1, n){
search(i);
}

A version of template given below

This can be use in competitive programming for faster coding.

C++

```
#include <iostream>
#include <math.h>
#include <string.h>
#define GFG 10
#define ans 0
#define n 20
#define sum 0
#define sum n + GFG
#define n1 10
#define i 10

using namespace std;

int main()
{
        // cin>>n1;
        for (int j = 0; j <= n1; j++) {
                cout << j * i << endl;
        }
}
```

C++

```
#include <bits/stdc++.h> // Include every standard library
using namespace std;

typedef long long LL;
typedef pair<int, int> pii;
typedef pair<LL, LL> pll;
typedef pair<string, string> pss;
typedef vector<int> vi;
typedef vector<vi> vvi;
typedef vector<pii> vii;
typedef vector<LL> vl;
typedef vector<vl> vvl;

double EPS = 1e-9;
int INF = 1000000005;
```

```
long long INFF = 10000000000000000005LL;
double PI = acos(-1);
int dirx[8] = { -1, 0, 0, 1, -1, -1, 1, 1 };
int diry[8] = { 0, 1, -1, 0, -1, 1, -1, 1 };

#ifdef TESTING
#define DEBUG fprintf(stderr, "====TESTING====\n")
#define VALUE(x) cerr << "The value of " << #x << " is " << x << endl
#define debug(...) fprintf(stderr, __VA_ARGS__)
#else
#define DEBUG
#define VALUE(x)
#define debug(...)
#endif

#define FOR(a, b, c) for (int(a) = (b); (a) < (c); ++(a))
#define FORN(a, b, c) for (int(a) = (b); (a) <= (c); ++(a))
#define FORD(a, b, c) for (int(a) = (b); (a) >= (c); --(a))
#define FORSQ(a, b, c) for (int(a) = (b); (a) * (a) <= (c); ++(a))
#define FORC(a, b, c) for (char(a) = (b); (a) <= (c); ++(a))
#define FOREACH(a, b) for (auto&(a) : (b))
#define REP(i, n) FOR(i, 0, n)
#define REPN(i, n) FORN(i, 1, n)
#define MAX(a, b) a = max(a, b)
#define MIN(a, b) a = min(a, b)
#define SQR(x) ((LL)(x) * (x))
#define RESET(a, b) memset(a, b, sizeof(a))
#define fi first
#define se second
#define mp make_pair
#define pb push_back
#define ALL(v) v.begin(), v.end()
#define ALLA(arr, sz) arr, arr + sz
#define SIZE(v) (int)v.size()
#define SORT(v) sort(ALL(v))
#define REVERSE(v) reverse(ALL(v))
#define SORTA(arr, sz) sort(ALLA(arr, sz))
#define REVERSEA(arr, sz) reverse(ALLA(arr, sz))
#define PERMUTE next_permutation
#define TC(t) while (t--)

inline string IntToString(LL a)
{
        char x[100];
        sprintf(x, "%lld", a);
        string s = x;
        return s;
}
```

```
inline LL StringToInt(string a)
{
	char x[100];
	LL res;
	strcpy(x, a.c_str());
	sscanf(x, "%lld", &res);
	return res;
}

inline string GetString(void)
{
	char x[1000005];
	scanf("%s", x);
	string s = x;
	return s;
}

inline string uppercase(string s)
{
	int n = SIZE(s);
	REP(i, n)
	if (s[i] >= 'a' && s[i] <= 'z')
		s[i] = s[i] - 'a' + 'A';
	return s;
}

inline string lowercase(string s)
{
	int n = SIZE(s);
	REP(i, n)
	if (s[i] >= 'A' && s[i] <= 'Z')
		s[i] = s[i] - 'A' + 'a';
	return s;
}

inline void OPEN(string s)
{
#ifndef TESTING
	freopen((s + ".in").c_str(), "r", stdin);
	freopen((s + ".out").c_str(), "w", stdout);
#endif
}

// end of Sektor_jr template v2.0.3 (BETA)

int main()
{
	freopen("A.in", "r", stdin);
	freopen("output.txt", "w", stdout);
```

```
        int a, b;
        fin >> a >> b;
        fout << a + b << endl;
        return 0;
}
```

4. IO Manipulation

C++ provides various I/O manipulators that can be used to format input/output operations. These manipulators can be used with std::cout and std::cin objects to format the output and input data respectively.

std::boolalpha - This manipulator is used to output boolean values as "true" or "false" instead of 1 or 0. For example, **std::cout << std::boolalpha << true;** will output **"true"**.

std::showpoint - This manipulator is used to display the decimal point in floating-point numbers even if there are no decimal places. For example, **std::cout << std::showpoint << 3.0;** will output **"3.0"**.

std::uppercase - This manipulator is used to output hexadecimal numbers in uppercase letters. For example, **std::cout << std::hex << std::uppercase << 255;** will output **"FF"**.

std::nouppercase - This manipulator is used to output hexadecimal numbers in lowercase letters. For example, **std::cout << std::hex << std::nouppercase << 255;** will output **"ff"**.

std::hex - This manipulator is used to output integer values in hexadecimal format. For example, **std::cout << std::hex << 255;** will output **"ff"**.

std::dec - This manipulator is used to output integer values in decimal format. For example, **std::cout << std::dec << 255;** will output **"255"**.

std::oct - This manipulator is used to output integer values in octal format. For example, **std::cout << std::oct << 255;** will output **"377"**.

Example -

C++

```
// Code implementation showing the use of hex, dec and oct

#include <iostream>
```

```cpp
#include <iomanip>

using namespace std;

int main() {
    cout << hex << 123 << endl;       // Outputs: 7b
    cout << oct << 456 << endl;       // Outputs: 710
    cout << dec << 789 << endl;       // Outputs: 789
}
```

Output

```
7b
710
789
```

These are just a few more examples of I/O manipulators in C++. By using these manipulators and others, you can customize the output of your data to fit your specific needs.

Prefix Sum, Two pointer, Sliding Window

1. Prefix Sum Introduction

Given an array arr[] of size n, its prefix sum array is another array prefixSum[] of the same size, such that the value of prefixSum[i] is arr[0] + arr[1] + arr[2] ... arr[i].

Examples :

```
Input  : arr[] = {10, 20, 10, 5, 15}
Output : prefixSum[] = {10, 30, 40, 45, 60}

Explanation : While traversing the array, update
the element by adding it with its previous element.
prefixSum[0] = 10,
prefixSum[1] = prefixSum[0] + arr[1] = 30,
prefixSum[2] = prefixSum[1] + arr[2] = 40 and so on.
```

To fill the prefix sum array, we run through index 1 to last and keep on adding the present element with the previous value in the prefix sum array.
Below is the implementation :

C++Java

```cpp
// C++ program for Implementing
// prefix sum array
#include <bits/stdc++.h>
using namespace std;

// Fills prefix sum array
void fillPrefixSum(int arr[], int n, int prefixSum[])
{
    prefixSum[0] = arr[0];

    // Adding present element
    // with previous element
    for (int i = 1; i < n; i++)
        prefixSum[i] = prefixSum[i - 1] + arr[i];
}

// Driver Code
int main()
{
    int arr[] = { 10, 4, 16, 20 };
    int n = sizeof(arr) / sizeof(arr[0]);
    int prefixSum[n];

    fillPrefixSum(arr, n, prefixSum);
    for (int i = 0; i < n; i++)
        cout << prefixSum[i] << " ";
}
```

Output:

```
10 14 30 50
```

2. Prefix Sum Implementation

Given an array arr[] of size n. Given Q queries and in each query given L and R, print sum of array elements from index L to R.

```
Input : n = 6
        a[ ] = {3, 6, 2, 8, 9, 2}
        q = 4
        l = 2, r = 3.
        l = 4, r = 6.
```

```
        l = 1, r = 5.
        l = 3, r = 6.
```

Implementation :

C++14Java

```cpp
#include <iostream>
using namespace std;
const int N = 1e5+10;
int a[N];
int pf[N];

int main() {
int n;
cin >> n;
for(int i=1; i<=n; i++){
    cin >> a[i];
    if(i==0)
        pf[i]=a[i];
    else
        pf[i] = pf[i-1] + a[i];
    //cout<<pf[i];
}
int q;
cin >> q;
while(q--){
    int l, r;
    cin >> l >> r;
//Calculating sum from l to r.
    if(r>n || l<1)
    {
        cout<<"Please input in range 1 to "<<n<<endl;
        break;
    }
    if(l==1)
        cout << pf[r] << endl;
    else
        cout << pf[r] - pf[l-1] << endl;

}
return 0;
}
```

Output :

```
8
19
28
21
```

Time Complexity: O(n)

Equilibrium Point

Equilibrium index of an array using Prefix-Sum:

To solve the problem follow the below idea:

This is a quite simple and straightforward method. The idea is to take the prefix sum of the array twice. Once from the front end of the array and another from the back end of the array. After taking both prefix sums run a loop and check for some i if both the prefix sum from one array is equal to prefix sum from the second array then that point can be considered as the Equilibrium point

Follow the given steps to solve the problem:

- Declare two arrays to store the prefix sum of the array from the front and the back
- Run a loop from 1 to N and check if at any point prefix sum of the array from the front is equal to the prefix sum of the array from the back
- If any such index is found then return that index
- Else return -1

Below is the implementation of the above approach:

C++Java

```
// C++ program to find equilibrium index of an array
#include <bits/stdc++.h>
using namespace std;

int equilibrium(int a[], int n)
{
    if (n == 1)
        return (0);
    int forward[n] = { 0 };
    int rev[n] = { 0 };

    // Taking the prefixsum from front end array
  //As We Know that in prefixsum from front end the first prefixsum will
be equal to first element in the array
  //As well as from the back end the first prefixsum will be equal to last
element in the array
   forward[0]=a[0];
 rev[n-1]=a[n-1];
```

```
    for (int i = 1; i < n; i++) {
            forward[i] = forward[i - 1] + a[i];

    }

    // Taking the prefixsum from back end of array
    for (int i = n - 2; i >= 0; i--) {

            rev[i] = rev[i + 1] + a[i];

    }

    // Checking if forward prefix sum
    // is equal to rev prefix
    // sum
    for (int i = 0; i < n; i++) {
        if (forward[i] == rev[i]) {
            return i;
        }
    }
    return -1;

    // If You want all the points
    // of equilibrium create
    // vector and push all equilibrium
    // points in it and
    // return the vector
}

// Driver code
int main()
{
    int arr[] = { -7, 1, 5, 2, -4, 3, 0 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function call
    cout << "First Point of equilibrium is at index "
         << equilibrium(arr, n) << "\n";
    return 0;
}
```

Output:

```
First Point of equilibrium is at index 3
```

Time Complexity: O(N)
Auxiliary Space: O(N)

Two Pointers Technique

Two pointers is really an easy and effective technique that is typically used for searching pairs in a sorted array.
Given a sorted array A (sorted in ascending order), having N integers, find if there exists any pair of elements (A[i], A[j]) such that their sum is equal to X.

Illustration :

```
A[] = {10, 20, 35, 50, 75, 80}
X = =70
i = 0
j = 5

A[i] + A[j] = 10 + 80 = 90
Since A[i] + A[j] > X, j--
i = 0
j = 4

A[i] + A[j] = 10 + 75 = 85
Since A[i] + A[j] > X, j--
i = 0
j = 3

A[i] + A[j] = 10 + 50 = 60
Since A[i] + A[j] < X, i++
i = 1
j = 3
m
```

```
A[i] + A[j] = 20 + 50 = 70
Thus this signifies that Pair is Found.
```

Let us do discuss the working of two pointer algorithm in brief which is as follows. The algorithm basically uses the fact that the input array is sorted. We start the sum of extreme values (smallest and largest) and conditionally move both pointers. We move left pointer 'i' when the sum of A[i] and A[j] is less than X. We do not miss any pair because the sum is already smaller than X. Same logic applies for right pointer j.

Methods:

Here we will be proposing a two-pointer algorithm by starting off with the naïve approach only in order to showcase the execution of operations going on in both methods and secondary to justify how two-pointer algorithm optimizes code via time complexities across all dynamic programming languages such as C+, Java, Python, and even JavaScript

1. Naïve Approach using loops
2. Optimal approach using two pointer algorithm

Implementation:

Method 1: Naïve Approach

Examples

C++CJavaPython3C#Javascript

```cpp
// C++ Program Illustrating Naive Approach to
// Find if There is a Pair in A[0..N-1] with Given Sum

// Importing all libraries
#include <bits/stdc++.h>

using namespace std;

bool isPairSum(int A[], int N, int X)
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            // as equal i and j means same element
            if (i == j)
                continue;

            // pair exists
            if (A[i] + A[j] == X)
                return true;

            // as the array is sorted
            if (A[i] + A[j] > X)
                break;
        }
    }

    // No pair found with given sum.
    return false;
}

// Driver code
int main()
{
    int arr[] = { 2, 3, 5, 8, 9, 10, 11 };
    int val = 17;
    int arrSize = *(&arr + 1) - arr;
```

```
    sort(arr, arr + arrSize); // Sort the array
    // Function call
    cout << isPairSum(arr, arrSize, val);

    return 0;
}
```

Output

```
1
```

Time Complexity: $O(n^2)$.

Method 2: Two Pointers Technique

Now let's see how the two-pointer technique works. We take two pointers, one representing the first element and other representing the last element of the array, and then we add the values kept at both the pointers. If their sum is smaller than X then we shift the left pointer to right or if their sum is greater than X then we shift the right pointer to left, in order to get closer to the sum. We keep moving the pointers until we get the sum as X.

Examples

C++CJavaPython3C#Javascript

```cpp
// C++ Program Illustrating Naive Approach to
// Find if There is a Pair in A[0..N-1] with Given Sum
// Using Two-pointers Technique

// Importing required libraries
#include <iostream>
#include <algorithm>

using namespace std;

// Two pointer technique based solution to find
// if there is a pair in A[0..N-1] with a given sum.
int isPairSum(int A[], int N, int X)
{
    // represents first pointer
    int i = 0;

    // represents second pointer
    int j = N - 1;

    while (i < j) {

        // If we find a pair
        if (A[i] + A[j] == X)
            return 1;

        // If sum of elements at current
        // pointers is less, we move towards
        // higher values by doing i++
        else if (A[i] + A[j] < X)
            i++;

        // If sum of elements at current
        // pointers is more, we move towards
        // lower values by doing j--
        else
            j--;
    }
    return 0;
}

// Driver code
int main()
{
    // array declaration
    int arr[] = { 2, 3, 5, 8, 9, 10, 11 };
```

```
    // value to search
    int val = 17;

    // size of the array
    int arrSize = *(&arr + 1) - arr;

      // array should be sorted before using two-pointer technique
      sort(arr, arr+7);

    // Function call
    cout << (bool)isPairSum(arr, arrSize, val);

    return 0;
}
```

Output
1

Time Complexity: O(n)

More problems based on two pointer technique.

- Find the closest pair from two sorted arrays
- Find the pair in array whose sum is closest to x
- Find all triplets with zero sum
- Find a triplet that sum to a given value
- Find a triplet such that sum of two equals to third element
- Find four elements that sum to a given value

Number Theory Basics

Floating Point Default Print Format

Default Printing Format

- No trailing zeros
- Precision means total digits(excluding the digits used after e).
- Default Precision value is 6.
- When value before decimal point does not fit in 6 digits, power format is used. For example - 1234568.3 is printed as 1.23457e+06.

C++

```
#include<iostream>
using namespace std;
int main()
{
    double x = 1.2300;
    cout << x << '\n';
    double y = 1567.56732;
    cout << y << '\n';
    double z = 1244567.45;
    cout << z << '\n';
    double w = 124456.67;
    cout << w << '\n';
    double u = 123e+2;
    cout << u << '\n';
    return 0;
}
```

Output

```
1.23
1567.57
1.24457e+06
124457
12300
```

Floating point Manipulating Default Format

Manipulating Default Format

setprecision(n) - Changes the default precision.

showpoint() - shows trailing zeros, noshowpoint() reverts it.

showpos() - Show + sign for positive values. no showpos() reverts it.

uppercase() - prints 'e' as 'E'. nouppercase() reverts it.

C++

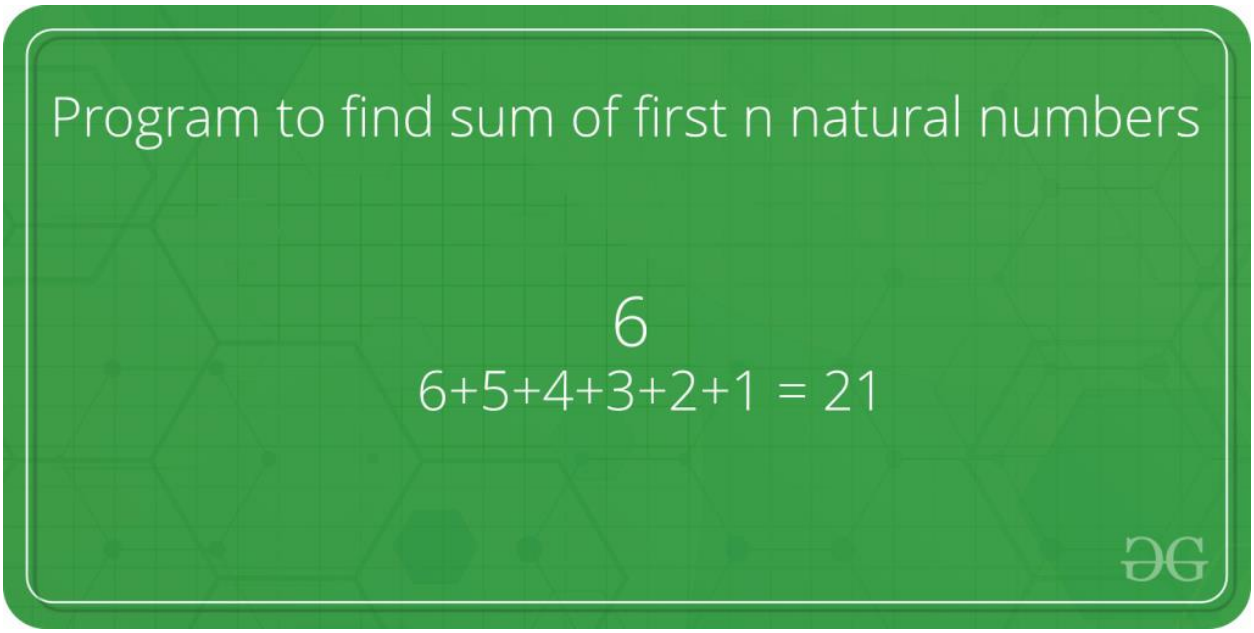
```
#include<iostream>
#include<iomanip>
using namespace std;
int main()
{
    cout << setprecision(4);
    double x = 15.5683, y = 34267.1;
    cout << x << ' ' << y << ' ' << '\n';
    double z = 1.23;
    cout << showpoint << z << '\n';
    cout << showpos   << z << '\n';
    cout << uppercase << y << '\n';
    return 0;
}
```

Output

```
15.57 3.427e+04
1.230
+1.230
+3.427E+04
```

Sum of N Natural Numbers

Given a number n, find the sum of first natural numbers.



Examples :

```
Input : n = 3
Output : 6
Explanation :
Note that 1 + 2 + 3 = 6

Input  : 5
Output : 15
Explanation :
Note that 1 + 2 + 3 + 4 + 5 = 15
```

A **simple solution** is to do the following.

```
1) Initialize : sum = 0
2) Run a loop from x = 1 to n and
   do following in loop.
     sum = sum + x
```

C++

```
// CPP program to find sum of first
// n natural numbers.
#include<iostream>
using namespace std;

// Returns sum of first n natural
// numbers
int findSum(int n)
{
   int sum = 0;
   for (int x=1; x<=n; x++)
```

```
      sum = sum + x;
    return sum;
}

// Driver code
int main()
{
  int n = 5;
  cout << findSum(n);
  return 0;
}
```

Output
15

Time Complexity: *O(n)*

Auxiliary Space: *O(1)*

An **efficient solution** is to use the below formula.

Sum of first n natural numbers = (n * (n + 1)) / 2

Examples :
n = 5
Sum = (5 * (5 + 1)) / 2 = (5 * 6) / 2 = 30/2 = 15

n = 10
Sum = (10 * (10 + 1)) / 2 = (10 * 11) / 2 = 110/2 = 55

How does this work?

We can prove this formula using induction.

It is true for n = 1 and n = 2
For n = 1, sum = 1 * (1 + 1)/2 = 1
For n = 2, sum = 2 * (2 + 1)/2 = 3

Let it be true for k = n-1.

Sum of k numbers = (k * (k+1))/2
Putting k = n-1, we get

```
Sum of k numbers = ((n-1) * (n-1+1))/2
                 = (n - 1) * n / 2

If we add n, we get,
Sum of n numbers = n + (n - 1) * n / 2
                 = (2n + n² - n)/2
                 = n * (n + 1)/2
```

C++

```cpp
// Efficient CPP program to find sum of first
// n natural numbers.
#include<iostream>
using namespace std;

// Returns sum of first n natural
// numbers
int findSum(int n)
{
   return n * (n + 1) / 2;
}

// Driver code
int main()
{
  int n = 5;
  cout << findSum(n);
  return 0;
}
```

Output

```
15
```

Time Complexity: *O(1)*

Auxiliary Space: *O(1)*

Sum of Squares

Given n, find sum of squares of first n natural numbers.

Examples :

```
Input : n = 2
Output : 5
Explanation: 1^2+2^2 = 5

Input : n = 8
```

```
Output : 204
Explanation :
1^2 + 2^2 + 3^2 + 4^2 + 5^2 + 6^2 + 7^2 + 8^2 = 204
```

Naive approach :

A naive approach will be to run a loop from 1 to n and sum up all the squares.

C++

```cpp
// CPP program to calculate
// 1^2+2^2+3^2+...
#include <iostream>
using namespace std;

// Function to calculate sum
int summation(int n)
{
        int sum = 0;
        for (int i = 1; i <= n; i++)
                sum += (i * i);

        return sum;
}

// Driver code
int main()
{
        int n = 2;
        cout << summation(n);
        return 0;
}
```

Output

5

Time Complexity: O(n)

Auxiliary Space: O(1), since no extra space has been taken.

Efficient Approach :

There exists a formula for finding the sum of squares of first n numbers.

$1 + 2 + \ldots\ldots\ldots + n = n(n+1) / 2$

$1^2 + 2^2 + \ldots\ldots\ldots + n^2 = n(n+1)(2n+1) / 6$

```
n * (n + 1) * (2*n + 1) / 6

Example : n = 3
        = 3 * (3 + 1) * (2*3 + 1) / 6
```

```
        = (3 * 4 * 7) / 6
        = 84 / 6
        = 14
```

How does this work?

```
We can prove this formula using induction.
We can easily see that the formula is true for
n = 1 and n = 2 as sums are 1 and 5 respectively.

Let it be true for n = k-1. So sum of k-1 numbers
is (k - 1) * k * (2 * k - 1)) / 6

In the following steps, we show that it is true
for k assuming that it is true for k-1.

Sum of k numbers = Sum of k-1 numbers + k²
           = (k - 1) * k * (2 * k - 1) / 6 + k²
           = ((k² - k) * (2*k - 1) + 6k²)/6
           = (2k³ - 2k² - k² + k + 6k²)/6
           = (2k³ + 3k² + k)/6
           = k * (k + 1) * (2*k + 1) / 6
```

C++

```cpp
// C++ program to get the sum
// of the following series
#include <iostream>
using namespace std;

// Function calculating
// the series
int summation(int n)
{
        return (n * (n + 1) *
                (2 * n + 1)) / 6;
}

// Driver Code
int main()
{
        int n = 10;
        cout << summation(n) << endl;
        return 0;
}

// This code is contributed by shubhamsingh10
```

Output

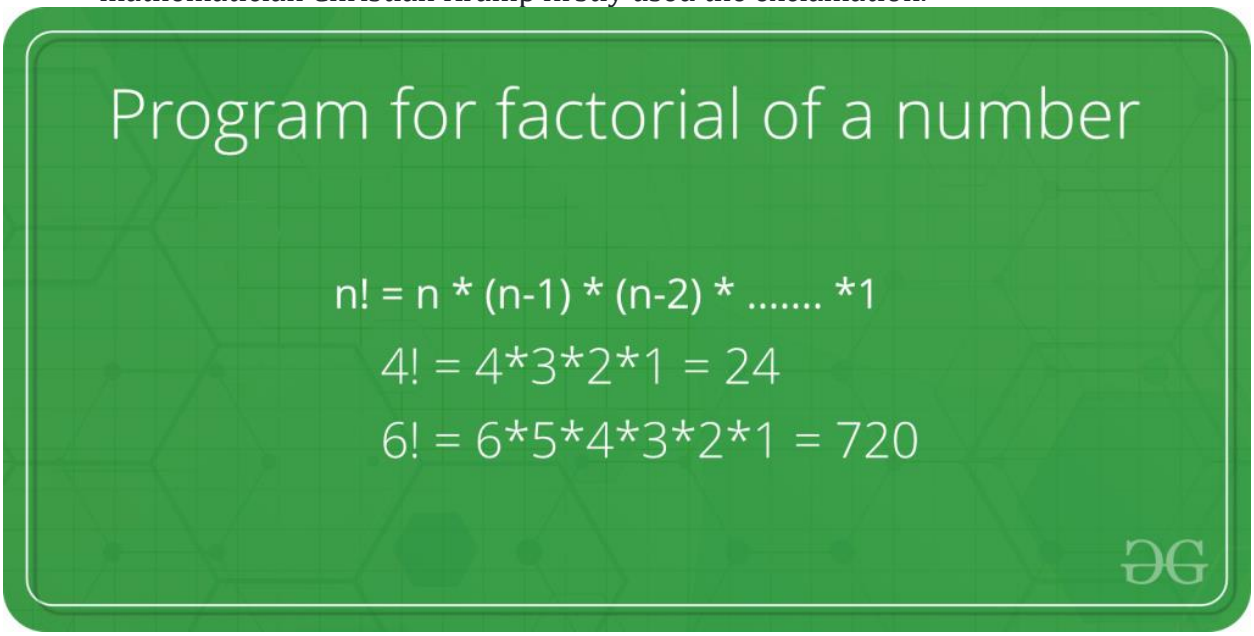
```
385
```

Time Complexity: O(1)
Auxiliary Space: O(1)

Factorial of A Number

What is the factorial of a number?

- Factorial of a non-negative integer is the multiplication of all positive integers smaller than or equal to n. For example factorial of 6 is 6*5*4*3*2*1 which is 720.
- A factorial is represented by a number and a ”!” mark at the end. It is widely used in permutations and combinations to calculate the total possible outcomes. A French mathematician Christian Kramp firstly used the exclamation.



C++

```cpp
#include<iostream>
using namespace std;

int main()
{
    int n, ans = 1;
    cin >> n;
    while(n>0)
    {
        ans *= n;
        n--;
    }
    cout << ans;
```

```
    return 0;
}
```

INPUT :

```
7
```

OUTPUT :

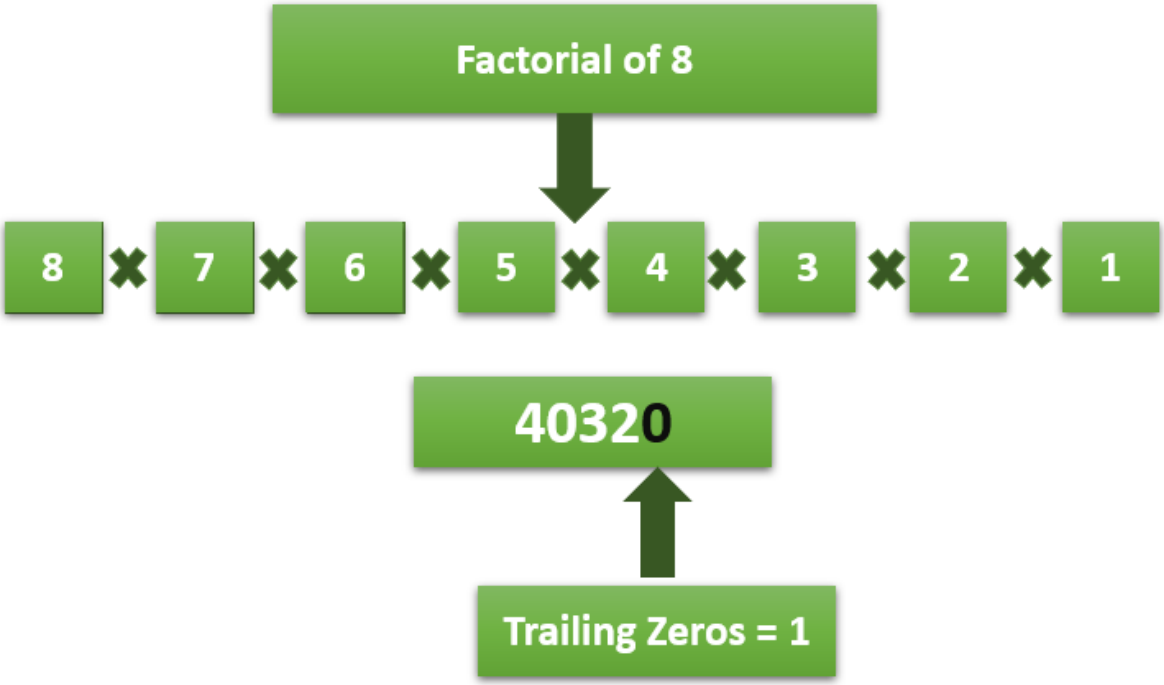
```
5040
```

Trailing Zeros in Factorial

In a realm where numbers hold secrets, a captivating challenge awaits, which is to, **Count Trailing Zeros in Factorial !!!**

Our Task: We are given a number. The task is to find the Number of Trailing Zeros in the factorial of the number.

The **Trailing Zeros** are the Zeros, which appear at the end of a number(factorial in that case)



Examples :

```
Input: 5
Output: 1
// Factorial of 5 = 5*4*3*2*1 = 120, which has one trailing 0.

Input: 20
Output: 4
// Factorial of 20 = 20*19*18*.... 3*2*1 = 2432902008176640000
which has 4 trailing zeroes.

Input: 100
Output: 24
```

We have 2 approaches to solve the problem: **Naive Approach** & **Efficient Approach**

1) Naive Approach

A simple method is to first calculate the factorial of n, then count trailing 0s in the result (We can count trailing 0s by repeatedly dividing the factorial by 10 till the remainder is not 0).

But, this method can cause overflow for slightly bigger numbers as the factorial of a number is a big number. So, we prefer the Efficient Approach

2) Efficient Approach

The idea is to consider prime factors of a factorial n. A trailing zero is always produced by prime factors 2 and 5. Our task is done if we can **count the number of 5s and 2s**. Consider the following examples:

- **n = 5:** There is one 5 and 3 2s in prime factors of 5! (2 * 2 * 2 * 3 * 5). So a count of trailing 0s is 1.
- **n = 11:** There are two 5s and eight 2s in prime factors of 11! ($2^8 * 3^4 * 5^2 * 7 * 11$). So the count of trailing 0s is 2.

We can easily observe that the number of 2s in prime factors is always more than or equal to the number of 5s. So if we **count 5s in prime factors**, we are done.

Following is the summarized formula for counting trailing 0s:

Trailing 0s in n! = Count of 5s in prime factors of n! = floor(n/5) + floor(n/25) + floor(n/125) + ...

The implementation is shown below:

C++Java

```
// C++ program to count
// trailing 0s in n!
#include <iostream>
using namespace std;

// Function to return trailing
// 0s in factorial of n
int findTrailingZeros(int n)
{
        if (n < 0) // Negative Number Edge Case
                return -1;

        // Initialize result
        int count = 0;

        // Keep dividing n by powers of
        // 5 and update count
        for (int i = 5; n / i >= 1; i *= 5)
                count += n / i;

        return count;
}

// Driver Code
int main()
```

```
{
        int n = 100;
        cout << "Count of trailing 0s in " << 100 << "! is "
                << findTrailingZeros(n);
        return 0;
}
```

Output :

```
Count of trailing 0s in 100! is 24
```

Time Complexity: O($\log_5 n$)

Auxiliary Space: O(1)

Maximum Power Dividing Factorial

Given two numbers, **fact** and **n**, find the largest power of n that divides **fact!** (Factorial of fact).

Examples:

Input :
fact = 5, n = 2
Output :
3
Explanation:
Value of 5! is 120. The largest power
of 2 that divides 120 is 8 (or 2^3

Input :
fact = 146, n = 15
Output :
35

The idea is based on Legendre's formula which finds largest power of a prime number that divides fact!. We find all prime factors of n. For every prime factor we find largest power of it that divides fact!. Finally we return minimum of all found powers.

Illustration :

```
fact = 146, n=15
First find the prime factor of 15 that are 3
and 5 then first divide with 3 and add i.e.
```

```
Applying Legendre's formula for prime
factor 3.
[146/3]+[48/3]+[16/3]+[5/3]+[1/3] = 70
   48  +   16  +  5  +  1  +  0   = 70
There is 70 is maximum power of 3 prime number.
146! is divisible by 3^70 which is maximum.

Applying Legendre's formula for prime
factor 5.
[146/5]+[29/5]+[5/5]+[1/5] = 35
   29  +   5  +  1  +  0   = 35
There is 35 is maximum power of 5 prime
number.
```

Minimum of two powers is 35 which is our answer.

Below is the implementation of the above approach:

C++

```cpp
// C++ program to count
// maximum power of n that divides a factorial
#include <iostream>
using namespace std;
int maxPF(int n)
{
  // to find the largest prime number that divides n
   int res= -1;
   for(int i=2; i*i<=n; i++)
  {
        while(n%i == 0)
       {
          res= i;
          n /= i;
       }

  }
         if(n > 1)
         res=n;
         return res;
}
int largestPow(int n, int fact)
{
       // to find maximum power of n which can divide fact number
       int p = maxPF(n);
       int res= 0;
       while(fact > 0)
       {
         fact = fact/p;
```

```
            res += fact;
        }
        return res;
}

// Driver Code

int main() {
        int n = 6,fact=20;
    cout<<largestPow(n,fact)<<endl;
        return 0;
}
```

Output
8

BIT MASKING

BITWISE OPERATOR

An **operator** is a symbol that operates on a value to perform specific mathematical or logical computations. They form the foundation of any programming language. In C++, we have built-in operators to provide the required functionality.

An operator operates the **operands**. For example,

```
int c = a + b;
```

Here, '+' is the addition operator. 'a' and 'b' are the operands that are being 'added'.

Operators in C++ can be classified into 6 types:

1. Arithmetic Operators
2. Relational Operators
3. Logical Operators
4. **Bitwise Operators**
5. Assignment Operators
6. Ternary or Conditional Operators

Bitwise Operators

These operators are used to perform bit-level operations on the operands. The operators are first converted to bit-level and then the calculation is performed on the operands. The

mathematical operations such as addition, subtraction, multiplication, etc. can be performed at the bit-level for faster processing.

Name	Symbol	Description	Example

Binary AND	&	Copies a bit to the evaluated result if it exists in both operands	int a = 2, b = 3;
			
			(a & b);
			//returns 2

Binary OR	|	Copies a bit to the evaluated result if it exists in any of the operand	int a = 2, b = 3;
			
			(a | b);
			//returns 3

Binary XOR	^	Copies the bit to the evaluated result if it is present in either of the operands but not both	int a = 2, b = 3;
			
			(a ^ b);
			//returns 1

Left Shift	<<	Shifts the value to left by the number of bits specified by the right operand.	int a = 2, b = 3;
			
			(a << 1);
			//returns 4

Right Shift	>>	Shifts the value to right by the number of bits specified by the right operand.	int a = 2, b = 3;
			
			(a >> 1);
			//returns 1

Name	Symbol	Description	Example

One's Complement	~	Changes binary digits 1 to 0 and 0 to 1	int b = 3;
			
			(~b); //returns -4

Note: *Only* ***char*** *and* ***int*** *data types can be used with Bitwise Operators.*

Example:

C++

```
// CPP Program to demonstrate the Relational Operators
#include <iostream>
using namespace std;

int main()
{
        int a = 6, b = 4;

        // Equal to operator
        cout << "a == b is " << (a == b) << endl;

        // Greater than operator
        cout << "a > b is " << (a > b) << endl;

        // Greater than or Equal to operator
        cout << "a >= b is " << (a >= b) << endl;

        // Lesser than operator
        cout << "a < b is " << (a < b) << endl;

        // Lesser than or Equal to operator
        cout << "a <= b is " << (a <= b) << endl;

        // true
        cout << "a != b is " << (a != b) << endl;

        return 0;
}
```

Output

```
a == b is 0
a > b is 1
a >= b is 1
```

```
a < b is 0
a <= b is 0
a != b is 1
```

NOTE -

The left shift and right shift operators should not be used for negative numbers.

The bitwise OR of two numbers is just the sum of those two numbers if there is no carry involved, otherwise you just add their bitwise AND.

C++

```cpp
#include <iostream>
using namespace std;

// Function to return the only odd
// occurring element
int findOdd(int arr[], int n)
{
        int res = 0, i;
        for (i = 0; i < n; i++)
                res ^= arr[i];
        return res;
}

// Driver Method
int main(void)
{
        int arr[] = { 12, 12, 14, 90, 14, 14, 14 };
        int n = sizeof(arr) / sizeof(arr[0]);
        cout << "The odd occurring element is "<< findOdd(arr, n);
        return 0;
}
```

Output

```
The odd occurring element is 90
```

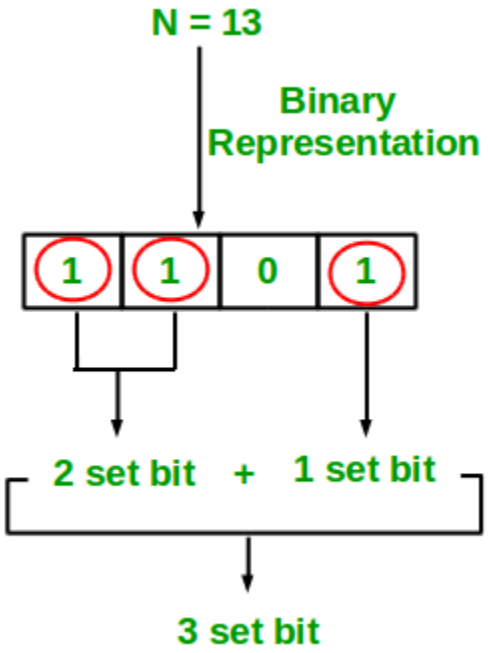
Count Set Bits

Write an efficient program to count the number of 1s in the binary representation of an integer.

Examples :

Input : *n = 6*
Output : *2*
Binary representation of 6 is 110 and has 2 set bits

Input : *n = 13*
Output : *3*
Binary representation of 13 is 1101 and has 3 set bit



Simple Method Loop through all bits in an integer, check if a bit is set and if it is, then increment the set bit count. See the program below.

C++Java

```
// C++ program to Count set
// bits in an integer
#include <bits/stdc++.h>
using namespace std;

/* Function to get no of set bits in binary
representation of positive integer n */
unsigned int countSetBits(unsigned int n)
{
        unsigned int count = 0;
        while (n) {
                count += n & 1;
                n >>= 1;
        }
        return count;
}
```

```
/* Program to test function countSetBits */
int main()
{
        int i = 9;
        cout << countSetBits(i);
        return 0;
}
```

Output

```
2
```

Time Complexity: O(log n)
Auxiliary Space: O(1)

Brian Kernighan's Algorithm:
Subtracting 1 from a decimal number flips all the bits after the rightmost set bit(which is 1) including the rightmost set bit.
for example :
10 in binary is 00001010
9 in binary is 00001001
8 in binary is 00001000
7 in binary is 00000111
So if we subtract a number by 1 and do it bitwise & with itself (n & (n-1)), we unset the rightmost set bit. If we do n & (n-1) in a loop and count the number of times the loop executes, we get the set bit count.
The beauty of this solution is the number of times it loops is equal to the number of set bits in a given integer.

```
1  Initialize count: = 0
2  If integer n is not zero
   (a) Do bitwise & with (n-1) and assign the value back to n
       n: = n&(n-1)
   (b) Increment count by 1
   (c) go to step 2
3  Else return count
```

Example for Brian Kernighan's Algorithm:

```
n =  9 (1001)
count = 0

Since 9 > 0, subtract by 1 and do bitwise & with (9-1)
```

```
n = 9&8  (1001 & 1000)
n = 8
count  = 1

Since 8 > 0, subtract by 1 and do bitwise & with (8-1)
n = 8&7  (1000 & 0111)
n = 0
count = 2

Since n = 0, return count which is 2 now.
```

Implementation of Brian Kernighan's Algorithm:

C++Java

```cpp
// C++ program to Count set
// bits in an integer
#include <iostream>
using namespace std;
class gfg {
	/* Function to get no of set bits in binary
representation of passed binary no. */
public:
	unsigned int countSetBits(int n)
	{
		unsigned int count = 0;
		while (n) {
			n &= (n - 1);
			count++;
		}
		return count;
	}
};
/* Program to test function countSetBits */
int main()
{
	gfg g;
	int i = 9;
	cout << g.countSetBits(i);
	return 0;
}
```

Output

2

Time Complexity: O(log n)

Auxiliary Space: O(1)

Using Lookup table: We can count bits in O(1) time using the lookup table.
Below is the implementation of the above approach:

C++Java

```cpp
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;

int BitsSetTable256[256];

// Function to initialise the lookup table
void initialize()
{

        // To initially generate the
        // table algorithmically
        BitsSetTable256[0] = 0;
        for (int i = 0; i < 256; i++)
        {
                BitsSetTable256[i] = (i & 1) +
                BitsSetTable256[i / 2];
        }
}

// Function to return the count
// of set bits in n
int countSetBits(int n)
{
        return (BitsSetTable256[n & 0xff] +
                        BitsSetTable256[(n >> 8) & 0xff] +
                        BitsSetTable256[(n >> 16) & 0xff] +
                        BitsSetTable256[n >> 24]);
}

// Driver code
int main()
{
        // Initialise the lookup table
        initialize();
        int n = 9;
        cout << countSetBits(n);
}
```

Output

2

Time Complexity: O(1)
Auxiliary Space: O(1)

Power of Two

Given a positive integer **n**, write a function to find if it is a power of 2 or not

Examples:

Input : *n = 4*
Output : *Yes*
Explanation: $2^2 = 4$

Input : *n = 32*
Output : *Yes*
Explanation: $2^5 = 32$

To solve the problem follow the below idea:

A simple method for this is to simply take the log of the number on base 2 and if you get an integer then the number is the power of 2

Below is the implementation of the above approach:

C++Java

```cpp
// C++ Program to find whether a
// no is a power of two
#include <bits/stdc++.h>
using namespace std;

// Function to check if x is power of 2
bool isPowerOfTwo(int n)
{
        if (n == 0)
                return false;

        return (ceil(log2(n)) == floor(log2(n)));
}

// Driver code
int main()
{
        // Function call
```

```
        isPowerOfTwo(31) ? cout << "Yes" << endl
                                 : cout << "No" << endl;
        isPowerOfTwo(64) ? cout << "Yes" << endl
                                 : cout << "No" << endl;

        return 0;
}
```

Output

```
No
Yes
```

Find whether a given number is a power of 2 using Brian Kernighan's algorithm:

To solve the problem follow the below idea:

As we know that the number which will be the power of two have only one set bit , therefore when we do bitwise AND with the number which is just less than the number which can be represented as the power of (2) then the result will be 0 .

Example : *4 can be represented as (2^2) ,*
(4 & 3)=0 or in binary (100 & 011=0)

Below is the implementation of the above approach:

C++Java

```
// C++ program of the above approach

#include <bits/stdc++.h>
using namespace std;

/* Function to check if x is power of 2*/
bool isPowerofTwo(long long n)
{
        return (n != 0) && ((n & (n - 1)) == 0);
}

// Driver code
int main()
{
        // Function call
        isPowerofTwo(30) ? cout << "Yes\n" : cout << "No\n";
```

```
        isPowerofTwo(128) ? cout << "Yes\n" : cout << "No\n";
        return 0;
}
```

Output

```
No
Yes
```

Time Complexity: O(1)
Auxiliary Space: O(1)

One Odd Occurring

Given an array of positive integers. All numbers occur an even number of times except one number which occurs an odd number of times. Find the number in O(n) time & constant space.

Examples :

Input : *arr = {1, 2, 3, 2, 3, 1, 3}*
Output : *3*

Input : *arr = {5, 7, 2, 7, 5, 2, 5}*
Output : *5*

A **Simple Solution** is to run two nested loops. The outer loop picks all elements one by one and the inner loop counts the number of occurrences of the element picked by the outer loop. The time complexity of this solution is $O(n^2)$.

Below is the implementation of the brute force approach :

C++Java

```
// C++ program to find the element
// occurring odd number of times
#include <bits/stdc++.h>
using namespace std;

// Function to find the element
// occurring odd number of times
int getOddOccurrence(int arr[], int arr_size)
{
```

```
    for (int i = 0; i < arr_size; i++) {

        int count = 0;

        for (int j = 0; j < arr_size; j++) {
            if (arr[i] == arr[j])
                count++;
        }
        if (count % 2 != 0)
            return arr[i];
    }
    return -1;
}

// driver code
int main()
{
    int arr[] = { 2, 3, 5, 4, 5, 2, 4, 3, 5, 2, 4, 4, 2 };
    int n = sizeof(arr) / sizeof(arr[0]);

    // Function calling
    cout << getOddOccurrence(arr, n);

    return 0;
}
```

Output :

```
5
```

Time Complexity: *O(n^2)*
Auxiliary Space: *O(1)*

The **Best Solution** is to do bitwise XOR of all the elements. XOR of all elements gives us odd occurring elements.

```
Here ^ is the XOR operators;
Note :
x^0 = x
x^y=y^x (Commutative property holds)
(x^y)^z = x^(y^z) (Associative property holds)
x^x=0
```

Below is the implementation of the above approach.

C++Java

```
// C++ program to find the element
```

```cpp
// occurring odd number of times
#include <bits/stdc++.h>
using namespace std;

// Function to find element occurring
// odd number of times
int getOddOccurrence(int ar[], int ar_size)
{
        int res = 0;
        for (int i = 0; i < ar_size; i++)
                res = res ^ ar[i];

        return res;
}

/* Driver function to test above function */
int main()
{
        int ar[] = {2, 3, 5, 4, 5, 2, 4, 3, 5, 2, 4, 4, 2};
        int n = sizeof(ar)/sizeof(ar[0]);

        // Function calling
        cout << getOddOccurrence(ar, n);

        return 0;
}
```

Output :

```
5
```

Time Complexity: O(n)
Auxiliary Space: O(1)

Two Odd Occurring

Given an unsorted array that contains even number of occurrences for all numbers except two numbers. Find the two numbers which have odd occurrences in O(n) time complexity and O(1) extra space.

Examples:

```
Input: {12, 23, 34, 12, 12, 23, 12, 45}
Output: 34 and 45

Input: {4, 4, 100, 5000, 4, 4, 4, 4, 100, 100}
Output: 100 and 5000

Input: {10, 20}
Output: 10 and 20
```

Naive method

To solve this problem is to run two nested loops. The outer loop picks an element and the inner loop counts the number of occurrences of the picked element. If the count of occurrences is odd then print the number. The time complexity of this method is O(n^2).

Python

```python
def oddappearing(arr) :
    for i in arr :
        count = 0

        for j in arr :
            if i == j :
                count += 1

        if count % 2 != 0 :
            print(i, end = " ")


if __name__ == "__main__" :
    arr = [3,4,3,4,5,4,4,6,7,7]
    oddappearing(arr)
```

Output:

```
 5 6
```

Time complexity: O(n^2)

Space complexity: O(1)

Efficient approach:

A O(n) time and O(1) extra space solution using XOR:

Let the two odd occurring numbers be x and y. We **use bitwise XOR** to get x and y. We try to make 2 groups such that x and y go to different groups. E.g. [a, a, b, b, x], . Then the problem will become “Find ‘one’ number with odd occurrence in an unsorted array”, which

becomes a simple problem and will be solved using XOR. Below are steps to group x and y differently.

1. The first step is to do XOR of all elements present in array. XOR of all elements gives us XOR of x and y because of the following properties of XOR operation.

1) XOR of any number n with itself gives us 0, i.e., n ^ n = 0
2) XOR of any number n with 0 gives us n, i.e., n ^ 0 = n
3) XOR is cumulative and associative.

So we have XOR of x and y after the first step, in decimal form. E.g. 5 ^ 6 returns 3, which is computed in bit form as 101 ^ 110 = 011. Let the 'value' of XOR be xor2. Every Set bit** in xor2 indicates that 'the corresponding bits in x and y have values different from each other' (XOR property- '1 when bits are different').

** (Set-bits are 1's in binary form. E.g. 101 has 2 set bits(1's), at 0th index and at 2nd index.)

For example, if x = 6 (0110) and y = 15 (1111), then xor2 will be (1001), the two set bits in xor2 indicate that the corresponding bits in x and y are different, at 0th index and at 3rd index both.

2. In the second step, we pick a set bit of xor2. Idea is to use the fact that xor2 is '1' in indexes where bits of x and y are different. So we separate x and y to different groups, along with rest of the numbers of list, based on whether the number has same set-bit or not.

We choose the rightmost set bit of xor2 as it is easy to get rightmost set bit of a number (bit magic). If we **bitwise AND** a number with its negative counterpart, we get rightmost set bit. (just an observation based property, do remember). So, (xor2) & (-xor2) will give us right set bit. Find (-number) by 2's complement, that is ((1's complement) +1). It can also be written as (~number)+1.

a) Example of 2's complement :

7 is 00111(any no. of preceding zeroes). 1's complement is obtained by flipping bits , 11000. Then add 1, so 2's complement of 7 is 11001. Since first bit is 1, its a negative no.

(-1)*16 + 1*8 +1*1 = -7

b) Example of (number) & (-number) = right set bit :

Continuing example of 7, 7 is 00111 and -7 is 11001 , 7 & -7 is 00001. So, rightmost set bit of 7 is 1.

Another example with 12 & -12:

12 is 01100 ** and -12 is calculated by flipping digits and adding 1. So, 10011 and adding 1 gives 10100. 12 & -12, 01100 & 10100 gives 00100 as set bit, that is returned as 4 in decimal system, also referred as **Set-bit Number** here.

** (since number is 32 bit, there are 28 0's left of 'left set-bit', but taking only a few is okay. Positive numbers have leftmost bit 0 and negative have 1)

3. In third step, we separate x and y in different groups : We now know that for selected set bit index, x and y have different corresponding bits. If we AND all numbers in list with set bit, some will give 0 and others will give 1. We will put all numbers giving zeroes in one group and ones in another. x and y will fall in different groups.

Explained with example:-

E.g. arr = [4, 2, 4, 10, 2, 3, 3, 12] ,

Step 1) XOR of all in arr will cancel all repeating nos. 10 ^12 will be ans. 1010 ^ 1100 will be 0110 that is xor=6.

Step 2) Set bit is 10 from 0110 from visualization. (number) & (-number) is also a quick way to find right set bit.

xor & (-xor) can be coded directly. 6 is 0110 and finding -6 by flipping digits and adding 1, 1001 +1 = 1010.

So 6 AND -6 is essentially 0110 & 1010, that is 0010 i.e. 2 – Set-bit Number.

Step 3) AND of all in list with 2 (Set bit no.) will give us numbers that give either 1 or 0, and we make groups.

[4, 4, 12] and [2, 10, 2, 3, 3], giving 0 and 1 respectively on AND with Set-bit number.

Step 4) XOR of 1st group will give us x=12, x ^ y is known from 1st step i.e. 6. x ^(x ^y) will give us y. 12 ^6 is 10.

x=12, y=10

This step works because of the same properties of XOR. All the occurrences of a number will go in same set. XOR of all occurrences of a number which occur even number of times will result in 0 in its set. And the xor of a set will be one of the odd occurring elements.

Python

```python
def oddappearing(arr):
    xors = 0  # This will hold the XOR of all elements in the array
    res1 = 0  # This will hold one of the results
    res2 = 0  # This will hold the other result

    # XOR all elements in the array
    for i in arr:
        xors = xors ^ i

    # Find the rightmost set bit in xors (which will be different in the
two odd occurring numbers)
    sn = xors & -(xors - 1)

    # Divide elements in two groups and XOR them separately
    for i in arr:
        if i & sn != 0:
            res1 = res1 ^ i  # XOR of first group
        else:
            res2 = res2 ^ i  # XOR of second group

    # Print the two odd occurring numbers
    print(res1, res2)

if __name__ == "__main__":
    arr = [3, 4, 3, 4, 5, 4, 4, 6, 7, 7]
    oddappearing(arr)
```

Output

```
6 5
```

Time Complexity: O(n)
Auxiliary Space: O(1)

Gray Code

Gray Code is the encoding of numbers such that the consecutive numbers differ with one bit.

```
3 bit Gray Codes for numbers from 0 to 7

000 001 011 010 110 111 101 100
 0   1   2   3   4   5   6   7
```

Note :

1. Above property is same for n bit binary numbers also.

2. The MSB in Gray Codes and Normal Binary is Same.

Application of Gray Code:

1. Error Correction :

Gray Code are used in detecting error in digital communication as, If there is any error in code, we can atleast be assured that there will be only one bit position change.

2. Boolean circuit minimization :

Gray code also finds application in K-Map, which can be used to minimise boolean expressions.

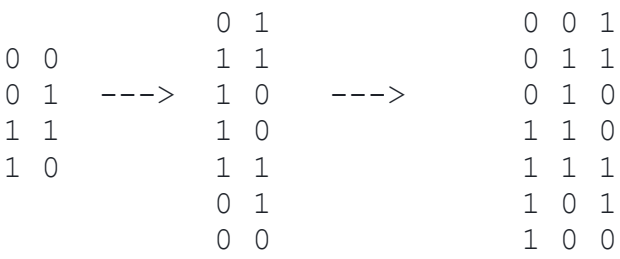
Process of Generating Gray Code

We can generate the 2 bit Gray code from 1 bit by copying the Bit and then reversing the Bit and then putting the 0 and 1 across the Bit

```
1 Bit                    2 Bit
            0              0 0
 0  --->    1   --->       0 1
 1           1              1 1
            0              1 0
```

Similarly, we will do the Conversion of 2 Bit into 3 Bit, in the same way.

```
2 Bit                     3 Bit
           0 0             0 0 0
```



Gray to Binary and Binary to Gray conversion

Binary Number is the default way to store numbers, but in many applications, binary numbers are difficult to use and a variety of binary numbers is needed. This is where Gray codes are very useful.

Gray code has a property that **two successive numbers differ in only one bit** because of this property gray code does the cycling through various states with minimal effort and is used in K-maps, error correction, communication, etc.

How to generate n bit Gray Codes?

```
Following is 2-bit sequence (n = 2)
  00 01 11 10
Following is 3-bit sequence (n = 3)
  000 001 011 010 110 111 101 100
And Following is 4-bit sequence (n = 4)
  0000 0001 0011 0010 0110 0111 0101 0100 1100 1101 1111
  1110 1010 1011 1001 1000
```

n-bit Gray Codes can be generated from a list of (n-1)-bit Gray codes using the following steps.

1. Let the list of (n-1)-bit Gray codes be L1. Create another list L2 which is the reverse of L1.
2. Modify the list L1 by prefixing a '0' in all codes of L1.
3. Modify the list L2 by prefixing a '1' in all codes of L2.
4. Concatenate L1 and L2. The concatenated list is the required list of n-bit Gray codes.

Please refer Generate n-bit Gray Codes for a detailed program.

How to Convert Binary To Gray and Vice Versa?

```
Binary : 0011
Gray   : 0010

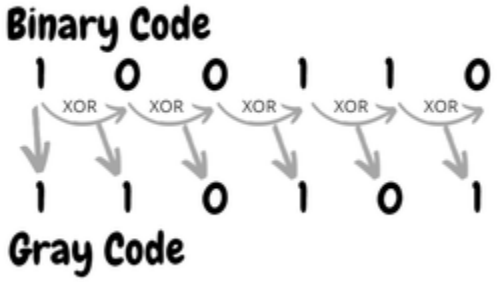
Binary : 01001
```

```
Gray    : 01101
```

In computer science many a time we need to convert binary code to gray code and vice versa. This conversion can be done by applying the following rules :

Binary to Gray conversion :

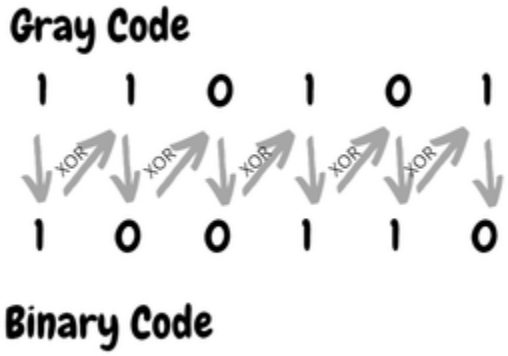
1. The Most Significant Bit (MSB) of the gray code is always equal to the MSB of the given binary code.
2. Other bits of the output gray code can be obtained by XORing binary code bit at that index and previous index.



Binary code to gray code conversion

Gray to binary conversion :

1. The Most Significant Bit (MSB) of the binary code is always equal to the MSB of the given gray code.
2. Other bits of the output binary code can be obtained by checking the gray code bit at that index. If the current gray code bit is 0, then copy the previous binary code bit, else copy the invert of the previous binary code bit.



Gray code to binary code conversion

Below is the implementation of the above steps.

C++JavaPython3C#PHPJavascript

```
// C++ program for Binary To Gray
// and Gray to Binary conversion
```

```cpp
#include <iostream>
using namespace std;

// Helper function to xor two characters
char xor_c(char a, char b) { return (a == b) ? '0' : '1'; }

// Helper function to flip the bit
char flip(char c) { return (c == '0') ? '1' : '0'; }

// function to convert binary string
// to gray string
string binarytoGray(string binary)
{
    string gray = "";

    // MSB of gray code is same as binary code
    gray += binary[0];

    // Compute remaining bits, next bit is computed by
    // doing XOR of previous and current in Binary
    for (int i = 1; i < binary.length(); i++) {
        // Concatenate XOR of previous bit
        // with current bit
        gray += xor_c(binary[i - 1], binary[i]);
    }

    return gray;
}

// function to convert gray code string
// to binary string
string graytoBinary(string gray)
{
    string binary = "";

    // MSB of binary code is same as gray code
    binary += gray[0];

    // Compute remaining bits
    for (int i = 1; i < gray.length(); i++) {
        // If current bit is 0, concatenate
        // previous bit
        if (gray[i] == '0')
            binary += binary[i - 1];

        // Else, concatenate invert of
        // previous bit
        else
            binary += flip(binary[i - 1]);
```

```
    }

    return binary;
}

// Driver program to test above functions
int main()
{
    string binary = "01001";
    cout << "Gray code of " << binary << " is "
         << binarytoGray(binary) << endl;

    string gray = "01101";
    cout << "Binary code of " << gray << " is "
         << graytoBinary(gray) << endl;
    return 0;
}
```

Output

```
Gray code of 01001 is 01101
Binary code of 01101 is 01001
```

Time Complexity: O(n)

Auxiliary Space: O(n)

Here, n is length of the binary string.

Binary to Gray using Bitwise Operators

C++JavaPython3C#Javascript

```
// C++ program for above approach
#include <bits/stdc++.h>
using namespace std;

int greyConverter(int n) { return n ^ (n >> 1); }

int main()
{
    int n = 3;
    cout << greyConverter(n) << endl;

    n = 9;
    cout << greyConverter(n) << endl;

    return 0;
}
```

Output
2
13

Time Complexity: O(1)

Auxiliary Space: O(1)

Gray to Binary using Bitwise Operators

C++JavaPython3C#Javascript

```cpp
// C++ program for above approach
#include <bits/stdc++.h>
using namespace std;

int binaryConverter(int n)
{
    int res = n;
    while (n > 0)
    {
        n >>= 1;
        res ^= n;
    }
    return res;
}

// Driver Code
int main()
{
    int n = 4;

    // Function call
    cout << binaryConverter(n) << endl;

    return 0;
}

// This code is contributed by sshrey47
```

Output
7

Time Complexity: O(log(n))

Auxiliary Space: O(1)

This article is contributed by Utkarsh Trivedi. Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above

Power Set using Bitwise

Power Set: Power set **P(S)** of a set **S** is the set of all subsets of **S**. For example S = {a, b, c} then P(s) = {{}, {a}, {b}, {c}, {a,b}, {a, c}, {b, c}, {a, b, c}}.
If **S** has **n** elements in it then **P(s)** will have $\mathbf{2^n}$ elements

Example:

Set = [a,b,c]
power_set_size = pow(2, 3) = 8
Run for binary counter = 000 to 111

Value of Counter	*Subset*
000	*-> Empty set*
001	*-> a*
010	*-> b*
011	*-> ab*
100	*-> c*
101	*-> ac*
110	*-> bc*
111	*-> abc*

Algorithm:

Input: Set[], set_size
1. Get the size of power set
    powet_set_size = pow(2, set_size)
2 Loop for counter from 0 to pow_set_size
  (a) Loop for i = 0 to set_size
    (i) If ith bit in counter is set
      Print ith element from set for this subset
  (b) Print separator for subsets i.e., newline

Method 1:
For a given set[] S, the power set can be found by generating all binary numbers

between **0** and 2^n-1, where **n** is the size of the set.
For example, for the set **S {*x*, *y*, *z*}**, generate all binary numbers from **0** to 2^3-1 and for each generated number, the corresponding set can be found by considering set bits in the number.

Below is the implementation of the above approach.

Javascript

```
// JavaScript program for the above approach

function printPowerSet(set, set_size) {
    // Set_size of power set of a set with set_size
    // n is (2^n-1)
    let pow_set_size = Math.pow(2, set_size);
    let counter, j;

    // Run from counter 000..0 to 111..1
    for (counter = 0; counter < pow_set_size; counter++) {
        for (j = 0; j < set_size; j++) {
            // Check if jth bit in the counter is set
            // If set then print jth element from set
            if (counter & (1 << j))
                console.log(set[j]);
        }
        console.log("\n");
    }
}

// Driver code
let set = ['a', 'b', 'c'];
printPowerSet(set, 3);
```

Output

```
a
b
ab
c
ac
bc
abc
```

Time Complexity: $O(n2^n)$
Auxiliary Space: $O(1)$

Fibonacci Numbers

Fibonacci Numbers (introduction and Properties)

Fibonacci numbers are a sequence of numbers in which each number is the sum of the two preceding numbers. The sequence starts with 0 and 1, and the next number in the sequence is the sum of the previous two numbers. So the sequence goes: **0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, ...**

Properties of Fibonacci Number :

1) Golden Ratio: Two numbers, say **A** and **B** are said to be in the golden ratio if their ratio equals the ratio of the sum of two numbers to the larger number, i.e.,

Suppose A > B, then
If A/B = (A + B)/A = □ = 1.618(Golden Ratio),
then these two numbers are said to be in golden ratio.

It is denoted by Ø and its value is equal to **1.6180339...**,

2)**Binet's formula:** It is used to find the **N**th term in the Fibonacci sequence.

where, F_N is the Nth term in the Fibonacci Sequence.

The above equation is known as **Binet's Formula**. And the value $\mathbf{(1+\sqrt{5})/2}$ is known as the **Golden Ratio**, which is equal to **1.618**. Therefore, the **N**th **Fibonacci Number** is given by:

$\boldsymbol{F_N \approx \square^N}$
where, where, □ is the Golden Ratio and F_n is the nth Fibonacci term.

*3)***Fibonacci is a complete sequence (like powers of 2) :** The Fibonacci sequence is a complete sequence in the sense that it contains every positive integer exactly once, as a sum of some of its terms. This is because every positive integer can be expressed as a sum of non-consecutive Fibonacci numbers in a unique way.

For example, the number 7 can be expressed as 1 + 2 + 4, which are non-consecutive Fibonacci numbers.

More Properties of Fibonacci Numbers

Fibonacci numbers, a sequence of numbers that starts with 0 and 1, and each subsequent number is the sum of the two preceding ones. In this article, we will explore three remarkable properties of Fibonacci numbers: their matrix form, Cassini's identity, and the addition rule.

1. **Matrix Form:** Fibonacci numbers can be represented in a matrix form that reveals some intriguing patterns. Consider the following matrix:

```
[1 1]
[1 0]
```

If we raise this matrix to a power of n, denoted as M^n, where n is a positive integer, and extract the value in the top-right corner of the resulting matrix, we obtain the n-th Fibonacci number. In other words, the (1,2)-th entry of M^n is the n-th Fibonacci number. For example:

```
M^0 = [1 1]^0 = [1 0]   => F(0) = 0
M^1 = [1 1]^1 = [1 1]   => F(1) = 1
M^2 = [1 1]^2 = [2 1]   => F(2) = 1
M^3 = [1 1]^3 = [3 2]   => F(3) = 2
M^4 = [1 1]^4 = [5 3]   => F(4) = 3
```

And so on. This matrix form of Fibonacci numbers provides an elegant and efficient way to compute Fibonacci numbers without using recursive formulas or iterative algorithms, and it has applications in various fields such as computer graphics, cryptography, and number theory.

2. **Cassini's Identity:** Cassini's identity is a fascinating property of Fibonacci numbers discovered by the French astronomer Jean-Dominique Cassini in the 18th century. It states that the difference between the product of two consecutive Fibonacci numbers and the square of their common neighbor is always equal to (-1)^n, where n is the position of the common neighbor in the Fibonacci sequence. Mathematically, Cassini's identity can be expressed as follows:

```
F(n-1) * F(n+1) - F(n)^2 = (-1)^n
```

For example:

```
n = 2: F(1) * F(3) - F(2)^2 = 1 * 2 - 1 = 1 = (-1)^2
```

```
n = 4: F(3) * F(5) - F(4)^2 = 2 * 5 - 3^2 = 1 = (-1)^4
n = 6: F(5) * F(7) - F(6)^2 = 5 * 13 - 8^2 = 1 = (-1)^6
```

And so on. Cassini's identity provides a fascinating insight into the deep connections and patterns present in the Fibonacci sequence, and it has been used in various mathematical proofs and applications, such as generating new identities and solving equations.

3. **Addition Rule:** The addition rule is a surprising property of Fibonacci numbers that states that the sum of any k consecutive Fibonacci numbers is equal to the difference between the (k+2)-th and (k-1)-th Fibonacci numbers, multiplied by a suitable power of 2, depending on the parity of k.

More formally, let F(n) denote the n-th Fibonacci number, where F(0) = 0, F(1) = 1, and F(n) = F(n-1) + F(n-2) for n ≥ 2. Then, for any positive integer k, the sum of k consecutive Fibonacci numbers, denoted as F(n) + F(n+1) + ... + F(n+k-1), can be expressed as:

```
F(n) + F(n+1) + ... + F(n+k-1) = F(n+k+1) - F(n-1)     if k is
even
F(n) + F(n+1) + ... + F(n+k-1) = F(n+k+1) - F(n)         if k is
odd
```

where n is any arbitrary positive integer.

Fibonacci Divisibility and GCD

In this article, we will explore three remarkable properties of Fibonacci numbers that shed light on their mathematical and practical significance.

1. **Two Consecutive Fibonacci Numbers are Always Co-prime**

One fascinating property of Fibonacci numbers is that any two consecutive numbers in the sequence are always coprime, which means they do not share any common factors other than 1. Mathematically, for any positive integer n, the greatest common divisor (GCD) of two consecutive Fibonacci numbers F(n) and F(n+1) is always 1.

To see why this is true, let's consider the definition of Fibonacci numbers. We start with F(0) = 0 and F(1) = 1, and then generate subsequent Fibonacci numbers using the recursive formula F(n) = F(n-1) + F(n-2) for n ≥ 2. Since each Fibonacci number is the sum of the two preceding ones, they do not have any common factors other than 1, as adding any common factor to both terms would result in a common factor in the sum, which contradicts the

definition of Fibonacci numbers. Therefore, two consecutive Fibonacci numbers are always coprime, which is a unique and intriguing property of this sequence.

This property has practical applications in various areas such as number theory, cryptography, and computer science. For example, it is used in cryptographic algorithms to ensure secure communication by leveraging the coprimality of consecutive Fibonacci numbers to generate encryption keys that are difficult to crack.

2. **GCD(Fm, Fn) = F GCD(m, n)**

Another interesting property of Fibonacci numbers is the relationship between the GCD of two Fibonacci numbers and the Fibonacci number corresponding to the GCD of their indices. Mathematically, for any positive integers m and n, the GCD of F(m) and F(n) is equal to the Fibonacci number corresponding to the GCD of m and n.

Formally, we can express this property as:

```
GCD(F(m), F(n)) = F(GCD(m, n))
```

This property can be proven using mathematical induction and properties of GCD. It has important implications in number theory and provides a useful tool for computing the GCD of Fibonacci numbers efficiently. It also has applications in various areas such as algorithm design, cryptography, and coding theory.

3. **If Fn is a Multiple of Fm, Then n is Also a Multiple of m**

A third intriguing property of Fibonacci numbers is that if F(n) is a multiple of F(m) for some positive integers n and m, then n is also a multiple of m. Mathematically, if F(n) is divisible by F(m), then n must be a multiple of m.

In other words, if F(n) = a * F(m), where a is an integer, then n = b * m, where b is also an integer.

This property can be proven using properties of divisibility and the recursive formula of Fibonacci numbers. It has important implications in number theory and has been used in various mathematical proofs and conjectures.

In conclusion, Fibonacci numbers exhibit remarkable properties that go beyond their simple recursive definition. From being coprime, to the relationship between their GCD and the GCD of their indices

Count Binary Strings With No two Consecutive 1's

Given a positive integer N, count all possible distinct binary strings of length N such that there are no consecutive 1's.

Examples:

```
Input:  N = 2
Output: 3
// The 3 strings are 00, 01, 10

Input: N = 3
Output: 5
// The 5 strings are 000, 001, 010, 100, 101
```

This problem can be solved using Dynamic Programming. Let a[i] be the number of binary strings of length i which do not contain any two consecutive 1's and which end in 0. Similarly, let b[i] be the number of such strings which end in 1. We can append either 0 or 1 to a string ending in 0, but we can only append 0 to a string ending in 1. This yields the recurrence relation:

```
a[i] = a[i - 1] + b[i - 1]
b[i] = a[i - 1]
```

The base cases of above recurrence are a[1] = b[1] = 1. The total number of strings of length i is just a[i] + b[i].

Following is the implementation of above solution. In the following implementation, indexes start from 0. So a[i] represents the number of binary strings for input length i+1. Similarly, b[i] represents binary strings for input length i+1.

Implementation:

C++

```cpp
// C++ program to count all distinct binary strings
// without two consecutive 1's
#include <iostream>
using namespace std;

int countStrings(int n)
{
    int a[n], b[n];
    a[0] = b[0] = 1;
    for (int i = 1; i < n; i++)
    {
        a[i] = a[i-1] + b[i-1];
```

```
        b[i] = a[i-1];
    }
    return (a[n-1] + b[n-1])%1000000007;
}


// Driver program to test above functions
int main()
{
    cout << countStrings(3) << endl;
    return 0;
}
```

Output
5
Time Complexity: O(N)
Auxiliary Space: O(N)

Source:

If we take a closer look at the pattern, we can observe that the count is actually (n+2)'th Fibonacci number for n >= 1. The Fibonacci Numbers are 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 141, ...

```
n = 1, count = 2  = fib(3)
n = 2, count = 3  = fib(4)
n = 3, count = 5  = fib(5)
n = 4, count = 8  = fib(6)
n = 5, count = 13 = fib(7)
.................
```

Therefore we can count the strings in O(Log n) time also using the method 5 here.

Another method :-

From the above method it is clear that we only want just previous value in the for loop which we can also do by replacing the array with the variable.

Below is the implementation of the above approach:-

C++

```
// C++ program to count all distinct binary strings
// without two consecutive 1's
#include <bits/stdc++.h>
using namespace std;
```

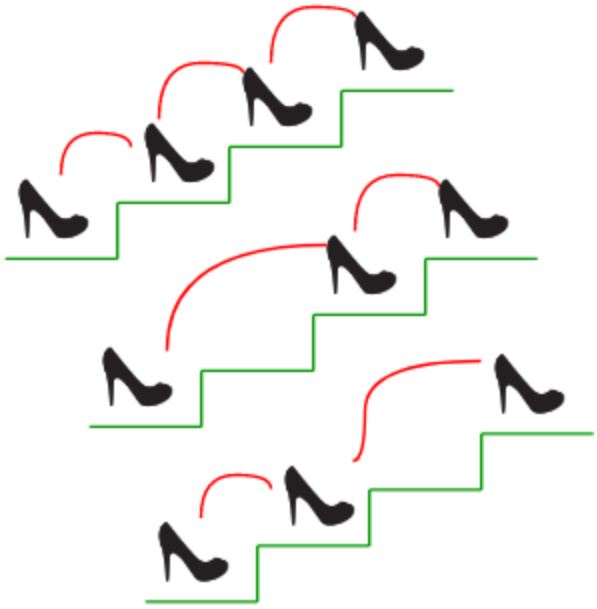
```
int countStrings(int n)
{
    int a = 1, b = 1;
    for (int i = 1; i < n; i++) {
        // Here we have used the temp variable because we
        // want to assign the older value of a to b
        int temp = a + b;
        b = a;
        a = temp;
    }
    return (a + b)%1000000007;
}

// Driver program to test above functions
int main()
{
    cout << countStrings(3) << endl;
    return 0;
}
```

Output
5
Time Complexity: O(N)
Auxiliary Space: O(1)

Count Ways to Reach n-th Stair

There are n stairs, a person standing at the bottom wants to reach the top. The person can climb either 1 stair or 2 stairs at a time. Count the number of ways, the person can reach the top.



Consider the example shown in the diagram. The value of n is 3. There are 3 ways to reach the top. The diagram is taken from [Easier Fibonacci puzzles]

Examples:

```
Input: n = 1
Output: 1
There is only one way to climb 1 stair

Input: n = 2
Output: 2
There are two ways: (1, 1) and (2)

Input: n = 4
Output: 5
(1, 1, 1, 1), (1, 1, 2), (2, 1, 1), (1, 2, 1), (2, 2)
```

Method 1: The first method uses the technique of recursion to solve this problem.
Approach: We can easily find the recursive nature in the above problem. The person can reach **n**th stair from either **(n-1)**th stair or from **(n-2)**th stair. Hence, for each stair **n**, we try to find out the number of ways to reach **n-1**th stair and **n-2**th stair and add them to give the answer for the **n**th stair. Therefore the expression for such an approach comes out to be :

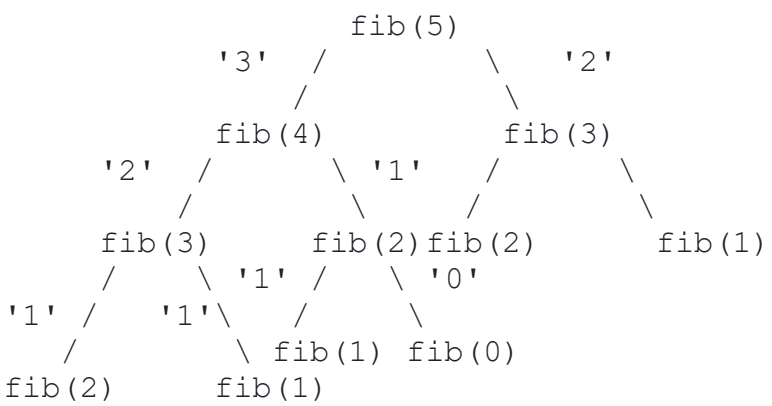
```
ways(n) = ways(n-1) + ways(n-2)
```

The above expression is actually the expression for Fibonacci numbers, but there is one thing to notice, the value of ways(n) is equal to fibonacci(n+1).

```
ways(1) = fib(2) = 1
ways(2) = fib(3) = 2
ways(3) = fib(4) = 3
```

For a better understanding, let's refer to the recursion tree below -:

Input: N = 4



So we can use the function for Fibonacci numbers to find the value of ways(n). Following is C++ implementation of the above idea.

C++

```cpp
// C++ program to count number of
// ways to reach Nth stair
#include <bits/stdc++.h>
using namespace std;

// A simple recursive program to
// find N'th fibonacci number
int fib(int n)
{
    if (n <= 1)
        return n;
    return fib(n - 1) + fib(n - 2);
}

// Returns number of ways to
// reach s'th stair
int countWays(int s)
{
```

```
    return fib(s + 1);
}

// Driver C
int main()
{
    int s = 4;

    cout << "Number of ways = " << countWays(s);

    return 0;
}
```

Output

```
Number of ways = 5
```

Complexity Analysis:

- **Time Complexity:** *O(2^n)*
  The time complexity of the above implementation is exponential (golden ratio raised to power n) due to redundant calculations.It can be optimized to work in O(Logn) time using the previously discussed Fibonacci function optimizations.
- **Auxiliary Space:** *O(1)*

Fibonacci Implementation

Given a number n, print n-th Fibonacci Number.

Examples:

```
Input  : n = 2
Output : 1

Input  : n = 9
Output : 34
```

Write a function *int fib(int n)* that returns F_n. For example, if *n* = 0, then *fib()* should return 0. If n = 1, then it should return 1. For n > 1, it should return $F_{n-1} + F_{n-2}$

```
For n = 9
Output:34
```

Method 1 (Naive) : Use recursion

C++

```
// C++ program for Fibonacci Implementation

#include<bits/stdc++.h>
using namespace std;

class GFG {
public:
    // Function to implemet fibonacci
    int fib(int n)
    {
        if (n <= 1)
                return n;
        return fib(n-1) + fib(n-2);
    }
};

int main ()
{
        int n = 4;
        class GFG obj;
        cout << obj.fib(n);
        getchar();
        return 0;
}
```

Output
3

Time Complexity: Exponential, as every function calls two other functions.
Auxiliary space complexity: O(n)

Method 2 (Efficient) :

C++

```
// C++ program for Fibonacci Implementation

#include<bits/stdc++.h>
using namespace std;

class GFG {
public:
    // Function to implemet fibonacci
    int fib(int n)
    {
        if (n <= 1)
                return n;
        int a=0;
        int b=1;
```

```
        int c;
        for(int i=2;i<=n;i++)
        {
            c=a+b;
            a=b;
            b=c;
        }
        return c;
    }
};

int main ()
{
        int n = 4;
        class GFG obj;
        cout << obj.fib(n);
        getchar();
        return 0;
}
```

Output
3
Time Complexity: θ(n)
Extra Space: O(1)

Sum of Fibonacci Numbers

Given a number positive number n, find value of $f_0 + f_1 + f_2 + ... + f_n$ where f_i indicates i'th Fibonacci number. Remember that $f_0 = 0$, $f_1 = 1$, $f_2 = 1$, $f_3 = 2$, $f_4 = 3$, $f_5 = 5$, ...
Examples :

Input : n = 3
Output : 4
Explanation : 0 + 1 + 1 + 2 = 4

Input : n = 4
Output : 7
Explanation : 0 + 1 + 1 + 2 + 3

Naive Solution : find all the Fibonacci numbers till f(n) and then add them up.

C++

```cpp
// C++ program to find Sum of Fibonacci Numbers
#include <iostream>
using namespace std;

int fib(int n)
{
    if(n<=1)
    return n;
    int a = 0, b = 1, sum=1;
    for (int i = 2; i <= n; i++)
    {
        int c = a + b;
        a = b;
        b = c;
        sum += c;
    }
    return sum;
}
// Driver Code
int main() {
       int n = 5;
    cout<<fib(n)<<endl;
       return 0;
}
```

Output
12

Time Complexity: θ(n)
Auxiliary Space: O(n)

Efficient Solution:

The idea is to find relationship between the sum of Fibonacci numbers and n'th Fibonacci number.
F(i) refers to the i'th Fibonacci number.
S(i) refers to sum of Fibonacci numbers till F(i),

```
We can rewrite the relation F(n+1) = F(n) + F(n-1) as below
F(n-1)    = F(n+1)   -  F(n)

Similarly,
F(n-2)    = F(n)     -  F(n-1)
.           .            .
.           .              .
.           .              .
```

```
F(0)       = F(2)     -   F(1)
------------------------------
```

Adding all the equations, on left side, we have
F(0) + F(1) + … F(n-1) which is S(n-1).
Therefore,
S(n-1) = F(n+1) – F(1)
S(n-1) = F(n+1) – 1
S(n) = F(n+2) – 1 —-(1)
In order to find S(n), simply calculate the (n+2)'th Fibonacci number and subtract 1 from the result.

Implementation :

C++

```
// C++ Program to find sum of fibonacci numbers

#include <iostream>
using namespace std;
int MAX = 1000;
int f[100005];
// fib returns nth fibonacci number using table f[]
int fib(int n)
{
    // Base cases
    if (n == 0)
    return 0;

    if (n == 1 || n == 2)
    return (f[n] = 1);

    // If fib(n) is already computed
    if (f[n] != 0)
    return f[n];

    int k = (n & 1) == 1? (n + 1) / 2 : n / 2;

    // Applying above formula [Note value
    // n&1 is 1 if n is odd, else 0.
    f[n] = (n & 1) == 1? (fib(k) * fib(k) + fib(k - 1) * fib(k - 1)): (2 *
fib(k - 1) + fib(k))* fib(k);

    return f[n];
}
int sumFibo(int n)
{
    return fib(n+2) - 1;
```

```
}
 /* Driver program to test above function */
int main() {
        int n = 5;
    //f= int[MAX];
    cout<<sumFibo(n)<<endl;
        return 0;
}
```

Output
12

Time Complexity: O(logn)

Sum of Squares of Fibonacci Numbers

Given a positive integer N. The task is to find the sum of squares of all Fibonacci numbers up to N-th Fibonacci number. That is,

$f_0^2 + f_1^2 + f_2^2 + ... + f_n^2$

where f_i indicates i-th fibonacci number.

Fibonacci numbers: $f_0=0$ and $f_1=1$ and $f_i=f_{i-1} + f_{i-2}$ for all i>=2.

Examples:

Input: N = 3
Output: 6
Explanation: 0 + 1 + 1 + 4 = 6

Input: N = 6
Output: 104
Explanation: 0 + 1 + 1 + 4 + 9 + 25 + 64 = 104

C++

```
#include<bits/stdc++.h>
using namespace std;

int sumSquaresFib(int n)
    {
        if(n<=1)
            return n;
        int a = 0, b = 1, sum=1;
        for (int i = 2; i <= n; i++)
        {
            int c = a + b;
            a = b;
```

```
            b = c;
            sum += c*c;
        }
        return sum;
    }

    // Driver Code
    int main(){

    int n = 6;


    cout<<sumSquaresFib(n);
}
```

Output
104

Time Complexity: O(n)
Auxiliary Space: O(n)

Method 2 (Efficient) :

We know that for i-th Fibonacci number,

$f_{i+1} = f_i + f_{i-1}$ for all $i>0$

Or, $f_i = f_{i+1} - f_{i-1}$ for all $i>0$
Or, $f_i^2 = f_i f_{i+1} - f_{i-1} f_i$
So for any n>0 we see,

$f_0^2 + f_1^2 + f_2^2 + \ldots\ldots + f_n^2$
$= f_0^2 + (f_1 f_2 - f_0 f_1) + (f_2 f_3 - f_1 f_2) + \ldots\ldots\ldots\ldots + (f_n f_{n+1} - f_{n-1} f_n)$
$= f_n f_{n+1}$ (Since $f_0 = 0$)

This identity also satisfies for n=0 (For n=0, $f_0^2 = 0 = f_0 f_1$).

Below is the implementation of the above approach:

C++

```
// C++ Program to find sum of squares of
// Fibonacci numbers
```

```cpp
#include <bits/stdc++.h>
using namespace std;
const int MAX = 1000;

// Create an array for memoization
int f[MAX] = { 0 };

// Returns n'th Fibonacci number using table f[]
int fib(int n)
{
    // Base cases
    if (n == 0)
        return 0;
    if (n == 1 || n == 2)
        return (f[n] = 1);

    // If fib(n) is already computed
    if (f[n])
        return f[n];

    int k = (n & 1) ? (n + 1) / 2 : n / 2;

    // Applying above formula [Note value n&1 is 1
    // if n is odd, else 0].
    f[n] = (n & 1) ? (fib(k) * fib(k) + fib(k - 1) * fib(k - 1))
                   : (2 * fib(k - 1) + fib(k)) * fib(k);

    return f[n];
}

// Function to calculate sum of
// squares of Fibonacci numbers
int calculateSumOfSquares(int n)
{
    return fib(n) * fib(n + 1);
}

// Driver Code
int main()
{
    int n= 6;


    cout << "Sum of Squares of Fibonacci numbers is : "
         << calculateSumOfSquares(n) << endl;

    return 0;
}
```

Output

```
Sum of Squares of Fibonacci numbers is : 104
```

Time Complexity: O(logn)

Auxiliary Space: O(n)

Fibonacci LogN Implementation

Fibonacci Log N Implementation

C++

```cpp
// CPP Program to find n'th fibonacci
// Number with O(Log n) arithmetic operations

#include<bits/stdc++.h>
using namespace std;

class GFG {
public:

        int f[1000];

        // Returns n'th fibonacci number using
        // table f[]
        int fib(int n)
        {
                // Base cases
                if (n == 0)
                        return 0;

                if (n == 1 || n == 2)
                        return (f[n] = 1);

                // If fib(n) is already computed
                if (f[n] != 0)
                        return f[n];

                int k = (n & 1) == 1? (n + 1) / 2
                                                : n / 2;

                // Applying above formula [Note value
                // n&1 is 1 if n is odd, else 0.
                f[n] = (n & 1) == 1? (fib(k) * fib(k) +
```

```
                                                fib(k - 1) * fib(k - 1))
                                        : (2 * fib(k - 1) + fib(k))
                                        * fib(k);

                return f[n];
        }
};


/* Driver program to test above function */
int main()
{
        int n = 9;
        class GFG obj;
        cout << obj.fib(n) << endl;
}
```

Output

```
34
```

Pisano's Period

Pisano's period, also known as the Fibonacci sequence modulo a number, refers to the repeating pattern that emerges when the Fibonacci sequence is taken modulo a specific number.

When the Fibonacci sequence is taken modulo a number, the resulting sequence is periodic, meaning that it repeats itself after a certain number of terms. This repeating pattern is called Pisano's period. The length of Pisano's period depends on the chosen modulus.

For example, if we take the Fibonacci sequence modulo 3, we get the following sequence:

```
0, 1, 1, 2, 0, 2, 2, 1, 0, 1, 1, 2, 0, 2, 2, 1, 0, 1, 1, 2, 0,
2, 2, 1, ...
```

In this case, Pisano's period has a length of 8. Similarly, if we take the Fibonacci sequence modulo 5, we get the following sequence:

```
0, 1, 1, 2, 3, 0, 3, 3, 1, 4, 0, 4, 4, 3, 2, 0, 2, 2, 4, 1, 0,
1, 1, 2, 3, ...
```

In this case, Pisano's period has a length of 20.

Why does this period exist ?

For modulo m, the period length is upper bounded by m^2

Let us consider below m^2+1 ordered pairs

$(F_0, F_1), (F_1, F_2) \ldots\ldots\ldots\ldots\ldots\ldots (Fm^2, Fm^2+1)$

If we consider remainders of all pairs with m, we can claim , there will be at least two pairs with some remainders

$(F_i \% m, F_{i+1} \% m) = (F_j \% m, F_{j+1} \% m)$

$F_i \% m = F_j \% m$ $\qquad\qquad\qquad\qquad\qquad\qquad i < j \leq m^2$

$F_{i+1} \% m = F_{j+1} \% m$

Please note the sequences starting from F_i and F_j and generated using (F_i, F_{i+1}) and (F_j ,F_{j+1}) respectively. So F_{i+k} %. m = F_{j+k} % m

Fibonacci Mod M

Given two number **N** and **M**. The task is to find the **N-th** fibonacci number **mod M**.
In general let $\mathbf{F_N}$ be the **N-th** fibonacci number then the output should be $\mathbf{F_N}$ % **M**.

The Fibonacci sequence is a series of numbers in which each no. is the sum of two preceding nos. It is defined by the recurrence relation:

$F_0 = 0$
$F_1 = 1$
$F_n = F_{n-1} + F_{n-2}$

These nos. are in the following sequence: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, …
Here **N** can be large.

Examples:

Input: *N = 438, M = 900*
Output: *44*

Input: *N = 1548276540, M = 235*
Output: *185*

Approach:
However, for such values of *N* , a simple recursive approach to keep calculating *N* Fibonacci numbers with a time complexity of $O(2^N)$ should be avoided. Even an iterative or a Dynamic Programming approach with an algorithm looping for *N* iterations will not be time-

efficient.
This problem can be solved using the properties of **Pisano Period**.

For a given value of *N* and M >= 2, the series generated with F_i *modulo M* (for i in range(N)) is periodic.

Implementation :

C++

```
// C++ program to calculate Fibbonaci Mod M

#include <bits/stdc++.h>
using namespace std;


class GFG{
    public:

    // Function to Calculate Pisano Period
    long pisano(long m)
    {
        long prev = 0;
        long curr = 1;
        long res = 0;

        for(int i = 0; i < m * m; i++)
        {
                long temp = 0;
                temp = curr;
                curr = (prev + curr) % m;
                prev = temp;

                if (prev == 0 && curr == 1)
                        res = i + 1;
        }
        return res;
    }

    // Function to Calculate Fibbonaci Mod M
    long fibonacciModulo(long n, long m)
    {
        // Getting the period
        long pisanoPeriod = pisano(m);

        n = n % pisanoPeriod;

        long prev = 0;
```

```
        long curr = 1;

        if (n == 0)
                return 0;
        else if (n == 1)
                return 1;

        for(int i = 0; i < n - 1; i++)
        {
                long temp = 0;
                temp = curr;
                curr = (prev + curr) % m;
                prev = temp;
        }
        return curr % m;
    }
};



// Driver Code
int main()
{
        long n = 9;
        long m = 4;
        class GFG obj;

        cout << (obj.fibonacciModulo(n, m));
        return 0;
}
```

Output
2

Time Complexity: θ(p)

Auxiliary Space: θ(p)

Further Optimizations for Large Composite m

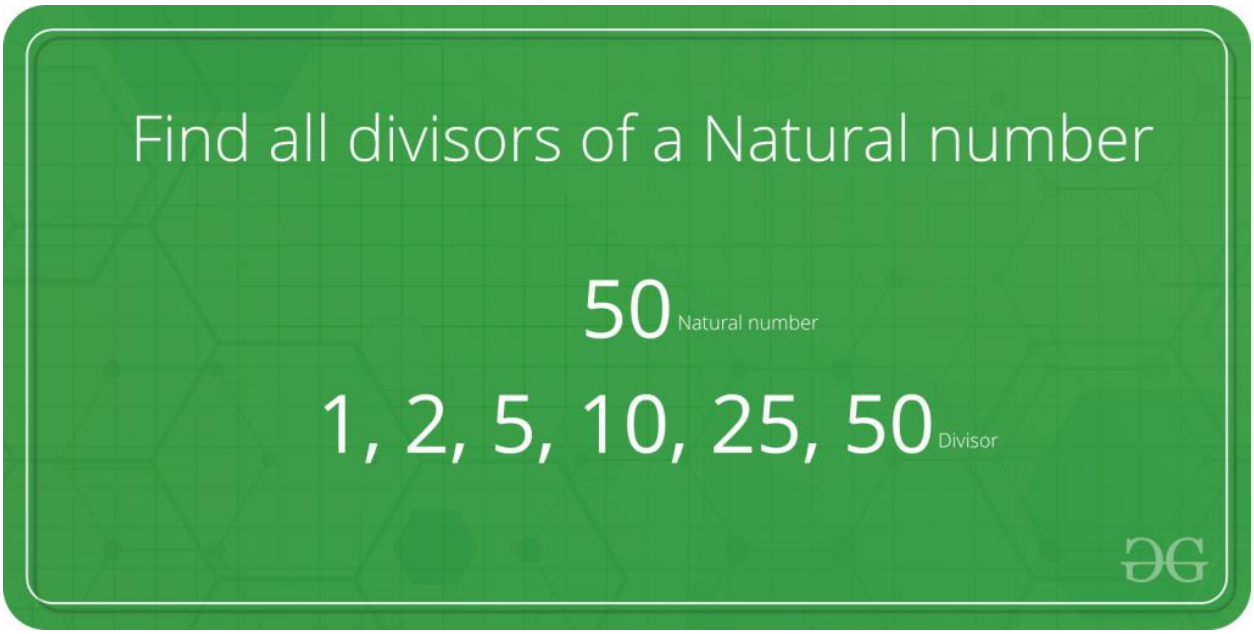
- pisano (p_1x p_2) = lcm (pisano (p_1) x pisano (p_2)) Where p_1 and p_2 are cobrime
- pisano (p^k) = p^{k-1} x pisano (p) where p in a prime number

We use above on prime factors of m.

DIVISORS

All Divisors of a Number

Given a natural number n, print all distinct divisors of it.



Examples:

```
Input : n = 10
Output: 1 2 5 10

Input:  n = 100
Output: 1 2 4 5 10 20 25 50 100

Input:  n = 125
Output: 1 5 25 125
```

A **Naive Solution** would be to iterate all the numbers from 1 to n, checking if that number divides n and printing it. Below is a program for the same:

C++Java

```
// C++ implementation of Naive method to print all
// divisors
#include <iostream>
using namespace std;
```

```
// function to print the divisors
void printDivisors(int n)
{
        for (int i = 1; i <= n; i++)
                if (n % i == 0)
                        cout <<" " << i;
}

/* Driver program to test above function */
int main()
{
        cout <<"The divisors of 100 are: \n";
        printDivisors(100);
        return 0;
}
```

Output:

```
The divisors of 100 are:
1 2 4 5 10 20 25 50 100
```

Time Complexity : O(n)
Auxiliary Space : O(1)

Can we improve the above solution?
If we look carefully, all the divisors are present in pairs. For example if n = 100, then the various pairs of divisors are: (1,100), (2,50), (4,25), (5,20), (10,10)
Using this fact we could speed up our program significantly.
We, however, have to be careful if there are two equal divisors as in the case of (10, 10). In such case, we'd print only one of them.

Below is an implementation for the same:

C++Java

```
// A Better (than Naive) Solution to find all divisors
#include <iostream>
#include <math.h>
using namespace std;

// Function to print the divisors
void printDivisors(int n)
{
        // Note that this loop runs till square root
        for (int i=1; i<=sqrt(n); i++)
        {
```

```
                if (n%i == 0)
                {
                        // If divisors are equal, print only one
                        if (n/i == i)
                                cout <<" "<< i;

                        else // Otherwise print both
                                cout << " "<< i << " " << n/i;
                }
        }
}

/* Driver program to test above function */
int main()
{
        cout <<"The divisors of 100 are: \n";
        printDivisors(100);
        return 0;
}
```

Output:

```
The divisors of 100 are:
1 100 2 50 4 25 5 20 10
```

Time Complexity: O(sqrt(n))

Auxiliary Space : O(1)

Printing all the divisors in sorted order:

C++Java

```
// A O(sqrt(n)) program that prints all divisors
// in sorted order
#include <iostream>
#include <math.h>
using namespace std;

// Function to print the divisors
void printDivisors(int n)
{
        int i;
        for (i = 1; i * i < n; i++) {
                if (n % i == 0)
                        cout<<i<<" ";
        }
        if (i - (n / i) == 1) {
```

```
                i--;
        }
        for (; i >= 1; i--) {
                if (n % i == 0)
                        cout<<n / i<<" ";
        }
}

// Driver code
int main()
{
        cout << "The divisors of 100 are: \n";

        printDivisors(100);

        return 0;
}

// This code is contributed by siteshbiswal
```

Output:

```
1 2 4 5 10 20 25 50 100
```

Time Complexity: O(sqrt(n))

Auxiliary Space : O(1)

Numbers with exactly 3 divisors

Given a number **N**, print all numbers in the range from 1 to N having exactly 3 divisors.

Examples:

Input: N = 16
Output: 4 9
Explanation: 4 and 9 have exactly three divisors.

Input: N = 49
Output: 4 9 25 49
Explanation: 4, 9, 25 and 49 have exactly three divisors.

Mathematical approach to find Numbers with exactly 3 divisors:

To solve the problem follow the below idea:

Idea: After having a close look at the examples mentioned above, you have noticed that all the required numbers are perfect squares and that too of only prime numbers.

Proof: Suppose the number is N, and it is a perfect square with square root X such that X is prime.

Now if we find the factors of N, it will always have following combinations:

- 1*N
- X*X

Therefore the required numbers will have only three numbers as their divisors:

- 1,
- that number itself, and
- just a single divisor in between 1 and the number.

Algorithm: We can generate all primes within a set using any sieve method efficiently and then we should take all primes i, such that **i*i <=N**.

Follow the below steps to solve the problem:

- Generate the prime numbers from 1 to N using any sieve method efficiently
- Print all the prime numbers(X) between 1 to N, such as X^2 is less than or equal to N

Below is the implementation of the above approach:

C++Java

```cpp
// C++ program to print all
// three-primes smaller than
// or equal to N using Sieve
// of Eratosthenes
#include <bits/stdc++.h>
using namespace std;

// Generates all primes upto N and
// prints their squares
void numbersWith3Divisors(int N)
{
    bool prime[N + 1];
```

```
    memset(prime, true, sizeof(prime));
    prime[0] = prime[1] = 0;

    for (int p = 2; p * p <= N; p++) {
        // If prime[p] is not changed,
        // then it is a prime
        if (prime[p] == true) {
            // Update all multiples of p
            for (int i = p * 2; i <= N; i += p)
                prime[i] = false;
        }
    }

    // Print squares of primes upto n.
    cout << "Numbers with 3 divisors :\n";
    for (int i = 0; i * i <= N; i++)
        if (prime[i])
            cout << i * i << " ";
}

// Driver code
int main()
{
    int N = 96;

    // Function call
    numbersWith3Divisors(N);

    return 0;
}
```

Output

```
Numbers with 3 divisors :
4 9 25 49
```

Time Complexity: O(N*log(log(N)))
Auxiliary Space: O(N)

Numbers with exactly 3 divisors using constant space:

Run a loop from 2 to sqrt(N) and check if the current element is prime or not, if it is so then print that number, but this method will increase the time complexity of the solution

Follow the below steps to solve the problem:

- Start a loop for integer **i** from **2** to **N.**

- Check if **i** is prime or not, which can be done easily using the **isPrime(n)** method.
- If **i** is prime, check if its square is less than or equal to the given number. This will be reviewed only for squares of prime numbers, therefore reducing the number of checks.
- If the above condition is satisfied, the number will be printed and the loop will continue till **i <= n.**

Below is the implementation of the above approach:

C++Java

```
// C++ program to print all
// three-primes smaller than
// or equal to n without using
// extra space
#include <bits/stdc++.h>
using namespace std;

void numbersWith3Divisors(int);
bool isPrime(int);

// Generates all primes upto n and
// prints their squares
void numbersWith3Divisors(int N)
{
    cout << "Numbers with 3 divisors : " << endl;

    for (int i = 2; i * i <= N; i++) {

        // Check prime
        if (isPrime(i)) {
            if (i * i <= N) {

                // Print numbers in
                // the order of
                // occurrence
                cout << i * i << " ";
            }
        }
    }
}

// Check if a number is prime or not
bool isPrime(int N)
{
    for (int i = 2; i * i <= N; i++) {
        if (N % i == 0)
            return false;
    }
    return true;
```

```
}

// Driver code
int main()
{
    int N = 122;

    // Function call
    numbersWith3Divisors(N);

    return 0;
}
```

Output

```
Numbers with 3 divisors :
4 9 25 49 121
```

Time Complexity: O(sqrt N^2)

Auxiliary Space: O(1)

Count Divisors (Factors)

Given a number n, count all distinct divisors of it.

Examples:

```
Input : 18
Output : 6
Divisors of 18 are 1, 2, 3, 6, 9 and 18.

Input : 100
Output : 9
Divisors of 100 are 1, 2, 4, 5, 10, 20,
25, 50 and 100
```

Naive Approach:

This problem can be solved using various methods, but one of the most common and simple ways is to use a loop to iterate through all the possible divisors of the given number and count the ones that actually divide it without leaving any remainder.

C++Java

```
// C++ implementation to count all divisors
#include <iostream>
```

```
using namespace std;
// function to count the divisors
int countDivisors(int n)
{
        int cnt = 0;
        for (int i = 1; i <= n; i++)
        {
                if (n % i == 0)
                cnt++;
        }
        return cnt;
}
// Driver program to test above function
int main() {
        int n=72;
        cout<<countDivisors(n)<<endl;
        return 0;
}
```

Output:
12

Time Complexity : (O(n))

Space Complexity: O(1)

Better Approach:

A Better Solution would be to iterate all the numbers from 1 to sqrt(n), checking if that number divides n and incrementing number of divisors. This approach takes O(sqrt(n)) time.

C++Java

```
// C implementation of Naive method to count all
// divisors
#include <bits/stdc++.h>
using namespace std;

// function to count the divisors
int countDivisors(int n)
{
        int cnt = 0;
        for (int i = 1; i <= sqrt(n); i++) {
                if (n % i == 0) {
```

```
                        // If divisors are equal,
                        // count only one
                        if (n / i == i)
                                cnt++;

                        else // Otherwise count both
                                cnt = cnt + 2;
                }
        }
        return cnt;
}

/* Driver program to test above function */
int main()
{
        cout << "Total distinct divisors of 100 are " <<
countDivisors(100));
        return 0;
}
```

Output :

```
Total distinct divisors of 100 are : 9
```

Time Complexity : (O(n^1/2))

Space Complexity: O(1)

Efficient Approach :

The approach is implemented as a function called "countDivisors", which takes an integer 'n' as input and returns an integer as the count of divisors.

The function starts by initializing a variable 'res' to 1, which will hold the result of the count of divisors.

The function uses a for loop that iterates from 2 to the square root of 'n' (i*i <= n). This is because, for any number 'n', its divisors will always be present in the range from 1 to the square root of 'n'. Therefore, checking for divisors up to the square root of 'n' is sufficient, as divisors occur in pairs. For example, if 'n' has a divisor 'a', then it will also have a divisor 'b' such that ab = n.

Inside the for loop, a while loop is used to count the occurrences of each prime factor of 'n'. The while loop continues until 'n' is divisible by the current prime factor 'i'. For each occurrence, the counter 'cnt' is incremented. The division 'n = n/i' is performed to reduce 'n' by its prime factors, so that the next prime factor can be checked in the next iteration.

After counting the occurrences of the prime factor 'i', the result 'res' is updated by multiplying it with 'cnt+1'. This is because, for a prime factor 'i' with 'cnt' occurrences, there will be 'cnt+1' ways to choose from the occurrences of 'i' (including choosing zero occurrences), resulting in 'cnt+1' divisors.

Once the while loop ends, if 'n' is still greater than or equal to 2, it indicates that there is one more prime factor remaining for 'n' which is greater than the square root of 'n'. In that case, the result 'res' is updated by multiplying it with 2. This is because, the remaining prime factor itself and its occurrence of 1 (since 'n' is divisible by it only once) will also contribute to the count of divisors.

Finally, the function returns the calculated result 'res', which represents the total count of divisors of the given number 'n'.

C++Java

```
// C++ implementation to count all divisors
#include <iostream>
using namespace std;
// function to count the divisors
int countDivisors(int n)
{
        int res = 1;
        for (int i = 2; i*i <=n ; i++)
        {
                int cnt = 0;
            while (n % i == 0) {
                n = n/i;
                cnt++;
                }
                res = res * (cnt+1);
        }
        if(n >= 2)
    res = res * 2;

        return res;
}
/* Driver program to test above function */
int main() {
        int n=72;
        cout<<countDivisors(n)<<endl;
```

```
        return 0;
}
```

Output:
12

Time Complexity : (O(n^1/2))

Space Complexity: O(1)

Count Factors for Multiple Queries

```
Input :
3
72
15
11

Output :
12
4
2
```

Implementation :

C++

```
// C++ program to count factors for multiple queries
#include<bits/stdc++.h>
using namespace std;
int MAX=1000000;
vector<int>least_prime(MAX+1,0);
void fillSmallestSieve()
{
    // function to calculate minimum prime factor
    for(int i=1;i<=MAX;i++)
    least_prime[i]=i;
    for(int i=2;i*i<=MAX;i++)
    {
        if(least_prime[i]==i)
        {
            for(int j=i*i;j<=MAX;j+=i)
            {
                if(least_prime[j]==j)
                least_prime[j]=i;
            }
        }
```

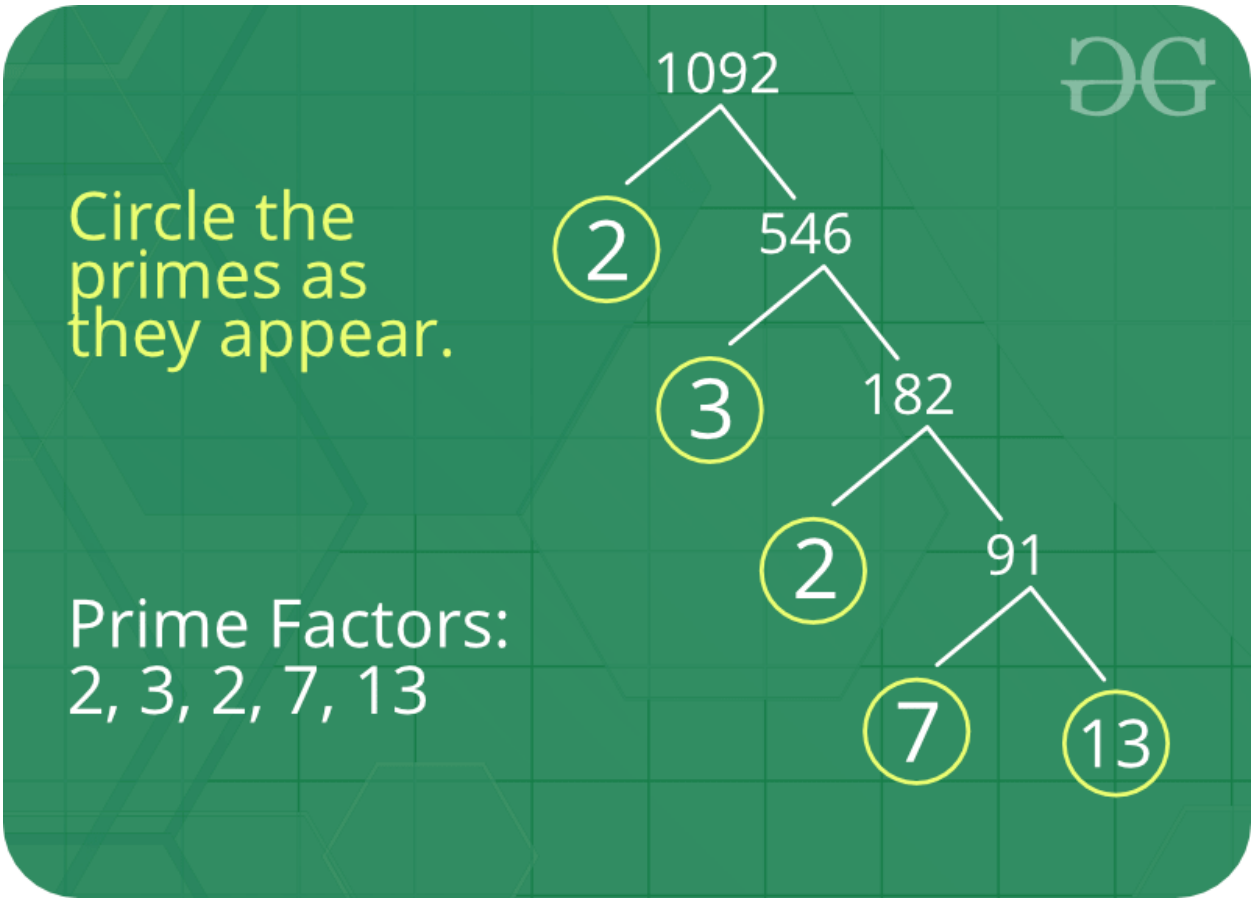
```
    }
}
int query(int n)
{
    int res=1, prev=least_prime[n];
    int count=1;
    n/=prev;
    while(n>1)
    {
        if(least_prime[n]==prev)
        count++;
        else
        {
            prev=least_prime[n];
            res=res*(count+1);
            count=1;
        }
        n/=prev;
    }
    res=res*(count+1);
    return res;
}
// Driver function
int main() {
    fillSmallestSieve(); // pre-processing
       int q;
       cin>>q;
       while(q--)
       {
           int n;
           cin>>n;
           cout<<query(n)<<endl;
       }
       return 0;
}
```

Prime Factorization

Prime Factors

Prime factor is the factor of the given number which is a prime number. Factors are the numbers you multiply together to get another number. In simple words, prime factor is finding which prime numbers multiply together to make the original number.

Example: The prime factors of 15 are 3 and 5 (because 3×5=15, and 3 and 5 are prime numbers).



Some interesting fact about Prime Factor :

1. There is only one (unique!) set of prime factors for any number.
2. In order to maintain this property of unique prime factorizations, it is necessary that the number one, 1, be categorized as neither prime nor composite.
3. Prime factorizations can help us with divisibility, simplifying fractions, and finding common denominators for fractions.
4. Pollard's Rho is a prime factorization algorithm, particularly fast for a large composite number with small prime factors.
5. Cryptography is the study of secret codes. Prime Factorization is very important to people who try to make (or break) secret codes based on numbers.

How to print a prime factor of a number?

Naive solution:

Given a number n, write a function to print all prime factors of n. For example, if the input number is 12, then output should be "2 2 3" and if the input number is 315, then output should be "3 3 5 7".

Following are the steps to find all prime factors:

1. While *n* is divisible by 2, print 2 and divide n by 2.
2. After step 1, *n* must be odd. Now start a loop from i = 3 to square root of *n*. While *i* divides *n*, print *i* and divide *n* by *i*, increment *i* by 2 and continue.
3. If *n* is a prime number and is greater than 2, then *n* will not become 1 by above two steps. So print *n* if it is greater than 2.

C++Java

```
// Program to print all prime factors
# include <stdio.h>
# include <math.h>

// A function to print all prime factors of a given number n
void primeFactors(int n)
{
        // Print the number of 2s that divide n
        while (n%2 == 0)
        {
                printf("%d ", 2);
                n = n/2;
        }

        // n must be odd at this point. So we can skip
        // one element (Note i = i +2)
        for (int i = 3; i <= sqrt(n); i = i+2)
        {
                // While i divides n, print i and divide n
                while (n%i == 0)
                {
                        printf("%d ", i);
                        n = n/i;
                }
        }

        // This condition is to handle the case when n
        // is a prime number greater than 2
        if (n > 2)
                printf ("%d ", n);
}

/* Driver program to test above function */
int main()
{
        int n = 315;
        primeFactors(n);
        return 0;
}
```

Output:

```
3 3 5 7
```

Time Complexity: O(sqrt(n))

Auxiliary Space: O(1)

More efficient solution:

C++Java

```
#include <iostream>
#include <limits.h>
using namespace std;

 void printPrimeFactors(int n)
{
        if(n <= 1)
                return;

        while(n % 2 == 0)
        {
                cout<<2<<" ";

                n = n / 2;
        }

        while(n % 3 == 0)
        {
                cout<<3<<" ";

                n = n / 3;
        }

        for(int i=5; i*i<=n; i=i+6)
        {
                while(n % i == 0)
                {
                        cout<<i<<" ";

                        n = n / i;
                }

                while(n % (i + 2) == 0)
```

```
                {
                        cout<<(i + 2)<<" ";

                        n = n / (i + 2);
                }
        }

        if(n > 3)
                cout<<n<<" ";

        cout<<endl;
}

int main() {

        int n = 315;

        printPrimeFactors(n);

        return 0;
}
```

Output:

```
3 3 5 7
```

Smallest Prime Factors

Given a number **n**, print **least prime factors** of all numbers from 1 to n. The least prime factor of an integer n is the smallest prime number that divides the number. The least prime factor of all even numbers is 2. A prime number is its own least prime factor (as well as its own greatest prime factor).
Note: We need to print 1 for 1.
Example :

```
Input : 6
Output : Least Prime factor of 1: 1
         Least Prime factor of 2: 2
         Least Prime factor of 3: 3
         Least Prime factor of 4: 2
         Least Prime factor of 5: 5
         Least Prime factor of 6: 2
```

Naive Approach:

The naive approach to print the least prime factors of numbers from 1 to n involves iterating through each number and finding its least prime factor using nested loops.

C++Java

```cpp
// C++ program for snallest prime factors
#include <bits/stdc++.h>
using namespace std;

class GFG{
    public:
    //Function for smallest prime factors
    int smallestPrime(int n){
        for(int i=2;i*i<=n;i++)
        {
            if(n%i==0)
            {
                return i;
            }
        }
        return n;
    }

    void printSmallestPrimeFactor(int n){
         for(int i=1;i<=n;i++)
         {
             cout<<smallestPrime(i)<<" ";
         }
    }
};


// Driver Code
int main()
{
        int n = 6;
    class GFG obj;
        // Function call
        obj.printSmallestPrimeFactor(n);

        return 0;
}
```

Output:

1 2 3 2 5 2

Efficient Approach:

We can use a variation of sieve of Eratosthenes to solve the above problem.

1. Create a list of consecutive integers from 2 through n: (2, 3, 4, ..., n).
2. Initially, let i equal 2, the smallest prime number.
3. Enumerate the multiples of i by counting to n from 2i in increments of i, and mark them as having least prime factor as i (if not already marked). Also mark i as least prime factor of i (i itself is a prime number).
4. Find the first number greater than i in the list that is not marked. If there was no such number, stop. Otherwise, let i now equal this new number (which is the next prime), and repeat from step 3.

Below is the implementation of the algorithm, where least_prime[] saves the value of the least prime factor corresponding to the respective index.

C++Java

```
// C++ program to print the least prime factors
// of numbers less than or equal to
// n using modified Sieve of Eratosthenes
#include<bits/stdc++.h>
using namespace std;

void leastPrimeFactor(int n)
{
        // Create a vector to store least primes.
        // Initialize all entries as 0.
        vector<int> least_prime(n+1, 0);

        // We need to print 1 for 1.
        least_prime[1] = 1;

        for (int i = 2; i <= n; i++)
        {
                // least_prime[i] == 0
                // means it i is prime
                if (least_prime[i] == 0)
                {
                        // marking the prime number
                        // as its own lpf
                        least_prime[i] = i;

                        // mark it as a divisor for all its
                        // multiples if not already marked
                        for (int j = i*i; j <= n; j += i)
                                if (least_prime[j] == 0)
                                least_prime[j] = i;
```

```
                }
        }

        // print least prime factor of
        // of numbers till n
        for (int i = 1; i <= n; i++)
                cout << "Least Prime factor of "
                        << i << ": " << least_prime[i] << "\n";
}

// Driver program to test above function
int main()
{
        int n = 10;
        leastPrimeFactor(n);
        return 0;
}
```

Output

```
Least Prime factor of 1: 1
Least Prime factor of 2: 2
Least Prime factor of 3: 3
Least Prime factor of 4: 2
Least Prime factor of 5: 5
Least Prime factor of 6: 2
Least Prime factor of 7: 7
Least Prime factor of 8: 2
Least Prime factor of 9: 3
Least Prime factor of 10: 2
```

Time Complexity: O(nloglog(n))
Auxiliary Space: O(n)

Prime Factorization for Multiple Queries

We can calculate the prime factorization of a number **"n"** in **O(sqrt(n))** as discussed here. But O(sqrt n) method times out when we need to answer multiple queries regarding prime factorization.
In this article, we study an efficient method to calculate the prime factorization using O(n) space and **O(log n)** time complexity with pre-computation allowed.

Key Concept: Our idea is to store the Smallest Prime Factor(SPF) for every number. Then to calculate the prime factorization of the given number by dividing the given number repeatedly with its smallest prime factor till it becomes 1.

To calculate to smallest prime factor for every number we will use the sieve of eratosthenes. In original Sieve, every time we mark a number as not-prime, we store the corresponding smallest prime factor for that number.

Now, after we are done with pre-calculating the smallest prime factor for every number we will divide our number n (whose prime factorization is to be calculated) by its corresponding smallest prime factor till n becomes 1.

Pseudo Code for prime factorization assuming SPFs are computed :

```
PrimeFactors[] // To store result

i = 0  // Index in PrimeFactors

while n != 1 :

    // SPF : smallest prime factor
    PrimeFactors[i] = SPF[n]
    i++
    n = n / SPF[n]
```

The implementation for the above method is given below :

C++

```
// C++ program to find prime factorization of a
// number n in O(Log n) time with precomputation
// allowed.
#include "bits/stdc++.h"
using namespace std;

#define MAXN 100001

// stores smallest prime factor for every number
int spf[MAXN];

// Calculating SPF (Smallest Prime Factor) for every
// number till MAXN.
// Time Complexity : O(nloglogn)
void sieve()
{
```

```
        spf[1] = 1;
        for (int i=2; i<MAXN; i++)

                // marking smallest prime factor for every
                // number to be itself.
                spf[i] = i;

        // separately marking spf for every even
        // number as 2
        for (int i=4; i<MAXN; i+=2)
                spf[i] = 2;

        for (int i=3; i*i<MAXN; i++)
        {
                // checking if i is prime
                if (spf[i] == i)
                {
                        // marking SPF for all numbers divisible by i
                        for (int j=i*i; j<MAXN; j+=i)

                                // marking spf[j] if it is not
                                // previously marked
                                if (spf[j]==j)
                                        spf[j] = i;
                }
        }
}

// A O(log n) function returning primefactorization
// by dividing by smallest prime factor at every step
vector<int> getFactorization(int x)
{
        vector<int> ret;
        while (x != 1)
        {
                ret.push_back(spf[x]);
                x = x / spf[x];
        }
        return ret;
}

// driver program for above function
int main(int argc, char const *argv[])
{
        // precalculating Smallest Prime Factor
        sieve();
        int x = 12246;
        cout << "prime factorization for " << x << " : ";
```

```
        // calling getFactorization function
        vector <int> p = getFactorization(x);

        for (int i=0; i<p.size(); i++)
                cout << p[i] << " ";
        cout << endl;
        return 0;
}
```

Output

```
prime factorization for 12246 : 2 3 13 157
```

Time Complexity: O(log n), for each query (Time complexity for precomputation is not included)
Auxiliary Space: O(1)

Note : The above code works well for n upto the order of 10^7. Beyond this we will face memory issues.

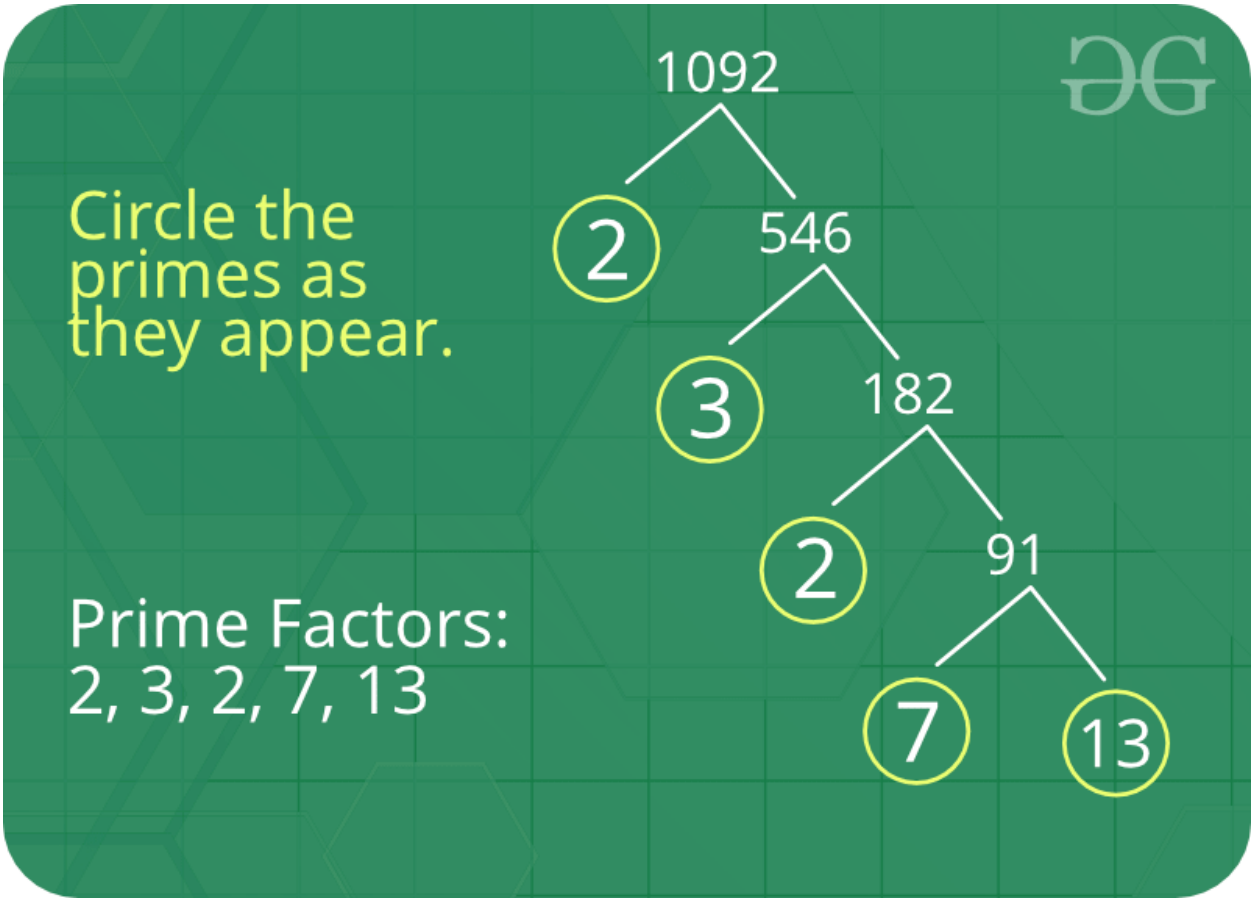
Time Complexity: The precomputation for smallest prime factor is done in O(n log log n) using sieve. Whereas in the calculation step we are dividing the number every time by the smallest prime number till it becomes 1.

Prime Factorization

Prime Factors

Prime factor is the factor of the given number which is a prime number. Factors are the numbers you multiply together to get another number. In simple words, prime factor is finding which prime numbers multiply together to make the original number.

Example: The prime factors of 15 are 3 and 5 (because 3×5=15, and 3 and 5 are prime numbers).



Some interesting fact about Prime Factor :

1. There is only one (unique!) set of prime factors for any number.
2. In order to maintain this property of unique prime factorizations, it is necessary that the number one, 1, be categorized as neither prime nor composite.
3. Prime factorizations can help us with divisibility, simplifying fractions, and finding common denominators for fractions.
4. Pollard's Rho is a prime factorization algorithm, particularly fast for a large composite number with small prime factors.
5. Cryptography is the study of secret codes. Prime Factorization is very important to people who try to make (or break) secret codes based on numbers.

How to print a prime factor of a number?

Naive solution:

Given a number n, write a function to print all prime factors of n. For example, if the input number is 12, then output should be "2 2 3" and if the input number is 315, then output should be "3 3 5 7".

Following are the steps to find all prime factors:

1. While *n* is divisible by 2, print 2 and divide n by 2.

2. After step 1, *n* must be odd. Now start a loop from i = 3 to square root of *n*. While *i* divides *n*, print *i* and divide *n* by *i*, increment *i* by 2 and continue.
3. If *n* is a prime number and is greater than 2, then *n* will not become 1 by above two steps. So print *n* if it is greater than 2.

C++Java

```
// Program to print all prime factors
# include <stdio.h>
# include <math.h>

// A function to print all prime factors of a given number n
void primeFactors(int n)
{
        // Print the number of 2s that divide n
        while (n%2 == 0)
        {
                printf("%d ", 2);
                n = n/2;
        }

        // n must be odd at this point. So we can skip
        // one element (Note i = i +2)
        for (int i = 3; i <= sqrt(n); i = i+2)
        {
                // While i divides n, print i and divide n
                while (n%i == 0)
                {
                        printf("%d ", i);
                        n = n/i;
                }
        }

        // This condition is to handle the case when n
        // is a prime number greater than 2
        if (n > 2)
                printf ("%d ", n);
}

/* Driver program to test above function */
int main()
{
        int n = 315;
        primeFactors(n);
        return 0;
}
```

Output:

```
3 3 5 7
```

Time Complexity: O(sqrt(n))

Auxiliary Space: O(1)

More efficient solution:

C++Java

```cpp
#include <iostream>
#include <limits.h>
using namespace std;

 void printPrimeFactors(int n)
{
        if(n <= 1)
                return;

        while(n % 2 == 0)
        {
                cout<<2<<" ";

                n = n / 2;
        }

        while(n % 3 == 0)
        {
                cout<<3<<" ";

                n = n / 3;
        }

        for(int i=5; i*i<=n; i=i+6)
        {
                while(n % i == 0)
                {
                        cout<<i<<" ";

                        n = n / i;
                }

                while(n % (i + 2) == 0)
                {
                        cout<<(i + 2)<<" ";

                        n = n / (i + 2);
                }
```

```
        }

        if(n > 3)
                cout<<n<<" ";

        cout<<endl;
}

int main() {

        int n = 315;

        printPrimeFactors(n);

        return 0;
}
```

Output:

```
3 3 5 7
```

Mark as Read

Report An Issue

Smallest Prime Factors

Given a number **n**, print **least prime factors** of all numbers from 1 to n. The least prime factor of an integer n is the smallest prime number that divides the number. The least prime factor of all even numbers is 2. A prime number is its own least prime factor (as well as its own greatest prime factor).

Note: We need to print 1 for 1.

Example :

```
Input : 6
Output : Least Prime factor of 1: 1
         Least Prime factor of 2: 2
         Least Prime factor of 3: 3
         Least Prime factor of 4: 2
         Least Prime factor of 5: 5
         Least Prime factor of 6: 2
```

Naive Approach:

The naive approach to print the least prime factors of numbers from 1 to n involves iterating through each number and finding its least prime factor using nested loops.

C++Java

```cpp
// C++ program for snallest prime factors
#include <bits/stdc++.h>
using namespace std;

class GFG{
    public:
    //Function for smallest prime factors
    int smallestPrime(int n){
        for(int i=2;i*i<=n;i++)
        {
            if(n%i==0)
            {
                return i;
            }
        }
        return n;
    }

    void printSmallestPrimeFactor(int n){
         for(int i=1;i<=n;i++)
         {
             cout<<smallestPrime(i)<<" ";
         }
    }
};


// Driver Code
int main()
{
        int n = 6;
    class GFG obj;
        // Function call
        obj.printSmallestPrimeFactor(n);

        return 0;
}
```

Output:

```
1 2 3 2 5 2
```

Efficient Approach:

We can use a variation of sieve of Eratosthenes to solve the above problem.

1. Create a list of consecutive integers from 2 through n: (2, 3, 4, ..., n).
2. Initially, let i equal 2, the smallest prime number.
3. Enumerate the multiples of i by counting to n from 2i in increments of i, and mark them as having least prime factor as i (if not already marked). Also mark i as least prime factor of i (i itself is a prime number).
4. Find the first number greater than i in the list that is not marked. If there was no such number, stop. Otherwise, let i now equal this new number (which is the next prime), and repeat from step 3.

Below is the implementation of the algorithm, where least_prime[] saves the value of the least prime factor corresponding to the respective index.

C++Java

```
// C++ program to print the least prime factors
// of numbers less than or equal to
// n using modified Sieve of Eratosthenes
#include<bits/stdc++.h>
using namespace std;

void leastPrimeFactor(int n)
{
        // Create a vector to store least primes.
        // Initialize all entries as 0.
        vector<int> least_prime(n+1, 0);

        // We need to print 1 for 1.
        least_prime[1] = 1;

        for (int i = 2; i <= n; i++)
        {
                // least_prime[i] == 0
                // means it i is prime
                if (least_prime[i] == 0)
                {
                        // marking the prime number
                        // as its own lpf
                        least_prime[i] = i;

                        // mark it as a divisor for all its
                        // multiples if not already marked
                        for (int j = i*i; j <= n; j += i)
                                if (least_prime[j] == 0)
                                least_prime[j] = i;
                }
        }
```

```
        // print least prime factor of
        // of numbers till n
        for (int i = 1; i <= n; i++)
                cout << "Least Prime factor of "
                        << i << ": " << least_prime[i] << "\n";
}

// Driver program to test above function
int main()
{
        int n = 10;
        leastPrimeFactor(n);
        return 0;
}
```

Output

```
Least Prime factor of 1: 1
Least Prime factor of 2: 2
Least Prime factor of 3: 3
Least Prime factor of 4: 2
Least Prime factor of 5: 5
Least Prime factor of 6: 2
Least Prime factor of 7: 7
Least Prime factor of 8: 2
Least Prime factor of 9: 3
Least Prime factor of 10: 2
```

Time Complexity: O(nloglog(n))
Auxiliary Space: O(n)

Prime Factorization for Multiple Queries

We can calculate the prime factorization of a number **"n"** in **O(sqrt(n))** as discussed here. But O(sqrt n) method times out when we need to answer multiple queries regarding prime factorization.
In this article, we study an efficient method to calculate the prime factorization using O(n) space and **O(log n)** time complexity with pre-computation allowed.

Key Concept: Our idea is to store the Smallest Prime Factor(SPF) for every number. Then to calculate the prime factorization of the given number by dividing the given number

repeatedly with its smallest prime factor till it becomes 1.

To calculate to smallest prime factor for every number we will use the sieve of eratosthenes. In original Sieve, every time we mark a number as not-prime, we store the corresponding smallest prime factor for that number.

Now, after we are done with pre-calculating the smallest prime factor for every number we will divide our number n (whose prime factorization is to be calculated) by its corresponding smallest prime factor till n becomes 1.

Pseudo Code for prime factorization assuming SPFs are computed :

```
PrimeFactors[] // To store result

i = 0  // Index in PrimeFactors

while n != 1 :

    // SPF : smallest prime factor
    PrimeFactors[i] = SPF[n]
    i++
    n = n / SPF[n]
```

The implementation for the above method is given below :

C++

```
// C++ program to find prime factorization of a
// number n in O(Log n) time with precomputation
// allowed.
#include "bits/stdc++.h"
using namespace std;

#define MAXN 100001

// stores smallest prime factor for every number
int spf[MAXN];

// Calculating SPF (Smallest Prime Factor) for every
// number till MAXN.
// Time Complexity : O(nloglogn)
void sieve()
{
        spf[1] = 1;
        for (int i=2; i<MAXN; i++)
```

```
                // marking smallest prime factor for every
                // number to be itself.
                spf[i] = i;

        // separately marking spf for every even
        // number as 2
        for (int i=4; i<MAXN; i+=2)
                spf[i] = 2;

        for (int i=3; i*i<MAXN; i++)
        {
                // checking if i is prime
                if (spf[i] == i)
                {
                        // marking SPF for all numbers divisible by i
                        for (int j=i*i; j<MAXN; j+=i)

                                // marking spf[j] if it is not
                                // previously marked
                                if (spf[j]==j)
                                        spf[j] = i;
                }
        }
}

// A O(log n) function returning primefactorization
// by dividing by smallest prime factor at every step
vector<int> getFactorization(int x)
{
        vector<int> ret;
        while (x != 1)
        {
                ret.push_back(spf[x]);
                x = x / spf[x];
        }
        return ret;
}

// driver program for above function
int main(int argc, char const *argv[])
{
        // precalculating Smallest Prime Factor
        sieve();
        int x = 12246;
        cout << "prime factorization for " << x << " : ";

        // calling getFactorization function
        vector <int> p = getFactorization(x);
```

```
        for (int i=0; i<p.size(); i++)
                cout << p[i] << " ";
        cout << endl;
        return 0;
}
```

Output

```
prime factorization for 12246 : 2 3 13 157
```

Time Complexity: O(log n), for each query (Time complexity for precomputation is not included)
Auxiliary Space: O(1)

Note : The above code works well for n upto the order of 10^7. Beyond this we will face memory issues.

Time Complexity: The precomputation for smallest prime factor is done in O(n log log n) using sieve. Whereas in the calculation step we are dividing the number every time by the smallest prime number till it becomes 1.

Prime Numbers

Check for Prime

What are prime numbers?

- A prime number is a natural number greater than **1**, which is only divisible by 1 and itself. First few prime numbers are: 2 3 5 7 11 13 17 19 23.....



Prime numbers

- In other words, the prime number is a positive integer greater than 1 that has exactly two factors, 1 and the number itself.
- There are many prime numbers, such as 2, 3, 5, 7, 11, 13, etc.
- Keep in mind that 1 cannot be either prime or composite.
- The remaining numbers, except for 1, are classified as prime and composite numbers.

Some interesting facts about Prime numbers:

- Except for 2, which is the smallest prime number and the only even prime number, all prime numbers are odd numbers.
- Every prime number can be represented in form of **6n + 1** or **6n – 1** except the prime numbers 2 and 3, where n is a natural number.
- 2 and 3 are only two consecutive natural numbers that are prime.
- Goldbach Conjecture: Every even integer greater than 2 can be expressed as the sum of two primes.
- Wilson Theorem: Wilson's theorem states that a natural number p > 1 is a prime number if and only if

```
(p - 1) ! ≡  -1   mod p
OR  (p - 1) ! ≡  (p-1) mod p
```

- Fermat's Little Theorem: If n is a prime number, then for every a, 1 <= a < n,

```
  a^(n-1) ≡ 1 (mod n)
        OR
  a^(n-1) % n = 1
```

- Prime Number Theorem: The probability that a given, randomly chosen number n is prime is inversely proportional to its number of digits, or to the logarithm of n.
- Lemoine's Conjecture: Any odd integer greater than 5 can be expressed as a sum of an odd prime (all primes other than 2 are odd) and an even semiprime. A semiprime number is a product of two prime numbers. This is called Lemoine's conjecture.

Properties of prime numbers:

- Every number greater than 1 can be divided by at least one prime number.
- Every even positive integer greater than 2 can be expressed as the sum of two primes.
- Except 2, all other prime numbers are odd. In other words, we can say that 2 is the only even prime number.
- Two prime numbers are always coprime to each other.
- Each composite number can be factored into prime factors and individually all of these are unique in nature.

Prime numbers and co-prime numbers:

It is important to distinguish between prime numbers and co-prime numbers. Listed below are the differences between prime and co-prime numbers.

- A coprime number is always considered as a pair, whereas a prime number is considered as a single number.
- Co-prime numbers are numbers that have no common factor except 1. In contrast, prime numbers do not have such a condition.
- A co-prime number can be either prime or composite, but its greatest common factor (GCF) must always be 1. Unlike composite numbers, prime numbers have only two factors, 1 and the number itself.
- **Example of co-prime:** 13 and 15 are co-primes. The factors of 13 are 1 and 13 and the factors of 15 are 1, 3 and 5. We can see that they have only 1 as their common factor, therefore, they are coprime numbers.
- **Example of prime:** A few examples of prime numbers are 2, 3, 5, 7 and 11 etc.

How do we check whether a number is Prime or not?

Naive Approach: A naive solution is to iterate through all numbers from 2 to sqrt(n) and for every number check if it divides n. If we find any number that divides, we return false.

Below is the implementation:

C++Java

```
// A school method based C++ program to
// check if a number is prime
#include <bits/stdc++.h>
using namespace std;

// function check whether a number
// is prime or not
bool isPrime(int n)
{
        // Corner case
        if (n <= 1)
                return false;

        // Check from 2 to square root of n
        for (int i = 2; i <= sqrt(n); i++)
                if (n % i == 0)
                        return false;

        return true;
}

// Driver Code
int main()
```

```
{
        isPrime(11) ? cout << " true\n" : cout << " false\n";
        return 0;
}
```

Output

```
 true
```

Time Complexity: O(sqrt(n))
Auxiliary space: O(1)

Efficient approach: To check whether the number is prime or not follow the below idea:

In the previous approach given if the size of the given number is too large then its square root will be also very large, so to deal with large size input we will deal with a few numbers such as 1, 2, 3, and the numbers which are divisible by 2 and 3 in separate cases and for remaining numbers, we will iterate our loop from 5 to sqrt(n) and check for each iteration whether that (iteration) or (that iteration + 2) divides n or not. If we find any number that divides, we return false.

Below is the implementation for the above idea:

C++Java

```
// A school method based C++ program to
// check if a number is prime
#include <bits/stdc++.h>
using namespace std;

// function check whether a number
// is prime or not
bool isPrime(int n)
{
        // Check if n=1 or n=0
        if (n <= 1)
                return false;
        // Check if n=2 or n=3
        if (n == 2 || n == 3)
                return true;
        // Check whether n is divisible by 2 or 3
        if (n % 2 == 0 || n % 3 == 0)
                return false;
        // Check from 5 to square root of n
        // Iterate i by (i+6)
        for (int i = 5; i <= sqrt(n); i = i + 6)
```

```
                if (n % i == 0 || n % (i + 2) == 0)
                        return false;

        return true;
}

// Driver Code
int main()
{
        isPrime(11) ? cout << "true\n" : cout << "false\n";
        return 0;
}
```

Output :

```
true
```

Time complexity: O(sqrt(n))
Auxiliary space: O(1)

Sieve of Eratosthenes

Given a number n, print all primes smaller than or equal to n. It is also given that n is a small number.

Example:

Input : *n =10*
Output : *2 3 5 7*

Input : *n = 20*
Output: *2 3 5 7 11 13 17 19*

The sieve of Eratosthenes is one of the most efficient ways to find all primes smaller than n when n is smaller than 10 million or so.

Following is the algorithm to find all the prime numbers less than or equal to a given integer n by the Eratosthene's method:
When the algorithm terminates, all the numbers in the list that are not marked are prime.

Explanation with Example:

Let us take an example when n = 50. So we need to print all prime numbers smaller than or equal to 50.

We create a list of all numbers from 2 to 50.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

According to the algorithm we will mark all the numbers which are divisible by 2 and are greater than or equal to the square of it.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

Now we move to our next unmarked number 3 and mark all the numbers which are multiples of 3 and are greater than or equal to the square of it.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

We move to our next unmarked number 5 and mark all multiples of 5 and are greater than or equal to the square of it.

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

We continue this process and our final table will look like below:

	2	3	4	5	6	7	8	9	10
11	12	13	14	15	16	17	18	19	20
21	22	23	24	25	26	27	28	29	30
31	32	33	34	35	36	37	38	39	40
41	42	43	44	45	46	47	48	49	50

So the prime numbers are the unmarked ones: 2,3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43, 47.

Implementation:

Following is the implementation of the above algorithm. In the following implementation, a boolean array arr[] of size n is used to mark multiples of prime numbers.

C++Java

```
// C++ program to print all primes smaller than or equal to
// n using Sieve of Eratosthenes
#include <bits/stdc++.h>
using namespace std;

void SieveOfEratosthenes(int n)
{
        // Create a boolean array "prime[0..n]" and initialize
        // all entries it as true. A value in prime[i] will
        // finally be false if i is Not a prime, else true.
        bool prime[n + 1];
        memset(prime, true, sizeof(prime));

        for (int p = 2; p * p <= n; p++) {
                // If prime[p] is not changed, then it is a prime
                if (prime[p] == true) {
                        // Update all multiples of p greater than or
                        // equal to the square of it numbers which are
```

```
                        // multiple of p and are less than p^2 are
                        // already been marked.
                        for (int i = p * p; i <= n; i += p)
                                prime[i] = false;
                }
        }

        // Print all prime numbers
        for (int p = 2; p <= n; p++)
                if (prime[p])
                        cout << p << " ";
}

// Driver Code
int main()
{
        int n = 30;
        cout << "Following are the prime numbers smaller "
                << " than or equal to " << n << endl;
        SieveOfEratosthenes(n);
        return 0;
}
```

Output

```
Following are the prime numbers smaller than or equal to 30
2 3 5 7 11 13 17 19 23 29
```

Time Complexity: O(n*log(log(n)))
Auxiliary Space: O(n)

Segmented Sieve (Print Primes in a Range)

Given a range [low, high], print all primes in this range? For example, if the given range is [10, 20], then output is 11, 13, 17, 19.

A **Naive approach** is to run a loop from low to high and check each number for primeness. A Better Approach is to precalculate primes up to the maximum limit using Sieve of Eratosthenes, then print all prime numbers in range.
The above approach looks good, but consider the input range [50000, 55000]. the above Sieve approach would precalculate primes from 2 to 50100. This causes a waste of memory as well as time. Below is the Segmented Sieve based approach.

Segmented Sieve (Background)

Below are basic steps to get an idea of how Segmented Sieve works

1. Use Simple Sieve to find all primes up to a predefined limit (square root of 'high' is used in below code) and store these primes in an array "prime[]". Basically we call Simple Sieve for a limit and we not only find prime numbers, but also puts them in a separate array prime[].
2. Create an array mark[high-low+1]. Here we need only O(n) space where **n** is number of elements in given range.
3. Iterate through all primes found in step 1. For every prime, mark its multiples in given range [low..high].

So unlike simple sieve, we don't check for all multiples of every number smaller than square root of high, we only check for multiples of primes found in step 1. And we don't need O(high) space, we need O(sqrt(high) + n) space.

Below is the implementation of above idea.

C++Java

```
#include <bits/stdc++.h>
using namespace std;
// fillPrime function fills primes from 2 to sqrt of high in
chprime(vector) array
void fillPrimes(vector<int>& prime, int high)
{
    bool ck[high + 1];
    memset(ck, true, sizeof(ck));
    ck[1] = false;
    ck[0] = false;
    for (int i = 2; (i * i) <= high; i++) {
        if (ck[i] == true) {
            for (int j = i * i; j <= sqrt(high); j = j + i) {
                ck[j] = false;
            }
        }
    }
    for (int i = 2; i * i <= high; i++) {
        if (ck[i] == true) {
            prime.push_back(i);
        }
    }
}
// in segmented sieve we check for prime from range [low, high]
void segmentedSieve(int low, int high)
{
    if (low<2 and high>=2){
        low = 2;
    }//for handling corner case when low = 1 and we all know 1 is not
prime no.
```

```
    bool prime[high - low + 1];
  //here prime[0] indicates whether low is prime or not similarly prime[1]
indicates whether low+1 is prime or not
    memset(prime, true, sizeof(prime));

    vector<int> chprime;
    fillPrimes(chprime, high);
    //chprimes has primes in range [2,sqrt(n)]
     // we take primes from 2 to sqrt[n] because the multiples of all
primes below high are marked by these
   // primes in range 2 to sqrt[n] for eg: for number 7 its multiples i.e
14 is marked by 2, 21 is marked by 3,
   // 28 is marked by 4, 35 is marked by 5, 42 is marked 6, so 49 will be
first marked by 7 so all number before 49
  // are marked by primes in range [2,sqrt(49)]
    for (int i : chprime) {
        int lower = (low / i);
        //here lower means the first multiple of prime which is in range
[low,high]
        //for eg: 3's first multiple in range [28,40] is 30
        if (lower <= 1) {
            lower = i + i;
        }
        else if (low % i) {
            lower = (lower * i) + i;
        }
        else {
            lower = (lower * i);
        }
        for (int j = lower; j <= high; j = j + i) {
            prime[j - low] = false;
        }
    }

    for (int i = low; i <= high; i++) {
        if (prime[i - low] == true) {
            cout << (i) << " ";
        }
    }
}
int main()
{
    // low should be greater than or equal to 2
    int low = 2;
    // low here is the lower limit
    int high = 100;
    // high here is the upper limit
      // in segmented sieve we calculate primes in range [low,high]
```

```
   // here we initially we find primes in range [2,sqrt(high)] to mark all
their multiples as not prime
   //then we mark all their(primes) multiples in range [low,high] as false
   // this is a modified sieve of eratosthenes , in standard sieve of
eratosthenes we find prime from 1 to n(given number)
   // in segmented sieve we only find primes in a given interval
  cout<<"Primes in range "<<low<<" to "<< high<<" are\n";
    segmentedSieve(low, high);
}
```

Output

```
Primes in range 2 to 100 are
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83
89 97
```

Time Complexity : O(n)

Auxiliary Space: O(n)

Segmented Sieve (What if 'high' value of range is too high and range is also big)

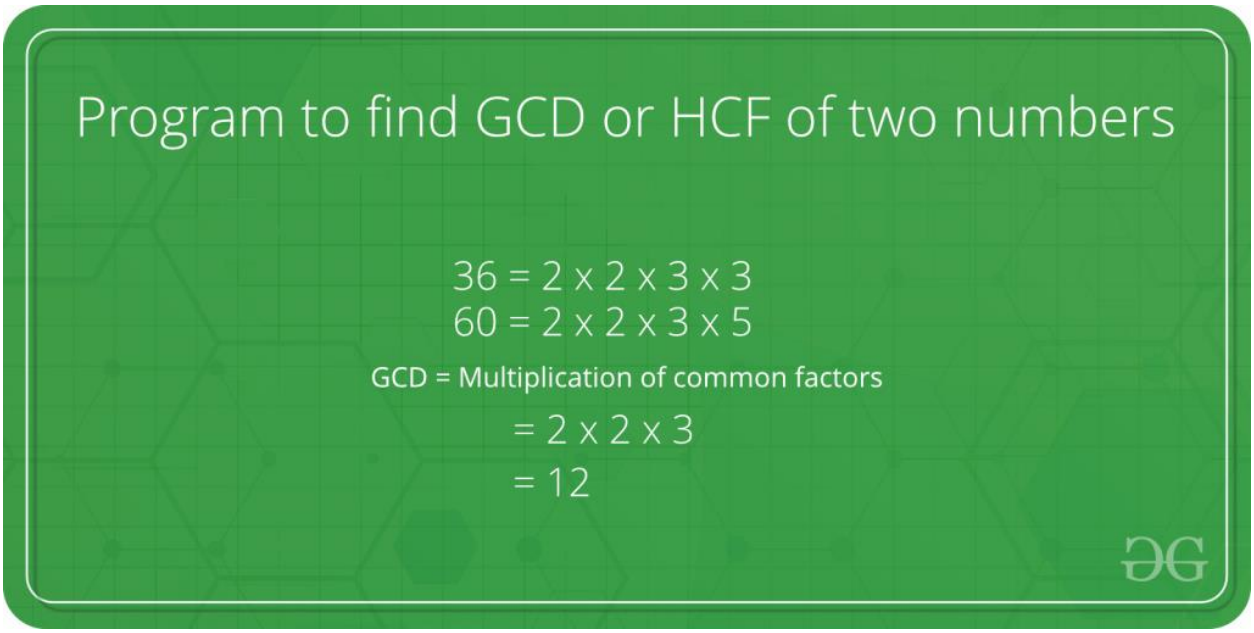
Consider a situation where given high value is so high that neither sqrt(high) nor O(high-low+1) can fit in memory. How to find primes in range. For this situation, we run step 1 (Simple Sieve) only for a limit that can fit in memory. Then we divide given range in different segments. For every segment, we run step 2 and 3 considering low and high as end points of current segment. We add primes of current segment to prime[] before running the next segment.

This article is contributed by Utkarsh Trivedi. Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above

GCD and LCM

GCD of Two Numbers

GCD (Greatest Common Divisor) or HCF (Highest Common Factor) of two numbers is the largest number that divides both of them.



For example, GCD of 20 and 28 is 4 and GCD of 98 and 56 is 14.

C++

```cpp
#include<iostream>
using namespace std;

int main()
{
    int a, b;
    cin >> a >> b;

    int x = min(a,b);
    int ans = 1;

    for(int i = 1; i<=x; i++){
        if(a%i==0 and b%i==0)
            ans = i;
    }
    cout << ans;
    return 0;
}
```

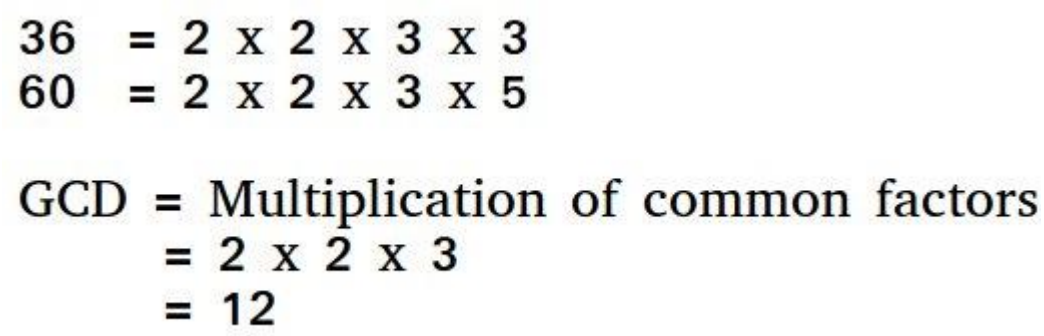
INPUT :

```
15 6
```

OUTPUT :

Euclidean algorithms (Basic and Extended)

GCD of two numbers is the largest number that divides both of them. A simple way to find GCD is to factorize both numbers and multiply common prime factors.



Basic Euclidean Algorithm for GCD: The algorithm is based on the below facts.

- If we subtract a smaller number from a larger one (we reduce a larger number), GCD doesn't change. So if we keep subtracting repeatedly the larger of two, we end up with GCD.
- Now instead of subtraction, if we divide the smaller number, the algorithm stops when we find the remainder 0.

Below is a recursive function to evaluate gcd using Euclid's algorithm.

CPPCJavaPython3C#PHPJavascript

```cpp
// C++ program to demonstrate
// Basic Euclidean Algorithm

#include <bits/stdc++.h>
using namespace std;

// Function to return
// gcd of a and b
int gcd(int a, int b)
{
    if (a == 0)
        return b;
    return gcd(b % a, a);
}
```

```
// Driver Code
int main()
{
    int a = 10, b = 15;
    cout << "GCD(" << a << ", "
         << b << ") = " << gcd(a, b)
                        << endl;
    a = 35, b = 10;
    cout << "GCD(" << a << ", "
         << b << ") = " << gcd(a, b)
                        << endl;
    a = 31, b = 2;
    cout << "GCD(" << a << ", "
         << b << ") = " << gcd(a, b)
                        << endl;
    return 0;
}
```

Output :

```
GCD(10, 15) = 5
GCD(35, 10) = 5
GCD(31, 2) = 1
```

Time Complexity: O(Log min(a, b)), **Auxiliary Space: O(log(min(a,b)).**

Extended Euclidean Algorithm:

Extended Euclidean algorithm also finds integer coefficients x and y such that:

```
  ax + by = gcd(a, b)
```

Examples:

```
Input: a = 30, b = 20
Output: gcd = 10
        x = 1, y = -1
(Note that 30*1 + 20*(-1) = 10)

Input: a = 35, b = 15
Output: gcd = 5
        x = 1, y = -2
(Note that 35*1 + 15*(-2) = 5)
```

The extended Euclidean algorithm updates the results of gcd(a, b) using the results calculated by recursive call gcd(b%a, a). Let values of x and y calculated by the recursive call be x_1 and y_1. x and y are updated using the below expressions.

```
ax + by = gcd(a,b)
gcd(a,b) = gcd(b%a, a)
gcd(b%a, a) = (b%a)x₁ + ay₁
```

$ax + by = (b\%a)x_1 + ay_1$
$ax + by = (b - [b/a] * a)x_1 + ay_1$
$ax + by = a(y_1 - [b/a] * x_1) + bx_1$
Comparing LHS and RHS,

$x = y_1 - \lfloor b/a \rfloor * x_1$
$y = x_1$

Below is an implementation based on the above formulas.

C++CJavaPython3C#PHPJavascript

```cpp
// C++ program to demonstrate working of
// extended Euclidean Algorithm
#include <bits/stdc++.h>
using namespace std;

// Function for extended Euclidean Algorithm
int gcdExtended(int a, int b, int *x, int *y)
{
    // Base Case
    if (a == 0)
    {
        *x = 0;
        *y = 1;
        return b;
    }

    int x1, y1; // To store results of recursive call
    int gcd = gcdExtended(b%a, a, &x1, &y1);

    // Update x and y using results of
    // recursive call
    *x = y1 - (b/a) * x1;
    *y = x1;

    return gcd;
}

// Driver Code
int main()
{
    int x, y, a = 35, b = 15;
    int g = gcdExtended(a, b, &x, &y);
    cout << "GCD(" << a << ", " << b
         << ") = " << g << endl;
    return 0;
}
```

Output :

```
gcd(35, 15) = 5
```

Time Complexity: O(logn), **Auxiliary Space:** O(logn)

How does Extended Algorithm Work?

As seen above, x and y are results for inputs a and b,

$a.x + b.y = gcd$ ----(1)

And x_1 and y_1 are results for inputs b%a and a

$(b\%a).x_1 + a.y_1 = gcd$

When we put $b\%a = (b - (\lfloor b/a \rfloor).a)$ in above,
we get following. Note that $\lfloor b/a \rfloor$ is floor(b/a)

$(b - (\lfloor b/a \rfloor).a).x_1 + a.y_1 = gcd$

Above equation can also be written as below

$b.x_1 + a.(y_1 - (\lfloor b/a \rfloor).x_1) = gcd$ ---(2)

After comparing coefficients of 'a' and 'b' in (1) and (2), we get following

$x = y_1 - \lfloor b/a \rfloor * x_1$

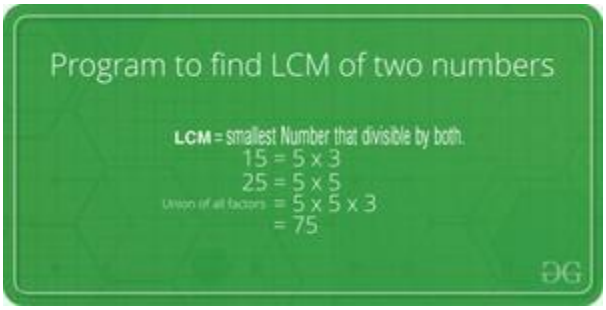
$y = x_1$

How is Extended Algorithm Useful?

The extended Euclidean algorithm is particularly useful when a and b are coprime (or gcd is 1). Since x is the modular multiplicative inverse of "a modulo b", and y is the modular multiplicative inverse of "b modulo a". In particular, the computation of the modular multiplicative inverse is an essential step in RSA public-key encryption method.

This article is contributed by **Ankur**. Please write comments if you find anything incorrect, or if you want to share more information about the topic discussed above

LCM of Two Numbers



LCM (Least Common Multiple) of two numbers is the smallest number which can be divided by both numbers.

For example, LCM of 15 and 20 is 60, and LCM of 5 and 7 is 35.

A **simple solution** is to find all prime factors of both numbers, then find union of all factors present in both numbers. Finally, return the product of elements in union.

An **efficient solution** is based on the below formula for LCM of two numbers 'a' and 'b'.

```
a x b = LCM(a, b) * GCD (a, b)

LCM(a, b) = (a x b) / GCD(a, b)
```

We have discussed function to find GCD of two numbers. Using GCD, we can find LCM.

Below is the implementation of the above idea:

C++Java

```
// C++ program to find LCM of two numbers
#include <iostream>
using namespace std;

// Recursive function to return gcd of a and b
long long gcd(long long int a, long long int b)
{
if (b == 0)
        return a;
return gcd(b, a % b);
}

// Function to return LCM of two numbers
long long lcm(int a, int b)
{
        return (a*b / gcd(a, b)) ;
}

// Driver program to test above function
int main()
```

```
{
        int a = 15, b = 20;
        cout <<"LCM of " << a << " and "
                << b << " is " << lcm(a, b);
        return 0;
}
```

Output

```
LCM of 15 and 20 is 60
```

Time Complexity: *O(log(min(a,b))*

Auxiliary Space: *O(log(min(a,b))*

Extended Euclidean Algorithm

Extended Euclidean Algorithm:

Extended Euclidean algorithm also finds integer coefficients x and y such that: ax + by = gcd(a, b)

Examples:

Input: *a = 30, b = 20*
Output: *gcd = 10, x = 1, y = -1*
*(Note that 30*1 + 20*(-1) = 10)*

Input: *a = 35, b = 15*
Output: *gcd = 5, x = 1, y = -2*
*(Note that 35*1 + 15*(-2) = 5)*

The extended Euclidean algorithm updates the results of gcd(a, b) using the results calculated by the recursive call gcd(b%a, a). Let values of x and y calculated by the recursive call be x_1 and y_1. x and y are updated using the below expressions.

$ax + by = gcd(a, b)$
$gcd(a, b) = gcd(b\%a, a)$
$gcd(b\%a, a) = (b\%a)x_1 + ay_1$

$ax + by = (b\%a)x_1 + ay_1$
$ax + by = (b - [b/a] * a)x_1 + ay_1$
$ax + by = a(y_1 - [b/a] * x_1) + bx_1$

Comparing LHS and RHS,
$x = y_1 - ⌊b/a⌋ * x_1$
$y = x_1$

Below is an implementation of the above approach:

C++Java

```cpp
// C++ program to demonstrate working of
// extended Euclidean Algorithm
#include <bits/stdc++.h>
using namespace std;

// Function for extended Euclidean Algorithm
int gcdExtended(int a, int b, int *x, int *y)
{
	// Base Case
	if (a == 0)
	{
		*x = 0;
		*y = 1;
		return b;
	}

	int x1, y1; // To store results of recursive call
	int gcd = gcdExtended(b%a, a, &x1, &y1);

	// Update x and y using results of
	// recursive call
	*x = y1 - (b/a) * x1;
	*y = x1;

	return gcd;
}

// Driver Code
int main()
{
	int x, y, a = 35, b = 15;
	int g = gcdExtended(a, b, &x, &y);
	cout << "GCD(" << a << ", " << b
		<< ") = " << g << endl;
	return 0;
}
```

Output :

```
gcd(35, 15) = 5
```

Time Complexity: O(log N)
Auxiliary Space: O(log N)

How does Extended Algorithm Work?

As seen above, x and y are results for inputs a and b,

$a.x + b.y = gcd$ —-(1)

And x_1 and y_1 are results for inputs b%a and a

$(b\%a).x_1 + a.y_1 = gcd$

When we put $b\%a = (b - (\lfloor b/a \rfloor).a)$ in above,
we get following. Note that $\lfloor b/a \rfloor$ is floor(b/a)

$(b - (\lfloor b/a \rfloor).a).x_1 + a.y_1 = gcd$

Above equation can also be written as below

$b.x_1 + a.(y_1 - (\lfloor b/a \rfloor).x_1) = gcd$ —(2)

After comparing coefficients of 'a' and 'b' in (1) and
(2), we get following,
$x = y_1 - \lfloor b/a \rfloor * x_1$
$y = x_1$

How is Extended Algorithm Useful?

The extended Euclidean algorithm is particularly useful when a and b are coprime (or gcd is 1). Since x is the modular multiplicative inverse of "a modulo b", and y is the modular multiplicative inverse of "b modulo a". In particular, the computation of the modular multiplicative inverse is an essential step in RSA public-key encryption method.

Bezout's identity (Bezout's lemma)

Let ***a*** and ***b*** be any integer and ***g*** be its greatest common divisor of ***a*** and ***b***. Then, there exists integers ***x*** and ***y*** such that ***ax + by = g ...(1)***.

ax+by=gg=gcd(a,b)

The pair ***(x, y)*** satisfying the above equation is not unique. However, all possible solutions can be calculated.

We can find ***x'*** and ***y'*** which satisfies ***(1)*** using Euclidean algorithms . All possible solutions of ***(1)*** is given by,

x=x′+bgky=y′−agkk∈\Z

where ***k*** is any integer.

It is easy to see why this holds. Just plug in the solutions to ***(1)*** to have an intuition.

Also, it is *important* to see that for general equation of the form,

ax+by=u

u=gcd(a, b) is the smallest positive integer for which ***ax+by=u*** has a solution with integral values of ***x*** and ***y***.

Statement: If gcd(a, c)=1 and gcd(b, c)=1, then gcd(ab, c)=1.

Proof:

Above can be easily proved using Bezout's Identity.

ax+cy=1 and bu+cv=1

Multiply the above two equations,

(ax+cy)(bu+cv)=1

The above implies,

ab(ux)+c(axv+buy+cyv)=1

1 is the only integer dividing L.H.S and R.H.S .

Hence, gcd(ab, c) = 1.

Linear Diophantine Equation (Check for Solution)

A Diophantine equation is a polynomial equation, usually in two or more unknowns, such that only the integral solutions are required. An Integral solution is a solution such that all the unknown variables take only integer values.

Given three integers a, b, c representing a linear equation of the form : ax + by = c.
Determine if the equation has a solution such that x and y are both integral values.

Examples:

```
Input : a = 3, b = 6, c = 9
Output: Possible
Explanation : The Equation turns out to be,
3x + 6y = 9 one integral solution would be
x = 1 , y = 1

Input : a = 3, b = 6, c = 8
Output : Not Possible
Explanation : o integral values of x and y
exists that can satisfy the equation 3x + 6y = 8

Input : a = 2, b = 5, c = 1
Output : Possible
Explanation : Various integral solutions
possible are, (-2,1) , (3,-1) etc.
```

For linear Diophantine equation equations, integral solutions exist if and only if, the GCD of coefficients of the two variables divides the constant term perfectly. In other words, the integral solution exists if, GCD(a ,b) divides c.

Below is the implementation

C++

```cpp
// C++ program for the above approach
#include <iostream>
using namespace std;
int x,y;
// Function to implement the extended euclid algorithm
int gcd_extend(int a, int b)
{
```

```
        // Base Case
        if (b == 0)
        {
                x = 1;
                y = 0;
                return a;
        }

        // Recursively find the gcd
        else
        {
                int g = gcd_extend(b, a % b);
                int x1 = x, y1 = y;
                x = y1;
                y = x1 - (a / b) * y1;
                return g;
        }
}
// Function to print the solutions of the given equations ax + by = c
void print_solution(int a, int b, int c)
{
        if (a == 0 && b == 0)
        {

                // Condition for infinite solutions
                if (c == 0)
                {
                        cout<<"Infinite Solutions "<<"Exist"<<"\n";
                }

                // Condition for no solutions exist
                else
                {
                        cout<<"No Solution exists"<<"\n";
                }
        }
        int gcd = gcd_extend(a, b);
        // Condition for no solutions exist
        if (c % gcd != 0)
        {
                cout<<"No Solution exists"<<"\n";
        }
        else
        {
                // Print the solution
                cout<<"x = "<<x*(c / gcd)<<", y = "<<y*(c / gcd)<<"\n";
        }
}
```

```
// Driver Code
int main() {
        int a, b, c;

        // Given coefficients
        a = 4;
        b = 18;
        c = 10;

        // Function Call
        print_solution(a, b, c);
        return 0;
}
```

Output

```
x = -20, y = 5
```

Linear Diophantine Equation (Find any Solution)

```
Input : a = 4, b = 6, c = 8
Output: x = -4 , y = 4

Input : a = 3, b = 6, c = 8
Output: No Solution
```

Implementation :

C++

```
#include <bits/stdc++.h>
using namespace std;


int gcd_extend(int a, int b,
               int& x, int& y)
{
    // Base Case
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }

    // Recursively find the gcd
    else {
        int g = gcd_extend(b,
```

```
                                a % b, x, y);
        int x1 = x, y1 = y;
        x = y1;
        y = x1 - (a / b) * y1;
        return g;
    }
}

// Function to print the solutions of
// the given equations ax + by = c
void print_solution(int a, int b, int c)
{
    int x, y;
    if (a == 0 && b == 0) {

        // Condition for infinite solutions
        if (c == 0) {
            cout
                << "Infinite Solutions Exist"
                << endl;
        }

        // Condition for no solutions exist
        else {
            cout
                << "No Solution exists"
                << endl;
        }
    }
    int gcd = gcd_extend(a, b, x, y);

    // Condition for no solutions exist
    if (c % gcd != 0) {
        cout
            << "No Solution exists"
            << endl;
    }
    else {

        // Print the solution
        cout << "x = " << x * (c / gcd)
             << ", y = " << y * (c / gcd)
             << endl;
    }
}

// Driver Code
int main(void)
{
```

```
    int a, b, c;
    cin>>a>>b>>c;

    // Function Call
    print_solution(a, b, c);
    return 0;
}
```

Output
```
x = 0, y = 0
```
Time Complexity :O(log(max(a,b)))

Auxillary Space : O(log(max(a,b)))

Mathematical Principles

Inclusion Exclusion Principle

Sum Rule – If a task can be done in one of n_1 ways or one of n_2 ways, where none of the set of n_1 ways is the same as any of the set of n_2 ways, then there are $n_1 + n_2$ ways to do the task. The sum-rule mentioned above states that if there are multiple sets of ways of doing a task, there shouldn't be any way that is common between two sets of ways because if there is, it would be counted twice and the enumeration would be wrong.

The principle of **inclusion-exclusion** says that in order to count only unique ways of doing a task, we must add the number of ways to do it in one way and the number of ways to do it in another and then subtract the number of ways to do the task that are common to both sets of ways.

The principle of inclusion-exclusion is also known as the **subtraction principle**. For two sets of ways $A_i$and A_2, the enumeration would like-
$| A_1 \cup A_2 | = |A_1| + | A_2| - |A_1 \cap A_2|$

Below are some examples to explain the application of inclusion-exclusion principle:

Example 1: How many binary strings of length 8 either start with a '1' bit or end with two bits '00'?

Solution: If the string starts with one, there are 7 characters left which can be filled in 2^7 = 128 ways.
If the string ends with '00' then 6 characters can be filled in 2^6 = 64 ways.

Now if we add the above sets of ways and conclude that it is the final answer, then it would be wrong. This is because there are strings with start with '1' and end with '00' both, and since they satisfy both criteria they are counted twice.
So we need to subtract such strings to get a correct count.
Strings that start with '1' and end with '00' have five characters that can be filled in 2^5=32 ways.
So by the inclusion-exclusion principle we get- Total strings = 128 + 64 – 32 = 160

Example 2: How many numbers between 1 and 1000, including both, are divisible by 3 or 4?
Solution: Number of numbers divisible by 3 =

=

. Number of numbers divisible by 4 =

=

.Number of numbers divisible by 3 and 4 =

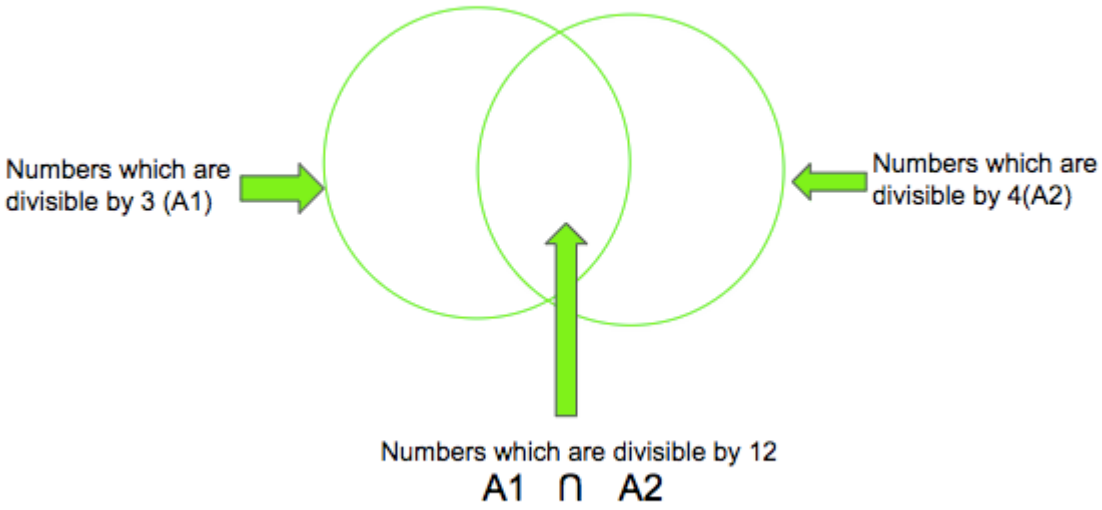
=

Therefore, number of numbers divisible by 3 or 4 =

= 333 + 250 – 83 = 500

Implementation

Problem 1: How many numbers between 1 and 1000, including both, are divisible by 3 or 4?
The Approach will be the one discussed above, we add the number of numbers that are divisible by 3 and 4 and subtract the numbers which are divisible by 12.



C++

```cpp
// CPP program to count the
// number of numbers between
// 1 and 1000, including both,
// that are divisible by 3 or 4
#include <bits/stdc++.h>
using namespace std;

// function to count the divisors
int countDivisors(int N, int a, int b)
{
        // Counts of numbers
        // divisible by a and b
        int count1 = N / a;
        int count2 = N / b;

        // inclusion-exclusion
        // principle applied
        int count3 = (N / (a * b));

        return count1 + count2 - count3;
}

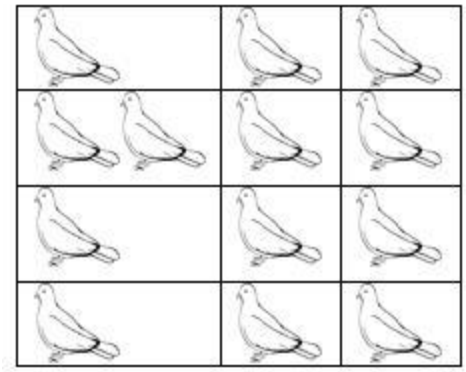
// Driver Code
int main()
{
```

```
        int N = 1000, a = 3, b = 4;
        cout << countDivisors(N, a, b);
        return 0;
}
```

Output
500

The Pigeonhole Principle

Suppose that a flock of 13 pigeons flies into a set of 12 pigeonholes to roost. Because there are 13 pigeons but only 12 pigeonholes, at least one of these 19 pigeonholes must have at least two pigeons in it. To see why this is true, note that if each pigeonhole had at most one pigeon in it, at most 12 pigeons, one per hole, could be accommodated. This illustrates a general principle called the pigeonhole principle, which states that if there are more pigeons than pigeonholes, then there must be at least one pigeonhole with at least two pigeons in it.



Theorem :

I) If "A" is the average number of pigeons per hole, where A is not an integer then

- At least one pigeon hole contains **ceil[A]** (smallest integer greater than or equal to A) pigeons
- Remaining pigeon holes contains at most **floor[A]** (largest integer less than or equal to A) pigeons

Or

II) We can say as, if n + 1 objects are put into n boxes, then at least one box contains two or more objects.

The abstract formulation of the principle: Let X and Y be finite sets and let be a function.

- If X has more elements than Y, then f is not one-to-one.
- If X and Y have the same number of elements and f is onto, then f is one-to-one.
- If X and Y have the same number of elements and f is one-to-one, then f is onto.

Pigeonhole principle is one of the simplest but most useful ideas in mathematics. We will see more applications that proof of this theorem.

Example 1: If (Kn+1) pigeons are kept in n pigeon holes where K is a positive integer, what is the average no. of pigeons per pigeon hole?

Solution: average number of pigeons per hole = (Kn+1)/n
= K + 1/n
Therefore there will be at least one pigeonhole which will contain at least (K+1) pigeons i.e., ceil[K +1/n] and remaining will contain at most K i.e., floor[k+1/n] pigeons.
i.e., the minimum number of pigeons required to ensure that at least one pigeon hole contains (K+1) pigeons is (Kn+1).

Example 2: A bag contains 10 red marbles, 10 white marbles, and 10 blue marbles. What is the minimum no. of marbles you have to choose randomly from the bag to ensure that we get 4 marbles of same color?

Solution: Apply pigeonhole principle.
No. of colors (pigeonholes) n = 3
No. of marbles (pigeons) K+1 = 4
Therefore the minimum no. of marbles required = Kn+1
By simplifying we get Kn+1 = 10.
Verification: ceil[Average] is [Kn+1/n] = 4
[Kn+1/3] = 4
Kn+1 = 10
i.e., 3 red + 3 white + 3 blue + 1(red or white or blue) = 10

Pigeonhole principle strong form :

Theorem: Let $q_1, q_2, \ldots, q_n$ be positive integers.
If $q_1 + q_2 + \ldots + q_n - n + 1$ objects are put into n boxes, then either the 1st box contains at least q_1 objects, or the 2nd box contains at least q_2 objects, . . ., the nth box contains at least q_n objects.

Application of this theorem is more important, so let us see how we apply this theorem in problem solving.

Example 1: In a computer science department, a student club can be formed with either 10 members from first year or 8 members from second year or 6 from third year or 4 from final year. What is the minimum no. of students we have to choose randomly from department to ensure that a student club is formed?

Solution: we can directly apply from the above formula where,
$q_1 = 10$, $q_2 = 8$, $q_3 = 6$, $q_4 = 4$ and $n = 4$
Therefore the minimum number of students required to ensure department club to be formed is
$10 + 8 + 6 + 4 - 4 + 1 = 25$

Example 2: A box contains 6 red, 8 green, 10 blue, 12 yellow and 15 white balls. What is the minimum no. of balls we have to choose randomly from the box to ensure that we get 9 balls of same color?

Solution: Here in this we cannot blindly apply pigeon principle. First we will see what happens if we apply above formula directly.
From the above formula we have get answer 47 because $6 + 8 + 10 + 12 + 15 - 5 + 1 = 47$
But it is not correct. In order to get the correct answer we need to include only blue, yellow and white balls because red and green balls are less than 9. But we are picking randomly so we include after we apply pigeon principle.
i.e., 9 blue + 9 yellow + 9 white – 3 + 1 = 25
Since we are picking randomly so we can get all the red and green balls before the above 25 balls. Therefore we add 6 red + 8 green + 25 = 39
We can conclude that in order to pick 9 balls of same color randomly, one has to pick 39 balls from a box.

Derangements

A Derangement is a permutation of n elements, such that no element appears in its original position. For example, a derangement of {0, 1, 2, 3} is {2, 3, 1, 0}.
Given a number n, find the total number of Derangements of a set of n elements.

Examples :

```
Input: n = 2
Output: 1
For two elements say {0, 1}, there is only one
possible derangement {1, 0}

Input: n = 3
Output: 2
For three elements say {0, 1, 2}, there are two
possible derangements {2, 0, 1} and {1, 2, 0}

Input: n = 4
Output: 9
For four elements say {0, 1, 2, 3}, there are 9
possible derangements {1, 0, 3, 2} {1, 2, 3, 0}
{1, 3, 0, 2}, {2, 3, 0, 1}, {2, 0, 3, 1}, {2, 3,
1, 0}, {3, 0, 1, 2}, {3, 2, 0, 1} and {3, 2, 1, 0}
```

Counting Derangements

A Derangement is a permutation of n elements, such that no element appears in its original position. For example, a derangement of {0, 1, 2, 3} is {2, 3, 1, 0}.
Given a number n, find the total number of Derangements of a set of n elements.

Examples :

```
Input: n = 2
Output: 1
For two elements say {0, 1}, there is only one
possible derangement {1, 0}

Input: n = 3
Output: 2
For three elements say {0, 1, 2}, there are two
possible derangements {2, 0, 1} and {1, 2, 0}

Input: n = 4
Output: 9
```

```
For four elements say {0, 1, 2, 3}, there are 9
possible derangements {1, 0, 3, 2} {1, 2, 3, 0}
{1, 3, 0, 2}, {2, 3, 0, 1}, {2, 0, 3, 1}, {2, 3,
1, 0}, {3, 0, 1, 2}, {3, 2, 0, 1} and {3, 2, 1, 0}
```

Let countDer(n) be count of derangements for n elements. Below is the recursive relation to it.

```
countDer(n) = (n - 1) * [countDer(n - 1) + countDer(n - 2)]
```

How does above recursive relation work?

There are n – 1 way for element 0 (this explains multiplication with n – 1).
Let ***0 be placed at index i***. There are now two possibilities, depending on whether or not element i is placed at 0 in return.

1. ***i is placed at 0:*** This case is equivalent to solving the problem for n-2 elements as two elements have just swapped their positions.
2. ***i is not placed at 0:*** This case is equivalent to solving the problem for n-1 elements as now there are n-1 elements, n-1 positions and every element has n-2 choices

Below is the simple solution based on the above recursive formula:

C++

```cpp
// A Naive Recursive C++ program
// to count derangements
#include <bits/stdc++.h>
using namespace std;

int countDer(int n)
{
// Base cases
if (n == 1) return 0;
if (n == 2) return 1;

// countDer(n) = (n-1)[countDer(n-1) + der(n-2)]
return (n - 1) * (countDer(n - 1) + countDer(n - 2));
}

// Driver Code
int main()
{
        int n = 4;
        cout << "Count of Derangements is "
                << countDer(n);
        return 0;
}
```

Output

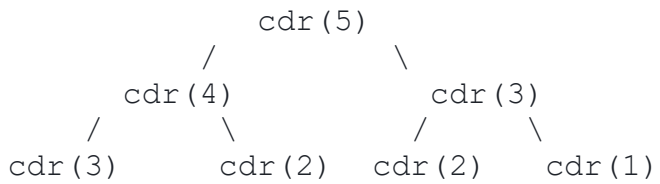
```
Count of Derangements is 9
```

ime Complexity: O(2^n) since **T(n) = T(n-1) + T(n-2)** which is exponential.

Auxiliary Space: O(h) where **h= log n** is the maximum height of the tree.

We can observe that this implementation does repetitive work. For example, see recursion tree for countDer(5), countDer(3) is being evaluated twice.

```
cdr() ==> countDer()
```



An **Efficient Solution** is to use Dynamic Programming

C++

```cpp
// C++ implementation of the above
// approach

#include <iostream>
using namespace std;

int countDer(int n)
{

	// base case
	if (n == 1 or n == 2) {
		return n - 1;
	}

	// Variable for just storing
	// previous values
	int a = 0;
	int b = 1;

	// using above recursive formula
	for (int i = 3; i <= n; ++i) {
		int cur = (i - 1) * (a + b);
		a = b;
		b = cur;
	}

	// Return result for n
	return b;
}
```

```
// Driver Code
int main()
{

        cout << "Count of Derangements is " << countDer(4);
        return 0;
}

// Code contributed by skagnihotri
```

Output

```
Count of Derangements is 9
```

Time Complexity: *O(n)*

Auxiliary Space: *O(1)*

Subarray Sum Divisible By Size

Given an array **arr[]** of length **N**. The task is to check if there exists any subarray whose sum is a multiple of **N**. If there exists such subarray, then print the starting and ending index of that subarray else print **-1**. If there are multiple such subarrays, print any of them.

Examples:

Input: *arr[] = {7, 5, 3, 7}*
Output: *0 1*
Sub-array from index 0 to 1 is [7, 5]
sum of this subarray is 12 which is a multiple of 4

Recursive method :

C++

```
// C++ implementation to print subarray sum divisible by size
#include <iostream>
using namespace std;
// function to print the starting and ending indices of subarray whose sum
is divisible by n
void printSubArray(int arr[],int n)
{
    for(int i=0;i<n;i++)
    {
        int sum=0;
```

```
        for(int j=i;j<n;j++)
        {
            sum+=arr[j];
            if(sum%n==0)
            {
             cout<<i<<" "<<j<<endl;
             return;
            }
        }
    }
}
// Driver function
int main() {
        int arr[4]={10,5,7,4};
        int n=4;
        printSubArray(arr,n);
        return 0;
}
```

Output

```
1 2
```

Efficient Solution :

Dynamic Programming :

C++

```
// C++ implementation to print subarray sum divisible by size
#include<bits/stdc++.h>
using namespace std;
// function to print the starting and ending indices of subarray whose sum
is divisible by n
void printSubArray(int arr[],int n)
{
    vector<int>preSum(n),rem(n,-1);
    preSum[0]=arr[0];
    if(preSum[0]%n==0)
    {
        cout<<0<<" "<<0;
        return;
    }
    rem[preSum[0]%n]=0;
    for(int i=1;i<n;i++)
    {
        preSum[i]=preSum[i-1]+arr[i];
        int r=preSum[i]%n;
        if(r==0)
        {
           cout<<0<<" "<<i;
```

```
            return;
        }
        if(rem[r]!=-1)
        {
            cout<<(rem[r]+1)<<" "<<i;
            return;
        }
        rem[r]=i;
    }
}
// Driver function
int main() {
        int arr[4]={2,7,5,3};
        int n=4;
        printSubArray(arr,n);
        return 0;
}
```

Output

```
1 2
```

Number Theoretic Functions

Euler's Totient Function

Euler's Totient function Φ (n) for an input n is the count of numbers in {1, 2, 3, ..., n-1} that are relatively prime to n, i.e., the numbers whose GCD (Greatest Common Divisor) with n is 1.

Examples :

```
Φ(1) = 1
gcd(1, 1) is 1

Φ(2) = 1
gcd(1, 2) is 1, but gcd(2, 2) is 2.

Φ(3) = 2
gcd(1, 3) is 1 and gcd(2, 3) is 1

Φ(4) = 2
gcd(1, 4) is 1 and gcd(3, 4) is 1
```

```
Φ(5) = 4
gcd(1, 5) is 1, gcd(2, 5) is 1,
gcd(3, 5) is 1 and gcd(4, 5) is 1

Φ(6) = 2
gcd(1, 6) is 1 and gcd(5, 6) is 1,
```

How to compute Φ(n) for an input nΦ

A **simple solution** is to iterate through all numbers from 1 to n-1 and count numbers with gcd with n as 1. Below is the implementation of the simple method to compute Euler's Totient function for an input integer n.

C++

```cpp
// A simple C++ program to calculate
// Euler's Totient Function
#include <iostream>
using namespace std;

// Function to return gcd of a and b
int gcd(int a, int b)
{
	if (a == 0)
		return b;
	return gcd(b % a, a);
}

// A simple method to evaluate Euler Totient Function
int phi(unsigned int n)
{
	unsigned int result = 1;
	for (int i = 2; i < n; i++)
		if (gcd(i, n) == 1)
			result++;
	return result;
}

// Driver program to test above function
int main()
{
	int n;
	for (n = 1; n <= 10; n++)
		cout << "phi("<<n<<") = " << phi(n) << endl;
	return 0;
}
```

Output

```
phi(1) = 1
phi(2) = 1
```

```
phi(3) = 2
phi(4) = 2
phi(5) = 4
phi(6) = 2
phi(7) = 6
phi(8) = 4
phi(9) = 6
phi(10) = 4
```

Euler Totient (Efficient Implement)

The idea is to count all prime factors and their multiples and subtract this count from n to get the totient function value (Prime factors and multiples of prime factors won't have gcd as 1)

```
1) Initialize result as n
2) Consider every number 'p' (where 'p' varies from 2 to Φn).
   If p divides n, then do following
   a) Subtract all multiples of p from 1 to n [all multiples of
p
      will have gcd more than 1 (at least p) with n]
   b) Update n by repeatedly dividing it by p.
3) If the reduced n is more than 1, then remove all multiples
   of n from result.
```

Below is the implementation of the above algorithm.

C++

```cpp
// C++ program to calculate Euler's
// Totient Function
#include <bits/stdc++.h>
using namespace std;

int phi(int n)
{
        // Initialize result as n
        int result = n;

        // Consider all prime factors of n
        // and subtract their multiples
        // from result
        for(int p = 2; p * p <= n; ++p)
        {

                // Check if p is a prime factor.
```

```
                if (n % p == 0)
                {

                        // If yes, then update n and result
                        while (n % p == 0)
                                n /= p;

                        result -= result / p;
                }
        }

        // If n has a prime factor greater than sqrt(n)
        // (There can be at-most one such prime factor)
        if (n > 1)
                result -= result / n;

        return result;
}

// Driver code
int main()
{
        int n;
        for(n = 1; n <= 10; n++)
        {
                cout << "Phi" << "("
                        << n << ")" << " = "
                        << phi(n) << endl;
        }
        return 0;
}

// This code is contributed by koulick_sadhu
```

Output

```
Phi(1) = 1
Phi(2) = 1
Phi(3) = 2
Phi(4) = 2
Phi(5) = 4
Phi(6) = 2
Phi(7) = 6
Phi(8) = 4
Phi(9) = 6
Phi(10) = 4
```

Time Complexity: $O(\sqrt{n})$

Auxiliary Space: $O(1)$

Let us take an example to understand the above algorithm.

```
n = 10.
Initialize: result = 10

2 is a prime factor, so n = n/i = 5, result = 5
3 is not a prime factor.

The for loop stops after 3 as 4*4 is not less than or equal
to 10.

After for loop, result = 5, n = 5
Since n > 1, result = result - result/n = 4
```

Properties of Euler Totient Function

1) For a **prime number p**,

2) For **two prime numbers a and b** ,

3)For **a prime number p**,

4) For **two number a and b**

5)Sum of values of totient functions of all divisors of n is equal to n.

$$\sum_{d|n} \varphi(d) = n,$$

Euler Totient for 1 To n

Euler's totient function, also known as Euler's phi function, is a mathematical function that counts the number of positive integers less than or equal to n that are relatively prime to n. In other words, it calculates the number of positive integers less than or equal to n that do

not share any common factors with n other than 1. The Euler totient function is denoted by the symbol φ(n).

```
I/P : n=4
O/P : 1 1 2 2

I/P : n=10
O/P: 1 1 2 2 4 2 6 4 6 4
```

Calculating the Euler totient function for all positive integers from 1 to n can be done using a simple algorithm that checks each number less than or equal to n for relative primality with n. However, this algorithm is inefficient for large values of n.

Efficient Soln.

C++

```
#include<iostream>
#include<bits/stdc++.h>
using namespace std;

class GFG {

// Computes and prints totient of all numbers
// smaller than or equal to n.
public :
void computeTotient(int n) {

        // Create and initialize an array to store
        // phi or totient values
        long int phi[n+1];
        for (int i = 1; i <= n; i++)
        phi[i] = i; // indicates not evaluated yet
                                // and initializes for product
                                // formula.

        // Compute other Phi values
        for (int p = 2; p <= n; p++) {

        // If phi[p] is not computed already,
        // then number p is prime
        if (phi[p] == p) {

                // Phi of a prime number p is
                // always equal to p-1.
                phi[p] = p - 1;

                // Update phi values of all
                // multiples of p
```

```
                for (int i = 2 * p; i <= n; i += p) {

                // Add contribution of p to its
                // multiple i by multiplying with
                // (1 - 1/p)
                phi[i] = (phi[i] / p) * (p - 1);
                }
        }
        }

        // Print precomputed phi values
        for (int i = 1; i <= n; i++)
        cout<<"Totient of "<<" "<<i<<" " <<"is"<<" "<<phi[i]<<endl;
                                                ;
}
};

// Driver code
 int main() {
    class GFG obj;
        int n = 12;
   obj.computeTotient(n);
}
```

Output

```
Totient of  1 is 1
Totient of  2 is 1
Totient of  3 is 2
Totient of  4 is 2
Totient of  5 is 4
Totient of  6 is 2
Totient of  7 is 6
Totient of  8 is 4
Totient of  9 is 6
Totient of  10 is 4
Totient of  11 is 10
Totient of  12 is 4
```

Time Complexity - O(n log log n)

Aux. Space - $\Theta(n)$

Binomial Coefficients

Permutation

Permutation is a fundamental concept in combinatorics that deals with the arrangement of items in a specific order. It is used to calculate the number of possible ways that a set of items can be arranged or distributed. In this article, we will explore four types of permutations: distribution of n distinct items to n people, distribution of n distinct items to m people where m<=n and order matters, permutation with repetition, and arrangements of n items with some items repeated.

1. **Distribution of n distinct items to n people:** In this type of permutation, n distinct items are to be distributed among n people, where each person gets exactly one item. The order in which the items are distributed is essential. To calculate the number of permutations, we use the formula n!. For example, suppose we have three distinct items A, B, and C that need to be distributed among three people X, Y, and Z. The possible permutations are:

AXBXC, AXCXB, BXAXC, BXCAX, CXAXB, and CXBAX.

Therefore, there are 6 possible permutations.

2. **Distribution of n distinct items to m people where m<=n and order matters:** In this type of permutation, n distinct items are to be distributed among m people, where m is less than or equal to n, and the order in which the items are distributed is essential. To calculate the number of permutations, we use the formula nPm = n!/(n-m)!. For example, suppose we have four distinct items A, B, C, and D that need to be distributed among two people X and Y. The possible permutations are:

AXBY, AYBX, BAXY, BAYX, XABY, XAYB, YABX, YAXB, XBAY, and YBAX.

Therefore, there are 10 possible permutations.

3. **Permutation with repetition:** In this type of permutation, we have n items, some of which may be identical. The order in which the items are arranged is essential. To calculate the number of permutations, we use the formula n!/r1!r2!...rn!, where r1, r2,...,rn are the frequencies of each distinct item. For example, suppose we have the word 'MISISSIPPI,' and we want to find the number of permutations of the letters. There are 11 letters in total, with four 'I's, four 'S's, and two 'P's. The number of permutations is:

11!/(4!4!2!) = 34650.

Therefore, there are 34650 possible permutations.

4. **Arrangements of n items with some items repeated:** In this type of permutation, we have n items, some of which may be identical. However, the order in which the items are arranged is not essential. To calculate the number of permutations, we use the formula n!/r1!r2!...rn! where r1, r2,...,rn are the frequencies of each distinct item. For example, suppose we have the word 'MISSISSIPPI,' and we want to find the number of arrangements of the letters. There are 11 letters in total, with four 'I's, four 'S's, and two 'P's. The number of arrangements is:

11!/(4!4!2!) = 34650/4!4!2! = 34650/576 = 60.

Therefore, there are 60 possible arrangements.

In conclusion, permutations are a fundamental concept in combinatorics that are used to calculate the number of possible ways that a set of items can be arranged or distributed.

Printing all Permutations

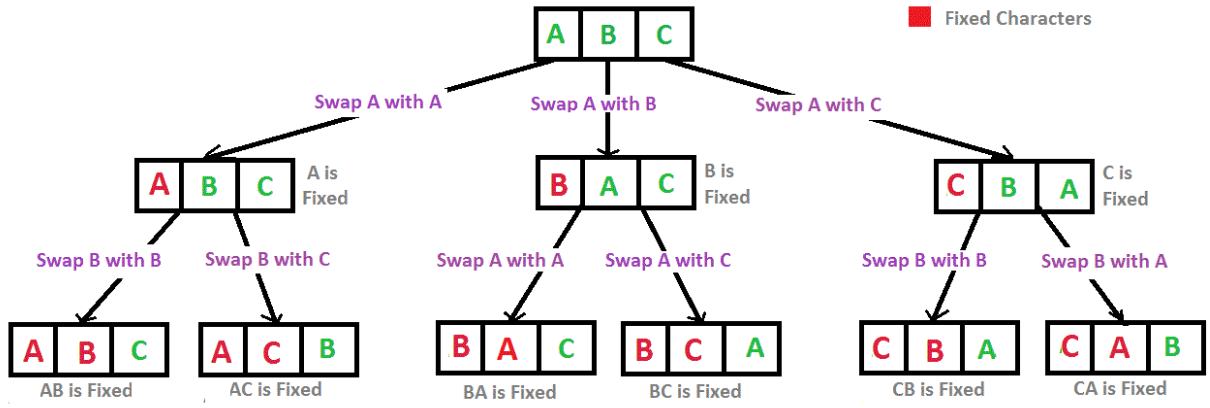
Given a string, print all permutations of it.

Input : str = "ABC"
Output : ABC ACB BAC BCA CAB CBA

Idea: We iterate from first to last index. For every index i, we swap the i-th character with the first index. This is how we fix characters at the current first index, then we recursively generate all permutations beginning with fixed characters (by parent recursive calls). After we have recursively generated all permutations with the first character fixed, then we move the first character back to its original position so that we can get the original string back and fix the next character at first position.

Illustration:
We swap 'A' with 'A'. Then we recursively generate all permutations beginning with A.

While returning from the recursive calls, we revert the changes made by them using the same swap again. So we get the original string "ABC".
Then we swap 'A' with 'B' and generate all permutations beginning with 'B'.
Similarly, we generate all permutations beginning with 'C'



Recursion Tree for Permutations of String "ABC"

C++Java

```cpp
// C++ program to print all
// permutations with duplicates allowed
#include <bits/stdc++.h>
using namespace std;

/* Function to print permutations of string
This function takes three parameters:
1. String
2. Starting index of the string
3. Ending index of the string. */
void permute(string &str, int l, int r)
{
    if (l == r)
    {

        cout << str << " ";
        return ;
    }
    else
    {
        for (int i = l; i <= r; i++)
        {
            swap(str[l], str[i]);
```

```
            permute(str, l+1, r);
            swap(str[l], str[i]);
        }
    }
}

/* Driver program to test above functions */
int main()
{
    string str = "ABC";
    permute(str, 0, str.length()-1);
    return 0;
}
```

Output
```
ABC ACB BAC BCA CBA CAB
```

Time Complexity: O(N * N!), where N is the length of the string. Note that there are N! permutations and it requires O(N) time to print a permutation.
Auxiliary Space: O(N), , where N is the length of the string.

Combination

Combinatorics is a fascinating branch of mathematics that deals with counting and arranging objects, particularly when order and repetition are important. One common problem in combinatorics is determining the number of ways to select a certain number of distinct items from a larger set of distinct items, without replacement. This problem is often encountered in various fields such as statistics, probability theory, computer science, and engineering. In this article, we will explore the concept of combinatorics and how it can be applied to solve the problem of selecting r distinct items from n distinct items.

The problem of selecting r distinct items from n distinct items can be represented mathematically using the notation "nCr", which stands for the number of ways to choose r items from n items without replacement. This is also known as a combination, where order does not matter. The formula to calculate nCr is given by:

```
nCr = n! / (r! * (n - r)!)
```

where "!" denotes the factorial function. The factorial of a positive integer n, denoted by n!, is the product of all positive integers from 1 to n. For example, 5! = 5 * 4 * 3 * 2 * 1 = 120.

Let's take an example to illustrate the concept of combinations. Suppose we have a set of 5 distinct items, labeled A, B, C, D, and E, and we want to select 3 items from this set without replacement. We can use the formula nCr = n! / (r! * (n - r)!) to calculate the number of ways to do this:

n = 5 (since we have 5 distinct items) r = 3 (since we want to select 3 items) nCr = 5! / (3! * (5 - 3)!) = 5! / (3! * 2!) = (5 * 4 * 3 * 2 * 1) / ((3 * 2 * 1) * (2 * 1)) = 10

So, there are 10 different ways to select 3 distinct items from a set of 5 distinct items without replacement.

Combinations have several interesting properties that make them useful in various applications.

nCr (Simple Method)

Following are the common definitions of Binomial Coefficients.

- A binomial coefficient C(n, k) can be defined as the coefficient of $\mathbf{X}^k$ in the expansion of $(\mathbf{1 + X})^n$.
- A binomial coefficient C(n, k) also gives the number of ways, disregarding order, that **k** objects can be chosen from among **n** objects; more formally, the number of k-element subsets (or k-combinations) of an n-element set.

Given two numbers **N** and **r**, The task is to find the value of ${}^{N}\mathbf{C}_{r}$

Examples :

Input: N = 5, r = 2
Output: 10
Explanation: The value of ${}^{5}C_{2}$ is 10

Input: N = 3, r = 1
Output: 3

Approach: Below is the idea to solve the problem:

The total number of ways for selecting **r** elements out of **n** options are ${}^{n}C_{r}$ **= (n!) / (r! * (n-r)!)**
where **n! = 1 * 2 * . . . * n**.

Below is the Implementation of the above approach:

C++Java

```cpp
// CPP program To calculate The Value Of nCr
#include <bits/stdc++.h>
using namespace std;

int fact(int n);

int nCr(int n, int r)
{
    return fact(n) / (fact(r) * fact(n - r));
}

// Returns factorial of n
int fact(int n)
{
      if(n==0)
      return 1;
    int res = 1;
    for (int i = 2; i <= n; i++)
        res = res * i;
    return res;
}

// Driver code
int main()
{
    int n = 5, r = 3;
    cout << nCr(n, r);
    return 0;
}
```

Output
10

Time Complexity: O(N)
Auxiliary Space: O(1)
Complexity Analysis:

The time complexity of the above approach is O(N).
This is because the function fact() has a time complexity of O(N), and it is called twice for each call to nCr().

The space complexity of the above approach is O(1).
Because the function does not make any recursive calls and only uses a constant amount of memory.

Another Approach:

Using loops for a lesser number of computations.

C++Java

```cpp
// CPP program To calculate The Value Of nCr
#include <bits/stdc++.h>
using namespace std;


int nCr(int n, int r)
{
    int res = 1;
        for (int i = n; i >= (n-r+1); i--)
                res = res * i;
        for (int i = 2; i <= r; i++)
         res = res/i;
        return res;
}

// Driver code
int main()
{
    int n = 5, r = 3;
    cout << nCr(n, r);
    return 0;
}
```

10

Time Complexity: O(N)
Auxiliary Space: O(1)

nCr(Recursive Addition Formula)

Problems with previous solution :

1)Causes overflow even when answer can fit in **long long in C++** or **long in java.**

2)When we need result mod m, the previous method works only when r! is co-prime to m.

One very interesting way to find the C(n,r) is the recursive method which is based on the recursive equation.

`C(n,r) = C(n-1,r-1) + C(n-1,r)`

Below is the implementation of the above approach using DP as follows:

C++

```cpp
// C++ program to calculate the value Of nCr
#include<bits/stdc++.h>
using namespace std;
int dp[10005][10005];
int nCr(int n,int r)
{

    for(int i=0;i<=n;i++)
    {
        dp[i][0]=1;
        for(int j=1;j<=min(i,r);j++)
        {
            if(i==j)
            {
                dp[i][j]=1;
            }
            else
            {
                dp[i][j]=dp[i-1][j-1] + dp[i-1][j];
            }
        }
    }
    return dp[n][r];
}
// Driver code
int main() {
        int n = 5, r = 3;
        cout<<nCr(n, r)<<endl;
        return 0;
}
```

Output

```
10
```

Time Complexity - Θ(nr)

Auxillary Space - Θ(nr)

Space Optimized Solution :

C++

```cpp
// C++ program to calculate the Value Of nCr (space optimized)
#include <iostream>
using namespace std;
int dp[100005];
int nCr(int n,int r)
{
    dp[0]=1;
    for(int i=1;i<=n;i++)
    {
        for(int j=min(i,r);j>0;j--)
        {
            dp[j]=dp[j]+dp[j-1];
        }
    }
    return dp[r];
}
// Driver code
int main() {
        int n = 5, r = 3;
        cout<<nCr(n, r)<<endl;
        return 0;
}
```

Output

```
10
```

Time Complexity - Θ(nr)

Auxiliary Space: Θ(r)

nCr Mod p for Large Prime

Input: 4 $p=10^9+7$, $1 \le n \le 10^5$, $0 \le r \le 10^5$

5 2

10 3

4 4

7 6

Output: 10

120

1
7

We know the formula for ^nC_r

^nC_r = fact(n) / (fact(r) x fact(n-r))
Here fact() means factorial.

^nC_r % p = (fac[n]* modIverse(fac[r]) % p *
modIverse(fac[n-r]) % p) % p;
Here modIverse() means modular inverse under modulo p.

Implementation :

C++

```
// C++ program for nCr Mod  p forLarge Prime
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

#define ll long long
const int maxn=100001,p=1000000007;
ll fact[maxn],invf[maxn];


class GFG{
public:
    //Iterative Function to calculate (x^y)%p in O(log y)
    long long p;
    unsigned long long power(unsigned long long x, int y, int p){
        unsigned long long res = 1; // Initialize result

        x = x % p; // Update x if it is more than or
        // equal to p

        while (y > 0)
        {

            // If y is odd, multiply x with result
            if (y & 1)
                res = (res * x) % p;

            // y must be even now
            y = y >> 1; // y = y/2
            x = (x * x) % p;
        }
        return res;
    }
```

```
    // Returns n^(-1) mod p
    unsigned long long modInv(unsigned long long n, int p){
        return power(n, p - 2, p);
    }
    void premcompute(){
        fact[0]=1;
        fact[1]=1;
        invf[0]=1;
        invf[0]=1;

        for(int i = 2; i < maxn; i++)
        {
            fact[i] = (fact[i-1] * i) % p;
            invf[i] = modInv(fact[i] , p);
        }
    }
    ll query(int n, int r){
        return ((fact[n]*invf[r] % p) * invf[n-r]) % p;
    }
};


// Driver program
int main()
{
        // p must be a prime greater than n.
        int n = 10, r = 2, p = 13;
        class GFG obj;
        obj.p = p;
        obj.premcompute();
        cout << "Value of nCr % p is " << obj.query(n, r);
        return 0;
}
```

Output

```
Value of nCr % p is 6
```

Catalan Number

Catalan Number

Catalan numbers are defined as a mathematical sequence that consists of positive integers, which can be used to find the number of possibilities of various combinations.

The **nth** term in the sequence denoted $\mathbf{C_n}$, is found in the following formula:

The first few Catalan numbers for n = 0, 1, 2, 3, … are : 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, …

Catalan numbers occur in many interesting counting problems like the following.

1. Count the number of expressions containing n pairs of parentheses that are correctly matched. For n = 3, possible expressions are **((()))**, **()(())**, **()()()**, **(())()**, **(()())**.
2. Count the number of possible Binary Search Trees with **n** keys
3. Count the number of full binary trees (A rooted binary tree is full if every vertex has either two children or no children) with **n+1** leaves.
4. Given a number **n**, return the number of ways you can draw n chords in a circle with **2 x n** points such that no **2** chords intersect.

Catalan Number Implementation (Part 1)

```
Input: n = 6
Output: 132
Input: n = 8
Output: 1430
```

Recursive Solution for Catalan number:

Catalan numbers satisfy the following recursive formula:

Follow the steps below to implement the above recursive formula

- Base condition for the recursive approach, when **n <= 1**, return **1**
- Iterate from i = 0 to i < n
    - Make a recursive call **catalan(i)** and **catalan(n – i – 1)** and keep adding the product of both into **res**.
- Return the **res**.

Following is the implementation of the above recursive formula.

C++

```
#include <iostream>
using namespace std;
```

```
// A recursive function to find nth catalan number
unsigned long int catalan(unsigned int n)
{
	// Base case
	if (n <= 1)
		return 1;

	// catalan(n) is sum of
	// catalan(i)*catalan(n-i-1)
	unsigned long int res = 0;
	for (int i = 0; i < n; i++)
		res += catalan(i) * catalan(n - i - 1);

	return res;
}

// Driver code
int main()
{
	for (int i = 0; i < 10; i++)
		cout << catalan(i) << " ";
	return 0;
}
```

Output

```
1 1 2 5 14 42 132 429 1430 4862
```

Dynamic Programming Solution for Catalan number:

We can observe that the above recursive implementation does a lot of repeated work. Since there are overlapping subproblems, we can use dynamic programming for this.

Below is the implementation of the above idea:

- Create an array **catalan[]** for storing **ith** Catalan number.
- Initialize, **catalan[0] and catalan[1] = 1**
- Loop through **i = 2** to the given Catalan number **n**.
  - Loop through **j = 0** to **j < i** and Keep adding value of **catalan[j] * catalan[i – j – 1]** into **catalan[i]**.
- Finally, return **catalan[n]**

Follow the steps below to implement the above approach :

C++

```
#include <iostream>
using namespace std;
```

```
// A dynamic programming based function to find nth
// Catalan number
unsigned long int catalanDP(unsigned int n)
{
        // Table to store results of subproblems
        unsigned long int catalan[n + 1];

        // Initialize first two values in table
        catalan[0] = catalan[1] = 1;

        // Fill entries in catalan[] using recursive formula
        for (int i = 2; i <= n; i++) {
                catalan[i] = 0;
                for (int j = 0; j < i; j++)
                        catalan[i] += catalan[j] * catalan[i - j - 1];
        }

        // Return last entry
        return catalan[n];
}

// Driver code
int main()
{
        for (int i = 0; i < 10; i++)
                cout << catalanDP(i) << " ";
        return 0;
}
```

Output

```
1 1 2 5 14 42 132 429 1430 4862
```

Time Complexity: $O(n^2)$

Auxiliary Space: $O(n)$

Catalan Number Implementation (Part 2)

Catalan numbers are defined as a mathematical sequence that consists of positive integers, which can be used to find the number of possibilities of various combinations.

The **nth** term in the sequence denoted $\mathbf{C_n}$, is found in the following formula:

The first few Catalan numbers for n = 0, 1, 2, 3, … are : 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, … **Catalan numbers** are defined as a mathematical sequence that consists of positive integers, which can be used to find the number of possibilities of various combinations.

The **nth** term in the sequence denoted $\mathbf{C_n}$, is found in the following formula:

The first few Catalan numbers for n = 0, 1, 2, 3, … are : 1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, …

Dynamic Programming Solution for Catalan number:

We can observe that the above recursive implementation does a lot of repeated work. Since there are overlapping subproblems, we can use dynamic programming for this.
Below is the implementation of the above idea:

- Create an array **catalan[]** for storing **ith** Catalan number.
- Initialize, **catalan[0] and catalan[1] = 1**
- Loop through **i = 2** to the given Catalan number **n**.
  - Loop through **j = 0** to **j < i** and Keep adding value of **catalan[j] * catalan[i – j – 1]** into **catalan[i]**.
- Finally, return **catalan[n]**

Follow the steps below to implement the above approach:

C++Java

```
#include <iostream>
using namespace std;

// A dynamic programming based function to find nth
// Catalan number
unsigned long int catalanDP(unsigned int n)
{
        // Table to store results of subproblems
        unsigned long int catalan[n + 1];

        // Initialize first two values in table
        catalan[0] = catalan[1] = 1;

        // Fill entries in catalan[] using recursive formula
        for (int i = 2; i <= n; i++) {
                catalan[i] = 0;
                for (int j = 0; j < i; j++)
```

```
                        catalan[i] += catalan[j] * catalan[i - j - 1];
        }

        // Return last entry
        return catalan[n];
}

// Driver code
int main()
{
        for (int i = 0; i < 10; i++)
                cout << catalanDP(i) << " ";
        return 0;
}
```

Output

```
1 1 2 5 14 42 132 429 1430 4862
```

Time Complexity: O(n²)
Auxiliary Space: O(n)

Binomial Coefficient Solution for Catalan number:

*We can also use the below formula to find **nth** Catalan number in **O(n)** time.*

Here is the proof for the Expression:–

$$C_n = \frac{2n!}{(n+1)! * n!} \text{ or } C_n = \frac{1}{(n+1)}\binom{2n}{n}$$

This is the expression for which we are going to see the proof

In the pascal triangle,

```
        1
    1       1
```

```
          1      2      1
       1     3      3     1
    1     4      6     4      1
 1     5      10    10     5      1
```

$$
\begin{array}{ccccccccccc}
 & & & & \binom{0}{0} & & & & & & \\
 & & & & \binom{1}{0} & & \binom{1}{1} & & & & \\
 & & & \binom{2}{0} & & \binom{2}{1} & & \binom{2}{2} & & & \\
 & & \binom{3}{0} & & \binom{3}{1} & & \binom{3}{2} & & \binom{3}{3} & & \\
 & \binom{4}{0} & & \binom{4}{1} & & \binom{4}{2} & & \binom{4}{3} & & \binom{4}{4} & \\
\binom{5}{0} & & \binom{5}{1} & & \binom{5}{2} & & \binom{5}{3} & & \binom{5}{4} & & \binom{5}{5}
\end{array}
$$

Pascal's triangle as binomial coefficients

$$\mathbf{C}(n,k) = \mathbf{C}_k^n = {}_nC_k = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

the formula for a cell of Pascal's triangle

K→ N↓	0	1	2	3	4	5	6	7	8	9	10
0	1										
1	1	1									
2	1	2	1								
3	1	3	**3**	**1**							
4	1	4	6	**4**	**1**						
5	1	5	10	**10**	**5**	1					
6	1	6	15	20	**15**	**6**	1				
7	1	7	21	35	**35**	**21**	7	1			
8	1	8	28	56	70	**56**	**28**	8	1		
9	1	9	36	84	126	**126**	**84**	36	9	1	
10	1	10	45	120	210	252	**210**	**120**	45	10	1

Pascal's triangle

Nth term in Catalan:-
0th = 1
1st = 1
2nd = 2
3rd = 5 => ((3 + 4) - (1 + 1))
4th = 14 => ((10 + 15) - (5 + 6))
5th = 42 => ((35 + 56) - (21 + 28))
6th = 132 => ((126 + 210) - (84 + 120))
Now if we try to write the nth catalan number in terms of Binomial coefficient

n=0th:- 1(base case)
n=1st:- 1 (base case)
n=2nd:-2 (base case)

n=3rd:- ($(C^3_2 + C^4_3)$ - $(C^3_3 + C^4_4)$)

n=4th:- ($(C^5_3 + C^6_4)$ - $(C^5_4 + C^6_5)$)

n=5th:- ($(C^7_4 + C^8_5)$ - $(C^7_5 + C^8_6)$)

n=6th:- ($(C^9_5 + C^{10}_6)$ - $(C^9_6 + C^{10}_7)$)

n=7th:-($(C^{11}_6 + C^{12}_7)$ - $(C^{11}_7 + C^{12}_8)$)

n=8th:- ($(C^{13}_7 + C^{14}_8)$ - $(C^{13}_8 + C^{14}_9)$)

. . . .

Similarly if you try to generalize these for the nth term then the equation will be look like this,

n=nth:- ($(C^{2(n-1)-1}_{n-1} + C^{2n-2}_n)$ - $(C^{2n-3}_n + C^{2(n+1)-4}_{n+1})$)

=> nth = ($(C^{2n-3}_{n-1} + C^{2n-2}_n)$ - $(C^{2n-3}_n + C^{2n-2}_{n+1})$)

After some rearrangements i can write this above equation in following manner,

=> nth = ($(C^{2n-3}_{n-1} - C^{2n-3}_n)$ + $(C^{2n-2}_n - C^{2n-2}_{n+1})$)

=> nth = $\left(\frac{(2n-3)!}{(n-1)!(n-2)!} - \frac{(2n-3)!}{(n)!(n-3)!}\right) + \left(\frac{(2n-2)!}{(n)!(n-2)!} - \frac{(2n-2)!}{(n+1)!(n-3)!}\right)$

=> nth = $\frac{(2n-3)!}{(n-1)!(n-3)!}\left(\frac{1}{n-2} - \frac{1}{n}\right) + \frac{(2n-2)!}{(n)!(n-3)!}\left(\frac{1}{n-2} - \frac{1}{n+1}\right)$

=> nth = $\frac{(2n-3)!}{(n-1)!(n-3)!}\left(\frac{n-(n-2)}{n(n-2)}\right) + \frac{(2n-2)!}{(n)!(n-3)!}\left(\frac{(n+1)-(n-2)}{(n-2)(n+1)}\right)$

=> nth = $\frac{2(2n-3)!}{n(n-1)!*(n-2)(n-3)!} + \frac{3(2n-2)!}{(n+1)(n)!*(n-2)(n-3)!}$

=> nth = $\frac{2(2n-3)!}{n(n-1)!*(n-2)(n-3)!} + \frac{3(2n-2)!}{(n+1)(n)!*(n-2)(n-3)!}$

$$\Rightarrow nth = \frac{2(2n-3)!}{n!*(n-2)!} + \frac{3(2n-2)!}{(n+1)!*(n-2)!}$$

$$\Rightarrow nth = \frac{(2n-3)!}{n!*(n-2)!}\left(2 + \frac{3*(2n-2)}{n+1}\right)$$ [as,(2n-2)! = (2n-2)*(2n-3)! & (n+1)! = (n+1)*n!]

$$\Rightarrow nth = \frac{(2n-3)!}{n!*(n-2)!} * \frac{4*(2n-1)}{(n+1)}$$

Now, multiply **(n-1)** in numerator and denominator.

$$\Rightarrow nth = \frac{(2n-3)!}{n!*(n-2)!}\frac{4*(2n-1)}{(n+1)} * \frac{(n-1)}{(n-1)}$$

$$\Rightarrow nth = \frac{(2n-3)!}{(n+1)!*(n-2)!} * \frac{2*(2n-1)*(2n-2)}{n-1}$$

[as, (n+1)*n! = (n+1)! and 2*2*(2n-1)*(n-1)=>2*(2n-1)*(2n-2)]

$$\Rightarrow nth = \frac{2*(2n-1)!}{(n+1)!*(n-1)!}$$ [as, 2*(2n-1)*(2n-2)*(2n-3)! = 2*(2n-1)! and (n-1)*(n-2)! = (n-1)!]

Now, multiply (2n) in numerator and denominator.

$$\Rightarrow nth = \frac{2*(2n-1)!}{(n+1)!*(n-1)!} * \frac{2n}{2n}$$

$$\Rightarrow nth = \frac{2n!}{(n+1)!*n*(n-1)!} \Rightarrow \frac{2n!}{(n+1)!*n!} = C_n$$

$$\therefore C_n = \frac{2n!}{(n+1)!*n!} \text{ or } C_n = \frac{1}{(n+1)}\binom{2n}{n}$$

$$C_n = \frac{1}{(n+1)}\binom{2n}{n} \Rightarrow \frac{1}{(n+1)} * \frac{2n!}{n!(2n-n)!} \Rightarrow \frac{1}{(n+1)} * \frac{2n!}{n!(n)!} \Rightarrow \frac{2n!}{n!*(n+1)!}$$

Below are the steps for calculating $\mathbf{nC_r}$.

- Create a variable to store the answer and change **r** to **n – r** if **r** is greater than **n – r** because we know that **C(n, r) = C(n, n-r)** if r > n – r
- Run a loop from **0** to **r-1**
    - In every iteration update ans as **(ans*(n-i))/(i+1)**, where i is the loop counter.
- So the answer will be equal to **((n/1)*((n-1)/2)*...*((n-r+1)/r)**, which is equal to **nC_r**.

Below are steps to calculate Catalan numbers using the formula: $2nC_n/(n+1)$

- Calculate **$2nC_n$** using the similar steps that we use to calculate **nC_r**
- Return the value $2nC_n/ (n + 1)\backslash$

Below is the implementation of the above approach:

C++Java

```
// C++ program for nth Catalan Number
#include <iostream>
using namespace std;

// Returns value of Binomial Coefficient C(n, k)
unsigned long int binomialCoeff(unsigned int n,
                                unsigned int k)
{
        unsigned long int res = 1;

        // Since C(n, k) = C(n, n-k)
        if (k > n - k)
                k = n - k;

        // Calculate value of [n*(n-1)*---*(n-k+1)] /
        // [k*(k-1)*---*1]
        for (int i = 0; i < k; ++i) {
                res *= (n - i);
                res /= (i + 1);
        }

        return res;
}

// A Binomial coefficient based function to find nth catalan
// number in O(n) time
unsigned long int catalan(unsigned int n)
{
        // Calculate value of 2nCn
        unsigned long int c = binomialCoeff(2 * n, n);

        // return 2nCn/(n+1)
        return c / (n + 1);
}

// Driver code
```

```
int main()
{
        for (int i = 0; i < 10; i++)
                cout << catalan(i) << " ";
        return 0;
}
```

Output

```
1 1 2 5 14 42 132 429 1430 4862
```

Time Complexity: O(n).
Auxiliary Space: O(1)

We can also use the below formulas to find **nth** Catalan number in O(n) time.

Count Ways To Reach Grid Top

The number of paths with 2n steps on a rectangular grid from the bottom left corner, denoted as (n-1, 0), to the top right corner, denoted as (0, n-1), that do not cross above the main diagonal can be calculated using Catalan numbers. Catalan numbers are a sequence of natural numbers that have wide-ranging applications in various fields of mathematics, including combinatorics and geometry.

To better understand the concept, let's consider a rectangular grid with n columns and n rows, where each cell represents a step that can be taken either to the right (horizontal) or upward (vertical). The goal is to start from the bottom left corner and reach the top right corner, moving only in the upward and rightward directions, without crossing above the main diagonal. The main diagonal of the grid is the diagonal that runs from the top left corner to the bottom right corner.

Now, let's denote the steps to the right as 'R' and the steps upward as 'U'. For a valid path that does not cross above the main diagonal, at any point along the path, the number of 'R's must be greater than or equal to the number of 'U's. This ensures that the path stays below the main diagonal.

For example, let's consider a rectangular grid with n = 3, where the bottom left corner is denoted as (2, 0) and the top right corner is denoted as (0, 2):

```
. . .
. . .
x . .
```

Here, 'x' represents the starting point at (2, 0), and the goal is to reach the top right corner at (0, 2). Following the condition mentioned earlier, there must be at least as many 'R's as 'U's at any point along the path.

One way to calculate the number of such paths is to use Catalan numbers. The nth Catalan number, denoted as Cn, is given by the formula:

```
Cn = (2n)! / ((n+1)! * n!)
```

where '!' denotes the factorial operation, which is the product of all positive integers up to a given number.

In our case, we can use Catalan numbers to calculate the number of valid paths from the bottom left corner to the top right corner without crossing above the main diagonal. Since the grid has n columns and n rows, the number of steps in total is 2n. Therefore, the number of such paths is Cn.

For example, when n = 3, we can plug in the value into the formula for Catalan numbers:

```
C3 = (2 * 3)! / ((3+1)! * 3!)
  = 6! / (4! * 3!)
  = (6 * 5 * 4 * 3 * 2 * 1) / ((4 * 3 * 2 * 1) * (3 * 2 * 1))
  = 5
```

So, there are a total of 5 valid paths from the bottom left corner to the top right corner of a 3x3 rectangular grid without crossing above the main diagonal.

In general, the Catalan numbers provide a way to count the number of valid paths in a rectangular grid with 2n steps from the bottom left corner to the top right corner without crossing above the main diagonal. These numbers have many applications in various fields of mathematics and can be used to solve a wide range of combinatorial problems involving paths, trees, and parentheses, among others.

Number of Binary Trees

A Binary Tree is labeled if every node is assigned a label and a Binary Tree is unlabelled if nodes are not assigned any label.

```
Below two are considered same unlabelled trees
    o                o
  /   \            /   \
 o     o          o     o

Below two are considered different labelled trees
    A                C
  /   \            /  \
 B     C          A    B
```

How many different Unlabelled Binary Trees can be there with n nodes?

```
For n  = 1, there is only one tree
   o

For n  = 2, there are two trees
   o      o
  /        \
 o          o

For n  = 3, there are five trees
    o      o           o         o      o
   /        \         /  \      /        \
  o          o       o    o    o          o
 /            \                 \        /
o              o                 o      o
```

The idea is to consider all possible pairs of counts for nodes in left and right subtrees and multiply the counts for a particular pair. Finally, add the results of all pairs.

```
For example, let T(n) be count for n nodes.
T(0) = 1  [There is only 1 empty tree]
T(1) = 1
T(2) = 2

T(3) =  T(0)*T(2) + T(1)*T(1) + T(2)*T(0) = 1*2 + 1*1 + 2*1 = 5

T(4) =  T(0)*T(3) + T(1)*T(2) + T(2)*T(1) + T(3)*T(0)
     =  1*5 + 1*2 + 2*1 + 5*1
     =  14
```

The above pattern basically represents n'th Catalan Numbers. First few Catalan numbers are 1 1 2 5 14 42 132 429 1430 4862,...

Here,
T(i-1) represents the number of nodes on the left-sub-tree
T(n−i-1) represents the number of nodes on the right-sub-tree

n'th Catalan Number can also be evaluated using the direct formula.

```
T(n) = (2n)! / (n+1)!n!
```

The number of Binary Search Trees (BST) with n nodes is also the same as the number of unlabelled trees. The reason for this is simple, in BST also we can make any key a root, If the root is i'th key in sorted order, then i-1 keys can go on one side, and (n-i) keys can go on another side.

How many labeled Binary Trees can be there with n nodes?
To count labeled trees, we can use the above count for unlabelled trees. The idea is simple, every unlabelled tree with n nodes can create n! different labeled trees by assigning different permutations of labels to all nodes.

Therefore,

```
Number of Labelled Trees = (Number of unlabelled trees) * n!
                         = [(2n)! / (n+1)!n!]  × n!
```

For example for n = 3, there are 5 * 3! = 5*6 = 30 different labelled trees

Counting Polygon Triangulation

A triangulation is a partition of the polygon into non-intersecting triangles by adding diagonals (line segments connecting non-adjacent vertices of the polygon). For example, a square can be divided into two triangles by adding a diagonal from one vertex to the opposite vertex.

Now, let T(n) denote the number of ways to triangulate a convex polygon. To count the number of triangulations, we can use a recurrence relation based on the structure of triangulations. Specifically, we can choose any vertex of the polygon and draw a diagonal from that vertex to some other vertex. This diagonal divides the polygon into two smaller polygons, each of which can be triangulated independently. Therefore, the number of triangulations of the n-gon is given by the sum of the products of the number of triangulations of the two smaller polygons, summed over all possible choices of the dividing diagonal:

```
T(n) = T(1)T(n-1) + T(2)T(n-2) + ... + T(n-1)T(1)
```
where T(1) = 1, since a single vertex is already a triangle. This recurrence relation can be derived by considering all possible choices of the diagonal and the number of ways to triangulate the two smaller polygons.

Using this recurrence relation, we can compute T(n) recursively. For example, T(3) = 1, T(4) = 2, T(5) = 5, T(6) = 14, and so on. However, computing the number of triangulations for larger values of n can be quite time-consuming.

Alternatively, we can use the closed-form formula for the nth Catalan number, which is:

```
C(n) = (2n)! / (n!(n+1)!)
```
It turns out that T(n) is equal to the (n-2)nd Catalan number, which can be seen by considering that a triangulation of an n-gon corresponds to a valid sequence of n-2 diagonal choices. There are (n-2)! ways to order the diagonals, but each triangulation can be obtained in two ways, since each diagonal can be flipped. Therefore, the total number of triangulations is:

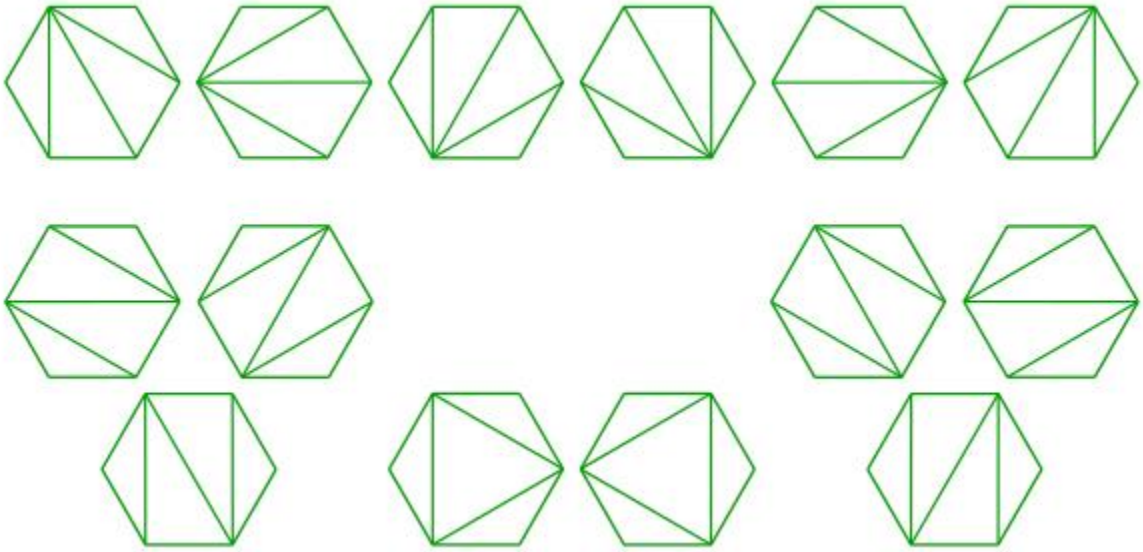
```
T(n) = (n-2)!/2 * C(n-2)
```
This formula directly gives us the number of ways we can divide a convex n-gon into triangles, without needing to compute the recurrence relation. For example, the number of ways to divide a hexagon (6-gon) into triangles is given by T(6) = C(4) = (2*6-4)! / (4!*4!) = 14.

More Applications of Catalan

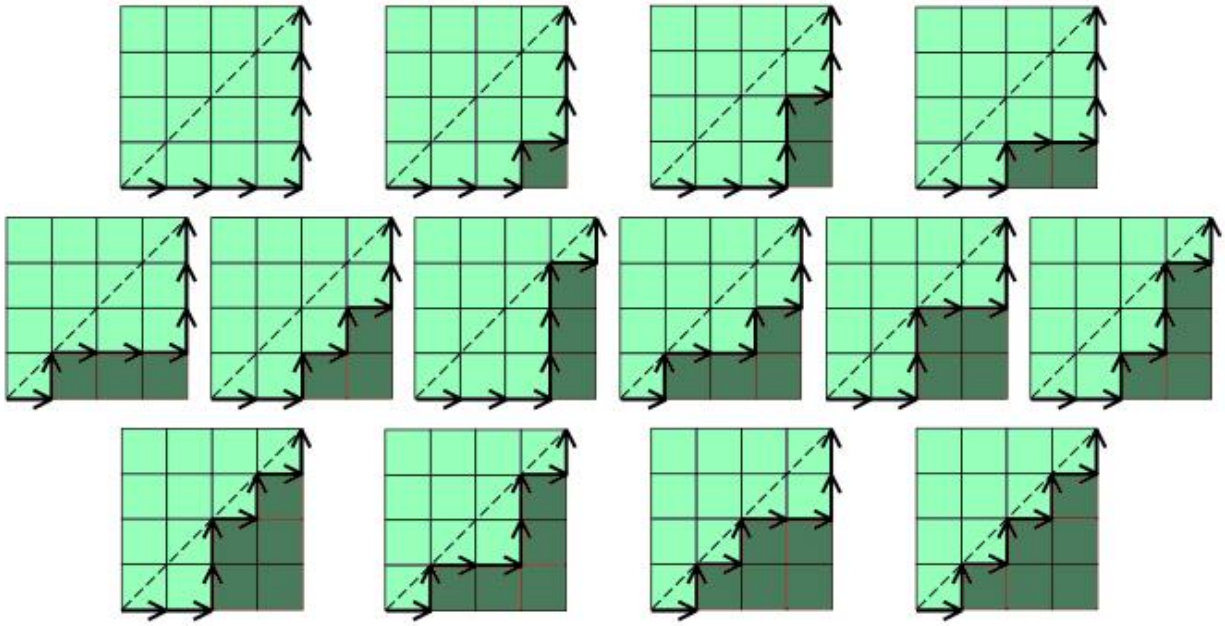
Applications :

1. Number of possible Binary Search Trees with n keys.
2. Number of expressions containing n pairs of parentheses which are correctly matched. For n = 3, possible expressions are ((())), ()(()), ()()(), (())(), (()()).

3. Number of ways a convex polygon of n+2 sides can split into triangles by connecting vertices.

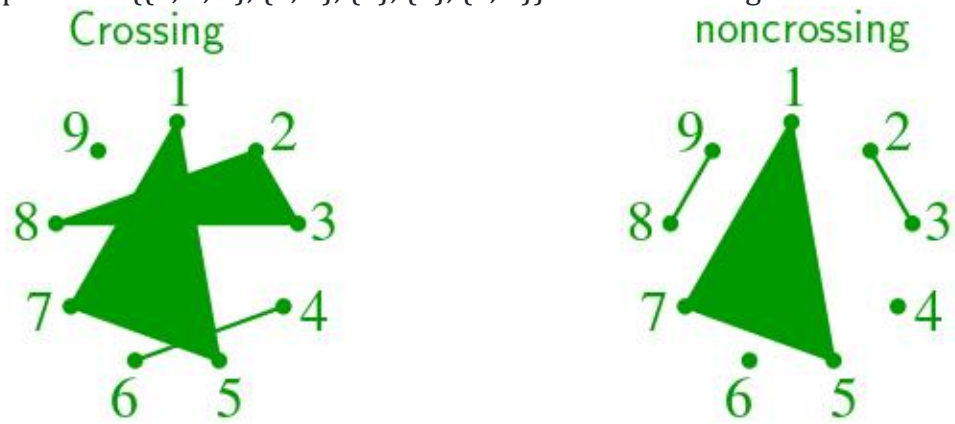


4. Number of full binary trees (A rooted binary tree is full if every vertex has either two children or no children) with n+1 leaves.
5. Number of different Unlabeled Binary Trees can be there with n nodes.
6. The number of paths with 2n steps on a rectangular grid from bottom left, i.e., (n-1, 0) to top right (0, n-1) that do not cross above the main diagonal.

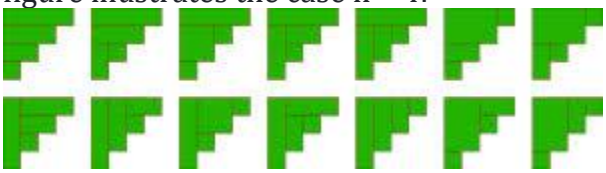


7. Number of ways to insert n pairs of parentheses in a word of n+1 letters, e.g., for n=2 there are 2 ways: ((ab)c) or (a(bc)). For n=3 there are 5 ways, ((ab)(cd)), (((ab)c)d), ((a(bc))d), (a((bc)d)), (a(b(cd))).
8. Number of noncrossing partitions of the set {1, …, 2n} in which every block is of size 2. A partition is noncrossing if and only if in its planar diagram, the blocks are

disjoint (i.e. don't cross). For example, below two are crossing and non-crossing partitions of {1, 2, 3, 4, 5, 6, 7, 8, 9}. The partition {{1, 5, 7}, {2, 3, 8}, {4, 6}, {9}} is crossing and partition {{1, 5, 7}, {2, 3}, {4}, {6}, {8, 9}} is non-crossing.



9. Number of Dyck words of length 2n. A Dyck word is a string consisting of n X's and n Y's such that no initial segment of the string has more Y's than X's. For example, the following are the Dyck words of length 6:
   XXXYYY XYXXYY XYXYXY XXYYXY XXYXYY.
10. Number of ways to tile a stairstep shape of height n with n rectangles. The following figure illustrates the case n = 4:



11. Number of ways to connect the points on a circle disjoint chords. This is similar to point 3 above.
12. Number of ways to form a "mountain ranges" with n upstrokes and n down-strokes that all stay above the original line.The mountain range interpretation is that the mountains will never go below the horizon.

$n = 0$:	*	1 way
$n = 1$:	/\	1 way
$n = 2$:	/\ /\/\, / \	2 ways
$n = 3$:	/\ /\ /\ /\/\ / \ /\/\/\, /\/ \, / \/\, / \, / \	5 ways

Mountain Ranges

13. Number of stack-sortable permutations of {1, …, n}. A permutation w is called stack-sortable if S(w) = (1, …, n), where S(w) is defined recursively as follows: write w = unv where n is the largest element in w and u and v are shorter sequences, and set S(w) = S(u)S(v)n, with S being the identity for one-element sequences.

14. Number of permutations of {1, …, n} that avoid the pattern 123 (or any of the other patterns of length 3); that is, the number of permutations with no three-term increasing subsequence. For n = 3, these permutations are 132, 213, 231, 312 and 321. For n = 4, they are 1432, 2143, 2413, 2431, 3142, 3214, 3241, 3412, 3421, 4132, 4213, 4231, 4312 and 4321

Modular Arithmetic

Why did we choose 10^9+7?

In most programming competitions, we are required to answer the result in 10^9+7 modulo. The reason behind this is, if problem constraints are large integers, only efficient algorithms can solve them in an allowed limited time.

What is modulo operation:

The remainder obtained after the division operation on two operands is known as modulo operation. The operator for doing modulus operation is **'%'**. For ex: a % b = c which means, when a is divided by b it gives the remainder c, 7%2 = 1, 17%3 = 2.

Why do we need modulo:

- The reason of taking Mod is to prevent integer overflows. The largest integer data type in C/C++ is *unsigned long long int* which is of 64 bit and can handle integer from 0 to (2^64 - 1). But in some problems where the growth rate of output is very high, this high range of unsigned long long may be insufficient.
  Suppose in a 64 bit variable 'A', 2^62 is stored and in another 64 bit variable 'B', 2^63 is stored. When we multiply A and B, the system does not give a runtime error or exception. It just does some bogus computation and stores the bogus result because the big size of the result comes after multiplication overflows.

- In some of the problems, to compute the result modulo inverse is needed and this number helps a lot because it is prime. Also, this number should be large enough otherwise modular inverse techniques may fail in some situations.

Due to these reasons, problem setters require to give the answer as a result of modulo of some number **N**.

There are certain criteria on which the value of N depends:

1. It should just be large enough to fit in the largest integer data type i.e it makes sure that there is no overflow in the result.
2. It should be a prime number because if we take mod of a number by Prime the result is generally spaced i.e. the results are very different results in comparison to mod the number by non-prime, that is why primes are generally used for mod.

10^9+7 fulfills both the criteria. It is the first 10-digit prime number and fits in int data type as well. In fact, any prime number less than 2^30 will be fine in order to prevent possible overflows.

How modulo is used:

A few distributive properties of modulo are as follows:

1. (a + b) % c = ((a % c) + (b % c)) % c
2. (a * b) % c = ((a % c) * (b % c)) % c
3. (a - b) % c = ((a % c) - (b % c)) % c
4. ~~(a / b) % c = ((a % c) / (b % c)) % c~~

So, modulo is distributive over +, * and – but not over /

NOTE: The result of (a % b) will always be less than b.

In the case of computer programs, due to the size of variable limitations, we **perform modulo M at each intermediate stage** so that range overflow never occurs

Range of int types

Integers are a fundamental data type in programming languages, used to represent whole numbers. However, not all integer types are created equal. Different programming languages have different implementations of integer types, which can vary in terms of size, range, and behavior. In this article, we will explore the ranges of integer types in three popular programming languages: C++, Java, and Python.

1. C++: C++ provides a variety of integer types with different sizes and ranges. The most common integer types in C++ are:

- **int**: This is a signed integer type that typically represents a signed integer with a range of at least -2^31 to 2^31-1 .
- **unsigned int**: This is an unsigned integer type that represents a non-negative integer with a range of at least 0 to 2^32-1

- **long long**: This is a signed integer type that represents a signed integer with a larger range than **int**, typically -2^62 to 2^63-1.
- **unsigned long long**: This is an unsigned integer type with a larger range than **unsigned int**, typically 0 to 2^64-1

It's important to note that the exact size and range of integer types in C++ may vary depending on the platform and compiler used.

2. Java: Java also provides several integer types with different sizes and ranges. The most commonly used integer types in Java are:

- **int**: This is a signed integer type that represents a signed integer with a range of at least -2^31 to 2^31-1.
- **long**: This is a signed integer type that represents a signed integer with a larger range than **int**, typically -2^63 to 2^63-1 .

Java also provides corresponding unsigned integer types through the use of the **long** type with bitwise operations. However, Java does not have a built-in unsigned integer type like C++.

3. Python: Python's integer types behave differently from those in C++ and Java, as Python has dynamic typing and supports arbitrary precision integers. In Python, integers can grow or shrink in size as needed to accommodate their values, without worrying about overflow or underflow errors. Python provides two main integer types:

- **int**: This is a signed integer type that can represent signed integers of arbitrary size. Python automatically converts integers to long integers when they exceed the range of a regular **int**, and long integers can have virtually unlimited precision.
- **bool**: This is a boolean integer type that represents either **True** or **False** values, which are used in Python for logical operations.

In conclusion, the ranges of integer types in C++, Java, and Python vary in terms of size,

Modular Arithmetic Properties

Modular arithmetic is the branch of arithmetic mathematics related with the "mod" functionality. Basically, modular arithmetic is related with computation of "mod" of expressions. Expressions may have digits and computational symbols of addition, subtraction, multiplication, division or any other. Here we will discuss briefly about all modular arithmetic operations.

Quotient Remainder Theorem:

It states that, for any pair of integers a and b (b is positive), there exist two unique integers q and r such that:

a = b x q + r where 0 <= r < b

Example: If a = 20, b = 6 then q = 3, r = 2 20 = 6 x 3 + 2

Modular Addition:

Rule for modular addition is:

(a + b) mod m = ((a mod m) + (b mod m)) mod m

Example:

```
(15 + 17) % 7
= ((15 % 7) + (17 % 7)) % 7
= (1 + 3) % 7
= 4 % 7
= 4
```

The same rule is to modular subtraction. We don't require much modular subtraction but it can also be done in the same way.

Modular Multiplication:

The Rule for modular multiplication is:

(a x b) mod m = ((a mod m) x (b mod m)) mod m

Example:

```
(12 x 13) % 5
= ((12 % 5) x (13 % 5)) % 5
= (2 x 3) % 5
= 6 % 5
= 1
```

Modular Division:

The modular division is totally different from modular addition, subtraction and multiplication. It also does not exist always.

`(a / b) mod m is not equal to ((a mod m) / (b mod m)) mod m.`

This is calculated using the following formula:

(a / b) mod m = (a x (inverse of b if exists)) mod m

Modular Inverse:

The modular inverse of a mod m exists only if a and m are relatively prime i.e. gcd(a, m) = 1. Hence, for finding the inverse of an under modulo m, if (a x b) mod m = 1 then b is the modular inverse of a.

Example: a = 5, m = 7 (5 x 3) % 7 = 1 hence, 3 is modulo inverse of 5 under 7.

Modulo Inverse Using Fermet's Little Theorem

Fermat's little theorem states that if p is a prime number, then for any integer a, the number a^p – a is an integer multiple of p.

Here p is a prime number
$a^p \equiv a \pmod p$.

Special Case: If a is not divisible by p, Fermat's little theorem is equivalent to the statement that a^{p-1}-1 is an integer multiple of p.

$a^{p-1} \equiv 1 \pmod p$
OR
$a^{p-1} \% p = 1$
Here a is not divisible by p.
Take an Example How Fermat's little theorem works

Example :

```
P = an integer Prime number
a = an integer which is not multiple of P
Let a = 2 and P = 17

According to Fermat's little theorem
```

$2^{17-1} \equiv 1 \mod(17)$
we got 65536 % 17 ≡ 1
that mean (65536-1) is an multiple of 17

Use of Fermat's little theorem

If we know m is prime, then we can also use Fermat's little theorem to find the inverse.

$$a^{m-1} \equiv 1 \pmod{m}$$

If we multiply both sides with a^{-1}, we get

$$a^{-1} \equiv a^{m-2} \pmod{m}$$

Below is the Implementation of above

C++

```
// C++ program for Modulo Inverse Using Fermet's Little Theorem

#include<iostream>
#include <bits/stdc++.h>
using namespace std;



class GFG{
public:
    // Function for modular inverse of a under modulo m where m is prime
    void modInverse(int a, int m)
    {
        if (__gcd(a, m) != 1)
            cout << "Inverse doesn't exist";

        else {

            // If a and m are relatively prime, then
            // modulo inverse is a^(m-2) mode m
            cout << "Modular multiplicative inverse is "
                 << power(a, m - 2, m);
        }
    }

    // for computing x^y under modulo m
    int power(int x, unsigned int y, unsigned int m)
    {
        if (y == 0)
            return 1;
        int p = power(x, y / 2, m) % m;
        p = (p * p) % m;

        return (y % 2 == 0) ? p : (x * p) % m;
```

```
    }

};


// Driver Program
int main()
{
    int a = 3, m = 11;
    class GFG obj;
    obj.modInverse(a, m);
    return 0;
}
```

Output

```
Modular multiplicative inverse is 4
```

Time Complexity: O(log m)

Modulo Inverse of 1 to N

Give a positive integer n, find modular multiplicative inverse of all integer from 1 to n with respect to a big prime number, say, 'prime'.

```
Input : n = 10, prime = 17
Output : 1 9 6 13 7 3 5 15 2 12
Explanation :
For 1, modular inverse is 1 as (1 * 1)%17 is 1
For 2, modular inverse is 9 as (2 * 9)%17 is 1
For 3, modular inverse is 6 as (3 * 6)%17 is 1
.......

Input : n = 5, prime = 7
Output : 1 4 5 2 3
```

A **simple solution** is to one by one find modular inverse for every number.

C++

```
// CPP program to find modular inverse of
// all numbers from 1 to n using naive
// method

#include <bits/stdc++.h>
```

```
using namespace std;


class GFG {
public:

        // A naive method to find modular
        // multiplicative inverse of 'a'
        // under modulo 'prime'
        int modInverse(int a, int prime)
        {
                a = a % prime;
                for (int x = 1; x <prime; x++)
                if ((a * x) % prime == 1)
                        return x;

                return -1;
        }

        void printModIverses(int n, int prime)
        {
                for (int i = 1; i <= n; i++)
                cout << modInverse(i, prime) << " ";
        }
};



int main() {
        int n = 5, prime = 13;
        class GFG obj;
        obj.printModIverses(n, prime);
        return 0;
}
```

Output

```
1 7 9 10 8
```

Time Complexity: O(n*prime)

Auxiliary Space: O(1)

An **efficient solution** is based on extended Euclid algorithm.

Extended Euclidean algorithm finds integer coefficients x and y such that:

```
  ax + by = gcd(a, b)

Let us put b = prime, we get
  ax + prime * y = gcd(a, prime)
```

```
We know gcd(a, prime) = 1 because
one of the numbers is prime. So we
know
  ax + prime * y = 1

Since prime * y is a multiple of prime,
x is modular multiplicative inverse of a.

 ax  ≡ 1 (mod prime)
```

We can recursively find x using below expression (see extended Euclid algorithm for details).
The extended Euclidean algorithm updates results of gcd(a, b) using the results calculated by recursive call gcd(b%a, a). Let values of x and y calculated by the recursive call be x_{prev} and y_{prev}. x and y are updated using below expressions.

$x = y_{prev} - \lfloor prime/a \rfloor * x_{prev}$
$y = x_{prev}$

We use above relation to compute inverse using previously computed values.

```
inverse(a) = (inverse(prime % a) *
              (prime - prime/a)) % primeTime Complexity:
O(n*prime)
Auxiliary Space: O(1)

An efficient solution is based on extended Euclid algorithm.

Extended Euclidean algorithm finds integer coefficients x and y
such that:

  ax + by = gcd(a, b)

Let us put b = prime, we get
  ax + prime * y = gcd(a, prime)

We know gcd(a, prime) = 1 because
one of the numbers is prime. So we
know
  ax + prime * y = 1

Since prime * y is a multiple of prime,
x is modular multiplicative inverse of a.

 ax  ≡ 1 (mod prime)
```

The extended Euclidean algorithm updates results of gcd(a, b) using the results calculated by recursive call gcd(b%a, a). Let values of x and y calculated by the recursive call be x_{prev} and y_{prev}. x and y are updated using below expressions.

```
x = y_prev - ⌊prime/a⌋ * x_prev
y = x_prev
```

We use above relation to compute inverse using previously computed values.

```
inverse(a) = (inverse(prime % a) *
              (prime - prime/a)) % prime
```

We use Dynamic Programming approach that uses above recursive structure.

Dynamic Approach :
dp[1] = 1,
dp[2] = dp[17%2]*(17-17/2)%17 = 9
dp[3] = dp[17%3]*(17-17/3)%17 = 6
and so on..

C++

```cpp
// CPP code to find modular inverse
// from 1 to n w.r.t a big prime number
#include <iostream>
using namespace std;

// Function to calculate modular
// inverse using D.P
void modularInverse(int n, int prime)
{
        int dp[n + 1];
        dp[0] = dp[1] = 1;
        for (int i = 2; i <= n; i++)
                dp[i] = dp[prime % i] *
                        (prime - prime / i) % prime;

        for (int i = 1; i <= n; i++)
                cout << dp[i] << ' ';
}

// Driver code
int main()
{
        int n = 10, prime = 17;
        modularInverse(n, prime);
        return 0;
}
```

Output

```
1 9 6 13 7 3 5 15 2 12
```

Time Complexity: θ *(n)*
Auxiliary Space: θ *(n)*

Chinese Remainder Theorem (Introduction)

We are given two arrays num[0..k-1] and rem[0..k-1]. In num[0..k-1], every pair is coprime (gcd for every pair is 1). We need to find minimum positive number x such that:

```
x % num[0]    =  rem[0],
x % num[1]    =  rem[1],
.....................
x % num[k-1]  =  rem[k-1]
```

Basically, we are given k numbers which are pairwise coprime, and given remainders of these numbers when an unknown number x is divided by them. We need to find the minimum possible value of x that produces given remainders.

Examples :

```
Input:  num[] = {5, 7}, rem[] = {1, 3}
Output: 31
Explanation:
31 is the smallest number such that:
  (1) When we divide it by 5, we get remainder 1.
  (2) When we divide it by 7, we get remainder 3.

Input:  num[] = {3, 4, 5}, rem[] = {2, 3, 1}
Output: 11
Explanation:
11 is the smallest number such that:
  (1) When we divide it by 3, we get remainder 2.
  (2) When we divide it by 4, we get remainder 3.
  (3) When we divide it by 5, we get remainder 1.
```

Chinese Remainder Theorem states that there always exists an x that satisfies given congruences. Below is theorem statement adapted from wikipedia.

Let num[0], num[1], …num[k-1] be positive integers that are pairwise coprime. Then, for any given sequence of integers rem[0], rem[1], … rem[k-1], there exists an integer x solving the following system of simultaneous congruences.

$$\begin{cases} x \equiv rem[0] & (\text{mod } num[0]) \\ \quad \dots \\ x \equiv rem[k-1] & (\text{mod } num[k-1]) \end{cases}$$

Furthermore, all solutions x of this system are congruent modulo the product, prod = num[0] * num[1] * ... * nun[k-1]. Hence

$$x \equiv y \ (\text{mod } num[i]), \quad 0 \leq i \leq k-1 \quad \Longleftrightarrow \quad x \equiv y \ (\text{mod } prod).$$

The first part is clear that there exists an x. The second part basically states that all solutions (including the minimum one) produce the same remainder when divided by- product of num[0], num[1], .. num[k-1]. In the above example, the product is 3*4*5 = 60. And 11 is one solution, other solutions are 71, 131, .. etc. All these solutions produce the same remainder when divided by 60, i.e., they are of form 11 + m*60 where m >= 0.
A **Naive Approach to find x** is to start with 1 and one by one increment it and check if dividing it with given elements in num[] produces corresponding remainders in rem[]. Once we find such an x, we return it.
Below is the implementation of Naive Approach.

C++

```
// A C++ program to demonstrate working of Chinese remainder
// Theorem
#include<bits/stdc++.h>
using namespace std;

// k is size of num[] and rem[]. Returns the smallest
// number x such that:
// x % num[0] = rem[0],
// x % num[1] = rem[1],
// ..................
// x % num[k-2] = rem[k-1]
// Assumption: Numbers in num[] are pairwise coprime
// (gcd for every pair is 1)
int findMinX(int num[], int rem[], int k)
{
        int x = 1; // Initialize result

        // As per the Chinese remainder theorem,
        // this loop will always break.
        while (true)
        {
                // Check if remainder of x % num[j] is
                // rem[j] or not (for all j from 0 to k-1)
```

```
                int j;
                for (j=0; j<k; j++ )
                        if (x%num[j] != rem[j])
                        break;

                // If all remainders matched, we found x
                if (j == k)
                        return x;

                // Else try next number
                x++;
        }

        return x;
}

// Driver method
int main(void)
{
        int num[] = {3, 4, 5};
        int rem[] = {2, 3, 1};
        int k = sizeof(num)/sizeof(num[0]);
        cout << "x is " << findMinX(num, rem, k);
        return 0;
}
```

Output

```
x is 11
```

CRT Efficient Implementation

We are given two arrays num[0..k-1] and rem[0..k-1]. In num[0..k-1], every pair is coprime (gcd for every pair is 1). We need to find minimum positive number x such that:

```
x % num[0]     =  rem[0],
x % num[1]     =  rem[1],
.....................
x % num[k-1]   =  rem[k-1]
```

Example:

```
Input:  num[] = {3, 4, 5}, rem[] = {2, 3, 1}
Output: 11
Explanation:
```

```
11 is the smallest number such that:
  (1) When we divide it by 3, we get remainder 2.
  (2) When we divide it by 4, we get remainder 3.
  (3) When we divide it by 5, we get remainder 1.
```

an efficient solution to find x is discussed.

The solution is based on below formula.

```
x =  ( ∑ (rem[i]*pp[i]*inv[i]) ) % prod
   Where 0 <= i <= n-1

rem[i] is given array of remainders

prod is product of all given numbers
prod = num[0] * num[1] * ... * num[k-1]

pp[i] is product of all divided by num[i]
pp[i] = prod / num[i]

inv[i] = Modular Multiplicative Inverse of
         pp[i] with respect to num[i]
```

Example:

```
Let us take below example to understand the solution
   num[] = {3, 4, 5}, rem[] = {2, 3, 1}
   prod  = 60
   pp[]  = {20, 15, 12}
   inv[] = {2,  3,  3}  // (20*2)%3 = 1, (15*3)%4 = 1
                        // (12*3)%5 = 1

   x = (rem[0]*pp[0]*inv[0] + rem[1]*pp[1]*inv[1] +
        rem[2]*pp[2]*inv[2]) % prod
     = (2*20*2 + 3*15*3 + 1*12*3) % 60
     = (80 + 135 + 36) % 60
     = 11
```

Below is the implementation of above formula.

C++

```cpp
// A C++ program to demonstrate
// working of Chinese remainder
// Theorem
#include <bits/stdc++.h>
using namespace std;

// Returns modulo inverse of a
// with respect to m using
// extended Euclid Algorithm.
// Refer below post for details:
```

```
// https://www.geeksforgeeks.org/
// multiplicative-inverse-under-modulo-m/
int inv(int a, int m)
{
	int m0 = m, t, q;
	int x0 = 0, x1 = 1;

	if (m == 1)
		return 0;

	// Apply extended Euclid Algorithm
	while (a > 1) {
		// q is quotient
		q = a / m;

		t = m;

		// m is remainder now, process same as
		// euclid's algo
		m = a % m, a = t;

		t = x0;

		x0 = x1 - q * x0;

		x1 = t;
	}

	// Make x1 positive
	if (x1 < 0)
		x1 += m0;

	return x1;
}

// k is size of num[] and rem[]. Returns the smallest
// number x such that:
// x % num[0] = rem[0],
// x % num[1] = rem[1],
// ..................
// x % num[k-2] = rem[k-1]
// Assumption: Numbers in num[] are pairwise coprime
// (gcd for every pair is 1)
int findMinX(int num[], int rem[], int k)
{
	// Compute product of all numbers
	int prod = 1;
	for (int i = 0; i < k; i++)
		prod *= num[i];
```

```
        // Initialize result
        int result = 0;

        // Apply above formula
        for (int i = 0; i < k; i++) {
                int pp = prod / num[i];
                result += rem[i] * inv(pp, num[i]) * pp;
        }

        return result % prod;
}

// Driver method
int main(void)
{
        int num[] = { 3, 4, 5 };
        int rem[] = { 2, 3, 1 };
        int k = sizeof(num) / sizeof(num[0]);
        cout << "x is " << findMinX(num, rem, k);
        return 0;
}
```

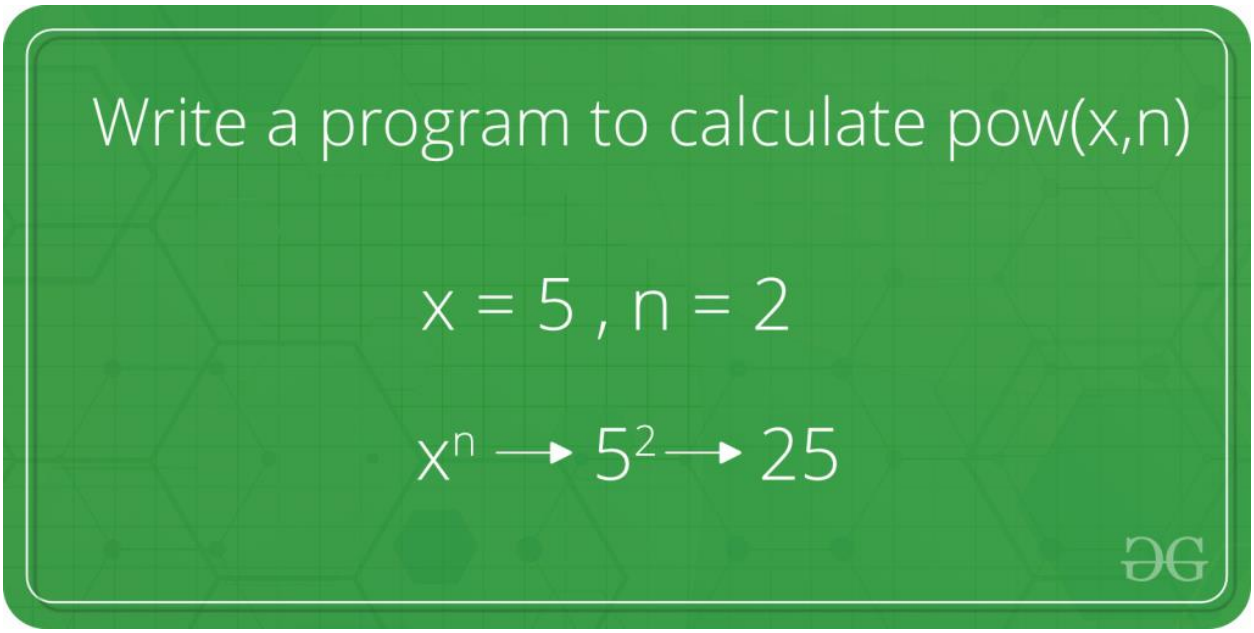
Output
```
x is 11
```
Time Complexity : O(N*LogN)

Auxiliary Space : O(1)

Matrix Exponentiation

Computing Power

Given two integers **x** and **n**, write a function to compute $\mathbf{x}^{\mathbf{n}}$. We may assume that x and n are small and overflow doesn't happen.



Examples :

Input : *x = 2, n = 3*
Output : *8*

Input : *x = 7, n = 2*
Output : *49*
Naive Approach: To solve the problem follow the below idea:

A simple solution to calculate pow(x, n) would multiply x exactly n times. We can do that by using a simple for loop
Below is the implementation of the above approach:

C++Java

```
// C++ program for the above approach
#include <bits/stdc++.h>
using namespace std;

// Naive iterative solution to calculate pow(x, n)
long power(int x, unsigned n)
{
        // Initialize result to 1
        long long pow = 1;

        // Multiply x for n times
        for (int i = 0; i < n; i++) {
                pow = pow * x;
```

```
        }

        return pow;
}

// Driver code
int main(void)
{

        int x = 2;
        unsigned n = 3;

        // Function call
        int result = power(x, n);
        cout << result << endl;

        return 0;
}
```

Output

8

Time Complexity: O(n)
Auxiliary Space: O(1)

Program to calculate pow(x, n) using Divide and Conqueror approach:

To solve the problem follow the below idea:

The problem can be recursively defined by:

- *power(x, n) = power(x, n / 2) * power(x, n / 2); // if n is even*
- *power(x, n) = x * power(x, n / 2) * power(x, n / 2); // if n is odd*

Below is the implementation of the above approach:

C++Java

```
// C++ program to calculate pow(x,n)
#include <bits/stdc++.h>
using namespace std;
class gfg {
```

```
        /* Function to calculate x raised to the power y */
public:
        int power(int x, unsigned int y)
        {
                if (y == 0)
                        return 1;
                else if (y % 2 == 0)
                        return power(x, y / 2) * power(x, y / 2);
                else
                        return x * power(x, y / 2) * power(x, y / 2);
        }
};

/* Driver code */
int main()
{
        gfg g;
        int x = 2;
        unsigned int y = 3;

        // Function call
        cout << g.power(x, y);
        return 0;
}

// This code is contributed by SoM15242
```

Output

8

Time Complexity: O(n)
Auxiliary Space: O(n)

An Optimized Divide and Conquer Solution:
To solve the problem follow the below idea:

There is a problem with the above solution, the same subproblem is computed twice for each recursive call. We can optimize the above function by computing the solution of the subproblem once only.
Below is the implementation of the above approach:

C++Java

```
/* Function to calculate x raised to the power y in
* O(logn)*/
```

```
int power(int x, unsigned int y)
{
        int temp;
        if (y == 0)
                return 1;
        temp = power(x, y / 2);
        if (y % 2 == 0)
                return temp * temp;
        else
                return x * temp * temp;
}

// This code is contributed by Shubhamsingh10
```

Output

```
8
```

Time Complexity: O(log n)
Auxiliary Space: O(log n), for recursive call stack

Iterative Power

Given an integer x and a positive number y, write a function that computes x^y under following conditions.
a) Time complexity of the function should be O(Log y)
b) Extra Space is O(1)

Examples:

```
Input: x = 3, y = 5
Output: 243

Input: x = 2, y = 5
Output: 32
```

Following is implementation to compute x^y.

C++Java

```
// Iterative C program to implement pow(x, n)
#include <iostream>
using namespace std;
```

```
/* Iterative Function to calculate (x^y) in O(logy) */
int power(int x, unsigned int y)
{
        int res = 1; // Initialize result

        while (y > 0) {
                // If y is odd, multiply x with result
                if (y & 1)
                        res = res * x;

                // y must be even now
                y = y >> 1; // y = y/2
                x = x * x; // Change x to x^2
        }
        return res;
}

// Driver program to test above functions
int main()
{
        int x = 3;
        unsigned int y = 5;

        cout<<"Power is "<<power(x, y);

        return 0;
}

// this code is contributed by shivanisinghss2110
```

Output

```
Power is 243
```

Time Complexity: O(log y), since in loop each time the value of y decreases by half it's current value.
Auxiliary Space: O(1), since no extra space has been taken

Matrix Exponentiation

This is one of the most used techniques in competitive programming. Let us first consider below simple question.

In this post, a general implementation of Matrix Exponentiation is discussed.

For solving the matrix exponentiation we are assuming a linear recurrence equation like below:

```
F(n) = a*F(n-1) + b*F(n-2) + c*F(n-3)   for n >= 3
                                  . . . . . Equation (1)
```

where a, b and c are constants.

For this recurrence relation, it depends on three previous values.
Now we will try to represent Equation (1) in terms of the matrix.

```
[First Matrix] = [Second matrix] * [Third Matrix]
| F(n)   |     =   Matrix 'C'    *  | F(n-1) |
| F(n-1) |                           | F(n-2) |
| F(n-2) |                           | F(n-3) |
```

Dimension of the first matrix is 3 x 1 .
Dimension of the third matrix is also 3 x 1.

So the dimension of the second matrix must be 3 x 3
[For multiplication rule to be satisfied.]

Now we need to fill the Matrix 'C'.

So according to our equation.
F(n) = a*F(n-1) + b*F(n-2) + c*F(n-3)
F(n-1) = F(n-1)
F(n-2) = F(n-2)

```
C = [a b c
     1 0 0
     0 1 0]
```

Now the relation between matrix becomes :

```
[First Matrix]  [Second matrix]       [Third Matrix]
| F(n)   |  =  | a b c |  *          | F(n-1) |
| F(n-1) |     | 1 0 0 |             | F(n-2) |
| F(n-2) |     | 0 1 0 |             | F(n-3) |
```

Lets assume the initial values for this case :-
F(0) = 0
F(1) = 1
F(2) = 1

So, we need to get F(n) in terms of these values.

So, for n = 3 Equation (1) changes to

```
| F(3) |  =  | a b c |  *          | F(2) |
| F(2) |     | 1 0 0 |             | F(1) |
| F(1) |     | 0 1 0 |             | F(0) |
```

Now similarly for n = 4

```
| F(4) |  =  | a b c |  *          | F(3) |
| F(3) |     | 1 0 0 |             | F(2) |
| F(2) |     | 0 1 0 |             | F(1) |

             - - - -  2 times - - -
| F(4) |  =  | a b c |  * | a b c | *       | F(2) |
| F(3) |     | 1 0 0 |    | 1 0 0 |         | F(1) |
| F(2) |     | 0 1 0 |    | 0 1 0 |         | F(0) |
```

So for n, the Equation (1) changes to

```
                - - - - - - - - - n -2 times - - - -  -
| F(n)   |  =  | a b c | * | a b c | * ... * | a b c | * | F(2)
|
| F(n-1) |     | 1 0 0 |   | 1 0 0 |         | 1 0 0 |   | F(1)
|
| F(n-2) |     | 0 1 0 |   | 0 1 0 |         | 0 1 0 |   | F(0)
|


| F(n)   |  =  [ | a b c | ] ^ (n-2)   *  | F(2) |
| F(n-1) |     [ | 1 0 0 | ]              | F(1) |
| F(n-2) |     [ | 0 1 0 | ]              | F(0) |
```

So we can simply multiply our Second matrix n-2 times and then multiply it with the third matrix to get the result. Multiplication can be done in (log n) time using Divide and Conquer algorithm for power (See this or this)

Let us consider the problem of finding n'th term of a series defined using below recurrence.

```
n'th term,
    F(n) = F(n-1) + F(n-2) + F(n-3), n >= 3
Base Cases :
    F(0) = 0, F(1) = 1, F(2) = 1
```

We can find n'th term using following :

```
Putting a = 1, b = 1 and c = 1 in above formula
```

```
| F(n)   |   =   [ | 1 1 1 | ] ^ (n-2)    *   | F(2) |
| F(n-1) |       [ | 1 0 0 | ]                | F(1) |
| F(n-2) |       [ | 0 1 0 | ]                | F(0) |
```

Below is the implementation of above idea.

C++Java

```
// C++ program to find value of f(n) where f(n)
// is defined as
//    F(n) = F(n-1) + F(n-2) + F(n-3), n >= 3
// Base Cases :
//    F(0) = 0, F(1) = 1, F(2) = 1
#include<bits/stdc++.h>
using namespace std;

// A utility function to multiply two matrices
// a[][] and b[][].  Multiplication result is
// stored back in b[][]
void multiply(int a[3][3], int b[3][3])
{
    // Creating an auxiliary matrix to store elements
    // of the multiplication matrix
    int mul[3][3];
    for (int i = 0; i < 3; i++)
    {
        for (int j = 0; j < 3; j++)
        {
            mul[i][j] = 0;
            for (int k = 0; k < 3; k++)
                mul[i][j] += a[i][k]*b[k][j];
        }
    }

    // storing the multiplication result in a[][]
    for (int i=0; i<3; i++)
        for (int j=0; j<3; j++)
            a[i][j] = mul[i][j];  // Updating our matrix
}

// Function to compute F raise to power n-2.
int power(int F[3][3], int n)
{
    int M[3][3] = {{1,1,1}, {1,0,0}, {0,1,0}};

    // Multiply it with initial values i.e with
    // F(0) = 0, F(1) = 1, F(2) = 1
    if (n==1)
        return F[0][0] + F[0][1];
```

```
    power(F, n/2);

    multiply(F, F);

    if (n%2 != 0)
        multiply(F, M);

    // Multiply it with initial values i.e with
    // F(0) = 0, F(1) = 1, F(2) = 1
    return F[0][0] + F[0][1] ;
}

// Return n'th term of a series defined using below
// recurrence relation.
// f(n) is defined as
//    f(n) = f(n-1) + f(n-2) + f(n-3), n>=3
// Base Cases :
//    f(0) = 0, f(1) = 1, f(2) = 1
int findNthTerm(int n)
{

    int F[3][3] = {{1,1,1}, {1,0,0}, {0,1,0}} ;

    //Base cases
    if(n==0)
        return 0;
    if(n==1 || n==2)
        return 1;

    return power(F, n-2);
}

// Driver code
int main()
{
   int n = 5;

   cout << "F(5) is " << findNthTerm(n);

   return 0;
}
```

Output

```
F(5) is 7
```

Time Complexity: O(logN)

Auxiliary Space: O(logN)

Combinatorial Game Theory

Combinational Game Theory Introduction

Combinatorial Game Theory (CGT) is a branch of mathematics that deals with the study of two-player games where the players take turns making moves according to certain rules. In CGT, the emphasis is on the structure of the game rather than on the specific strategies or tactics that the players may use. CGT has applications in a wide range of fields, including computer science, economics, and physics.

A combinatorial game can be defined as a game where the players have perfect information and the game always ends after a finite number of moves, without any element of chance or randomness. Each move in a combinatorial game reduces the available options, and the player who is unable to make a move loses the game.

CGT analyzes games in terms of their **"value"** and **"outcome"**. The value of a game is a mathematical object that represents the advantage that a player has over the other, assuming both play perfectly. The outcome of a game is simply who wins or loses.

Combinatorial games can be classified into two categories: **impartial games** and **partisan games**. In impartial games, the rules and moves are the same for both players. Examples of impartial games include Nim, Sprague-Grundy, and Hackenbush. In partisan games, the rules and moves are different for each player. Examples of partisan games include Chess, Checkers, and Go.

CGT also involves the study of game equivalence, which is a concept that allows us to compare different games and determine if they have the same outcome. Two games are said to be equivalent if they have the same value and outcome, even if their rules and structure are different.

Overall, Combinatorial Game Theory provides a powerful framework for the analysis and study of a wide range of two-player games. Its applications span a wide range of fields, including computer science, economics, and physics, and it continues to be an active area of research and development.

Impartial Game Examples (Find the player who will win the Coin game)

Given **N** coins, the task is to find who win the coin game.
Coin game is a game in which each player picks coins from the given **N** coins in such a way that he can pick coins ranging from 1 to 5 coins in one turn and the game continues for both the players. The player who picks the last coin loses the game.
Examples:

```
Input: N = 4
Output: First Player
Explanation:
Player 1 pick 3 coins and
Player 2 pick last coin

Input: N = 7
Output: Second Player
```

Approach:

- As the player can take coins ranging from 1 to 5 inclusively and if a player loses it means that he had only 1 coin, otherwise, he could have taken 1 less coin than available coins and force another player to lose. So now we will consider the case when the second player is going to win, which means the first player had only one coin.
- For N = 1, second player is going to win, for N = 2 to 6, first player can choose 1 less coin than N, and force the second player to lose so discard them, for N = 7, first player can choose coin 1 to 5, that's going to leave coin ranging from 6 to 2, and then second player can choose 1 to 5, and to win second player will intelligently choose 1 less coin forcing first to lose, So basically starting from 1, all the numbers on a gap of 6(as whatever first player choose, second player will choose coins equal to difference of 6 and coins chosen by the first player) will be the winning for second player.
- Finally, we just have to check if n is of form 6*c+1 if it is then the second player going to win, otherwise, the first player going to win.

Below is the implementation for the above approach:

C++JavaPython3C#Javascript

```
// C++ program to find the player
// who wins the game
```

```cpp
#include <bits/stdc++.h>
using namespace std;

// Function to check the
// winning player
void findWinner(int n)
{
    // As discussed in the
    // above approach
    if ((n - 1) % 6 == 0) {
        cout << "Second Player wins the game";
    }
    else {
        cout << "First Player wins the game";
    }
}

// Driver function
int main()
{

    int n = 7;
    findWinner(n);
}
```

Output:

```
Second Player wins the game
```

Time Complexity:O(1)

Auxiliary Space: O(1)

Mex and Grundy Number

Grundy Numbers or Numbers determine how any Impartial Game (not only the Game of Nim) can be solved once we have calculated the Grundy Numbers associated with that game using Sprague-Grundy Theorem.
But before calculating Grundy Numbers, we need to learn about another term- Mex.

What is Mex?
'Minimum excludant' a.k.a 'Mex' of a set of numbers is the smallest non-negative number

not present in the set.

$$\begin{aligned}
&\mathrm{mex}(\emptyset) = 0\\
&\mathrm{mex}(\{1,2,3\}) = 0\\
&\mathrm{mex}(\{0,2,4,6,\ldots\}) = 1\\
&\mathrm{mex}(\{0,1,4,7,12\}) = 2\\
&\mathrm{mex}(\{0,1,2,3,\ldots\}) = \omega\\
&\mathrm{mex}(\{0,1,2,3,\ldots,\omega\}) = \omega+1
\end{aligned}$$

How to calculate Grundy Numbers?
We use this definition- The Grundy Number/ number is equal to 0 for a game that is lost immediately by the first player and is equal to Mex of the numbers of all possible next positions for any other game.
Below are three example games and programs to calculate Grundy Number and Mex for each of them. Calculation of Grundy Numbers is done basically by a recursive function called as calculateGrundy() function which uses calculateMex() function as its sub-routine.

Example 1
The game starts with a pile of n stones, and the player to move may take any positive number of stones. Calculate the Grundy Numbers for this game. The last player to move wins. Which player wins the game?
Since if the first player has 0 stones, he will lose immediately, so Grundy(0) = 0
If a player has 1 stones, then he can take all the stones and win. So the next possible position of the game (for the other player) is (0) stones
Hence, Grundy(1) = Mex(0) = 1 [According to the definition of Mex]
Similarly, If a player has 2 stones, then he can take only 1 stone or he can take all the stones and wins. So the next possible position of the game (for the other player) is (1, 0) stones respectively.
Hence, Grundy(2) = Mex(0, 1) = 2 [According to the definition of Mex]
Similarly, If a player has 'n' stones, then he can take only 1 stone, or he can take 2 stones..... or he can take all the stones and win. So the next possible position of the game (for the other player) is (n-1, n-2,....1) stones respectively.
Hence, Grundy(n) = Mex (0, 1, 2, ...n-1) = n [According to the definition of Mex]

We summarize the first the Grundy Value from 0 to 10 in the below table-

N	0	1	2	3	4	5	6	7	8	9	10
Grundy (n)	0	1	2	3	4	5	6	7	8	9	10

C++Java

```cpp
/* A recursive C++ program to find Grundy Number for
   a game which is like a one-pile version of Nim.
  Game Description : The game starts with a pile of n stones,
  and the player to move may take any positive number of stones.
The last player to move wins. Which player wins the game? */
#include<bits/stdc++.h>
using namespace std;

// A Function to calculate Mex of all the values in
// that set.
int calculateMex(unordered_set<int> Set)
{
    int Mex = 0;

    while (Set.find(Mex) != Set.end())
        Mex++;

    return (Mex);
}

// A function to Compute Grundy Number of 'n'
// Only this function varies according to the game
int calculateGrundy(int n)
{
    if (n == 0)
        return (0);

    unordered_set<int> Set; // A Hash Table

    for (int i=0; i<=n-1; i++)
            Set.insert(calculateGrundy(i));

    return (calculateMex(Set));
}

// Driver program to test above functions
int main()
{
    int n = 10;
    printf("%d", calculateGrundy(n));
    return (0);
```

```
}
```

Output :

```
10
```

The above solution can be optimized using Dynamic Programming as there are overlapping subproblems. The Dynamic programming based implementation can be found here.

Example 2

The game starts with a pile of n stones, and the player to move may take any positive number of stones up to 3 only. The last player to move wins. Which player wins the game?
This game is 1 pile version of Nim.
Since if the first player has 0 stones, he will lose immediately, so Grundy(0) = 0
If a player has 1 stones, then he can take all the stones and win. So the next possible position of the game (for the other player) is (0) stones

Hence, Grundy(1) = Mex(0) = 1 [According to the definition of Mex]
Similarly, if a player has 2 stones, then he can take only 1 stone or he can take 2 stones and win. So the next possible position of the game (for the other player) is (1, 0) stones respectively.
Hence, Grundy(2) = Mex(0, 1) = 2 [According to the definition of Mex]
Similarly, Grundy(3) = Mex(0, 1, 2) = 3 [According to the definition of Mex]

But what about 4 stones ?
If a player has 4 stones, then he can take 1 stone or he can take 2 stones or 3 stones, but he can't take 4 stones (see the constraints of the game). So the next possible position of the game (for the other player) is (3, 2, 1) stones respectively.
Hence, Grundy(4) = Mex (1, 2, 3) = 0 [According to the definition of Mex]
So we can define Grundy Number of any n >= 4 recursively as-
Grundy(n) = Mex[Grundy (n-1), Grundy (n-2), Grundy (n-3)]

We summarize the first the Grundy Value from 0 to 10 in the below table-

N	0	1	2	3	4	5	6	7	8	9	10
Grundy (n)	0	1	2	3	0	1	2	3	0	1	2

C++Java

```cpp
/* A recursive C++ program to find Grundy Number for
a game which is one-pile version of Nim.
Game Description : The game starts with a pile of
n stones, and the player to move may take any
positive number of stones up to 3 only.
The last player to move wins. */
#include<bits/stdc++.h>
using namespace std;

// A Function to calculate Mex of all the values in
// that set.

// A function to Compute Grundy Number of 'n'
// Only this function varies according to the game
int calculateGrundy(int n)
{
    if (n == 0)
        return (0);
    if (n == 1)
        return (1);
    if (n == 2)
        return (2);
    if (n == 3)
        return (3);
    else
        return (n%(3+1));
}

// Driver program to test above functions
int main()
{
    int n = 10;
    printf("%d", calculateGrundy(n));
    return (0);
}
```

Output :

```
2
```

The general solution for above code when we are allowed to pick upto k stones can be found here.

Example 3

The game starts with a number- 'n' and the player to move divides the number- 'n' with 2, 3

or 6 and then takes the floor. If the integer becomes 0, it is removed. The last player to move wins. Which player wins the game?

We summarize the first the Grundy Value from 0 to 10 in the below table:

N	0	1	2	3	4	5	6	7	8	9	10
Grundy (n)	0	1	2	2	3	3	0	0	0	0	0

Think about how we generated this table.

C++Java

```
/* A recursive C++ program to find Grundy Number for
   a game.
 Game Description:  The game starts with a number- 'n'
 and the player to move divides the number- 'n' with 2, 3
 or 6 and then takes the floor. If the integer becomes 0,
 it is removed. The last player to move wins.  */
#include<bits/stdc++.h>
using namespace std;

// A Function to calculate Mex of all the values in
// that set.
int calculateMex(unordered_set<int> Set)
{
    int Mex = 0;

    while (Set.find(Mex) != Set.end())
        Mex++;

    return (Mex);
}

// A function to Compute Grundy Number of 'n'
// Only this function varies according to the game
int calculateGrundy (int n)
{
    if (n == 0)
        return (0);

    unordered_set<int> Set; // A Hash Table

    Set.insert(calculateGrundy(n/2));
    Set.insert(calculateGrundy(n/3));
    Set.insert(calculateGrundy(n/6));
```

```
    return (calculateMex(Set));
}

// Driver program to test above functions
int main()
{
    int n = 10;
    printf("%d", calculateGrundy (n));
    return (0);
}
```

Output :

```
0
```

The above solution can be optimized using Dynamic Programming as there are overlapping subproblems. The Dynamic programming based implementation can be found here.

Sprague Grundy Theorem

What is Sprague-Grundy Theorem?

Suppose there is a composite game (more than one sub-game) made up of N sub-games and two players, A and B. Then Sprague-Grundy Theorem says that if both A and B play optimally (i.e., they don't make any mistakes), then the player starting first is guaranteed to win if the XOR of the grundy numbers of position in each sub-games at the beginning of the game is non-zero. Otherwise, if the XOR evaluates to zero, then player A will lose definitely, no matter what.

How to apply Sprague Grundy Theorem ?

We can apply Sprague-Grundy Theorem in any impartial game and solve it. The basic steps are listed as follows:

1. Break the composite game into sub-games.
2. Then for each sub-game, calculate the Grundy Number at that position.
3. Then calculate the XOR of all the calculated Grundy Numbers.
4. If the XOR value is non-zero, then the player who is going to make the turn (First Player) will win else he is destined to lose, no matter what.

Example Game : The game starts with 3 piles having 3, 4 and 5 stones, and the player to move may take any positive number of stones upto 3 only from any of the piles [Provided

that the pile has that much amount of stones]. The last player to move wins. Which player wins the game assuming that both players play optimally?

How to tell who will win by applying Sprague-Grundy Theorem?
As, we can see that this game is itself composed of several sub-games.
First Step : The sub-games can be considered as each piles.
Second Step : We see from the below table that

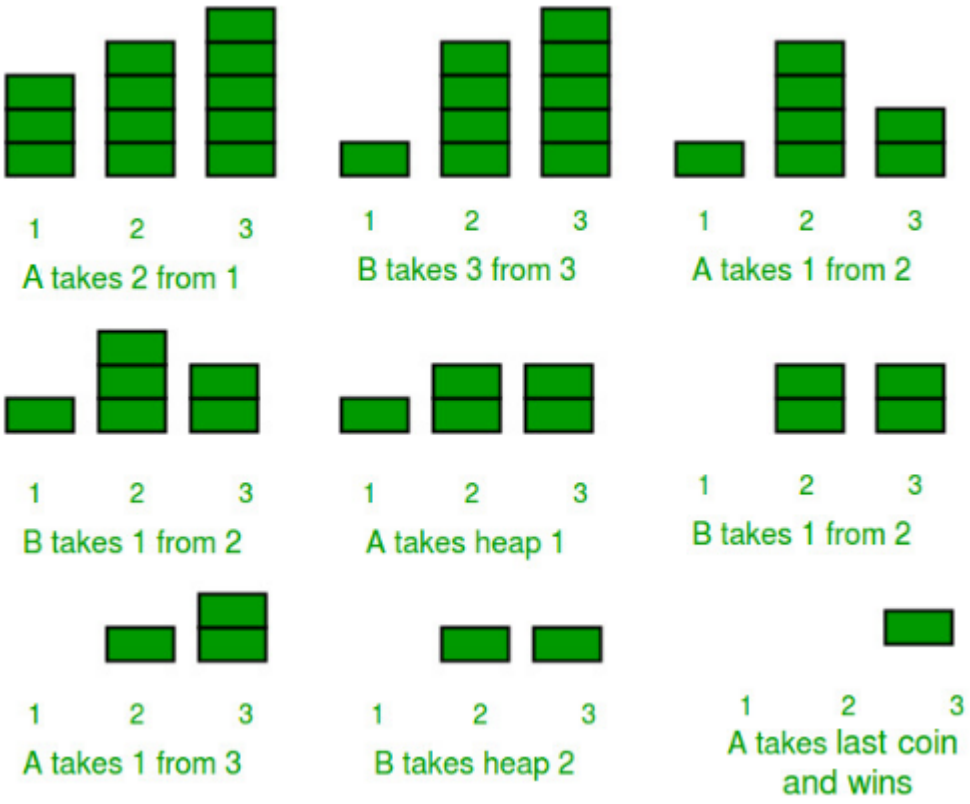
```
Grundy(3) = 3
Grundy(4) = 0
Grundy(5) = 1
```

N	0	1	2	3	4	5	6	7	8	9	10
Grundy (n)	0	1	2	3	0	1	2	3	0	1	2

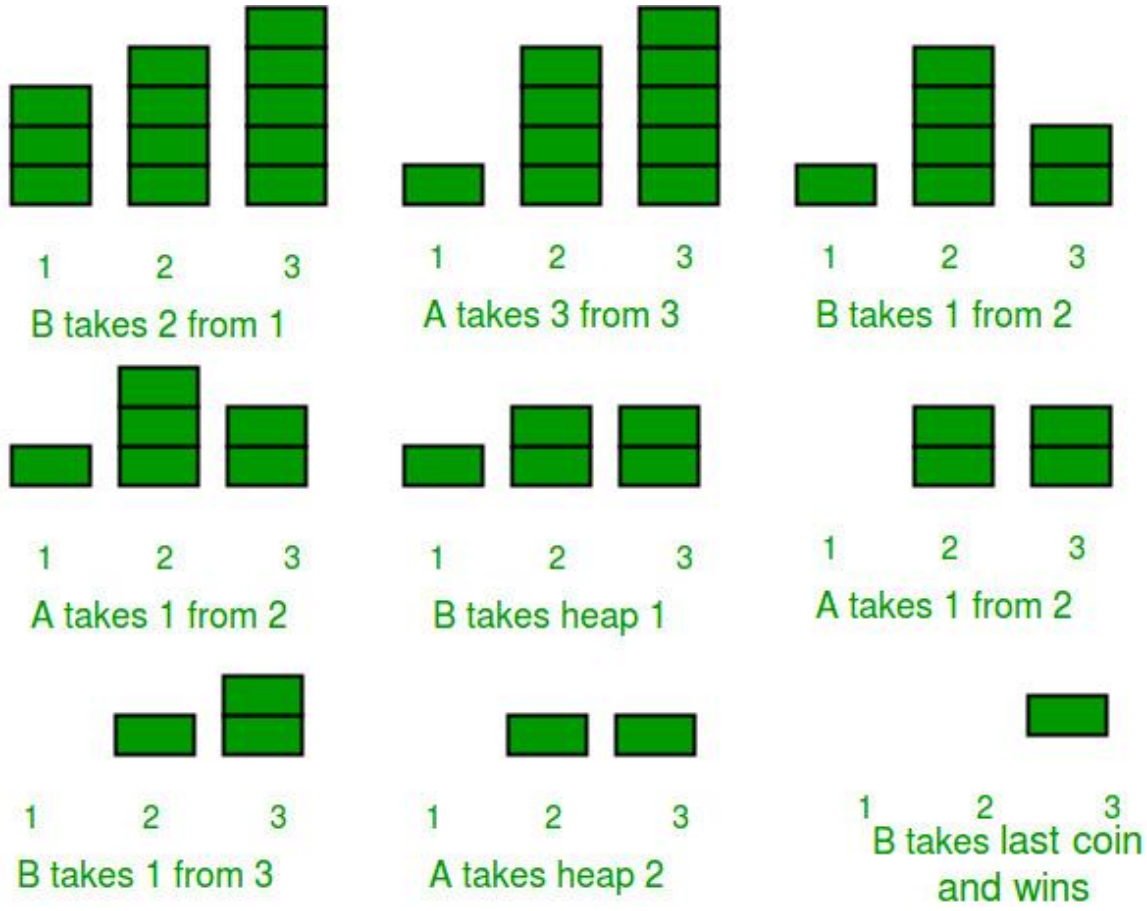
We have already seen how to calculate the Grundy Numbers of this game in the previous article.
Third Step : The XOR of 3, 0, 1 = 2
Fourth Step : Since XOR is a non-zero number, so we can say that the first player will win.

The Game of Nim

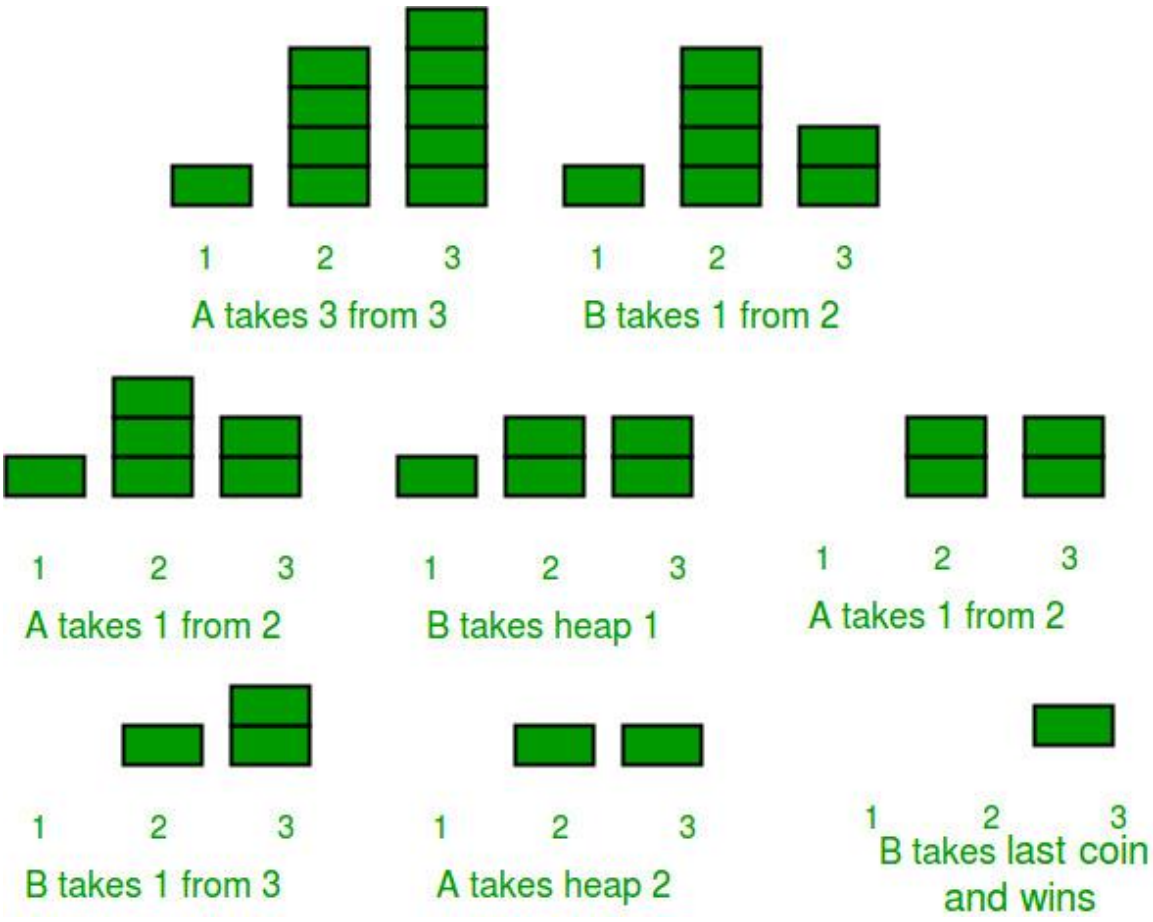
The Game of Nim is described by the following rules- " *Given a number of piles in which each pile contains some numbers of stones/coins. In each turn, a player can choose only one pile and remove any number of stones (at least one) from that pile. The player who cannot move is considered to lose the game (i.e., one who take the last stone is the winner).* " For example, consider that there are two players- **A** and **B**, and initially there are three piles of coins initially having **3, 4, 5** coins in each of them as shown below. We assume that first move is made by **A**. See the below figure for clear understanding of the whole game play.



. A Won the match (Note: A made the first move) So was **A** having a strong expertise in this game ? or he/she was having some edge over **B** by starting first ? Let us now play again, with the same configuration of the piles as above but this time **B** starting first instead of **A**.



B Won the match (Note: B made the first move) **B**y the above figure, it must be clear that the game depends on one important factor – Who starts the game first ? **So does the player who starts first will win everytime ?** Let us again play the game, starting from **A** , and this time with a different initial configuration of piles. The piles have 1, 4, 5 coins initially. Will **A** win again as he has started first ? Let us see.



A made the first move, but lost the Game. So, the result is clear. **A** has lost. But how? We know that this game depends heavily on which player starts first. Thus, there must be another factor which dominates the result of this simple-yet-interesting game. That factor is the initial configuration of the heaps/piles. This time the initial configuration was different from the previous one. So, we can conclude that this game depends on two factors-

1. The player who starts first.
2. The initial configuration of the piles/heaps.

Working of Nim-Sum Formula

Nim-Sum : The cumulative XOR value of the number of coins/stones in each piles/heaps at any point of the game is called Nim-Sum at that point. ***"If both A and B play optimally (i.e- they don't make any mistakes), then the player starting first is guaranteed to win if the Nim-Sum at the beginning of the game is non-zero. Otherwise, if the Nim-Sum evaluates***

to zero, then player A will lose definitely." For the proof of the above theorem, see- https://en.wikipedia.org/wiki/Nim#Proof_of_the_winning_formula

Optimal Strategy :

- If the XOR sum of 'n' numbers is already zero then there is no possibility to make the XOR sum zero by single reduction of a number.
- If the XOR sum of 'n' numbers is non-zero then there is at least a single approach by which if you reduce a number, the XOR sum is zero.

Initially two cases could exist.

- **Case 1: Initial Nim Sum is zero** As we know, in this case if played optimally **B** wins, which means **B** would always prefer to have Nim sum of zero for **A**'s turn. So, as the Nim Sum is initially zero, whatever number of items **A** removes the new Nim Sum would be non-zero (as mentioned above). Also, as **B** would prefer Nim sum of zero for **A**'s turn, he would then play a move so as to make the Nim Sum zero again (which is always possible, as mentioned above). The game will run as long as there are items in any of the piles and in each of their respective turns **A** would make Nim sum non-zero and **B** would make it zero again and eventually there will be no elements left and **B** being the one to pick the last wins the game. **It is evident by above explanation that the optimal strategy for each player is to make the Nim Sum for his opponent zero in each of their turn, which will not be possible if it's already zero.**
- **Case 2: Initial Nim Sum is non-zero** Now going by the optimal approach **A** would make the Nim Sum to be zero now (which is possible as the initial Nim sum is non-zero, as mentioned above). Now, in **B**'s turn as the nim sum is already zero whatever number **B** picks, the nim sum would be non-zero and **A** can pick a number to make the nim sum zero again. This will go as long as there are items available in any pile. And **A** will be the one to pick the last item. So, as discussed in the above cases, it should be obvious now that Optimal strategy for any player is to make the nim sum zero if it's non-zero and if it is already zero then whatever moves the player makes now, it can be countered.

Let us apply the above theorem in the games played above. In the first game **A** started first and the Nim-Sum at the beginning of the game was, 3 XOR 4 XOR 5 = 2, which is a non-zero value, and hence **A** won. Whereas in the second game-play, when the initial configuration of the piles were 1, 4, and 5 and **A** started first, then **A** was destined to lose as the Nim-Sum at the beginning of the game was 1 XOR 4 XOR 5 = 0 .

Working of Grundy Number Theorem

Grundy Number Theorem is a mathematical concept used in game theory to analyze and solve certain types of games. It states that in a game where each player takes turns making a move and the game is played on a finite state graph, each vertex of the graph is assigned a non-negative integer called the Grundy number, which represents the "value" of that position.

The Grundy number of a position is calculated as the minimum non-negative integer that is not assigned to any of its immediate successors in the state graph. In other words, the Grundy number of a position is the smallest non-negative integer that is not assigned to any position that can be reached in one move from that position.

Once the Grundy numbers are assigned to all positions in the game, the value of the entire game can be calculated by taking the XOR (exclusive or) of the Grundy numbers of all positions that are reachable from the starting position. If the XOR is zero, then the starting position is a losing position (i.e., the player who moves first will lose no matter what moves they make), and if the XOR is non-zero, then the starting position is a winning position (i.e., the player who moves first can always force a win).

The Grundy number theorem can be used to analyze various combinatorial games, including Nim, Kayles, and Sprague-Grundy theory. It provides a systematic way of determining the winning and losing positions in a game and can be used to develop optimal strategies for playing such games.

Geometric Algorithms

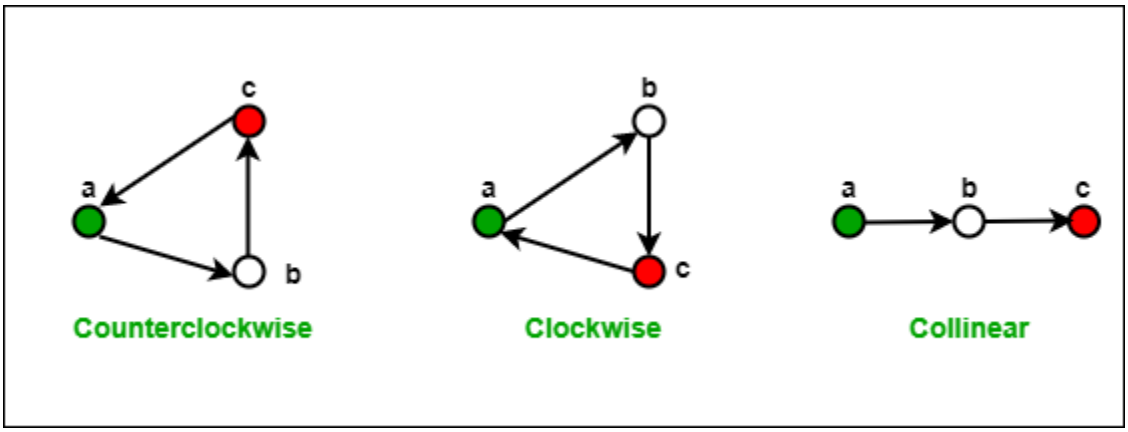
Orientation of 3 ordered points

Given three points p1, p2 and p3, the task is to determine the orientation of these three points.

Orientation of an ordered triplet of points in the plane can be

- counterclockwise
- clockwise
- collinear

The following diagram shows different possible orientations of (a,b,c)



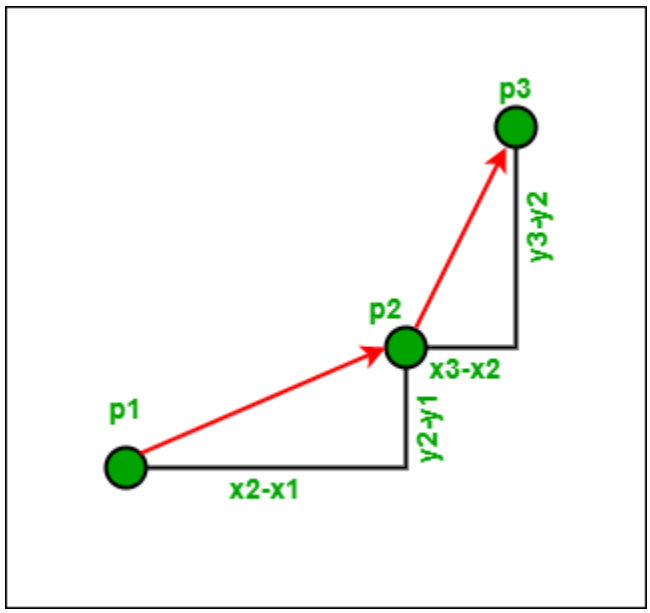
If orientation of (p1, p2, p3) is collinear, then orientation of (p3, p2, p1) is also collinear.
If orientation of (p1, p2, p3) is clockwise, then orientation of (p3, p2, p1) is counterclockwise and vice versa is also true.
Example:

Input: p1 = {0, 0}, p2 = {4, 4}, p3 = {1, 2}
Output: CounterClockWise

Input: p1 = {0, 0}, p2 = {4, 4}, p3 = {1, 1}
Output: Collinear

How to compute Orientation?

The idea is to use slope.



Slope of line segment (p1, p2): σ = (y2 - y1)/(x2 - x1)
Slope of line segment (p2, p3): τ = (y3 - y2)/(x3 - x2)

If σ > τ, the orientation is clockwise (right turn)

Using above values of σ and τ, we can conclude that,
the orientation depends on sign of below expression:

(y2 - y1)*(x3 - x2) - (y3 - y2)*(x2 - x1)

Above expression is negative when σ < τ, i.e., counterclockwise

Below is the implementation of above idea.

C++Java

```
// A C++ program to find orientation of three points
#include <iostream>
using namespace std;

struct Point {
    int x, y;
};

// To find orientation of ordered triplet (p1, p2, p3).
// The function returns following values
// 0 --> p, q and r are collinear
// 1 --> Clockwise
// 2 --> Counterclockwise
int orientation(Point p1, Point p2, Point p3)
```

```
{
    // See 10th slides from following link for derivation
    // of the formula
    int val = (p2.y - p1.y) * (p3.x - p2.x)
              - (p2.x - p1.x) * (p3.y - p2.y);

    if (val == 0)
        return 0; // collinear

    return (val > 0) ? 1 : 2; // clock or counterclock wise
}

// Driver program to test above functions
int main()
{
    Point p1 = { 0, 0 }, p2 = { 4, 4 }, p3 = { 1, 2 };
    int o = orientation(p1, p2, p3);
    if (o == 0)
        cout << "Linear";
    else if (o == 1)
        cout << "Clockwise";
    else
        cout << "CounterClockwise";
    cout << endl;

    p1 = { 0, 0 }, p2 = { 4, 4 }, p3 = { 1, 1 };
    o = orientation(p1, p2, p3);
    if (o == 0)
        cout << "Linear";
    else if (o == 1)
        cout << "Clockwise";
    else
        cout << "CounterClockwise";
    cout << endl;

    p1 = { 1, 2 }, p2 = { 4, 4 }, p3 = { 0, 0 };
    o = orientation(p1, p2, p3);
    if (o == 0)
        cout << "Linear";
    else if (o == 1)
        cout << "Clockwise";
    else
        cout << "CounterClockwise";
    return 0;
}
```

Output

```
CounterClockwise
```

```
Linear
Clockwise
```

Time Complexity: O(1)
Auxiliary Space: O(1)

Approach#2: Using slope

This approach checks the orientation of 3 ordered points in the plane by calculating the slopes of the line segments formed by the points. If the slopes are equal, then the points are collinear. If the slope of the line segment formed by the first two points is less than the slope of the line segment formed by the last two points, then the orientation is counter-clockwise, otherwise it is clockwise.

Algorithm

1. Calculate slope of lines formed by (p1,p2) and (p2,p3)
2. If slopes are equal, then points are collinear
3. If slope of (p1,p2) < slope of (p2,p3), then points are in counter clockwise orientation
4. If slope of (p1,p2) > slope of (p2,p3), then points are in clockwise orientation

C++

```cpp
#include <iostream>
using namespace std;

string orientation(int p1[], int p2[], int p3[])
{
    // Calculate slopes
    int slope1 = (p2[1] - p1[1]) * (p3[0] - p2[0]);
    int slope2 = (p3[1] - p2[1]) * (p2[0] - p1[0]);
    // Check orientation
    if (slope1 == slope2) {
        return "Collinear";
    }
    else if (slope1 < slope2) {
        return "CounterClockWise";
    }
    else {
        return "ClockWise";
    }
}

int main()
{
    // Example usage
    int p1[] = { 0, 0 };
    int p2[] = { 4, 4 };
```

```
    int p3[] = { 1, 1 };
    cout << orientation(p1, p2, p3) << endl;

    int p4[] = { 0, 0 };
    int p5[] = { 4, 4 };
    int p6[] = { 1, 2 };
    cout << orientation(p4, p5, p6) << endl;

    return 0;
}
// This code is contributed by user_dtewbxkn77n
```

Output

```
Collinear
CounterClockWise
```

Time Complexity: O(1)

Space Complexity: O(1)

How to check if two given line segments intersect?

Given two line segments (p1, q1) and (p2, q2), find if the given line segments intersect with each other.

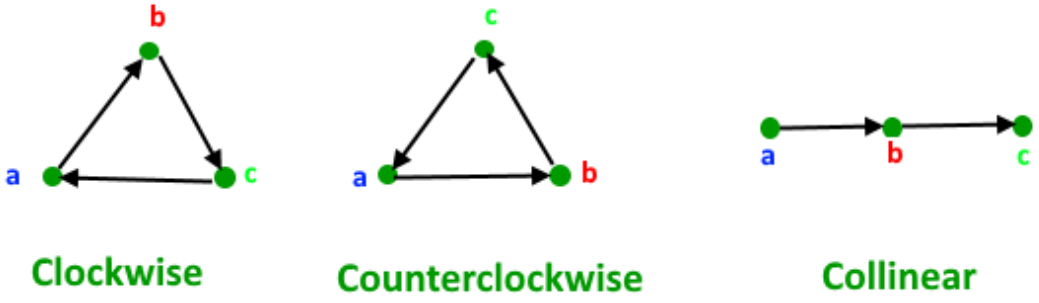
Before we discuss the solution, let us define notion of **orientation**. Orientation of an ordered triplet of points in the plane can be

--counterclockwise

--clockwise

--collinear

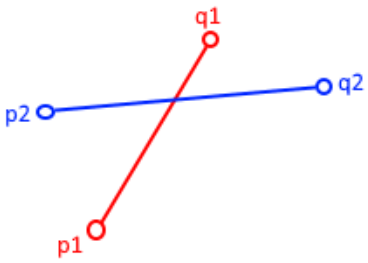
The following diagram shows different possible orientations of (a, b, c)



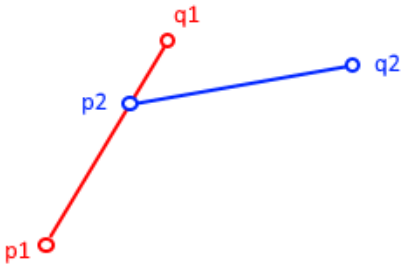
How is Orientation useful here?
Two segments (p1,q1) and (p2,q2) intersect if and only if one of the following two conditions is verified

1. ***General Case:***
- (p1, q1, p2) and (p1, q1, q2) have different orientations and
- (p2, q2, p1) and (p2, q2, q1) have different orientations.

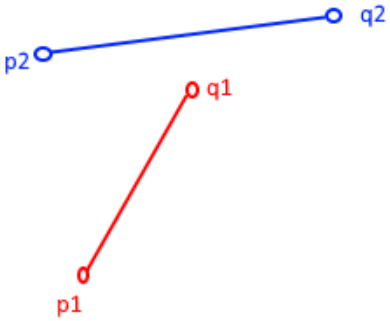
Examples:



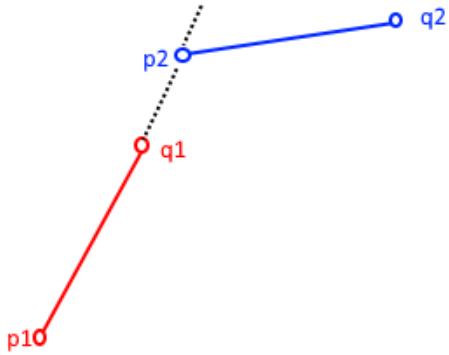
Example : Orientations of (p1,q1,p2) and (p1,q1,q2) are different. Orientations of (p2,q2,p1) and (p2,q2,q1) also differnet.



Example: Orientations of (p1,q1,p2) and (p1,q1,q2) are different. Orientations of (p2,q2,p1) and (p2,q2,q1) also different



Example: Orientations of (p1,q1,p2) and (p1,q1,q2) are different. Orientations of (p2,q2,p1) and (p2,q2,q1) are same

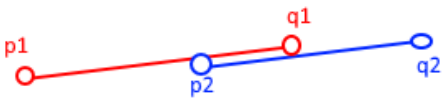


Example: Orientations of (p1,q1,p2) and (p1,q1,q2) are different. Orientations of (p2,q2,p1) and (p2,q2,q1) are same.

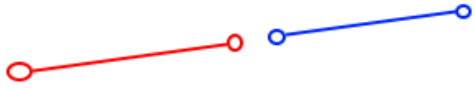
2. ***Special Case***

- (p1, q1, p2), (p1, q1, q2), (p2, q2, p1), and (p2, q2, q1) are all collinear and
- the x-projections of (p1, q1) and (p2, q2) intersect
- the y-projections of (p1, q1) and (p2, q2) intersect

Examples:



Example: All points are collinear. The x-projections pf (p1,q1) and (p2,q2) intersect. The y-projection of (p1,q1) and(p2,q2) also intersect



Example: All points are collinear .The x-projections of (p1,q1) and (p2,q2) not intersect. The y-projection of (p1,q1) and (p2,q2) do not intersect

Following is the implementation based on the above idea.

C++Java

```cpp
// A C++ program to check if two given line segments intersect
#include <iostream>
using namespace std;

struct Point
{
    int x;
    int y;
};

// Given three collinear points p, q, r, the function checks if
// point q lies on line segment 'pr'
bool onSegment(Point p, Point q, Point r)
{
    if (q.x <= max(p.x, r.x) && q.x >= min(p.x, r.x) &&
        q.y <= max(p.y, r.y) && q.y >= min(p.y, r.y))
       return true;

    return false;
}

// To find orientation of ordered triplet (p, q, r).
// The function returns following values
// 0 --> p, q and r are collinear
// 1 --> Clockwise
// 2 --> Counterclockwise
int orientation(Point p, Point q, Point r)
{
    // See https://www.cdn.geeksforgeeks.org/orientation-3-ordered-points/
    // for details of below formula.
    int val = (q.y - p.y) * (r.x - q.x) -
              (q.x - p.x) * (r.y - q.y);

    if (val == 0) return 0;  // collinear

    return (val > 0)? 1: 2; // clock or counterclock wise
}

// The main function that returns true if line segment 'p1q1'
// and 'p2q2' intersect.
bool doIntersect(Point p1, Point q1, Point p2, Point q2)
{
    // Find the four orientations needed for general and
    // special cases
    int o1 = orientation(p1, q1, p2);
    int o2 = orientation(p1, q1, q2);
    int o3 = orientation(p2, q2, p1);
    int o4 = orientation(p2, q2, q1);
```

```
    // General case
    if (o1 != o2 && o3 != o4)
        return true;

    // Special Cases
    // p1, q1 and p2 are collinear and p2 lies on segment p1q1
    if (o1 == 0 && onSegment(p1, p2, q1)) return true;

    // p1, q1 and q2 are collinear and q2 lies on segment p1q1
    if (o2 == 0 && onSegment(p1, q2, q1)) return true;

    // p2, q2 and p1 are collinear and p1 lies on segment p2q2
    if (o3 == 0 && onSegment(p2, p1, q2)) return true;

     // p2, q2 and q1 are collinear and q1 lies on segment p2q2
    if (o4 == 0 && onSegment(p2, q1, q2)) return true;

    return false; // Doesn't fall in any of the above cases
}

// Driver program to test above functions
int main()
{
    struct Point p1 = {1, 1}, q1 = {10, 1};
    struct Point p2 = {1, 2}, q2 = {10, 2};

    doIntersect(p1, q1, p2, q2)? cout << "Yes\n": cout << "No\n";

    p1 = {10, 0}, q1 = {0, 10};
    p2 = {0, 0}, q2 = {10, 10};
    doIntersect(p1, q1, p2, q2)? cout << "Yes\n": cout << "No\n";

    p1 = {-5, -5}, q1 = {0, 0};
    p2 = {1, 1}, q2 = {10, 10};
    doIntersect(p1, q1, p2, q2)? cout << "Yes\n": cout << "No\n";

    return 0;
}
```

Output:

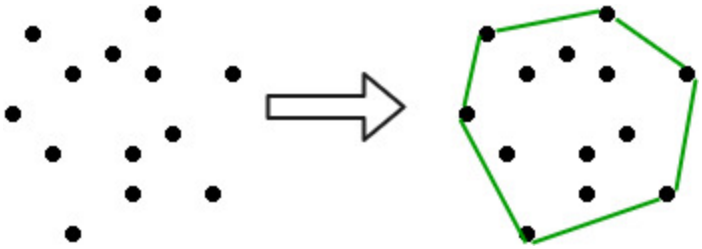
```
No
Yes
No
```

Time Complexity: *O(1)*

Space Complexity*: O(1)*

Convex Hull using Divide and Conquer Algorithm

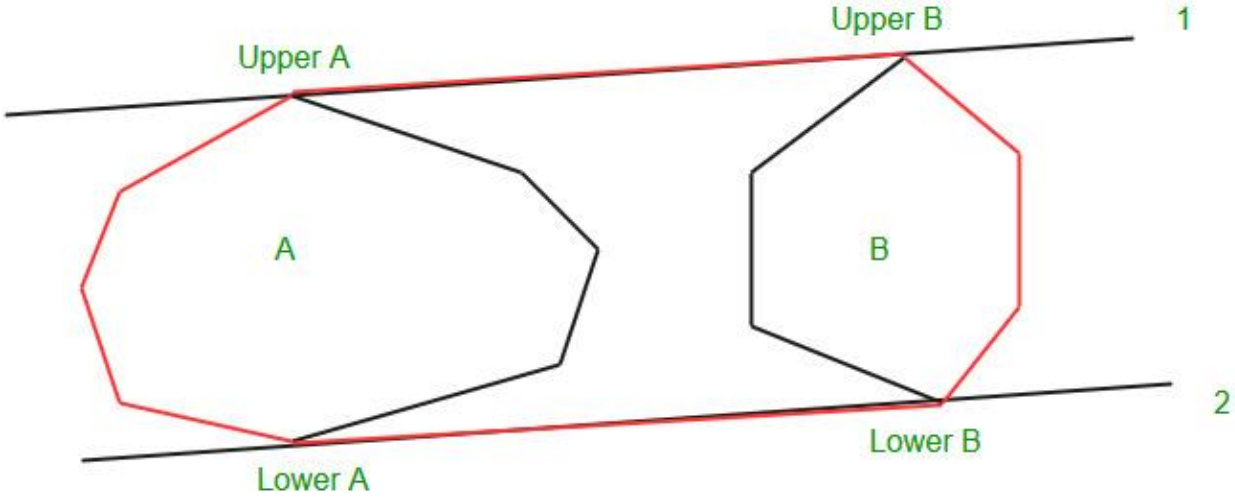
A convex hull is the smallest convex polygon containing all the given points.



Input is an array of points specified by their x and y coordinates. The output is the convex hull of this set of points. Examples:

```
Input : points[] = {(0, 0), (0, 4), (-4, 0), (5, 0),
                    (0, -6), (1, 0)};
Output : (-4, 0), (5, 0), (0, -6), (0, 4)
```

Pre-requisite: Tangents between two convex polygons **Algorithm:** Given the set of points for which we have to find the convex hull. Suppose we know the convex hull of the left half points and the right half points, then the problem now is to merge these two convex hulls and determine the convex hull for the complete set. This can be done by finding the upper and lower tangent to the right and left convex hulls. This is illustrated here Tangents between two convex polygons Let the left convex hull be a and the right convex hull be b. Then the lower and upper tangents are named as 1 and 2 respectively, as shown in the figure. Then the red outline shows the final convex hull.



Now the problem remains, how to find the convex hull for the left and right half. Now recursion comes into the picture, we divide the set of points until the number of points in the set is very small, say 5, and we can find the convex hull for these points by the brute

algorithm. The merging of these halves would result in the convex hull for the complete set of points. **Note:** We have used the brute algorithm to find the convex hull for a small number of points and it has a time complexity of O(n3). But some people suggest the following, the convex hull for 3 or fewer points is the complete set of points. This is correct but the problem comes when we try to merge a left convex hull of 2 points and right convex hull of 3 points, then the program gets trapped in an infinite loop in some special cases. So, to get rid of this problem I directly found the convex hull for 5 or fewer points by O(n3)algorithm, which is somewhat greater but does not affect the overall complexity of the algorithm.

CPPJava

```
// A divide and conquer program to find convex
// hull of a given set of points.
#include<bits/stdc++.h>
using namespace std;

// stores the centre of polygon (It is made
// global because it is used in compare function)
pair<int, int> mid;

// determines the quadrant of a point
// (used in compare())
int quad(pair<int, int> p)
{
    if (p.first >= 0 && p.second >= 0)
        return 1;
    if (p.first <= 0 && p.second >= 0)
        return 2;
    if (p.first <= 0 && p.second <= 0)
        return 3;
    return 4;
}

// Checks whether the line is crossing the polygon
int orientation(pair<int, int> a, pair<int, int> b,
                pair<int, int> c)
{
    int res = (b.second-a.second)*(c.first-b.first) -
              (c.second-b.second)*(b.first-a.first);

    if (res == 0)
        return 0;
    if (res > 0)
        return 1;
    return -1;
}
```

```
// compare function for sorting
bool compare(pair<int, int> p1, pair<int, int> q1)
{
    pair<int, int> p = make_pair(p1.first - mid.first,
                                 p1.second - mid.second);
    pair<int, int> q = make_pair(q1.first - mid.first,
                                 q1.second - mid.second);

    int one = quad(p);
    int two = quad(q);

    if (one != two)
        return (one < two);
    return (p.second*q.first < q.second*p.first);
}

// Finds upper tangent of two polygons 'a' and 'b'
// represented as two vectors.
vector<pair<int, int>> merger(vector<pair<int, int> > a,
                              vector<pair<int, int> > b)
{
    // n1 -> number of points in polygon a
    // n2 -> number of points in polygon b
    int n1 = a.size(), n2 = b.size();

    int ia = 0, ib = 0;
    for (int i=1; i<n1; i++)
        if (a[i].first > a[ia].first)
            ia = i;

    // ib -> leftmost point of b
    for (int i=1; i<n2; i++)
        if (b[i].first < b[ib].first)
            ib=i;

    // finding the upper tangent
    int inda = ia, indb = ib;
    bool done = 0;
    while (!done)
    {
        done = 1;
        while (orientation(b[indb], a[inda], a[(inda+1)%n1]) >=0)
            inda = (inda + 1) % n1;

        while (orientation(a[inda], b[indb], b[(n2+indb-1)%n2]) <=0)
        {
            indb = (n2+indb-1)%n2;
            done = 0;
```

```
        }
    }

    int uppera = inda, upperb = indb;
    inda = ia, indb=ib;
    done = 0;
    int g = 0;
    while (!done)//finding the lower tangent
    {
        done = 1;
        while (orientation(a[inda], b[indb], b[(indb+1)%n2])>=0)
            indb=(indb+1)%n2;

        while (orientation(b[indb], a[inda], a[(n1+inda-1)%n1])<=0)
        {
            inda=(n1+inda-1)%n1;
            done=0;
        }
    }

    int lowera = inda, lowerb = indb;
    vector<pair<int, int>> ret;

    //ret contains the convex hull after merging the two convex hulls
    //with the points sorted in anti-clockwise order
    int ind = uppera;
    ret.push_back(a[uppera]);
    while (ind != lowera)
    {
        ind = (ind+1)%n1;
        ret.push_back(a[ind]);
    }

    ind = lowerb;
    ret.push_back(b[lowerb]);
    while (ind != upperb)
    {
        ind = (ind+1)%n2;
        ret.push_back(b[ind]);
    }
    return ret;

}

// Brute force algorithm to find convex hull for a set
// of less than 6 points
vector<pair<int, int>> bruteHull(vector<pair<int, int>> a)
{
    // Take any pair of points from the set and check
```

```
    // whether it is the edge of the convex hull or not.
    // if all the remaining points are on the same side
    // of the line then the line is the edge of convex
    // hull otherwise not
    set<pair<int, int> >s;

    for (int i=0; i<a.size(); i++)
    {
        for (int j=i+1; j<a.size(); j++)
        {
            int x1 = a[i].first, x2 = a[j].first;
            int y1 = a[i].second, y2 = a[j].second;

            int a1 = y1-y2;
            int b1 = x2-x1;
            int c1 = x1*y2-y1*x2;
            int pos = 0, neg = 0;
            for (int k=0; k<a.size(); k++)
            {
                if (a1*a[k].first+b1*a[k].second+c1 <= 0)
                    neg++;
                if (a1*a[k].first+b1*a[k].second+c1 >= 0)
                    pos++;
            }
            if (pos == a.size() || neg == a.size())
            {
                s.insert(a[i]);
                s.insert(a[j]);
            }
        }
    }

    vector<pair<int, int>>ret;
    for (auto e:s)
        ret.push_back(e);

    // Sorting the points in the anti-clockwise order
    mid = {0, 0};
    int n = ret.size();
    for (int i=0; i<n; i++)
    {
        mid.first += ret[i].first;
        mid.second += ret[i].second;
        ret[i].first *= n;
        ret[i].second *= n;
    }
    sort(ret.begin(), ret.end(), compare);
    for (int i=0; i<n; i++)
        ret[i] = make_pair(ret[i].first/n, ret[i].second/n);
```

```
    return ret;
}

// Returns the convex hull for the given set of points
vector<pair<int, int>> divide(vector<pair<int, int>> a)
{
    // If the number of points is less than 6 then the
    // function uses the brute algorithm to find the
    // convex hull
    if (a.size() <= 5)
        return bruteHull(a);

    // left contains the left half points
    // right contains the right half points
    vector<pair<int, int>>left, right;
    for (int i=0; i<a.size()/2; i++)
        left.push_back(a[i]);
    for (int i=a.size()/2; i<a.size(); i++)
        right.push_back(a[i]);

    // convex hull for the left and right sets
    vector<pair<int, int>>left_hull = divide(left);
    vector<pair<int, int>>right_hull = divide(right);

    // merging the convex hulls
    return merger(left_hull, right_hull);
}

// Driver code
int main()
{
    vector<pair<int, int> > a;
    a.push_back(make_pair(0, 0));
    a.push_back(make_pair(1, -4));
    a.push_back(make_pair(-1, -5));
    a.push_back(make_pair(-5, -3));
    a.push_back(make_pair(-3, -1));
    a.push_back(make_pair(-1, -3));
    a.push_back(make_pair(-2, -2));
    a.push_back(make_pair(-1, -1));
    a.push_back(make_pair(-2, -1));
    a.push_back(make_pair(-1, 1));

    int n = a.size();

    // sorting the set of points according
    // to the x-coordinate
    sort(a.begin(), a.end());
```

```
    vector<pair<int, int> >ans = divide(a);

    cout << "convex hull:\n";
    for (auto e:ans)
    cout << e.first << " "
            << e.second << endl;

    return 0;
}
```

Output

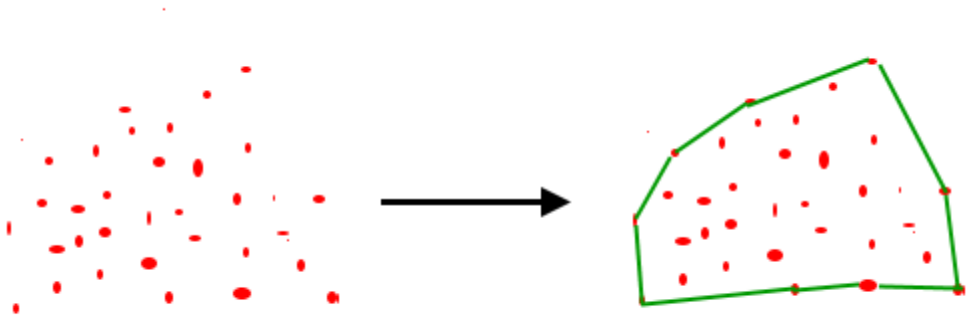
```
convex hull:
-5 -3
-1 -5
1 -4
0 0
-1 1
```

Time Complexity: The merging of the left and the right convex hulls take O(n) time and as we are dividing the points into two equal parts, so the time complexity of the above algorithm is O(n * log n).

Auxiliary Space: O(n)

Convex Hull using Jarvis' Algorithm or Wrapping

Given a set of points in the plane. the convex hull of the set is the smallest convex polygon that contains all the points of it.



We strongly recommend to see the following post first.
How to check if two given line segments intersect?
The idea of Jarvis's Algorithm is simple, we start from the leftmost point (or point with minimum x coordinate value) and we keep wrapping points in counterclockwise direction.

The big question is, given a point p as current point, how to find the next point in output?

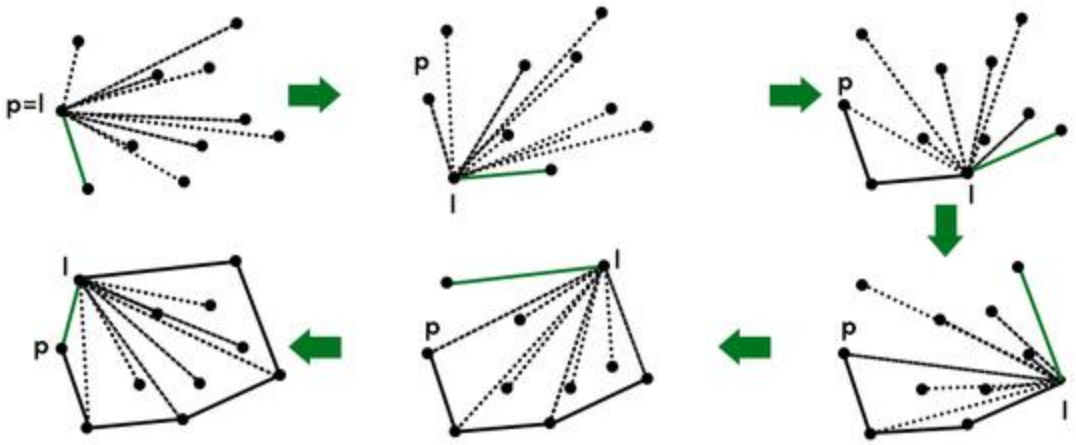
The idea is to use orientation() here. Next point is selected as the point that beats all other points at counterclockwise orientation, i.e., next point is q if for any other point r, we have "orientation(p, q, r) = counterclockwise".

Algorithm:
Step 1) Initialize p as leftmost point.
Step 2) Do following while we don't come back to the first (or leftmost) point.
2.1) The next point q is the point, such that the triplet (p, q, r) is counter clockwise for any other point r.

To find this, we simply initialize q as next point, then we traverse through all points.

For any point i, if i is more counter clockwise, i.e., orientation(p, i, q) is counter clockwise, then we update q as i.

Our final value of q is going to be the most counter clockwise point.
2.2) next[p] = q (Store q as next of p in the output convex hull).
2.3) p = q (Set p as q for next iteration).



The execution of Jarvis March

Below is the implementation of above algorithm.

C++Java

```
// A C++ program to find convex hull of a set of points. Refer
// https://www.cdn.geeksforgeeks.org/orientation-3-ordered-points/
// for explanation of orientation()
#include <bits/stdc++.h>
using namespace std;

struct Point
{
    int x, y;
};

// To find orientation of ordered triplet (p, q, r).
// The function returns following values
// 0 --> p, q and r are collinear
// 1 --> Clockwise
// 2 --> Counterclockwise
int orientation(Point p, Point q, Point r)
{
    int val = (q.y - p.y) * (r.x - q.x) -
              (q.x - p.x) * (r.y - q.y);
```

```
    if (val == 0) return 0;  // collinear
    return (val > 0)? 1: 2; // clock or counterclock wise
}

// Prints convex hull of a set of n points.
void convexHull(Point points[], int n)
{
    // There must be at least 3 points
    if (n < 3) return;

    // Initialize Result
    vector<Point> hull;

    // Find the leftmost point
    int l = 0;
    for (int i = 1; i < n; i++)
        if (points[i].x < points[l].x)
            l = i;

    // Start from leftmost point, keep moving counterclockwise
    // until reach the start point again.  This loop runs O(h)
    // times where h is number of points in result or output.
    int p = l, q;
    do
    {
        // Add current point to result
        hull.push_back(points[p]);

        // Search for a point 'q' such that orientation(p, q,
        // x) is counterclockwise for all points 'x'. The idea
        // is to keep track of last visited most counterclock-
        // wise point in q. If any point 'i' is more counterclock-
        // wise than q, then update q.
        q = (p+1)%n;
        for (int i = 0; i < n; i++)
        {
           // If i is more counterclockwise than current q, then
           // update q
           if (orientation(points[p], points[i], points[q]) == 2)
               q = i;
        }

        // Now q is the most counterclockwise with respect to p
        // Set p as q for next iteration, so that q is added to
        // result 'hull'
        p = q;

    } while (p != l);  // While we don't come to first point
```

```
    // Print Result
    for (int i = 0; i < hull.size(); i++)
        cout << "(" << hull[i].x << ", "
             << hull[i].y << ")\n";
}

// Driver program to test above functions
int main()
{
    Point points[] = {{0, 3}, {2, 2}, {1, 1}, {2, 1},
                      {3, 0}, {0, 0}, {3, 3}};
    int n = sizeof(points)/sizeof(points[0]);
    convexHull(points, n);
    return 0;
}
```

Output: The output is points of the convex hull.

```
(0, 3)
(0, 0)
(3, 0)
(3, 3)
```

Time Complexity: O(m * n), where n is number of input points and m is number of output or hull points (m <= n). For every point on the hull we examine all the other points to determine the next point.

Worst case, Time complexity: $O(n^2)$. The worst case occurs when all the points are on the hull (m = n).

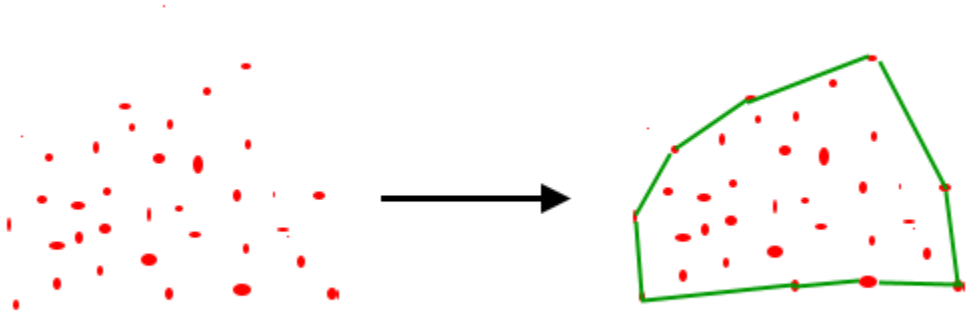
Auxiliary Space: O(n), since n extra space has been taken.

Set 2- Convex Hull (Graham Scan)

Note : The above code may produce different results for different order of inputs, when there are collinear points in the convex hull. For example, it produces output as (0, 3) (0, 0) (3, 0) (3, 3) for input (0, 3), (0, 0), (0, 1), (3, 0), (3, 3) and output as (0, 3) (0, 1) (0, 0) (3, 0) (3, 3) for input as (0, 3), (0, 1), (0, 0), (3, 0), (3, 3). We generally need the farthest next point in case of collinear, we can get the desired result in case of collinear points by adding one more if condition. Please refer this modified code.

Graham Scan Algorithm for Convex Hull

Given a set of points in the plane. the convex hull of the set is the smallest convex polygon that contains all the points of it.



We strongly recommend to see the following post first.
How to check if two given line segments intersect?
We have discussed Jarvis's Algorithm for Convex Hull. The worst case time complexity of Jarvis's Algorithm is O(n^2). Using Graham's scan algorithm, we can find Convex Hull in O(nLogn) time. Following is Graham's algorithm
Let points[0..n-1] be the input array.
1) Find the bottom-most point by comparing y coordinate of all points. If there are two points with the same y value, then the point with smaller x coordinate value is considered. Let the bottom-most point be P0. Put P0 at first position in output hull.
2) Consider the remaining n-1 points and sort them by polar angle in counterclockwise order around points[0]. If the polar angle of two points is the same, then put the nearest point first.
3 After sorting, check if two or more points have the same angle. If two more points have the same angle, then remove all same angle points except the point farthest from P0. Let the size of the new array be m.
4) If m is less than 3, return (Convex Hull not possible)
5) Create an empty stack 'S' and push points[0], points[1] and points[2] to S.
6) Process remaining m-3 points one by one. Do following for every point 'points[i]'
4.1) Keep removing points from stack while orientation of following 3 points is not counterclockwise (or they don't make a left turn).
a) Point next to top in stack

b) Point at the top of stack

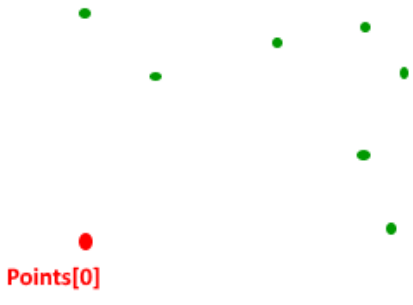
c) points[i]

4.2) Push points[i] to S

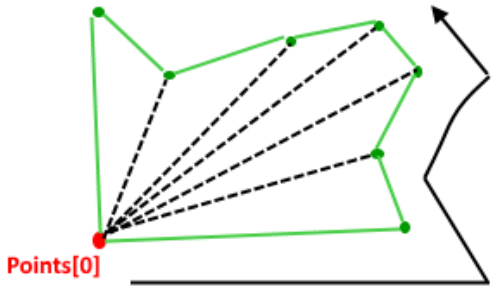
5) Print contents of S

The above algorithm can be divided into two phases.

Phase 1 (Sort points): We first find the bottom-most point. The idea is to pre-process points be sorting them with respect to the bottom-most point. Once the points are sorted, they form a simple closed path (See the following diagram).



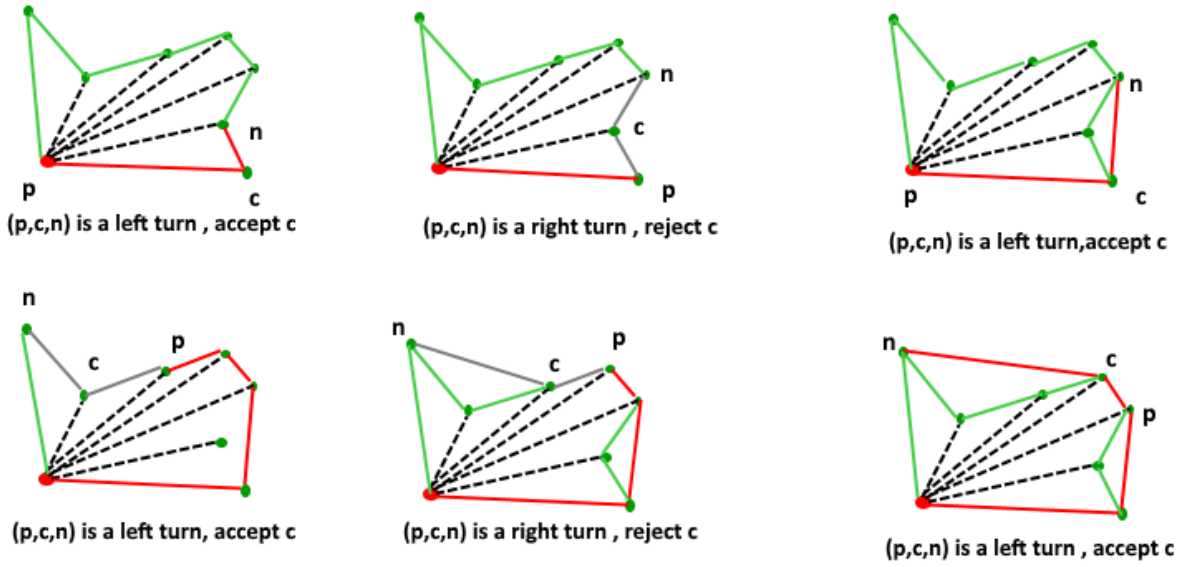
Given points



Points consider according to increasing angle with respect to points[0] from a simple closed path

What should be the sorting criteria? computation of actual angles would be inefficient since trigonometric functions are not simple to evaluate. The idea is to use the orientation to compare angles without actually computing them (See the compare() function below)

Phase 2 (Accept or Reject Points): Once we have the closed path, the next step is to traverse the path and remove concave points on this path. How to decide which point to remove and which to keep? Again, orientation helps here. The first two points in sorted array are always part of Convex Hull. For remaining points, we keep track of recent three points, and find the angle formed by them. Let the three points be prev(p), curr(c) and next(n). If orientation of these points (considering them in same order) is not counterclockwise, we discard c, otherwise we keep it. Following diagram shows step by step process of this phase.



In the above algorithm and below code , a stack of points is used to store convex hull points. With reference to the code ,p is next-to-top in stack, c is top of stack and n is point[i]

Graham Scan Algorithm in C++

C++ implementation

C++

```
//C++ program for graham scan algorithm
#include <iostream>
#include <bits/stdc++.h>
using namespace std;

struct Point
{
        int x, y;
};

// A global point needed for sorting points with reference
// to the first point Used in compare function of qsort()
Point p0;

// A utility function to find next to top in a stack
Point nextToTop(stack<Point> &S)
{
        Point p = S.top();
```

```
        S.pop();
        Point res = S.top();
        S.push(p);
        return res;
}

// A utility function to swap two points
void swap(Point &p1, Point &p2)
{
        Point temp = p1;
        p1 = p2;
        p2 = temp;
}

// A utility function to return square of distance
// between p1 and p2
int distSq(Point p1, Point p2)
{
        return (p1.x - p2.x)*(p1.x - p2.x) +
               (p1.y - p2.y)*(p1.y - p2.y);
}

// To find orientation of ordered triplet (p, q, r).
// The function returns following values
// 0 --> p, q and r are collinear
// 1 --> Clockwise
// 2 --> Counterclockwise
int orientation(Point p, Point q, Point r)
{
        int val = (q.y - p.y) * (r.x - q.x) -
                  (q.x - p.x) * (r.y - q.y);

        if (val == 0) return 0; // collinear
        return (val > 0)? 1: 2; // clock or counterclock wise
}

// A function used by library function qsort() to sort an array of
// points with respect to the first point
int compare(const void *vp1, const void *vp2)
{
Point *p1 = (Point *)vp1;
Point *p2 = (Point *)vp2;

// Find orientation
int o = orientation(p0, *p1, *p2);
if (o == 0)
        return (distSq(p0, *p2) >= distSq(p0, *p1))? -1 : 1;

return (o == 2)? -1: 1;
```

```
}

// Prints convex hull of a set of n points.
void convexHull(Point points[], int n)
{
// Find the bottommost point
int ymin = points[0].y, min = 0;
for (int i = 1; i < n; i++)
{
        int y = points[i].y;

        // Pick the bottom-most or chose the left
        // most point in case of tie
        if ((y < ymin) || (ymin == y &&
                points[i].x < points[min].x))
                ymin = points[i].y, min = i;
}

// Place the bottom-most point at first position
swap(points[0], points[min]);

// Sort n-1 points with respect to the first point.
// A point p1 comes before p2 in sorted output if p2
// has larger polar angle (in counterclockwise
// direction) than p1
p0 = points[0];
qsort(&points[1], n-1, sizeof(Point), compare);

// If two or more points make same angle with p0,
// Remove all but the one that is farthest from p0
// Remember that, in above sorting, our criteria was
// to keep the farthest point at the end when more than
// one points have same angle.
int m = 1; // Initialize size of modified array
for (int i=1; i<n; i++)
{
        // Keep removing i while angle of i and i+1 is same
        // with respect to p0
        while (i < n-1 && orientation(p0, points[i],

        points[i+1]) == 0)
                i++;


        points[m] = points[i];
        m++; // Update size of modified array
}

// If modified array of points has less than 3 points,
```

```
// convex hull is not possible
if (m < 3) return;

// Create an empty stack and push first three points
// to it.
stack<Point> S;
S.push(points[0]);
S.push(points[1]);
S.push(points[2]);

// Process remaining n-3 points
for (int i = 3; i < m; i++)
{
        // Keep removing top while the angle formed by
        // points next-to-top, top, and points[i] makes
        // a non-left turn
        while (S.size()>1 && orientation(nextToTop(S), S.top(), points[i])
!= 2)
                S.pop();
        S.push(points[i]);
}

// Now stack has the output points, print contents of stack
while (!S.empty())
{
        Point p = S.top();
        cout << "(" << p.x << ", " << p.y <<")" << endl;
        S.pop();
}
}

// Driver program to test above functions
int main()
{
        Point points[] = {{0, 3}, {1, 1}, {2, 2}, {4, 4},
                                        {0, 0}, {1, 2}, {3, 1}, {3, 3}};
        int n = sizeof(points)/sizeof(points[0]);
        convexHull(points, n);
        return 0;
}
```

Output

```
(0, 3)
(4, 4)
(3, 1)
(0, 0)
```

Mark as Read
Report An Issue

Closest Pair of Points

Input :arr[]= {(10,20),(3,5),(5,6),(11,20)}
Output :1.00

Input : arr[]= {(2,3),(10,5),(3,4)}
Output : 1.414

We are given an array of n points in the plane, and the problem is to find out the closest pair of points in the array. This problem arises in a number of applications. For example, in air-traffic control, you may want to monitor planes that come too close together, since this may indicate a possible collision. Recall the following formula for distance between two points p and q.

Naive Solution: Brute Force

C++

```
// Naive Code

#include <bits/stdc++.h>
using namespace std;

struct point {
   int x, y;
};

//find distance between p1 and p2
double dist(point p1, point p2)
{
   return sqrt((p1.x - p2.x)*(p1.x - p2.x) + (p1.y - p2.y)*(p1.y - p2.y));
}

double minDist(point p[], int n)
{
    double res = DBL_MAX;
    for(int i = 0; i < n-1; i++)
    {
        for(int j = i+1; j < n; j++)
        {
            res = min(res,dist(p[i],p[j]));
        }
    }
    return res;
}
```

```
int main() {
   point P[] ={{2, 3}, {12, 30}, {40, 50}, {5, 1}, {12, 10}, {3, 4}};
   int n = 6;
   cout<< "The minimum distance is " << minDist(P, n);
}
```

Output

```
The minimum distance is 1.41421
```

Time Complexity: $\theta(n^2)$

Auxiliary Space: $\theta(1)$

Efficient Solution :

1) We sort all points according to x coordinates.
2) Divide all points in two halves.
3) Recursively find the smallest distances in both subarrays.
4) Take the minimum of two smallest distances. Let the minimum be d.
5) Create an array strip[] that stores all points which are at most d distance away from the middle line dividing the two sets.
6) Find the smallest distance in strip[].
7) Return the minimum of d and the smallest distance calculated in above step 6.

Working of Closest Pair Algorithm

The **closest pair algorithm** is a well-known algorithm for solving the closest pair of points problem, which involves finding the pair of points with the smallest distance between them in a given set of points. The closest pair algorithm has a time complexity of O(n log n), making it much faster than the brute-force algorithm (which has a time complexity of O(n^2)) for large input sizes.

The closest pair algorithm works as follows:

1)Sort the set of points by their x-coordinates. This can be done using any sorting algorithm with a time complexity of O(n log n), such as merge sort or quick sort.

2)Divide the set of points into two equal-sized subsets by a vertical line that passes through the midpoint of the sorted list of points. The left and right subsets will each contain roughly half of the points.

3)Recursively compute the closest pair of points in each of the two subsets. This can be done by applying the same algorithm to each subset. The recursion stops when the subsets have only one or two points.

4)Let d be the minimum distance between any two points in the left subset and the right subset, respectively. The closest pair of points must either be a pair of points in one of the two subsets whose distance is less than d, or a pair of points consisting of one point in the left subset and one point in the right subset whose distance is less than d.

5)Consider a band of width 2d around the vertical line passing through the midpoint of the sorted list of points. Any pair of points whose distance is less than d must be within this band. Therefore, we only need to consider pairs of points that are within this band.

6)Sort the points in the band by their y-coordinates. This can be done using any sorting algorithm with a time complexity of O(n log n), such as merge sort or quick sort.

7)For each point in the band, consider the next 7 points (or fewer, if there are fewer than 7 remaining points). The distance between each of these pairs of points is computed and the minimum distance is stored.

8)The minimum of the distances computed in step 7 is the minimum distance between any two points in the band, and is also the minimum distance between any two points in the original set of points.

9)Return the minimum distance computed in step 8.

The closest pair algorithm can be implemented efficiently using a combination of divide and conquer and sorting techniques. This algorithm has a time complexity of O(n log n)

Sweep Line Algorithm for Any Two Intersection

Given n line segments (p1, q1), (p2, q2), … (pn, qn), find if the given line segments intersect with each other or not.

Example:

Input: {{1, 5}, {4, 5}}, {{2, 5}, {10, 1}},{{3, 2}, {10, 3}},{{6, 4}, {9, 4}},{{7, 1}, {8, 1}}
Output: 2
Explanation: Lines {{1, 5}, {4, 5}}, {{2, 5}, {10, 1}} are intersecting, as well as {{2, 5}, {10, 1}},{{3, 2}, {10, 3}}.

Naive Algorithm :

A naive solution to solve this problem is to check every pair of lines and check if the pair intersects or not. Therefore, this approach takes $O(n^2)$.

Sweep Line Algorithm:

We can solve this problem in **O(nLogn)** time using Sweep Line Algorithm. The algorithm first sorts the end points along the x axis from left to right, then it passes a vertical line through all points from left to right and checks for intersections. Following are detailed steps.
1) Let there be n given lines. There must be 2n end points to represent the n lines. Sort all points according to x coordinates. While sorting maintain a flag to indicate whether this point is left point of its line or right point.
2) Start from the leftmost point. Do following for every point
…..**a)** If the current point is a left point of its line segment, check for intersection of its line segment with the segments just above and below it. And add its line to *active* line segments (line segments for which left end point is seen, but right end point is not seen yet). Note that we consider only those neighbors which are still active.
….**b)** If the current point is a right point, remove its line segment from active list and check whether its two active neighbors (points just above and below) intersect with each other.
The step 2 is like passing a vertical line from all points starting from the leftmost point to the rightmost point. That is why this algorithm is called Sweep Line Algorithm. The Sweep Line technique is useful in many other geometric algorithms like calculating the 2D Voronoi diagram

Sweep Line Implementation

C++ Implementation :

C++14

```
// Implementation of Sweep Line Algorithm
#include <bits/stdc++.h>
using namespace std;

// A point in 2D plane
struct Point
{
	int x, y;
};

// A line segment with left as Point
// with smaller x value and right with
// larger x value.
struct Segment
{
	Point left, right;
};


// An event for sweep line algorithm
// An event has a point, the position
// of point (whether left or right) and
// index of point in the original input
// array of segments.
struct Event {
	int x, y;
	bool isLeft;
	int index;

	Event(int x, int y, bool l, int i) : x(x), y(y), isLeft(l),
index(i) {}

	// This is for maintaining the order in set.
	bool operator<(const Event& e) const {
		return y < e.y;
	}
};


// Given three colinear points p, q, r, the function checks if
// point q lies on line segment 'pr'
bool onSegment(Point p, Point q, Point r)
{
	if (q.x <= max(p.x, r.x) && q.x >= min(p.x, r.x) &&
		q.y <= max(p.y, r.y) && q.y >= min(p.y, r.y))
	return true;

	return false;
```

```
}

// To find orientation of ordered triplet (p, q, r).
// The function returns following values
// 0 --> p, q and r are colinear
// 1 --> Clockwise
// 2 --> Counterclockwise
int orientation(Point p, Point q, Point r)
{
	// See https://www.geeksforgeeks.org/orientation-3-ordered-points/
	// for details of below formula.
	int val = (q.y - p.y) * (r.x - q.x) -
			(q.x - p.x) * (r.y - q.y);

	if (val == 0) return 0; // colinear

	return (val > 0)? 1: 2; // clock or counterclock wise
}

// The main function that returns true if line segment 'p1q1'
// and 'p2q2' intersect.
bool doIntersect(Segment s1, Segment s2)
{
	Point p1 = s1.left, q1 = s1.right, p2 = s2.left, q2 = s2.right;

	// Find the four orientations needed for general and
	// special cases
	int o1 = orientation(p1, q1, p2);
	int o2 = orientation(p1, q1, q2);
	int o3 = orientation(p2, q2, p1);
	int o4 = orientation(p2, q2, q1);

	// General case
	if (o1 != o2 && o3 != o4)
		return true;

	// Special Cases
	// p1, q1 and p2 are colinear and p2 lies on segment p1q1
	if (o1 == 0 && onSegment(p1, p2, q1)) return true;

	// p1, q1 and q2 are colinear and q2 lies on segment p1q1
	if (o2 == 0 && onSegment(p1, q2, q1)) return true;

	// p2, q2 and p1 are colinear and p1 lies on segment p2q2
	if (o3 == 0 && onSegment(p2, p1, q2)) return true;

	// p2, q2 and q1 are colinear and q1 lies on segment p2q2
	if (o4 == 0 && onSegment(p2, q1, q2)) return true;
```

```
        return false; // Doesn't fall in any of the above cases
}


// Find predecessor of iterator in s.
auto pred(set<Event> &s, set<Event>::iterator it) {
        return it == s.begin() ? s.end() : --it;
}

// Find successor of iterator in s.
auto succ(set<Event> &s, set<Event>::iterator it) {
        return ++it;
}

// Returns true if any two lines intersect.
bool sweepline(Segment arr[], int n)
{
        // Pushing all points to a vector of events
        vector<Event> e;
        for (int i = 0; i < n; ++i) {
                e.push_back(Event(arr[i].left.x, arr[i].left.y, true, i));
                e.push_back(Event(arr[i].right.x, arr[i].right.y, false,
i));
        }

        // Sorting all events according to x coordinate.
        sort(e.begin(), e.end(), [](Event &e1, Event &e2) {return e1.x <
e2.x;});

        // For storing active segments.
        set<Event> s;

        // Traversing through sorted points
        for (int i=0; i<2*n; i++)
        {
                Event curr = e[i];
                int index = curr.index;

                // If current point is left of its segment
                if (curr.isLeft)
                {
                        // Get above and below points
                        auto next = s.lower_bound(curr);
                        auto prev = pred(s, next);

                        // Check if current point intersects with
                        // any of its adjacent
                        if (next != s.end() && doIntersect(arr[next->index],
arr[index]))
```

```
                        return true;
                        if (prev != s.end() && doIntersect(arr[prev->index],
arr[index]))
                        return true;

                        // Insert current point (or event)
                        s.insert(curr);
                }

                // If current point is right of its segment
                else
                {
                        // Find the iterator
                        auto it = s.find(curr);

                        // Find above and below points
                        auto next = succ(s, it);
                        auto prev = pred(s, it);

                        // If above and below point intersect
                        if (next != s.end() && prev != s.end() &&
doIntersect(arr[prev->index], arr[next->index]))
                                return true;

                        // Return current point
                        s.erase(curr);
                }
        }
        return false;
}

// Driver code
int main() {
        Segment arr[] = { {{0, 0}, {4, 4}}, {{1, 3}, {3, 1}}};
        int n = sizeof(arr)/sizeof(arr[0]);
        cout << sweepline(arr, n);
        return 0;
}
```

Output
1

Time Complexity:θ(nLogn)

Auxiliary Space: θ(N)

Recursion

Generate all the balanced string of parentheses for given length

Write a function to generate all possible n pairs of balanced parentheses.

Examples:

```
Input: n=1
Output: {}
Explanation: This the only sequence of balanced
parenthesis formed using 1 pair of balanced parenthesis.

Input : n=2
Output:
{}{}
{{}}
Explanation: This the only two sequences of balanced
parenthesis formed using 2 pair of balanced parenthesis.
```

Approach 1: To form all the sequences of balanced bracket subsequences with n pairs. So there are n opening brackets and n closing brackets.
So the subsequence will be of length 2*n. There is a simple idea, the i'th character can be '{' if and only if the count of '{' till i'th is less than n and i'th character can be '}' if and only if the count of '{' is greater than the count of '}' till index i. If these two cases are followed then the resulting subsequence will always be balanced.
So form the recursive function using the above two cases.

Algorithm:

- Create a recursive function that accepts a string (s), count of opening brackets (o) and count of closing brackets (c) and the value of n.
- if the value of opening bracket and closing bracket is equal to n then print the string and return.
- If the count of opening bracket is greater than count of closing bracket then call the function recursively with the following parameters String *s + "}"*, count of opening bracket *o*, count of closing bracket *c + 1*, and n.
- If the count of opening bracket is less than n then call the function recursively with the following parameters String *s + "{"*, count of opening bracket *o + 1*, count of closing bracket *c*, and n.

Implementation:

C++Java

```
#include <iostream>

#define MAX_SIZE 100

void _printParenthesis(int pos, int n, int open, int close);

// Wrapper over _printParenthesis()
void printParenthesis(int n)
{
    if (n > 0)
        _printParenthesis(0, n, 0, 0);
    return;
}

void _printParenthesis(int pos, int n, int open, int close)
{
    static char str[MAX_SIZE];

    if (close == n) {
        std::cout << str << std::endl;
        return;
    }
    else {
        if (open > close) {
            str[pos] = '}';
            _printParenthesis(pos + 1, n, open, close + 1);
        }

        if (open < n) {
            str[pos] = '{';
            _printParenthesis(pos + 1, n, open + 1, close);
        }
    }
}

// Driver program to test above functions
int main()
{
    int n = 3;
    printParenthesis(n);
    return 0;
}
```

Output

```
{{}}
{}{}
```

Time complexity: O(2^n), as there are 2^n possible combinations of '(' and ')' parentheses.
Auxiliary space: O(n), as n characters are stored in the str array.

Let's see the implementation of the same algorithm in a slightly different, simple and concise way :

C++Java

```
// c++ program to print all the combinations of balanced
// parenthesis.
#include <bits/stdc++.h>
using namespace std;
// function which generates all possible n pairs of balanced
// parentheses. open : count of the number of open
// parentheses used in generating the current string s. close
// : count of the number of closed parentheses used in
// generating the current string s. s : currently generated
// string/ ans : a vector of strings to store all the valid
// parentheses.
void generateParenthesis(int n, int open, int close,
                         string s, vector<string>& ans)
{
    // if the count of both open and close parentheses
    // reaches n, it means we have generated a valid
    // parentheses. So, we add the currently generated string
    // s to the final ans and return.
    if (open == n && close == n) {
        ans.push_back(s);
        return;
    }
    // At any index i in the generation of the string s, we
    // can put an open parentheses only if its count until
    // that time is less than n.
    if (open < n) {
        generateParenthesis(n, open + 1, close, s + "{",
                            ans);
    }
    // At any index i in the generation of the string s, we
    // can put a closed parentheses only if its count until
    // that time is less than the count of open parentheses.
    if (close < open) {
        generateParenthesis(n, open, close + 1, s + "}",
                            ans);
    }
}
int main()
{
```

```
    int n = 3;
    // vector ans is created to store all the possible valid
    // combinations of the parentheses.
    vector<string> ans;
    // initially we are passing the counts of open and close
    // as 0, and the string s as an empty string.
    generateParenthesis(n, 0, 0, "", ans);
    // Now, here we print out all the combinations.
    for (auto s : ans) {
        cout << s << endl;
    }
    return 0;
}
```

Output

```
{{{}}}
{{}{}}
{{}}{}
{}{{}}
{}{}{}
```

Complexity Analysis:

- **Time Complexity:** O(2^n).
  For every index there can be two options '{' or '}'. So it can be said that the upper bound of time complexity is O(2^n) or it has exponential time complexity.
- **Auxiliary Space:** O(n).
  The space complexity can be reduced to O(n) if global variable or static variable is used to store the output string.

Generate all passwords from given character set.

Given a set of characters generate all possible password from them. this means we should generate all possible permutation of words using the given characters with repetition and upto a given length.

Input:

```
arr[]={a,b}
len=2
```

Output:

```
a b aa ab ba bb
```

Explanation:

For every character we have two choice either use the current character or not. In this way we can generate all the passwords upto given length,

C++Java

```cpp
 // C++ program to generate all passwords for given
  // characters
#include <bits/stdc++.h>
    using namespace std;

// int cnt;

// Recursive helper function, adds/removes characters
// until len is reached
void generate(char* arr, int i, string s, int len)
{
    // base case
    if (i == 0) // when len has been reached
    {
        // print it out
        cout << s << "\n";
        // cnt++;
        return;
    }

    // iterate through the array
    for (int j = 0; j < len; j++) {

        // Create new string with next character
        // Call generate again until string has
        // reached its len
        string appended = s + arr[j];
        generate(arr, i - 1, appended, len);
    }

    return;
}

// function to generate all possible passwords
void crack(char* arr, int len)
{
    // call for all required lengths
    for (int i = 1; i <= len; i++) {
        generate(arr, i, "", len);
    }
}
```

```
// driver function
int main()
{
    char arr[] = { 'a', 'b', 'c' };
    int len = sizeof(arr) / sizeof(arr[0]);
    crack(arr, 2);

    return 0;
}
```

Output:

```
a
b
aa
ab
ba
bb
```

Distinct way to write N as sum of positive integers

For a given number n > 0, find the number of different ways in which n can be written as a sum of two or more positive integers.

Examples:

```
Input : n = 5
Output : 6
Explanation : All possible six ways are :
4 + 1
3 + 2
3 + 1 + 1
2 + 2 + 1
2 + 1 + 1 + 1
1 + 1 + 1 + 1 + 1

Input : 4
Output : 4
Explanation : All possible four ways are :
3 + 1
2 + 2
2 + 1 + 1
1 + 1 + 1 + 1
```

This problem can be solved in a similar fashion as coin change problem, the difference is only that in this case we should iterate for 1 to n-1 instead of particular values of coin as in coin-change problem.

C++JavaPython3C#PHPJavascript

```cpp
// Program to find the number of ways, n can be
// written as sum of two or more positive integers.
#include <bits/stdc++.h>
using namespace std;

// Returns number of ways to write n as sum of
// two or more positive integers
int countWays(int n)
{
    // table[i] will be storing the number of
    // solutions for value i. We need n+1 rows
    // as the table is constructed in bottom up
    // manner using the base case (n = 0)
    int table[n+1];

    // Initialize all table values as 0
    memset(table, 0, sizeof(table));

    // Base case (If given value is 0)
    table[0] = 1;

    // Pick all integer one by one and update the
    // table[] values after the index greater
    // than or equal to n
    for (int i=1; i<n; i++)
        for (int j=i; j<=n; j++)
            table[j] += table[j-i];

    return table[n];
}

// Driver program
int main()
{
    int n = 7;
    cout << countWays(n);
    return 0;
}
```

Output

```
14
```

Time Complexity: $O(n^2)$
Auxiliary Space: O(n)

This article is contributed by **Shivam Pradhan (anuj_charm)**. If you like GeeksforGeeks and would like to contribute, you can also write an article using contribute.geeksforgeeks.org or mail your article to review-team@geeksforgeeks.org. See your article appearing on the GeeksforGeeks main page and help other Geeks.
Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

Number of Paths in a Grid

The problem is to count all the possible paths from the top left to the bottom right of a M X N matrix with the constraints that from each cell you can either move only to the right or down

Examples:

Input: M = 2, N = 2
Output: 2
Explanation: There are two paths
(0, 0) -> (0, 1) -> (1, 1)
(0, 0) -> (1, 0) -> (1, 1)

Input: M = 2, N = 3
Output: 3
Explanation: There are three paths
(0, 0) -> (0, 1) -> (0, 2) -> (1, 2)
(0, 0) -> (0, 1) -> (1, 1) -> (1, 2)
(0, 0) -> (1, 0) -> (1, 1) -> (1, 2)
To solve the problem follow the below idea:

We can recursively move to right and down from the start until we reach the destination and then add up all valid paths to get the answer.
Follow the below steps to solve the problem:

- Create a recursive function with parameters as row and column index
- Call this recursive function for N-1 and M-1
- In the recursive function
    - If N == 1 or M == 1 then return 1
    - else call the recursive function with (N-1, M) and (N, M-1) and return the sum of this
- Print the answer

Below is the implementation of the above approach:

C++JavaPython3C#PHPJavascript

```cpp
// A C++  program to count all possible paths
// from top left to bottom right

#include <bits/stdc++.h>
using namespace std;

// Returns count of possible paths to reach cell at row
// number m and column number n from the topmost leftmost
// cell (cell at 1, 1)
int numberOfPaths(int m, int n)
{
    // If either given row number is first or given column
    // number is first
    if (m == 1 || n == 1)
        return 1;

    // If diagonal movements are allowed then the last
    // addition is required.
    return numberOfPaths(m - 1, n)
           + numberOfPaths(m, n - 1);
    // + numberOfPaths(m-1, n-1);
}

// Driver code
int main()
{
    cout << numberOfPaths(3, 3);
    return 0;
}
```

Output

```
6
```

Time Complexity: $O(2^N)$

Auxiliary Space: $O(N + M)$

Count all possible paths from top left to the bottom right of a M X N matrix using Memoization:

To solve the problem follow the below idea:

As the above recursive solution has overlapping subproblems so we can declare a 2-D array to save the values for different states of the recursive function and later on use the values of this dp array to get the answer for already solved subproblems
Follow the below steps to solve the problem:

- Declare a 2-D array of size N X M
- Create a recursive function with parameters as row and column index and 2-D array
- Call this recursive function for N-1 and M-1
- In the recursive function
  - If N == 1 or M == 1 then return 1
  - If the value of this recursive function is not stored in the 2-D array then call the recursive function for (N-1, M, dp) and (N, M-1, dp) and assign the sum of answers of these functions in the 2-D array and return this value
  - else return the value of this function stored in the 2-D array
- Print the answer

Below is the implementation of the above approach:

C++JavaPython3C#Javascript

```
// A C++ program to count all possible paths from
// top left to top bottom right using
// Recursive Dynamic Programming
#include <bits/stdc++.h>
using namespace std;

// Returns count of possible paths to reach
// cell at row number m and column number n from
// the topmost leftmost cell (cell at 1, 1)
int numberOfPaths(int n, int m, int DP[4][4])
{

    if (n == 1 || m == 1)
        return DP[n][m] = 1;

    // Add the element in the DP table
    // If it was not computed before
    if (DP[n][m] == 0)
        DP[n][m] = numberOfPaths(n - 1, m, DP)
                   + numberOfPaths(n, m - 1, DP);

    return DP[n][m];
```

```
}

// Driver code
int main()
{
    // Create an empty 2D table
    int DP[4][4] = { 0 };
    memset(DP, 0, sizeof(DP));

    cout << numberOfPaths(3, 3, DP);

    return 0;
}
// This code is contributed
// by Gatea David
```

Output

6

Time Complexity: O(N * M)
Auxiliary Space: (N * M)

Count all possible paths from the top left to the bottom right of a M X N matrix using DP:

To solve the problem follow the below idea:

So this problem has both properties (see this and this) of a dynamic programming problem. Like other typical Dynamic Programming(DP) problems, recomputations of the same subproblems can be avoided by constructing a temporary array count[][] in a bottom-up manner using the above recursive formula
Follow the below steps to solve the problem:

- Declare a 2-D array count of size M * N
- Set value of count[i][0] equal to 1 for 0 <= i < M as the answer of subproblem with a single column is equal to 1
- Set value of count[0][j] equal to 1 for 0 <= j < N as the answer of subproblem with a single row is equal to 1
- Create a nested for loop for 0 <= i < M and 0 <= j < N and assign count[i][j] equal to count[i-1][j] + count[i][j-1]
- Print value of count[M-1][N-1]

Below is the implementation of the above approach:

C++JavaPython3C#PHPJavascript

```
// A C++ program to count all possible paths
```

```cpp
// from top left to bottom right
#include <bits/stdc++.h>
using namespace std;

// Returns count of possible paths to reach cell at
// row number m and column  number n from the topmost
// leftmost cell (cell at 1, 1)
int numberOfPaths(int m, int n)
{
    // Create a 2D table to store results of subproblems
    int count[m][n];

    // Count of paths to reach any cell in first column is 1
    for (int i = 0; i < m; i++)
        count[i][0] = 1;

    // Count of paths to reach any cell in first row is 1
    for (int j = 0; j < n; j++)
        count[0][j] = 1;

    // Calculate count of paths for other cells in
    // bottom-up manner using the recursive solution
    for (int i = 1; i < m; i++) {
        for (int j = 1; j < n; j++)

            // By uncommenting the last part the code
            // calculates the total possible paths if the
            // diagonal Movements are allowed
            count[i][j]
                = count[i - 1][j]
                  + count[i][j - 1]; //+ count[i-1][j-1];
    }
    return count[m - 1][n - 1];
}

// Driver code
int main()
{
    cout << numberOfPaths(3, 3);
    return 0;
}
```

Output

```
6
```

Time Complexity: O(M * N) - Due to nested for loops.
Auxiliary Space: O(M * N) - We have used a 2D array of size M x N

Space optimization of the above approach:
To solve the problem follow the below idea:

We can space optimize the above dp approach as for calculating the values of the current row we require only previous row
Follow the below steps to solve the problem:

- Declare an array dp of size N
- Set dp[0] = 1
- Create a nested for loop for 0 <= i < M and 0 <= j < N and add dp[j-1] to dp[j]
- Print value of dp[n - 1]

Below is the implementation of the above approach:

C++JavaPython3C#PHPJavascript

```cpp
// C++ program for the above approach

#include <bits/stdc++.h>
using namespace std;

// Returns count of possible paths to reach
// cell at row number m and column number n from
// the topmost leftmost cell (cell at 1, 1)
int numberOfPaths(int m, int n)
{
    // Create a 1D array to store results of subproblems
    int dp[n] = { 1 };
    dp[0] = 1;

    for (int i = 0; i < m; i++) {
        for (int j = 1; j < n; j++) {
            dp[j] += dp[j - 1];
        }
    }

    return dp[n - 1];
}

// Driver code
int main() { cout << numberOfPaths(3, 3); }

// This code is contributed by mohit kumar 29
```

Output

6

Time Complexity: O(M * N), The program uses nested loops to fill the 1D array "dp". The outer loop runs "m" times, and the inner loop runs "n-1" times. Therefore, the time complexity of the program is O(M*N).
Auxiliary Space: O(N), The program uses a 1D array "dp" of size "n" to store the results of subproblems. Hence, the space complexity of the program is O(N).

This code is contributed by **Vivek Singh**

Note: the count can also be calculated using the formula (M-1 + N-1)!/(M-1)! * (N-1)!

Count all possible paths from top left to the bottom right of a M X N matrix using combinatorics:

To solve the problem follow the below idea:

In this approach, We have to calculate ${}^{m+n-2}C_{n-1}$ here which will be (m+n-2)! / (n-1)! (m-1)!
m = number of rows, n = number of columns

Total number of moves in which we have to move down to reach the last row = m - 1 (m rows, since we are starting from (1, 1) that is not included)
Total number of moves in which we have to move right to reach the last column = n - 1 (n column, since we are starting from (1, 1) that is not included)

Down moves = (m - 1)
Right moves = (n - 1)
Total moves = Down moves + Right moves = (m - 1) + (n - 1)

Now think of moves as a string of 'R' and 'D' characters where 'R' at any ith index will tell us to move 'Right' and 'D' will tell us to move 'Down'. Now think of how many unique strings (moves) we can make where in total there should be (n - 1 + m - 1) characters and there should be (m - 1) 'D' character and (n - 1) 'R' character?

Choosing positions of (n - 1) 'R' characters results in the automatic choosing of (m - 1) 'D' character positions

The number of ways to choose positions for (n - 1) 'R' character = ${}^{\text{Total positions}}C_{n-1}$ = ${}^{\text{Total positions}}C_{m-1}$ = (n - 1 + m - 1) != (n−1+m−1)!(n−1)!(m−1)!

Another way to think about this problem:

Count the Number of ways to make an N digit Binary String (String with 0s and 1s only) with **'X' zeros** and **'Y' ones** (here we have replaced 'R' with '0' or '1' and 'D' with '1' or '0' respectively whichever suits you better)
Follow the below steps to solve the problem:

- Declare a variable path equal to 1
- Create a for loop from i equal to n to (m + n - 1)
  - Set path equal to path * i
  - Set path equal to path divided by (i - n + 1)
- Return path

Below is the implementation of the above approach:

C++JavaPython3C#PHPJavascript

```cpp
// A C++ program to count all possible paths from
// top left to top bottom using combinatorics

#include <bits/stdc++.h>
using namespace std;

int numberOfPaths(int m, int n)
{
    // We have to calculate m+n-2 C n-1 here
    // which will be (m+n-2)! / (n-1)! (m-1)!
    int path = 1;
    for (int i = n; i < (m + n - 1); i++) {
        path *= i;
        path /= (i - n + 1);
    }
    return path;
}

// Driver code
int main()
{
    cout << numberOfPaths(3, 3);
    return 0;
}

// This code is suggested by Kartik Sapra
```

Output

```
6
```

Time Complexity: O(M), The time complexity is O(M), where n is the maximum of m and n. This is because the for loop iterates n times, and each iteration involves a constant number of arithmetic operations.

Auxiliary Space: O(1), The space complexity of the program is also O(1), which means that it uses a constant amount of memory regardless of the input size. This is because the program does not create any data structures or arrays that grow with the input size, and all calculations are done using only a few variables.

This article is contributed by **Hariprasad NG**. Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

Minimum Path Sum

Given an mXn grid filled with non-negative numbers, find a path from the top left to the bottom suitable, which minimizes the sum of all numbers in the path.

Note: You can only move down or right at any point in the path.

Input:

```
arr[][]={{1,3,1},{1,5,1},{4,2,1}}
```

Output:

7

Explanation: The path is 1->3->1->**1->1**

Approach: To solve this problem using recursion, we can define a recursive function that takes the current cell coordinates (i, j) and returns the minimum sum of numbers from the top left to the cell (i, j). The function can recursively call itself on the two neighboring cells to the right and below, and return the minimum of the two sums plus the current cell's value.

C++

```
#include <bits/stdc++.h>
using namespace std;

#define INF 1000000000

int rec(vector<vector<int> >& grid, int i, int j, int n,
        int m)
{
    if (i == n - 1 && j == m - 1) {
        return grid[i][j];
    }
```

```
    int ans = INF;

    if (i + 1 < n)
        ans = min(ans,
                  grid[i][j] + rec(grid, i + 1, j, n, m));
    if (j + 1 < m)
        ans = min(ans,
                  grid[i][j] + rec(grid, i, j + 1, n, m));

    return ans;
}

int main()
{
    vector<vector<int> > grid{ { 5, 9, 6 }, { 11, 5, 2 } };

    int n = grid.size();
    int m = grid[0].size();

    cout << rec(grid, 0, 0, n, m);
    return 0;
}
```

Output:

```
21
```

Time Complexity: O(2^n)

Binomial Coefficient

The following are the common definitions of Binomial Coefficients.

A binomial coefficient C(n, k) can be defined as the coefficient of x^k in the expansion of (1 + x)^n.

A binomial coefficient C(n, k) also gives the number of ways, disregarding order, that k objects can be chosen from among n objects more formally, the number of k-element subsets (or k-combinations) of a n-element set.

The Problem

Write a function that takes two parameters n and k and returns the value of Binomial

Coefficient C(n, k). For example, your function should return 6 for n = 4 and k = 2, and it should return **10** for **n = 5** and **k = 2**.

1) Optimal Substructure

The value of C(n, k) can be recursively calculated using the following standard formula for Binomial Coefficients.

```
C(n, k) = C(n-1, k-1) + C(n-1, k)
C(n, 0) = C(n, n) = 1
```

Following is a simple recursive implementation that simply follows the recursive structure mentioned above.

C++CJavaPython3C#PHPJavascript

```cpp
// A naive recursive C++ implementation
#include <bits/stdc++.h>
using namespace std;

// Returns value of Binomial Coefficient C(n, k)
int binomialCoeff(int n, int k)
{
    // Base Cases
    if (k > n)
        return 0;
    if (k == 0 || k == n)
        return 1;

    // Recur
    return binomialCoeff(n - 1, k - 1)
           + binomialCoeff(n - 1, k);
}

/* Driver code*/
int main()
{
    int n = 5, k = 2;
    cout << "Value of C(" << n << ", " << k << ") is "
         << binomialCoeff(n, k);
    return 0;
}

// This is code is contributed by rathbhupendra
```

Output

```
Value of C(5, 2) is 10
```

Time Complexity: O(n*max(k,n-k))

Auxiliary Space: O(n*max(k,n-k))

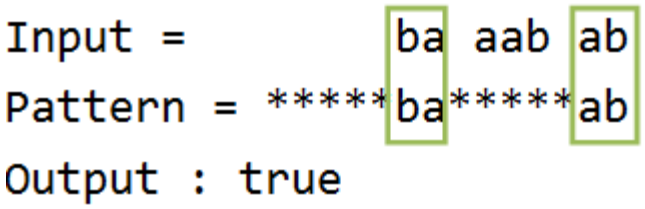
Text Pattern Matching

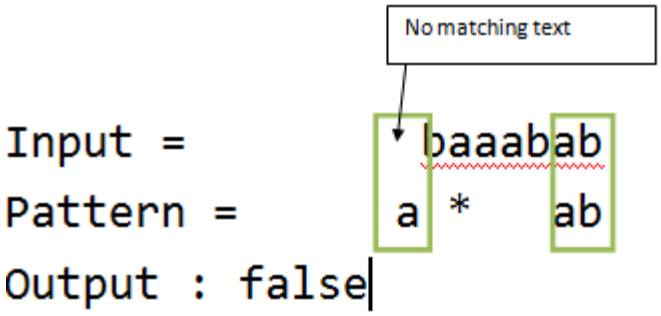
Given a text and a wildcard pattern, implement wildcard pattern matching algorithm that finds if wildcard pattern is matched with text. The matching should cover the entire text (not partial text). The wildcard pattern can include the characters '?' and '*'

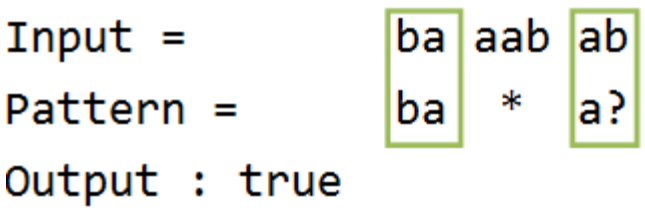
- '?' – matches any single character
- '*' – Matches any sequence of characters (including the empty sequence)

For example:

```
Text = "baaabab",
Pattern = "*****ba*****ab", output : true
Pattern = "baaa?ab", output : true
Pattern = "ba*a?", output : true
Pattern = "a*ab", output : false
```







Each occurrence of '?' character in wildcard pattern can be replaced with any other character and each occurrence of '*' with a sequence of characters such that the wildcard pattern becomes identical to the input string after replacement.

Let's consider any character in the pattern.

Case 1: The character is '*' . Here two cases arises as follows:

1. We can ignore '*' character and move to next character in the Pattern.
2. '*' character matches with one or more characters in Text. Here we will move to next character in the string.

Case 2: The character is '?'
We can ignore current character in Text and move to next character in the Pattern and Text.

Case 3: The character is not a wildcard character
If current character in Text matches with current character in Pattern, we move to next character in the Pattern and Text. If they do not match, wildcard pattern and Text do not match.
We can use Dynamic Programming to solve this problem:

Let **T[i][j]** is true if first i characters in given string match the first j characters of pattern.

Method 1:Using Backtracking(Brute Force)

Firstly we should be going thorugh the backtracking method:

The implementation of the code:

C++JavaC#

```
#include <iostream>
using namespace std;

bool isMatch(string s, string p) {
        //dry run this sample case on paper , if unable to understand what
soln does
        // p = "a*bc" s = "abcbc"
        int sIdx = 0, pIdx = 0, lastWildcardIdx = -1, sBacktrackIdx = -1,
nextToWildcardIdx = -1;
        while (sIdx < s.size()) {
            if (pIdx < p.size() && (p[pIdx] == '?' || p[pIdx] == s[sIdx]))
{
```

```
                // chars match
                ++sIdx;
                ++pIdx;
            } else if (pIdx < p.size() && p[pIdx] == '*') {
                // wildcard, so chars match - store index.
                lastWildcardIdx = pIdx;
                nextToWildcardIdx = ++pIdx;
                sBacktrackIdx = sIdx;

                //storing the pidx+1 as from there I want to match the
remaining pattern
            } else if (lastWildcardIdx == -1) {
                // no match, and no wildcard has been found.
                return false;
            } else {
                // backtrack - no match, but a previous wildcard was
found.
                pIdx = nextToWildcardIdx;
                sIdx = ++sBacktrackIdx;
                //backtrack string from previousbacktrackidx + 1 index to
see if then new pidx and sidx have same chars, if that is the case that
means wildcard can absorb the chars in b/w and still further we can run
the algo, if at later stage it fails we can backtrack
            }
        }
        for(int i = pIdx; i < p.size(); i++){
            if(p[i] != '*') return false;
        }
        return true;
        // true if every remaining char in p is wildcard
    }

int main() {

    string str = "baaabab";
    string pattern = "*****ba*****ab";

    if (isMatch(str, pattern))
        cout << "Yes" << endl;
    else
        cout << "No" << endl;
}
```

Output
Yes

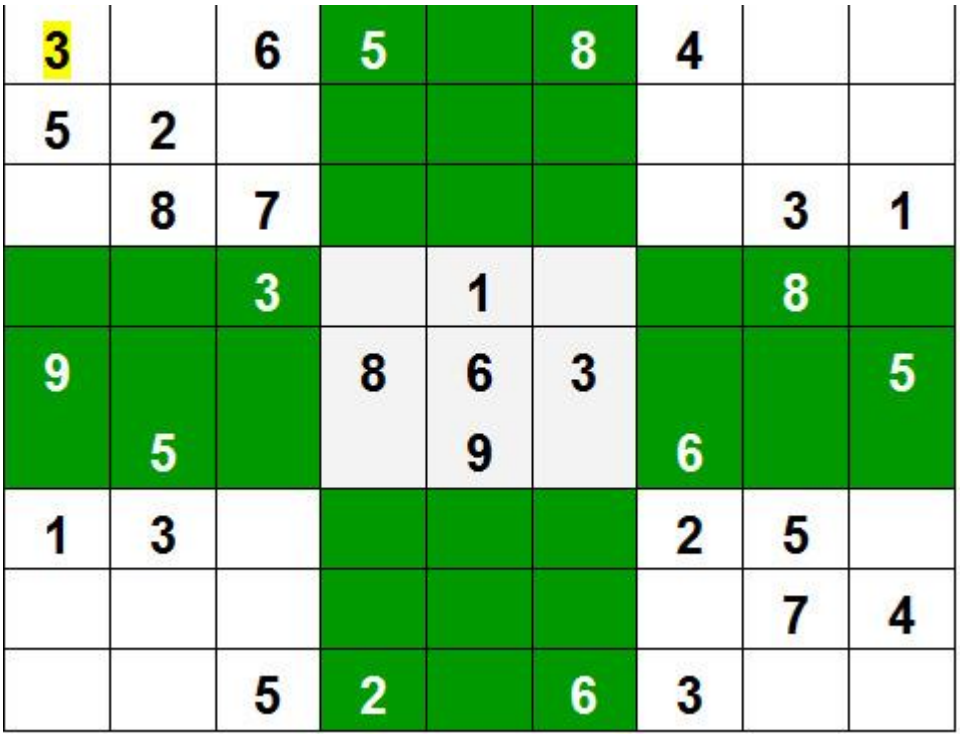
Time complexity: O(m x n)

Auxiliary space: O(m x n)

Backtracking

Sudoku | Backtracking

Given a partially filled 9x9 2D array 'grid[9][9]', the goal is to assign digits (from 1 to 9) to the empty cells so that every row, column, and subgrid of size 3x3 contains exactly one instance of the digits from 1 to 9.



Examples:

Input: grid
{ {3, 0, 6, 5, 0, 8, 4, 0, 0},
{5, 2, 0, 0, 0, 0, 0, 0, 0},
{0, 8, 7, 0, 0, 0, 0, 3, 1},
{0, 0, 3, 0, 1, 0, 0, 8, 0},
{9, 0, 0, 8, 6, 3, 0, 0, 5},
{0, 5, 0, 0, 9, 0, 6, 0, 0},

{1, 3, 0, 0, 0, 0, 2, 5, 0},
{0, 0, 0, 0, 0, 0, 0, 7, 4},
{0, 0, 5, 2, 0, 6, 3, 0, 0} }

Output:

3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

Explanation: Each row, column and 3*3 box of the output matrix contains unique numbers.

Input: grid

{ { 3, 1, 6, 5, 7, 8, 4, 9, 2 },
{ 5, 2, 9, 1, 3, 4, 7, 6, 8 },
{ 4, 8, 7, 6, 2, 9, 5, 3, 1 },
{ 2, 6, 3, 0, 1, 5, 9, 8, 7 },
{ 9, 7, 4, 8, 6, 0, 1, 2, 5 },
{ 8, 5, 1, 7, 9, 2, 6, 4, 3 },
{ 1, 3, 8, 0, 4, 7, 2, 0, 6 },
{ 6, 9, 2, 3, 5, 1, 8, 7, 4 },
{ 7, 4, 5, 0, 8, 6, 3, 1, 0 } };

Output:

3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9

Explanation: Each row, column and 3*3 box of the output matrix contains unique numbers.

Naive Approach:

The naive approach is to generate all possible configurations of numbers from 1 to 9 to fill the empty cells. Try every configuration one by one until the correct configuration is found, i.e. for every unassigned position fill the position with a number from 1 to 9. After filling all the unassigned positions check if the matrix is safe or not. If safe print else recurs for other cases.

Follow the steps below to solve the problem:

- Create a function that checks if the given matrix is valid sudoku or not. Keep Hashmap for the row, column and boxes. If any number has a frequency greater than 1 in the hashMap return false else return true;
- Create a recursive function that takes a grid and the current row and column index.
- Check some base cases.
    - If the index is at the end of the matrix, i.e. i=N-1 and j=N then check if the grid is safe or not, if safe print the grid and return true else return false.
    - The other base case is when the value of column is N, i.e j = N, then move to next row, i.e. i++ and j = 0.
- If the current index is not assigned then fill the element from 1 to 9 and recur for all 9 cases with the index of next element, i.e. i, j+1. if the recursive call returns true then break the loop and return true.
- If the current index is assigned then call the recursive function with the index of the next element, i.e. i, j+1

Below is the implementation of the above approach:

C++CJavaPython3C#Javascript

```
#include <iostream>

using namespace std;

// N is the size of the 2D matrix   N*N
#define N 9

/* A utility function to print grid */
void print(int arr[N][N])
{
    for (int i = 0; i < N; i++)
    {
        for (int j = 0; j < N; j++)
            cout << arr[i][j] << " ";
        cout << endl;
    }
}
```

```
// Checks whether it will be
// legal to assign num to the
// given row, col
bool isSafe(int grid[N][N], int row,
                        int col, int num)
{

    // Check if we find the same num
    // in the similar row , we
    // return false
    for (int x = 0; x <= 8; x++)
        if (grid[row][x] == num)
            return false;

    // Check if we find the same num in
    // the similar column , we
    // return false
    for (int x = 0; x <= 8; x++)
        if (grid[x][col] == num)
            return false;

    // Check if we find the same num in
    // the particular 3*3 matrix,
    // we return false
    int startRow = row - row % 3,
            startCol = col - col % 3;

    for (int i = 0; i < 3; i++)
        for (int j = 0; j < 3; j++)
            if (grid[i + startRow][j +
                            startCol] == num)
                return false;

    return true;
}

/* Takes a partially filled-in grid and attempts
to assign values to all unassigned locations in
such a way to meet the requirements for
Sudoku solution (non-duplication across rows,
columns, and boxes) */
bool solveSudoku(int grid[N][N], int row, int col)
{
    // Check if we have reached the 8th
    // row and 9th column (0
    // indexed matrix) , we are
    // returning true to avoid
    // further backtracking
```

```
    if (row == N - 1 && col == N)
        return true;

    // Check if column value  becomes 9 ,
    // we move to next row and
    //  column start from 0
    if (col == N) {
        row++;
        col = 0;
    }

    // Check if the current position of
    // the grid already contains
    // value >0, we iterate for next column
    if (grid[row][col] > 0)
        return solveSudoku(grid, row, col + 1);

    for (int num = 1; num <= N; num++)
    {

        // Check if it is safe to place
        // the num (1-9)  in the
        // given row ,col  ->we
        // move to next column
        if (isSafe(grid, row, col, num))
        {

           /* Assigning the num in
              the current (row,col)
              position of the grid
              and assuming our assigned
              num in the position
              is correct     */
            grid[row][col] = num;

            //  Checking for next possibility with next
            //  column
            if (solveSudoku(grid, row, col + 1))
                return true;
        }

        // Removing the assigned num ,
        // since our assumption
        // was wrong , and we go for
        // next assumption with
        // diff num value
        grid[row][col] = 0;
    }
    return false;
```

```
}

// Driver Code
int main()
{
    // 0 means unassigned cells
    int grid[N][N] = { { 3, 0, 6, 5, 0, 8, 4, 0, 0 },
                       { 5, 2, 0, 0, 0, 0, 0, 0, 0 },
                       { 0, 8, 7, 0, 0, 0, 0, 3, 1 },
                       { 0, 0, 3, 0, 1, 0, 0, 8, 0 },
                       { 9, 0, 0, 8, 6, 3, 0, 0, 5 },
                       { 0, 5, 0, 0, 9, 0, 6, 0, 0 },
                       { 1, 3, 0, 0, 0, 0, 2, 5, 0 },
                       { 0, 0, 0, 0, 0, 0, 0, 7, 4 },
                       { 0, 0, 5, 2, 0, 6, 3, 0, 0 } };

    if (solveSudoku(grid, 0, 0))
        print(grid);
    else
        cout << "no solution  exists " << endl;

    return 0;
    // This is code is contributed by Pradeep Mondal P
}
```

Output

```
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Time complexity: O($9^{(N*N)}$), For every unassigned index, there are 9 possible options so the time complexity is O(9^(n*n)).
Space Complexity: O(N*N), To store the output array a matrix is needed.

Sudoku using Backtracking:

Like all other Backtracking problems, Sudoku can be solved by assigning numbers one by one to empty cells. Before assigning a number, check whether it is safe to assign.

Check that the same number is not present in the current row, current column and current 3X3 subgrid. After checking for safety, assign the number, and recursively check whether this assignment leads to a solution or not. If the assignment doesn't lead to a solution, then try the next number for the current empty cell. And if none of the number (1 to 9) leads to a solution, return false and print no solution exists.

Follow the steps below to solve the problem:

- Create a function that checks after assigning the current index the grid becomes unsafe or not. Keep Hashmap for a row, column and boxes. If any number has a frequency greater than 1 in the hashMap return false else return true; hashMap can be avoided by using loops.
- Create a recursive function that takes a grid.
- Check for any unassigned location.
    - If present then assigns a number from 1 to 9.
    - Check if assigning the number to current index makes the grid unsafe or not.
    - If safe then recursively call the function for all safe cases from 0 to 9.
    - If any recursive call returns true, end the loop and return true. If no recursive call returns true then return false.
- If there is no unassigned location then return true.

Below is the implementation of the above approach:

C++JavaPython3C#Javascript

```
// A Backtracking program in
// C++ to solve Sudoku problem
#include <bits/stdc++.h>
using namespace std;

// UNASSIGNED is used for empty
// cells in sudoku grid
#define UNASSIGNED 0

// N is used for the size of Sudoku grid.
// Size will be NxN
#define N 9

// This function finds an entry in grid
// that is still unassigned
bool FindUnassignedLocation(int grid[N][N],
                            int& row, int& col);

// Checks whether it will be legal
// to assign num to the given row, col
bool isSafe(int grid[N][N], int row,
            int col, int num);
```

```
/* Takes a partially filled-in grid and attempts
to assign values to all unassigned locations in
such a way to meet the requirements for
Sudoku solution (non-duplication across rows,
columns, and boxes) */
bool SolveSudoku(int grid[N][N])
{
    int row, col;

    // If there is no unassigned location,
    // we are done
    if (!FindUnassignedLocation(grid, row, col))
        // success!
        return true;

    // Consider digits 1 to 9
    for (int num = 1; num <= 9; num++)
    {

        // Check if looks promising
        if (isSafe(grid, row, col, num))
        {

            // Make tentative assignment
            grid[row][col] = num;

            // Return, if success
            if (SolveSudoku(grid))
                return true;

            // Failure, unmake & try again
            grid[row][col] = UNASSIGNED;
        }
    }

    // This triggers backtracking
    return false;
}

/* Searches the grid to find an entry that is
still unassigned. If found, the reference
parameters row, col will be set the location
that is unassigned, and true is returned.
If no unassigned entries remain, false is returned. */
bool FindUnassignedLocation(int grid[N][N],
                            int& row, int& col)
{
    for (row = 0; row < N; row++)
        for (col = 0; col < N; col++)
```

```
            if (grid[row][col] == UNASSIGNED)
                return true;
    return false;
}

/* Returns a boolean which indicates whether
an assigned entry in the specified row matches
the given number. */
bool UsedInRow(int grid[N][N], int row, int num)
{
    for (int col = 0; col < N; col++)
        if (grid[row][col] == num)
            return true;
    return false;
}

/* Returns a boolean which indicates whether
an assigned entry in the specified column
matches the given number. */
bool UsedInCol(int grid[N][N], int col, int num)
{
    for (int row = 0; row < N; row++)
        if (grid[row][col] == num)
            return true;
    return false;
}

/* Returns a boolean which indicates whether
an assigned entry within the specified 3x3 box
matches the given number. */
bool UsedInBox(int grid[N][N], int boxStartRow,
               int boxStartCol, int num)
{
    for (int row = 0; row < 3; row++)
        for (int col = 0; col < 3; col++)
            if (grid[row + boxStartRow]
                    [col + boxStartCol] ==
                                        num)
                return true;
    return false;
}

/* Returns a boolean which indicates whether
it will be legal to assign num to the given
row, col location. */
bool isSafe(int grid[N][N], int row,
            int col, int num)
{
    /* Check if 'num' is not already placed in
```

```
    current row, current column
    and current 3x3 box */
    return !UsedInRow(grid, row, num)
           && !UsedInCol(grid, col, num)
           && !UsedInBox(grid, row - row % 3,
                         col - col % 3, num)
           && grid[row][col] == UNASSIGNED;
}

/* A utility function to print grid */
void printGrid(int grid[N][N])
{
    for (int row = 0; row < N; row++)
    {
        for (int col = 0; col < N; col++)
            cout << grid[row][col] << " ";
        cout << endl;
    }
}

// Driver Code
int main()
{
    // 0 means unassigned cells
    int grid[N][N] = { { 3, 0, 6, 5, 0, 8, 4, 0, 0 },
                       { 5, 2, 0, 0, 0, 0, 0, 0, 0 },
                       { 0, 8, 7, 0, 0, 0, 0, 3, 1 },
                       { 0, 0, 3, 0, 1, 0, 0, 8, 0 },
                       { 9, 0, 0, 8, 6, 3, 0, 0, 5 },
                       { 0, 5, 0, 0, 9, 0, 6, 0, 0 },
                       { 1, 3, 0, 0, 0, 0, 2, 5, 0 },
                       { 0, 0, 0, 0, 0, 0, 0, 7, 4 },
                       { 0, 0, 5, 2, 0, 6, 3, 0, 0 } };
    if (SolveSudoku(grid) == true)
        printGrid(grid);
    else
        cout << "No solution exists";

    return 0;
}

// This is code is contributed by rathbhupendra
```

Output

```
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
```

```
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Time complexity: O($9^{(N*N)}$), For every unassigned index, there are 9 possible options so the time complexity is O(9^(n*n)). The time complexity remains the same but there will be some early pruning so the time taken will be much less than the naive algorithm but the upper bound time complexity remains the same.
Space Complexity: O(N*N), To store the output array a matrix is needed.

Sudoku using Bit Masks:

This method is a slight optimization to the above 2 methods. For each row/column/box create a bitmask and for each element in the grid set the bit at position 'value' to 1 in the corresponding bitmasks, for O(1) checks.

Follow the steps below to solve the problem:

- Create 3 arrays of size N (one for rows, columns, and boxes).
- The boxes are indexed from 0 to 8. (in order to find the box index of an element we use the following formula: **row / 3 * 3 + column / 3**).
- Map the initial values of the grid first.
- Each time we **add/remove** an element to/from the grid **set the bit to 1/0** to the corresponding bitmasks.

Below is the implementation of the above approach:

C++JavaPython3C#Javascript

```cpp
#include <bits/stdc++.h>
using namespace std;

#define N 9

// Bitmasks for each row/column/box
int row[N], col[N], box[N];
bool seted = false;

// Utility function to find the box index
// of an element at position [i][j] in the grid
int getBox(int i, int j) { return i / 3 * 3 + j / 3; }

// Utility function to check if a number
// is present in the corresponding row/column/box
bool isSafe(int i, int j, int number)
{
```

```
    return !((row[i] >> number) & 1)
           && !((col[j] >> number) & 1)
           && !((box[getBox(i, j)] >> number) & 1);
}

// Utility function to set the initial values of a Sudoku
// board (map the values in the bitmasks)
void setInitialValues(int grid[N][N])
{
    for (int i = 0; i < N; i++)
        for (int j = 0; j < N; j++)
            row[i] |= 1 << grid[i][j],
                col[j] |= 1 << grid[i][j],
                box[getBox(i, j)] |= 1 << grid[i][j];
}

/* Takes a partially filled-in grid and attempts
to assign values to all unassigned locations in
such a way to meet the requirements for
Sudoku solution (non-duplication across rows,
columns, and boxes) */
bool SolveSudoku(int grid[N][N], int i, int j)
{
    // Set the initial values
    if (!seted)
        seted = true, setInitialValues(grid);

    if (i == N - 1 && j == N)
        return true;
    if (j == N)
        j = 0, i++;

    if (grid[i][j])
        return SolveSudoku(grid, i, j + 1);

    for (int nr = 1; nr <= N; nr++) {
        if (isSafe(i, j, nr)) {
            /*  Assign nr in the
                current (i, j)
                position and
                add nr to each bitmask
            */
            grid[i][j] = nr;
            row[i] |= 1 << nr;
            col[j] |= 1 << nr;
            box[getBox(i, j)] |= 1 << nr;

            if (SolveSudoku(grid, i, j + 1))
                return true;
```

```
            // Remove nr from each bitmask
            // and search for another possibility
            row[i] &= ~(1 << nr);
            col[j] &= ~(1 << nr);
            box[getBox(i, j)] &= ~(1 << nr);
        }

        grid[i][j] = 0;
    }

    return false;
}

// Utility function to print the solved grid
void print(int grid[N][N])
{
    for (int i = 0; i < N; i++, cout << '\n')
        for (int j = 0; j < N; j++)
            cout << grid[i][j] << ' ';
}

// Driver Code
int main()
{
    // 0 means unassigned cells
    int grid[N][N] = { { 3, 0, 6, 5, 0, 8, 4, 0, 0 },
                       { 5, 2, 0, 0, 0, 0, 0, 0, 0 },
                       { 0, 8, 7, 0, 0, 0, 0, 3, 1 },
                       { 0, 0, 3, 0, 1, 0, 0, 8, 0 },
                       { 9, 0, 0, 8, 6, 3, 0, 0, 5 },
                       { 0, 5, 0, 0, 9, 0, 6, 0, 0 },
                       { 1, 3, 0, 0, 0, 0, 2, 5, 0 },
                       { 0, 0, 0, 0, 0, 0, 0, 7, 4 },
                       { 0, 0, 5, 2, 0, 6, 3, 0, 0 } };

    if (SolveSudoku(grid, 0, 0))
        print(grid);
    else
        cout << "No solution exists\n";

    return 0;
}
// This code is contributed
// by Gatea David
```

Output

```
3 1 6 5 7 8 4 9 2
```

```
5 2 9 1 3 4 7 6 8
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Time complexity: O($9^{(N*N)}$). For every unassigned index, there are 9 possible options so the time complexity is O(9^(n*n)). The time complexity remains the same but checking if a number is safe to use is much faster, O(1).
Space Complexity: O(N*N). To store the output array a matrix is needed, and 3 extra arrays of size n are needed for the bitmasks.

Sudoku using Cross-Hatching with backtracking:

This method is an optimization of the above method 2. It runs 5X times faster than method 2. Like we used to fill sudoku by first identifying the element which is almost filled. It starts with identifying the row and column where the element should be placed. Picking the almost-filled elements first will give better pruning.

Follow the steps below to solve the problem:

1. Build a graph with pending elements mapped to row and column coordinates where they can be fitted in the original matrix.
2. Pick the elements from the graph sorted by fewer remaining elements to be filled.
3. Recursively fill the elements using a graph into the matrix. Backtrack once an unsafe position is discovered.

Below is the implementation of the above approach:

C++Python3JavascriptJavaC#

```cpp
// C++ Code
#include <algorithm>
#include <bits/stdc++.h>
#include <map>
#include <vector>

using namespace std;

// Input matrix
vector<vector<int> > arr
    = { { 3, 0, 6, 5, 0, 8, 4, 0, 0 },
        { 5, 2, 0, 0, 0, 0, 0, 0, 0 },
        { 0, 8, 7, 0, 0, 0, 0, 3, 1 },
        { 0, 0, 3, 0, 1, 0, 0, 8, 0 },
```

```
        { 9, 0, 0, 8, 6, 3, 0, 0, 5 },
        { 0, 5, 0, 0, 9, 0, 6, 0, 0 },
        { 1, 3, 0, 0, 0, 0, 2, 5, 0 },
        { 0, 0, 0, 0, 0, 0, 0, 7, 4 },
        { 0, 0, 5, 2, 0, 6, 3, 0, 0 } };

// Position of the input elements in the arr
// pos = {
//     element: [[position 1], [position 2]]
// }
map<int, vector<vector<int> > > pos;

// Count of the remaining number of the elements
// rem = {
//     element: pending count
// }
map<int, int> rem;

// Graph defining tentative positions of the elements to be
// filled graph = {
//     key: {
//         row1: [columns],
//         row2: [columns]
//     }
// }
map<int, map<int, vector<int> > > graph;

// Print the matrix array
void printMatrix()
{
  for (int i = 0; i < 9; i++) {
    for (int j = 0; j < 9; j++) {
      cout << arr[i][j] << " ";
    }
    cout << endl;
  }
}

// Method to check if the inserted element is safe
bool is_safe(int x, int y)
{
  int key = arr[x][y];
  for (int i = 0; i < 9; i++) {
    if (i != y && arr[x][i] == key) {
      return false;
    }
    if (i != x && arr[i][y] == key) {
      return false;
    }
```

```
  }

  int r_start = floor(x / 3) * 3;
  int r_end = r_start + 3;

  int c_start = floor(y / 3) * 3;
  int c_end = c_start + 3;

  for (int i = r_start; i < r_end; i++) {
    for (int j = c_start; j < c_end; j++) {
      if (i != x && j != y && arr[i][j] == key) {
        return false;
      }
    }
  }
  return true;
}

// method to fill the matrix
// input keys: list of elements to be filled in the matrix
//        k   : index number of the element to be picked up
//        from keys rows: list of row index where element is
//        to be inserted r   : index number of the row to be
//        inserted
//
bool fill_matrix(int k, vector<int> keys, int r,
                 vector<int> rows)
{
  int c = 0;
  arr[rows[r]][c] = keys[k];
  if (is_safe(rows[r], c)) {
    if (r < rows.size() - 1) {
      if (fill_matrix(k, keys, r + 1, rows)) {
        return true;
      }
      else {
        arr[rows[r]][c] = 0;
      }
    }
    else {
      if (k < keys.size() - 1) {
        if (fill_matrix(
          k + 1, keys, 0,
          rows)) {
          return true;
        }
        else {
          arr[rows[r]][c] = 0;
```

```
        }
      }
      return true;
    }
  }
  arr[rows[r]][c] = 0;

  return false;
}

// Fill the pos and rem dictionary. It will be used to build
// graph
void build_pos_and_rem()
{
  for (int i = 0; i < 9; i++) {
    for (int j = 0; j < 9; j++) {
      if (arr[i][j] > 0) {
        if (!pos.count(arr[i][j])) {
          pos[arr[i][j]] = {};
        }
        pos[arr[i][j]].push_back({ i, j });
        if (!rem.count(arr[i][j])) {
          rem[arr[i][j]] = 9;
        }
        rem[arr[i][j]] -= 1;
      }
    }
  }

  // Fill the elements not present in input matrix.
  // Example: 1 is missing in input matrix
  for (int i = 1; i < 10; i++) {
    if (!pos.count(i)) {
      pos[i] = {};
    }
    if (!rem.count(i)) {
      rem[i] = 9;
    }
  }
}

int main() { printMatrix(); }

// This code is contributed by ishankhandelwals.
```

Output

```
3 1 6 5 7 8 4 9 2
5 2 9 1 3 4 7 6 8
```

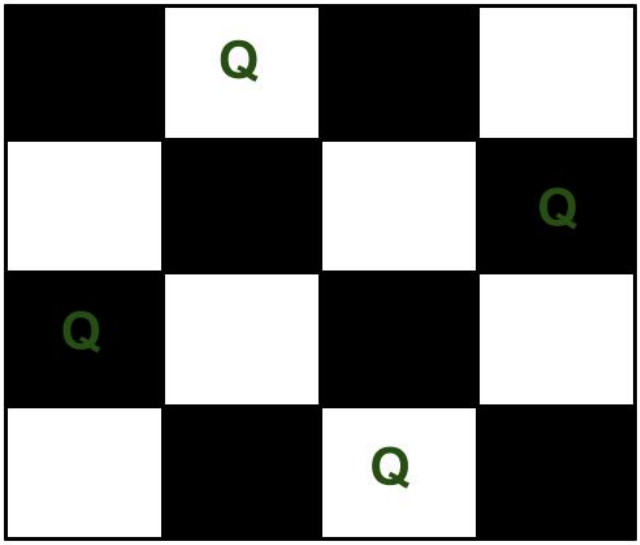
```
4 8 7 6 2 9 5 3 1
2 6 3 4 1 5 9 8 7
9 7 4 8 6 3 1 2 5
8 5 1 7 9 2 6 4 3
1 3 8 9 4 7 2 5 6
6 9 2 3 5 1 8 7 4
7 4 5 2 8 6 3 1 9
```

Time complexity: O(9^(n*n)). Due to the element that needs to fit in a cell will come earlier as we are filling almost filled elements first, it will help in less number of recursive calls. So the time taken will be much less than the naive approaches but the upper bound time complexity remains the same.

Auxiliary Space: O(n*n). A graph of the remaining positions to be filled for the respected elements is created.

N-Queen | Backtracking

Let us discuss N Queen as another example problem that can be solved using backtracking. The N Queen is the problem of placing N chess queens on an N×N chessboard so that no two queens attack each other. For example, the following is a solution for the 4 Queen problem.



The expected output is in form of a matrix that has 'Q's for the blocks where queens are placed and the empty spaces are represented by '.'s . For example, the following is the output matrix for the above 4 queen solution.

. . Q .
Q . . .
. . . Q
. Q . .

Algorithm for N queen problem:-

- Initialize an empty chessboard of size NxN.
- Start with the leftmost column and place a queen in the first row of that column.
- Move to the next column and place a queen in the first row of that column.
- Repeat step 3 until either all N queens have been placed or it is impossible to place a queen in the current column without violating the rules of the problem.
- If all N queens have been placed, print the solution.

- If it is not possible to place a queen in the current column without violating the rules of the problem, backtrack to the previous column.
- Remove the queen from the previous column and move it down one row.
- Repeat steps 4-7 until all possible configurations have been tried.

Pseudo-code implementation:

```
function solveNQueens(board, col, n):
  if col >= n:
    print board
    return true
  for row from 0 to n-1:
    if isSafe(board, row, col, n):
      board[row][col] = 1
      if solveNQueens(board, col+1, n):
        return true
      board[row][col] = 0
  return false

function isSafe(board, row, col, n):
  for i from 0 to col-1:
    if board[row][i] == 1:
      return false
  for i,j from row-1, col-1 to 0, 0 by -1:
    if board[i][j] == 1:
      return false
  for i,j from row+1, col-1 to n-1, 0 by 1, -1:
    if board[i][j] == 1:
      return false
  return true

board = empty NxN chessboard
solveNQueens(board, 0, N)
```

Naive Algorithm

Generate all possible configurations of queens on board and print a configuration that satisfies the given constraints.

```
while there are untried configurations
{
   generate the next configuration
   if queens don't attack in this configuration then
   {
      print this configuration;
   }
}
```

Backtracking Algorithm Method 1:

The idea is to place queens one by one in different columns, starting from the leftmost column. When we place a queen in a column, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return false.

```
Method 1:
1) Start in the leftmost column
2) If all queens are placed
    return true
3) Try all rows in the current column.
   Do following for every tried row.
    a) If the queen can be placed safely in this row
       then mark this [row, column] as part of the
       solution and recursively check if placing
       queen here leads to a solution.
    b) If placing the queen in [row, column] leads to
       a solution then return true.
    c) If placing queen doesn't lead to a solution then
       unmark this [row, column] (Backtrack) and go to
       step (a) to try other rows.
4) If all rows have been tried and nothing worked,
   return false to trigger backtracking.
```

Implementation of Backtracking solution by method 1:

C++CJavaPython3C#Javascript

```cpp
/* C++ program to solve N Queen Problem using
   backtracking */

#include <bits/stdc++.h>
#define N 4
using namespace std;

/* A utility function to print solution */
void printSolution(int board[N][N])
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
           if(board[i][j])
            cout << "Q ";
           else cout<<". ";
        printf("\n");
    }
}
```

```
/* A utility function to check if a queen can
   be placed on board[row][col]. Note that this
   function is called when "col" queens are
   already placed in columns from 0 to col -1.
   So we need to check only left side for
   attacking queens */
bool isSafe(int board[N][N], int row, int col)
{
    int i, j;

    /* Check this row on left side */
    for (i = 0; i < col; i++)
        if (board[row][i])
            return false;

    /* Check upper diagonal on left side */
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--)
        if (board[i][j])
            return false;

    /* Check lower diagonal on left side */
    for (i = row, j = col; j >= 0 && i < N; i++, j--)
        if (board[i][j])
            return false;

    return true;
}

/* A recursive utility function to solve N
   Queen problem */
bool solveNQUtil(int board[N][N], int col)
{
    /* base case: If all queens are placed
      then return true */
    if (col >= N)
        return true;

    /* Consider this column and try placing
       this queen in all rows one by one */
    for (int i = 0; i < N; i++) {
        /* Check if the queen can be placed on
          board[i][col] */
        if (isSafe(board, i, col)) {
            /* Place this queen in board[i][col] */
            board[i][col] = 1;

            /* recur to place rest of the queens */
            if (solveNQUtil(board, col + 1))
                return true;
```

```
            /* If placing queen in board[i][col]
               doesn't lead to a solution, then
               remove queen from board[i][col] */
            board[i][col] = 0; // BACKTRACK
        }
    }

    /* If the queen cannot be placed in any row in
        this column col  then return false */
    return false;
}

/* This function solves the N Queen problem using
   Backtracking. It mainly uses solveNQUtil() to
   solve the problem. It returns false if queens
   cannot be placed, otherwise, return true and
   prints placement of queens in the form of 1s.
   Please note that there may be more than one
   solutions, this function prints one  of the
   feasible solutions.*/
bool solveNQ()
{
    int board[N][N] = { { 0, 0, 0, 0 },
                        { 0, 0, 0, 0 },
                        { 0, 0, 0, 0 },
                        { 0, 0, 0, 0 } };

    if (solveNQUtil(board, 0) == false) {
        cout << "Solution does not exist";
        return false;
    }

    printSolution(board);
    return true;
}

// driver program to test above function
int main()
{
    solveNQ();
    return 0;
}

// This code is contributed by Aditya Kumar (adityakumar129)
```

Output

```
. . Q .
```

Q . . .
. . . Q
. Q . .

Time Complexity: O(N!)
Auxiliary Space: O(N2)

Backtracking Algorithm Method 2:
The idea is to place queens one by one in different rows, starting from the topmost row. When we place a queen in a row, we check for clashes with already placed queens. In the current column, if we find a row for which there is no clash, we mark this row and column as part of the solution. If we do not find such a row due to clashes, then we backtrack and return false.

Method 2:
0) Make a board, make a space to collect all solution states.
1) Start in the topmost row.
2) Make a recursive function which takes state of board and the current row number as its parameter.
3) Fill a queen in a safe place and use this state of board to advance to next recursive call, add 1 to the current row. Revert the state of board after making the call.
 a) Safe function checks the current column, left top diagonal and right top diagonal.
 b) If no queen is present then fill else return false and stop exploring that state and track back to the next possible solution state
4) Keep calling the function till the current row is out of bound.
5) If current row reaches the number of rows in the board then the board is filled.
6) Store the state and return.

Implementation of Backtracking solution by Method 2:

C++JavaPython3C#Javascript

```
#include <bits/stdc++.h>
using namespace std;
// store all the possible answers
vector<vector<string> > answer;
// print the board
void print_board()
{
    for (auto& str : answer[1]) {
        for (auto& letter : str)
            cout << letter << " ";
        cout << endl;
    }
```

```
    return;
}
// we need to check in three directions
// 1. in the same column above the current position
// 2. in the left top diagonal from the given cell
// 3. in the right top diagonal from the given cell
int safe(int row, int col, vector<string>& board)
{
    for (int i = 0; i < board.size(); i++) {
        if (board[i][col] == 'Q')
            return false;
    }
    int i = row, j = col;
    while (i >= 0 && j >= 0)
        if (board[i--][j--] == 'Q')
            return false;
    i = row, j = col;
    while (i >= 0 && j < board.size())
        if (board[i--][j++] == 'Q')
            return false;
    return true;
}
// rec function here will fill the queens
// 1. there can be only one queen in one row
// 2. if we filled the final row in the board then row will
// be equal to total number of rows in board
// 3. push that board configuration in answer set because
// there will be more than one answers for filling the board
// with n-queens
void rec(vector<string> board, int row)
{
    if (row == board.size()) {
        answer.push_back(board);
        return;
    }
    for (int i = 0; i < board.size(); i++) {
        // for each position check if it is safe and if it
        // safe make a recursive call with
        // row+1,board[i][j]='Q' and then revert the change
        // in board that is make the board[i][j]='.' again to
        // generate more solutions
        if (safe(row, i, board)) {
            board[row][i] = 'Q';
            rec(board, row + 1);
            board[row][i] = '.';
        }
    }
    return;
}
```

```
// function to solve n queens
vector<vector<string> > solveNQueens(int n)
{
    string s;
    for (int i = 0; i < n; i++)
        s += '.';
    // vector of string will make our board which is
    // initially all empty
    vector<string> board(n, s);
    rec(board, 0);
    return answer;
}
int main()
{
    clock_t start, end; // this is to calculate the
                        // execution time for n-queens
    start = clock();
    // size 4x4 is taken and we can pass some other
    // dimension for chess board as well
    cout << solveNQueens(4).size() << endl;
    end = clock();
    double time_taken
        = double(end - start) / double(CLOCKS_PER_SEC);
    cout << time_taken << " time was taken(in miliseconds)"
         << endl;
    cout << "Out of " << answer.size()
         << " solutions one is following" << endl;
    print_board();
}
```

Output

```
2
9.7e-05 time was taken(in seconds)
Out of 2 solutions one is following
. . Q .
Q . . .
. . . Q
. Q . .
```

Time Complexity: O(N!)

Auxiliary Space: O(N^2)

Optimization in is_safe() function

The idea is not to check every element in right and left diagonal, instead use the property of diagonals:

1. The sum of i and j is constant and unique for each right diagonal, where i is the row of

elements and j is the
column of elements.
2. The difference between i and j is constant and unique for each left diagonal, where i and j are row and column of element respectively.

Implementation of Backtracking solution(with optimization)

C++CJavaPython3C#Javascript

```cpp
/* C++ program to solve N Queen Problem using
   backtracking */
#include<bits/stdc++.h>
using namespace std;
#define N 4
/* ld is an array where its indices indicate row-col+N-1
 (N-1) is for shifting the difference to store negative
 indices */
int ld[30] = { 0 };
/* rd is an array where its indices indicate row+col
   and used to check whether a queen can be placed on
   right diagonal or not*/
int rd[30] = { 0 };
/*column array where its indices indicates column and
  used to check whether a queen can be placed in that
    row or not*/
int cl[30] = { 0 };
/* A utility function to print solution */
void printSolution(int board[N][N])
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            cout<<" "<< board[i][j]<<" ";
        cout<<endl;
    }
}

/* A recursive utility function to solve N
   Queen problem */
bool solveNQUtil(int board[N][N], int col)
{
    /* base case: If all queens are placed
      then return true */
    if (col >= N)
        return true;

    /* Consider this column and try placing
       this queen in all rows one by one */
```

```
    for (int i = 0; i < N; i++) {
        /* Check if the queen can be placed on
          board[i][col] */
        /* A check if a queen can be placed on
           board[row][col].We just need to check
           ld[row-col+n-1] and rd[row+coln] where
           ld and rd are for left and right
           diagonal respectively*/
        if ((ld[i - col + N - 1] != 1 && rd[i + col] != 1) && cl[i] != 1)
{
            /* Place this queen in board[i][col] */
            board[i][col] = 1;
            ld[i - col + N - 1] = rd[i + col] = cl[i] = 1;

            /* recur to place rest of the queens */
            if (solveNQUtil(board, col + 1))
                return true;

            /* If placing queen in board[i][col]
               doesn't lead to a solution, then
               remove queen from board[i][col] */
            board[i][col] = 0; // BACKTRACK
            ld[i - col + N - 1] = rd[i + col] = cl[i] = 0;
        }
    }

    /* If the queen cannot be placed in any row in
        this column col  then return false */
    return false;
}
/* This function solves the N Queen problem using
   Backtracking. It mainly uses solveNQUtil() to
   solve the problem. It returns false if queens
   cannot be placed, otherwise, return true and
   prints placement of queens in the form of 1s.
   Please note that there may be more than one
   solutions, this function prints one  of the
   feasible solutions.*/
bool solveNQ()
{
    int board[N][N] = { { 0, 0, 0, 0 },
                        { 0, 0, 0, 0 },
                        { 0, 0, 0, 0 },
                        { 0, 0, 0, 0 } };

    if (solveNQUtil(board, 0) == false) {
        cout<<"Solution does not exist";
        return false;
    }
```

```
    printSolution(board);
    return true;
}

// driver program to test above function
int main()
{
    solveNQ();
    return 0;
}

// This code is contributed by Aditya Kumar (adityakumar129)
```

Output

```
 0   0   1   0
 1   0   0   0
 0   0   0   1
 0   1   0   0
```

Time Complexity: O(N!)
Auxiliary Space: O(N)

Printing all solutions in N-Queen Problem

Sources:

http://see.stanford.edu/materials/icspacs106b/H19-RecBacktrackExamples.pdf
http://en.literateprograms.org/Eight_queens_puzzle_%28C%29
http://en.wikipedia.org/wiki/Eight_queens_puzzle

Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

Rat in a maze | Backtracking

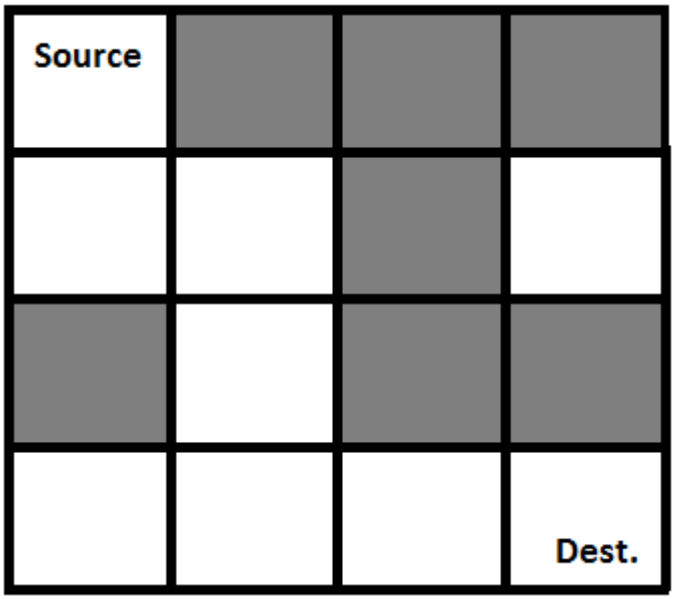
Let us discuss Rat in a Maze as another example problem that can be solved using Backtracking.

A Maze is given as N*N binary matrix of blocks where source block is the upper left most block i.e., maze[0][0] and destination block is lower rightmost block i.e., maze[N-1][N-1]. A rat starts from source and has to reach the destination. The rat can move only in two directions: forward and down.

In the maze matrix, 0 means the block is a dead end and 1 means the block can be used in the path from source to destination. Note that this is a simple version of the typical Maze problem. For example, a more complex version can be that the rat can move in 4 directions and a more complex version can be with a limited number of moves.

Following is an example maze.

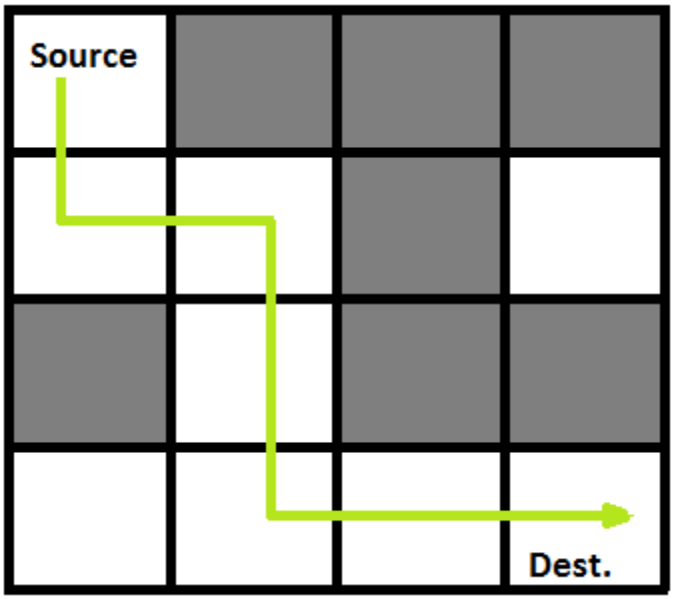
```
Gray blocks are dead ends (value = 0).
```



Following is a binary matrix representation of the above maze.

```
{1, 0, 0, 0}
{1, 1, 0, 1}
{0, 1, 0, 0}
{1, 1, 1, 1}
```

Following is a maze with highlighted solution path.



Following is the solution matrix (output of program) for the above input matrix.

```
{1, 0, 0, 0}
{1, 1, 0, 0}
{0, 1, 0, 0}
{0, 1, 1, 1}
All entries in solution path are marked as 1.
```

Backtracking Algorithm: Backtracking is an algorithmic-technique for solving problems recursively by trying to build a solution incrementally. Solving one piece at a time, and removing those solutions that fail to satisfy the constraints of the problem at any point of time (by time, here, is referred to the time elapsed till reaching any level of the search tree) is the process of backtracking.

Approach: Form a recursive function, which will follow a path and check if the path reaches the destination or not. If the path does not reach the destination then backtrack and try other paths.

Algorithm:

1. Create a solution matrix, initially filled with 0's.
2. Create a recursive function, which takes initial matrix, output matrix and position of rat (i, j).
3. if the position is out of the matrix or the position is not valid then return.
4. Mark the position output[i][j] as 1 and check if the current position is destination or not. If destination is reached print the output matrix and return.
5. Recursively call for position (i+1, j) and (i, j+1).

6. Unmark position (i, j), i.e output[i][j] = 0.

C++CJavaPython3C#Javascript

```cpp
// C++ program to solve Rat in a Maze problem using
// backtracking
#include <bits/stdc++.h>
using namespace std;
// Maze size
#define N 4

bool solveMazeUtil(int maze[N][N], int x, int y,int sol[N][N]);

// A utility function to print solution matrix sol[N][N]
void printSolution(int sol[N][N])
{
    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++)
            cout<<" "<<sol[i][j]<<" ";
        cout<<endl;
    }
}

// A utility function to check if x, y is valid index for
// N*N maze
bool isSafe(int maze[N][N], int x, int y)
{
    // if (x, y outside maze) return false
    if (x >= 0 && x < N && y >= 0 && y < N && maze[x][y] == 1)
        return true;
    return false;
}

// This function solves the Maze problem using Backtracking.
// It mainly uses solveMazeUtil() to solve the problem. It
// returns false if no path is possible, otherwise return
// true and prints the path in the form of 1s. Please note
// that there may be more than one solutions, this function
// prints one of the feasible solutions.
bool solveMaze(int maze[N][N])
{
    int sol[N][N] = { { 0, 0, 0, 0 },
                      { 0, 0, 0, 0 },
                      { 0, 0, 0, 0 },
                      { 0, 0, 0, 0 } };
    if (solveMazeUtil(maze, 0, 0, sol) == false) {
        cout<<"Solution doesn't exist";
        return false;
    }
    printSolution(sol);
    return true;
```

```
}

// A recursive utility function to solve Maze problem
bool solveMazeUtil(int maze[N][N], int x, int y, int sol[N][N])
{
    // if (x, y is goal) return true
    if (x == N - 1 && y == N - 1 && maze[x][y] == 1) {
        sol[x][y] = 1;
        return true;
    }
    // Check if maze[x][y] is valid
    if (isSafe(maze, x, y) == true) {
        // Check if the current block is already part of
        // solution path.
        if (sol[x][y] == 1)
            return false;
        // mark x, y as part of solution path
        sol[x][y] = 1;
        /* Move forward in x direction */
        if (solveMazeUtil(maze, x + 1, y, sol) == true)
            return true;
        // If moving right didn't work
        // move left
          if (solveMazeUtil(maze, x - 1, y, sol) == true)
            return true;
        // If moving in x direction doesn't give solution
        // then Move down in y direction
        if (solveMazeUtil(maze, x, y + 1, sol) == true)
            return true;
        // If moving down didn't work
        // move up
          if (solveMazeUtil(maze, x, y - 1, sol) == true)
            return true;
        // If none of the above movements work then
        // BACKTRACK: unmark x, y as part of solution path
        sol[x][y] = 0;
        return false;
    }
    return false;
}

// driver program to test above function
int main()
{
    int maze[N][N] = { { 1, 0, 0, 0 },
                       { 1, 1, 0, 1 },
                       { 0, 1, 0, 0 },
                       { 1, 1, 1, 1 } };
    solveMaze(maze);
```

```
    return 0;
}

// This code is contributed by Aditya Kumar (adityakumar129)
// Changes made for N>4 by Ch. Abdul Wasay
```

Output:

The 1 values show the path for rat

```
 1   0   0   0
 1   1   0   0
 0   1   0   0
 0   1   1   1
```

Complexity Analysis:

- **Time Complexity:** O(2^(n^2)).
  The recursion can run upper-bound 2^(n^2) times.
- **Auxiliary Space:** O(n^2).
  Output matrix is required so an extra space of size n*n is needed.

Below is an extended version of this problem. [Count number of ways to reach destination in a Maze]

Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

search a word in a grid | Backtracking

Given a 2D grid of characters and a single word/an array of words, find all occurrences of the given word/words in the grid. A word can be matched in all 8 directions at any point. Word is said to be found in a direction if all characters match in this direction (not in zig-zag form).

The 8 directions are, Horizontally Left, Horizontally Right, Vertically Up, Vertically Down and 4 Diagonal directions.

Example:

```
Input:  grid[][] = {"GEEKSFORGEEKS",
                    "GEEKSQUIZGEEK",
                    "IDEQAPRACTICE"};
        word = "GEEKS"

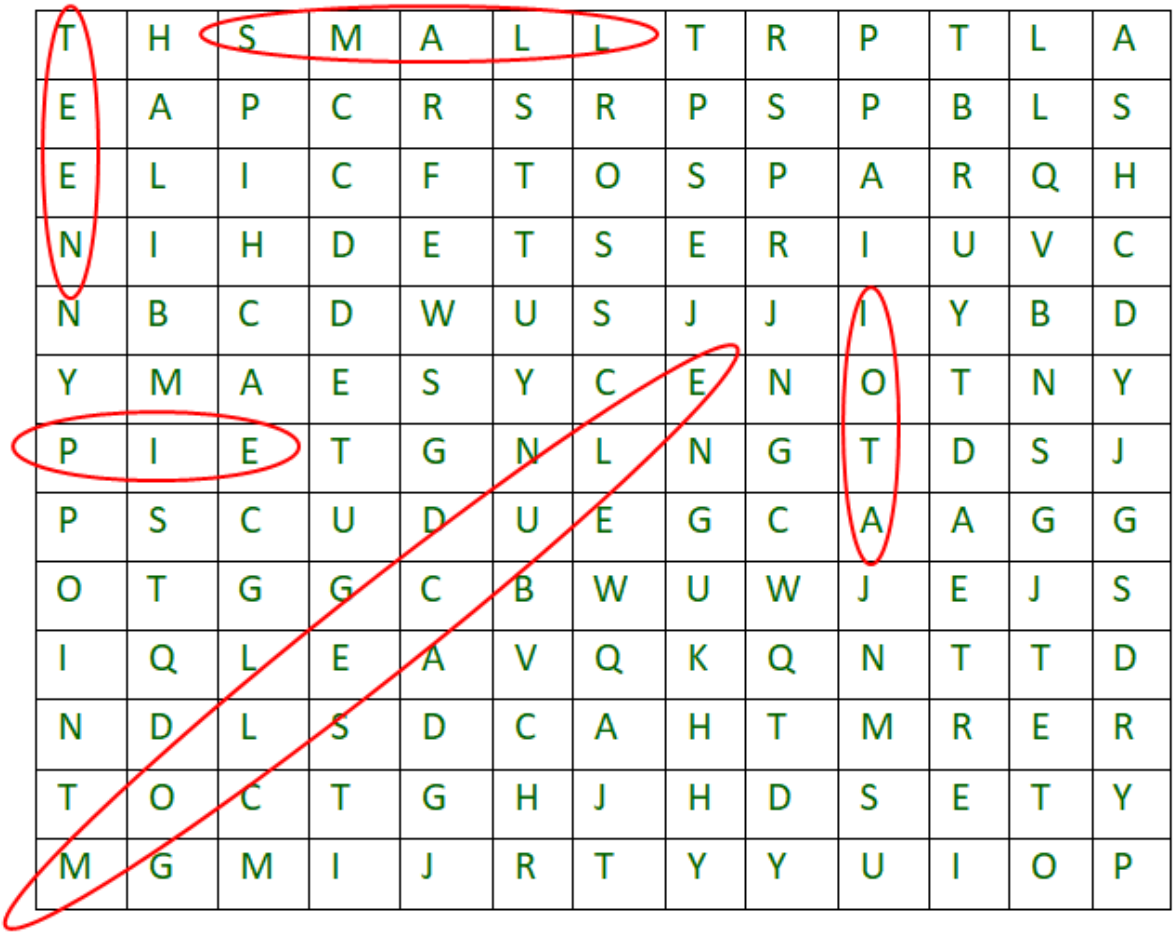
Output: pattern found at 0, 0
```

```
        pattern found at 0, 8
        pattern found at 1, 0
Explanation: 'GEEKS' can be found as prefix of
1st 2 rows and suffix of first row

Input:  grid[][] = {"GEEKSFORGEEKS",
                    "GEEKSQUIZGEEK",
                    "IDEQAPRACTICE"};
        word = "EEE"

Output: pattern found at 0, 2
        pattern found at 0, 10
        pattern found at 2, 2
        pattern found at 2, 12
Explanation: EEE can be found in first row
twice at index 2 and index 10
and in second row at 2 and 12
```

Below diagram shows a bigger grid and presence of different words in it.



Source: Microsoft Interview Question.

Approach when a single word is given: The idea used here is simple, we check every cell. If cell has first character, then we one by one try all 8 directions from that cell for a match. Implementation is interesting though. We use two arrays x[] and y[] to find next move in all 8 directions.
Below are implementation of the same:

C++JavaPython3C#Javascript

```
// C++ programs to search a word in a 2D grid
#include <bits/stdc++.h>
using namespace std;

// For searching in all 8 direction
int x[] = { -1, -1, -1,  0, 0,  1, 1, 1 };
int y[] = { -1,  0,  1, -1, 1, -1, 0, 1 };

// This function searches in
// all 8-direction from point
// (row, col) in grid[][]
bool search2D(char *grid, int row, int col,
              string word, int R, int C)
{
    // If first character of word doesn't
    // match with given starting point in grid.
    if (*(grid+row*C+col) != word[0])
        return false;

    int len = word.length();

    // Search word in all 8 directions
    // starting from (row, col)
    for (int dir = 0; dir < 8; dir++) {
        // Initialize starting point
        // for current direction
        int k, rd = row + x[dir], cd = col + y[dir];

        // First character is already checked,
        // match remaining characters
        for (k = 1; k < len; k++) {
            // If out of bound break
            if (rd >= R || rd < 0 || cd >= C || cd < 0)
                break;
```

```
            // If not matched,  break
            if (*(grid+rd*C+cd) != word[k])
                break;

            // Moving in particular direction
            rd += x[dir], cd += y[dir];
        }

        // If all character matched, then value of k must
        // be equal to length of word
        if (k == len)
            return true;
    }
    return false;
}

// Searches given word in a given
// matrix in all 8 directions
void patternSearch(char *grid, string word,
                   int R, int C)
{
    // Consider every point as starting
    // point and search given word
    for (int row = 0; row < R; row++)
        for (int col = 0; col < C; col++)
            if (search2D(grid, row, col, word, R, C))
                cout << "pattern found at "
                     << row << ", "
                     << col << endl;
}

// Driver program
int main()
{
      int R = 3, C = 13;
    char grid[R][C] = { "GEEKSFORGEEKS",
                        "GEEKSQUIZGEEK",
                        "IDEQAPRACTICE" };

    patternSearch((char *)grid, "GEEKS", R, C);
    cout << endl;
    patternSearch((char *)grid, "EEE", R, C);
    return 0;
}
```

Output

```
pattern found at 0, 0
pattern found at 0, 8
```

```
pattern found at 1, 0

pattern found at 0, 2
pattern found at 0, 10
pattern found at 2, 2
pattern found at 2, 12
```

Complexity Analysis:

- **Time complexity:** O(R*C*8*len(str)).
  All the cells will be visited and traversed in all 8 directions, where R and C is side of matrix so time complexity is O(R*C).
- **Auxiliary Space:** O(1).
  As no extra space is needed.

Approach when array of words is given: The idea used here is simple, we check every cell. If cell has first character, then we one by one try all 8 directions from that cell for a match, put this check in a loop. We use mover array to store the manner in which next moves are possible={left,right,up,down,...diagonals}.
Below are implementation of the same:

C++JavaPython3C#Javascript

```cpp
#include <bits/stdc++.h>
using namespace std;

// making a solution class to solve the problem and to keep
// the components and functions of solution together
class Solution {
public:
    // making the possible moves in movers array
    vector<vector<int> > mover
        = { { 1, 0 }, { 0, 1 },   { -1, 0 }, { 0, -1 },
            { 1, 1 }, { -1, -1 }, { 1, -1 }, { -1, 1 } };
    // making the board global variable
    vector<vector<char> > board;
    // depth first search for the string, with the
    // coordinates and a visited array to take care that we
    // do not overlap the places visited already
    bool dfs(int x, int y, string& s,
             vector<vector<bool> > vis)
    {
        // if string length becomes 0 means string is found
        if (s.length() == 0)
            return true;
        vis[x][y] = true;
        // making a solution boolean to see if we can
        // perform depth search to find answer
        bool sol = false;
        // making possible moves
```

```
        for (int i = 0; i < mover.size(); i++) {
            int curr_x = mover[i][0] + x;
            int curr_y = mover[i][1] + y;
            // checking for out of bound areas
            if (curr_x >= 0 && curr_x < board.size()) {
                if (curr_y >= 0
                    && curr_y < board[0].size()) {
                    // checking for similarity in the first
                    // letter and the visited array
                    if (board[curr_x][curr_y] == s[0]
                        && vis[curr_x][curr_y] == false) {
                        string k = s.substr(
                            1); // removing the first letter
                                // from the string
                        sol |= dfs(curr_x, curr_y, k, vis);
                    }
                }
            }
        }
        return sol;
    }
    // making a function findwords to find words along with
    // their location which inputs the board and list of
    // words
    vector<string> findWords(vector<vector<char> >& board,
                             vector<string>& words)
    {
        this->board
            = board; // making board a global variable
        vector<string> ans;
        vector<vector<bool> > vis(
            board.size(),
            vector<bool>(board[0].size(),
                         false)); // visited array
        for (auto& word : words) {
            for (int i = 0; i < board.size(); i++) {
                for (int j = 0; j < board[i].size(); j++) {
                    if (board[i][j] == word[0]) {
                        // if first letter of(i,j)==
                        // string's first letter then we can
                        // perform dfs to check the
                        // possibility of string being present
                        // from location (i,j)
                        string s = word.substr(1);
                        if (dfs(i, j, s, vis)) {
                            ans.push_back(
                                word + "->{" + to_string(i)
                                + "," + to_string(j) + "}");
                        }
```

```
                    }
                }
                if (ans.size() && ans.back() == word)
                    break;
            }
        }
        return ans;
    }
};
int main()
{
    // making 1 instance of class solution as solver
    Solution solver;
    vector<vector<char> > board
        = { { 'o', 'a', 'a', 'n' },
            { 'e', 't', 'a', 'e' },
            { 'i', 'h', 'k', 'r' },
            { 'i', 'f', 'l', 'v' } };
    vector<string> words = { "oath", "pea", "eat", "rain" };
    // using the function findwords from our solution class
    // to find the answer
    vector<string> ans = solver.findWords(board, words);
    // printing the answer
    for (auto& part : ans)
        cout << part << endl;
    return 0;
}//contributed by NamanAnand
```

Output

```
oath->{0,0}
eat->{1,0}
eat->{1,3}
```

Complexity Analysis:

- **Time complexity: O(R*C*len(str)*Number(str)*len(str))**.
  All the cells will be visited and traversed in all 8 directions, where R and C is side of matrix so time complexity is O(R*C) for each string.
- **Auxiliary Space: O(R*C*Numberof(str)*len(str))**. (due to visited array)

Another Approach:

Approach: The code implements a solution class that takes a board of characters and a list of words as input. It then performs a depth-first search to find if the words are present on the board. If a word is found, it is added to the output along with its position on the board. The code uses a visited array to keep track of the cells that

have been visited during the search. Finally, the function returns a vector of strings, where each string is a word followed by its position on the board.

Implementation is given below:

C++JavaPython3C#Javascript

```
#include <iostream>
#include <vector>
#include <string>

using namespace std;

class Solution {
private:
    vector<pair<int, int>> mover = {{1, 0}, {0, 1}, {-1, 0},
                                    {0, -1}, {1, 1}, {-1, -1},
                                    {1, -1}, {-1, 1}};

public:
    bool dfs(int x, int y, string word,
             vector<vector<bool>>& visited,
             vector<vector<char>>& board) {

        // If word length becomes 0, the string is found
        if (word.empty()) {
            return true;
        }

        visited[x][y] = true;
        bool sol = false;

        // Making possible moves
        for (auto move : mover) {
            int curr_x = move.first + x;
            int curr_y = move.second + y;

            // Checking for out of bound areas
            if (0 <= curr_x && curr_x < board.size() && 0 <= curr_y &&
curr_y < board[0].size()) {

              // Checking for similarity in the first letter and the
visited array
                if (board[curr_x][curr_y] == word[0] &&
!visited[curr_x][curr_y]) {
                    string s = word.substr(1);
                    sol |= dfs(curr_x, curr_y, s, visited, board);
                }
            }
```

```
        }

        visited[x][y] = false;
        return sol;
    }

    vector<string> findWords(vector<vector<char>>& board, vector<string>&
words) {
        vector<string> ans;
        vector<vector<bool>> visited(board.size(),
vector<bool>(board[0].size(), false));

        for (string word : words) {
            for (int i = 0; i < board.size(); i++) {
                for (int j = 0; j < board[0].size(); j++) {
                    if (board[i][j] == word[0]) {
                        string s = word.substr(1);
                        if (dfs(i, j, s, visited, board)) {
                            ans.push_back(word + " -> {" + to_string(i) +
"," + to_string(j) + "}");
                        }
                    }
                }
            }
        }

        return ans;
    }
};

int main() {
    vector<vector<char>> board = {{'o', 'a', 'a', 'n'},
                                  {'e', 't', 'a', 'e'},
                                  {'i', 'h', 'k', 'r'},
                                  {'i', 'f', 'l', 'v'}};
    vector<string> words = {"oath", "pea", "eat", "rain"};
    Solution solver;
    vector<string> result = solver.findWords(board, words);

    for (string str : result) {
        cout << str << endl;
    }

    return 0;
}
```

Output

```
['oath -> {0,0}', 'eat -> {1,0}', 'eat -> {1,3}']
```

The time complexity of the findWords function is O(N * M * L * 8^L), where N is the number of rows in the board, M is the number of columns in the board, L is the maximum length of the word in the dictionary. For each starting point on the board, the algorithm may explore up to 8 directions, and at each step, it needs to check if the next letter in the word matches the character in the board. Since the maximum length of the word is L, this gives a factor of L in the time complexity.

The space complexity of the algorithm is O(N * M + L), where N and M are the dimensions of the board, and L is the length of the longest word in the dictionary. The space is used to store the visited array, which has the same size as the board, and the recursion stack, which can be as deep as the length of the longest word in the dictionary.

Exercise: The above solution only print locations of word. Extend it to print the direction where word is present.
See this for solution of exercise.
This article is contributed by **Utkarsh Trivedi**. Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.
Odd-Even

Given number of digits n, print all n-digit numbers whose absolute difference between sum of digits at even and odd positions is 1. Solution should not consider leading 0's as digits.
Examples:

```
Input:  n = 2
Output:
10 12 21 23 32 34 43 45 54 56 65 67 76 78 87 89 98

Input:  n = 3
Output:
100 111 120 122 131 133 142 144 153 155 164 166 175 177 186
188 197 199 210 221 230 232 241 243 252 254 263 265 274 276
285 287 296 298 320 331 340 342 351 353 362 364 373 375 384
386 395 397 430 441 450 452 461 463 472 474 483 485 494 496
540 551 560 562 571 573 582 584 593 595 650 661 670 672 681
683 692 694 760 771 780 782 791 793 870 881 890 892 980 991
```

The idea is to generate all n-digit numbers using recursion and maintain sum of even and odd digits so far in two separate variables. For a given position, we fill it with all digits from 0 to 9 and based on whether current position is even or odd, we increment even or odd sum. We handle leading 0's case separately as they are not counted as digits.
We have followed Zero-based numbering like array indexes. i.e. leading(leftmost) digit is considered to be present in even position and digit next to it is considered at odd position and so on.
Below is implementation of above idea –

C++JavaPython 3C#PHPJavascript

```
// A C++ recursive program to print all N-digit numbers with
// absolute difference between sum of even and odd digits is 1
#include <bits/stdc++.h>
using namespace std;

// Recursive function to print all N-digit numbers with absolute
// difference between sum of even and odd digits is 1.
// This function considers leading zero as a digit

// n --> value of input
// out --> output array
// index --> index of next digit to be filled in output array
// evenSum, oddSum --> sum of even and odd digits so far
void findNDigitNumsUtil(int n, char* out, int index, int evenSum,
                                                     int oddSum)
{
    // Base case
    if (index > n)
        return;

    // If number becomes n-digit
    if (index == n)
    {
        // if absolute difference between sum of even and
        // odd digits is 1, print the number
        if (abs(evenSum - oddSum) == 1)
        {
            out[index] = ' ';
            cout << out;
        }
        return;
    }

    // If current index is odd, then add it to odd sum and recurse
    if (index & 1)
```

```
    {
        for (int i = 0; i <= 9; i++)
        {
            out[index] = i + '0';
            findNDigitNumsUtil(n, out, index + 1, evenSum, oddSum + i);
        }
    }
    else // else add to even sum and recurse
    {
        for (int i = 0; i <= 9; i++)
        {
            out[index] = i + '0';
            findNDigitNumsUtil(n, out, index + 1, evenSum + i, oddSum);
        }
    }
}

// This is mainly a wrapper over findNDigitNumsUtil.
// It explicitly handles leading digit and calls
// findNDigitNumsUtil() for remaining indexes.
int findNDigitNums(int n)
{
    // output array to store n-digit numbers
    char out[n + 1];

    // Initialize number index considered so far
    int index = 0;

    // Initialize even and odd sums
    int evenSum = 0, oddSum = 0;

    // Explicitly handle first digit and call recursive function
    // findNDigitNumsUtil for remaining indexes. Note that the
    // first digit is considered to be present in even position.
    for (int i = 1; i <= 9; i++)
    {
        out[index] = i + '0';
        findNDigitNumsUtil(n, out, index + 1, evenSum + i, oddSum);
    }
}

// Driver program
int main()
{
    int n = 3;

    findNDigitNums(n);

    return 0;
```

```
}
```

Output:

```
100 111 120 122 131 133 142 144 153 155 164 166 175 177 186
188 197 199 210 221 230 232 241 243 252 254 263 265 274 276
285 287 296 298 320 331 340 342 351 353 362 364 373 375 384
386 395 397 430 441 450 452 461 463 472 474 483 485 494 496
540 551 560 562 571 573 582 584 593 595 650 661 670 672 681
683 692 694 760 771 780 782 791 793 870 881 890 892 980 991
```

We can avoid using two variables evenSum and oddSum. Instead we can maintain single variable diff that stores difference between sum of even and odd digits. The implementation can be seen here.

The Knight Tour | BackTracking

Backtracking is an algorithmic paradigm that tries different solutions until finds a solution that "works". Problems that are typically solved using the backtracking technique have the following property in common. These problems can only be solved by trying every possible configuration and each configuration is tried only once. A Naive solution for these problems is to try all configurations and output a configuration that follows given problem constraints. Backtracking works incrementally and is an optimization over the Naive solution where all possible configurations are generated and tried.
For example, consider the following Knight's Tour problem.

Problem Statement:
Given a N*N board with the Knight placed on the first block of an empty board. Moving according to the rules of chess knight must visit each square exactly once. Print the order of each cell in which they are visited.

Example:

```
Input :
N = 8
Output:
0  59  38  33  30  17   8  63
37  34  31  60   9  62  29  16
58   1  36  39  32  27  18   7
```

```
35   48   41   26   61   10   15   28
42   57    2   49   40   23    6   19
47   50   45   54   25   20   11   14
56   43   52    3   22   13   24    5
51   46   55   44   53    4   21   12
```

The path followed by Knight to cover all the cells

Following is a chessboard with 8 x 8 cells. Numbers in cells indicate the move number of Knight.

0	59	38	33	30	17	8	63
37	34	31	60	9	62	29	16
58	1	36	39	32	27	18	7
35	48	41	26	61	10	15	28
42	57	2	49	40	23	6	19
47	50	45	54	25	20	11	14
56	43	52	3	22	13	24	5
51	46	55	44	53	4	21	12

Let us first discuss the Naive algorithm for this problem and then the Backtracking algorithm.

Naive Algorithm for Knight's tour

The Naive Algorithm is to generate all tours one by one and check if the generated tour satisfies the constraints.

```
while there are untried tours
{
   generate the next tour
   if this tour covers all squares
   {
      print this path;
   }
}
```

Backtracking works in an incremental way to attack problems. Typically, we start from an empty solution vector and one by one add items (Meaning of item varies from problem to problem. In the context of Knight's tour problem, an item is a Knight's move). When we add an item, we check if adding the current item violates the problem constraint, if it does then we remove the item and try other alternatives. If none of the alternatives works out then we go to the previous stage and remove the item added in the previous stage. If we reach

the initial stage back then we say that no solution exists. If adding an item doesn't violate constraints then we recursively add items one by one. If the solution vector becomes complete then we print the solution.

Backtracking Algorithm for Knight's tour

Following is the Backtracking algorithm for Knight's tour problem.

```
If all squares are visited
    print the solution
Else
   a) Add one of the next moves to solution vector and
recursively
   check if this move leads to a solution. (A Knight can make
maximum
   eight moves. We choose one of the 8 moves in this step).
   b) If the move chosen in the above step doesn't lead to a
solution
   then remove this move from the solution vector and try other
   alternative moves.
   c) If none of the alternatives work then return false
(Returning false
   will remove the previously added item in recursion and if
false is
   returned by the initial call of recursion then "no solution
exists" )
```

Following are implementations for Knight's tour problem. It prints one of the possible solutions in 2D matrix form. Basically, the output is a 2D 8*8 matrix with numbers from 0 to 63 and these numbers show steps made by Knight.

C++CJavaPython3C#Javascript

```cpp
// C++ program for Knight Tour problem
#include <bits/stdc++.h>
using namespace std;

#define N 8

int solveKTUtil(int x, int y, int movei, int sol[N][N],
                int xMove[], int yMove[]);

/* A utility function to check if i,j are
valid indexes for N*N chessboard */
int isSafe(int x, int y, int sol[N][N])
{
    return (x >= 0 && x < N && y >= 0 && y < N
```

```
            && sol[x][y] == -1);
}

/* A utility function to print
solution matrix sol[N][N] */
void printSolution(int sol[N][N])
{
    for (int x = 0; x < N; x++) {
        for (int y = 0; y < N; y++)
            cout << " " << setw(2) << sol[x][y] << " ";
        cout << endl;
    }
}

/* This function solves the Knight Tour problem using
Backtracking. This function mainly uses solveKTUtil()
to solve the problem. It returns false if no complete
tour is possible, otherwise return true and prints the
tour.
Please note that there may be more than one solutions,
this function prints one of the feasible solutions. */
int solveKT()
{
    int sol[N][N];

    /* Initialization of solution matrix */
    for (int x = 0; x < N; x++)
        for (int y = 0; y < N; y++)
            sol[x][y] = -1;

    /* xMove[] and yMove[] define next move of Knight.
    xMove[] is for next value of x coordinate
    yMove[] is for next value of y coordinate */
    int xMove[8] = { 2, 1, -1, -2, -2, -1, 1, 2 };
    int yMove[8] = { 1, 2, 2, 1, -1, -2, -2, -1 };

    // Since the Knight is initially at the first block
    sol[0][0] = 0;

    /* Start from 0,0 and explore all tours using
    solveKTUtil() */
    if (solveKTUtil(0, 0, 1, sol, xMove, yMove) == 0) {
        cout << "Solution does not exist";
        return 0;
    }
    else
        printSolution(sol);

    return 1;
```

```
}

/* A recursive utility function to solve Knight Tour
problem */
int solveKTUtil(int x, int y, int movei, int sol[N][N],
                int xMove[8], int yMove[8])
{
    int k, next_x, next_y;
    if (movei == N * N)
        return 1;

    /* Try all next moves from
    the current coordinate x, y */
    for (k = 0; k < 8; k++) {
        next_x = x + xMove[k];
        next_y = y + yMove[k];
        if (isSafe(next_x, next_y, sol)) {
            sol[next_x][next_y] = movei;
            if (solveKTUtil(next_x, next_y, movei + 1, sol,
                            xMove, yMove)
                == 1)
                return 1;
            else

                // backtracking
                sol[next_x][next_y] = -1;
        }
    }
    return 0;
}

// Driver Code
int main()
{
      // Function Call
    solveKT();
    return 0;
}

// This code is contributed by ShubhamCoder
```

Output

```
  0  59  38  33  30  17   8  63
 37  34  31  60   9  62  29  16
 58   1  36  39  32  27  18   7
 35  48  41  26  61  10  15  28
 42  57   2  49  40  23   6  19
 47  50  45  54  25  20  11  14
```

```
 56   43   52    3   22   13   24    5
 51   46   55   44   53    4   21   12
```

Time Complexity :
There are N^2 Cells and for each, we have a maximum of 8 possible moves to choose from, so the worst running time is $O(8^{N^2})$.

Auxiliary Space: $O(N^2)$

Important Note:
No order of the xMove, yMove is wrong, but they will affect the running time of the algorithm drastically. For example, think of the case where the 8th choice of the move is the correct one, and before that our code ran 7 different wrong paths. It's always a good idea a have a heuristic than to try backtracking randomly. Like, in this case, we know the next step would probably be in the south or east direction, then checking the paths which lead their first is a better strategy.

Note that Backtracking is not the best solution for the Knight's tour problem. See the below article for other better solutions. The purpose of this post is to explain Backtracking with an example.
Warnsdorff's algorithm for Knight's tour problem

Divide and Conquer – CP

Binary Search

Binary Search

Binary Search is defined as a searching algorithm used in a sorted array by **repeatedly dividing the search interval in half**. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to O(log N).



Introduction to Binary Search

Conditions for when to apply Binary Search in a Data Structure:

To apply binary search in any data structure, the data structure must maintain the following properties:

- The data structure must be sorted.
- Access to any element of the data structure takes constant time.

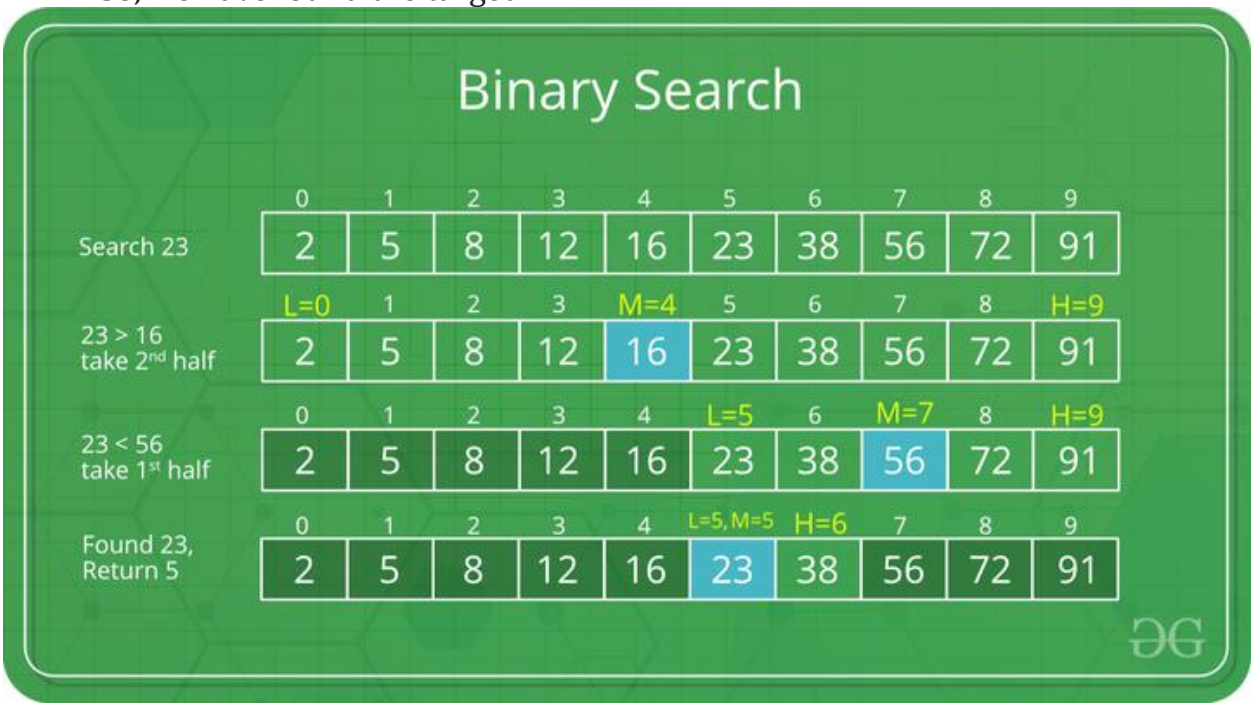
How does Binary Search work?

To understand the working of binary search, consider the following illustration:

Consider an array **arr[] = {2, 5, 8, 12, 16, 23, 38, 56, 72, 91}**, and the **target = 23**.

- **First Step:**
  - Initially the search space is from 0 to 9.
  - Let's denote the boundary by **L** and **H** where **L = 0** and **H = 9** initially.
  - Now mid of this search space is **M = 4**.
  - So compare **target** with **arr[M]**.
- **Second Step:**
  - As arr[4] is less than **target**, switch the search space to the right of 16, i.e., [5, 9].
  - Now **L = 5**, **H = 9** and **M** becomes **7**.
  - Compare target with **arr[M]**.

- **Third Step:**
  - arr[7] is greater than **target**.
  - Shift the search space to the left of **M**, i.e., [5, 6].
  - So, now **L = 5**, **H = 6** and **M = 6**.
  - Compare arr[M] with target.
  - Here **arr[M]** and target are the same.
- So, we have found the target.



Example of Binary Search Algorithm

When to use Binary Search?

- When searching a large dataset as it has a time complexity of O(log n), which means that it is much faster than linear search.
- When the dataset is sorted.
- When data is stored in contiguous memory.
- Data does not have a complex structure or relationships.

How to Implement Binary Search?

The basic steps to perform Binary Search are:

1. Set the low index to the first element of the array and the high index to the last element.
2. Set the middle index to the average of the low and high indices.
   - If the element at the middle index is the target element, return the middle index.
   - Otherwise, based on the value of the key to be found and the value of the middle element, decide the next search space.

- If the target is less than the element at the middle index, set the high index to **middle index - 1**.
- If the target is greater than the element at the middle index, set the low index to **middle index + 1**.

3. Perform step 2 repeatedly until the target element is found or the search space is exhausted.

The **Binary Search Algorithm** can be implemented in the following two ways

- Iterative Binary Search Algorithm
- Recursive Binary Search Algorithm

Given below are the pseudocodes for the approaches.

Iterative Binary Search Algorithm:

Pseudocode of Iterative Binary Search Algorithm:

```
binarySearch(arr, x, low, high)
  repeat till low = high
      mid = (low + high)/2
      if (x = arr[mid])
        return mid
      else if (x > arr[mid])
        low = mid + 1
      else
        high = mid - 1
```

Implementation of Iterative Binary Search Algorithm:

C++Java

```cpp
// C++ program to implement iterative Binary Search
#include <bits/stdc++.h>
using namespace std;

// A iterative binary search function. It returns
// location of x in given array arr[l..r] if present,
// otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
    while (l <= r) {
        int m = l + (r - l) / 2;

        // Check if x is present at mid
        if (arr[m] == x)
            return m;

        // If x greater, ignore left half
        if (arr[m] < x)
            l = m + 1;
```

```
        // If x is smaller, ignore right half
        else
            r = m - 1;
    }

    // if we reach here, then element was
    // not present
    return -1;
}

int main(void)
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = binarySearch(arr, 0, n - 1, x);
    (result == -1)
        ? cout << "Element is not present in array"
        : cout << "Element is present at index " << result;
    return 0;
}
```

Output

```
Element is present at index 3
```

Time Complexity: O(log N)
Auxiliary Space: O(1)

Recursive Binary Search Algorithm:

Pseudocode of Recursive Binary Search Algorithm:

```
binarySearch(arr, x, low, high)
    if low > high
      return False
    else
      mid = (low + high) / 2
      if x = arr[mid]
        return mid
      else if x > arr[mid]
        return binarySearch(arr, x, mid + 1, high)
      else
        return binarySearch(arr, x, low, mid - 1)
```

Implementation of Recursive Binary Search Algorithm:
C++Java

```cpp
// C++ program to implement recursive Binary Search
#include <bits/stdc++.h>
using namespace std;

// A recursive binary search function. It returns
// location of x in given array arr[l..r] is present,
// otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
    if (r >= l) {
        int mid = l + (r - l) / 2;

        // If the element is present at the middle
        // itself
        if (arr[mid] == x)
            return mid;

        // If element is smaller than mid, then
        // it can only be present in left subarray
        if (arr[mid] > x)
            return binarySearch(arr, l, mid - 1, x);

        // Else the element can only be present
        // in right subarray
        return binarySearch(arr, mid + 1, r, x);
    }

    // We reach here when element is not
    // present in array
    return -1;
}

int main()
{
    int arr[] = { 2, 3, 4, 10, 40 };
    int x = 10;
    int n = sizeof(arr) / sizeof(arr[0]);
    int result = binarySearch(arr, 0, n - 1, x);
    (result == -1)
        ? cout << "Element is not present in array"
        : cout << "Element is present at index " << result;
    return 0;
}
```

Output

```
Element is present at index 3
```

Time Complexity: O(logN)
Auxiliary Space: O(1), If the recursive call stack is considered then the auxiliary space will be O(logN).

Binary Search Complexity Analysis:

The time and space complexities of the binary search algorithm are mentioned below.

Time Complexity of Binary Search Algorithm:

- **Best Case:** O(1)

Best case is when the element is at the middle index of the array. It takes only one comparison to find the target element. So the best case complexity is **O(1)**.

- **Average Case:** O(log N)

Consider array **arr[]** of length **N** and element **X** to be found. There can be two cases:

- Case1: Element is present in the array
- Case2: Element is not present in the array.

There are **N** Case1 and **1** Case2. So total number of cases = **N+1**. Now notice the following:

- An element at index N/2 can be found in **1** comparison
- Elements at index N/4 and 3N/4 can be found in **2** comparisons.
- Elements at indices N/8, 3N/8, 5N/8 and 7N/8 can be found in **3** comparisons and so on.

Based on this we can conclude that elements that require:

- 1 comparison = 1
- 2 comparisons = 2
- 3 comparisons = 4
- **x** comparisons = $\mathbf{2^{x-1}}$ where **x** belongs to the range **[1, logN]** because maximum comparisons = maximum time N can be halved = maximum comparisons to reach 1st element = logN.

So, total comparisons
= 1*(elements requiring 1 comparisons) + 2*(elements requiring 2 comparisons) + . . . + logN*(elements requiring logN comparisons)
= $1*1 + 2*2 + 3*4 + \ldots + \log N * (2^{\log N-1})$
= $2^{\log N} * (\log N - 1) + 1$
= **N * (logN - 1) + 1**

Total number of cases = **N+1**.

Therefore, the average complexity = (**N*(logN - 1) + 1)/N+1** = **N*logN / (N+1) + 1/(N+1)**. Here the dominant term is N*logN/(N+1) which is approximately **logN**. So the average case complexity is **O(logN)**

- **Worst Case:** O(log N)

The worst case will be when the element is present in the first position. As seen in the average case, the comparison required to reach the first element is **logN**. So the time complexity for the worst case is **O(logN)**.

Auxiliary Space Complexity of Binary Search Algorithm

- **O(1)** as no extra space is used

Correct way to calculate "mid" in Binary Search:

Here to calculate "**mid**" we are doing the following:

int mid = low + (high - low)/2;

Maybe, you wonder why we are calculating the ***middle index*** this way, we can simply add the **lower** and **higher** index and divide it by 2 like

int mid = (low + high)/2;

But if we calculate the **middle index** like this, it fails for larger values of **int** variables. Specifically, it fails if the sum of **low** and **high** is greater than the maximum positive **int** value($2^{31} - 1$). The sum overflows to a negative value and the value stays negative when divided by 2.

So it's better to calculate "**mid**" in the way it is done in the above code.

Advantages of Binary Search:

- Binary search is faster than linear search, especially for large arrays. As the size of the array increases, the time it takes to perform a linear search increases linearly, while the time it takes to perform a binary search increases logarithmically.
- Binary search is more efficient than other searching algorithms that have a similar time complexity, such as interpolation search or exponential search.
- Binary search is relatively simple to implement and easy to understand, making it a good choice for many applications.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.
- Binary search can be used as a building block for more complex algorithms, such as those used in computer graphics and machine learning.

Drawbacks of Binary Search:

- We require the array to be sorted. If the array is not sorted, we must first sort it before performing the search. This adds an additional O(N * logN) time complexity for the sorting step, which makes it irrelevant to use binary search.
- Binary search requires that the data structure being searched be stored in contiguous memory locations. This can be a problem if the data structure is too large to fit in memory, or if it is stored on external memory such as a hard drive or in the cloud.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered. This can be a problem if the elements of the array are not naturally ordered, or if the ordering is not well-defined.
- Binary search can be less efficient than other algorithms, such as hash tables, for searching very large datasets that do not fit in memory.

Applications of Binary Search:

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- Commonly used in Competitive Programming.
- Can be used for searching in computer graphics. Binary search can be used as a building block for more complex algorithms used in computer graphics, such as algorithms for ray tracing or texture mapping.
- Can be used for searching a database. Binary search can be used to efficiently search a database of records, such as a customer database or a product catalog.

Conclusion:

To wrap it up, it can be said that Binary search is an efficient algorithm for finding an element within a sorted array. The time complexity of the binary search is O(log n). Though, one of the main drawbacks of binary search is that the array must be sorted. this is a useful algorithm for building more complex algorithms in computer graphics and machine learning or several other cases.

Binary Search functions in C++ STL (binary_search, lower_bound and upper_bound)

Binary search is an important component in competitive programming or any algorithmic competition, having knowledge of shorthand functions reduces the time to code them. Binary search is the most efficient search algorithm.

Binary Search *is a searching algorithm used in a* ***sorted*** *array by repeatedly dividing the search interval in half. The idea of binary search is to use the information that the array is sorted and reduce the time complexity to* ***O(Log N).***

General operations performed using binary search:

1. finding an element
2. lower_bound
3. upper_bound

1. binary_search:

binary_search(start_ptr, end_ptr, num): This function returns true if the element is present in the container, else returns false. The **start_ptr** variable holds the starting point of the binary search and **end_ptr** holds the ending position of binary search space and **num** is the value to be found.

Coding implementation of binary_search function:
CPP

```cpp
// C++ code to demonstrate the working of binary_search()

#include <bits/stdc++.h>
using namespace std;

// Driver's code
int main()
{
    // initializing vector of integers
    vector<int> arr = { 10, 15, 20, 25, 30, 35 };

    // using binary_search to check if 15 exists
    if (binary_search(arr.begin(), arr.end(), 15))
        cout << "15 exists in vector";
    else
        cout << "15 does not exist";

    cout << endl;

    // using binary_search to check if 23 exists
    if (binary_search(arr.begin(), arr.end(), 23))
        cout << "23 exists in vector";
    else
        cout << "23 does not exist";

    cout << endl;
}
```

Output

```
15 exists in vector
23 does not exist
```

Time Complexity: O(log N) - where N is the number of elements in the array.
Auxiliary Space: O(1)

2. lower_bound:

lower_bound(start_ptr, end_ptr, num): Returns pointer to the position of num if the container contains only **one occurrence** of num. Returns a pointer to the first position of num if the container contains **multiple occurrences** of num. Returns pointer to the position of a number just higher than num, if the container **does not contain an occurrence** of num which is the position of the number when inserted in the already sorted array and sorted again. Subtracting the first position i.e vect.begin() from the pointer, returns the actual index. The **start_ptr** variable holds the starting point of the binary search and **end_ptr** holds the ending position of binary search space and **num** is the value to be found.

Coding implementation of lower_bound function:
CPP

```cpp
// C++ code to demonstrate the working of lower_bound()
#include <bits/stdc++.h>
using namespace std;

// Driver's code
int main()
{
    // initializing vector of integers
    // for single occurrence
    vector<int> arr1 = { 10, 15, 20, 25, 30, 35 };

    // initializing vector of integers
    // for multiple occurrences
    vector<int> arr2 = { 10, 15, 20, 20, 25, 30, 35 };

    // initializing vector of integers
    // for no occurrence
    vector<int> arr3 = { 10, 15, 25, 30, 35 };

    // using lower_bound() to check if 20 exists
    // single occurrence
    // prints 2
    cout << "The position of 20 using lower_bound "
            " (in single occurrence case) : ";
    cout << lower_bound(arr1.begin(), arr1.end(), 20)
                - arr1.begin();
```

```
    cout << endl;

    // using lower_bound() to check if 20 exists
    // multiple occurrence
    // prints 2
    cout << "The position of 20 using lower_bound "
            "(in multiple occurrence case) : ";
    cout << lower_bound(arr2.begin(), arr2.end(), 20)
                - arr2.begin();

    cout << endl;

    // using lower_bound() to check if 20 exists
    // no occurrence
    // prints 2 ( index of next higher)
    cout << "The position of 20 using lower_bound "
            "(in no occurrence case) : ";
    cout << lower_bound(arr3.begin(), arr3.end(), 20)
                - arr3.begin();

    cout << endl;
}
```

Output

```
The position of 20 using lower_bound  (in single occurrence
case) : 2
The position of 20 using lower_bound (in multiple occurrence
case) : 2
The position of 20 using lower_bound (in no occurrence case) : 2
```

Time Complexity: O(log N) - where N is the number of elements in the array.
Auxiliary Space: O(1)

3. upper_bound:

upper_bound(start_ptr, end_ptr, num): Returns pointer to the position of next higher number than num if the container contains **one occurrence** of num. Returns pointer to the first position of the next higher number than the last occurrence of num if the container contains **multiple occurrences** of num. Returns pointer to position of next higher number than num if the container **does not contain an occurrence** of num. Subtracting the first position i.e vect.begin() from the pointer, returns the actual index. The **start_ptr** variable holds the starting point of the binary search and **end_ptr** holds the ending position of binary search space and **num** is the value to be found.

Coding implementation of upper_bound function:
CPP

```cpp
// C++ code to demonstrate the working of upper_bound()
#include <bits/stdc++.h>
using namespace std;

// Driver's code
int main()
{
    // initializing vector of integers
    // for single occurrence
    vector<int> arr1 = { 10, 15, 20, 25, 30, 35 };

    // initializing vector of integers
    // for multiple occurrences
    vector<int> arr2 = { 10, 15, 20, 20, 25, 30, 35 };

    // initializing vector of integers
    // for no occurrence
    vector<int> arr3 = { 10, 15, 25, 30, 35 };

    // using upper_bound() to check if 20 exists
    // single occurrence
    // prints 3
    cout << "The position of 20 using upper_bound"
            " (in single occurrence case) : ";
    cout << upper_bound(arr1.begin(), arr1.end(), 20)
                - arr1.begin();

    cout << endl;

    // using upper_bound() to check if 20 exists
    // multiple occurrence
    // prints 4
    cout << "The position of 20 using upper_bound "
            "(in multiple occurrence case) : ";
    cout << upper_bound(arr2.begin(), arr2.end(), 20)
                - arr2.begin();

    cout << endl;

    // using upper_bound() to check if 20 exists
    // no occurrence
    // prints 2 ( index of next higher)
    cout << "The position of 20 using upper_bound"
            " (in no occurrence case) : ";
    cout << upper_bound(arr3.begin(), arr3.end(), 20)
                - arr3.begin();
```

```
    cout << endl;
}
```

Output

```
The position of 20 using upper_bound (in single occurrence case)
: 3
The position of 20 using upper_bound (in multiple occurrence
case) : 4
The position of 20 using upper_bound (in no occurrence case) : 2
```

Time Complexity: O(log N) - where N is the number of elements in the array.
Auxiliary Space: O(1)

Binary Search in Java

Binary search is one of the searching techniques applied when the input is sorted as here we are focusing on finding the middle element that acts as a reference frame whether to go left or right to it as the elements are already sorted. This searching helps in optimizing the search technique with every iteration is referred to as binary search and readers do stress over it as it is indirectly applied in solving questions. Now you must be thinking what if the input is not sorted then the results are undefined.

Note: If there are duplicates, there is no guarantee which one will be found.
Now let us adhere to the ***significant value of the negative value returned by both functions***?

The function returns an index of the search key, if it is contained in the array; otherwise, (-(insertion point) – 1). The insertion point is defined as the point at which the key would be inserted into the array: the index of the first element greater than the key, or a.length if all elements in the array are less than the specified key. Note that this guarantees that the return value will be >= 0 if and only if the key is found.

Implementation of Binary Search in Java

Java

```
// Java implementation of recursive Binary Search
class BinarySearch
{
```

```
    // Returns index of x if it is present in arr[l..
    // r], else return -1
    int binarySearch(int arr[], int l, int r, int x)
    {
        if (r>=l)
        {
            int mid = l + (r - l)/2;

            // If the element is present at the
            // middle itself
            if (arr[mid] == x)
               return mid;

            // If element is smaller than mid, then
            // it can only be present in left subarray
            if (arr[mid] > x)
               return binarySearch(arr, l, mid-1, x);

            // Else the element can only be present
            // in right subarray
            return binarySearch(arr, mid+1, r, x);
        }

        // We reach here when element is not present
        //  in array
        return -1;
    }

    // Driver method to test above
    public static void main(String args[])
    {
        BinarySearch ob = new BinarySearch();
        int arr[] = {2,3,4,10,40};
        int n = arr.length;
        int x = 10;
        int result = ob.binarySearch(arr,0,n-1,x);
        if (result == -1)
            System.out.println("Element not present");
        else
            System.out.println("Element found at index " +
                                                   result);
    }
}
```

Output:

```
Element found at index 3
```

Tip: Geeks you must be wondering out whether there is any function like lower_bound() or upper_bound() just likely found in C++ STL. so the straight answer is that there was no function only till Java 9, later onwards they were added.

Types of Binary Search in Java

There are two ways to do a binary search in Java

- Arrays.binarysearch
- Collections.binarysearch

Type 1: Arrays.binarysearch()

It works for arrays which can be of primitive data type also.

Example:

Java

```
// Java Program to demonstrate working of binarySearch()
// Method of Arrays class In a sorted array

// Importing required classes
import java.util.Arrays;

// Main class
public class GFG {

    // Main driver method
    public static void main(String[] args)
    {
        // Declaring an integer array
        int arr[] = { 10, 20, 15, 22, 35 };

        // Sorting the above array
        // using sort() method of Arrays class
        Arrays.sort(arr);

        int key = 22;
        int res = Arrays.binarySearch(arr, key);

        if (res >= 0)
            System.out.println(
                key + " found at index = " + res);
        else
            System.out.println(key + " Not found");
```

```
        key = 40;
        res = Arrays.binarySearch(arr, key);
        if (res >= 0)
            System.out.println(
                key + " found at index = " + res);
        else
            System.out.println(key + " Not found");
    }
}
```

Output:

```
22 found at index = 3
40 Not found
```

Now let us see how does Collections.binarySearch() work for LinkedList. So basically as discussed above this method runs in log(n) time for a "random access" list like ArrayList. If the specified list does not implement the RandomAccess interface and is large, this method will do an iterator-based binary search that performs O(n) link traversals and O(log n) element comparisons.

Type 2: Collections.binarysearch()

It works for objects Collections like ArrayList and LinkedList.

Example

Java

```
// Java Program to Demonstrate Working of binarySearch()
// method of Collections class

// Importing required classes
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

// Main class
public class GFG {
    // Main driver method
    public static void main(String[] args)
    {
        // Creating an empty ArrayList of integer type
        List<Integer> al = new ArrayList<Integer>();
```

```
        // Populating the Arraylist
        al.add(1);
        al.add(2);
        al.add(3);
        al.add(10);
        al.add(20);

        // 10 is present at index 3
        int key = 10;
        int res = Collections.binarySearch(al, key);

        if (res >= 0)
            System.out.println(
                key + " found at index = " + res);
        else
            System.out.println(key + " Not found");

        key = 15;
        res = Collections.binarySearch(al, key);

        if (res >= 0)
            System.out.println(
                key + " found at index = " + res);
        else
            System.out.println(key + " Not found");
    }
}
```

Output:

```
10 found at index = 3
15 Not found
```

Time complexity: O(logn)

Auxiliary space: O(1)

Binary Search on Real Number Theory

Given two number x and n, find n-th root of x.

Examples:

Input : 5 2
Output : 2.2360679768025875

Input : x = 5, n = 3
Output : 1.70997594668

In order to calculate n^{th} root of a number, we can use the following procedure.

1. If x lies in the range **[0, 1)** then we set the lower limit **low = x** and upper limit **high = 1**, because for this range of numbers the nth root is always greater than the given number and can never exceed 1.
   eg- $0.09=0.3$.
2. Otherwise, we take **low = 1** and **high = x**.
3. Declare a variable named **epsilon** and initialize it for accuracy you need.
   Say epsilon=0.01, then we can guarantee that our guess for nth root of the given number will be
   correct up to 2 decimal places.
4. Declare a variable guess and initialize it to **guess=(low+high)/2.**
5. Run a loop such that:
    - if the **absolute error** of our **guess** is more than **epsilon** then do:
        1. if **$guess^n$ > x**, then **high=guess**
        2. else **low=guess**
        3. Making a new better **guess** i.e., **guess=(low+high)/2.**
    - If the **absolute error** of our **guess** is less than epsilon then exit the loop.

Absolute Error: Absolute Error can be calculated as **abs($guess^n$ -x)**

C++Java

```
// C++ Program to find
// n-th real root of x
#include <bits/stdc++.h>
using namespace std;

void findNthRoot(double x, int n)
{

    // Initialize boundary values
    double low, high;
    if (x >= 0 and x <= 1)
    {
        low = x;
        high = 1;
    }
```

```
    else
    {
        low = 1;
        high = x;
    }

    // Used for taking approximations
    // of the answer
    double epsilon = 0.00000001;

    // Do binary search
    double guess = (low + high) / 2;
    while (abs((pow(guess, n)) - x) >= epsilon)
    {
        if (pow(guess, n) > x)
        {
            high = guess;
        }
        else
        {
            low = guess;
        }
        guess = (low + high) / 2;
    }

    cout << fixed << setprecision(16) << guess;
}

// Driver code
int main()
{
    double x = 5;
    int n = 2;
    findNthRoot(x, n);
}

// This code is contributed
// by Subhadeep
```

Output
2.2360679768025875

Time Complexity: O(log(x * 10^d)*$\log_{guess}$(n))

Auxiliary Space: O(1)

Here d is the number of decimal places upto which we want the result accurately.

Explanation of first example with epsilon = 0.01

```
Since taking too small value of epsilon as taken in our program
might not be feasible for
explanation because it will increase the number of steps
drastically so for the sake of
simplicity we are taking epsilon = 0.01

The above procedure will work as follows:

Say we have to calculate the 5,then x = 5, low = 1, high =
5.Taking epsilon = 0.01
First Guess:guess = (1 + 5) / 2 = 3Absolute error = |32 - 5| = 4
> epsilonguess^2 = 9 > 5(x)
then high = guess --> high = 3

Second Guess:guess = (1 + 3) / 2 = 2Absolute error = |22 - 5| =
1 > epsilonguess^2 = 4 > 5(x) then
ow = guess --> low = 2Third Guess:guess = (2 + 3) / 2 = 2.5
Absolute error = |2.5^2 - 5| = 1.25 > epsilonguess^2 = 6.25 >
5(x)

then high = guess --> high = 2.5and proceeding so on we will get
the 5 correct up to 2
decimal places i.e., 5 = 2.23600456We will ignore the digits
after 2 decimal places
since they may or may not be correct.
```

Need for Monotonic Function

Binary search is a well-known algorithm for finding a specific value in a sorted array. It has an average time complexity of O(log n) and is widely used in various computer science applications. However, binary search requires that the array be sorted and that the search function is monotonic. In this article, we will discuss the importance of monotonic functions for binary search.

Firstly, let us define a monotonic function. A function is said to be monotonic if it either increases or decreases as its input increases. In other words, a function f(x) is monotonic if, for any x1 and x2, if x1 is less than x2, then f(x1) is less than or equal to f(x2) if it is monotone non-decreasing, or f(x1) is greater than or equal to f(x2) if it is monotone non-increasing.

Binary search works by dividing the array into halves until the desired value is found. This is achieved by comparing the search value with the middle value of the array. If the search value is smaller than the middle value, the search continues in the left half of the array. If the search value is larger than the middle value, the search continues in the right half of the array. This process is repeated until the desired value is found or until there are no more values to search.

The requirement of a monotonic function arises because it guarantees that the values in the array are sorted. If the function is not monotonic, it is possible that the array is not sorted, which can lead to incorrect search results. In other words, if the function is not monotonic, the binary search algorithm can fail to find the desired value, or it may find a value that is not the desired value.

To illustrate this point, let us consider an example where the search function is not monotonic. Suppose we have an array of integers that are not sorted, and we want to find the index of the first occurrence of a specific value. If we apply binary search to this array without sorting it, we will not get the correct result. This is because the binary search algorithm assumes that the array is sorted, which is not the case in this example.

Another important aspect of a monotonic function is that it ensures that the binary search algorithm works correctly even if the array contains repeated values. If the function is not monotonic, the binary search algorithm may not be able to correctly handle repeated values, leading to incorrect results.

In conclusion, monotonic functions are essential for the binary search algorithm to work correctly. They guarantee that the array is sorted, which is a prerequisite for binary search, and they ensure that the algorithm can handle repeated values correctly. If the search function is not monotonic, the binary search algorithm can fail to find the desired value or find a value that is not the desired value, leading to incorrect search results. Therefore, it is important to ensure that the search function is monotonic before applying the binary search algorithm to a sorted array.

Applications of Binary Search

Applications of Binary Search:

- Binary search can be used as a building block for more complex algorithms used in machine learning, such as algorithms for training neural networks or finding the optimal hyperparameters for a model.
- Commonly used in Competitive Programming.
- Can be used for searching in computer graphics. Binary search can be used as a building block for more complex algorithms used in computer graphics, such as algorithms for ray tracing or texture mapping.
- Can be used for searching a database. Binary search can be used to efficiently search a database of records, such as a customer database or a product catalog.

Advantages of Binary Search:

- Binary search is faster than linear search, especially for large arrays. As the size of the array increases, the time it takes to perform a linear search increases linearly, while the time it takes to perform a binary search increases logarithmically.
- Binary search is more efficient than other searching algorithms that have a similar time complexity, such as interpolation search or exponential search.
- Binary search is relatively simple to implement and easy to understand, making it a good choice for many applications.
- Binary search is well-suited for searching large datasets that are stored in external memory, such as on a hard drive or in the cloud.
- Binary search can be used as a building block for more complex algorithms, such as those used in computer graphics and machine learning.

Drawbacks of Binary Search:

- We require the array to be sorted. If the array is not sorted, we must first sort it before performing the search. This adds an additional O(N * logN) time complexity for the sorting step, which makes it irrelevant to use binary search.
- Binary search requires that the data structure being searched be stored in contiguous memory locations. This can be a problem if the data structure is too large to fit in memory, or if it is stored on external memory such as a hard drive or in the cloud.
- Binary search requires that the elements of the array be comparable, meaning that they must be able to be ordered. This can be a problem if the elements of the array are not naturally ordered, or if the ordering is not well-defined.
- Binary search can be less efficient than other algorithms, such as hash tables, for searching very large datasets that do not fit in memory.

Conclusion:

To wrap it up, it can be said that Binary search is an efficient algorithm for finding an element within a sorted array. The time complexity of the binary search is O(log n).

Though, one of the main drawbacks of binary search is that the array must be sorted. this is a useful algorithm for building more complex algorithms in computer graphics and machine learning or several other cases.

Find the farthest smaller number in the right side

Given an array **arr[]** of size **N**. For every element in the array, the task is to find the index of the farthest element in the array to the right which is smaller than the current element. If no such number exists then print **-1**.

Examples:

Input: arr[] = {3, 1, 5, 2, 4}
Output: 3 -1 4 -1 -1
arr[3] is the farthest smallest element to the right of arr[0].
arr[4] is the farthest smallest element to the right of arr[2].
And for the rest of the elements, there is no smaller element to their right.

Input: arr[] = {1, 2, 3, 4, 0}
Output: 4 4 4 4 -1

Approach 1 : (Brute Force Method)

A brute force approach to this problem can be, keep a variable **idx = -1** from beginning and for each element start traversing the same array from the backward upto **(i+1)**th index. And, if at any index **j** find smaller element from the current element, i.e. **(a[i] > a[j])** break from the loop.

Below is the implementation of the above approach :

C++Java

```
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;

// Function to find the farthest
// smaller number in the right side
void farthest_min(int a[], int n)
{
```

```
    for (int i = 0; i < n; i++) {

        // keeping the idx = -1 from beginning
        int idx = -1;

        // traverse the given array backward
        for (int j = n - 1; j > i; j--) {

            // if found any element smaller
            if (a[i] > a[j]) {
                // update that index and break
                idx = j;
                break;
            }
        }

        // Print the required index
        cout << idx << " ";
    }
}

// Driver code
int main()
{
    int a[] = { 3, 1, 5, 2, 4 };
    int n = sizeof(a) / sizeof(a[0]);

    farthest_min(a, n);

    return 0;
}

// this code is contributed by Rajdeep
```

Output

```
3 -1 4 -1 -1
```

Time Complexity : O(N^2)

Auxiliary Space : O(1)

Approach 2 : (Optimized Method)

An efficient approach is to create a **suffix_min[]** array where **suffix_min[i]** stores the minimum element from the subarray **arr[i ... N - 1]**. Now for any element **arr[i]**, binary search can be used on the subarray **suffix_min[i + 1 ... N - 1]** to find the farthest smallest element to the right of **arr[i]**.

Below is the implementation of the above approach:

C++Java

```cpp
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;

// Function to find the farthest
// smaller number in the right side
void farthest_min(int a[], int n)
{
    // To store minimum element
    // in the range i to n
    int suffix_min[n];
    suffix_min[n - 1] = a[n - 1];
    for (int i = n - 2; i >= 0; i--) {
        suffix_min[i] = min(suffix_min[i + 1], a[i]);
    }

    for (int i = 0; i < n; i++) {
        int low = i + 1, high = n - 1, ans = -1;

        while (low <= high) {
            int mid = (low + high) / 2;

            // If current element in the suffix_min
            // is less than a[i] then move right
            if (suffix_min[mid] < a[i]) {
                ans = mid;
                low = mid + 1;
            }
            else
                high = mid - 1;
        }

        // Print the required answer
        cout << ans << " ";
    }
}

// Driver code
int main()
{
    int a[] = { 3, 1, 5, 2, 4 };
    int n = sizeof(a) / sizeof(a[0]);

    farthest_min(a, n);

    return 0;
```

```
}
```

Output

```
3 -1 4 -1 -1
```

Time Complexity: O(N* log(N))

Auxiliary Space: O(N)

Aggresive Cows

Farmer John has built a new long barn, with N stalls. The stalls are located along a straight line at positions x1,...,xN .

His C cows don't like this barn layout and become aggressive towards each other once put into a stall. To prevent the cows from hurting each other, FJ wants to assign the cows to the stalls, such that the minimum distance between any two of them is as large as possible. What is the largest minimum distance?

Input Format:

The first line of the input contains a single integer **T** denoting the number of test cases. The description of **T** test cases is as follows:

- The first line of each test case contains an integer ***N.***
- For Next N lines, each line contain $\mathbf{A_1,A_2,...,A_n}$.

Output Format:

For each test case, print the answer to the problem followed by a newline character.

Constraints:

$2 <= \mathbf{N} <= 10^5$

$1<= \mathbf{C_i} <=10^5$

$0 <= X_i <= 1,000,000,000$

```
Example:

Input:
1
5 3
1 2 8 4 9
```

Output:
3
C++

```cpp
#include <iostream>
#include <algorithm>
using namespace std;

long long arr[100001];
bool check_cows(long long n,long long x,long long c){
    long long cows_count = 1;
    long long last_cows =arr[0];
    for (int i=1;i<n;i++){
        if((arr[i]-last_cows)>=x){
            cows_count++;
            last_cows=arr[i];

        }
        if(cows_count==c)return true;

    }
    return false;
}
int main(){
    long long t;
    cin>>t;
    while(t--){
    long long n,c;
    cin>>n>>c;

    for(int i=0;i<n;i++)cin>>arr[i];
    sort(arr,arr+n);
    long long low=0,high =1000000000,ans =0;
    while(low<=high){
        long long mid=(low+high)/2;
        if(check_cows(n,mid,c)){
            ans=mid;
            low = mid+1;
        }else{
            high = mid-1;
        }
    }
    cout<<ans<<"\n";
}return 0;
}
```

Minimum time to Fulfill all Orders

Geek is organizing a party at his house. For the party, he needs exactly **N** donuts for the guests. **Geek** decides to order the donuts from a nearby restaurant, which has **L** chefs and each chef has a rank **R**.
A chef with rank **R** can make 1 donut in the first **R** minutes, 1 more donut in the next **2R** minutes, 1 more donut in **3R** minutes, and so on.
For example. a chef with rank 2, can make one donut in 2 minutes, one more donut in the next 4 minutes, and one more in the next 6 minutes. So, it take 2 + 4 + 6 = 12 minutes to make 3 donuts. A chef can move on to making the next donut only after completing the previous one. All the chefs can work simultaneously.
Since, it's time for the party, **Geek** wants to know the **minimum** time required in completing **N** donuts. Return an integer denoting the minimum time

Example :

```
Input:
N = 10
L = 4
rank[] = {1, 2, 3, 4}
Output: 12
Explanation:
Chef with rank 1, can make 4 donuts in time 1 + 2 + 3 + 4 = 10
mins
Chef with rank 2, can make 3 donuts in time 2 + 4 + 6 = 12 mins
Chef with rank 3, can make 2 donuts in time 3 + 6 = 9 mins
Chef with rank 4, can make 1 donuts in time = 4 minutes
Total donuts = 4 + 3 + 2 + 1 = 10 and total time = 12 minutes.
```

C++

```cpp
#include<bits/stdc++.h>
using namespace std;

class Solution{
    public:
   int count(int N,vector<int>&arr,int L,long long t){
       int cnt =0;
       for(int i=0;i<L;i++){
           int rank =arr[i];
           long long x =t;
           while(x>0){
               x-=rank;
               if(x>=0){
                   cnt++;
                   rank+=arr[i];
```

```
                }
            }
        }
        return cnt;

    }

    int findMinTime(int N, vector<int>&arr,int L){
        int ans=0;
        long long low =1;
        long long high = 1e9;

        while(low<=high){
            long long mid = (low+high)/2;
            int x = count(N,arr,L,mid);
            if(x>=N){
                ans=mid;
                high = mid -1;
            }else {
                low = mid+1;
            }
        }
        return ans;
    }
};

int main()
{
        int t;
        cin>>t;
        while(t--)
        {
            int n ;
            cin >> n;
            int l ;
            cin >> l;
            vector<int>arr(l);
            for(int i = 0; i < l; i++){
                cin >> arr[i];
            }
            Solution ob;
            int ans = ob.findMinTime(n, arr, l);
            cout << ans <<endl;
        }
        return 0;
}
```

Ternary Search

Ternary Search

Find the minimum value of the function

$f(x) = (x^2 + b * x + c) / \sin(x)$

Ternary search is a decrease(by constant) and conquer algorithm that can be used to find an element in an array. It is similar to binary search where we divide the array into two parts but in this algorithm, we divide the given array into three parts and determine which has the key (searched element). We can divide the array into three parts by taking mid1 and mid2 which can be calculated as shown below. Initially, l and r will be equal to 0 and n-1 respectively, where n is the length of the array.

It is same as the binary search. The only difference is that, it reduces the time complexity a bit more. the algorithm involves 'N' steps. The searchable range would be the size = 3^N. Inversely, the number of steps needed to find the element is the log of the size of the collection. So the runtime is O(log N base 3).

C++

```cpp
#include <iostream>
#include <iomanip>
#include <cmath>
using namespace std;

#define PI 3.141592653589793

const double EPS = 1e-9;

double f(double x) {
    // Function to be minimized
    return sin(x) + cos(x);
}

double ternarySearch(double lo, double hi) {
    while (hi - lo > EPS) {
        double mid1 = lo + (hi - lo) / 3.0;
        double mid2 = hi - (hi - lo) / 3.0;
        double y1 = f(mid1);
        double y2 = f(mid2);
        if (y1 > y2) {
```

```
            lo = mid1;
        } else {
            hi = mid2;
        }
    }
    return f(lo);
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);

    double lo = 0.0, hi = PI / 2.0;
    double ans = ternarySearch(lo, hi);
    cout << fixed << setprecision(9) << ans << endl;

    return 0;
}
```

Output

```
1.000000000
```

Need for Parabolic Curve

In a ternary search, we divide the search interval into three parts and discard one part in each iteration, until we find the desired value or determine that it doesn't exist in the interval. One common optimization used in ternary search involves using a parabolic curve to guide the search.

To understand the need for a parabolic curve, let's consider a simple example. Suppose we have a unimodal function f(x) and we want to find its maximum value within a given interval [a, b]. One way to do this is to use a ternary search. We start by dividing the interval [a, b] into three parts, and then evaluate the function at the two points x1 and x2 that divide the interval.

Suppose that f(x1) > f(x2). This means that the maximum value of f(x) is more likely to be in the interval [a, x2], since we know that f(x1) is already greater than f(x2). So, we can discard the interval [x2, b] and continue the search within the interval [a, x2].

Now, instead of dividing the remaining interval into three parts again, we can use a parabolic curve to guide the search. We can fit a parabola to the three points (a, f(a)), (x1,

f(x1)), and (x2, f(x2)), and find the maximum point of the parabola. Let's call this point xm. If the maximum value of the parabola is at xm, then we can discard the interval [x2, b] and continue the search within the interval [a, x2]. Otherwise, we can discard the interval [a, x1] and continue the search within the interval [x1, b].

The advantage of using a parabolic curve is that it allows us to make more informed decisions about which part of the interval to discard in each iteration. By fitting a parabola to the function values at the three points, we can estimate the shape of the function and use this information to guide the search towards the maximum value more efficiently.

In summary, a parabolic curve is used in ternary search to improve the efficiency of the search by guiding the search towards the maximum or minimum value of a unimodal function. By fitting a parabola to the function values at three points, we can estimate the shape of the function and use this information to make more informed decisions about which part of the interval to discard in each iteration.

In what cases does it fail ?

Why is Binary Search preferred over Ternary Search?

The following is a simple recursive **Binary Search** function in C++

C++

```
// CPP program for the above approach
#include <bits/stdc++.h>
using namespace std;


// A recursive binary search function. It returns location of x in
// given array arr[l..r] is present, otherwise -1
int binarySearch(int arr[], int l, int r, int x)
{
if (r >= l)
{
                int mid = l + (r - l)/2;

                // If the element is present at the middle itself
                if (arr[mid] == x) return mid;

                // If element is smaller than mid, then it can only be
present
```

```
                // in left subarray
                if (arr[mid] > x) return binarySearch(arr, l, mid-1, x);

                // Else the element can only be present in right subarray
                return binarySearch(arr, mid+1, r, x);
}

// We reach here when element is not present in array
return -1;
}
```

The following is a simple recursive **Ternary Search** function :

C++

```
// A recursive ternary search function. It returns location of x in
// given array arr[l..r] is present, otherwise -1
int ternarySearch(int arr[], int l, int r, int x)
{
if (r >= l)
{
                int mid1 = l + (r - l)/3;
                int mid2 = mid1 + (r - l)/3;

                // If x is present at the mid1
                if (arr[mid1] == x) return mid1;

                // If x is present at the mid2
                if (arr[mid2] == x) return mid2;

                // If x is present in left one-third
                if (arr[mid1] > x) return ternarySearch(arr, l, mid1-1, x);

                // If x is present in right one-third
                if (arr[mid2] < x) return ternarySearch(arr, mid2+1, r, x);

                // If x is present in middle one-third
                return ternarySearch(arr, mid1+1, mid2-1, x);
}
// We reach here when element is not present in array
return -1;
}
```

Which of the above two does less comparisons in worst case?

From the first look, it seems the ternary search does less number of comparisons as it

makes Log_3n recursive calls, but binary search makes Log_2n recursive calls. Let us take a closer look.
The following is recursive formula for counting comparisons in worst case of Binary Search.

$T(n) = T(n/2) + 2, \quad T(1) = 1$

The following is recursive formula for counting comparisons in worst case of Ternary Search.

$T(n) = T(n/3) + 4, \; T(1) = 1$

In binary search, there are $2Log_2n + 1$ comparisons in worst case. In ternary search, there are $4Log_3n + 1$ comparisons in worst case.

$Time\ Complexity\ for\ Binary\ search = 2clog_2n + O(1)$
$Time\ Complexity\ for\ Ternary\ search = 4clog_3n + O(1)$

Therefore, the comparison of Ternary and Binary Searches boils down the comparison of expressions $2Log_3n$ and Log_2n . The value of $2Log_3n$ can be written as $(2 / Log_23) * Log_2n$. Since the value of $(2 / Log_23)$ is more than one, Ternary Search does more comparisons than Binary Search in worst case.

Counting Inversions using Merge Sort

Inversion Count for an array indicates – how far (or close) the array is from being sorted. If the array is already sorted, then the inversion count is 0, but if the array is sorted in reverse order, the inversion count is the maximum.

Given an array **a[]**. The task is to find the inversion count of **a[]**. Where two elements a[i] and a[j] form an inversion if a[i] > a[j] and i < j.

Examples:

Input: *arr[] = {8, 4, 2, 1}*
Output: *6*
Explanation: *Given array has six inversions: (8, 4), (4, 2), (8, 2), (8, 1), (4, 1), (2, 1).*

Input: *arr[] = {1, 20, 6, 4, 5}*
Output: *5*
Explanation: *Given array has five inversions: (20, 6), (20, 4), (20, 5), (6, 4), (6, 5).*

Naive Approach:

Traverse through the array, and for every index, find the number of smaller elements on its right side of the array. This can be done using a nested loop. Sum up the counts for all indices in the array and print the sum.

Follow the below steps to Implement the idea:

- Traverse through the array from start to end
- For every element, find the count of elements smaller than the current number up to that index using another loop.
- Sum up the count of inversion for every index.
- Print the count of inversions.

Below is the Implementation of the above approach

C++

```
// C++ program to Count Inversions
// in an array
#include <bits/stdc++.h>
using namespace std;

int getInvCount(int arr[], int n)
{
        int inv_count = 0;
        for (int i = 0; i < n - 1; i++)
                for (int j = i + 1; j < n; j++)
                        if (arr[i] > arr[j])
                                inv_count++;

        return inv_count;
}

// Driver Code
int main()
{
        int arr[] = { 1, 20, 6, 4, 5 };
        int n = sizeof(arr) / sizeof(arr[0]);
        cout << " Number of inversions are "
                << getInvCount(arr, n);
        return 0;
}
```

Output

```
Number of inversions are 5
```

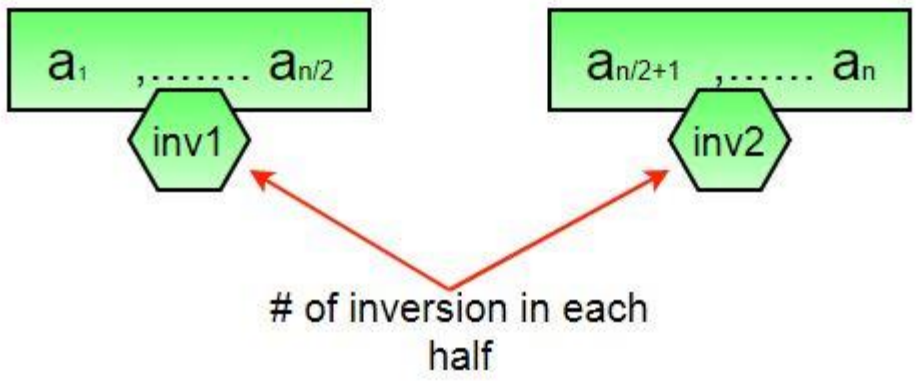
Time Complexity: O(N^2), Two nested loops are needed to traverse the array from start to end.

Auxiliary Space: O(1), No extra space is required.

Counting Inversions using Merge Sort

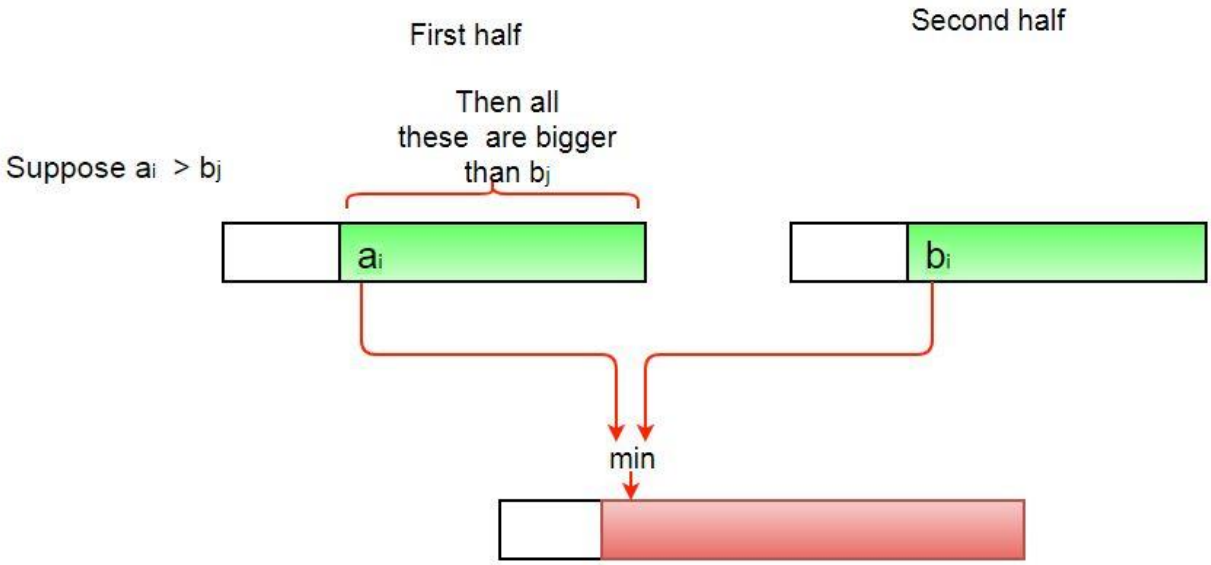
Illustration:

Suppose the number of inversions in the left half and right half of the array (let be inv1 and inv2); what kinds of inversions are not accounted for in Inv1 + Inv2? The answer is – the inversions that need to be counted during the merge step. Therefore, to get the total number of inversions that needs to be added are the number of inversions in the left subarray, right subarray, and merge().

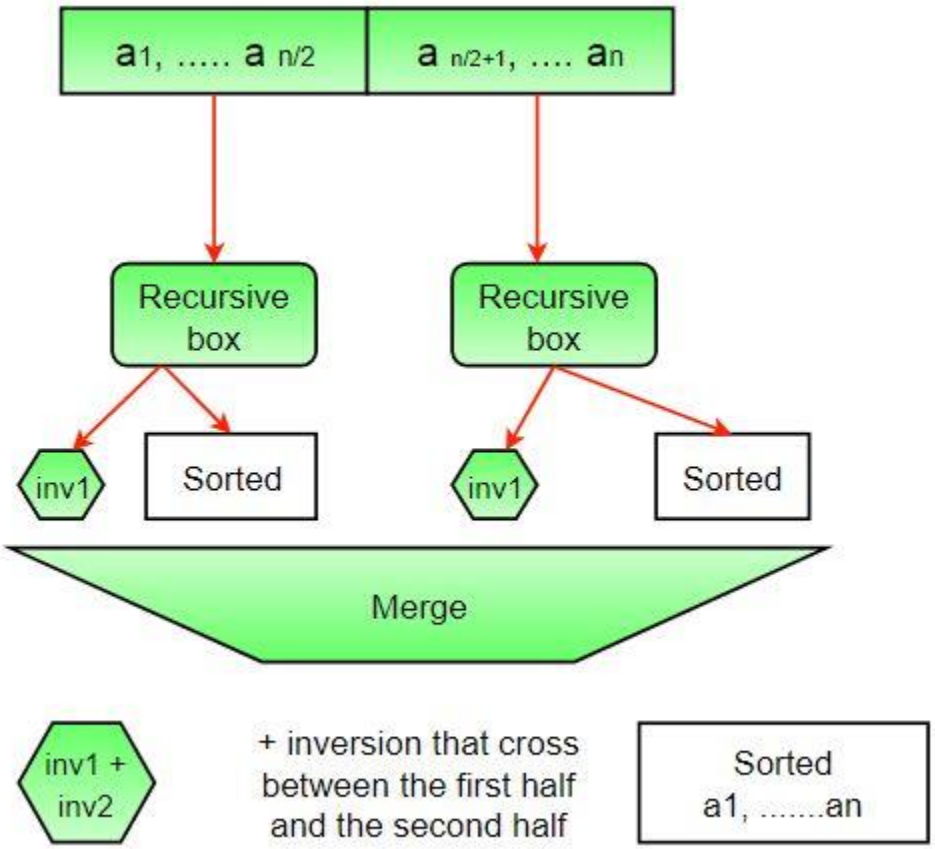


How to get** the **number of inversions in merge()?

In merge process, let i is used for indexing left sub-array and j for right sub-array. At any step in merge(), if a[i] is greater than a[j], then there are (mid – i) inversions. because left and right subarrays are sorted, so all the remaining elements in left-subarray (a[i+1], a[i+2] … a[mid]) will be greater than a[j]



The complete picture:



Follow the below steps to Implement the idea:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> merge(vector<int> a, vector<int> b, long long &invCount) {
    int n = a.size();
    int m = b.size();

    a.push_back(INT_MAX);
    b.push_back(INT_MAX);

    vector<int> res(n + m);

    int i = 0, j = 0;
    for (int k = 0; k < n + m; k++) {
        if (a[i] <= b[j]) {
            res[k] = a[i];
            i++;
            invCount += j;
        } else {
            res[k] = b[j];
            j++;
        }
    }

    return res;
}

vector<int> mergeSort(vector<int> v, long long &invCount) {
    int n = v.size();
    if (n == 1) {
        return v;
    }

    vector<int> left(v.begin(), v.begin() + n / 2);
    vector<int> right(v.begin() + n / 2, v.end());

    left = mergeSort(left, invCount);
    right = mergeSort(right, invCount);

    return merge(left, right, invCount);
}

int main() {
    int arr[] = { 1, 20, 6, 4, 5 };
    int n = sizeof(arr) / sizeof(arr[0]);
    vector<int> v(arr, arr + n);
    long long invCount = 0;
    vector<int> sorted = mergeSort(v, invCount);
```

```
    cout << "Number of inversions are " << invCount << endl;
    return 0;
}
```

Output

```
Number of inversions are 5
```

Time Complexity: O(n * log n), The algorithm used is divide and conquer i.e. merge sort whose complexity is O(n log n).

Minimum Range Problem

Given a function **$F(x) = (x^2 + b * x + c) / \sin x$,**

Find the **minimum** value of the over the range **$0 < x < \pi / 2$**.

Input Format:

The first line of the input contains a single integer **T** denoting the number of test cases. The description of **T** test cases is as follows:

- Each Line of Testcase contain b and c

Output Format:

For each test case, output the minimum value of f(x) over the given range. Absolute error of 10^{-9} is allowed.

Constraints:

$1 <= T <= 15000$

$1 <= B_i, C_i <= 20$

Example:

Input:

```
1
2 2
```

Output:

```
5.883172561
```

C++

```
#include <bits/stdc++.h>
using namespace std;

const double PI = 3.14159265359;
```

```cpp
const double eps = 1e-9;

double operation(double x, double b, double c) {
    return (x * x + b * x + c) / sin(x);
}

void solve() {
    double b, c;
    cin >> b >> c;
    double l = 0 + eps, r = PI / 2 - eps;
    while (r - l > eps) {
        double m1 = l + (r - l) / 3;
        double m2 = r - (r - l) / 3;

        double f1 = operation(m1, b, c);
        double f2 = operation(m2, b, c);
        if (f1 > f2)
            l = m1;
        else
            r = m2;
    }

    cout << setprecision(9) << fixed << operation(l, b, c) << "\n";
}

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);

    int t = 1;
    cin >> t;
    while (t--) {
        solve();
    }


    return 0;
}
```

Input :

```
1
2 2
```

Output :

5.883172561

Finding Peak in 1-D Array

Given an array of distinct integers, print index of a peak element in it. An array element is a peak if it is greater than its neighbours.

Example :

arr[] = {1,3,20,4,2,0}

- Condition for Peak element :
  arr[i] > arr[i-1] and arr[i] >arr[i+1]
- Edge Cases:
  If size of array is 1, that element will be peak element. For extreme indexes, there's only 1 neighbour.

Naive Solution : Brute Force

C++

```
#include <iostream>

using namespace std;

int findPeak(int arr[], int n) {
    if (n == 1) {
        return 0;
    }

    if (arr[0] > arr[1]) {
        return 0;
    }

    if (arr[n-1] > arr[n-2]) {
        return n - 1;
    }

    for (int i = 1; i < n - 1; i++) {
        if (arr[i] > arr[i-1] && arr[i] > arr[i+1]){
            return i;
        }
    }
```

```
    return -1; // peak element not found
}

int main() {
    int arr[] = {1,3,20,4,2,0};
    int n = sizeof(arr)/sizeof(arr[0]);
    int peakIndex = findPeak(arr, n);
    if (peakIndex != -1) {
        cout << "Peak element found at index " << peakIndex << endl;
    } else {
        cout << "Peak element not found" << endl;
    }
    return 0;
}
```

Output

```
Peak element found at index 2
```

Time complexity: O(n)

Auxiliary Space: O(1)

Efficient solution using Divide and Conquer

The idea is based on the technique of Binary Search to check if the middle element is the peak element or not.

- If middle element is peak, return it's index as result.
- If middle element is not the peak, then we reduce our search space to the left or right of middle based on some condition

Implementation :

C++

```
#include <iostream>

using namespace std;

int findPeak(int arr[], int lo, int hi, int n) {
    int mid = (lo + hi) / 2;

    if ((mid == 0 || arr[mid - 1] < arr[mid]) && (mid == n - 1 || arr[mid
+ 1] < arr[mid])) {
        return mid;
    } else if (mid > 0 && arr[mid - 1] > arr[mid]) {
        return findPeak(arr, lo, (mid - 1), n);
    } else {
        return findPeak(arr, (mid + 1), hi, n);
```

```
    }
}

int main() {
    int arr[] = {1, 3, 20, 4, 2, 0};
    int n = sizeof(arr)/sizeof(arr[0]);
    int peakIndex = findPeak(arr, 0, n-1, n);
    if (peakIndex != -1) {
        cout << "Peak element found at index " << peakIndex << endl;
    } else {
        cout << "Peak element not found" << endl;
    }
    return 0;
}
```

Output

```
Peak element found at index 2
```

Time Complexity: O(log N), Where N is the number of elements in the input array.
Auxiliary Space: O(log N)

Peak in a rotated array

Minimum Element in a rotated sorted array

Problem:

A sorted array containing all unique elements is rotated some unknown number of times, find the minimum element in it.

We can use binary search to solve this.

Implementation :

C++

```
#include <iostream>
using namespace std;

int findMin(int arr[], int n) {

    int lo = 0, hi = n - 1;

    while (lo < hi) {
        int mid = (lo + hi) / 2;
```

```
        if (arr[mid] < arr[hi]) {
            hi = mid;
        } else {
            lo = mid + 1;
        }
    }

    return arr[lo];
}

int main() {
    int arr[] = {4, 5, 6, 1, 2, 3};
    int n = sizeof(arr) / sizeof(arr[0]);
    int minElement = findMin(arr, n);
    cout << "Minimum element in the rotated array is: " << minElement <<
endl;
    return 0;
}
```

Output

```
Minimum element in the rotated array is: 1
```

Time Complexity :O(logn)

Auxiliary Space :O(1)

Stacks and Queues – CP

Stack

Next Greater Element

Given an array, print the Next Greater Element (NGE) for every element.

The **Next greater Element** for an element x is the first greater element on the right side of x in the array. Elements for which no greater element exist, consider the next greater element as -1.

Example:

Input: arr[] = [4 , 5 , 2 , 25]
Output: 4 --> 5
5 --> 25
2 --> 25
25 --> -1
Explanation: *except* ***25*** *every element has an element greater then them present on the right side*

Input: arr[] = [13 , 7, 6 , 12]
Output: 13 --> -1
7 --> 12
6 --> 12
12 --> -1
Explanation: ***13*** *and* ***12*** *don't have any element greater then them present on the right side*

Find Next Greater Element using Nested Loops:

The idea is to use two loops , The outer loop picks all the elements one by one. The inner loop looks for the first greater element for the element picked by the outer loop. If a greater element is found then that element is printed as next, otherwise, -1 is printed.

Follow the steps mentioned below to implement the idea:

- Traverse The array from **index 0** to **end**.
- For each element start another loop from **index i+1** to **end.**
- If a greater element is found in the second loop then print it and **break** the loop, else print **-1.**

Below is the implementation of the above approach:

C++Java

```cpp
// Simple C++ program to print
// next greater elements in a
// given array
#include <iostream>
using namespace std;

/* prints element and NGE pair
for all elements of arr[] of size n */
void printNGE(int arr[], int n)
{
```

```
    int next, i, j;
    for (i = 0; i < n; i++) {
        next = -1;
        for (j = i + 1; j < n; j++) {
            if (arr[i] < arr[j]) {
                next = arr[j];
                break;
            }
        }
        cout << arr[i] << " --> " << next << endl;
    }
}

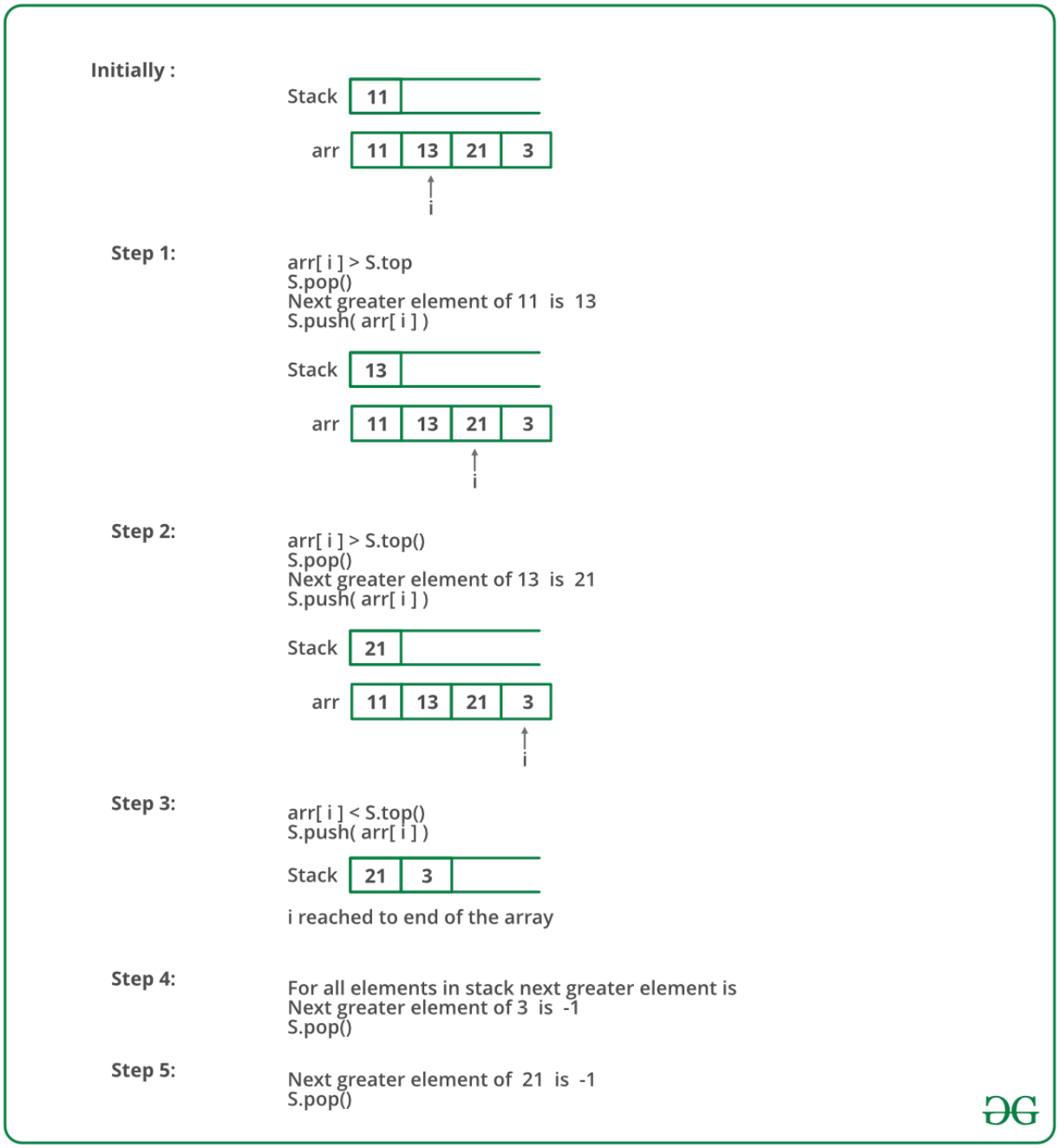
// Driver Code
int main()
{
    int arr[] = { 11, 13, 21, 3 };
    int n = sizeof(arr) / sizeof(arr[0]);
    printNGE(arr, n);
    return 0;
}
```

Time Complexity: $O(N^2)$
Auxiliary Space: $O(1)$

Find Next Greater Element using Stack:

The idea is to store the elements for which we have to find the next greater element in a stack and while traversing the array, if we find a greater element, we will pair it with the elements from the stack till the top element of the stack is less than the current element.

illustration:
Below is the illustration of the above approach:



Follow the steps mentioned below to implement the idea:

- Push the first element to stack.
- Pick the rest of the elements one by one and follow the following steps in the loop.
    - Mark the current element as **next.**
    - If the stack is not empty, compare top most element of stack with **next**.
    - If **next** is greater than the top element, Pop element from the stack. **next** is the next greater element for the popped element.

    - Keep popping from the stack while the popped element is smaller than **next. next** becomes the next greater element for all such popped elements.
- Finally, push the **next** in the stack.
- After the loop in step 2 is over, pop all the elements from the stack and print **-1** as the next element for them.

Below is the implementation of the above approach:

C++Java

```
// A Stack based C++ program to find next
// greater element for all array elements.
#include <bits/stdc++.h>
using namespace std;

/* prints element and NGE pair for all
elements of arr[] of size n */
void printNGE(int arr[], int n)
{
    stack<int> s;

    /* push the first element to stack */
    s.push(arr[0]);

    // iterate for rest of the elements
    for (int i = 1; i < n; i++) {

        if (s.empty()) {
            s.push(arr[i]);
            continue;
        }

        /* if stack is not empty, then
           pop an element from stack.
           If the popped element is smaller
           than next, then
        a) print the pair
        b) keep popping while elements are
        smaller and stack is not empty */
        while (s.empty() == false && s.top() < arr[i]) {
            cout << s.top() << " --> " << arr[i] << endl;
            s.pop();
        }

        /* push next to stack so that we can find
        next greater for it */
        s.push(arr[i]);
    }

    /* After iterating over the loop, the remaining
```

```
    elements in stack do not have the next greater
    element, so print -1 for them */
    while (s.empty() == false) {
        cout << s.top() << " --> " << -1 << endl;
        s.pop();
    }
}

/* Driver code */
int main()
{
    int arr[] = { 11, 13, 21, 3 };
    int n = sizeof(arr) / sizeof(arr[0]);
    printNGE(arr, n);
    return 0;
}
```

Time Complexity: O(N)
Auxiliary Space: O(N)

Find Next Greater Element using Map:

In this particular approach we are using the map as our main stack

- This is same as above method but the elements are pushed and popped only once into the stack. The array is changed in place. The array elements are pushed into the stack until it finds a greatest element in the right of array. In other words the elements are popped from stack when top of the stack value is smaller in the current array element.
- Once all the elements are processed in the array but stack is not empty. The left out elements in the stack doesn't encounter any greatest element . So pop the element from stack and change it's index value as -1 in the array.

C++

```
// C++ code to implement the approach

#include <bits/stdc++.h>

using namespace std;

void nextLargerElement(int arr[], int n)
{
    vector<unordered_map<string, int> > s;

    // iterating over the array
    for (int i = 0; i < n; i++) {
        while (s.size() > 0
```

```
                && s[s.size() - 1]["value"] < arr[i]) {
            // updating the array as per the stack top
            unordered_map<string, int> d = s[s.size() - 1];
            s.pop_back();
            arr[d["ind"]] = arr[i];
        }
        // pushing values to stack
        unordered_map<string, int> e;

        e["value"] = arr[i];
        e["ind"] = i;
        s.push_back(e);
    }

    // updating the array as per the stack top
    while (s.size() > 0) {
        unordered_map<string, int> d = s[s.size() - 1];
        s.pop_back();
        arr[d["ind"]] = -1;
    }
}

// Driver Code

int main()
{
    int arr[] = { 6, 8, 0, 1, 3 };
    int n = 5;

    // Function call
    nextLargerElement(arr, n);
    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";
}

// This code is contributed by phasing17
```

Output

```
8 -1 1 3 -1
```

Time Complexity: O(N)

Auxiliary Space: O(N)

Valid parentheses

Given an expression string **exp**, write a program to examine whether the pairs and the orders of "{", "}", "(", ")", "[", "]" are correct in the given expression.

Example:

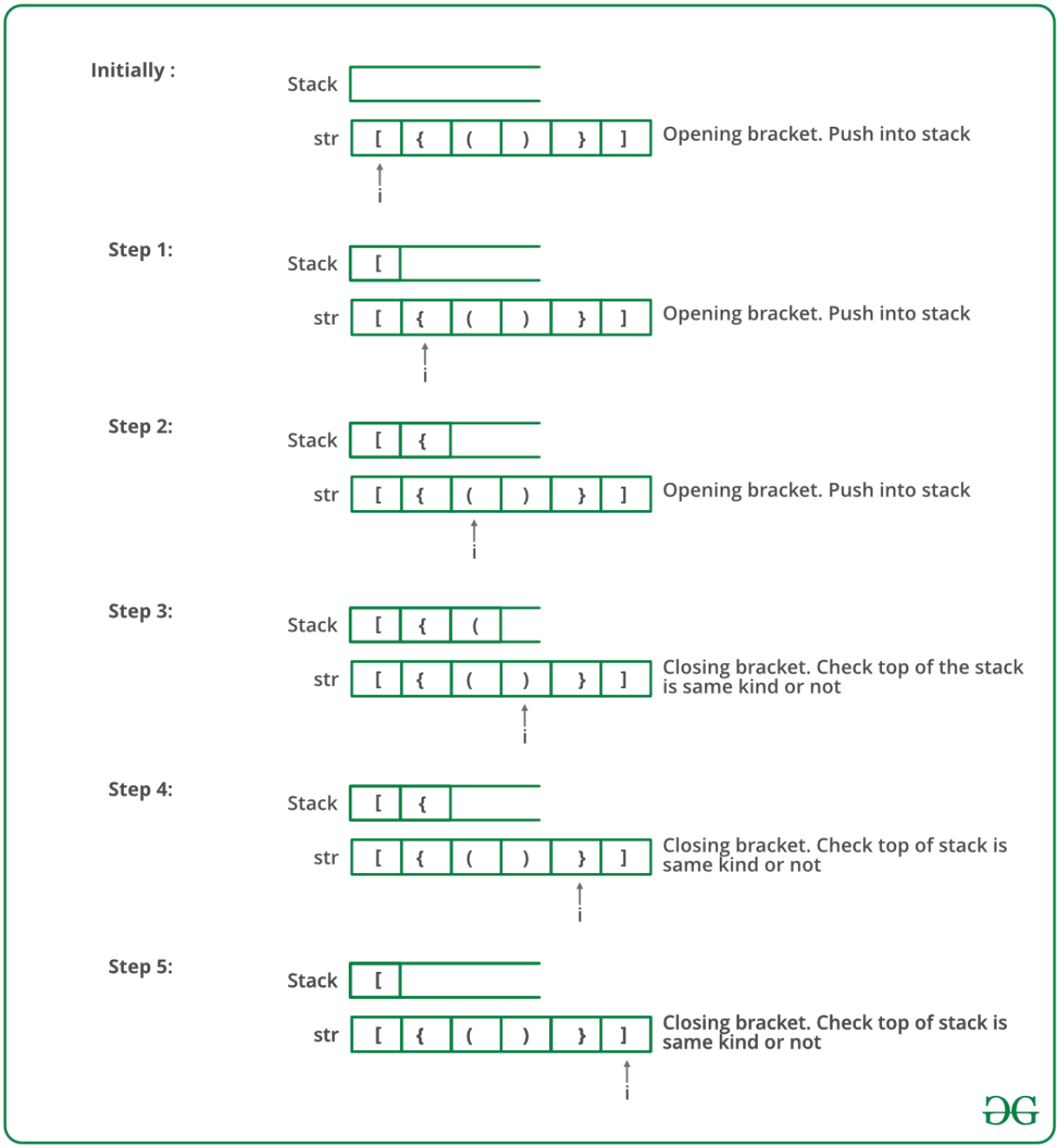
Input: exp = "[()]{}{[()()]()}"
Output: Balanced
Explanation: *all the brackets are well-formed*

Input: exp = "[(])"
Output: Not Balanced
Explanation: 1 *and* ***4*** *brackets are not balanced because there is a closing ']' before the closing '('*

Check for Balanced Bracket expression using Stack:

The idea is to put all the opening brackets in the stack. Whenever you hit a closing bracket, search if the top of the stack is the opening bracket of the same nature. If this holds then pop the stack and continue the iteration, in the end if the stack is empty, it means all brackets are well-formed . Otherwise, they are not balanced.

Illustration:
Below is the illustration of the above approach.



Follow the steps mentioned below to implement the idea:

- Declare a character stack (say **temp**).
- Now traverse the string exp.
    - If the current character is a starting bracket (**'(' or '{' or '['**) then push it to stack.
    - If the current character is a closing bracket (**')' or '}' or ']'**) then pop from stack and if the popped character is the matching starting bracket then fine.

    - Else brackets are **Not Balanced**.
- After complete traversal, if there is some starting bracket left in stack then **Not balanced**, else **Balanced**.

Below is the implementation of the above approach:

C++Java

```cpp
// C++ program to check for balanced brackets.

#include <bits/stdc++.h>
using namespace std;

// Function to check if brackets are balanced
bool areBracketsBalanced(string expr)
{
    // Declare a stack to hold the previous brackets.
    stack<char> temp;
    for (int i = 0; i < expr.length(); i++) {
        if (temp.empty()) {

            // If the stack is empty
            // just push the current bracket
            temp.push(expr[i]);
        }
        else if ((temp.top() == '(' && expr[i] == ')')
                 || (temp.top() == '{' && expr[i] == '}')
                 || (temp.top() == '[' && expr[i] == ']')) {

            // If we found any complete pair of bracket
            // then pop
            temp.pop();
        }
        else {
            temp.push(expr[i]);
        }
    }
    if (temp.empty()) {

        // If stack is empty return true
        return true;
    }
    return false;
}

// Driver code
int main()
{
    string expr = "{()}[]";
```

```
    // Function call
    if (areBracketsBalanced(expr))
        cout << "Balanced";
    else
        cout << "Not Balanced";
    return 0;
}
```

Output
Balanced

Time Complexity: O(N), Iteration over the string of size N one time.
Auxiliary Space: O(N) for stack.

Check for Balanced Bracket expression without using stack :

Following are the steps to be followed:

- Initialize a variable i with -1.
- Iterate through string and if it is a open bracket then increment the counter by +1.
- Else if it is a closing bracket then decrement the i by -1.
- At last if we get the (i==-1) then the string is balanced and we will return true otherwise the function will return false.

Below is the implementation of the above approach:

C++Java

```
#include <iostream>

using namespace std;

 bool areBracketsBalanced(string s) {
        int i=-1;
        for(auto& ch:s){
            if(ch=='(' || ch=='{' || ch=='[')
                s[++i]=ch;
            else{
                if(i>=0 && ((s[i]=='(' && ch==')') || (s[i]=='{' &&
ch=='}') || (s[i]=='[' && ch==']')))
                    i--;
                else
                    return false;
            }
        }
        return i==-1;
    }
```

```
int main()
{
    string expr = "{()}[]";

    // Function call
    if (areBracketsBalanced(expr))
        cout << "Balanced";
    else
        cout << "Not Balanced";
    return 0;
}
```

Output
```
Balanced
```
Time Complexity: O(N), Iteration over the string of size N one time.
Auxiliary Space: O(N) because we are using a char array of size length of the string.

Minimum in a stack in O(1) time complexity and O(n) space

Design a Data Structure SpecialStack that supports all the stack operations like push(), pop(), isEmpty(), isFull() and an additional operation getMin() which should return minimum element from the SpecialStack. All these operations of SpecialStack must be O(1). **Note:** To implement SpecialStack, you should only use standard Stack data structure and no other data structure like arrays, list, . etc.

Example:

```
Consider the following SpecialStack
16  --> TOP
15
29
19
18

When getMin() is called it should
return 15, which is the minimum
element in the current stack.

If we do pop two times on stack,
the stack becomes
29  --> TOP
```

```
19
18

When getMin() is called, it should
return 18 which is the minimum in
the current stack.
```

Solution:

Use two stacks: one to store actual stack elements and the other as an auxiliary stack to store minimum values. The idea is to do push() and pop() operations in such a way that the top of the auxiliary stack is always the minimum. Let us see how push() and pop() operations work.

Push(int x) // inserts an element x to Special Stack

1. 1) push x to the first stack (the stack with actual elements)
2. 2) compare x with the top element of the second stack (the auxiliary stack). Let the top element be y.
    1. If x is smaller than y then push x to the auxiliary stack.
    2. If x is greater than y then push y to the auxiliary stack.

int Pop() // removes an element from Special Stack and return the removed element

1. pop the top element from the auxiliary stack.
2. pop the top element from the actual stack and return it. Step 1 is necessary to make sure that the auxiliary stack is also updated for future operations.

int getMin() // returns the minimum element from Special Stack

1. Return the top element of the auxiliary stack.

We can see that **all the above operations are O(1)**.

Let us see an example. Let us assume that both stacks are initially empty and 18, 19, 29, 15, and 16 are inserted to the SpecialStack.

```
When we insert 18, both stacks change to following.
Actual Stack
18 <--- top
Auxiliary Stack
18 <---- top

When 19 is inserted, both stacks change to following.
Actual Stack
19 <--- top
18
Auxiliary Stack
18 <---- top
```

```
18

When 29 is inserted, both stacks change to following.
Actual Stack
29 <--- top
19
18
Auxiliary Stack
18 <---- top
18
18

When 15 is inserted, both stacks change to following.
Actual Stack
15 <--- top
29
19
18
Auxiliary Stack
15 <---- top
18
18
18

When 16 is inserted, both stacks change to following.
Actual Stack
16 <--- top
15
29
19
18
Auxiliary Stack
15 <---- top
15
18
18
18
```

The following is the implementation for SpecialStack class. In the below implementation, SpecialStack inherits from Stack and has one Stack object *min* which works as an auxiliary stack.

C++Java

```
#include <iostream>
#include <stdlib.h>

using namespace std;
```

```
/* A simple stack class with
basic stack functionalities */
class Stack {
private:
    static const int max = 100;
    int arr[max];
    int top;

public:
    Stack() { top = -1; }
    bool isEmpty();
    bool isFull();
    int pop();
    void push(int x);
};

/* Stack's member method to check
if the stack is empty */
bool Stack::isEmpty()
{
    if (top == -1)
        return true;
    return false;
}

/* Stack's member method to check
if the stack is full */
bool Stack::isFull()
{
    if (top == max - 1)
        return true;
    return false;
}

/* Stack's member method to remove
an element from it */
int Stack::pop()
{
    if (isEmpty()) {
        cout << "Stack Underflow";
        abort();
    }
    int x = arr[top];
    top--;
    return x;
}

/* Stack's member method to insert
an element to it */
```

```
void Stack::push(int x)
{
    if (isFull()) {
        cout << "Stack Overflow";
        abort();
    }
    top++;
    arr[top] = x;
}

/* A class that supports all the stack
operations and one additional
  operation getMin() that returns the
minimum element from stack at
  any time.  This class inherits from
the stack class and uses an
  auxiliary stack that holds minimum
elements */
class SpecialStack : public Stack {
    Stack min;

public:
    int pop();
    void push(int x);
    int getMin();
};

/* SpecialStack's member method to insert
 an element to it. This method
   makes sure that the min stack is also
updated with appropriate minimum
   values */
void SpecialStack::push(int x)
{
    if (isEmpty() == true) {
        Stack::push(x);
        min.push(x);
    }
    else {
        Stack::push(x);
        int y = min.pop();
        min.push(y);
        if (x < y)
            min.push(x);
        else
            min.push(y);
    }
}
```

```
/* SpecialStack's member method to
remove an element from it. This method
   removes top element from min stack also. */
int SpecialStack::pop()
{
    int x = Stack::pop();
    min.pop();
    return x;
}

/* SpecialStack's member method to get
 minimum element from it. */
int SpecialStack::getMin()
{
    int x = min.pop();
    min.push(x);
    return x;
}

/* Driver program to test SpecialStack
 methods */
int main()
{
    SpecialStack s;
    s.push(10);
    s.push(20);
    s.push(30);
    cout << s.getMin() << endl;
    s.push(5);
    cout << s.getMin();
    return 0;
}
```

Output

```
10
5
```

Complexity Analysis:

- **Time Complexity:**

    1. **For insert operation: O(1)** (As insertion 'push' in a stack takes constant time)
    2. **For delete operation: O(1)** (As deletion 'pop' in a stack takes constant time)
    3. **For 'Get Min' operation: O(1)** (As we have used an auxiliary stack which has it's top as the minimum element)

- **Auxiliary Space:** O(n).
  Use of auxiliary stack for storing values.

Minimum in a stack in o(1) time complexity and o(1) space

Design a Data Structure SpecialStack that supports all the stack operations like push(), pop(), isEmpty(), isFull() and an additional operation **getMin()** which should return minimum element from the SpecialStack. All these operations of SpecialStack must have a time and space complexity of O(1).
Note: To implement SpecialStack, you should only use standard Stack data structure and no other data structure like arrays, lists, etc

Example:

Input: Consider the following SpecialStack

16 --> TOP
15
29
19
18

When getMin() is called it should return 15,
which is the minimum element in the current stack.

If we do pop two times on stack, the stack becomes

29 --> TOP
19
18

When getMin() is called, it should return 18
which is the minimum in the current stack.

Approach: To solve the problem follow the below idea:

We define a variable **minEle** that stores the current minimum element in the stack. Now the interesting part is, how to handle the case when the minimum element is removed. To handle this, we push "2x - minEle" into the stack instead of x so that the previous minimum element can be retrieved using the current minEle and its value stored in the stack

Follow the given steps to implement the stack operations:

Push(x): Insert x at the top of the stack

- If the stack is empty, insert x into the stack and make minEle equal to x.
- If the stack is not empty, compare x with minEle. Two cases arise:
    - If x is greater than or equal to minEle, simply insert x.
    - If x is less than minEle, insert (2*x – minEle) into the stack and make minEle equal to x.
      For example, let the previous minEle be 3. Now we want to insert 2. We update minEle as 2 and insert 2*2 - 3 = 1 into the stack

Pop(): Removes an element from the top of the stack

- Remove the element from the top. Let the removed element be y. Two cases arise:
    - If y is greater than or equal to minEle, the minimum element in the stack is still minEle.
    - If y is less than minEle, the minimum element now becomes (2*minEle – y), so update (minEle = 2*minEle - y). This is where we retrieve the previous minimum from the current minimum and its value in the stack.
      For example, let the element to be removed be 1 and minEle be 2. We remove 1 and update minEle as 2*2 - 1 = 3

Important Points:

- Stack doesn't hold the actual value of an element if it is minimum so far.
- The actual minimum element is always stored in the minEle variable

Below is the illustration of the above approach:

Push(x)

Number Inserted	Present Stack	minELe
3	3	3
5	5 3	3
2	1 5 3	2
1	0 1 5 3	1
1	1 0 1 5 3	1
-1	-3 1 0 1 5 3	-1

- Number to be Inserted: 3, Stack is empty, so insert 3 into stack and minEle = 3.
- Number to be Inserted: 5, Stack is not empty, 5> minEle, insert 5 into stack and minEle = 3.
- Number to be Inserted: 2, Stack is not empty, 2< minEle, insert (2*2-3 = 1) into stack and minEle = 2.
- Number to be Inserted: 1, Stack is not empty, 1< minEle, insert (2*1-2 = 0) into stack and minEle = 1.
- Number to be Inserted: 1, Stack is not empty, 1 = minEle, insert 1 into stack and minEle = 1.
- Number to be Inserted: -1, Stack is not empty, -1 < minEle, insert (2*-1 – 1 = -3) into stack and minEle = -1.

Pop()

Number Removed	Orignal Number	Present Stack	minEle
-	-	-3 1 0 1 5 3	-1
-3	-1	1 0 1 5 3	1
1	1	0 1 5 3	1
0	1	1 5 3	2
1	2	5 3	3
5	5	3	3

- Initially the minimum element minEle in the stack is -1.
- Number removed: -3, Since -3 is less than the minimum element the original number being removed is minEle which is -1, and the new minEle = 2*-1 – (-3) = 1
- Number removed: 1, 1 == minEle, so number removed is 1 and minEle is still equal to 1.
- Number removed: 0, 0< minEle, original number is minEle which is 1 and new minEle = 2*1 – 0 = 2.
- Number removed: 1, 1< minEle, original number is minEle which is 2 and new minEle = 2*2 – 1 = 3.
- Number removed: 5, 5> minEle, original number is 5 and minEle is still 3

Below is the implementation of the above approach:

C++Java

```
// C++ program to implement a stack that supports
// getMinimum() in O(1) time and O(1) extra space.
#include <bits/stdc++.h>
using namespace std;

// A user defined stack that supports getMin() in
// addition to push() and pop()
struct MyStack {
    stack<int> s;
    int minEle;

    // Prints minimum element of MyStack
    void getMin()
```

```
    {
        if (s.empty())
            cout << "Stack is empty\n";

        // variable minEle stores the minimum element
        // in the stack.
        else
            cout << "Minimum Element in the stack is: "
                 << minEle << "\n";
    }

    // Prints top element of MyStack
    void peek()
    {
        if (s.empty()) {
            cout << "Stack is empty ";
            return;
        }

        int t = s.top(); // Top element.

        cout << "Top Most Element is: ";

        // If t < minEle means minEle stores
        // value of t.
        (t < minEle) ? cout << minEle : cout << t;
    }

    // Remove the top element from MyStack
    void pop()
    {
        if (s.empty()) {
            cout << "Stack is empty\n";
            return;
        }

        cout << "Top Most Element Removed: ";
        int t = s.top();
        s.pop();

        // Minimum will change as the minimum element
        // of the stack is being removed.
        if (t < minEle) {
            cout << minEle << "\n";
            minEle = 2 * minEle - t;
        }

        else
            cout << t << "\n";
```

```
    }

    // Removes top element from MyStack
    void push(int x)
    {
        // Insert new number into the stack
        if (s.empty()) {
            minEle = x;
            s.push(x);
            cout << "Number Inserted: " << x << "\n";
            return;
        }

        // If new number is less than minEle
        else if (x < minEle) {
            s.push(2 * x - minEle);
            minEle = x;
        }

        else
            s.push(x);

        cout << "Number Inserted: " << x << "\n";
    }
};

// Driver Code
int main()
{
    MyStack s;

      // Function calls
    s.push(3);
    s.push(5);
    s.getMin();
    s.push(2);
    s.push(1);
    s.getMin();
    s.pop();
    s.getMin();
    s.pop();
    s.peek();

    return 0;
}
```

Output

```
Number Inserted: 3
```

```
Number Inserted: 5
Minimum Element in the stack is: 3
Number Inserted: 2
Number Inserted: 1
Minimum Element in the stack is: 1
Top Most Element Removed: 1
Minimum Element in the stack is: 2
Top Most Element Removed: 2
Top Most Element is: 5
```

Time Complexity: O(1)

Auxiliary Space: O(1)

How does this approach work?

When the element to be inserted is less than minEle, we insert "2x - minEle". The important thing to note is, that 2x - minEle will always be less than x (proved below), i.e., new minEle and while popping out this element we will see that something unusual has happened as the popped element is less than the minEle. So we will be updating minEle.

How 2*x - minEle is less than x in push()?

x < minEle which means x - minEle < 0

// Adding x on both sides
x - minEle + x < 0 + x
2*x - minEle < x
We can conclude 2*x - minEle < new minEle

While popping out, if we find the element(y) less than the current minEle, we find the new minEle = 2*minEle - y

How previous minimum element, prevMinEle is, 2*minEle - y
in pop() is y the popped element?

// We pushed y as 2x - prevMinEle. Here
// prevMinEle is minEle before y was inserted

y = 2*x - prevMinEle

// Value of minEle was made equal to x
minEle = x .

new minEle = 2 * minEle - y
= 2*x - (2*x - prevMinEle)
= prevMinEle // This is what we wanted

Design a stack that supports getMin() in O(1) time and O(1) extra space by creating a MinStack class:

To solve the problem follow the below idea:

Create a class node that has two variables Val and min. Val will store the actual value that we are going to insert in the stack, whereas min will store the min value so far seen up to that node

Below is the implementation of the above approach:

C++Java

```
// C++ program for the above approach
#include <bits/stdc++.h>
using namespace std;

int mini(int a, int b) { return a > b ? b : a; }

class MinStack {
public:
    stack<pair<int, int> > s;

    void push(int element)
    {
        /* new max will be given no. if stack is empty else
        we compare given no. to max at current top of
        stack*/

        int new_min = s.empty()
                          ? element
                          : mini(element, s.top().second);

        // we push the pair of given_element,new_min in s

        s.push({ element, new_min });
    }

    int pop()
```

```
    {
        int popped;
        if (!s.empty()) {
            // popped has popped number
            popped = s.top().first;
            s.pop();
        }
        else {
            // print a message or throw exception etc
        }
        return popped;
    }

    int minimum()
    {
        int min_elem = s.top().second;
        return min_elem;
    }
};

// Driver code
int main()
{
    MinStack s;

    // Function calls
    s.push(-1);
    s.push(10);
    s.push(-4);
    s.push(0);
    cout << s.minimum() << endl;
    cout << s.pop() << endl;
    cout << s.pop() << endl;
    cout << s.minimum();
    return 0;
}

// this code is contributed by Apoorv Shrivastava - VITB
```

Output

```
-4
0
-4
-1
```

Related Article: An approach that uses O(1) time and O(n) extra space is discussed here.

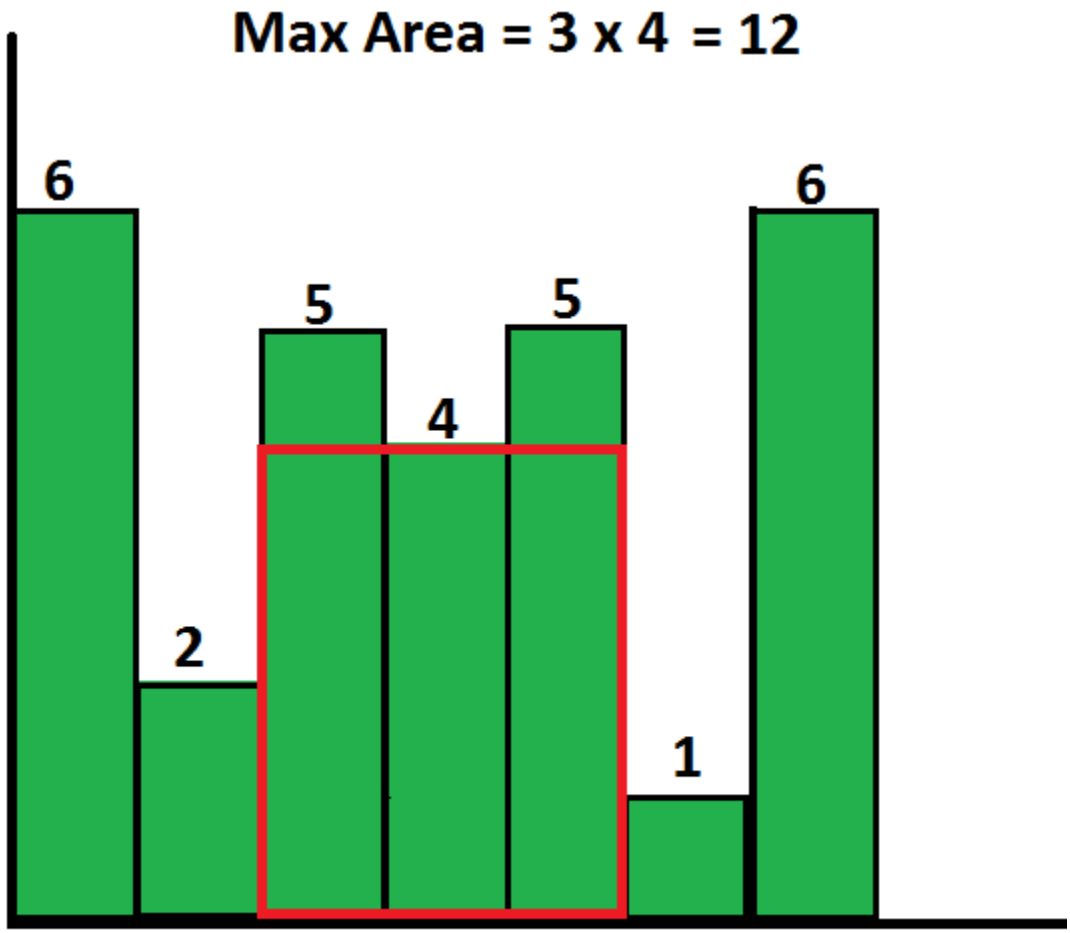
This article is contributed by Nikhil Tekwani. If you like GeeksforGeeks and would like to contribute, you can also write an article and mail your article to review-team@geeksforgeeks.org. See your article appearing on the GeeksforGeeks main page and help other Geeks.
Please write comments if you find anything incorrect, or you want to share more information about the topic discussed above.

Largest Rectangle in a Histogram (Stack)

Find the largest rectangular area possible in a given histogram where the largest rectangle can be made of a number of contiguous bars. Each bar is of width one

Example:

Input: histogram = {6, 2, 5, 4, 5, 1, 6}



Output: 12

Input: histogram = {3, 5, 1, 7, 5, 9}
Output: 15

To solve the problem follow the below idea:

For every bar 'x', we calculate the area with 'x' as the smallest bar in the rectangle. If we calculate the such area for every bar 'x' and find the maximum of all areas, our task is done.

How to calculate the area with 'x' as the smallest bar?

We need to know the index of the first smaller (smaller than 'x') bar on the left of 'x' and the index of the first smaller bar on the right of 'x'. Let us call these indexes as 'left index' and 'right index' respectively. We traverse all bars from left to right and

maintain a stack of bars. Every bar is pushed to stack once. A bar is popped from the stack when a bar of smaller height is seen. When a bar is popped, we calculate the area with the popped bar as the smallest bar.

How do we get the left and right indexes of the popped bar?

The current index tells us the right index and the index of the previous item in the stack is the left index

Follow the given steps to solve the problem:

1. Create an empty stack.
2. Start from the first bar, and do the following for every bar hist[i] where '**i**' varies from 0 to n-1
    1. If the stack is empty or hist[i] is higher than the bar at top of the stack, then push '**i**' to stack.
    2. If this bar is smaller than the top of the stack, then keep removing the top of the stack while the top of the stack is greater.
    3. Let the removed bar be hist[tp]. Calculate the area of the rectangle with hist[tp] as the smallest bar.
    4. For hist[tp], the 'left index' is previous (previous to tp) item in stack and 'right index' is '**i**' (current index).
3. If the stack is not empty, then one by one remove all bars from the stack and do step (2.2 and 2.3) for every removed bar

Below is the implementation of the above approach.

C++Java

```
// C++ program to find maximum rectangular area in
// linear time
#include <bits/stdc++.h>
using namespace std;

// The main function to find the maximum rectangular
// area under given histogram with n bars
int getMaxArea(int hist[], int n)
{
    // Create an empty stack. The stack holds indexes
    // of hist[] array. The bars stored in stack are
    // always in increasing order of their heights.
    stack<int> s;

    int max_area = 0; // Initialize max area
    int tp; // To store top of stack
    int area_with_top; // To store area with top bar
                       // as the smallest bar
```

```
    // Run through all bars of given histogram
    int i = 0;
    while (i < n) {
        // If this bar is higher than the bar on top
        // stack, push it to stack
        if (s.empty() || hist[s.top()] <= hist[i])
            s.push(i++);

        // If this bar is lower than top of stack,
        // then calculate area of rectangle with stack
        // top as the smallest (or minimum height) bar.
        // 'i' is 'right index' for the top and element
        // before top in stack is 'left index'
        else {
            tp = s.top(); // store the top index
            s.pop(); // pop the top

            // Calculate the area with hist[tp] stack
            // as smallest bar
            area_with_top
                = hist[tp]
                  * (s.empty() ? i : i - s.top() - 1);

            // update max area, if needed
            if (max_area < area_with_top)
                max_area = area_with_top;
        }
    }

    // Now pop the remaining bars from stack and calculate
    // area with every popped bar as the smallest bar
    while (s.empty() == false) {
        tp = s.top();
        s.pop();
        area_with_top
            = hist[tp] * (s.empty() ? i : i - s.top() - 1);

        if (max_area < area_with_top)
            max_area = area_with_top;
    }

    return max_area;
}

// Driver code
int main()
{
    int hist[] = { 6, 2, 5, 4, 5, 1, 6 };
    int n = sizeof(hist) / sizeof(hist[0]);
```

```
    // Function call
    cout << "Maximum area is " << getMaxArea(hist, n);
    return 0;
}
```

Output
```
Maximum area is 12
```
Time Complexity: O(N), Since every bar is pushed and popped only once
Auxiliary Space: O(N)

Merge Overlapping Intervals (Stack)

Given a set of time intervals in any order, merge all overlapping intervals into one and output the result which should have only mutually exclusive intervals.

Example:

Input: Intervals = {{1,3},{2,4},{6,8},{9,10}}
Output: {{1, 4}, {6, 8}, {9, 10}}
Explanation: Given intervals: [1,3],[2,4],[6,8],[9,10], we have only two overlapping intervals here,[1,3] and [2,4]. Therefore we will merge these two and return [1,4],[6,8], [9,10].

Input: Intervals = {{6,8},{1,9},{2,4},{4,7}}
Output: {{1, 9}}

Merge Overlapping Intervals Optimized Approach

The idea to solve this problem is, first sort the intervals according to the starting time. Once we have the sorted intervals, we can combine all intervals in a linear traversal. The idea is, in sorted array of intervals, if **interval[i]** doesn't overlap with **interval[i-1]**, then **interval[i+1]** cannot overlap with **interval[i-1]** because starting time of **interval[i+1]** must be greater than or equal to **interval[i]**.

Follow the steps mentioned below to implement the approach:

- Sort the intervals based on the increasing order of starting time.
- Push the first interval into a stack.

- For each interval do the following:
    - If the current interval does not overlap with the top of the stack then, push the current interval into the stack.
    - If the current interval overlap with the top of the stack then, update the stack top with the ending time of the current interval.
- The end stack contains the merged intervals.

Below is the implementation of the above approach:

C++Java

```cpp
// A C++ program for merging overlapping intervals
#include <bits/stdc++.h>
using namespace std;

// An interval has start time and end time
struct Interval {
    int start, end;
};

// Compares two intervals according to their starting time.
// This is needed for sorting the intervals using library
// function std::sort(). See http://goo.gl/iGspV
bool compareInterval(Interval i1, Interval i2)
{
    return (i1.start < i2.start);
}

// The main function that takes a set of intervals, merges
// overlapping intervals and prints the result
void mergeIntervals(Interval arr[], int n)
{
    // Test if the given set has at least one interval
    if (n <= 0)
        return;

    // Create an empty stack of intervals
    stack<Interval> s;

    // sort the intervals in increasing order of start time
    sort(arr, arr + n, compareInterval);

    // push the first interval to stack
    s.push(arr[0]);

    // Start from the next interval and merge if necessary
    for (int i = 1; i < n; i++) {
        // get interval from stack top
        Interval top = s.top();
```

```
        // if current interval is not overlapping with stack
        // top, push it to the stack
        if (top.end < arr[i].start)
            s.push(arr[i]);

        // Otherwise update the ending time of top if ending
        // of current interval is more
        else if (top.end < arr[i].end) {
            top.end = arr[i].end;
            s.pop();
            s.push(top);
        }
    }

    // Print contents of stack
    cout << "\n The Merged Intervals are: ";
    while (!s.empty()) {
        Interval t = s.top();
        cout << "[" << t.start << "," << t.end << "] ";
        s.pop();
    }
    return;
}

// Driver program
int main()
{
    Interval arr[]
        = { { 6, 8 }, { 1, 9 }, { 2, 4 }, { 4, 7 } };
    int n = sizeof(arr) / sizeof(arr[0]);
    mergeIntervals(arr, n);
    return 0;
}
```

Output

```
 The Merged Intervals are: [1,9]
```

Time complexity: O(N*log(N))

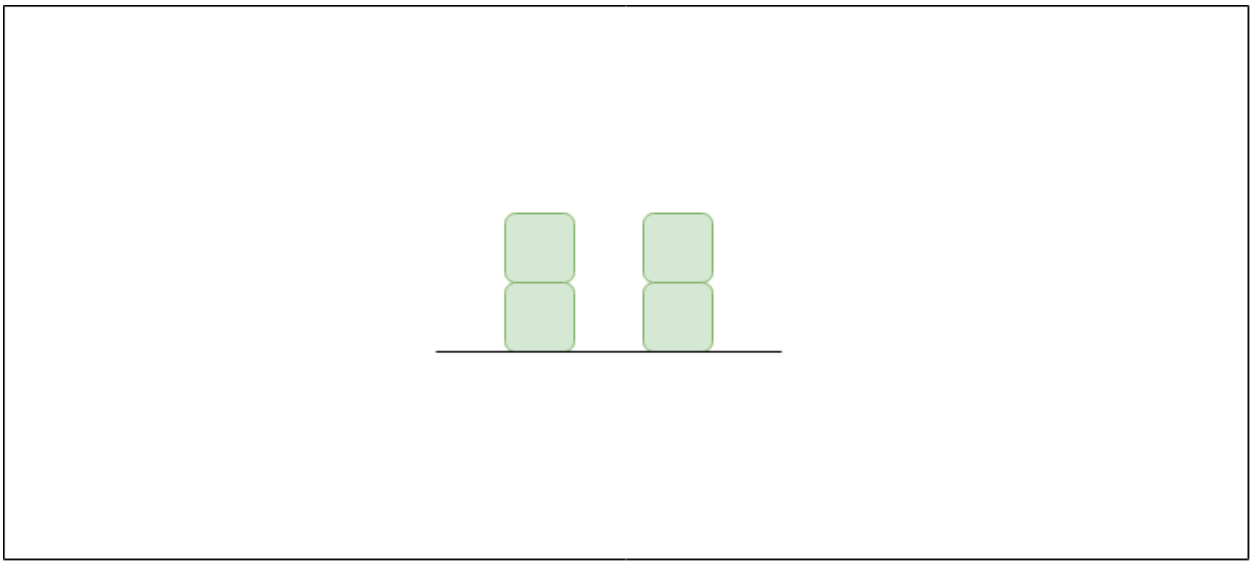
Auxiliary Space: O(N)

Trapping Rain Water (stack)

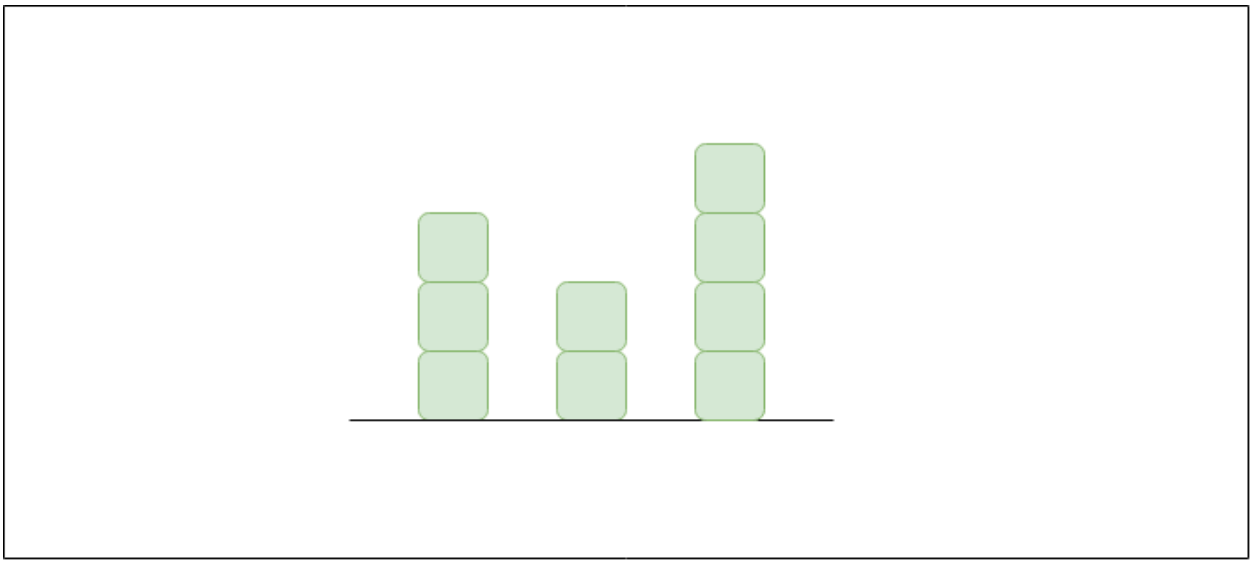
Given an array of **N** non-negative integers **arr[]** representing an elevation map where the width of each bar is **1**, compute how much water it is able to trap after raining.

Examples:

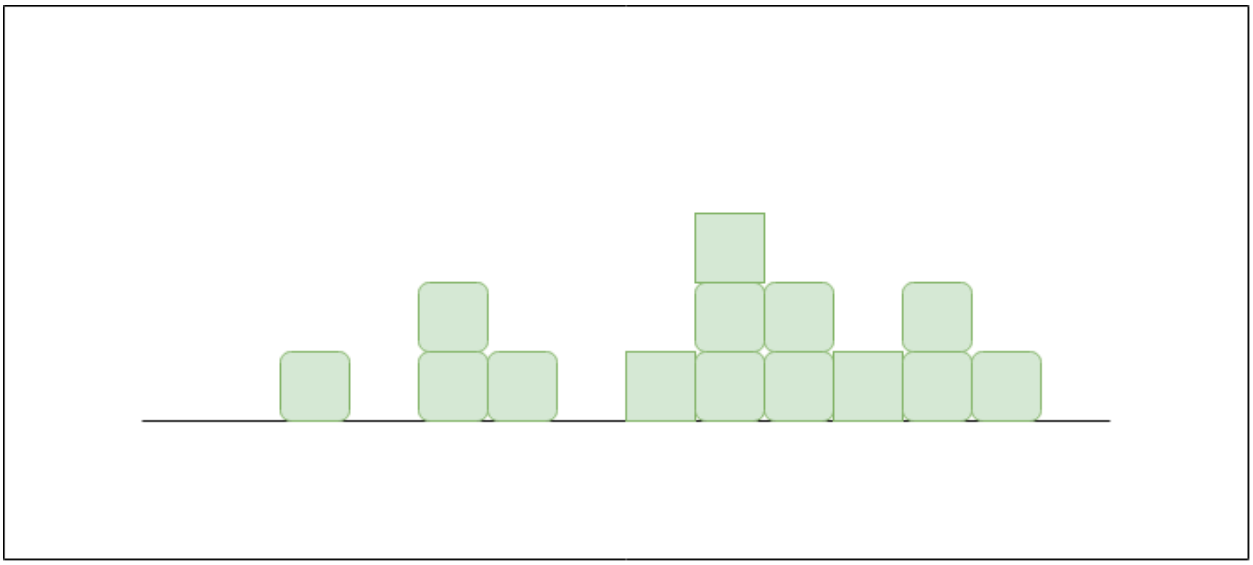
Input: arr[] = {2, 0, 2}
Output: 2
Explanation: The structure is like below.
We can trap 2 units of water in the middle gap.



Input: arr[] = {3, 0, 2, 0, 4}
Output: 7
Explanation: Structure is like below.
We can trap "3 units" of water between 3 and 2,
"1 unit" on top of bar 2 and "3 units" between 2 and 4.



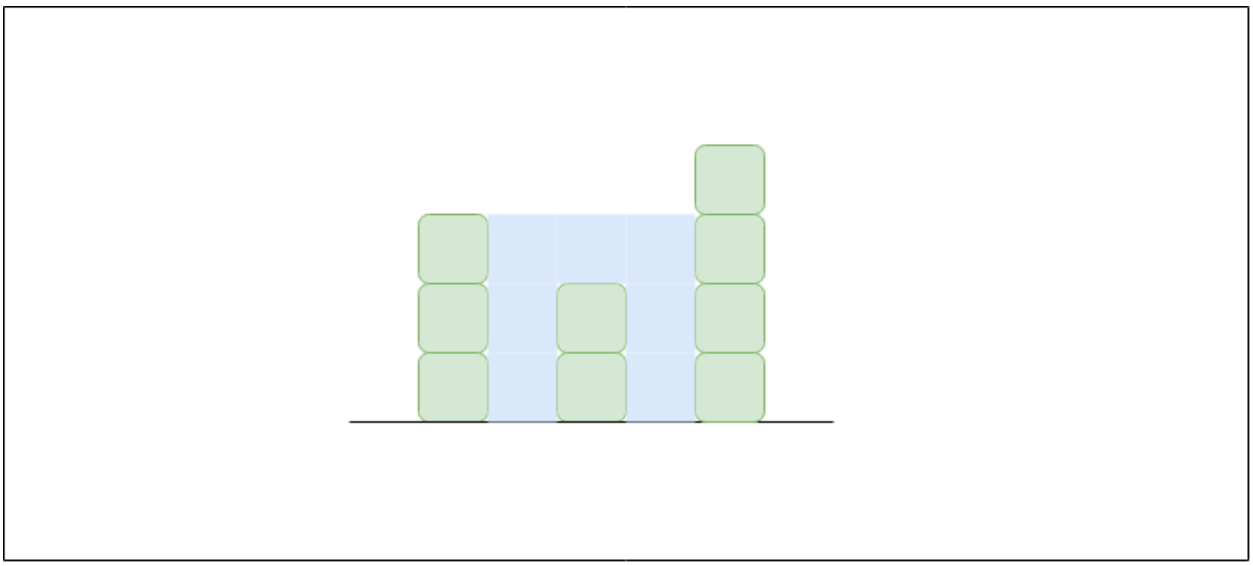
Input: arr[] = {0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1}
Output: 6
Explanation: The structure is like below.
Trap "1 unit" between first 1 and 2, "4 units" between
first 2 and 3 and "1 unit" between second last 1 and last 2



Intuition: The basic intuition of the problem is as follows:

- An element of the array can store water if there are higher bars on the left and the right.
- The amount of water to be stored in every position can be found by finding the heights of bars on the left and right sides.
- The total amount of water stored is the summation of the water stored in each index.

For example - Consider the array **arr[] = {3, 0, 2, 0, 4}**.
Three units of water can be stored in two indexes 1 and 3, and one unit of water at index 2.
Water stored in each index = 0 + 3 + 1 + 3 + 0 = **7**



We can use a stack to track the bars that are bounded by the higher left and right bars. This can be done using only one iteration.

- For this we will keep pushing elements in stack, until an element with higher value than the stack top is found. This denotes that there is a chance of storing position on the left side of the current element with the current bar being an end.
- So remove the smaller element on left and find the left bar (which is the current top of stack) and the amount of water stored by the left end bar and the current bar being the right end. Continue this till the stack is not empty or a higher value element is found.

Follow the below illustration for a better understanding.

Illustration:

Consider the array **arr[] = {3, 0, 2, 0, 4}** and an empty stack **st**.

For i = 0:

=> The stack is empty. So no elements with higher value on left.

=> Push the index into the stack. **st = {0}** [to keep track of the distance in between]

For i = 1:

=> arr[1] is less than arr[stack top]. So arr[1] has a higher left bound.

=> Push the index into stack. **st = {0, 1}**

For i = 2:

=> arr[2] is greater than arr[stack top]. So arr[2] is the higher right bound of current stack top.

=> Calculate the water stored in between the left and right bound of the stack top.

=> The stack top is the base height in between the left and right bound.

=> Pop the stack top. So **st = {0}**.

=> Water stored in between when arr[0] and arr[2] are the bound= {min(arr[0], arr[2]) - arr[1]} * (2 - 0 - 1) = **2**.

=> arr[0] is greater than arr[2] Push the index into stack. **st = {0, 2}.**

=> Total water stored = **0 + 2 = 2**.

For i = 3:

=> arr[3] is less than arr[2]. So arr[3] has a higher left bound.

=> Push the index into the stack. **st = {0, 2, 3}**.

For i = 4:

=> arr[4] is greater than arr[stack top]. So arr[4] is the higher right bound of current stack top.

=> Calculate the water stored in same way as for i = 2. The base height is arr[3].

=> Pop the stack top. So **st = {0, 2}**.

=> Water stored in between when arr[4] and arr[2] are the bound= {min(arr[4], arr[2]) - arr[3]} * (4 - 2 - 1) = **2**.

=> arr[4] is greater than arr[2].

=> Pop the stack. **st = {0}**.

=> Water stored in between arr[0] and arr[4] when arr[2] is the base height = {min(3, 4) - 2} * (4 - 0 - 1) = **3**

=> arr[0] less than arr[4]. So pop stack. **st = {}**.

=> As no element left in the stack push the index. **st = {4}**.

=> Total water stored = **2 + 2 + 3 = 7**.

So the total amount of water stored = **7**.

Follow the steps mentioned below to implement the idea:

- Loop through the indices of the bar array.
- For each bar, do the following:
    - Loop while the Stack is not empty and the current bar has a height greater than the top bar of the stack,
        - Store the index of the top bar in **pop_height** and pop it from the Stack.

        - Find the distance between the left bar(current top) of the popped bar and the current bar.
        - Find the minimum height between the **top bar** and the **current bar**.
        - The maximum water that can be trapped is **distance * min_height**.
        - The water trapped, including the popped bar, is **(distance * min_height) - height[pop_height]**.
        - Add that to the answer.
- Return the amount received as the total amount of water

Below is the implementation of the above approach.

C++Java

```
// C++ implementation of the approach
#include <bits/stdc++.h>
using namespace std;

// Function to return the maximum
// water that can be stored
int maxWater(int height[], int n)
{

    // Stores the indices of the bars
    stack<int> st;

    // Stores the final result
    int ans = 0;

    // Loop through the each bar
    for (int i = 0; i < n; i++) {

        // Remove bars from the stack
        // until the condition holds
        while ((!st.empty())
               && (height[st.top()] < height[i])) {

            // Store the height of the top
            // and pop it.
            int pop_height = height[st.top()];
            st.pop();

            // If the stack does not have any
            // bars or the popped bar
            // has no left boundary
            if (st.empty())
                break;

            // Get the distance between the
            // left and right boundary of
            // popped bar
```

```
            int distance = i - st.top() - 1;

            // Calculate the min. height
            int min_height
                = min(height[st.top()], height[i])
                  - pop_height;

            ans += distance * min_height;
        }

        // If the stack is either empty or
        // height of the current bar is less than
        // or equal to the top bar of stack
        st.push(i);
    }
    return ans;
}

// Driver code
int main()
{

    int arr[] = { 0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1 };
    int n = sizeof(arr) / sizeof(arr[0]);

    cout << maxWater(arr, n);

    return 0;
}
```

Output
6
Time Complexity: O(N)
Auxiliary Space: O(N)

Queue

Maximum subsequence sum with some constrained | queue

Given an integer array **nums** and an integer **K**, The task is to find the maximum sum of a non-empty subsequence of the array such that for every two consecutive integers in the

subsequence, nums[i] and nums[j], where **i < j**, the condition **j – i <= K** is satisfied.

A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Examples:

Input: nums = [10, 2, -10, 5, 20], K = 2
Output: 37
Explanation: The subsequence is [10, 2, 5, 20].

Input: nums = [-1, -2, -3], K = 1
Output: -1

Input: nums = [10, -2, -10, -5, 20], K = 2
Output: 23

- **Approach:**
  At any index i after calculating the maximum sum for i, nums[i] will now store the maximum possible sum that can be obtained from a subsequence ending at i.
- To calculate the maximum possible sum for every index i , check what is the maximum value that can be obtained from a window of size K before it. If the maximum value is negative, use zero instead.
- To use the values of calculated sums optimally we use a deque to store the sums along with their indexes in an increasing order of the sums . We also pop the element from the back when its index goes out of the window k of current index i.

C++Java

```
// C++ program to find the maximum sum
// subsequence under given constraint
#include <bits/stdc++.h>
using namespace std;

// Function return the maximum sum
int ConstrainedSubsetSum(vector<int>& nums,
                         int k)
{
   deque<pair<int, int> > d;

   // Iterating the given array
   for (int i = 0; i < nums.size(); i++)
   {

       // Check if deque is empty
```

```
        nums[i] += d.size()
            ? max(d.back().first, 0) : 0;

        while (d.size() &&
               d.front().first < nums[i])
            d.pop_front();

        d.push_front({ nums[i], i });

        if (d.back().second == i - k)
            d.pop_back();
    }

    int ans = nums[0];

    for (auto x : nums)
        ans = max(ans, x);

    return ans;
}

// Driver code
int main()
{
    vector<int> nums = { 10, -2, -10,
                         -5, 20 };
    int K = 2;

    // Function call
    cout << ConstrainedSubsetSum(nums, K)
        << endl;
    return 0;
}
```

Output:

```
23
```

Time Complexity: O(N)
Auxiliary Space: O(K)

maximum in a window | queue

Given an array and an integer **K**, find the maximum for each and every contiguous subarray of size K.

Examples :

Input: arr[] = {1, 2, 3, 1, 4, 5, 2, 3, 6}, K = 3
Output: 3 3 4 5 5 5 6
Explanation: Maximum of 1, 2, 3 is 3
Maximum of 2, 3, 1 is 3
Maximum of 3, 1, 4 is 4
Maximum of 1, 4, 5 is 5
Maximum of 4, 5, 2 is 5
Maximum of 5, 2, 3 is 5
Maximum of 2, 3, 6 is 6

Input: arr[] = {8, 5, 10, 7, 9, 4, 15, 12, 90, 13}, K = 4
Output: 10 10 10 15 15 90 90
Explanation: Maximum of first 4 elements is 10, similarly for next 4 elements (i.e from index 1 to 4) is 10, So the sequence generated is 10 10 10 15 15 90 90

Create a Deque, *Qi* of capacity K, that stores only useful elements of current window of K elements. An element is useful if it is in current window and is greater than all other elements on right side of it in current window. Process all array elements one by one and maintain *Qi* to contain useful elements of current window and these useful elements are maintained in sorted order. The element at front of the *Qi* is the largest and element at rear/back of *Qi* is the smallest of current window.

Below is the dry run of the above approach:

Initially :



, K = 3

Dequeue stores index of the maximum element at front and stores index of minimum element at back. At any time, it store indexes which belongs to current window

Step 1 :

Take the first window and keep necessary index in dequeue



dequeue : { 2 }
Maximum element is arr[2] = 3

Step 2 :



dequeue : { 2,3 }
Maximum element is arr[2] = 3

Step 3 :



dequeue : { 4 }
Maximum element is arr[4] = 4

Step 4 :



dequeue : { 5 }
Maximum element is arr[5] = 5

Step 5 :



dequeue : { 5,6 }
Maximum element is arr[5] = 5

Step 6 :



dequeue : { 5,7 }
Maximum element is arr[5] = 5

Step 7 :



dequeue : { 8 }
Maximum element is arr[8] = 6

Follow the given steps to solve the problem:

- Create a deque to store K elements.
- Run a loop and insert the first K elements in the deque. Before inserting the element, check if the element at the back of the queue is smaller than the current element, if it is so remove the element from the back of the deque until all elements left in the deque are greater than the current element. Then insert the current element, at the back of the deque.
- Now, run a loop from K to the end of the array.
- Print the front element of the deque.
- Remove the element from the front of the queue if they are out of the current window.
- Insert the next element in the deque. Before inserting the element, check if the element at the back of the queue is smaller than the current element, if it is so remove the element from the back of the deque until all elements left in the deque are greater than the current element. Then insert the current element, at the back of the deque.
- Print the maximum element of the last window.

Below is the implementation of the above approach:

C++Java

```cpp
// CPP program for the above approach
#include <bits/stdc++.h>
using namespace std;

// A Dequeue (Double ended queue) based
// method for printing maximum element of
// all subarrays of size k
void printKMax(int arr[], int N, int K)
{

    // Create a Double Ended Queue,
    // Qi that will store indexes
    // of array elements
    // The queue will store indexes
    // of useful elements in every
    // window and it will
    // maintain decreasing order of
    // values from front to rear in Qi, i.e.,
    // arr[Qi.front[]] to arr[Qi.rear()]
    // are sorted in decreasing order
    std::deque<int> Qi(K);

    /* Process first k (or first window)
     elements of array */
    int i;
    for (i = 0; i < K; ++i) {
```

```
        // For every element, the previous
        // smaller elements are useless so
        // remove them from Qi
        while ((!Qi.empty()) && arr[i] >= arr[Qi.back()])

            // Remove from rear
            Qi.pop_back();

        // Add new element at rear of queue
        Qi.push_back(i);
    }

    // Process rest of the elements,
    // i.e., from arr[k] to arr[n-1]
    for (; i < N; ++i) {

        // The element at the front of
        // the queue is the largest element of
        // previous window, so print it
        cout << arr[Qi.front()] << " ";

        // Remove the elements which
        // are out of this window
        while ((!Qi.empty()) && Qi.front() <= i - K)

            // Remove from front of queue
            Qi.pop_front();

        // Remove all elements
        // smaller than the currently
        // being added element (remove
        // useless elements)
        while ((!Qi.empty()) && arr[i] >= arr[Qi.back()])
            Qi.pop_back();

        // Add current element at the rear of Qi
        Qi.push_back(i);
    }

    // Print the maximum element
    // of last window
    cout << arr[Qi.front()];
}

// Driver's code
int main()
{
    int arr[] = { 12, 1, 78, 90, 57, 89, 56 };
```

```
    int N = sizeof(arr) / sizeof(arr[0]);
    int K = 3;

    // Function call
    printKMax(arr, N, K);
    return 0;
}
```

Output
```
78 90 90 90 89
```
Time Complexity: O(N). It seems more than O(N) at first look. It can be observed that every element of the array is added and removed at most once. So there are total of 2n operations.
Auxiliary Space: O(K). Elements stored in the dequeue take O(K) space.

Priority Queue

Skyline Problem | Priority Queue

Given n rectangular buildings in a 2-dimensional city, computes the skyline of these buildings, eliminating hidden lines. The main task is to view buildings from a side and remove all sections that are not visible. All buildings share common bottom and every building is represented by a triplet (left, height, right)

- **left**: is x coordinated of the left side (or wall).
- **height**: is the height of the building.
- **right**: is x coordinate of the right side

Examples:

Input: buildings[][] = { {1, 11, 5}, {2, 6, 7}, {3, 13, 9}, {12, 7, 16}, {14, 3, 25}, {19, 18, 22}, {23, 13, 29}, {24, 4, 28} }

Output: Skyline for given buildings:

{ {1, 11} {3, 13} {9, 0} {12, 7} {16, 3} {19, 18} {22, 3} {23, 13} {29, 0} }

Explanation:

The skyline is formed based on the keypoints (representing by "green" dots) eliminating hidden walls of the buildings.

Input: buildings[][] = { {1, 11, 5} }

Output: Skyline for given buildings:

{ {1, 11} {5, 0} }

Approach: ***Step1:*** From the triplet of buildings structure parameters, retrieve the left wall location, height and right wall location value. ***Step2:*** Store the left wall with its negative value of height and the right wall with its height value as a pair. Negative value of the height of left wall ensures it will be inserted to multiset first for the same abscissa(x) as right wall. ***Step3:*** Sort the walls in ascending order. ***Step4:*** Initialize a multiset to keep left wall heights sorted. ***Step5:*** Initialize current max height among all left wall heights (*integer variable: top*) ***Step6:*** While traversing through the walls: if the left wall is found, store the height of the left wall. Otherwise, for the right walls discard their heights. ***Step7:*** Through reverse operator on left wall heights, mark a skyline point if the value of *top* changes. ***Step8:*** Print the skyline as a set of keypoints.

C++ Implementation:

CPP

```
#include &lt;bits/stdc++.h&gt;
using namespace std;

// Function to create skyline
vector&lt;pair&lt;int, int&gt; &gt;
createSkyline(vector&lt;vector&lt;int&gt; &gt;&amp; buildings)
{
    // Get the number of buildings
    int N = buildings.size();

    // To store the left and right
    // wall position of the buildings
    vector&lt;pair&lt;int, int&gt; &gt; wall;

    // Triplet of building structure parameters
    int left, height, right;
    for (int i = 0; i &lt; N; i++) {
        // Get left point of building
        left = buildings[i][0];
        // Get height of building
        height = buildings[i][1];
        // Get right point of building
        right = buildings[i][2];
```

```
        // Store left point and height
        // of the left wall
        // Negative value ensures
        // left wall will be inserted to
        // multiset first for the
        // same abscissa(x) as right wall
        wall.push_back(make_pair(left, -height));

        // Store right point and height of the right wall
        wall.push_back(make_pair(right, height));
    }

    // Sort the walls in ascending order
    sort(wall.begin(), wall.end());

    // To store skyline: output
    vector&lt;pair&lt;int, int&gt; &gt; skyline;

    // Initialize a multiset to
    // keep left wall heights sorted
    multiset&lt;int&gt; leftWallHeight = { 0 };

    // Current max height among leftWallHeights
    int top = 0;

    // Traverse through the sorted walls
    for (auto w : wall) {
        // If left wall is found
        if (w.second &lt; 0) {
            // Insert the height
            leftWallHeight.insert(-w.second);
        }
        // If right wall is found
        else {
            // Remove the height
            leftWallHeight.erase(leftWallHeight.find(w.second));
        }

        // Mark a skyline point if top changes
        // .rbegin(): reverse iterator
        if (*leftWallHeight.rbegin() != top) {
            top = *leftWallHeight.rbegin();
            skyline.push_back(make_pair(w.first, top));
        }
    }

    // Return skyline to printSkyline
    return skyline;
}
```

```
// Function to print the output: skyline
void printSkyline(vector&lt;vector&lt;int&gt; &gt;&amp; buildings)
{
    // Function call for creating skyline
    vector&lt;pair&lt;int, int&gt; &gt; skyline =
createSkyline(buildings);
    cout &lt;&lt; &quot;Skyline for given buildings:&quot; &lt;&lt; endl;
    cout &lt;&lt; &quot;{ &quot;;
    for (auto it : skyline) {
        cout &lt;&lt; &quot;{&quot; &lt;&lt; it.first &lt;&lt; &quot;,
&quot;
             &lt;&lt; it.second &lt;&lt; &quot;} &quot;;
    }
    cout &lt;&lt; &quot;}&quot;;
}

// Main Funtion
int main()
{
    // Given matrix: left and right wall location and
    // height of the buildings in a view
    vector&lt;vector&lt;int&gt; &gt; buildings{ { 1, 11, 5 }, { 2, 6, 7 },
                                    { 3, 13, 9 }, { 12, 7, 16 },
                                    { 14, 3, 25 }, { 19, 18, 22 },
                                    { 23, 13, 29 }, { 24, 4, 28 } };

    // Function call to print the skyline
    printSkyline(buildings);
    return 0;
}
```

Output:

```
Skyline for given buildings:
{ {1, 11} {3, 13} {9, 0} {12, 7} {16, 3} {19, 18} {22, 3} {23,
13} {29, 0} }
```

Time Complexity: O(nlogn)

Median of Running Stream | Priority Queue

Given that integers are being read from a data stream. Find the median of all the elements read so far starting from the first integer till the last integer. This is also called the Median of Running Integers. The data stream can be any source of data, for example, a file, an array of integers, input stream etc.

What is Median?
Median can be defined as the element in the data set which separates the higher half of the data sample from the lower half. In other words, we can get the median element as, when the input size is odd, we take the middle element of sorted data. If the input size is even, we pick an average of middle two elements in the sorted stream.

Examples:

Input: 5 10 15
Output: 5, 7.5, 10
Explanation: Given the input stream as an array of integers [5,10,15]. Read integers one by one and print the median correspondingly. So, after reading first element 5,median is 5. After reading 10,median is 7.5 After reading 15 ,median is 10.
Input: 1, 2, 3, 4
Output: 1, 1.5, 2, 2.5
Explanation: Given the input stream as an array of integers [1, 2, 3, 4]. Read integers one by one and print the median correspondingly. So, after reading first element 1,median is 1. After reading 2,median is 1.5 After reading 3 ,median is 2.After reading 4 ,median is 2.5.

Approach: The idea is to use max heap and min heap to store the elements of higher half and lower half. Max heap and min heap can be implemented using priority_queue in C++ STL. Below is the step by step algorithm to solve this problem.

Algorithm:

1. Create two heaps. One max heap to maintain elements of lower half and one min heap to maintain elements of higher half at any point of time..
2. Take initial value of median as 0.
3. For every newly read element, insert it into either max heap or min-heap and calculate the median based on the following conditions:

    - If the size of max heap is greater than the size of min-heap and the element is less than the previous median then pop the top element from max heap and insert into min-heap and insert the new element to max heap else insert the new element to min-heap. Calculate the new median as the average of top of elements of both max and min heap.
    - If the size of max heap is less than the size of min-heap and the element is greater than the previous median then pop the top element from min-heap and insert into the max heap and insert the new element to min heap else

insert the new element to the max heap. Calculate the new median as the average of top of elements of both max and min heap.

- If the size of both heaps is the same. Then check if the current is less than the previous median or not. If the current element is less than the previous median then insert it to the max heap and a new median will be equal to the top element of max heap. If the current element is greater than the previous median then insert it to min-heap and new median will be equal to the top element of min heap.

Below is the implementation of above approach.

C++Java

```
// C++ program to find med in
// stream of running integers
#include<bits/stdc++.h>
using namespace std;

// function to calculate med of stream
void printMedians(double arr[], int n)
{
    // max heap to store the smaller half elements
    priority_queue<double> s;

    // min heap to store the greater half elements
    priority_queue<double,vector<double>,greater<double> > g;

    double med = arr[0];
    s.push(arr[0]);

    cout << med << endl;

    // reading elements of stream one by one
    /*  At any time we try to make heaps balanced and
        their sizes differ by at-most 1. If heaps are
        balanced,then we declare median as average of
        min_heap_right.top() and max_heap_left.top()
        If heaps are unbalanced,then median is defined
        as the top element of heap of larger size  */
    for (int i=1; i < n; i++)
    {
        double x = arr[i];

        // case1(left side heap has more elements)
        if (s.size() > g.size())
        {
            if (x < med)
```

```
            {
                g.push(s.top());
                s.pop();
                s.push(x);
            }
            else
                g.push(x);

            med = (s.top() + g.top())/2.0;
        }

        // case2(both heaps are balanced)
        else if (s.size()==g.size())
        {
            if (x < med)
            {
                s.push(x);
                med = (double)s.top();
            }
            else
            {
                g.push(x);
                med = (double)g.top();
            }
        }

        // case3(right side heap has more elements)
        else
        {
            if (x > med)
            {
                s.push(g.top());
                g.pop();
                g.push(x);
            }
            else
                s.push(x);

            med = (s.top() + g.top())/2.0;
        }

        cout << med << endl;
    }
}

// Driver program to test above functions
int main()
{
    // stream of integers
```

```
    double arr[] = {5, 15, 10, 20, 3};
    int n = sizeof(arr)/sizeof(arr[0]);
    printMedians(arr, n);
    return 0;
}
```

Output:

```
5
10
10
12.5
10
```

Complexity Analysis:

- **Time Complexity:** O(n Log n).
  Time Complexity to insert element in min heap is log n. So to insert n element is O(n log n).
- **Auxiliary Space :** O(n).
  The Space required to store the elements in Heap is O(n).

Trees

Trees - Module Walkthrough

What is Traversal?

Traversal is basically visiting all the nodes in a given binary tree using some way or the other.

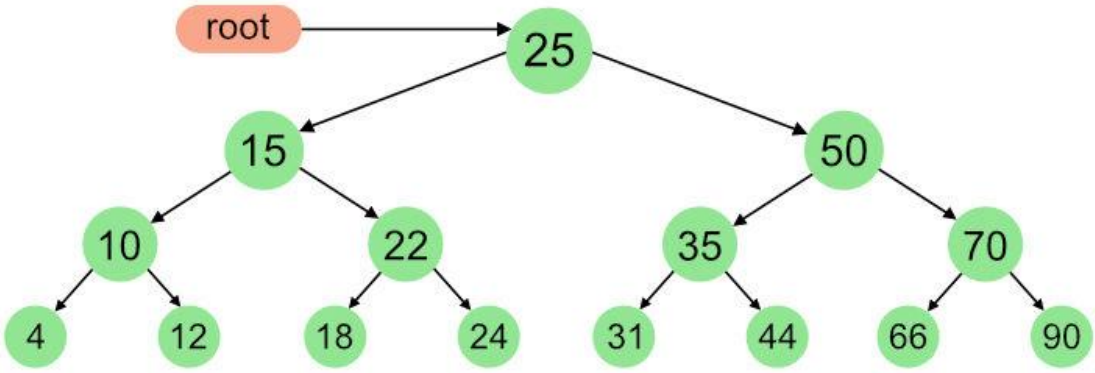
Types of Traversal in Binary Tree

1. Preorder Traversal
2. Inorder Traversal
3. Postorder Traversal
4. Level order Traversal

InOrder(root) visits nodes in the following order:
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



What is the Lowest Common Ancestor in a Binary Tree?

*The **lowest common ancestor** is the lowest node in the tree that has both n1 and n2 as **descendants,** where n1 and n2 are the nodes for which we wish to find the LCA. Hence, the LCA of a binary tree with nodes n1 and n2 is the shared ancestor of n1 and n2 that is located farthest from the root.*

Preorder Traversal Implementation

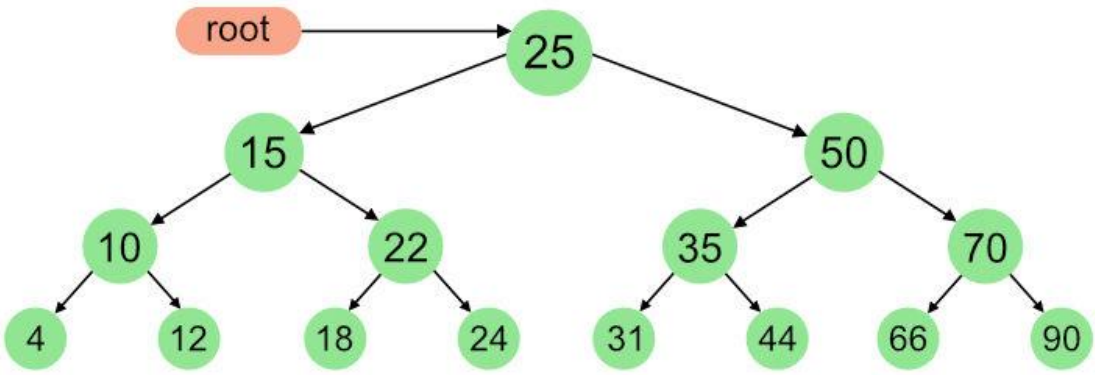
Preorder Traversal

In this traversal, we first visit the root node, then visit the left subtree recursively, and finally visit the right subtree recursively. For example, have a look at the figure below.

InOrder(root) visits nodes in the following order:
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



Algorithm Preorder(tree)

1. *Visit the root.*
2. *Traverse the left subtree, i.e., call Preorder(left->subtree)*
3. *Traverse the right subtree, i.e., call Preorder(right->subtree)*

Uses of Preorder:

Preorder traversal is used to create a copy of the tree. Preorder traversal is also used to get prefix expressions on an expression tree.

C

```c
void preorderTraversal(struct Node* root) {
        if (root == NULL) {
        return;
        }
        printf("%d ", root -> data);
        preorderTraversal(root -> left);
        preorderTraversal(root -> right);
}
```

C++

```cpp
void preorderTraversal(Node* root) {
```

```
        if (root == nullptr) {
                return;
        }

        cout << root -> data << " ";
        preorderTraversal(root -> left);
        preorderTraversal(root -> right);
}
```

Inorder Traversal Implementation

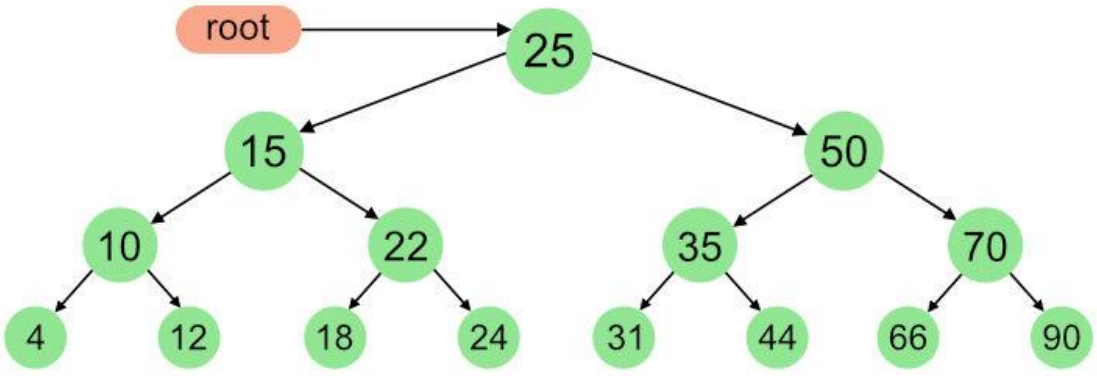
Inorder Traversal

In this traversal, we first visit the left subtree recursively, then visit the root node, and finally visit the right subtree recursively. This traversal results in visiting the nodes in ascending order when applied to a binary search tree. For example, refer to the figure below.

InOrder(root) visits nodes in the following order:
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



Algorithm Inorder(tree)

1. *Traverse the left subtree, i.e., call Inorder(left->subtree)*
2. *Visit the root.*
3. *Traverse the right subtree, i.e., call Inorder(right->subtree)*

Uses of Inorder Traversal:

In the case of binary search trees (BST), Inorder traversal gives nodes in non-decreasing order. To get nodes of BST in non-increasing order, a variation of Inorder traversal where Inorder traversal is reversed can be used.

C

```c
void inorderTraversal(struct TreeNode* root) {
    if(root == NULL) {
        return;
    }

    inorderTraversal(root -> left);
    printf("%d ", root -> val);
    inorderTraversal(root -> right);
}
```

C++

```cpp
void inorderTraversal(TreeNode* root) {
    if(root == NULL) {
        return;
    }

    inorderTraversal(root -> left);
    cout << root -> val << " ";
    inorderTraversal(root -> right);
}
```

Postorder Traversal Implementation

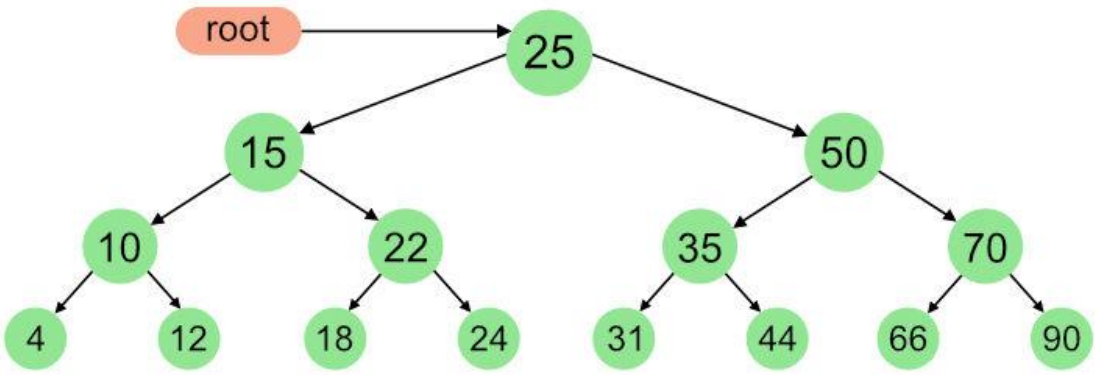
Postorder Traversal

In this traversal, we first visit the left subtree recursively, then visit the right subtree recursively, and finally visit the root node.

InOrder(root) visits nodes in the following order:
4, 10, 12, 15, 18, 22, 24, 25, 31, 35, 44, 50, 66, 70, 90

A Pre-order traversal visits nodes in the following order:
25, 15, 10, 4, 12, 22, 18, 24, 50, 35, 31, 44, 70, 66, 90

A Post-order traversal visits nodes in the following order:
4, 12, 10, 18, 24, 22, 15, 31, 44, 35, 66, 90, 70, 50, 25



Algorithm Postorder(tree)

1. *Traverse the left subtree, i.e., call Postorder(left->subtree)*
2. *Traverse the right subtree, i.e., call Postorder(right->subtree)*
3. *Visit the root*

Uses of Postorder:

Postorder traversal is used to delete the tree. Please see the question for the deletion of a tree for details. Postorder traversal is also useful to get the postfix expression of an expression tree.

C

```c
void postorder(struct Node* node) {
    if (node == NULL){
      return;
    }
    postorder(node -> left);
    postorder(node -> right);
    printf("%d ", node -> data);
}
```

C++

```
void postorder(Node* node) {
    if (node == nullptr){
        return;
    }

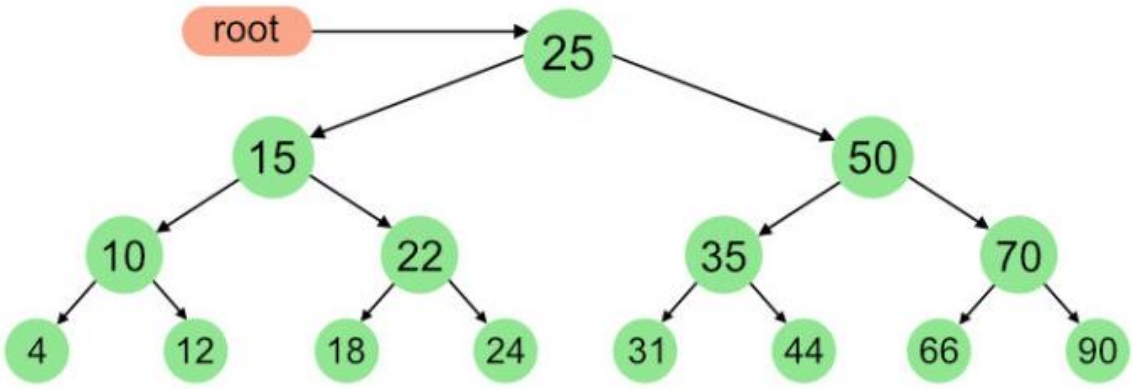
    postorder(node -> left);
    postorder(node -> right);
    cout << node -> data << " ";
}
```

Level order Tree Traversal

Level order Traversal

Level order traversal is another type of traversal in a binary tree. In level order traversal, we visit all nodes at the same level before moving on to the next level. This traversal starts at the root node and visits its left and right children, then visits the left and right children of the first level nodes, and so on. We can use a queue data structure to implement level order traversal.

Level order traversal is also known as breadth-first traversal because we visit nodes breadth-wise from left to right across each level of the tree.



Binary Tree

For the example above, the level order traversal would be:

```
25

15    50
```

```
10    22    35    70

4    12    18    24    31    44    66    90
```

Level Order Traversal Line by Line (Part 1)

Level Order Binary Tree Traversal Using Queue

For each node, first, the node is visited and then it's child nodes are put in a FIFO queue. Then again the first node is popped out and then it's child nodes are put in a FIFO queue and repeat until queue becomes empty.

Follow the below steps to Implement the above idea:

- Create an empty queue **q** and push **root** in **q**.
- Run a While loop until **q** is not empty.
    - Initialize temp_node = q.front() and print temp_node->data.
    - Push temp_node's children i.e. temp_node -> left then temp_node -> right to q
    - Pop front node from q.

C

```
struct Queue {
    int front, rear;
    int size;
    struct Node** arr;
};

struct Queue* createQueue(int size) {
    struct Queue* queue = (struct Queue*) malloc(sizeof(struct Queue));
    queue->front = queue->rear = -1;
    queue->size = size;
    queue->arr = (struct Node**) malloc(queue->size * sizeof(struct
Node*));
    return queue;
}

int isEmpty(struct Queue* queue) {
    return queue->front == -1;
}

int isFull(struct Queue* queue) {
    return queue->rear == queue->size - 1;
}

void enqueue(struct Queue* queue, struct Node* item) {
    if (isFull(queue)) {
```

```
        return;
    }
    queue->arr[++queue->rear] = item;
    if (queue->front == -1) {
        ++queue->front;
    }
}

struct Node* dequeue(struct Queue* queue) {
    if (isEmpty(queue)) {
        return NULL;
    }
    struct Node* item = queue->arr[queue->front];
    if (queue->front == queue->rear) {
        queue->front = queue->rear = -1;
    } else {
        ++queue->front;
    }
    return item;
}

void levelOrderTraversal(struct Node* root) {
    if (root == NULL) {
        return;
    }
    struct Queue* queue = createQueue(100);
    enqueue(queue, root);
    while (!isEmpty(queue)) {
        struct Node* node = dequeue(queue);
        printf("%d ", node->data);
        if (node->left) {
            enqueue(queue, node->left);
        }
        if (node->right) {
            enqueue(queue, node->right);
        }
    }
}

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

C++

```
void levelOrderTraversal(Node* root) {
    if (root == NULL) {
```

```
        return;
    }
    queue<Node*> q;
    q.push(root);
    while (!q.empty()) {
        Node* node = q.front();
        q.pop();
        cout << node->data << " ";
        if (node->left) {
            q.push(node->left);
        }
        if (node->right) {
            q.push(node->right);
        }
    }
}

Node* createNode(int data) {
    Node* newNode = new Node();
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}
```

Level Order Traversal Line by Line (Part 2)

Level Order Traversal Implementation using Queue - II
C++

```
#include <bits/stdc++.h>
using namespace std;

struct Node  {
  int key;
  struct Node *left;
  struct Node *right;
  Node(int k){
      key=k;
      left=right=NULL;
  }
};

void printLevel(Node *root){
    if(root==NULL)return;
    queue<Node *>q;
```

```
    q.push(root);
    while(q.empty()==false){
        int count=q.size();
        for(int i=0;i<count;i++){
        Node *curr=q.front();
        q.pop();
        cout<<curr->key<<" ";
        if(curr->left!=NULL)
            q.push(curr->left);
        if(curr->right!=NULL)
            q.push(curr->right);
        }
        cout<<"\n";
    }
}

int main() {

        Node *root=new Node(10);
        root->left=new Node(20);
        root->right=new Node(30);
        root->left->left=new Node(40);
        root->left->right=new Node(50);
        root->right->left=new Node(60);
        root->right->right=new Node(70);

        printLevel(root);
}
```

C

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int key;
    struct Node* left;
    struct Node* right;
};

struct Node* newNode(int k) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->key = k;
    node->left = NULL;
    node->right = NULL;
    return (node);
}

void printLevel(struct Node* root) {
    if (root == NULL)
```

```
        return;

    struct Node* queue[1000];
    int front = 0, rear = 0;

    queue[rear] = root;
    rear++;

    while (front < rear) {
        int count = rear - front;

        for (int i = 0; i < count; i++) {
            struct Node* curr = queue[front];
            front++;

            printf("%d ", curr->key);

            if (curr->left != NULL) {
                queue[rear] = curr->left;
                rear++;
            }

            if (curr->right != NULL) {
                queue[rear] = curr->right;
                rear++;
            }
        }
        printf("\n");
    }
}

int main() {
    struct Node* root = newNode(10);
    root->left = newNode(20);
    root->right = newNode(30);
    root->left->left = newNode(40);
    root->left->right = newNode(50);
    root->right->left = newNode(60);
    root->right->right = newNode(70);

    printLevel(root);

    return 0;
}
```

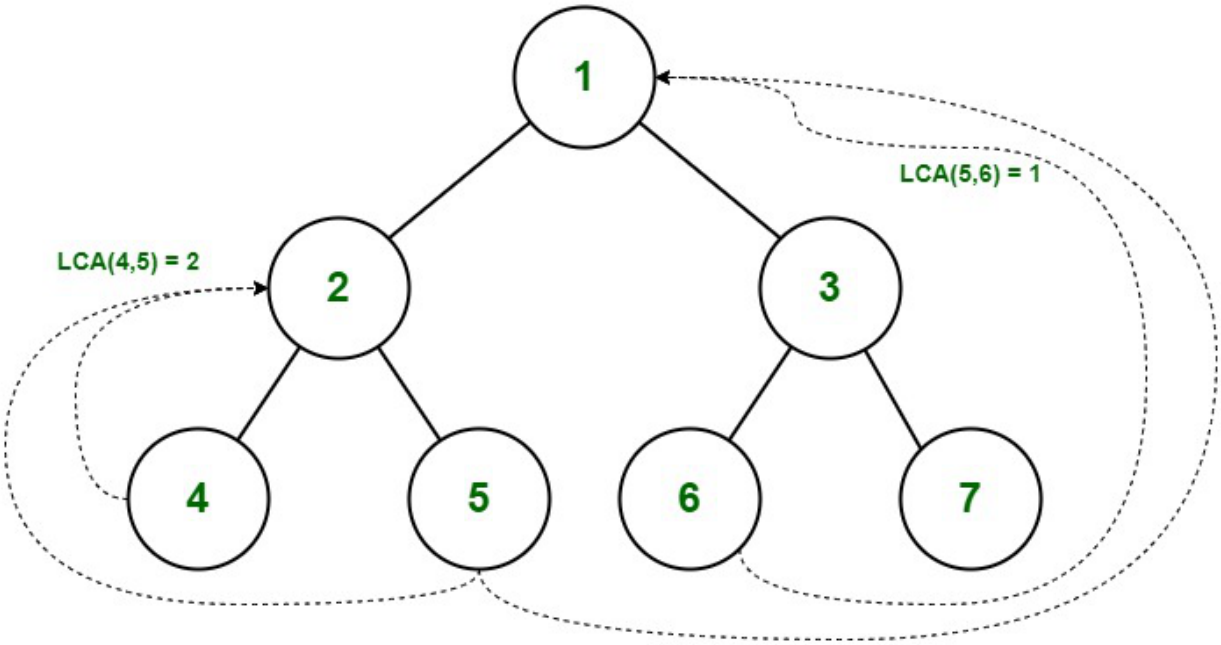
Here's the step-by-step approach of the level order traversal implemented in the given code:

- First, we create a **printLevel** function that takes the root node of the binary tree as its argument.
- Inside the **printLevel** function, we initialize a queue named q of type Node*. We push the root node into this queue using the push() method.
- Next, we start a while loop that will continue until the queue becomes empty.
- Inside the while loop, we use a for loop to iterate over all the nodes present at the current level of the binary tree. The count variable keeps track of the number of nodes present at the current level.
- In each iteration of the for loop, we dequeue the first node from the queue and assign it to a temporary pointer variable curr.
- We print the value of the key of the current node.
- Next, we check if the left child of the current node exists or not. If it exists, we push it into the queue using the push() method.
- Similarly, we check if the right child of the current node exists or not, and if it does, we push it into the queue using the push() method.
- After processing all the nodes at the current level, we print a new line character \n to move to the next level of the binary tree.
- The while loop continues until the queue becomes empty, which means we have traversed all the nodes of the binary tree.
- Finally, we create a main() function in which we create a binary tree by adding nodes and pass the root node of this binary tree to the printLevel function to perform level order traversal.

LCA of Binary Tree (Part I)

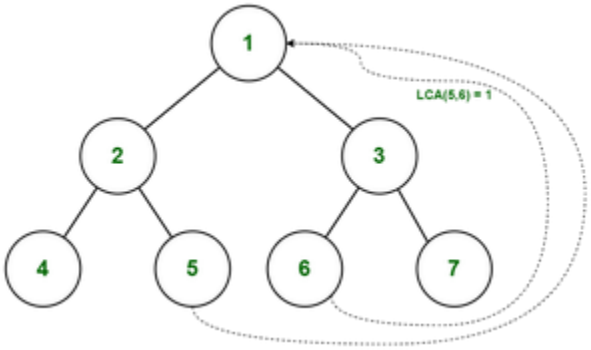
Lowest Common Ancestor

The lowest common ancestor is the lowest node in the tree that has both n1 and n2 as descendants, where n1 and n2 are the nodes for which we wish to find the LCA. Hence, the LCA of a binary tree with nodes n1 and n2 is the shared ancestor of n1 and n2 that is located farthest from the root.



Naive Solution

The naive solution to finding the LCA (Least Common Ancestor) of a binary tree using a two-path array approach involves storing the paths from the root to each of the nodes whose lowest common ancestor is to be computed. Once these paths are obtained, we can traverse the paths and find the last node that is common to both paths. This node will be the LCA of the given nodes.



Path from root to 5 = { 1, 2, 5 }
Path from root to 6 = { 1, 3, 6 }

We start checking from the 0 index. As both of the value matches(pathA[0] = pathB[0]), we move to the next index.
pathA[1] is not equal to pathB[1], there's a mismatch so we consider the previous value.
Therefore the LCA of (5,6) = 1

Follow the steps below to solve the problem:

- Find a path from the root to n1 and store it in a vector or array.
- Find a path from the root to n2 and store it in another vector or array.
- Traverse both paths till the values in arrays are the same. Return the common element just before the mismatch.

C

```c
#include <stdio.h>
#include <stdlib.h>
#include <stdbool.h>

struct Node {
    int key;
    struct Node *left, *right;
};

struct Node* newNode(int k){
    struct Node* temp = (struct Node*)malloc(sizeof(struct Node));
    temp->key = k;
    temp->left = temp->right = NULL;
    return temp;
}

bool findPath(struct Node* root, int path[], int pathLen, int k){
    if (root == NULL)
        return false;

    path[pathLen] = root->key;
    pathLen++;

    if (root->key == k)
        return true;

    if ((root->left && findPath(root->left, path, pathLen, k))
        || (root->right && findPath(root->right, path, pathLen, k)))
        return true;

    pathLen--;
    return false;
}

int findLCA(struct Node* root, int n1, int n2){
    int path1[1000], path2[1000];
    int pathLen1 = 0, pathLen2 = 0;

    if (!findPath(root, path1, pathLen1, n1)
        || !findPath(root, path2, pathLen2, n2))
        return -1;
```

```
    int i;
    for (i = 0; i < pathLen1 && i < pathLen2; i++)
        if (path1[i] != path2[i])
            break;
    return path1[i - 1];
}

int main(){
    struct Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->right->left = newNode(6);
    root->right->right = newNode(7);
    printf("LCA(4, 5) = %d\n", findLCA(root, 4, 5));
    printf("LCA(4, 6) = %d\n", findLCA(root, 4, 6));
    printf("LCA(3, 4) = %d\n", findLCA(root, 3, 4));
    printf("LCA(2, 4) = %d\n", findLCA(root, 2, 4));
    return 0;
}
```

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Node {
    int key;
    struct Node *left, *right;
};

Node* newNode(int k){
    Node* temp = new Node;
    temp->key = k;
    temp->left = temp->right = NULL;
    return temp;
}

bool findPath(Node* root, vector<int>& path, int k){
    if (root == NULL)
        return false;

    path.push_back(root->key);

    if (root->key == k)
        return true;

    if ((root->left && findPath(root->left, path, k))
```

```
        || (root->right && findPath(root->right, path, k)))
        return true;

    path.pop_back();
    return false;
}
int findLCA(Node* root, int n1, int n2){
    vector<int> path1, path2;

    if (!findPath(root, path1, n1)
        || !findPath(root, path2, n2))
        return -1;

    int i;
    for (i = 0; i < path1.size() && i < path2.size(); i++)
        if (path1[i] != path2[i])
            break;
    return path1[i - 1];
}

int main(){
    Node* root = newNode(1);
    root->left = newNode(2);
    root->right = newNode(3);
    root->left->left = newNode(4);
    root->left->right = newNode(5);
    root->right->left = newNode(6);
    root->right->right = newNode(7);
    cout << "LCA(4, 5) = " << findLCA(root, 4, 5);
    cout << "\nLCA(4, 6) = " << findLCA(root, 4, 6);
    cout << "\nLCA(3, 4) = " << findLCA(root, 3, 4);
    cout << "\nLCA(2, 4) = " << findLCA(root, 2, 4);
    return 0;
}
```

LCA of Binary Tree (Part 2)

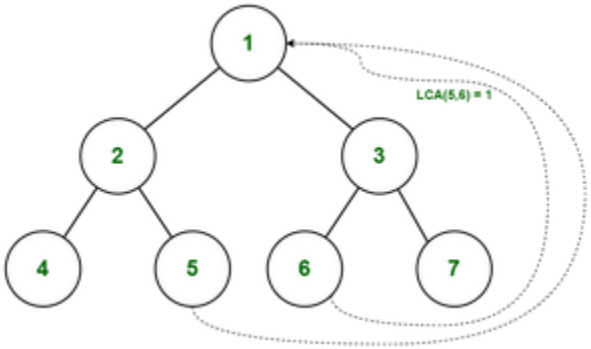
Lowest Common Ancestor in a Binary Tree By Single Traversal:

The idea is to traverse the tree starting from the root. If any of the given keys (n1 and n2) matches with the root, then the root is LCA (assuming that both keys are

present). If the root doesn't match with any of the keys, we recur for the left and right subtree.

- The node which has one key present in its left subtree and the other key present in the right subtree is the LCA.
- If both keys lie in the left subtree, then the left subtree has LCA also,
- Otherwise, LCA lies in the right subtree.

Illustration:



Find the LCA of 5 and 6

Root is pointing to the node with value 1, as its value doesn't match with { 5, 6 }. We look for the key in left subtree and right subtree.

- Left Subtree :
  - New Root = { 2 } ≠ 5 or 6, hence we will continue our recursion
  - New Root = { 4 } , it's left and right subtree is null, we will return NULL for this call
  - New Root = { 5 } , value matches with 5 so will return the node with value 5
  - The function call for root with value 2 will return a value of 5
- Right Subtree :
  - Root = { 3 } ≠ 5 or 6 hence we continue our recursion
  - Root = { 6 } = 5 or 6 , we will return the this node with value 6
  - Root = { 7 } ≠ 5 or 6, we will return NULL
  - So the function call for root with value 3 will return node with value 6
- As both the left subtree and right subtree of the node with value 1 is not NULL, so 1 is the LCA

Follow the steps below to solve the problem:

- We pass the root to a helper function and check if the value of the root matches any of n1 and n2.
  - If YES, return the root
  - else recursive call on the left and right subtree

- Basically, we do pre-order traversal, at first we check if the root->value matches with n1 or n2. Then traverse on the left and right subtree.
- If there is any root that returns one NULL and another NON-NULL value, we shall return the corresponding NON-NULL value for that node.
- The node that returns both NON-NULL values for both the left and right subtree, is our Lowest Common Ancestor.

Below is the implementation of the above approach:

Python3

```
# Python program to find LCA of n1 and n2 using one
# traversal of Binary tree

# A binary tree node


class Node:

    # Constructor to create a new tree node
    def __init__(self, key):
        self.key = key
        self.left = None
        self.right = None

# This function returns pointer to LCA of two given
# values n1 and n2
# This function assumes that n1 and n2 are present in
# Binary Tree


def findLCA(root, n1, n2):

    # Base Case
    if root is None:
        return None

    # If either n1 or n2 matches with root's key, report
    #  the presence by returning root (Note that if a key is
    #  ancestor of other, then the ancestor key becomes LCA
    if root.key == n1 or root.key == n2:
        return root

    # Look for keys in left and right subtrees
    left_lca = findLCA(root.left, n1, n2)
    right_lca = findLCA(root.right, n1, n2)

    # If both of the above calls return Non-NULL, then one key
    # is present in once subtree and other is present in other,
    # So this node is the LCA
```

```
    if left_lca and right_lca:
        return root

    # Otherwise check if left subtree or right subtree is LCA
    return left_lca if left_lca is not None else right_lca


# Driver code
if __name__ == '__main__':

    # Let us create a binary tree given in the above example
    root = Node(1)
    root.left = Node(2)
    root.right = Node(3)
    root.left.left = Node(4)
    root.left.right = Node(5)
    root.right.left = Node(6)
    root.right.right = Node(7)
    print("LCA(4, 5) = ", findLCA(root, 4, 5).key)
    print("LCA(4, 6) = ", findLCA(root, 4, 6).key)
    print("LCA(3, 4) = ", findLCA(root, 3, 4).key)
    print("LCA(2, 4) = ", findLCA(root, 2, 4).key)
```

Output

```
LCA(4, 5) = 2
LCA(4, 6) = 1
LCA(3, 4) = 1
LCA(2, 4) = 2
```

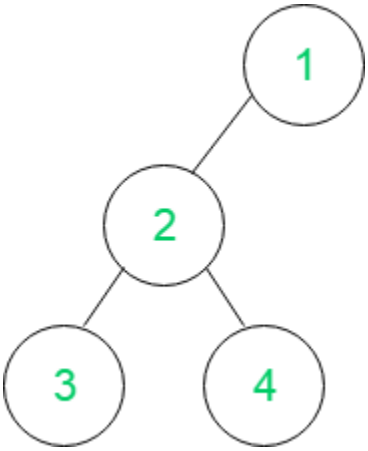
Time Complexity: O(N) as the method does a simple tree traversal in a bottom-up fashion.
Auxiliary Space: O(H), where H is the height of the tree.

Euler Tour of the Tree

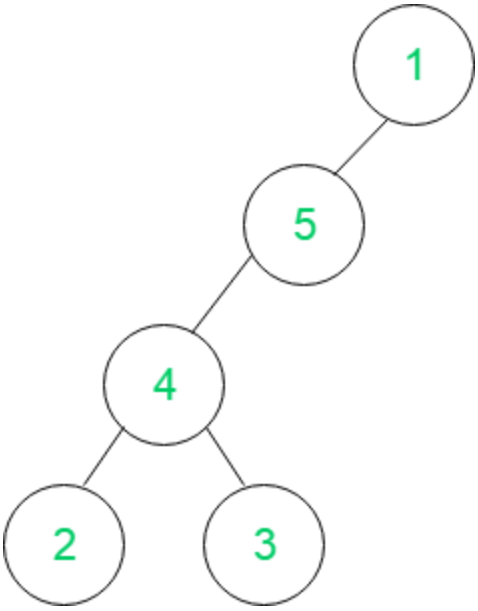
Euler Tour of the Tree

A Tree is a generalization of the connected graph where it has N nodes that will have exactly N-1 edges, i.e one edge between every pair of vertices. Find the Euler tour of the tree represented by the adjacency list.

Example:



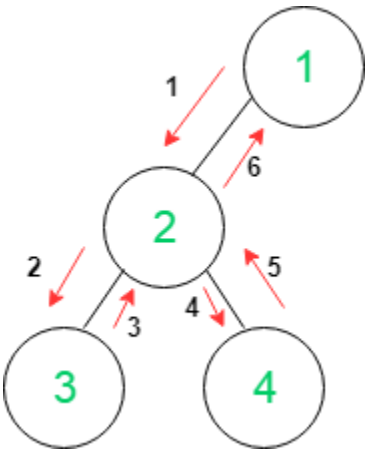
Output: 1 2 3 2 4 2 1



Output: 1 5 4 2 4 3 4 5 1

Euler tour is defined as a way of traversing a tree such that each vertex is added to the tour when we visit it (either moving down from the parent vertex or returning from the child vertex). We start from the root and reach back to the root after visiting all vertices. It requires exactly 2*N-1 vertices to store the **Euler tour**.

Approach: We will run DFS(Depth first search) algorithm on the Tree as:




```
1. Visit root node, i.e 1
vis[1]=1, Euler[0]=1
run dfs() for all unvisited adjacent nodes(2)
2. Visit node 2
vis[2]=1, Euler[1]=2
run dfs() for all unvisited adjacent nodes(3, 4)
3. Visit node 3
vis[3]=1, Euler[2]=3
All adjacent nodes are already visited, return to parent node
and add parent to Euler tour Euler[3]=2
4. Visit node 4
vis[4]=1, Euler[4]=4
All adjacent nodes are already visited, return to parent node
and add parent to Euler tour, Euler[5]=2
5. Visit node 2
All adjacent nodes are already visited, return to parent node
and add parent to Euler tour, Euler[6]=1
6. Visit node 1
All adjacent nodes are already visited, and node 1 is root node
so, we stop our recursion here.
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

#define MAX 1001

vector<int> adj[MAX];

int vis[MAX];

int Euler[2 * MAX];

void add_edge(int u, int v){
    adj[u].push_back(v);
    adj[v].push_back(u);
```

```
}

void eulerTree(int u, int &index){
    vis[u] = 1;
    Euler[index++] = u;
    for (auto it : adj[u]) {
        if (!vis[it]) {
            eulerTree(it, index);
            Euler[index++] = u;
        }
    }
}

void printEulerTour(int root, int N) {
    int index = 0;
    eulerTree(root, index);
    for (int i = 0; i < (2*N-1); i++)
        cout << Euler[i] << " ";
}

int main(){
    int N = 4;

    add_edge(1, 2);
    add_edge(2, 3);
    add_edge(2, 4);
    printEulerTour(1, N);

    return 0;
}
```

Euler Tour Problem

Subtree Updates

You are given a connected tree of **N** nodes numbered from **0** to **N-1** rooted at the **0th** node, where **p[i]** is the parent of **the ith** node and **p[0] = -1** as the 0th node is the root.
In the given tree each node has one of the two colors - black or white, if **A[i] = 1** then **the ith** node has **black** color else if **A[i] = 0** then **the ith** node has **white** color.
Each node also has a value, initially, the value of all nodes is 0.
You have to perform **Q** queries on the tree where the **ith** query is either of the two type:

1. **1 u x col**: you have to add value = **x** to every node of color = **col** which is in the subtree of

node **u (node inclusive).** ($0 \le u < N$, $1 \le x \le 10^9$ and $0 \le col \le 1$)
2. **2 u**: find the value of the node **u**. ($0 \le u < N$)

C++

```
#include <bits/stdc++.h>
using namespace std;

int cnt;

void eulerDFS(int cur, vector<vector<int>> &adj, vector<int> &euler,
              vector<int> &in, vector<int> &out){
    cnt++;
    in[cur] = cnt;
    euler.push_back(cur);
    for(int i = 0; i < adj[cur].size(); i++){
        eulerDFS(adj[cur][i], adj, euler, in, out);
    }
    cnt++;
    out[cur] = cnt;
    euler.push_back(cur);
}

vector<long long> seg_white(800100);
vector<long long> seg_black(800100);
vector<bool> marked(800100);
void push(int v){
    if(marked[v]){
        seg_white[2*v] += seg_white[v];
        seg_white[2*v+1] += seg_white[v];
        seg_black[2*v] += seg_black[v];
        seg_black[2*v+1] += seg_black[v];
        seg_white[v]=0;
        seg_black[v]=0;
        marked[v]=false;
        marked[2*v] = marked[2*v+1] = true;
    }
}
void build(int v, int l, int r){
    if(l == r){
        seg_white[v] = seg_black[v] = 0;
    }
    else{
        int m = (l+r)/2;
        build(2*v, l, m);
        build(2*v+1, m+1, r);
        seg_white[v] = seg_black[v] = 0;
    }
}
```

```
long long get(int v, int l, int r, int pos, int color){
    if(l == r){
        if(color == 1){
            return seg_black[v];
        }
        else{
            return seg_white[v];
        }
    }
    else{
        push(v);
        int m = (l+r)/2;
        if(pos <= m){
            return get(2*v, l, m, pos, color);
        }
        else{
            return get(2*v+1, m+1, r, pos, color);
        }
    }
}
void update(int v, int l, int r, int ql, int qr, int val, int color){
    if(l == ql && r == qr){
        if(color == 1){
            seg_black[v] += (long long )val;
        }
        else{
            seg_white[v] += (long long )val;
        }
        marked[v] = true;
    }
    else{
        push(v);
        int m = (l+r)/2;
        if(qr <= m){
            update(2*v, l, m, ql, qr, val, color);
        }
        else if(ql > m){
            update(2*v+1, m+1, r, ql, qr, val, color);
        }
        else{
            update(2*v, l, m, ql, m, val, color);
            update(2*v+1, m+1, r, m+1, qr, val, color);
        }
    }
}


int main(){
    int T;
```

```
    cin >> T;
    while(T--){
        int N, Q;
        cin >> N >> Q;
        vector<int> p(N), A(N);
        for(int i = 0; i < N; i++){
            cin >> p[i];
        }
        for(int i = 0; i < N; i++){
            cin >> A[i];
        }

        vector<vector<int>> adj(N);
        for(int i = 1; i < N; i++){
            adj[p[i]].push_back(i);
        }
        vector<int> euler{0};
        vector<int> in(N), out(N);
        cnt = 0;
        eulerDFS(0, adj, euler, in, out);
        int limit = (int )euler.size() - 1;
        build(1, 1, limit);



        vector<vector<int>> queries(Q);
        for(int i = 0; i < Q; i++){
            int type;
            cin >> type;
            if(type == 1){

                int u, x, col;
                cin >> u >> x >> col;

                update(1, 1, limit, in[u], out[u], x, col);
            }
            else{
                int u;
                cin >> u;
                cout << get(1, 1, limit, in[u], A[u]) << " ";

            }
        }
        cout << "\n";
    }
}
```

In the given solution, the eulerDFS function performs an Euler tour on the tree and records the order of nodes visited in the euler vector. It also records the entry and exit times of

each node in the in and out vectors respectively. These entry and exit times are used to identify the nodes that belong to the subtree of a particular node.

The segment tree is represented using three arrays - seg_white, seg_black, and marked. The seg_white array stores the sum of values of all nodes with color 0 while the seg_black array stores the sum of values of all nodes with color 1. The marked array keeps track of which nodes have been marked for update.

The push function propagates the updates from the parent node to its child nodes if the parent node has been marked for update. The build function initializes the values of seg_white and seg_black arrays to 0 for the leaf nodes. The get function returns the sum of values of the node at position pos based on its color. The update function updates the values of seg_white and seg_black arrays for the nodes that belong to the subtree of node u. It sets the marked flag for the parent node and propagates the update to its child nodes.

Finally, the main function reads in the queries and processes them sequentially. If the query is of type 1, it calls the update function to update the values of the nodes in the subtree of node u. If the query is of type 2, it calls the get function to retrieve the value of node u.

Overall, this solution uses Euler tour and segment tree data structure to efficiently process the given queries on a tree.

Greedy

Greedy - Module Walkthrough

What is a Greedy Algorithm?

A greedy algorithm is a type of algorithmic approach that always makes the locally optimal choice at each stage of the algorithm, with the goal of finding a globally optimal solution. In other words, at each step, the algorithm chooses the best option available to it, without worrying about the future consequences.

The greedy approach is often used when solving optimization problems, where the goal is to find the best solution out of a set of alternatives. Greedy algorithms are easy to code and computationally efficient, making them a popular choice in many applications.

One classic example of a greedy algorithm is the coin-changing problem. Given a set of coins with different denominations, and a target amount of money to make change for, the goal is to find the smallest number of coins needed to make up that amount. The greedy approach to this problem involves repeatedly selecting the largest denomination coin that can be used without exceeding the target amount until the total sum matches the target.

However, while greedy algorithms are simple and efficient, they may not always provide the optimal solution. In some cases, the locally optimal choice at one stage may lead to a suboptimal solution overall. For example, in the traveling salesman problem, a greedy algorithm that simply selects the nearest unvisited city at each stage may result in a longer overall route than a more complex algorithm that considers all possible paths.

Despite their limitations, greedy algorithms are still widely used in many applications, especially when efficiency is a primary concern. They can also serve as a useful starting point for more complex optimization approaches.

Greedy vs DP

How is a Greedy Algorithm different from Dynamic Programming?

While both greedy algorithms and dynamic programming are techniques used in algorithm design, there are significant differences between the two.

Greedy algorithms make local decisions at each step of the algorithm without considering the long-term consequences. They always choose the locally optimal solution that seems best at the moment, with the goal of finding a globally optimal solution in the end. In contrast, dynamic programming is a more systematic approach where the problem is broken down into smaller subproblems, and the solutions to those subproblems are stored and reused to solve larger problems.

The key difference between greedy algorithms and dynamic programming lies in their approach to decision-making. Greedy algorithms choose the option that looks best at the moment, while dynamic programming chooses the option that leads to the best outcome in the long run by considering all possible options and their outcomes.

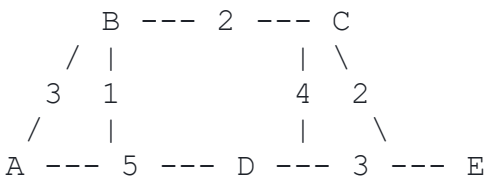
While greedy algorithms can be very efficient and easy to implement, they do not always provide the optimal solution. On the other hand, dynamic programming can guarantee an optimal solution, but it may be slower and more complex to implement.

In general, greedy algorithms are used when there is only one decision to be made at each step, whereas dynamic programming is used when there are multiple decisions to be made at each step, and the optimal solution depends on previous decisions.

Example:

To illustrate the difference between greedy algorithms and dynamic programming, let's consider the problem of finding the shortest path between two points in a graph.

A greedy approach to this problem would involve always choosing the next edge with the lowest weight at each step until you reach the destination. For example, if you were trying to get from point A to point E in the following graph:



A greedy algorithm would choose the edges (A, B), (B, C), (C, D), and (D, E) for a total weight of 9. However, the optimal solution is actually to take the edges (A, E) or (A, D, E) for a total weight of 8. Therefore, the greedy algorithm does not always provide the best solution.

In contrast, dynamic programming would break down the problem into smaller subproblems and store the solutions to those subproblems to solve larger problems. In this case, we could use Dijkstra's algorithm, which is an example of dynamic programming, to find the shortest path between two points. Dijkstra's algorithm considers all possible paths from the starting point and stores the shortest path to each subsequent node. This information is then used to find the overall shortest path from the starting point to the endpoint.

In summary, while a greedy algorithm may be simpler and faster to implement, it may not always provide the optimal solution. Dynamic programming, on the other hand, breaks the problem down into smaller subproblems and considers all possible options to find the optimal solution.

Policemen and Thieves

Policemen and Thieves

Given an array of size n that has the following specifications:

1. Each element in the array contains either a policeman or a thief.
2. Each policeman can catch only one thief.
3. A policeman cannot catch a thief who is more than K units away from the policeman.

We need to find the maximum number of thieves that can be caught.

Examples:

```
Input : arr[] = {'P', 'T', 'T', 'P', 'T'},
            k = 1.
Output : 2
Here maximum 2 thieves can be caught, first
policeman catches first thief and second police-
man can catch either second or third thief.

Input : arr[] = {'T', 'T', 'P', 'P', 'T', 'P'},
            k = 2.
Output : 3
```

A brute force approach would be to check all feasible sets of combinations of police and thief and return the maximum size set among them. Its time complexity is exponential and it can be optimized if we observe an important property.

An efficient solution is to use a greedy algorithm. But which greedy property to use can be tricky. We can try using: "**For each policeman from the left catch the nearest possible thief.**" This works, for example, three given above but fails for example two as it outputs 2 which is incorrect.

We may also try: "For each policeman from the left catch the farthest possible thief". This works, for example, two given above but fails for example three as it outputs 2 which is incorrect. A symmetric argument can be applied to show that traversing for these from the right side of the array also fails. We can observe that thinking irrespective of the policeman and focusing on just the allotment works:

1. Get the lowest index of policeman p and thief t. Make an allotment if |p-t| <= k and increment to the next p and t found.
2. Otherwise increment min(p, t) to the next p or t found.
3. Repeat the above two steps until the next p and t are found.
4. Return the number of allotments made.

Below is the implementation of the above algorithm. It uses vectors to store the indices of police and thief in the array and processes them.

C

```c
#include <stdio.h>
```

```c
#include <stdlib.h>

int policeThief(char arr[], int n, int k){
    int res = 0;
    int thi[n], pol[n];
    int thi_len = 0, pol_len = 0;

    for (int i = 0; i < n; i++) {
        if (arr[i] == 'P')
            pol[pol_len++] = i;
        else if (arr[i] == 'T')
            thi[thi_len++] = i;
    }

    int l = 0, r = 0;
    while (l < thi_len && r < pol_len) {
        if (abs(thi[l] - pol[r]) <= k) {
            l++;
            r++;
            res++;
        }
        else if (thi[l] < pol[r]) {
            l++;
        }
        else {
            r++;
        }
    }
    return res;
}

int main(){
    int k, n;
    char arr1[] = { 'P', 'T', 'T', 'P', 'T' };
    k = 2;
    n = sizeof(arr1) / sizeof(arr1[0]);
    printf("Maximum thieves caught: %d\n", policeThief(arr1, n, k));

    char arr2[] = { 'T', 'T', 'P', 'P', 'T', 'P' };
    k = 2;
    n = sizeof(arr2) / sizeof(arr2[0]);
    printf("Maximum thieves caught: %d\n", policeThief(arr2, n, k));

    char arr3[] = { 'P', 'T', 'P', 'T', 'T', 'P' };
    k = 3;
    n = sizeof(arr3) / sizeof(arr3[0]);
    printf("Maximum thieves caught: %d\n", policeThief(arr3, n, k));
    return 0;
}
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int policeThief(char arr[], int n, int k){
	int res = 0;
	vector<int> thi;
	vector<int> pol;

	for (int i = 0; i < n; i++) {
		if (arr[i] == 'P')
			pol.push_back(i);
		else if (arr[i] == 'T')
			thi.push_back(i);
	}
	int l = 0, r = 0;
	while (l < thi.size() && r < pol.size()) {
		if (abs(thi[l] - pol[r]) <= k) {
			l++;
			r++;
			res++;
		}
		else if (thi[l] < pol[r]) {
			l++;
		}
		else {
			r++;
		}
	}
	return res;
}

int main(){
	int k, n;
	char arr1[] = { 'P', 'T', 'T', 'P', 'T' };
	k = 2;
	n = sizeof(arr1) / sizeof(arr1[0]);
	cout << "Maximum thieves caught: "
		<< policeThief(arr1, n, k) << endl;

	char arr2[] = { 'T', 'T', 'P', 'P', 'T', 'P' };
	k = 2;
	n = sizeof(arr2) / sizeof(arr2[0]);
	cout << "Maximum thieves caught: "
		<< policeThief(arr2, n, k) << endl;

	char arr3[] = { 'P', 'T', 'P', 'T', 'T', 'P' };
	k = 3;
```

```
        n = sizeof(arr3) / sizeof(arr3[0]);
        cout << "Maximum thieves caught: "
               << policeThief(arr3, n, k) << endl;
        return 0;
}
```

Time Complexity: O(N)
Auxiliary Space: O(N)

Fractional Knapsack Problem

Given the weights and values of **N** items, put these items in a knapsack of capacity **W** to get the maximum total value in the knapsack. In **Fractional Knapsack**, we can break items for maximizing the total value of the knapsack

Note: In the 0-1 Knapsack problem, we are not allowed to break items. We either take the whole item or don't take it.

Input:
Items as (value, weight) pairs
arr[] = {{60, 10}, {100, 20}, {120, 30}}
Knapsack Capacity, W = 50

Output: *Maximum possible value = 240*
Explanation: *by taking items of weight 10 and 20 kg and 2/3 fraction of 30 kg.*
Hence total price will be 60+100+(2/3)(120) = 240

Input:
Items as (value, weight) pairs
arr[] = {{500, 30}}
Knapsack Capacity, W = 10
Output: *166.667*

Naive Approach: Try all possible subsets with all different fractions but that will be very inefficient.

Greedy approach for fractional knapsack problem:

*An **efficient solution** is to use the Greedy approach. The basic idea of the greedy approach is to calculate the ratio value/weight for each item and sort the item on the basis of this ratio. Then take the item with the highest ratio and add them until we can't add the next item as a whole and at the end add the next item as much as we can. Which will always be the optimal solution to this problem.*

Follow the given steps to solve the problem using the above approach:

- Calculate the ratio(value/weight) for each item.
- Sort all the items in decreasing order of the ratio.
- Initialize res =0, curr_cap = given_cap.
- Do the following for every item "i" in the sorted order:
  - If the weight of the current item is less than or equal to the remaining capacity then add the value of that item into the result
  - else add the current item as much as we can and break out of the loop
- Return res

Below is the implementation of the above approach:

Python

```
# Structure for an item which stores weight and
# corresponding value of Item
class Item:
	def __init__(self, value, weight):
		self.value = value
		self.weight = weight

# Main greedy function to solve problem
def fractionalKnapsack(W, arr):

	# sorting Item on basis of ratio
	arr.sort(key=lambda x: (x.value/x.weight), reverse=True)

	# Uncomment to see new order of Items with their
	# ratio
	# for item in arr:
	#	print(item.value, item.weight, item.value/item.weight)

	# Result(value in Knapsack)
	finalvalue = 0.0

	# Looping through all Items
	for item in arr:

		# If adding Item won't overflow, add it completely
```

```
                if item.weight <= W:
                        W -= item.weight
                        finalvalue += item.value

                # If we can't add current Item, add fractional part
                # of it
                else:
                        finalvalue += item.value * W / item.weight
                        break
        # Returning final value
        return finalvalue


# Driver's Code
if __name__ == "__main__":

        # Weight of Knapsack
        W = 50
        arr = [Item(60, 10), Item(100, 20), Item(120, 30)]

        # Function call
        max_val = fractionalKnapsack(W, arr)
        print ('Maximum value we can obtain = {}'.format(max_val))
```

Output

```
Maximum value we can obtain = 240
```

Time Complexity: O(N log N)
Auxiliary Space: O(N)

Greedy | XOR Problem

XOR Problem

Given an array A of length n, we are allowed to do the following operations on it at any number of times:

- Pick two adjacent elements; then remove them and place a single integer in their place: **their bitwise XOR.** Note that the length of the array decreases by one.

Determine if it is possible to make all the elements of the resultant array equal, after performing some operations. Number of elements in the resultant array should be at least 2.

Example:

```
Input: 3
          2 0 2
Output: YES
Explanation: 2 at the 0th index and 0 and the 1st index can be
XORed to get 2. So, after this operation, the array looks like:
[2, 2]. So we have all the elements the same and there are at
least 2 elements in the array.
Input: 4
          2 3 1 1 0
Output: NO
```

Greedy Approach

We can sort the input array in non-increasing order and then repeatedly apply the XOR operation on adjacent elements, starting from the highest bit set. At each step, we merge the two adjacent elements by taking their XOR, and remove them from the array. We repeat this process until there are less than two elements left in the array.

If at any point during this process, all the remaining elements in the array are equal, we return true. Otherwise, we return false.

The intuition behind this algorithm is that by repeatedly taking the XOR of adjacent elements with the highest bit set, we can "merge" those two elements into one element with a lower highest bit. This makes it more likely that we can eventually find a common value for all elements in the array.

C

```c
#include <stdio.h>
#include <stdlib.h>

int cmp(const void *a, const void *b) {
    return *(int*)b - *(int*)a;
}

int canMakeEqual(int* arr, int n){
    // Sort the array in non-increasing order
    qsort(arr, n, sizeof(int), cmp);

    // Apply XOR operations on adjacent elements
    while (n >= 2) {
        int x = arr[n - 1];
        int y = arr[n - 2];
        int z = x ^ y;
        if (z > x && z > y) {
```

```
            // The result of the XOR operation is greater than both x and
y,
            // which means we cannot make all elements equal
            return 0;
        }
        arr[n-1] = z;
        n--;
    }

    return (n == 1 ? 0 : 1);
}

int main(){
    int n;
    scanf("%d", &n);

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    if (canMakeEqual(arr, n)) {
        printf("YES\n");
    } else {
        printf("NO\n");
    }

    return 0;
}
```

C++

```
#include <iostream>
#include <vector>
#include <algorithm>

using namespace std;

bool canMakeEqual(vector<int>& arr){
    int n = arr.size();

    // Sort the array in non-increasing order
    sort(arr.begin(), arr.end(), greater<>());

    // Apply XOR operations on adjacent elements
    while (n >= 2) {
        int x = arr[n - 1];
        int y = arr[n - 2];
        int z = x ^ y;
        if (z > x && z > y) {
```

```
            // The result of the XOR operation is greater than both x and
y,
            // which means we cannot make all elements equal
            return false;
        }
        arr.pop_back();
        arr.pop_back();
        if (z != 0) {
            arr.push_back(z);
            n--;
        }
    }

    return false;
}

int main(){
    int n;
    cin >> n;

    vector<int> arr(n);
    for (int i = 0; i < n; i++) {
        cin >> arr[i];
    }

    if (canMakeEqual(arr)) {
        cout << "YES" << endl;
    } else {
        cout << "NO" << endl;
    }

    return 0;
}
```

Time complexity: O(nlogn)
Space Complexity: O(n)

Job Sequencing

Job Sequencing Problem

Given an array of jobs where every job has a deadline and associated profit if the job is finished before the deadline. It is also given that every job takes a single unit of time, so the

minimum possible deadline for any job is 1. Maximize the total profit if only one job can be scheduled at a time.

```
Input: Four Jobs with following deadlines and profits
JobID  Deadline  Profit
  a           4          20
  b           1          10
  c           1          40
  d          1          30
Output: Following is maximum profit sequence of jobs: c, a

Input:  Five Jobs with following deadlines and profits
JobID   Deadline  Profit
  a            2          100
  b            1          19
  c            2          27
 d            1          25
 e            3          15
Output: Following is maximum profit sequence of jobs: c, a, e
```

Greedy Approach

Greedily choose the jobs with maximum profit first, by sorting the jobs in decreasing order of their profit. This would help to maximize the total profit as choosing the job with maximum profit for every time slot will eventually maximize the total profit

Follow the given steps to solve the problem:

- Sort all jobs in decreasing order of profit.
- Iterate on jobs in decreasing order of profit.For each job , do the following :
  - Find a time slot i, such that slot is empty and i < deadline and i is greatest.Put the job in
    this slot and mark this slot filled.
  - If no such i exists, then ignore the job.

Below is the implementation of the above approach:

C

```c
#include <stdbool.h>
#include <stdio.h>
#include <stdlib.h>

// A structure to represent a job
typedef struct Job {

        char id; // Job Id
        int dead; // Deadline of job
        int profit; // Profit if job is over before or on
```

```
                                // deadline
} Job;

int compare(const void* a, const void* b){
        Job* temp1 = (Job*)a;
        Job* temp2 = (Job*)b;
        return (temp2->profit - temp1->profit);
}

// Find minimum between two numbers.
int min(int num1, int num2){
        return (num1 > num2) ? num2 : num1;
}

// Returns maximum profit from jobs
void printJobScheduling(Job arr[], int n){
        // Sort all jobs according to decreasing order of profit
        qsort(arr, n, sizeof(Job), compare);

        int result[n]; // To store result (Sequence of jobs)
        bool slot[n]; // To keep track of free time slots

        // Initialize all slots to be free
        for (int i = 0; i < n; i++)
                slot[i] = false;

        // Iterate through all given jobs
        for (int i = 0; i < n; i++) {

                // Find a free slot for this job (Note that we start
                // from the last possible slot)
                for (int j = min(n, arr[i].dead) - 1; j >= 0; j--) {

                        // Free slot found
                        if (slot[j] == false) {
                                result[j] = i; // Add this job to result
                                slot[j] = true; // Make this slot occupied
                                break;
                        }
                }
        }

        // Print the result
        for (int i = 0; i < n; i++)
                if (slot[i])
                        printf("%c ", arr[result[i]].id);
}

int main(){
```

```
        Job arr[] = { { 'a', 2, 100 },
                      { 'b', 1, 19 },
                      { 'c', 2, 27 },
                      { 'd', 1, 25 },
                      { 'e', 3, 15 } };
        int n = sizeof(arr) / sizeof(arr[0]);
        printf(
               "Following is maximum profit sequence of jobs \n");

        // Function call
        printJobScheduling(arr, n);
        return 0;
}
```

C++

```
#include <algorithm>
#include <iostream>
using namespace std;

struct Job {

        char id; // Job Id
        int dead; // Deadline of job
        int profit; // Profit if job is over before or on
                    // deadline
};

// Comparator function for sorting jobs
bool comparison(Job a, Job b){
        return (a.profit > b.profit);
}

// Returns maximum profit from jobs
void printJobScheduling(Job arr[], int n){
        // Sort all jobs according to decreasing order of profit
        sort(arr, arr + n, comparison);

        int result[n]; // To store result (Sequence of jobs)
        bool slot[n]; // To keep track of free time slots

        // Initialize all slots to be free
        for (int i = 0; i < n; i++)
               slot[i] = false;

        // Iterate through all given jobs
        for (int i = 0; i < n; i++) {
               // Find a free slot for this job (Note that we start
               // from the last possible slot)
               for (int j = min(n, arr[i].dead) - 1; j >= 0; j--) {
```

```
                        // Free slot found
                        if (slot[j] == false) {
                                result[j] = i; // Add this job to result
                                slot[j] = true; // Make this slot occupied
                                break;
                        }
                }
        }

        // Print the result
        for (int i = 0; i < n; i++)
                if (slot[i])
                        cout << arr[result[i]].id << " ";
}

int main(){
        Job arr[] = { { 'a', 2, 100 },
                                { 'b', 1, 19 },
                                { 'c', 2, 27 },
                                { 'd', 1, 25 },
                                { 'e', 3, 15 } };

        int n = sizeof(arr) / sizeof(arr[0]);
        cout << "Following is maximum profit sequence of jobs "
                        "\n";

        printJobScheduling(arr, n);
        return 0;
}
```

Time complexity: O(n²) in the worst case; while O(nlogn) in an average case
Space Complexity: O(n)

Dijkstra on Grid

Let's have a look at a very similar problem as discussed in the video.

Infinite paths
Given a directed weighted graph **G** and a source node **src** in the graph, for every node **x** in **G**, find the cost of the minimum path from **src** to **x**.

Input

- The first line of each test contains **T**, the number of test cases**. T** tests follow.

- The first line of each test contains three integers **N** (number of nodes), **M** (number of edges), and **src** (source node)
- Next **M** lines contain a description of **M** edges in the form of three integers **u, v,** and **w**, indicating that there is a directed edge from **u** to **v** with cost **w**

Output

- For every test case print **N** lines.
- If the $\mathbf{i}_{th}$ node is reachable from **src** print the minimum cost to reach **i** in the $\mathbf{i}_{th}$ line.
- If the $\mathbf{i}_{th}$ node is not reachable print "UNREACHABLE" without quotes in the $\mathbf{i}_{th}$ line.
- If the $\mathbf{i}_{th}$ node is reachable but there is no minimum cost to reach there, print "-inf" without quotes in the $\mathbf{i}_{th}$ line.

In this program, we first take input the number of test cases T. Then for each test case, we take input the number of nodes N, the number of edges M, and the source node src. We then create an adjacency list representation of the weighted directed graph using a vector of pairs.

We then call the Dijkstra function which takes as input the adjacency list, the number of nodes, and the source node. In the function, we first initialize a priority queue to store vertices in increasing order of their distance from the source vertex. We also initialize two vectors - dist to store the minimum distance from the source node to every other node and visited to keep track of whether or not a node has been visited.

We set the distance of the source node to 0 and push it into the priority queue. We then loop until the priority queue becomes empty. In each iteration, we extract the vertex with the minimum distance from the priority queue and mark it visited. We then iterate through all its adjacent vertices and update their distance if it can be reduced by going through the current vertex. Finally, we print the minimum distance from the source node to every other node.

Note: This program assumes that there are no negative weight cycles in the graph. If there are negative weight cycles, then the algorithm will go into an infinite loop.

C

```c
#include<stdio.h>
#include<stdlib.h>
#include<limits.h>

#define INF INT_MAX
#define N 100005

typedef struct node {
    int to, weight;
} node;
```

```c
typedef struct pair {
    int first, second;
} pair;

pair make_pair(int a, int b) {
    pair p = {a, b};
    return p;
}

typedef struct priority_queue {
    pair heap[N];
    int size;
} priority_queue;

void push(priority_queue *pq, pair p) {
    pq->heap[++pq->size] = p;
    int i = pq->size;
    while(i > 1 && pq->heap[i].first < pq->heap[i/2].first) {
        pair temp = pq->heap[i];
        pq->heap[i] = pq->heap[i/2];
        pq->heap[i/2] = temp;
        i = i/2;
    }
}

pair top(priority_queue *pq) {
    return pq->heap[1];
}

void pop(priority_queue *pq) {
    pq->heap[1] = pq->heap[pq->size--];
    int i = 1;
    while(2*i <= pq->size) {
        int child = 2*i;
        if(child+1 <= pq->size && pq->heap[child+1].first < pq-
>heap[child].first)
            child++;
        if(pq->heap[i].first > pq->heap[child].first) {
            pair temp = pq->heap[i];
            pq->heap[i] = pq->heap[child];
            pq->heap[child] = temp;
        }
        else
            break;
        i = child;
    }
}
```

```
int empty(priority_queue *pq) {
    return pq->size == 0;
}

void Dijkstra(node **adj, int V, int src) {
    priority_queue pq;
    pq.size = 0;
    int dist[N], visited[N];
    for(int i=1; i<=V; i++) {
        dist[i] = INF;
        visited[i] = 0;
    }

    push(&pq, make_pair(0, src));
    dist[src] = 0;

    while(!empty(&pq)) {
        int u = top(&pq).second;
        pop(&pq);
        visited[u] = 1;

        node *temp = adj[u];
        while(temp != NULL) {
            int v = temp->to;
            int weight = temp->weight;
            if(!visited[v] && dist[u]!=INF && dist[u]+weight < dist[v]) {
                dist[v] = dist[u] + weight;
                push(&pq, make_pair(dist[v], v));
            }
            temp = temp->next;
        }
    }
    for(int i=1; i<=V; i++) {
        if(dist[i] == INF)
            printf("UNREACHABLE\n");
        else if(i != src)
            printf("%d\n", dist[i]);
    }
}

int main() {
    int T;
    scanf("%d", &T);
    while(T--) {
        int N, M, src;
        scanf("%d %d %d", &N, &M, &src);
        node *adj[N+1];
        for(int i=1; i<=N; i++)
            adj[i] = NULL;
```

```
        for(int i=0; i<M; i++) {
            int u, v, w;
            scanf("%d %d %d", &u, &v, &w);
            node *new_node = (node*)malloc(sizeof(node));
            new_node->to = v;
            new_node->weight = w;
            new_node->next = adj[u];
            adj[u] = new_node;
        }
        Dijkstra(adj, N, src);
    }
    return 0;
}
```

C++

```
#include<bits/stdc++.h>
using namespace std;
#define INF INT_MAX
typedef pair<int, int> iPair;

void Dijkstra(vector<pair<int,int>> adj[], int V, int src) {
    priority_queue<iPair, vector<iPair>, greater<iPair>> pq;
    vector<int> dist(V+1, INF);
    vector<bool> visited(V+1, false);

    pq.push(make_pair(0, src));
    dist[src] = 0;

    while(!pq.empty()) {
        int u = pq.top().second;
        pq.pop();
        visited[u] = true;

        for(auto it : adj[u]) {
            int v = it.first;
            int weight = it.second;
            if(!visited[v] && dist[u]!=INF && dist[u]+weight < dist[v]) {
                dist[v] = dist[u] + weight;
                pq.push(make_pair(dist[v], v));
            }
        }
    }
    for(int i=1; i<=V; i++) {
        if(dist[i] == INF)
            cout << "UNREACHABLE" << endl;
        else if(i != src)
            cout << dist[i] << endl;
    }
}
```

```
int main() {
    int T;
    cin >> T;
    while(T--) {
        int N, M, src;
        cin >> N >> M >> src;
        vector<pair<int,int>> adj[N+1];
        for(int i=0; i<M; i++) {
            int u, v, w;
            cin >> u >> v >> w;
            adj[u].push_back(make_pair(v,w));
        }
        Dijkstra(adj, N, src);
    }
    return 0;
}
```

Make Permutation

Make Permutations

You are given an array of integers a of length n. Each element of the array is colored either blue or red. There are no unpainted elements in the array. One of the two operations described below can be applied on an array in a single step:

- either you can select blue element and decrease its value by 1
- or you can select any red element and increase the value by 1.

Situations in which there are no elements of some color at all are also possible. For example, if the whole array is colored blue or red, one of the operations becomes unavailable. Determine whether it is possible to make 0 or more steps such that the resulting array is a permutation of numbers from 1 to n.

Example:
Input:
1 2 5 2
B R B R
Output: Yes
Explanation:
1 -> 1
2 -> 2
5 -> 3
2 -> 4

Greedy Approach

The intuition behind the solution is to use a simple and efficient approach to check whether it is possible to obtain a permutation of 1 to n using the given array and color arrays.

The first step in the algorithm is to separate the elements of the input array into two vectors based on their color. This is because the two types of elements (red and blue) require different operations to transform the array into a permutation of 1 to n.

Next, we sort both vectors in non-decreasing order. Sorting allows us to easily compare each element with its corresponding index later on.

Then, we iterate over the blue vector and check whether any element is already equal to its corresponding index plus one. If such an element exists, it means that we cannot transform the array into a permutation of 1 to n, since there will be duplicate elements. This is because elements with different indices are required to have different values for a permutation to exist.

Similarly, we iterate over the red vector and check whether any element is greater than n-i, where i is the current index. If such an element exists, it means that we cannot transform the array into a permutation of 1 to n, since there will be at least one element greater than n. This is because we must have n distinct elements in a permutation of 1 to n.

If neither of the above conditions is met, it means that it is possible to transform the array into a permutation of 1 to n, and hence, we return true.

C

```c
int cmpfunc(const void *a, const void *b){
    return (*(int*)a - *(int*)b);
}

int makePermutation(int *a, char *colour, int n){
    int i, j;
    int *red = (int *)malloc(n*sizeof(int));
    int *blue = (int *)malloc(n*sizeof(int));

    j = 0;
    for(i=0; i<n; i++){
        if(colour[i] == 'R')
            red[j++] = a[i];
        else
            blue[i-j] = a[i];
    }
```

```
    qsort(blue, n-j, sizeof(int), cmpfunc);
    qsort(red, j, sizeof(int), cmpfunc);

    for(i=0; i<n-j; i++){
        if(blue[i] == i+1)
            return 0;
    }

    for(i=0; i<j; i++){
        if(red[i] > n-i)
            return 0;
    }

    free(red);
    free(blue);
    return 1;
}
```

C++

```
bool makePermutation(vector<int> &a, string &colour){
        int n = a.size();
        vector<int> red, blue;

        for(int i = 0; i < n; i++){
        if(colour[i] == 'R')
                red.emplace_back(a[i]);
         else
                blue.emplace_back(a[i]);
    }

        sort(blue.begin(), blue.end());
        sort(red.begin(), red.end());

        for(int i = 0; i < blue.size(); i++){
        if(blue[i] == i + 1)
                return false;
    }

        for(int i = 0; i < red.size(); i++){
        if(red[i] > n - i)
                return false;
    }

        return true;
}
```

Time complexity: O(nlogn)
Space Complexity: O(n)

DP – Basics:

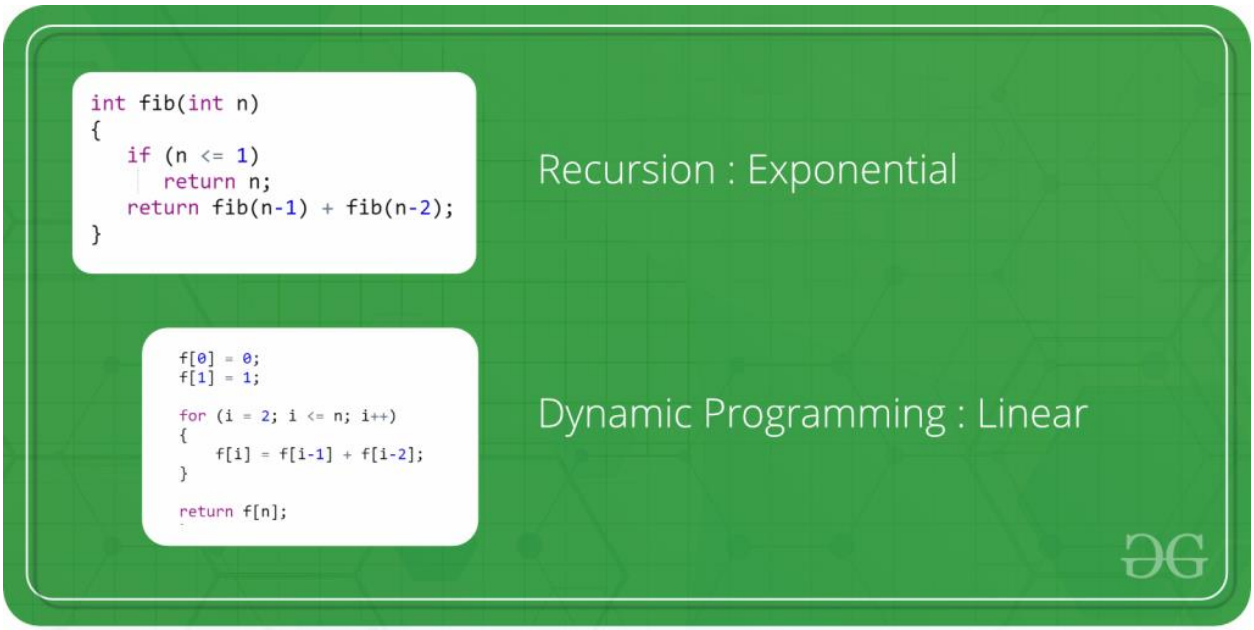
Standard DP Problems:

DP Module Walkthrough

What is Dynamic Programming?

Dynamic Programming is mainly an optimization over plain recursion. Wherever we see a recursive solution with repeated calls for the same inputs, we can optimize it using Dynamic Programming. The idea is to store the results of subproblems so that we do not have to re-compute them when needed later. This simple optimization reduces time complexities from exponential to polynomial.

For example, let's write a simple recursive solution for Fibonacci Numbers. We get exponential time complexity and if we optimize it by storing solutions of subproblems, time complexity reduces to linear.



Mark as Read

Report An Issue

Subset Sum Problem (Recursive Solution)

Subset Sum Problem

Given a set of non-negative integers and a value *sum*, determine if a subset of the given set has a sum equal to a given *sum*.

Example:

```
Input: set[] = {3, 34, 4, 12, 5, 2}, sum = 9
Output: True
There is a subset (4, 5) with sum 9.

Input: set[] = {3, 34, 4, 12, 5, 2}, sum = 30
Output: False
There is no subset that add up to 30.
```

Recursive Solution

Approach: For the recursive approach we will consider two cases.

1. Consider the last element and now the **required sum = target sum – value of 'last' element** and **number of elements = total elements – 1**
2. Leave the 'last' element and now the **required sum = target sum** and **number of elements = total elements – 1**

Following is the recursive formula for **isSubsetSum()** problem.

```
isSubsetSum(set, n, sum)
= isSubsetSum(set, n-1, sum) ||
  isSubsetSum(set, n-1, sum-set[n-1])
Base Cases:
isSubsetSum(set, n, sum) = false, if sum > 0 and n == 0
isSubsetSum(set, n, sum) = true, if sum == 0
```

Let's take a look at the simulation of the above approach-:

```
set[]={3, 4, 5, 2}
sum=9
(x, y)= 'x' is the left number of elements,
'y' is the required sum

                (4, 9)
               {True}
             /        \
         (3, 6)        (3, 9)

        /    \         /   \
     (2, 2)  (2, 6)   (2, 5)  (2, 9)
     {True}
     /   \
  (1, -3) (1, 2)
{False}  {True}
```

```
          /     \
        (0, 0)   (0, 2)
        {True} {False}
```

C

```c
#include <stdio.h>

bool isSubsetSum(int set[], int n, int sum){
	if (sum == 0)
		return true;
	if (n == 0)
		return false;

	if (set[n - 1] > sum)
		return isSubsetSum(set, n - 1, sum);

	return isSubsetSum(set, n - 1, sum) || isSubsetSum(set, n - 1, sum
- set[n - 1]);
}

int main(){
	int set[] = { 3, 34, 4, 12, 5, 2 };
	int sum = 9;
	int n = sizeof(set) / sizeof(set[0]);
	if (isSubsetSum(set, n, sum) == true)
		printf("Found a subset with given sum");
	else
		printf("No subset with given sum");
	return 0;
}
```

C++

```cpp
// A recursive solution for subset sum problem
#include <iostream>
using namespace std;

bool isSubsetSum(int set[], int n, int sum){

	if (sum == 0)
		return true;
	if (n == 0)
		return false;

	if (set[n - 1] > sum)
		return isSubsetSum(set, n - 1, sum);

	return isSubsetSum(set, n - 1, sum) || isSubsetSum(set, n - 1, sum
- set[n - 1]);
}
```

```
int main(){
        int set[] = { 3, 34, 4, 12, 5, 2 };
        int sum = 9;
        int n = sizeof(set) / sizeof(set[0]);
        if (isSubsetSum(set, n, sum) == true)
                cout <<"Found a subset with given sum";
        else
                cout <<"No subset with given sum";
        return 0;
}
```

Subset Sum Problem (DP Solution)

DP Solution

So we will create a 2D array of size (arr.size() + 1) * (target + 1) of type **boolean**. The state DP[i][j] will be **true** if there exists a subset of elements from A[0….i] with **sum value = 'j'.** The approach for the problem is:

```
if (A[i-1] > j)
        DP[i][j] = DP[i-1][j]
else
        DP[i][j] = DP[i-1][j] OR DP[i-1][j-A[i-1]]
```

1. This means that if the current element has a value greater than the 'current sum value' we will copy the answer for previous cases
2. And if the current sum value is greater than the 'ith' element we will see if any of the previous states have already experienced the **sum='j' OR any previous states experienced a value 'j – A[i]'** which will solve our purpose.

The below simulation will clarify the above approach:

set[]={3, 4, 5, 2}
target = 6

	0	1	2	3	4	5	6
0	T	F	F	F	F	F	F
3	T	F	F	T	F	F	F
4	T	F	F	T	T	F	F
5	T	F	F	T	T	T	F

2 T F T T T T T

C

```c
#include <stdio.h>

bool isSubsetSum(int set[], int n, int sum){
	bool subset[n + 1][sum + 1];

	for (int i = 0; i <= n; i++)
		subset[i][0] = true;

	for (int i = 1; i <= sum; i++)
		subset[0][i] = false;

	for (int i = 1; i <= n; i++) {
		for (int j = 1; j <= sum; j++) {
			if (j < set[i - 1])
				subset[i][j] = subset[i - 1][j];
			if (j >= set[i - 1])
				subset[i][j] = subset[i - 1][j] || subset[i -
1][j - set[i - 1]];
		}
	}

	return subset[n][sum];
}

int main(){
	int set[] = { 3, 34, 4, 12, 5, 2 };
	int sum = 9;
	int n = sizeof(set) / sizeof(set[0]);
	if (isSubsetSum(set, n, sum) == true)
		printf("Found a subset with given sum");
	else
		printf("No subset with given sum");
	return 0;
}
```

C++

```cpp
#include <iostream>
using namespace std;

bool isSubsetSum(int set[], int n, int sum){
	bool subset[n + 1][sum + 1];

	for (int i = 0; i <= n; i++)
		subset[i][0] = true;

	for (int i = 1; i <= sum; i++)
```

```
                subset[0][i] = false;

        for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= sum; j++) {
                        if (j < set[i - 1])
                                subset[i][j] = subset[i - 1][j];
                        if (j >= set[i - 1])
                                subset[i][j] = subset[i - 1][j] || subset[i -
1][j - set[i - 1]];
                }
        }

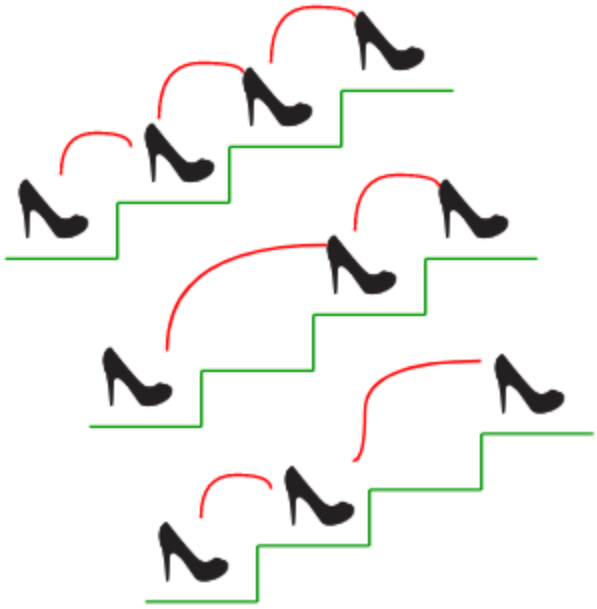
        return subset[n][sum];
}

int main(){
        int set[] = { 3, 34, 4, 12, 5, 2 };
        int sum = 9;
        int n = sizeof(set) / sizeof(set[0]);
        if (isSubsetSum(set, n, sum) == true)
                cout <<"Found a subset with given sum";
        else
                cout <<"No subset with given sum";
        return 0;
}
```

Count ways to reach nth stair

Count ways to reach the n'th stair

There are n stairs, a person standing at the bottom wants to reach the top. The person can climb either 1 stair or 2 stairs at a time. Count the number of ways, the person can reach the top.



Examples:

```
Input: n = 1
Output: 1
There is only one way to climb 1 stair

Input: n = 2
Output: 2
There are two ways: (1, 1) and (2)

Input: n = 4
Output: 5
(1, 1, 1, 1), (1, 1, 2), (2, 1, 1), (1, 2, 1), (2, 2)
```

Recursive Solution

The first method uses the technique of recursion to solve this problem.

Approach: We can easily find the recursive nature in the above problem. The person can reach **n**th stair from either **(n-1)**th stair or from **(n-2)**th stair. Hence, for each stair **n**, we try to find out the number of ways to reach **n-1**th stair and **n-2**th stair and add them to give the answer for the **n**th stair. Therefore the expression for such an approach comes out to be :

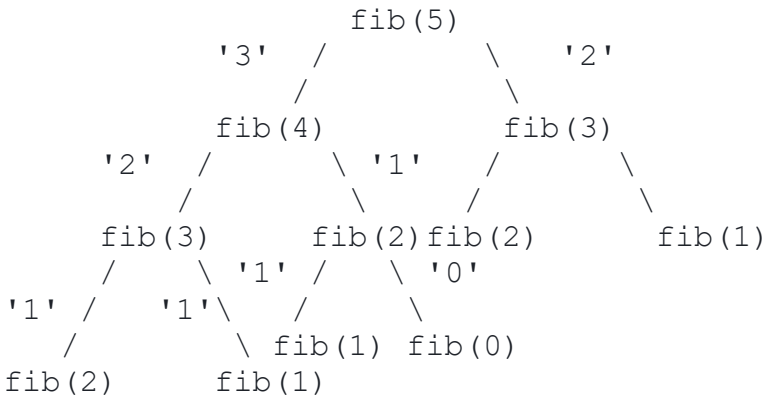
```
ways(n) = ways(n-1) + ways(n-2)
```

The above expression is actually the expression for Fibonacci numbers, but there is one thing to notice, the value of ways(n) is equal to fibonacci(n+1).

```
ways(1) = fib(2) = 1
ways(2) = fib(3) = 2
ways(3) = fib(4) = 3
```

For a better understanding, let's refer to the recursion tree below -:

Input: N = 4



So we can use the function for Fibonacci numbers to find the value of ways(n).

C

```c
#include <stdio.h>

int fib(int n){
	if (n <= 1)
		return n;
	return fib(n - 1) + fib(n - 2);
}

int countWays(int s){
	return fib(s + 1);
}

int main(){
	int s = 4;
	printf("Number of ways = %d", countWays(s));
	getchar();
	return 0;
}
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int fib(int n){
	if (n <= 1)
		return n;
	return fib(n - 1) + fib(n - 2);
}

int countWays(int s){
```

```
        return fib(s + 1);
}

int main(){
        int s = 4;

        cout << "Number of ways = " << countWays(s);

        return 0;
}
```

DP Solution

Approach: We create a table **res[]** in bottom-up manner using the following relation:

`res[i] = res[i] + res[i-j] for every (i-j) >= 0`

such that the **i**th index of the array will contain the number of ways required to reach the **i**th step considering all the possibilities of climbing (i.e. from 1 to **i**).

C

```
#include <stdio.h>

int countWaysUtil(int n, int m){
    int res[n];
    res[0] = 1;
    res[1] = 1;
    for (int i = 2; i < n; i++) {
        res[i] = 0;
        for (int j = 1; j <= m && j <= i; j++)
            res[i] += res[i - j];
    }
    return res[n - 1];
}

int countWays(int s, int m){
    return countWaysUtil(s + 1, m);
}

int main(){
    int s = 4, m = 2;
    printf("Number of ways = %d", countWays(s, m));
    return 0;
}
```

C++

```
#include <iostream>
using namespace std;

int countWaysUtil(int n, int m){
```

```
    int res[n];
    res[0] = 1;
    res[1] = 1;

    for(int i = 2; i < n; i++){
       res[i] = 0;

       for(int j = 1; j <= m && j <= i; j++)
          res[i] += res[i - j];
    }
    return res[n - 1];
}

int countWays(int s, int m){
    return countWaysUtil(s + 1, m);
}

int main(){
    int s = 4, m = 2;

    cout << "Number of ways = "
         << countWays(s, m);

    return 0;
}
```

Minimum Jumps to reach end

Minimum number of jumps to reach the end

Given an array **arr[]** where each element represents the max number of steps that can be made forward from that index. The task is to find the minimum number of jumps to reach the end of the array starting from index **0**. If the end isn't reachable, return **-1**.

Examples:

Input: *arr[] = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9}*
Output: *3 (1-> 3 -> 9 -> 9)*
Explanation: *Jump from 1st element to 2nd element as there is only 1 step.*
Now there are three options 5, 8 or 9. I
f 8 or 9 is chosen then the end node 9 can be reached. So 3 jumps are made.

Input: *arr[] = {1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1}*
Output: *10*
Explanation: *In every step a jump is needed so the count of jumps is 10.*

Recursive Approach

Start from the first element and recursively call for all the elements reachable from the first element. The minimum number of jumps to reach end from first can be calculated using the minimum value from the recursive calls.

minJumps(start, end) = Min (minJumps(k, end)) *for all k reachable from start.*
Follow the steps mentioned below to implement the idea:

- Create a recursive function.
- In each recursive call get all the reachable nodes from that index.
    - For each of the index call the recursive function.
    - Find the minimum number of jumps to reach the end from current index.
- Return the minimum number of jumps from the recursive call.

C

```c
#include <limits.h>
#include <stdio.h>

int minJumps(int arr[], int l, int h){
    if (h == l)
        return 0;

    if (arr[l] == 0)
        return INT_MAX;

    int min = INT_MAX;
    for (int i = l + 1; i <= h && i <= l + arr[l]; i++) {
        int jumps = minJumps(arr, i, h);
        if (jumps != INT_MAX && jumps + 1 < min)
            min = jumps + 1;
    }

    return min;
}

int main(){
    int arr[] = { 1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9 };
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Minimum number of jumps to reach end is %d ",
           minJumps(arr, 0, n - 1));
    return 0;
}
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

int minJumps(int arr[], int n){
    if (n == 1)
        return 0;

    int res = INT_MAX;
    for (int i = n - 2; i >= 0; i--) {
        if (i + arr[i] >= n - 1) {
            int sub_res = minJumps(arr, i + 1);
            if (sub_res != INT_MAX)
                res = min(res, sub_res + 1);
        }
    }

    return res;
}

int main(){
    int arr[] = { 1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9 };
    int n = sizeof(arr) / sizeof(arr[0]);
    cout << "Minimum number of jumps to";
    cout << " reach the end is " << minJumps(arr, n);
    return 0;
}
```

DP Solution

It can be observed that there will be overlapping subproblems.

For example in array, arr[] = {1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9} minJumps(3, 9) will be called two times as arr[3] is reachable from arr[1] and arr[2]. So this problem has both properties (optimal substructure and overlapping subproblems) of Dynamic Programming.

Follow the below steps to implement the idea:

- Create jumps[] array from **left to right** such that jumps[i] indicate the minimum number of jumps needed to reach arr[i] from arr[0].
- To fill the jumps array run a nested loop inner loop counter is **j** and the outer loop count is **i**.
    - Outer loop from **1 to n-1** and inner loop from **0 to i**.
    - If i is less than j + arr[j] then set jumps[i] to minimum of jumps[i] and jumps[j] + 1. initially set jump[i] to INT MAX
- Return jumps[n-1].

C

```c
#include <limits.h>
#include <stdio.h>

int min(int x, int y) { return (x < y) ? x : y; }
```

```
int minJumps(int arr[], int n){
	int jumps[n];
	int i, j;

	if (n == 0 || arr[0] == 0)
		return INT_MAX;

	jumps[0] = 0;
	for (i = 1; i < n; i++) {
		jumps[i] = INT_MAX;
		for (j = 0; j < i; j++) {
			if (i <= j + arr[j] && jumps[j] != INT_MAX) {
				jumps[i] = min(jumps[i], jumps[j] + 1);
				break;
			}
		}
	}
	return jumps[n - 1];
}

int main(){
	int arr[] = { 1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9 };
	int size = sizeof(arr) / sizeof(int);
	printf("Minimum number of jumps to reach end is %d ",
		minJumps(arr, size));
	return 0;
}
```

C++

```
// C++ program for Minimum number
// of jumps to reach end
#include <bits/stdc++.h>
using namespace std;

int min(int x, int y) { return (x < y) ? x : y; }

int minJumps(int arr[], int n){
	int* jumps = new int[n];
	int i, j;

	if (n == 0 || arr[0] == 0)
		return INT_MAX;

	jumps[0] = 0;

	for (i = 1; i < n; i++) {
		jumps[i] = INT_MAX;
		for (j = 0; j < i; j++) {
```

```
                        if (i <= j + arr[j] && jumps[j] != INT_MAX) {
                                jumps[i] = min(jumps[i], jumps[j] + 1);
                                break;
                        }
                }
        }
        return jumps[n - 1];
}

int main(){
        int arr[] = { 1, 3, 5, 8, 9, 2, 6, 7, 6, 8, 9 };
        int size = sizeof(arr) / sizeof(int);
        cout << "Minimum number of jumps to reach end is "
                << minJumps(arr, size);
        return 0;
}
```

House Thief

House Thief

There are **N** houses built in a line, each of which contains some value in it. A thief is going to steal the maximum value of these houses, but he can't steal in two adjacent houses because the owner of the stolen houses will tell his two neighbors left and right sides. The task is to find what is the maximum stolen value.

Examples:

Input: *hval[] = {6, 7, 1, 3, 8, 2, 4}*
Output: *19*
Explanation: *The thief will steal 6, 1, 8 and 4 from the house.*

Input: *hval[] = {5, 3, 4, 11, 2}*
Output: *16*
Explanation: *Thief will steal 5 and 11*

Recursive Solution

Given an array, the solution is to find the maximum sum subsequence where no two selected elements are adjacent. So the approach to the problem is a recursive solution.

So there are two cases:

1. If an element is selected then the next element cannot be selected.

2. if an element is not selected then the next element can be selected.

Use recursion to find the result for both cases.

C

```c
#include <stdio.h>

int max(int num1, int num2){
	return (num1 > num2) ? num1 : num2;
}

int maxLoot(int* hval, int n){
	if (n < 0)
		return 0;

	if (n == 0)
		return hval[0];
	int pick = hval[n] + maxLoot(hval, n - 2);
	int notPick = maxLoot(hval, n - 1);

	return max(pick, notPick);
}

int main(){
	int hval[] = { 6, 7, 1, 3, 8, 2, 4 };
	int n = sizeof(hval) / sizeof(hval[0]);
	printf("Maximum loot possible : %d ",
		maxLoot(hval, n - 1));
	return 0;
}
```

C++

```cpp
#include <iostream>
using namespace std;

int maxLoot(int* hval, int n){
	if (n < 0) {
		return 0;
	}

	if (n == 0) {
		return hval[0];
	}
	int pick = hval[n] + maxLoot(hval, n - 2);
	int notPick = maxLoot(hval, n - 1);
	return max(pick, notPick);
}

int main(){
```

```
        int hval[] = { 6, 7, 1, 3, 8, 2, 4 };
        int n = sizeof(hval) / sizeof(hval[0]);
        cout << "Maximum loot possible : "
                << maxLoot(hval, n - 1);
        return 0;
}
```

DP Solution

The sub-problems can be stored thus reducing the complexity and converting the recursive solution to a Dynamic programming problem.

Follow the below steps to Implement the idea:

- Create a **DP** vector of size n+1 and value **-1** and variables **pick** and **not pick**.
- Create a recursive function
    - If n < 0 possible stolen value is 0.
    - If n = 0 possible stolen value is hval[0].
    - If DP[n] != -1 possible stolen value is DP[n].
    - Set pick as pick = hval[n] + maxLoot(hval, n-2, DP)
    - Set not pick as notPick = maxLoot(hval, n-1, DP).
    - Set Dp[n] = max(pick, notPick) and return DP[n].

C++

```
#include <bits/stdc++.h>
using namespace std;

int maxLoot(int *hval, int n, vector<int> &dp){
        if(n < 0){
                return 0 ;
        }

        if(n == 0){
                return hval[0] ;
        }

        if(dp[n] != -1 ){
                return dp[n] ;
        }

        int pick = hval[n] + maxLoot(hval, n-2, dp) ;
        int notPick = maxLoot(hval, n-1, dp) ;

        return dp[n] = max(pick, notPick) ;
}

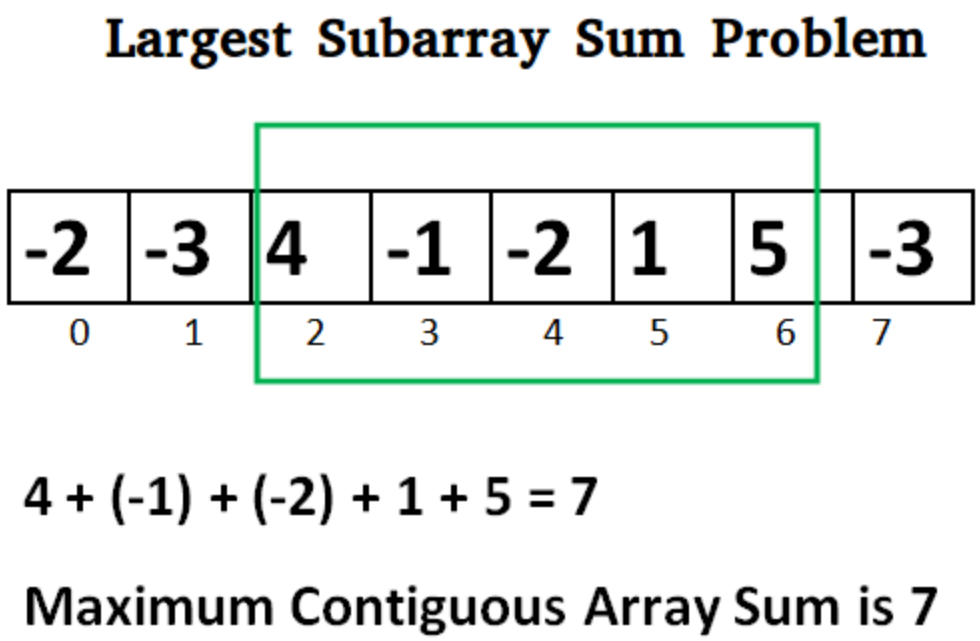
int main(){
        int hval[] = {6, 7, 1, 3, 8, 2, 4};
        int n = sizeof(hval)/sizeof(hval[0]);
```

```
        vector<int> dp(n+1, -1) ;
        cout << "Maximum loot possible : "
               << maxLoot(hval, n-1, dp);
        return 0;
}
```

Kadane's Algorithm

Kadane's Algorithm

Given an array **arr[]** of size **N**. The task is to find the sum of the contiguous subarray within a **arr[]** with the largest sum.



The idea of **Kadane's algorithm** is to maintain a variable **max_ending_here** that stores the maximum sum contiguous subarray ending at current index and a variable **max_so_far** stores the maximum sum of contiguous subarray found so far, Everytime there is a positive-sum value in **max_ending_here** compare it with **max_so_far** and update **max_so_far** if it is greater than **max_so_far**.

So the main **Intuition** behind **Kadane's algorithm** is,

– the subarray with negative sum is discarded (*by assigning max_ending_here = 0 in code*).

– we carry subarray till it gives positive sum.

Illustration:

Lets take the example: {-2, -3, 4, -1, -2, 1, 5, -3}
max_so_far = INT_MIN
max_ending_here = 0

for i=0, a[0] = -2
max_ending_here = max_ending_here + (-2)
Set max_ending_here = 0 because max_ending_here < 0
and set max_so_far = -2

for i=1, a[1] = -3
max_ending_here = max_ending_here + (-3)
Since max_ending_here = -3 and max_so_far = -2, max_so_far will remain -2
Set max_ending_here = 0 because max_ending_here < 0

for i=2, a[2] = 4
max_ending_here = max_ending_here + (4)
max_ending_here = 4
max_so_far is updated to 4 because max_ending_here greater
than max_so_far which was -2 till now

for i=3, a[3] = -1
max_ending_here = max_ending_here + (-1)
max_ending_here = 3

for i=4, a[4] = -2
max_ending_here = max_ending_here + (-2)
max_ending_here = 1

for i=5, a[5] = 1
max_ending_here = max_ending_here + (1)
max_ending_here = 2

for i=6, a[6] = 5
max_ending_here = max_ending_here + (5)
max_ending_here = 7

max_so_far is updated to 7 because max_ending_here is greater than max_so_far

for i=7, a[7] = -3
max_ending_here = max_ending_here + (-3)
max_ending_here = 4

Follow the below steps to Implement the idea:

- Initialize the variables **max_so_far** = INT_MIN and **max_ending_here** = 0
- Run a for loop from **0** to **N-1** and for each index **i**:
  - Add the arr[i] to max_ending_here.
  - If max_so_far is less than max_ending_here then update max_so_far to max_ending_here**.**
  - If max_ending_here < 0 then update max_ending_here = 0
- Return max_so_far

C

```c
#include <stdio.h>
#include <limits.h>

int maxSubArraySum(int a[], int size) {
    int max_so_far = INT_MIN, max_ending_here = 0;

    for (int i = 0; i < size; i++) {
        max_ending_here = max_ending_here + a[i];
        if (max_so_far < max_ending_here)
            max_so_far = max_ending_here;

        if (max_ending_here < 0)
            max_ending_here = 0;
    }

    return max_so_far;
}

int main() {
    int a[] = {-2, -3, 4, -1, -2, 1, 5, -3};
    int n = sizeof(a) / sizeof(a[0]);

    int max_sum = maxSubArraySum(a, n);
    printf("Maximum contiguous sum is %d", max_sum);
    return 0;
}
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
```

```
int maxSubArraySum(int a[], int size){
	int max_so_far = INT_MIN, max_ending_here = 0;

	for (int i = 0; i < size; i++) {
		max_ending_here = max_ending_here + a[i];
		if (max_so_far < max_ending_here)
			max_so_far = max_ending_here;

		if (max_ending_here < 0)
			max_ending_here = 0;
	}
	return max_so_far;
}

int main(){
	int a[] = { -2, -3, 4, -1, -2, 1, 5, -3 };
	int n = sizeof(a) / sizeof(a[0]);
	int max_sum = maxSubArraySum(a, n);
	cout << "Maximum contiguous sum is " << max_sum;
	return 0;
}
```

Matrix Chain Multiplication

Matrix Chain Multiplication

Given the dimension of a sequence of matrices in an array **arr[]**, where the dimension of the **i**th matrix is **(arr[i-1] * arr[i])**, the task is to find the most efficient way to multiply these matrices together such that the total number of element multiplications is minimum.

Examples:

Input: *arr[] = {40, 20, 30, 10, 30}*
Output: *26000*
Explanation:*There are 4 matrices of dimensions 40×20, 20×30, 30×10, 10×30.*
Let the input 4 matrices be A, B, C and D.
The minimum number of multiplications are obtained by
putting parenthesis in following way (A(BC))D.
*The minimum is 20*30*10 + 40*20*10 + 40*10*30*

Input: *arr[] = {1, 2, 3, 4, 3}*
Output: *30*

Explanation: *There are 4 matrices of dimensions 1×2, 2×3, 3×4, 4×3.*
Let the input 4 matrices be A, B, C and D.
The minimum number of multiplications are obtained by
putting parenthesis in following way ((AB)C)D.
*The minimum number is 1*2*3 + 1*3*4 + 1*4*3 = 30*

Input: *arr[] = {10, 20, 30}*
Output: *6000*
Explanation: *There are only two matrices of dimensions 10×20 and 20×30.*
*So there is only one way to multiply the matrices, cost of which is 10*20*30*

Recursive Solution

We can solve the problem using recursion based on the following facts and observations:

*Two matrices of size **m*n** and **n*p** when multiplied, they generate a matrix of size **m*p** and the number of multiplications performed are **m*n*p**.*

*Now, for a given chain of **N** matrices, the first partition can be done in **N-1** ways. For example, sequence of matrices A, B, C and D can be grouped as (A)(BCD), (AB)(CD) or (ABC)(D) in these 3 ways.*

*So a range **[i, j]** can be broken into two groups like **{[i, i+1], [i+1, j]}, {[i, i+2], [i+2, j]}, . . . , {[i, j-1], [j-1, j]}**.*

- *Each of the groups can be further partitioned into smaller groups and we can find the total required multiplications by solving for each of the groups.*
- *The minimum number of multiplications among all the first partitions is the required answer.*

Follow the steps mentioned below to implement the approach:

- Create a recursive function that takes **i** and **j** as parameters that determines the range of a group.
  - Iterate from **k = i to j** to partition the given range into two groups.
  - Call the recursive function for these groups.
  - Return the minimum value among all the partitions as the required minimum number of multiplications to multiply all the matrices of this group.
- The minimum value returned for the range **0** to **N-1** is the required answer.

C

```c
#include <limits.h>
#include <stdio.h>

int MatrixChainOrder(int p[], int i, int j){
        if (i == j)
```

```
                return 0;
        int k;
        int min = INT_MAX;
        int count;

        for (k = i; k < j; k++){
                count = MatrixChainOrder(p, i, k)
                                + MatrixChainOrder(p, k + 1, j)
                                + p[i - 1] * p[k] * p[j];

                if (count < min)
                        min = count;
        }

        return min;
}

int main(){
        int arr[] = { 1, 2, 3, 4, 3 };
        int N = sizeof(arr) / sizeof(arr[0]);

        printf("Minimum number of multiplications is %d ",
                MatrixChainOrder(arr, 1, N - 1));
        getchar();
        return 0;
}
```

C++

```
#include <bits/stdc++.h>
using namespace std;

int MatrixChainOrder(int p[], int i, int j){
        if (i == j)
                return 0;
        int k;
        int mini = INT_MAX;
        int count;

        for (k = i; k < j; k++){
                count = MatrixChainOrder(p, i, k)
                                + MatrixChainOrder(p, k + 1, j)
                                + p[i - 1] * p[k] * p[j];

                mini = min(count, mini);
        }

        return mini;
}
```

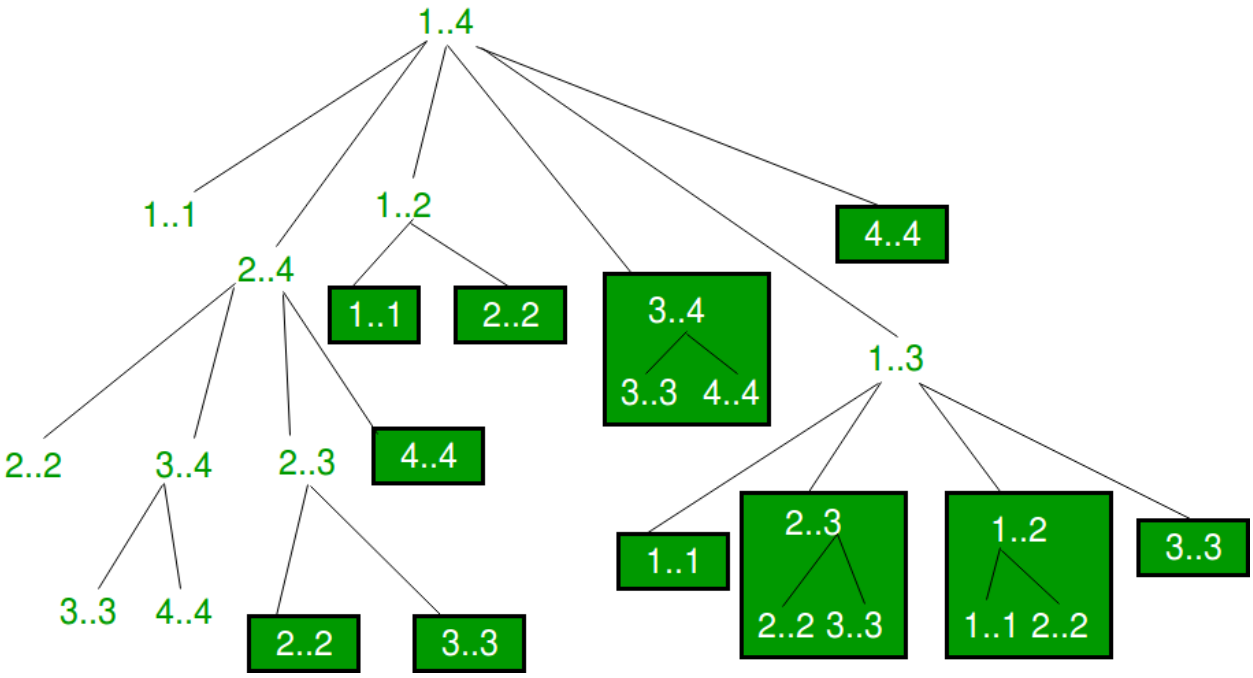
```
int main(){
        int arr[] = { 1, 2, 3, 4, 3 };
        int N = sizeof(arr) / sizeof(arr[0]);

        // Function call
        cout << "Minimum number of multiplications is "
                << MatrixChainOrder(arr, 1, N - 1);
        return 0;
}
```

Matrix Chain Multiplication - DP

DP Solution (Memoization)

Below is the recursion tree for the 2nd example of the above recursive approach:



If observed carefully you can find the following two properties:

1) Optimal Substructure: *In the above case, we are breaking the bigger groups into smaller subgroups and solving them to finally find the minimum number of multiplications. Therefore, it can be said that the problem has optimal substructure property.*

2) Overlapping Subproblems: *We can see in the recursion tree that the same subproblems are called again and again and this problem has the Overlapping Subproblems property.*

So Matrix Chain Multiplication problem has both properties of a dynamic programming problem. So recomputations of same subproblems can be avoided by constructing a temporary array ***dp[][]*** *in a bottom up manner.*
Follow the below steps to solve the problem:

- Build a matrix **dp[][]** of size **N*N** for memoization purposes.
- Use the same recursive call as done in the above approach:
  - When we find a range **(i, j)** for which the value is already calculated, return the minimum value for that range (i.e., **dp[i][j]**).
  - Otherwise, perform the recursive calls as mentioned earlier.
- The value stored at **dp[0][N-1]** is the required answer.

C++

```
#include <bits/stdc++.h>
using namespace std;
int dp[100][100];

int matrixChainMemoised(int* p, int i, int j){
    if (i == j){
        return 0;
    }
    if (dp[i][j] != -1){
        return dp[i][j];
    }
    dp[i][j] = INT_MAX;
    for (int k = i; k < j; k++){
        dp[i][j] = min(
            dp[i][j], matrixChainMemoised(p, i, k)
                    + matrixChainMemoised(p, k + 1, j)
                      + p[i - 1] * p[k] * p[j]);
    }
    return dp[i][j];
}
int MatrixChainOrder(int* p, int n){
    int i = 1, j = n - 1;
    return matrixChainMemoised(p, i, j);
}

int main(){
    int arr[] = { 1, 2, 3, 4 };
    int n = sizeof(arr) / sizeof(arr[0]);
    memset(dp, -1, sizeof dp);
```

```
    cout << "Minimum number of multiplications is "
         << MatrixChainOrder(arr, n);
}
```

DP Solution (Iterative)

in iterative approach, we initially need to find the number of multiplications required to multiply two adjacent matrices. We can use these values to find the minimum multiplication required for matrices in a range of length 3 and further use those values for ranges with higher lengths.

Build on the answer in this manner till the range becomes [0, N-1].

Follow the steps mentioned below to implement the idea:

- Iterate from **l = 2 to N-1** which denotes the length of the range:
  - Iterate from **i = 0 to N-1**:
    - Find the right end of the range (**j**) having **l** matrices.
    - Iterate from **k = i+1 to j** which denotes the point of partition.
      - Multiply the matrices in range (**i, k**) and (**k, j**).
      - This will create two matrices with dimensions **arr[i-1]*arr[k]** and **arr[k]*arr[j]**.
      - The number of multiplications to be performed to multiply these two matrices (say **X**) are **arr[i-1]*arr[k]*arr[j]**.
      - The total number of multiplications is **dp[i][k]+ dp[k+1][j] + X**.
- The value stored at **dp[1][N-1]** is the required answer.

C

```
#include <limits.h>
#include <stdio.h>
int MatrixChainOrder(int p[], int n){

        int m[n][n];
        int i, j, k, L, q;

        for (i = 1; i < n; i++)
                m[i][i] = 0;

        for (L = 2; L < n; L++) {
                for (i = 1; i < n - L + 1; i++){
                        j = i + L - 1;
                        m[i][j] = INT_MAX;
                        for (k = i; k <= j - 1; k++){
                                q = m[i][k] + m[k + 1][j]
                                        + p[i - 1] * p[k] * p[j];
                                if (q < m[i][j])
                                        m[i][j] = q;
                        }
                }
        }
```

```
        return m[1][n - 1];
}

int main(){
        int arr[] = { 1, 2, 3, 4 };
        int size = sizeof(arr) / sizeof(arr[0]);

        printf("Minimum number of multiplications is %d ",
                MatrixChainOrder(arr, size));

        getchar();
        return 0;
}
```

C++

```
#include <bits/stdc++.h>
using namespace std;
int MatrixChainOrder(int p[], int n){

        int m[n][n];

        int i, j, k, L, q;
        for (i = 1; i < n; i++)
                m[i][i] = 0;

        for (L = 2; L < n; L++){
                for (i = 1; i < n - L + 1; i++){
                        j = i + L - 1;
                        m[i][j] = INT_MAX;
                        for (k = i; k <= j - 1; k++){
                                q = m[i][k] + m[k + 1][j]
                                        + p[i - 1] * p[k] * p[j];
                                if (q < m[i][j])
                                        m[i][j] = q;
                        }
                }
        }

        return m[1][n - 1];
}

int main(){
        int arr[] = { 1, 2, 3, 4 };
        int size = sizeof(arr) / sizeof(arr[0]);

        cout << "Minimum number of multiplications is "
                << MatrixChainOrder(arr, size);
```

```
        getchar();
        return 0;
}
```

Allocate Minimum Number of Pages

Allocate a Minimum Number of Pages

Given a number of pages in **N** different books and **M** students. The books are arranged in ascending order of the number of pages. Every student is assigned to read some consecutive books. The task is to assign books in such a way that the maximum number of pages assigned to a student is minimum.

Example :

Input : *pages[] = {12, 34, 67, 90} , m = 2*
Output : *113*
Explanation: *There are 2 number of students. Books can be distributed in following fashion :*
1) [12] and [34, 67, 90]
Max number of pages is allocated to student '2' with 34 + 67 + 90 = 191 pages
2) [12, 34] and [67, 90] Max number of pages is allocated to student '2' with 67 + 90 = 157 pages
3) [12, 34, 67] and [90] Max number of pages is allocated to student '1' with 12 + 34 + 67 = 113 pages

Of the 3 cases, Option 3 has the minimum pages = 113.
Approach: A Binary Search method for solving the book allocation problem:

Case 1: When no valid answer exists.

If the number of students is greater than the number of books ***(i.e, M > N)****, In this case at least 1 student will be left to which no book has been assigned.*
Case 2: When a valid answer exists.

The maximum possible answer could be when there is only one student. So, all the book will be assigned to him and the result would be the sum of pages of all the books.

The minimum possible answer could be when number of student is equal to the number of book (i.e, M == N) , In this case all the students will get at most one book. So, the result would be the maximum number of pages among them ***(i.e, maximum(pages[])).***

Hence, we can apply binary search in this given range and each time we can consider the mid value as the maximum limit of pages one can get. And check for the limit if answer is valid then update the limit accordingly.

Below is the implementation of the above idea:

- Initialise the **start** to **maximum(pages[])** and **end =** sum of **pages[]**,
- Do while start <= end
    - Calculate the mid and check if **mid** number of pages can assign any student by satisfying the given condition such that all students will get at least one book. Follow the steps to check for validity.
        - Initialise the **studentsRequired = 1** and **curr_sum = 0** for sum of consecutive pages of book
        - Iterate over all books or say pages[]
            - Add the pages to curr_sum and check **curr_sum > curr_min** then increment the count of **studentRequired** by 1.
            - Check if the studentRequired > M, return false.
        - Return true.
    - If mid is valid then, update the **result** and move the **end = mid – 1**
    - Otherwise, move the **start = mid + 1**
- Finally, return the **result**.

C

```c
#include <limits.h>
#include <stdbool.h>
#include <stdio.h>

bool isPossible(int arr[], int n, int m, int curr_min){
        int studentsRequired = 1;
        int curr_sum = 0;

        for (int i = 0; i < n; i++) {
                if (arr[i] > curr_min)
                        return false;

                if (curr_sum + arr[i] > curr_min) {
                        studentsRequired++;

                        curr_sum = arr[i];
                        if (studentsRequired > m)
                                return false;
                }
```

```
                else
                        curr_sum += arr[i];
        }
        return true;
}

int findPages(int arr[], int n, int m){
        long long sum = 0;

        if (n < m)
                return -1;
        int mx = arr[0] ;

        for (int i = 0; i < n; i++){
        sum += arr[i];
        mx = (arr[i] > mx ? arr[i] : mx) ;
        }
        int start = mx, end = sum;
        int result = INT_MAX;

        while (start <= end) {
                int mid = (start + end) / 2;
                if (isPossible(arr, n, m, mid)) {
                        result = mid;
                        end = mid - 1;
                }

                else
                        start = mid + 1;
        }

        return result;
}

int main(){
        int arr[] = { 12, 34, 67, 90 };
        int n = sizeof arr / sizeof arr[0];
        int m = 2;

        printf("Minimum number of pages = %d\n",
                findPages(arr, n, m));
        return 0;
}
```

Output:

```
Minimum number of pages = 113
```

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

bool isPossible(int arr[], int n, int m, int curr_min){
	int studentsRequired = 1;
	int curr_sum = 0;
	for (int i = 0; i < n; i++) {
		if (arr[i] > curr_min)
			return false;

		if (curr_sum + arr[i] > curr_min) {
			studentsRequired++;
			curr_sum = arr[i];

			if (studentsRequired > m)
				return false;
		}
		else
			curr_sum += arr[i];
	}
	return true;
}
int findPages(int arr[], int n, int m){
	long long sum = 0;

	if (n < m)
		return -1;
	int mx = INT_MIN;

	for (int i = 0; i < n; i++) {
		sum += arr[i];
		mx = max(mx, arr[i]);
	}
	int start = mx, end = sum;
	int result = INT_MAX;

	while (start <= end) {
		int mid = (start + end) / 2;
		if (isPossible(arr, n, m, mid)) {
			result = mid;
			end = mid - 1;
		}

		else
			start = mid + 1;
	}
```

```
        return result;
}

int main(){
        int arr[] = { 12, 34, 67, 90 };
        int n = sizeof arr / sizeof arr[0];
        int m = 2;

        cout << "Minimum number of pages = "
                << findPages(arr, n, m) << endl;
        return 0;
}
```

Output:

```
Minimum number of pages = 113
```

2D – DP

Count Ways in a Grid

Count Ways in a Grid

The problem is to count all the possible paths from the top left to the bottom right of a M X N matrix with the constraints that from each cell you can either move only to the right or down.

Examples:

Input: *M = 2, N = 2*
Output: *2*
Explanation: *There are two paths*
(0, 0) -> (0, 1) -> (1, 1)
(0, 0) -> (1, 0) -> (1, 1)

Input: *M = 2, N = 3*
Output: *3*
Explanation: *There are three paths*
(0, 0) -> (0, 1) -> (0, 2) -> (1, 2)

(0, 0) -> (0, 1) -> (1, 1) -> (1, 2)
(0, 0) -> (1, 0) -> (1, 1) -> (1, 2)

Recursion Solution

To solve the problem follow the below idea:

We can recursively move to right and down from the start until we reach the destination and then add up all valid paths to get the answer.

Follow the below steps to solve the problem:

- Create a recursive function with parameters as row and column index
- Call this recursive function for N-1 and M-1
- In the recursive function
  - If N == 1 or M == 1 then return 1
  - else call the recursive function with (N-1, M) and (N, M-1) and return the sum of this
- Print the answer

C++

```
#include <bits/stdc++.h>
using namespace std;

int numberOfPaths(int m, int n){
        if (m == 1 || n == 1)
                return 1;
        return numberOfPaths(m - 1, n) + numberOfPaths(m, n - 1);
}

int main(){
        cout << numberOfPaths(3, 3);
        return 0;
}
```

DP Approach

To solve the problem follow the below idea:

So this problem has both properties (see this and this) of a dynamic programming problem. Like other typical Dynamic Programming(DP) problems, recomputations of the same subproblems can be avoided by constructing a temporary array count[][] in a bottom-up manner using the above recursive formula

Follow the below steps to solve the problem:

- Declare a 2-D array count of size M * N
- Set value of count[i][0] equal to 1 for 0 <= i < M as the answer of subproblem with a single column is equal to 1
- Set value of count[0][j] equal to 1 for 0 <= j < N as the answer of subproblem with a single row is equal to 1

- Create a nested for loop for 0 <= i < M and 0 <= j < N and assign count[i][j] equal to count[i-1][j] + count[i][j-1]
- Print value of count[M-1][N-1]

C++

```
#include <bits/stdc++.h>
using namespace std;

int numberOfPaths(int m, int n){
        int count[m][n];

        for (int i = 0; i < m; i++)
                count[i][0] = 1;

        for (int j = 0; j < n; j++)
                count[0][j] = 1;

        for (int i = 1; i < m; i++) {
                for (int j = 1; j < n; j++)
                        count[i][j]
                                = count[i - 1][j] + count[i][j - 1];
        }
        return count[m - 1][n - 1];
}

int main(){
        cout << numberOfPaths(3, 3);
        return 0;
}
```

0 - 1 Knapsack Problem

0 - 1 Knapsack Problem

We are given N items where each item has some weight and profit associated with it. We are also given a bag with capacity W, [i.e., the bag can hold at most **W** weight in it]. The target is to put the items into the bag such that the sum of profits associated with them is the maximum possible.

Note: The constraint here is we can either put an item completely into the bag or cannot put it at all [It is not possible to put a part of an item into the bag].

Examples:
Input: *N = 3, W = 4, profit[] = {1, 2, 3}, weight[] = {4, 5, 1}*
Output: *3*
Explanation: *There are two items which have weight less than or equal to 4. If we select the item with weight 4, the possible profit is 1. And if we select the item with weight 1, the possible profit is 3. So the maximum possible profit is 3. Note that we cannot put both the items with weight 4 and 1 together as the capacity of the bag is 4.*

Input: *N = 3, W = 3, profit[] = {1, 2, 3}, weight[] = {4, 5, 6}*
Output: *0*

Recursive Solution

To solve the problem follow the below idea:

A simple solution is to consider all subsets of items and calculate the total weight and profit of all subsets. Consider the only subsets whose total weight is smaller than W. From all such subsets, pick the subset with maximum profit.

Optimal Substructure: *To consider all subsets of items, there can be two cases for every item.*

- ***Case 1:*** *The item is included in the optimal subset.*
- ***Case 2:*** *The item is not included in the optimal set.*

Follow the below steps to solve the problem:

The maximum value obtained from 'N' items is the max of the following two values.

- Maximum value obtained by N-1 items and W weight (excluding n^{th} item)
- Value of nth item plus maximum value obtained by N-1 items and (W – weight of the Nth item) [including N^{th} item].
- If the weight of the 'N^{th}' item is greater than 'W', then the Nth item cannot be included and **Case 1** is the only possibility.

C++

```
#include <bits/stdc++.h>
using namespace std;

int max(int a, int b) { return (a > b) ? a : b; }

int knapSack(int W, int wt[], int val[], int n){

        if (n == 0 || W == 0)
                return 0;

        if (wt[n - 1] > W)
```

```
                return knapSack(W, wt, val, n - 1);
        else
                return max(
                        val[n - 1]
                                + knapSack(W - wt[n - 1], wt, val, n - 1),
                        knapSack(W, wt, val, n - 1));
}

int main(){
        int profit[] = { 60, 100, 120 };
        int weight[] = { 10, 20, 30 };
        int W = 50;
        int n = sizeof(profit) / sizeof(profit[0]);
        cout << knapSack(W, weight, profit, n);
        return 0;
}
```

DP Solution

To solve the problem follow the below idea:

Since subproblems are evaluated again, this problem has Overlapping Sub-problems property. So the 0/1 Knapsack problem has both properties (see this and this) of a dynamic programming problem. Like other typical Dynamic Programming(DP) problems, re-computation of the same subproblems can be avoided by constructing a temporary array K[][] in a bottom-up manner.

Follow the below steps to solve the problem:

- Consider the same cases as mentioned in the recursive approach.
- In a DP[][] table let's consider all the possible weights from '1' to 'W' as the columns and the element that can be kept as rows.
- The state DP[i][j] will denote the maximum value of 'j-weight' considering all values from '1 to i^{th}'. So if we consider 'w_i' (weight in 'i^{th}' row) we can fill it in all columns which have 'weight values > w_i'. Now two possibilities can take place:
  - Fill 'w_i' in the given column.
  - Do not fill 'w_i' in the given column.
- Now we have to take a maximum of these two possibilities,
  - Formally if we do not fill the 'i^{th}' weight in the 'j^{th}' column then the DP[i][j] state will be the same as DP[i-1][j]
  - But if we fill the weight, DP[i][j] will be equal to the value of ('w_i'+ value of the column weighing 'j-wi') in the previous row.
- So we take the maximum of these two possibilities to fill the current state.

Illustration:

Below is the illustration of the above approach:

Let, ***weight[] = {1, 2, 3}, profit[] = {10, 15, 40}, Capacity = 6***

- *If no element is filled, then the possible profit is 0.*

weight → / ***item ↓***	***0***	***1***	***2***	***3***	***4***	***5***	***6***
0	*0*	*0*	*0*	*0*	*0*	*0*	*0*
1							
2							
3							

- ***For filling the first item in the bag:*** *If we follow the above mentioned procedure, the table will look like the following.*

weight → / ***item ↓***	***0***	***1***	***2***	***3***	***4***	***5***	***6***
0	*0*	*0*	*0*	*0*	*0*	*0*	*0*
1	*0*	*10*	*10*	*10*	*10*	*10*	*10*
2							

weight→ / item↓	0	1	2	3	4	5	6
3							

- ***For filling the second item:***
  When j = 2, then maximum possible profit is max (10, DP[1][2-2] + 15) = max(10, 15) = 15.
  When j = 3, then maximum possible profit is max(2 not put, 2 is put into bag) = max(DP[1][3], 15+DP[1][3-2]) = max(10, 25) = 25.

weight→ / item↓	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	15	25	25	25	25
3							

- ***For filling the third item:***
  When j = 3, the maximum possible profit is max(DP[2][3], 40+DP[2][3-3]) = max(25, 40) = 40.
  When j = 4, the maximum possible profit is max(DP[2][4], 40+DP[2][4-3]) = max(25, 50) = 50.
  When j = 5, the maximum possible profit is max(DP[2][5], 40+DP[2][5-3]) = max(25, 55) = 55.
  When j = 6, the maximum possible profit is max(DP[2][6], 40+DP[2][6-3]) = max(25, 65) = 65.

weight→ / item↓	0	1	2	3	4	5	6
0	0	0	0	0	0	0	0
1	0	10	10	10	10	10	10
2	0	10	15	25	25	25	25
3	0	10	15	40	50	55	65

C++

```
#include <bits/stdc++.h>
using namespace std;

int max(int a, int b) { return (a > b) ? a : b; }

int knapSack(int W, int wt[], int val[], int n){
        int i, w;
        vector<vector<int> > K(n + 1, vector<int>(W + 1));

        for (i = 0; i <= n; i++) {
                for (w = 0; w <= W; w++) {
                        if (i == 0 || w == 0)
                                K[i][w] = 0;
                        else if (wt[i - 1] <= w)
                                K[i][w] = max(val[i - 1]
                                                + K[i - 1][w -
wt[i - 1]],
                                                K[i - 1][w]);
                        else
                                K[i][w] = K[i - 1][w];
                }
        }
        return K[n][W];
}
```

```
int main(){
        int profit[] = { 60, 100, 120 };
        int weight[] = { 10, 20, 30 };
        int W = 50;
        int n = sizeof(profit) / sizeof(profit[0]);

        cout << knapSack(W, weight, profit, n);

        return 0;
}
```

Maximize Points

Maximize Points

Sachin is free for N days. So, he decided to increase his knowledge. For each i (0 <= i <= N - 1), Sachin whill choose one of the following activities and do it on the i-th day:

- **Activity 0:** Learn maths. Gain a_{10} points of knowledge.
- **Activity 1:** Learn DSA. Gain a_{11} points of knowledge.
- **Activity 2:** Solve problems. Gain a_{12} points of knowledge.

As Sachin gets bored easily, he cannot do the same activities for two or more consecutive days. Find the maximum possible total points of knowledge that Sachin gains.

Recursive Solution

The only constraint over here is: We can not choose the same activity on 2 consecutive days.

go(i, j): Let's say we are currently choosing the activity for i-th day, and the activity done on the previous day was activity number j.

Now on i'th day, we can do any activity number k, such that k != j and k € 10, 1, 27, and we will get a, + go (i + 1, k) points.

Call: go (0, -1)

C++

```
int go(int i, int j){
        if(i == n){
        return 0;
    }
```

```
        int res = 0;
        for(int k : {0, 1, 2}){
        if(j != k){
                res = max(res, a[i][k] + go(i + 1, k));
         }
    }

        return res;
}
```

DP Solution

C++

```
int go(int i, int j){
        int dp[n][3];
        memset(dp, 0, sizeof(dp));

        for(int j : {0, 1, 2}){
        dp[n-1][j] = a[n-1][j];
        }

        for(int i = n-2; i >= 0; i--){
        for(int j : {0, 1, 2}){
                dp[i][j] = INT_MIN;
                for(int k : {0, 1, 2}){
                if(j != k){
                        dp[i][j] = max(dp[i][j], a[i][j] + dp[i+1][k]);
                }
                }
        }
        }

        int res = 0;
        for(int j : {0, 1, 2}){
        res = max(res, dp[0][j]);
        }

        return res;
}
```

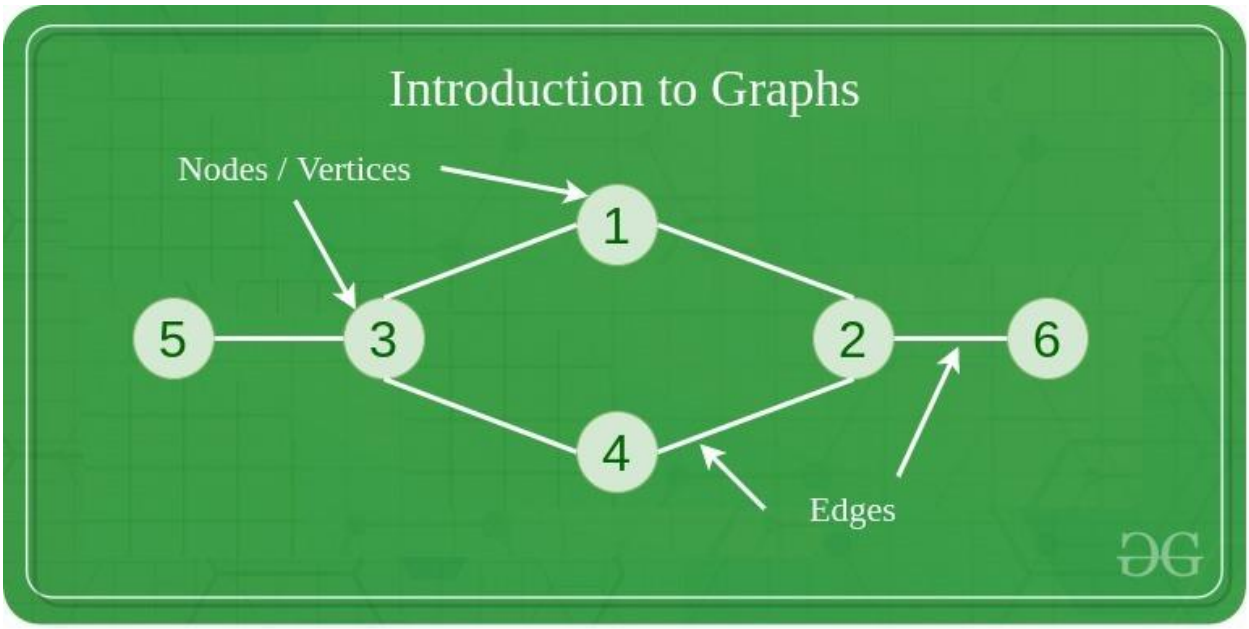
Graphs – CP

Walkthrough & BFS

Graph Walkthrough and BFS

Graph data structure

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by G(E, V).



Components of a Graph

- **Vertices:** Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertices or nodes. Every node/vertex can be labeled or unlabelled.
- **Edges:** Edges are drawn or used to connect two nodes of the graph. It can be an ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labeled/unlabelled.

Graphs are used to solve many real-life problems. Graphs are used to represent networks. The networks may include paths in a city or telephone network or circuit network. Graphs are also used in social networks like Linkedin, and Facebook. For example, on Facebook, each person is represented with a vertex(or node). Each node is a structure and contains information like person id, name, gender, locale, etc.

Find level of each node in a graph

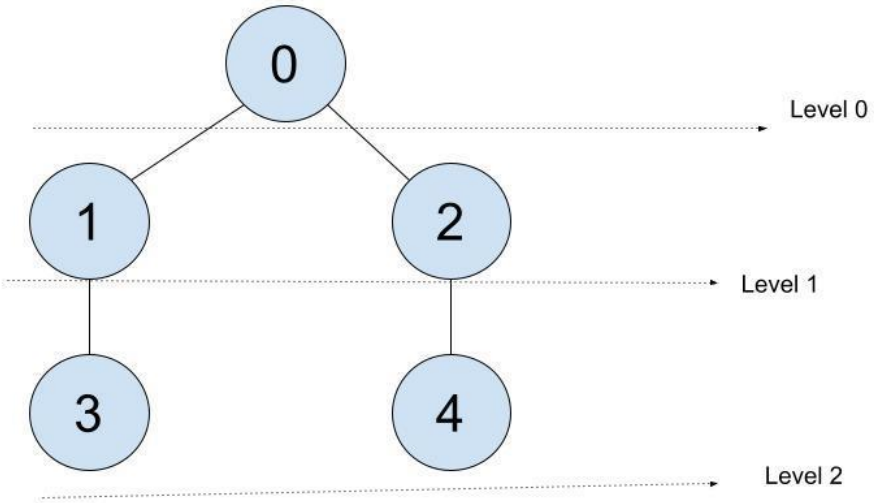
Finding the level of each node in a Graph

Given an undirected graph with **V** vertices and **E** edges and a node **X**, The task is to find the level of node **X** in the undirected graph. If X does not exist in the graph then return -1.

Note: Traverse the graph starting from vertex 0.

Examples:

Input: *V = 5, Edges = {{0, 1}, {0, 2}, {1, 3}, {2, 4}}, X = 3*
Output: *2*
Explanation: *Starting from vertex 0, at level 0 we have node 0, at level 1 we have nodes 1 and 2 and at level 2 we have nodes 3 and 4. So the answer is 2*



Approach: The problem can be solved based on the following idea:

The given problem can be solved with the help of level order traversal. We can perform a BFS traversal in order to find the level of the given vertex

Follow the steps mentioned below to implement the idea:

- Find the maximum vertex of the graph and store it in a variable (say **maxVertex**).
- create adjacency list **adj[]** of size **maxVertex+1**.
- Check if **X** is present or not, if not then return **-1**.
- To traverse the graph, create a queue for level order traversal.
- Push vertex 0 in a queue, and set a counter level to 0.
- Create a visited array of size **maxVertex+1** to mark the visited nodes.
- Start BFS traversal if the value of **X** is found in front of the queue then return the **level**.
    - Keep popping nodes from the queue till it becomes empty and increment the counter level
    - In one iteration, push all the unvisited nodes in the queue connected with the current node
    - Increment the level variable to jump to the next level.

C++

```cpp
// C++ code to implement the approach

#include <bits/stdc++.h>
using namespace std;

int findLevel(int N, vector<vector<int> >& edges, int X){

        int maxVertex = 0;
        for (auto it : edges) {
                maxVertex = max({ maxVertex, it[0], it[1] });
        }

        vector<int> adj[maxVertex + 1];
        for (int i = 0; i < edges.size(); i++) {
                adj[edges[i][0]].push_back(edges[i][1]);
                adj[edges[i][1]].push_back(edges[i][0]);
        }

        if (X > maxVertex || adj[X].size() == 0)
                return -1;

        queue<int> q;
        q.push(0);
        int level = 0;

        vector<int> visited(maxVertex + 1, 0);
        visited[0] = 1;

        while (!q.empty()) {
                int sz = q.size();
                while (sz--) {
```

```
                        auto currentNode = q.front();
                        q.pop();
                        if (currentNode == X) {
                                return level;
                        }

                        for (auto it : adj[currentNode]) {
                                if (!visited[it]) {
                                        q.push(it);
                                        visited[it] = 1;
                                }
                        }
                }
                level++;
        }

        return -1;
}

int main(){
        int V = 5;
        vector<vector<int> > edges
                = { { 0, 1 }, { 0, 2 }, { 1, 3 }, { 2, 4 } };
        int X = 3;

        int level = findLevel(V, edges, X);
        cout << level << endl;

        return 0;
}
```

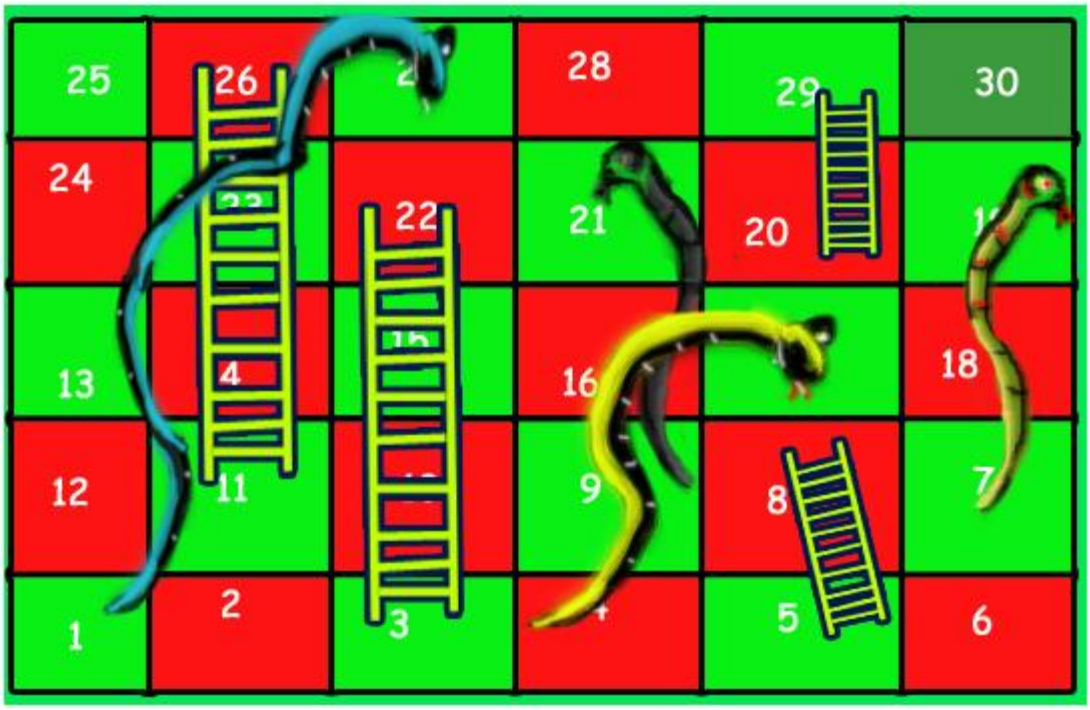
Snakes and Ladder

Snakes and Ladder

Given a snake and ladder board, find the minimum number of dice throws required to reach the destination or last cell from the source or 1st cell. Basically, the player has total control over the outcome of the dice throw and wants to find out the minimum number of throws required to reach the last cell.

If the player reaches a cell which is the base of a ladder, the player has to climb up that ladder and if reaches a cell is the mouth of the snake, and has to go down to the tail of the

snake without a dice throw.



For example, consider the board shown, the minimum number of dice throws required to reach cell 30 from cell 1 is 3.
The following are the steps:
a) First throw two dice to reach cell number 3 and then ladder to reach 22
b) Then throw 6 to reach 28.
c) Finally through 2 to reach 30.
There can be other solutions as well like (2, 2, 6), (2, 4, 4), (2, 3, 5).. etc.

The idea is to consider the given snake and ladder board as a directed graph with a number of vertices equal to the number of cells in the board. The problem reduces to finding the shortest path in a graph. Every vertex of the graph has an edge to the next six vertices if the next 6 vertices do not have a snake or ladder. If any of the next six vertices has a snake or ladder, then the edge from the current vertex goes to the top of the ladder or tail of the snake. Since all edges are of equal weight, we can efficiently find the shortest path using the Breadth-First Search of the graph.
Following is the implementation of the above idea. The input is represented by two things, the first is 'N' which is the number of cells in the given board, and the second is an array

‘move[0…N-1]’ of size N. An entry move[i] is -1 if there is no snake and no ladder from i, otherwise, move[i] contains the index of the destination cell for the snake or the ladder at i.

C++

```
#include <iostream>
#include <queue>
using namespace std;

struct queueEntry {
	int v;
	int dist;
};

int getMinDiceThrows(int move[], int N){
	bool* visited = new bool[N];
	for (int i = 0; i < N; i++)
		visited[i] = false;

	queue<queueEntry> q;

	visited[0] = true;
	queueEntry s
		= { 0, 0 };
	q.push(s);
	queueEntry qe;
	while (!q.empty()) {
		qe = q.front();
		int v = qe.v;
		if (v == N - 1)
			break;

		q.pop();
		for (int j = v + 1; j <= (v + 6) && j < N; ++j) {
			if (!visited[j]) {
				queueEntry a;
				a.dist = (qe.dist + 1);
				visited[j] = true;

				if (move[j] != -1)
					a.v = move[j];
				else
					a.v = j;
				q.push(a);
			}
		}
	}

	return qe.dist;
```

```
}

int main(){
        int N = 30;
        int moves[N];
        for (int i = 0; i < N; i++)
                moves[i] = -1;

        // Ladders
        moves[2] = 21;
        moves[4] = 7;
        moves[10] = 25;
        moves[19] = 28;

        // Snakes
        moves[26] = 0;
        moves[20] = 8;
        moves[16] = 3;
        moves[18] = 6;

        cout << "Min Dice throws required is "
                << getMinDiceThrows(moves, N);
        return 0;
}
```

Output:

```
Min Dice throws required is 3
```

Multi-source BFS

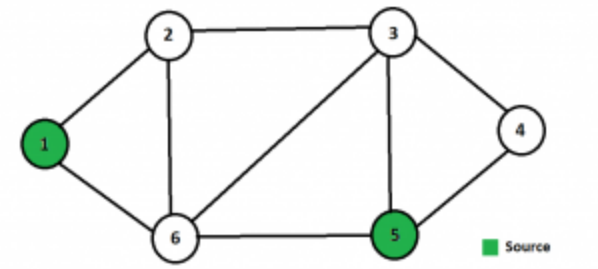
Multisource BFS

Suppose there are n towns connected by m bidirectional roads. There are s towns among them with a police station. We want to find out the distance of each town from the nearest police station. If the town itself has one the distance is 0.

Example:

```
Input :
Number of Vertices = 6
Number of Edges = 9
Towns with Police Station : 1, 5
```

```
Edges:
1 2
1 6
2 6
2 3
3 6
5 4
6 5
3 4
5 3
```



Output :

```
1 0
2 1
3 1
4 1
5 0
6 1
```

Approach

We can loop through the vertices and from each vertex run a BFS to find the closest town with a police station from that vertex. This will take O(V.E).

C++

```cpp
#include <bits/stdc++.h>
using namespace std;
#define N 100000 + 1
#define inf 1000000

int dist[N];

int source[N];

deque<pair<int, int> > BFSQueue;

// visited array for remembering visited vertices
int visited[N];
```

```
// The BFS function
void BFS(vector<int> graph[], int start){
	while (!BFSQueue.empty())
		BFSQueue.pop_back();

	BFSQueue.push_back({ start, 0 });

	while (!BFSQueue.empty()) {

		int s = BFSQueue.front().first;
		int d = BFSQueue.front().second;
		visited[s] = 1;
		BFSQueue.pop_front();

		if (source[s] == 1) {
			dist[start] = d;
			return;
		}

		for (int i = 0; i < graph[s].size(); i++)
			if (visited[graph[s][i]] == 0)
				BFSQueue.push_back({ graph[s][i], d + 1 });
	}
}

void nearestTown(vector<int> graph[], int n, int sources[], int S){
	for (int i = 1; i <= n; i++)
		source[i] = 0;
	for (int i = 0; i <= S - 1; i++)
		source[sources[i]] = 1;
	for (int i = 1; i <= n; i++) {
		for (int i = 1; i <= n; i++)
			visited[i] = 0;
		BFS(graph, i);
	}

	for (int i = 1; i <= n; i++)
		cout << i << " " << dist[i] << endl;
}

void addEdge(vector<int> graph[], int u, int v){
	graph[u].push_back(v);
	graph[v].push_back(u);
}

int main(){
	int n = 6;
```

```
        vector<int> graph[n + 1];

        addEdge(graph, 1, 2);
        addEdge(graph, 1, 6);
        addEdge(graph, 2, 6);
        addEdge(graph, 2, 3);
        addEdge(graph, 3, 6);
        addEdge(graph, 5, 4);
        addEdge(graph, 6, 5);
        addEdge(graph, 3, 4);
        addEdge(graph, 5, 3);

        int sources[] = { 1, 5 };

        int S = sizeof(sources) / sizeof(sources[0]);

        nearestTown(graph, n, sources, S);

        return 0;
}
```

Output:

```
1 0
2 1
3 1
4 1
5 0
6 1
```

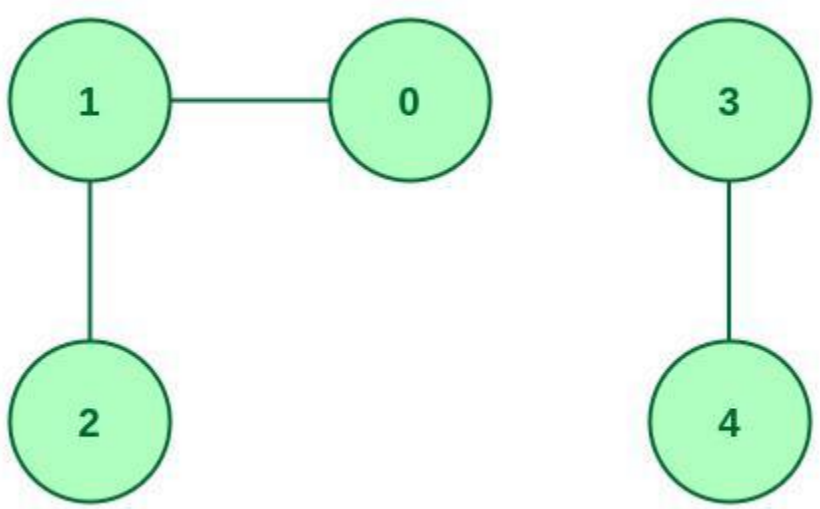
DFS

Connected components in a Graph

Connected Components in a Graph

Given an undirected graph, the task is to print all the connected components line by line.

Consider the following graph:



Output:
0 1 2
3 4
Explanation: There are 2 different connected components. They are {0, 1, 2} and {3, 4}.

Connected Components for an undirected graph using DFS:

Follow the steps mentioned below to implement the idea using DFS:

- Initialize all vertices as not visited.
- Do the following for every vertex **v**:
  - If **v** is not visited before, call the DFS. and print the newline character to print each component in a new line
    - Mark **v** as visited and print **v**.
    - For every adjacent **u** of **v**, If **u** is not visited, then recursively call the DFS.

C++

```
#include <bits/stdc++.h>
using namespace std;

class Graph {
        int V;
        list<int>* adj;

        void DFSUtil(int v, bool visited[]);

public:
        Graph(int V);
        ~Graph();
        void addEdge(int v, int w);
        void connectedComponents();
};
```

```
void Graph::connectedComponents(){
	bool* visited = new bool[V];
	for (int v = 0; v < V; v++)
		visited[v] = false;

	for (int v = 0; v < V; v++) {
		if (visited[v] == false) {
			DFSUtil(v, visited);

			cout << "\n";
		}
	}
	delete[] visited;
}

void Graph::DFSUtil(int v, bool visited[]){
	visited[v] = true;
	cout << v << " ";

	list<int>::iterator i;
	for (i = adj[v].begin(); i != adj[v].end(); ++i)
		if (!visited[*i])
			DFSUtil(*i, visited);
}

Graph::Graph(int V){
	this->V = V;
	adj = new list<int>[V];
}

Graph::~Graph() { delete[] adj; }

void Graph::addEdge(int v, int w){
	adj[v].push_back(w);
	adj[w].push_back(v);
}

int main(){
	Graph g(5);
	g.addEdge(1, 0);
	g.addEdge(2, 1);
	g.addEdge(3, 4);

	cout << "Following are connected components \n";
	g.connectedComponents();

	return 0;
}
```

Output:

```
Following are connected components
0 1 2
3 4
```

Count the number of islands in a Grid

Count the number of Islands in a grid

Given a binary 2D matrix, find the number of islands. A group of connected 1s forms an island. **For example,** the below matrix contains 5 islands.

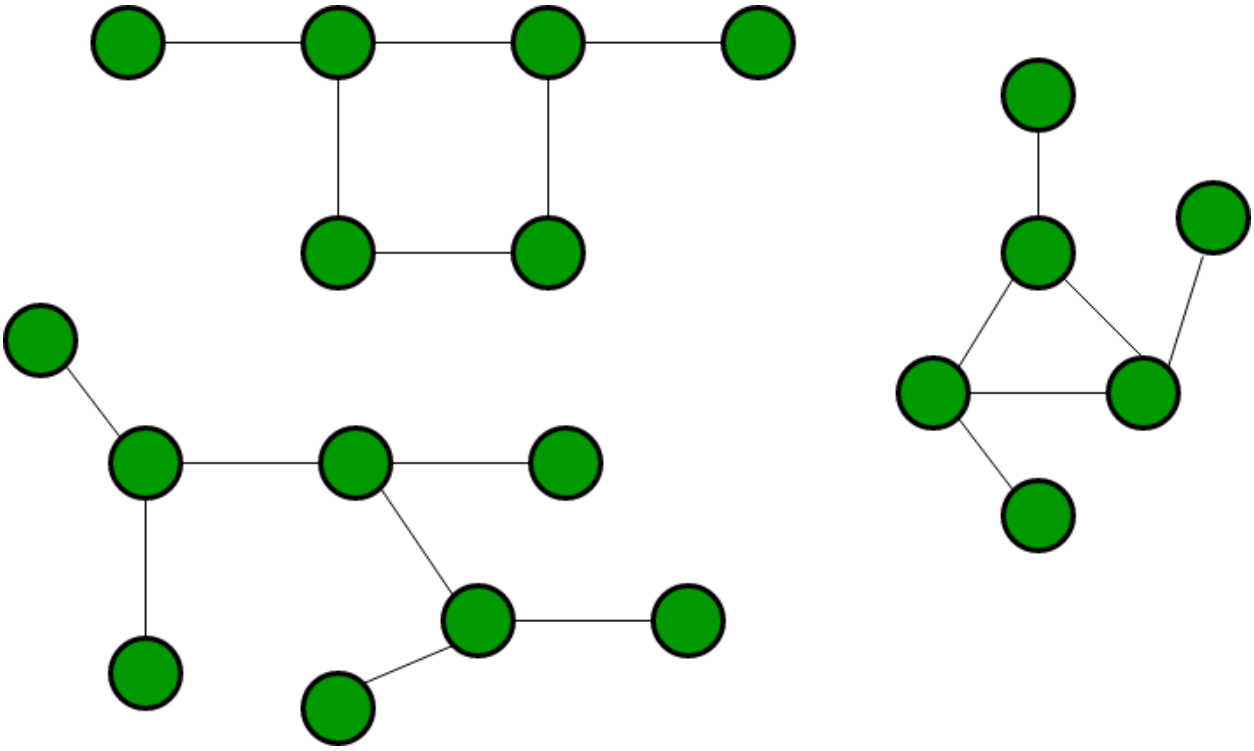
Example:

Input: *mat[][] = {{1, 1, 0, 0, 0},*
{0, 1, 0, 0, 1},
{1, 0, 0, 1, 1},
{0, 0, 0, 0, 0},
{1, 0, 1, 0, 0}}

Output: *5*

This is a variation of the standard problem: "Counting the number of connected components in an undirected graph".

Before we go to the problem, let us understand what is a connected component.
A connected component of an undirected graph is a subgraph in which every two vertices are connected to each other by a path(s), and which is connected to no other vertices outside the subgraph.
For example, the graph shown below has three connected components.



A graph where all vertices are connected with each other has exactly one connected component, consisting of the whole graph. Such a graph with only one connected component is called a Strongly Connected Graph.
This problem can be easily solved by applying DFS() on each component. In each DFS() call, a component or a sub-graph is visited. We will call DFS on the next un-visited component. The number of calls to DFS() gives the number of connected components. BFS can also be used.

What is an island?
A group of connected 1s forms an island. **For example,** the below matrix contains 4 islands:

```
{ { 1 , 1, 0, 0, 0 },
  { 0, 1, 0, 0, 1 },
  { 1 , 0, 0, 1, 1 },
  { 0 , 0, 0, 0, 0 },
  { 1, 0, 1, 1, 0 } }
```

The idea is to modify the given matrix, and perform DFS to find the total number of islands

Follow the steps below to solve the problem:

- Initialize **count = 0**, to store the answer.
- Traverse a loop from **0** till **ROW**
  - Traverse a nested loop from **0** to **COL**
    - If the value of the current cell in the given matrix is **1**
      - Increment **count** by **1**
      - Call **DFS** function
        - If the cell exceeds the boundary or the value at the current cell is 0
          - Return.
        - Update the value at the current cell as **0**.
        - Call **DFS** on the neighbor recursively
- Return **count** as the final answer.

C++

```
#include <bits/stdc++.h>
using namespace std;

void DFS(vector<vector<int> >& M, int i, int j, int ROW, int COL){
	if (i < 0 || j < 0 || i > (ROW - 1) || j > (COL - 1)
		|| M[i][j] != 1) {
		return;
	}

	if (M[i][j] == 1) {
		M[i][j] = 0;
		DFS(M, i + 1, j, ROW, COL);
		DFS(M, i - 1, j, ROW, COL);
		DFS(M, i, j + 1, ROW, COL);
		DFS(M, i, j - 1, ROW, COL);
		DFS(M, i + 1, j + 1, ROW, COL);
		DFS(M, i - 1, j - 1, ROW, COL);
		DFS(M, i + 1, j - 1, ROW, COL);
		DFS(M, i - 1, j + 1, ROW, COL);
	}
}

int countIslands(vector<vector<int> >& M){
	int ROW = M.size();
	int COL = M[0].size();
	int count = 0;
	for (int i = 0; i < ROW; i++) {
		for (int j = 0; j < COL; j++) {
			if (M[i][j] == 1) {
				count++;
				DFS(M, i, j, ROW, COL);
```

```
                        }
                    }
            }
            return count;
}

int main(){
            vector<vector<int> > M = { { 1, 1, 0, 0, 0 },
                                       { 0, 1, 0, 0, 1 },
                                       { 1, 0, 0, 1, 1 },
                                       { 0, 0, 0, 0, 0 },
                                       { 1, 0, 1, 0, 1 } };

            cout << "Number of islands is: " << countIslands(M);
            return 0;
}
```

Output:

```
Number of islands is: 5
```

Topological Sort

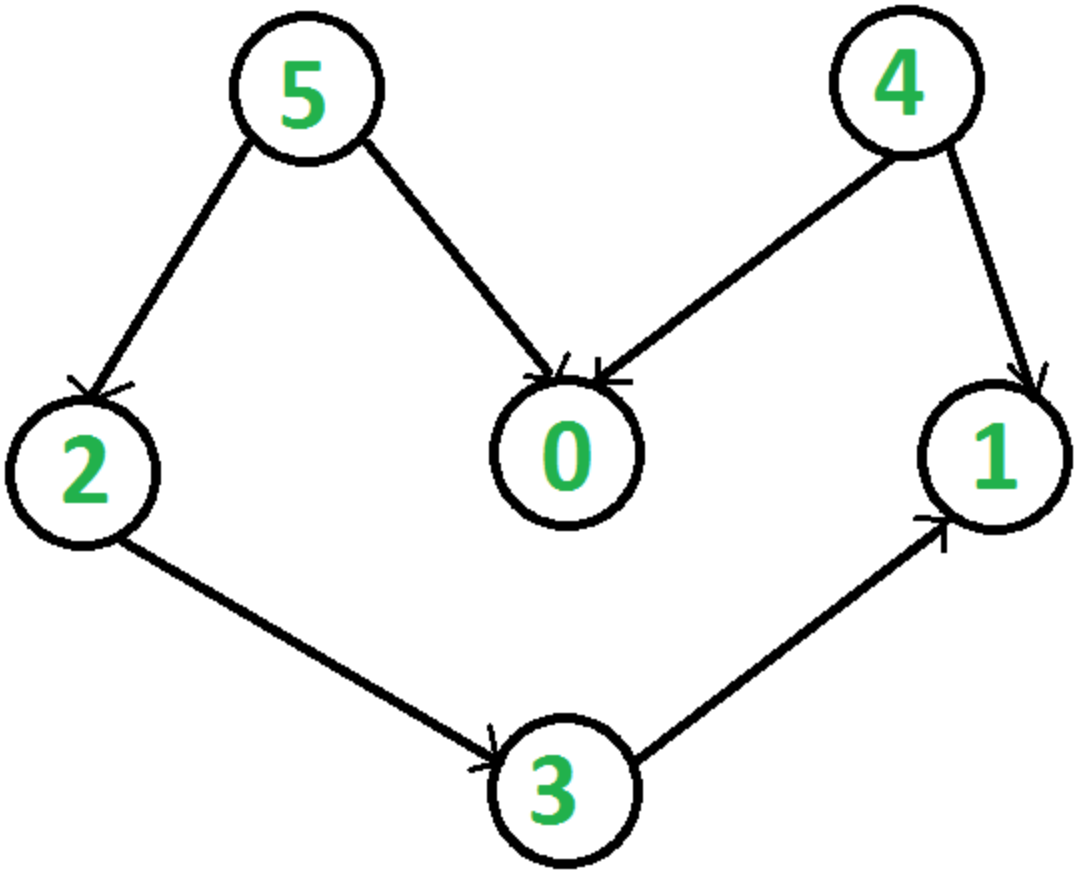
Topological Sorting Introduction

Topological Sorting Introduction

Topological sorting for Directed Acyclic Graph (DAG) is a linear ordering of vertices such that for every directed edge u v, vertex u comes before v in the ordering.

Note: Topological Sorting for a graph is not possible if the graph is not a DAG.

For example, *a topological sorting of the following graph is "5 4 2 3 1 0". There can be more than one topological sorting for a graph. Another topological sorting of the following graph is "4 5 2 3 1 0". The first vertex in topological sorting is always a vertex with an in-degree of 0 (a vertex with no incoming edges).*



Topological Sorting vs Depth First Traversal (DFS):
In DFS, we print a vertex and then recursively call DFS for its adjacent vertices. In topological sorting, we need to print a vertex before its adjacent vertices.

For example, in the given graph, the vertex '5' should be printed before vertex '0', but unlike DFS, the vertex '4' should also be printed before vertex '0'. So Topological sorting is different from DFS. For example, a DFS of the shown graph is "5 2 3 1 0 4", but it is not a topological sorting.

Topological Sorting - BFS (Kahn's Algorithm)

Kahn's Algorithm (Topological Sorting using BFS Approach)
The approach is based on the below fact:
A DAG(Directed Acyclic Graph) has at least one vertex with in-degree 0 and one vertex with out-degree 0.

Proof: There's simple proof to the above fact that a DAG does not contain a cycle which means that all paths will be of finite length. Now let S be the longest path from u(source) to v(destination). Since S is the longest path there can be no incoming edge to u and no outgoing edge from v, if this situation had occurred then S would not have been the longest path
=> indegree(u) = 0 and outdegree(v) = 0

Intuition:

Topological sorting is a kind of dependencies problem so, we can start with the tasks which do not have any dependencies and can be done straight away or simply if we talk about in the term of nodes,

- We will always try to execute those nodes that have outdegree 0.
- Then after execution of all those 0 outdegrees, we will execute which are directly dependent on currently resolved tasks (currently resolved tasks' outdegrees will become 0 now) and so on will execute all other tasks.

We look closely we are doing these executions are done level-wise or in a Breadth-first search (BFS) manner. Similarly, we can also perform the same task for indegree=0.

Algorithm: Steps involved in finding the topological ordering of a DAG:
Step-1: Compute in-degree (number of incoming edges) for each of the vertex presents in the DAG and initialize the count of visited nodes as 0.
Step-2: Pick all the vertices with in-degree as 0 and add them into a queue (Enqueue operation)
Step-3: Remove a vertex from the queue (Dequeue operation) and then.

1. Increment the count of visited nodes by 1.
2. Decrease in-degree by 1 for all its neighboring nodes.
3. If the in-degree of neighboring nodes is reduced to zero, then add it to the queue.

Step 4: Repeat Step 3 until the queue is empty.
Step 5: If the count of visited nodes is **not** equal to the number of nodes in the graph then the topological sort is not possible for the given graph.
How to find the in-degree of each node?
There are 2 ways to calculate the in-degree of every vertex:

1. Take an in-degree array which will keep track of
   Traverse the array of edges and simply increase the counter of the destination node by 1.

```
for each node in Nodes
```

```
    indegree[node] = 0;
for each edge(src, dest) in Edges
    indegree[dest]++
```

1. Time Complexity: O(V+E)
2. Traverse the list for every node and then increment the in-degree of all the nodes connected to it by 1.

```
    for each node in Nodes
        If (list[node].size()!=0) then
        for each dest in list
            indegree[dest]++;
```

1. Time Complexity: The outer for loop will be executed V a number of times and the inner for loop will be executed E a number of times, Thus overall time complexity is O(V+E).
   The overall time complexity of the algorithm is O(V+E)

C++

```
#include <bits/stdc++.h>
using namespace std;

class Graph {
        int V;

        list<int>* adj;

public:
        // Constructor
        Graph(int V);

        void addEdge(int u, int v);

        void topologicalSort();
};

Graph::Graph(int V){
        this->V = V;
        adj = new list<int>[V];
}

void Graph::addEdge(int u, int v){
        adj[u].push_back(v);
}

void Graph::topologicalSort(){
        vector<int> in_degree(V, 0);

        for (int u = 0; u < V; u++) {
                list<int>::iterator itr;
                for (itr = adj[u].begin();
                        itr != adj[u].end(); itr++)
```

```
                        in_degree[*itr]++;
        }

        // Create an queue and enqueue
        // all vertices with indegree 0
        queue<int> q;
        for (int i = 0; i < V; i++)
                if (in_degree[i] == 0)
                        q.push(i);

        // Initialize count of visited vertices
        int cnt = 0;

        vector<int> top_order;

        while (!q.empty()) {
                int u = q.front();
                q.pop();
                top_order.push_back(u);

                list<int>::iterator itr;
                for (itr = adj[u].begin();
                        itr != adj[u].end(); itr++)

                        // If in-degree becomes zero,
                        // add it to queue
                        if (--in_degree[*itr] == 0)
                                q.push(*itr);

                cnt++;
        }

        // Check if there was a cycle
        if (cnt != V) {
                cout << "There exists a cycle in the graph\n";
                return;
        }

        // Print topological order
        for (int i = 0; i < top_order.size(); i++)
                cout << top_order[i] << " ";
        cout << endl;
}

int main(){
        Graph g(6);
        g.addEdge(5, 2);
        g.addEdge(5, 0);
        g.addEdge(4, 0);
```

```
        g.addEdge(4, 1);
        g.addEdge(2, 3);
        g.addEdge(3, 1);

        cout << "Following is a Topological Sort of\n";
        g.topologicalSort();

        return 0;
}
```

Application of Kahn's algorithm for Topological Sort:

1. **Course sequencing**: Courses at universities frequently have prerequisites for other courses. The courses can be scheduled using Kahn's algorithm so that the prerequisites are taken before the courses that call for them.

2. **Management of software dependencies**: When developing software, libraries and modules frequently rely on other libraries and modules. The dependencies can be installed in the proper order by using Kahn's approach.

3. **Scheduling tasks**: In project management, activities frequently depend on one another. The tasks can be scheduled using Kahn's method so that the dependent tasks are finished before the tasks that depend on them.

4. **Data processing**: In data processing pipelines, the outcomes of some processes may be dependent. The stages can be carried out in the right order by using Kahn's algorithm.

5. **Circuit design**: In the creation of an electronic circuit, some components may be dependent on the output of others. The components can be joined in the right order by using Kahn's technique.

Topological Sorting - DFS

Algorithm for Topological Sorting using DFS:

We can modify DFS to find the Topological Sorting of a graph. In DFS,

- We start from a vertex, we first print it, and then
- Recursively call DFS for its adjacent vertices.

In topological sorting,

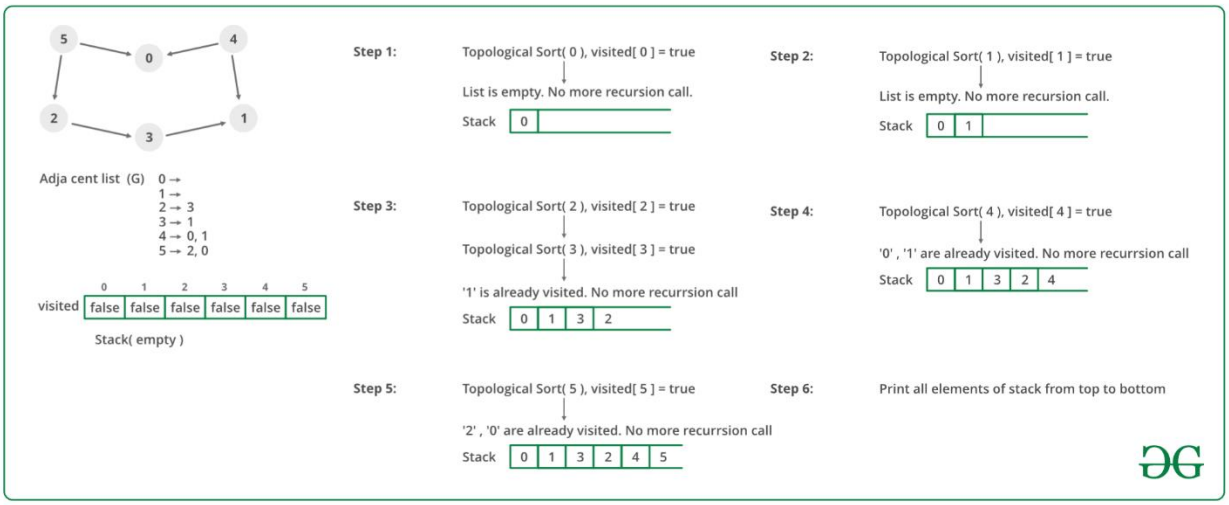
- We use a temporary stack.
- We don't print the vertex immediately,
- we first recursively call topological sorting for all its adjacent vertices, then push it to a stack.
- Finally, print the contents of the stack.

Note: A vertex is pushed to stack only when all of its adjacent vertices (and their adjacent vertices and so on) are already in the stack

Approach:

- Create a **stack** to store the nodes.
- Initialize **visited** array of size **N** to keep the record of visited nodes.
- Run a loop from **0** till **N**
    - if the node is not marked **True** in **visited** array
        - Call the recursive function for topological sort and perform the following steps.
            - Mark the current node as **True** in the **visited** array.
            - Run a loop on all the nodes which has a directed edge to the current node
                - if the node is not marked **True** in the **visited** array:
                - Recursively call the topological sort function on the node
            - Push the current node in the stack.
- Print all the elements in the stack.

The below image is an illustration of the above approach:



C++

```
#include <bits/stdc++.h>
using namespace std;

// Class to represent a graph
class Graph {
```

```
        // No. of vertices'
        int V;

        // Pointer to an array containing adjacency listsList
        list<int>* adj;

        // A function used by topologicalSort
        void topologicalSortUtil(int v, bool visited[],
                                                stack<int>& Stack);

public:
        // Constructor
        Graph(int V);

        // function to add an edge to graph
        void addEdge(int v, int w);

        // prints a Topological Sort of
        // the complete graph
        void topologicalSort();
};

Graph::Graph(int V)
{
        this->V = V;
        adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
        // Add w to v's list.
        adj[v].push_back(w);
}

// A recursive function used by topologicalSort
void Graph::topologicalSortUtil(int v, bool visited[],
                                                        stack<int>&
Stack)
{
        // Mark the current node as visited.
        visited[v] = true;

        // Recur for all the vertices
        // adjacent to this vertex
        list<int>::iterator i;
        for (i = adj[v].begin(); i != adj[v].end(); ++i)
                if (!visited[*i])
                        topologicalSortUtil(*i, visited, Stack);
```

```
        // Push current vertex to stack
        // which stores result
        Stack.push(v);
}

// The function to do Topological Sort.
// It uses recursive topologicalSortUtil()
void Graph::topologicalSort()
{
        stack<int> Stack;

        // Mark all the vertices as not visited
        bool* visited = new bool[V];
        for (int i = 0; i < V; i++)
                visited[i] = false;

        // Call the recursive helper function
        // to store Topological
        // Sort starting from all
        // vertices one by one
        for (int i = 0; i < V; i++)
                if (visited[i] == false)
                        topologicalSortUtil(i, visited, Stack);

        // Print contents of stack
        while (Stack.empty() == false) {
                cout << Stack.top() << " ";
                Stack.pop();
        }

        delete [] visited;
}

// Driver Code
int main()
{
        // Create a graph given in the above diagram
        Graph g(6);
        g.addEdge(5, 2);
        g.addEdge(5, 0);
        g.addEdge(4, 0);
        g.addEdge(4, 1);
        g.addEdge(2, 3);
        g.addEdge(3, 1);

        cout << "Following is a Topological Sort of the given "
                        "graph \n";

        // Function Call
```

```
        g.topologicalSort();

        return 0;
}
```

Lowest Common Ancestor (LCA)

Lowest Common Ancestor - Introduction

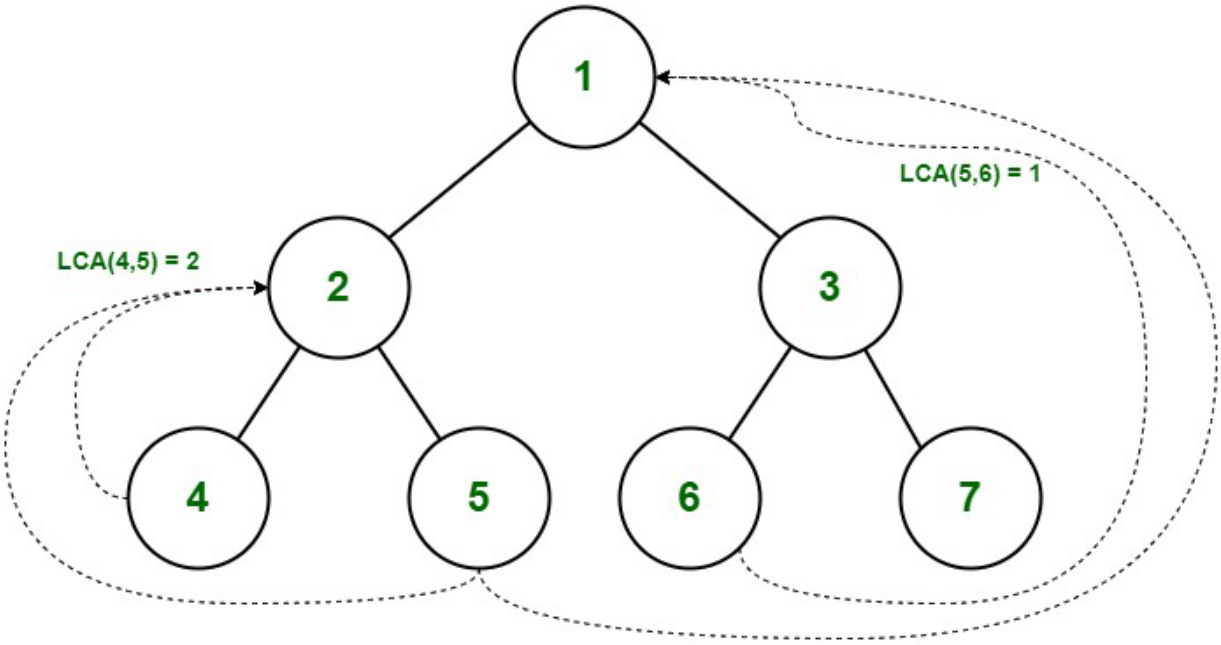
Lowest Common Ancestor - Introduction

We can understand LCA of a graph exactly similar to that of LCA in a Binary Tree. So, let's have a look at the explanation again:

*The **lowest common ancestor** is the lowest node in the tree that has both n1 and n2 as **descendants,** where n1 and n2 are the nodes for which we wish to find the LCA. Hence, the LCA of a binary tree with nodes n1 and n2 is the shared ancestor of n1 and n2 that is located farthest from the root.*

Application of Lowest Common Ancestor(LCA):

To determine the distance between pairs of nodes in a tree: the distance from n1 to n2 can be computed as the distance from the root to n1, plus the distance from the root to n2, minus twice the distance from the root to their lowest common ancestor.



Lowest Common Ancestor - Brute Force

Lowest Common Ancestor - Brute Force

Finding the lowest common ancestor (LCA) of two nodes in a general graph is a complex problem with no efficient brute-force solution. In general, you would need to perform a depth-first search starting from each of the two nodes and record all the nodes that are visited during each traversal. Then, you can iterate over these two lists of visited nodes to find the last node that appears in both lists. This last node will be the LCA of the two original nodes.

However, this approach has a time complexity of O(N^2), where N is the number of nodes in the graph. For large graphs, this can quickly become impractical.

If your graph is a tree or has some additional structure (such as being a directed acyclic graph), there might be more efficient algorithms available for finding the LCA. For example, in a binary tree, you can use a recursive algorithm that starts at the root and recursively traverses left or right based on whether the two target nodes lie in different subtrees. This algorithm has a time complexity of O(h), where h is the height of the tree.

C++

```cpp
#include <iostream>
#include <vector>
#include <queue>

using namespace std;

const int MAXN = 100005;

vector<int> adj[MAXN];
int parent[MAXN];

void bfs(int start) {
    queue<int> q;
    q.push(start);
    parent[start] = -1; // mark start as root
    while (!q.empty()) {
        int u = q.front();
        q.pop();
        for (int v : adj[u]) {
            if (parent[v] == -1) {
                parent[v] = u;
                q.push(v);
```

```
            }
        }
    }
}

int lca(int u, int v) {
    vector<int> path_u, path_v;
    while (u != -1) {
        path_u.push_back(u);
        u = parent[u];
    }
    while (v != -1) {
        path_v.push_back(v);
        v = parent[v];
    }
    int i = path_u.size() - 1, j = path_v.size() - 1;
    while (i >= 0 && j >= 0 && path_u[i] == path_v[j]) {
        i--;
        j--;
    }
    return path_u[i+1];
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        adj[u].push_back(v);
        adj[v].push_back(u);
    }
    int u, v;
    cin >> u >> v;
    // perform BFS to compute parents
    memset(parent, -1, sizeof(parent));
    bfs(1); // assume 1 is the root
    // find LCA of u and v
    int result = lca(u, v);
    cout << "LCA of " << u << " and " << v << " is " << result << endl;
    return 0;
}
```

LCA in a Graph using Binary Lifting Technique

LCA in a Graph using Binary Lifting Technique

We can again visualize the graph as a Tree Data Structure.

Binary Lifting is a dynamic programming approach where we pre-compute an array memo[1, n][1, log(n)] where memo[i][j] contains 2^j-th ancestor of node i. For computing the values of memo[][], the following recursion may be used:

```
memo state:

   memo[i][j] = i-th node's 2^(j)th ancestor in the path

memo initialization:

   memo[i][j] = memo[i][0] (first parent (2^0) of each node is
given)

memo trans:

   memo[i][j] = memo[ memo [i][j - 1]]

meaning: A(i,2^j)=A( A(i , 2^(j-1) ) , 2^(j-1) )

To find the (2^j)-th ancestor of i, recursively find i-th node's
2^(j-1)th ancestor's 2^(j-1)th ancestor. (2^(j) = 2^(j-1) +
2^(j-1))

So:

memo[i][j] = parent[i] if j = 0 and
memo[i][j] = memo[memo[i][j - 1]][j - 1] if j > 0.
```

We first check whether a node is an ancestor of another or not and if one node is ancestor of another then it is the LCA of these two nodes otherwise we find a node that is not the common ancestor of both u and v and is the highest(i.e. a node x such that x is not the common ancestor of u and v but memo[x][0] is) in the tree. After finding such a node (let it be x), we print the first ancestor of x i.e. memo[x][0] which will be the required LCA.

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

void dfs(int u, int p, int **memo, vector<int> &lev, int log, vector<int>
*g){
        memo[u][0] = p;
        for (int i = 1; i <= log; i++)
                memo[u][i] = memo[memo[u][i - 1]][i - 1];
```

```
        for (int v : g[u]){
                if (v != p){
                        lev[v] = lev[u] + 1;
                        dfs(v, u, memo, lev, log, g);
                }
        }
}

int lca(int u, int v, int log, vector<int> &lev, int **memo){
        if (lev[u] < lev[v])
                swap(u, v);

        for (int i = log; i >= 0; i--)
                if ((lev[u] - pow(2, i)) >= lev[v])
                        u = memo[u][i];

        if (u == v)
                return u;

        for (int i = log; i >= 0; i--){
                if (memo[u][i] != memo[v][i]){
                        u = memo[u][i];
                        v = memo[v][i];
                }
        }

        return memo[u][0];
}

int main(){
        // Number of nodes
        int n = 9;

        // vector to store tree
        vector<int> g[n + 1];

        int log = (int)ceil(log2(n));
        int **memo = new int *[n + 1];
        for (int i = 0; i < n + 1; i++)
                memo[i] = new int[log + 1];

        // Stores the level of each node
        vector<int> lev(n + 1);

        // Initialising memo values with -1
        for (int i = 0; i <= n; i++)
                memset(memo[i], -1, sizeof memo[i]);

        // Add edges
```

```
        g[1].push_back(2);
        g[2].push_back(1);
        g[1].push_back(3);
        g[3].push_back(1);
        g[1].push_back(4);
        g[4].push_back(1);
        g[2].push_back(5);
        g[5].push_back(2);
        g[3].push_back(6);
        g[6].push_back(3);
        g[3].push_back(7);
        g[7].push_back(3);
        g[3].push_back(8);
        g[8].push_back(3);
        g[4].push_back(9);
        g[9].push_back(4);
        dfs(1, 1, memo, lev, log, g);
        cout << "The LCA of 6 and 9 is " << lca(6, 9, log, lev, memo) <<
endl;
        cout << "The LCA of 5 and 9 is " << lca(5, 9, log, lev, memo) <<
endl;
        cout << "The LCA of 6 and 8 is " << lca(6, 8, log, lev, memo) <<
endl;
        cout << "The LCA of 6 and 1 is " << lca(6, 1, log, lev, memo) <<
endl;

        return 0;
}
```

Bridges and Articulation Point

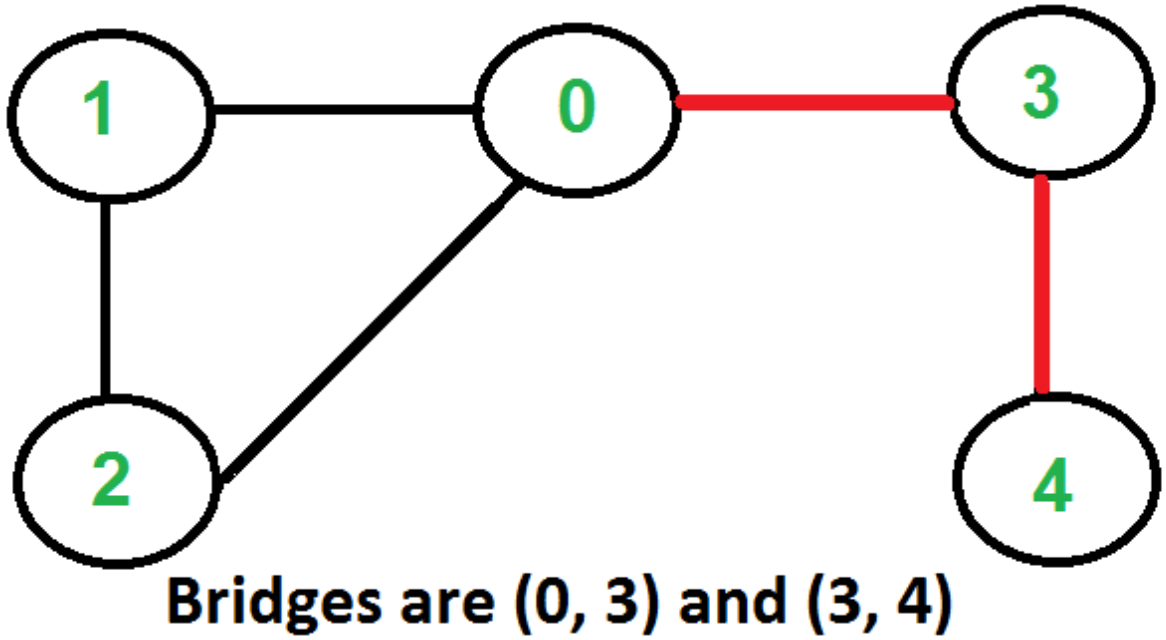
Bridges and Articulation Point - Introduction

Bridges in a Graph

An edge in an undirected connected graph is a bridge if removing it disconnects the graph. For a disconnected undirected graph, the definition is similar, a bridge is an edge removal that increases the number of disconnected components.

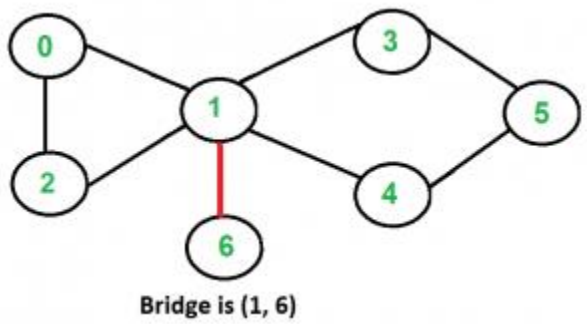
Examples:

Input:



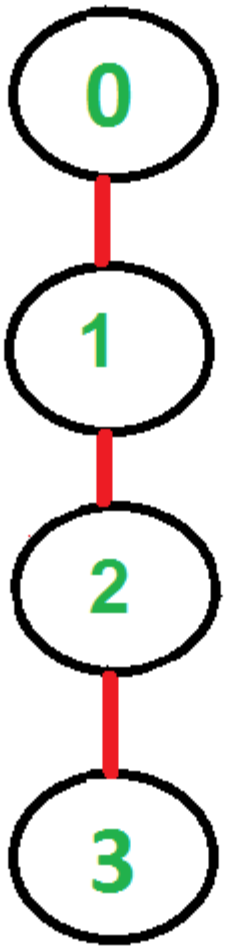
Output: *(0, 3) and (3, 4)*

Input:



Output: *(1, 6)*

Input:



Bridges are (0,1), (1,2) and (2,3)

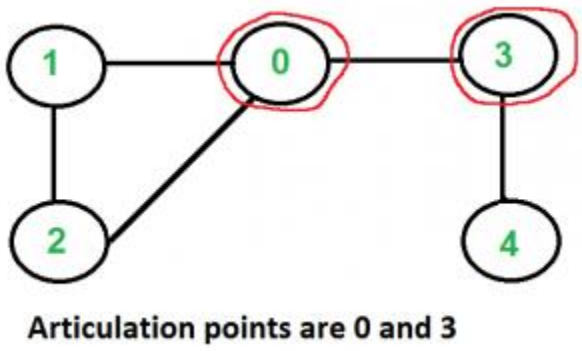
Output: *(0, 1), (1, 2), and (2, 3)*

Articulation Point in Graphs

Note: A vertex in an undirected connected graph is an articulation point (or cut vertex) if removing it (and edges through it) disconnects the graph. Articulation points represent vulnerabilities in a connected network – single points whose failure would split the network into 2 or more components. They are useful for designing reliable networks. For a disconnected undirected graph, an articulation point is a vertex removal which increases the number of connected components.

Examples:

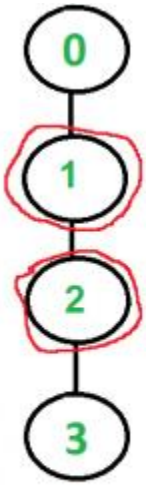
Input:



Articulation points are 0 and 3

Output: *0, 3*

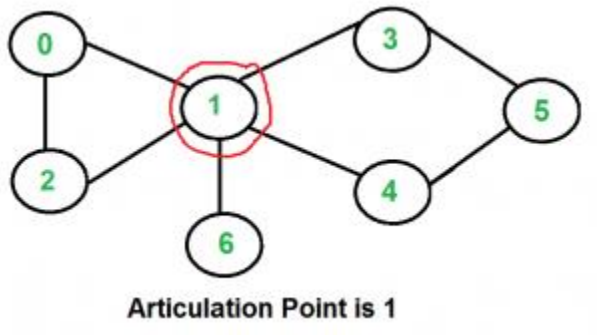
Input:



Artiulation
Points are 1 & 2

Output: *1, 2*

Input:



Output: *1*

Bridges - Algorithm and Code

Naive Approach: Below is the idea to solve the problem:

One by one remove all edges and see if removal of an edge causes disconnected graph.
Follow the below steps to Implement the idea:

- For every edge (u, v), do the following:
  - Remove (u, v) from the graph
  - See if the graph remains connected (either uses BFS or DFS)
  - Add (u, v) back to the graph.

Time Complexity: O(E*(V+E)) for a graph represented by an adjacency list.
Auxiliary Space: O(V+E)

Approach:
Do DFS traversal of the given graph. In DFS tree an edge (u, v) (u is parent of v in DFS tree) is bridge if there does not exist any other alternative edge to reach u or an ancestor of u from subtree rooted with v.

C++

```
#include<iostream>
#include <list>
#define NIL -1
using namespace std;

// A class that represents an undirected graph
class Graph
{
    int V;    // No. of vertices
```

```
    list<int> *adj;    // A dynamic array of adjacency lists
    void bridgeUtil(int v, bool visited[], int disc[], int low[],
                    int parent[]);
public:
    Graph(int V);   // Constructor
    void addEdge(int v, int w);   // to add an edge to graph
    void bridge();    // prints all bridges
};

Graph::Graph(int V)
{
    this->V = V;
    adj = new list<int>[V];
}

void Graph::addEdge(int v, int w)
{
    adj[v].push_back(w);
    adj[w].push_back(v);  // Note: the graph is undirected
}

void Graph::bridgeUtil(int u, bool visited[], int disc[], int low[], int
parent[]){
    static int time = 0;

    visited[u] = true;

    disc[u] = low[u] = ++time;

    list<int>::iterator i;
    for (i = adj[u].begin(); i != adj[u].end(); ++i){
        int v = *i;

        if (!visited[v]){
            parent[v] = u;
            bridgeUtil(v, visited, disc, low, parent);

            low[u]  = min(low[u], low[v]);

            if (low[v] > disc[u])
              cout << u <<" " << v << endl;
        }

        else if (v != parent[u])
            low[u]  = min(low[u], disc[v]);
    }
}

void Graph::bridge(){
```

```
    bool *visited = new bool[V];
    int *disc = new int[V];
    int *low = new int[V];
    int *parent = new int[V];

    for (int i = 0; i < V; i++){
        parent[i] = NIL;
        visited[i] = false;
    }

    for (int i = 0; i < V; i++)
        if (visited[i] == false)
            bridgeUtil(i, visited, disc, low, parent);
}

// Driver program to test above function
int main()
{
    // Create graphs given in above diagrams
    cout << "\nBridges in first graph \n";
    Graph g1(5);
    g1.addEdge(1, 0);
    g1.addEdge(0, 2);
    g1.addEdge(2, 1);
    g1.addEdge(0, 3);
    g1.addEdge(3, 4);
    g1.bridge();

    cout << "\nBridges in second graph \n";
    Graph g2(4);
    g2.addEdge(0, 1);
    g2.addEdge(1, 2);
    g2.addEdge(2, 3);
    g2.bridge();

    cout << "\nBridges in third graph \n";
    Graph g3(7);
    g3.addEdge(0, 1);
    g3.addEdge(1, 2);
    g3.addEdge(2, 0);
    g3.addEdge(1, 3);
    g3.addEdge(1, 4);
    g3.addEdge(1, 6);
    g3.addEdge(3, 5);
    g3.addEdge(4, 5);
    g3.bridge();

    return 0;
}
```

Articulation Point - Algorithm and Code

Naive Approach:

Below is the idea to solve the problem:

A simple approach is to one by one remove all vertices and see if removal of a vertex causes disconnected graph.
Follow the below steps to Implement the idea:

- Iterate over all the vertices and for every vertex do the following:
  - Remove v from the graph
  - See if the graph remains connected (We can either use BFS or DFS)
  - Add v back to the graph

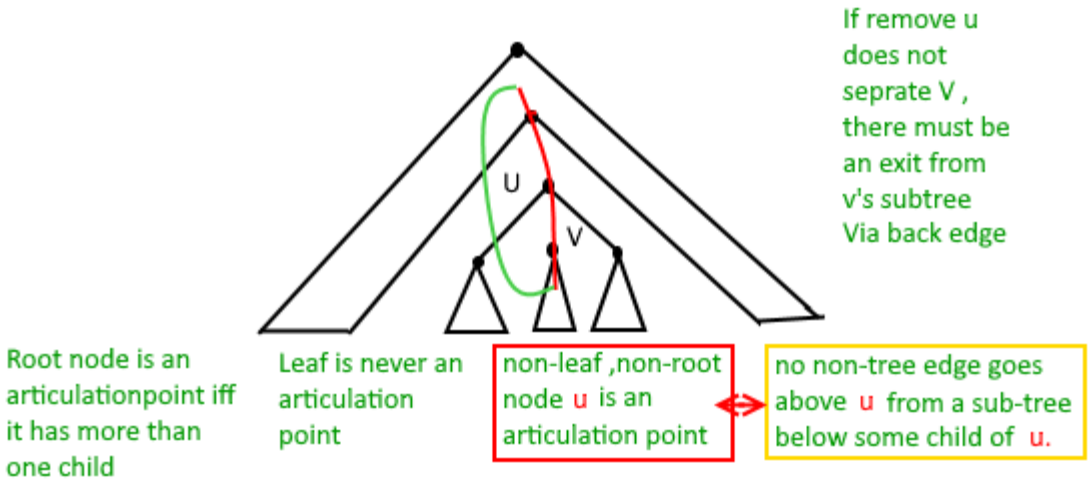
Time Complexity: O(V*(V+E)) for a graph represented using an adjacency list.
Auxiliary Space: O(V+E)

Finding Articulation Point using Tarjan's Algorithm
*The idea is to use DFS (Depth First Search). In DFS, follow vertices in a tree form called the **DFS tree**. In the DFS tree, a vertex **u** is the parent of another vertex **v**, if **v** is discovered by **u**.*

*In DFS tree, a vertex **u** is an articulation point if one of the following two conditions is true.*

- ***u** is the root of the DFS tree and it has at least two children.*
- ***u** is not the root of the DFS tree and it has a child v such that no vertex in the subtree rooted with **v** has a back edge to one of the ancestors in DFS tree of **u**.*

The following figure shows the same points as above with one additional point that a leaf in DFS Tree can never be an articulation point.



Follow the below steps to Implement the idea:

- Do DFS traversal of the given graph
  - In DFS traversal, maintain a **parent[]** array where **parent[u]** stores the parent of vertex **u**.
  - To check if **u** is the root of the DFS tree and it has at least two children. For every vertex, count children. If the currently visited vertex **u** is root (parent[u] is NULL) and has more than two children, print it.
  - To handle a second case where **u** is not the root of the DFS tree and it has a child **v** such that no vertex in the subtree rooted with **v** has a back edge to one of the ancestors in DFS tree of **u** maintain an array **disc[]** to store the discovery time of vertices.
  - For every node **u**, find out the earliest visited vertex (the vertex with minimum discovery time) that can be reached from the subtree rooted with **u**. So we maintain an additional array **low[]** such that:
    low[u] = min(disc[u], disc[w]) , Here **w** is an ancestor of **u** and there is a back edge from some descendant of **u** to **w**.

C++

```
#include <bits/stdc++.h>
using namespace std;

void APUtil(vector<int> adj[], int u, bool visited[],
                        int disc[], int low[], int& time, int parent,
                        bool isAP[]){
        int children = 0;

        visited[u] = true;

        disc[u] = low[u] = ++time;

        for (auto v : adj[u]) {
                if (!visited[v]) {
```

```
                        children++;
                        APUtil(adj, v, visited, disc, low, time, u, isAP);

                        low[u] = min(low[u], low[v]);

                        if (parent != -1 && low[v] >= disc[u])
                                isAP[u] = true;
                }

                else if (v != parent)
                        low[u] = min(low[u], disc[v]);
        }

        if (parent == -1 && children > 1)
                isAP[u] = true;
}

void AP(vector<int> adj[], int V){
        int disc[V] = { 0 };
        int low[V];
        bool visited[V] = { false };
        bool isAP[V] = { false };
        int time = 0, par = -1;

        for (int u = 0; u < V; u++)
                if (!visited[u])
                        APUtil(adj, u, visited, disc, low,
                                time, par, isAP);

        for (int u = 0; u < V; u++)
                if (isAP[u] == true)
                        cout << u << " ";
}

void addEdge(vector<int> adj[], int u, int v){
        adj[u].push_back(v);
        adj[v].push_back(u);
}

int main(){
        // Create graphs given in above diagrams
        cout << "Articulation points in first graph \n";
        int V = 5;
        vector<int> adj1[V];
        addEdge(adj1, 1, 0);
        addEdge(adj1, 0, 2);
        addEdge(adj1, 2, 1);
        addEdge(adj1, 0, 3);
        addEdge(adj1, 3, 4);
```

```
        AP(adj1, V);

        cout << "\nArticulation points in second graph \n";
        V = 4;
        vector<int> adj2[V];
        addEdge(adj2, 0, 1);
        addEdge(adj2, 1, 2);
        addEdge(adj2, 2, 3);
        AP(adj2, V);

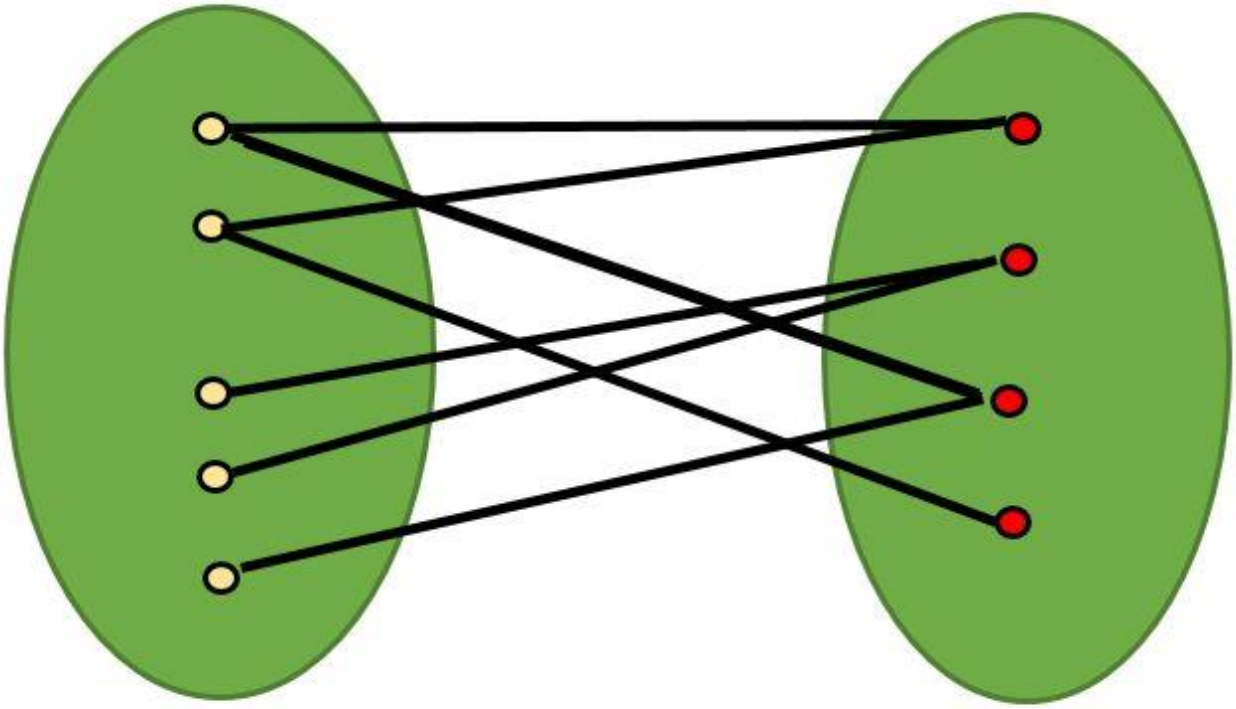
        cout << "\nArticulation points in third graph \n";
        V = 7;
        vector<int> adj3[V];
        addEdge(adj3, 0, 1);
        addEdge(adj3, 1, 2);
        addEdge(adj3, 2, 0);
        addEdge(adj3, 1, 3);
        addEdge(adj3, 1, 4);
        addEdge(adj3, 1, 6);
        addEdge(adj3, 3, 5);
        addEdge(adj3, 4, 5);
        AP(adj3, V);

        return 0;
}
```

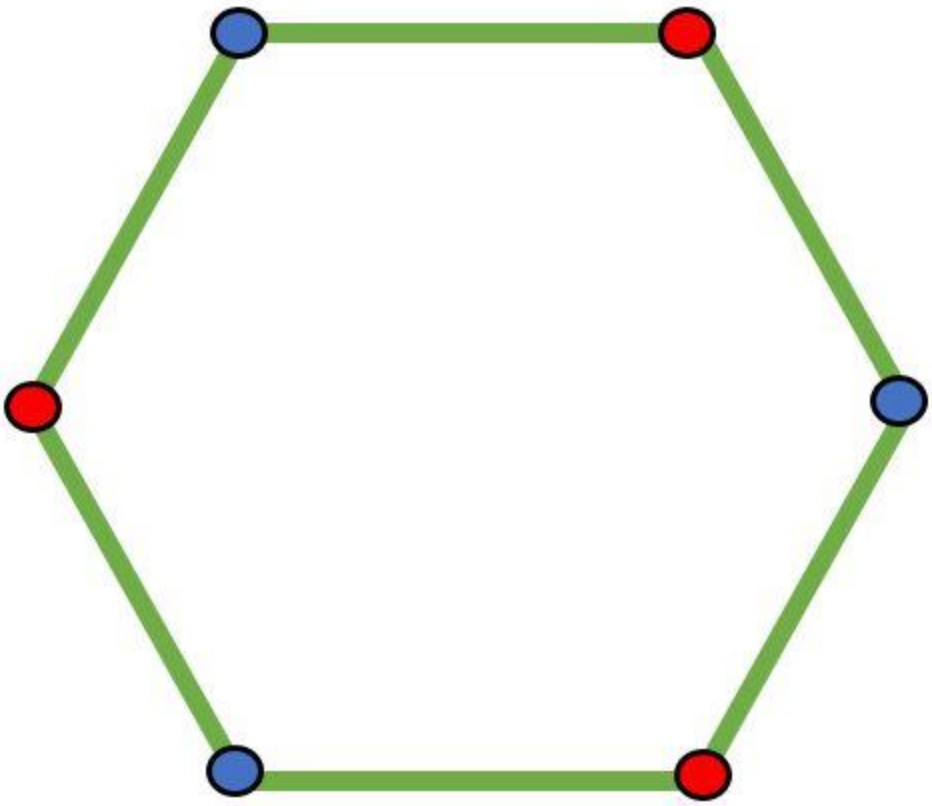
Bipartite Graph

Bipartite Graph - Introduction

A Bipartite Graph is a graph whose vertices can be divided into two independent sets, U and V such that every edge (u, v) either connects a vertex from U to V or a vertex from V to U. In other words, for every edge (u, v), either u belongs to U and v to V, or u belongs to V and v to U. We can also say that there is no edge that connects vertices of same set.

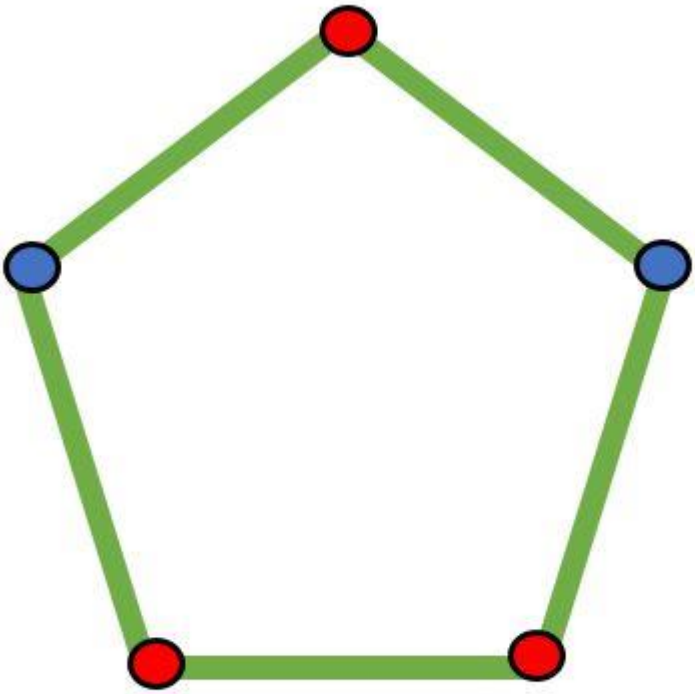


A bipartite graph is possible if the graph coloring is possible using two colors such that vertices in a set are colored with the same color. Note that it is possible to color a cycle graph with even cycle using two colors. For example, see the following graph.



Cycle graph of length 6

It is not possible to color a cycle graph with odd cycle using two colors.



Cycle graph of length 5

Bipartite Graph - Algorithm and Code

Algorithm to check if a graph is Bipartite:

One approach is to check whether the graph is 2-colorable or not using the backtracking algorithm m coloring problem.

Following is a simple algorithm to find out whether a given graph is Bipartite or not using **Breadth First Search (BFS).**

1. Assign RED color to the source vertex (putting into set U).
2. Color all the neighbors with BLUE color (putting into set V).
3. Color all neighbor's neighbors with RED color (putting into set U).
4. This way, assign a color to all vertices such that it satisfies all the constraints of the m way coloring problem where m = 2.

5. While assigning colors, if we find a neighbor which is colored with the same color as the current vertex, then the graph cannot be colored with 2 vertices (or the graph is not Bipartite)

C++

```
#include <iostream>
#include <queue>
#define V 4

using namespace std;

bool isBipartite(int G[][V], int src){
	int colorArr[V];
	for (int i = 0; i < V; ++i)
		colorArr[i] = -1;

	colorArr[src] = 1;

	queue <int> q;
	q.push(src);

	while (!q.empty()){
		// Dequeue a vertex from queue ( Refer http://goo.gl/35oz8
)
		int u = q.front();
		q.pop();

		// Return false if there is a self-loop
		if (G[u][u] == 1)
		return false;

		// Find all non-colored adjacent vertices
		for (int v = 0; v < V; ++v){
			if (G[u][v] && colorArr[v] == -1){
				// Assign alternate color to this adjacent v
of u
				colorArr[v] = 1 - colorArr[u];
				q.push(v);
			}
			else if (G[u][v] && colorArr[v] == colorArr[u])
				return false;
		}
	}

	return true;
}
```

```
int main(){
	int G[][V] = {{0, 1, 0, 1},
		{1, 0, 1, 0},
		{0, 1, 0, 1},
		{1, 0, 1, 0}
	};

	isBipartite(G, 0) ? cout << "Yes" : cout << "No";
	return 0;
}
```

The above algorithm works only if the graph is connected. In above code, we always start with source 0 and assume that vertices are visited from it. One important observation is a graph with no edges is also Bipartite. Note that the Bipartite condition says all edges should be from one set to another.

We can extend the above code to handle cases when a graph is not connected. The idea is repeatedly called above method for all not yet visited vertices.

C++

```
#include <bits/stdc++.h>

using namespace std;

const int V = 4;

bool isBipartiteUtil(int G[][V], int src, int colorArr[]){
	colorArr[src] = 1;

	queue<int> q;
	q.push(src);

	while (!q.empty()) {
		int u = q.front();
		q.pop();

		if (G[u][u] == 1)
			return false;
		for (int v = 0; v < V; ++v) {
			if (G[u][v] && colorArr[v] == -1) {
				colorArr[v] = 1 - colorArr[u];
				q.push(v);
			}

			else if (G[u][v] && colorArr[v] == colorArr[u])
```

```
                                return false;
                        }
                }

                return true;
}

bool isBipartite(int G[][V]){
        int colorArr[V];
        for (int i = 0; i < V; ++i)
                colorArr[i] = -1;

        for (int i = 0; i < V; i++)
                if (colorArr[i] == -1)
                        if (isBipartiteUtil(G, i, colorArr) == false)
                                return false;

        return true;
}

int main(){
        int G[][V] = { { 0, 1, 0, 1 },
                                { 1, 0, 1, 0 },
                                { 0, 1, 0, 1 },
                                { 1, 0, 1, 0 } };

        isBipartite(G) ? cout << "Yes" : cout << "No";
        return 0;
}
```

DFS Approach

C++

```
#include <iostream>

using namespace std;
#define V 4


bool colorGraph(int G[][V],int color[],int pos, int c){

        if(color[pos] != -1 && color[pos] !=c)
                return false;

        color[pos] = c;
        bool ans = true;
        for(int i=0;i<V;i++){
                if(G[pos][i]){
                        if(color[i] == -1)
                                ans &= colorGraph(G,color,i,1-c);
```

```
                        if(color[i] !=-1 && color[i] != 1-c)
                                return false;
                }
                if (!ans)
                        return false;
        }

        return true;
}

bool isBipartite(int G[][V]){
        int color[V];
        for(int i=0;i<V;i++)
                color[i] = -1;

        //start is vertex 0;
        int pos = 0;
        // two colors 1 and 0
        return colorGraph(G,color,pos,1);

}

int main(){
        int G[][V] = {{0, 1, 0, 1},
                {1, 0, 1, 0},
                {0, 1, 0, 1},
                {1, 0, 1, 0}
        };

        isBipartite(G) ? cout<< "Yes" : cout << "No";
        return 0;
}
```

Disjoint Sets

DSU - Introduction

Introduction

A disjoint-set data structure is defined as a data structure that keeps track of a set of elements partitioned into a number of disjoint (non-overlapping) subsets.

A union-find algorithm is an algorithm that performs two useful operations on such a data structure:

- **Find:** Determine which subset a particular element is in. This can be used for determining if two elements are in the same subset.
- **Union:** Join two subsets into a single subset. Here first we have to check if the two subsets belong to the same set. If not, then we cannot perform union.

UNION and FIND Operations for Disjoint Sets

A relation over a set of elements a1, a2,…an can be divided into equivalent classes. The equivalent class of an element a is the subset of S that contains all the elements of S that are related to a.

Divide a set of elements into equivalent classes through the two operations

1. UNION

2. FIND

A set is divided into subsets. Each subset contains related elements. If we come to know that the two-element ai and aj are related, then we can do the followings:

1. Find the subset: Si-containing ai

2. Find the subset: Sj containing aj

3. If Si and Sj are two independent subsets

then we create a new subset by taking the union of Si and Sj

New subset = Si □ Sj

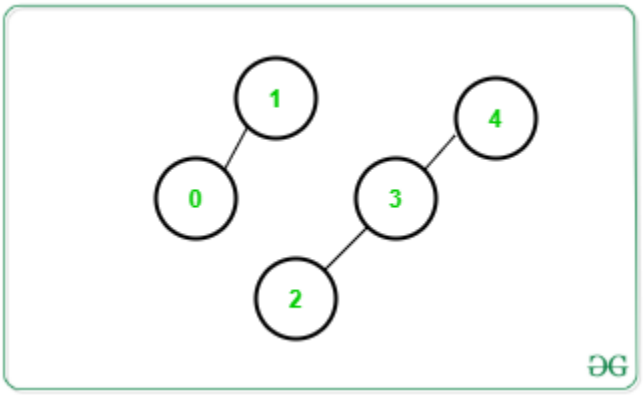
This algorithm is dynamic as, during the course of the algorithm, the sets can change via the union operation.

DSU | Finding connected Components

Given an undirected graph **G** with vertices numbered in the range **[0, N]** and an array **Edges[][]** consisting of **M** edges, the task is to find the total number of connected components in the graph using Disjoint Set Union algorithm.

Examples:

Input: *N = 4, Edges[][] = {{1, 0}, {2, 3}, {3, 4}}*
Output: *2*
Explanation: *There are only 2 connected components as shown below:*



Input: *N = 4, Edges[][] = {{1, 0}, {0, 2}, {3, 5}, {3, 4}, {6, 7}}*
Output: *3*
Explanation: *There are only 3 connected components as shown below:*

Approach: The problem can be solved using Disjoint Set Union algorithm. Follow the steps below to solve the problem:

- In DSU algorithm, there are two main functions, i.e. **connect()** and **root()** function.
- **connect():** Connects an edge.
- **root():** Recursively determine the topmost parent of a given edge.
- For each edge **{a, b},** check if **a** is connected to **b** or not. If found to be false, connect them by appending their top parents.
- After completing the above step for every edge, print the total number of the distinct top-most parents for each vertex.

Below is the implementation of the above approach:

C++

```
// C++ program for the above approach

#include <bits/stdc++.h>
using namespace std;

// Stores the parent of each vertex
int parent[1000000];
```

```
// Function to find the topmost
// parent of vertex a
int root(int a)
{
        // If current vertex is
        // the topmost vertex
        if (a == parent[a]) {
                return a;
        }

        // Otherwise, set topmost vertex of
        // its parent as its topmost vertex
        return parent[a] = root(parent[a]);
}

// Function to connect the component
// having vertex a with the component
// having vertex b
void connect(int a, int b)
{
        // Connect edges
        a = root(a);
        b = root(b);

        if (a != b) {
                parent[b] = a;
        }
}

// Function to find unique top most parents
void connectedComponents(int n)
{
        set<int> s;

        // Traverse all vertices
        for (int i = 0; i < n; i++) {

                // Insert all topmost
                // vertices obtained
                s.insert(root(parent[i]));
        }

        // Print count of connected components
        cout << s.size() << '\n';
}

// Function to print answer
void printAnswer(int N,
                                vector<vector<int> > edges)
```

```
{

        // Setting parent to itself
        for (int i = 0; i <= N; i++) {
                parent[i] = i;
        }

        // Traverse all edges
        for (int i = 0; i < edges.size(); i++) {
                connect(edges[i][0], edges[i][1]);
        }

        // Print answer
        connectedComponents(N);
}

// Driver Code
int main()
{
        // Given N
        int N = 8;

        // Given edges
        vector<vector<int> > edges = {
                { 1, 0 }, { 0, 2 }, { 5, 3 }, { 3, 4 }, { 6, 7 }
        };

        // Function call
        printAnswer(N, edges);

        return 0;
}
```

Time Complexity: *O(NLOGN+M)*
Auxiliary Space: *O(N+M)*

DSU | Detecting Cycles in an undirected graph

Problem: Given an undirected graph, the task is to check if the graph contains a cycle or not.

Examples:

Input: The following is the graph

Output: Yes
Explanation: There is a cycle of vertices {0, 1, 2}.
We already have discussed an algorithm to detect cycles in a directed graph. Here Union-Find Algorithm can be used to check whether an undirected graph contains a cycle or not. The idea is that,

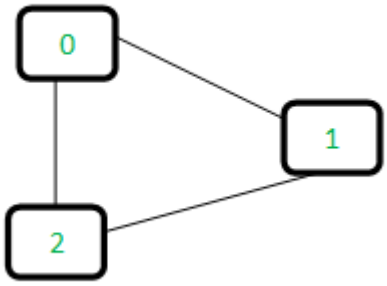
Initially create subsets containing only a single node which are the parent of itself. Now while traversing through the edges, if the two end nodes of the edge belongs to the same set then they form a cycle. Otherwise, perform union to merge the subsets together.

Note: *This method assumes that the graph doesn't contain any self-loops.*

Illustration:

Follow the below illustration for a better understanding

Let us consider the following graph:



Use an array to keep track of the subsets and which nodes belong to that subset. Let the array be ***parent[]****.*

Initially, all slots of parent array are initialized to hold the same values as the node.

parent[] = {0, 1, 2}*. Also when the value of the node and its parent are same, that is the root of that subset of nodes.*

Now process all edges one by one.
Edge 0-1:
=> Find the subsets in which vertices 0 and 1 are.

=> 0 and 1 belongs to subset 0 and 1.
=> Since they are in different subsets, take the union of them.
=> For taking the union, either make node 0 as parent of node 1 or vice-versa.
=> 1 is made parent of 0 (1 is now representative of subset {0, 1})
=> parent[] = {1, 1, 2}

Edge 1-2:

=> 1 is in subset 1 and 2 is in subset 2.
=> Since they are in different subsets, take union.
=> Make 2 as parent of 1. (2 is now representative of subset {0, 1, 2})
=> parent[] = {1, 2, 2}

Edge 0-2:

=> 0 is in subset 2 and 2 is also in subset 2.
=> Because 1 is parent of 0 and 2 is parent of 1. So 0 also belongs to subset 2
=> Hence, including this edge forms a cycle.

Therefore, the above graph contains a cycle.

Follow the below steps to implement the idea:

- Initially create a **parent[]** array to keep track of the subsets.
- Traverse through all the edges:
  - Check to which subset each of the nodes belongs by finding the parent[] array till the node and the parent are the same.
  - If the two nodes belong to the same subset then they belong to a cycle.
  - Otherwise, perform union operation on those two subsets.
- If no cycle is found, return false.

C++

```
// A union-find algorithm to detect cycle in a graph
#include <bits/stdc++.h>
using namespace std;

// a structure to represent an edge in graph
class Edge {
public:
        int src, dest;
};

// a structure to represent a graph
class Graph {
public:
        // V-> Number of vertices, E-> Number of edges
        int V, E;
```

```
        // graph is represented as an array of edges
        Edge* edge;
};

// Creates a graph with V vertices and E edges
Graph* createGraph(int V, int E){
        Graph* graph = new Graph();
        graph->V = V;
        graph->E = E;

        graph->edge = new Edge[graph->E * sizeof(Edge)];

        return graph;
}

// A utility function to find the subset of an element i
int find(int parent[], int i){
        if (parent[i] == i)
                return i;
        return find(parent, parent[i]);
}

// A utility function to do union of two subsets
void Union(int parent[], int x, int y) { parent[x] = y; }

// The main function to check whether a given graph contains
// cycle or not
int isCycle(Graph* graph){
        // Allocate memory for creating V subsets
        int* parent = new int[graph->V];

        // Initialize all subsets as single element sets
        for(int i = 0; i < graph->V; i++) {
                parent[i] = i;
        }

        // Iterate through all edges of graph, find subset of
        // both vertices of every edge, if both subsets are
        // same, then there is cycle in graph.
        for (int i = 0; i < graph->E; ++i) {
                int x = find(parent, graph->edge[i].src);
                int y = find(parent, graph->edge[i].dest);

                if (x == y)
                        return 1;

                Union(parent, x, y);
        }
```

```
        return 0;
}

int main(){
        /* Let us create the following graph
                0
                | \
                | \
                1---2 */
        int V = 3, E = 3;
        Graph* graph = createGraph(V, E);

        // add edge 0-1
        graph->edge[0].src = 0;
        graph->edge[0].dest = 1;

        // add edge 1-2
        graph->edge[1].src = 1;
        graph->edge[1].dest = 2;

        // add edge 0-2
        graph->edge[2].src = 0;
        graph->edge[2].dest = 2;

        if (isCycle(graph))
                cout << "Graph contains cycle";
        else
                cout << "Graph doesn't contain cycle";

        return 0;
}
```

The time and space complexity of the given code is as follows:

Time Complexity:

Creating the graph takes O(V + E) time, where V is the number of vertices and E is the number of edges.
Finding the subset of an element takes O(log V) time in the worst case, where V is the number of vertices. The worst case occurs when the tree is skewed, and the depth of the tree is V.
The union of two subsets takes O(1) time.
The loop iterating through all edges takes O(E) time.
Therefore, the overall time complexity of the algorithm is O(E log V). However, in practice,

it can be much faster than O(E log V) because the worst-case scenario of finding the subset of an element does not happen often.

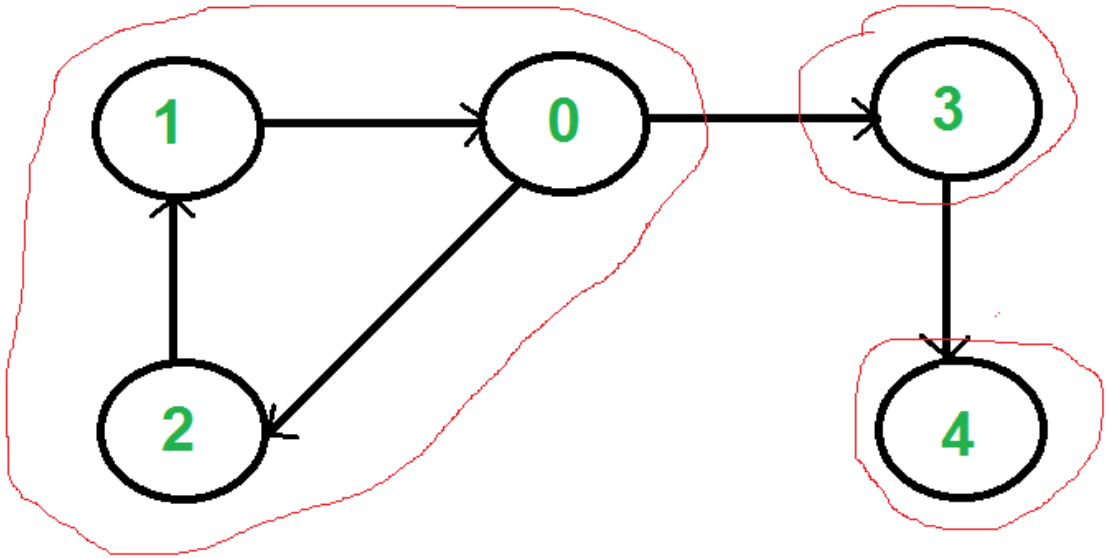
Space Complexity:

The space complexity of creating the graph is O(E).
The space complexity of creating the parent array is O(V).
The space complexity of the algorithm is O(max(V,E)) because at any point in time, there can be at most max(V,E) subsets.
Therefore, the overall space complexity of the algorithm is O(max(V,E)).

Strongly connected components

Strongly Connected Components

A directed graph is strongly connected if there is a path between all pairs of vertices. A strongly connected component (**SCC**) of a directed graph is a maximal strongly connected subgraph. For example, there are 3 SCCs in the following graph.



Properties

Strongly Connected Components (SCCs) in a graph are a set of vertices where each vertex can be reached from every other vertex in the same component. Here are some properties of SCCs:

1. A graph can have multiple SCCs.
2. Every vertex in an SCC has a path to every other vertex in the same SCC.
3. An SCC cannot be a subset of any other SCC in the same graph.
4. The vertices in an SCC are strongly connected, but there may not be any edges between different SCCs.
5. If a graph contains only one SCC, then it is strongly connected.
6. SCCs can be found using algorithms like Tarjan's algorithm or Kosaraju's algorithm.

These properties make SCCs a useful concept in many applications such as finding strongly connected components in social networks, identifying cycles in circuits, detecting deadlocks in computer systems, and more.

Strongly Connected Components - Kosaraju's Algorithm

We can find all strongly connected components in O(V+E) time using Kosaraju's algorithm. Following is detailed Kosaraju's algorithm.

1. *Create an empty stack 'S' and do DFS traversal of a graph. In DFS traversal, after calling recursive DFS for adjacent vertices of a vertex, push the vertex to stack. In the above graph, if we start DFS from vertex 0, we get vertices in stack as 1, 2, 4, 3, 0.*
2. *Reverse directions of all arcs to obtain the transpose graph.*
3. *One by one pop a vertex from S while S is not empty. Let the popped vertex be 'v'. Take v as source and do DFS (call DFSUtil(v)). The DFS starting from v prints strongly connected component of v. In the above example, we process vertices in order 0, 3, 4, 2, 1 (One by one popped from stack).*

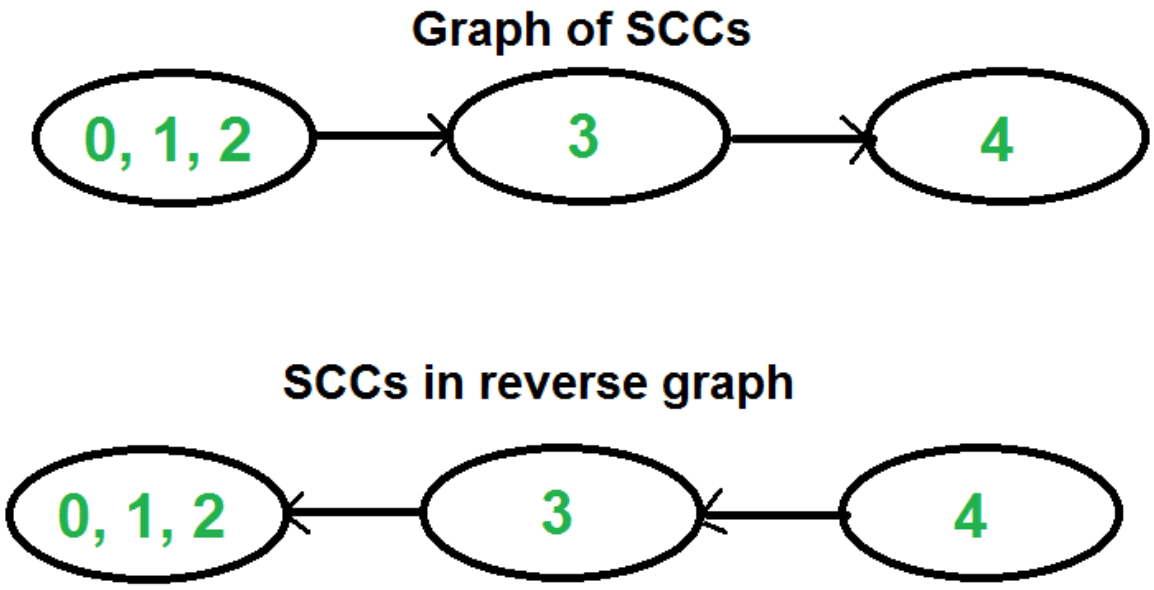
How does this work?

The above algorithm is DFS based. It does DFS two times. The DFS of a graph produces a single tree if all vertices are reachable from the DFS starting point. Otherwise, DFS produces a forest. So DFS of a graph with only one SCC always produces a tree. The important point to note is DFS may produce a tree or a forest when there are more than one SCCs depending upon the chosen starting point. For example, in the above diagram, if we start DFS from vertices 0 or 1, or 2, we get a tree as output. And if we start from 3 or 4, we get a forest. To find and print all SCCs, we would want to start DFS from vertex 4 (which

is a sink vertex), then move to 3 which is a sink in the remaining set (set excluding 4), and finally any of the remaining vertices (0, 1, 2). So how do we find this sequence of picking vertices as starting points of DFS? Unfortunately, there is no direct way for getting this sequence.

However, if we do a DFS of the graph and store vertices according to their finish times, we make sure that the finish time of a vertex that connects to other SCCs (other than its own SCC), will always be greater than the finish time of vertices in the other SCC (See this for proof). For example, in the DFS of the above example graph, a finish time of 0 is always greater than 3 and 4 (irrespective of the sequence of vertices considered for DFS). And a finish time of 3 is always greater than 4. DFS doesn't guarantee other vertices, for example, finish times of 1 and 2 may be smaller or greater than 3 and 4 depending upon the sequence of vertices considered for DFS. So to use this property, we do a DFS traversal of the complete graph and push every finished vertex to a stack. In stack, 3 always appears after 4, and 0 appears after both 3 and 4.

In the next step, we reverse the graph. Consider the graph of SCCs. In the reversed graph, the edges that connect two components are reversed. So the SCC {0, 1, 2} becomes sink and the SCC {4} becomes source. As discussed above, in stack, we always have 0 before 3 and 4. So if we do a DFS of the reversed graph using a sequence of vertices in stack, we process vertices from sink to the source (in the reversed graph). That is what we wanted to achieve and that is all needed to print SCCs one by one.



Following is the C++ implementation of Kosaraju's algorithm:

C++

```
#include <iostream>
#include <list>
#include <stack>
using namespace std;

class Graph
{
	int V;
	list<int> *adj;

	void fillOrder(int v, bool visited[], stack<int> &Stack);

	void DFSUtil(int v, bool visited[]);
public:
	Graph(int V);
	void addEdge(int v, int w);

	void printSCCs();

	Graph getTranspose();
};

Graph::Graph(int V){
	this->V = V;
	adj = new list<int>[V];
}

void Graph::DFSUtil(int v, bool visited[]){
	visited[v] = true;
	cout << v << " ";

	list<int>::iterator i;
	for (i = adj[v].begin(); i != adj[v].end(); ++i)
		if (!visited[*i])
			DFSUtil(*i, visited);
}

Graph Graph::getTranspose(){
	Graph g(V);
	for (int v = 0; v < V; v++){
		list<int>::iterator i;
		for(i = adj[v].begin(); i != adj[v].end(); ++i){
			g.adj[*i].push_back(v);
		}
	}
	return g;
```

```
}

void Graph::addEdge(int v, int w){
	adj[v].push_back(w); // Add w to v's list.
}

void Graph::fillOrder(int v, bool visited[], stack<int> &Stack){
	visited[v] = true;

	list<int>::iterator i;
	for(i = adj[v].begin(); i != adj[v].end(); ++i)
		if(!visited[*i])
			fillOrder(*i, visited, Stack);

	Stack.push(v);
}

void Graph::printSCCs(){
	stack<int> Stack;

	bool *visited = new bool[V];
	for(int i = 0; i < V; i++)
		visited[i] = false;

	for(int i = 0; i < V; i++)
		if(visited[i] == false)
			fillOrder(i, visited, Stack);

	Graph gr = getTranspose();

	for(int i = 0; i < V; i++)
		visited[i] = false;

	while (Stack.empty() == false){
		int v = Stack.top();
		Stack.pop();

		if (visited[v] == false){
			gr.DFSUtil(v, visited);
			cout << endl;
		}
	}
}

int main(){
	Graph g(5);
	g.addEdge(1, 0);
	g.addEdge(0, 2);
	g.addEdge(2, 1);
```

```
        g.addEdge(0, 3);
        g.addEdge(3, 4);

        cout << "Following are strongly connected components in "
                        "given graph \n";
        g.printSCCs();

        return 0;
}
```

Time Complexity: The above algorithm calls DFS, finds the reverse of the graph and again calls DFS. DFS takes O(V+E) for a graph represented using an adjacency list. Reversing a graph also takes O(V+E) time. For reversing the graph, we simply traverse all adjacency lists. The above algorithm is asymptotically the best algorithm, but there are other algorithms like Tarjan's algorithm and path-based which have the same time complexity but find SCCs using a single DFS.

Space Complexity: O(V) as we are using a stack to store the vertices.

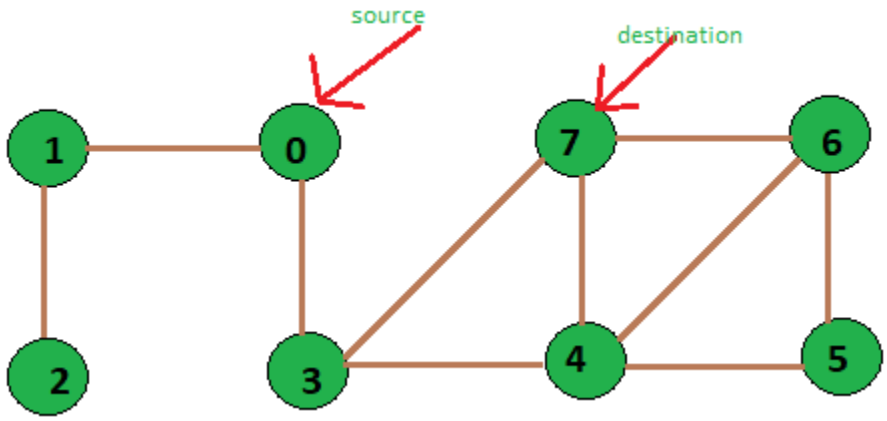
Applications: SCC algorithms can be used as a first step in many graph algorithms that work only on strongly connected graphs. In social networks, a group of people is generally strongly connected (For example, students of a class or any other commonplace). Many people in these groups generally like some common pages or play common games. The SCC algorithms can be used to find such groups and suggest the commonly liked pages or games to the people in the group who have not yet liked commonly liked a page or played a game.

Shortest Path Algorithms

Shortest Path in an undirected Graph

Introduction

Given an unweighted graph, a source, and a destination, we need to find the shortest path from source to destination in the graph in the most optimal way.




```
Input: source vertex = 0 and destination vertex is = 7.
Output: Shortest path length is:2
        Path is::
        0 3 7

Input: source vertex is = 2 and destination vertex is = 6.
Output: Shortest path length is:5
        Path is::
        2 1 0 3 4 6
```

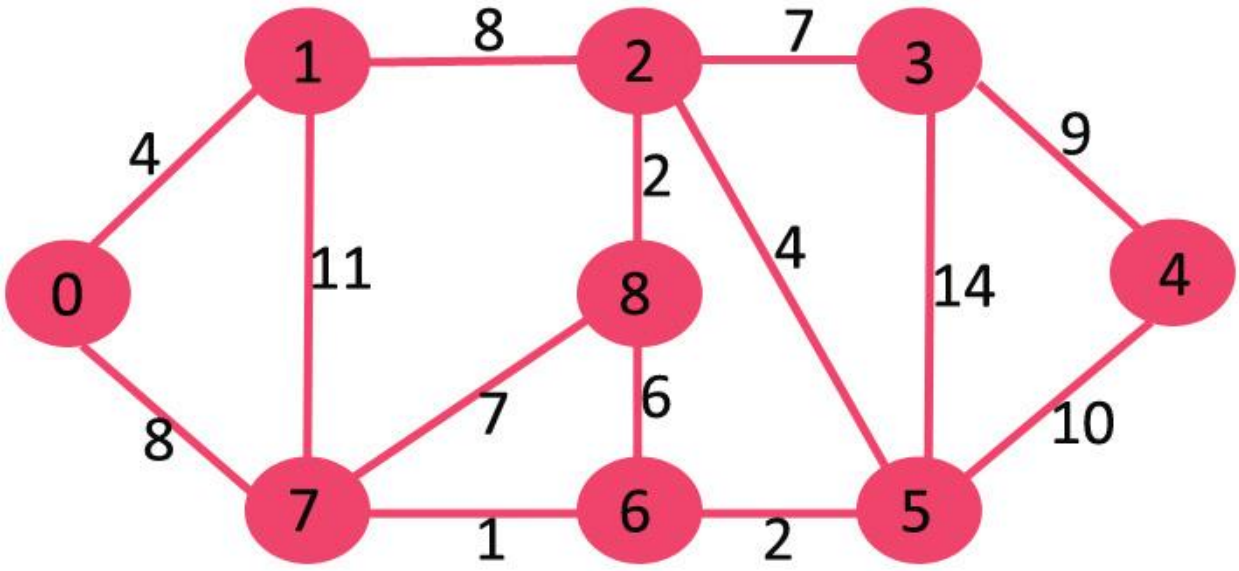
Dijkstra's Algorithm

Dijkstra's Algorithm

Given a graph and a source vertex in the graph, find the **shortest paths** from the source to all vertices in the given graph.

Examples:

Input: *src = 0, the graph is shown below.*



Output: *0 4 12 19 21 11 9 8 14*
Explanation: *The distance from 0 to 1 = 4.*
The minimum distance from 0 to 2 = 12. 0->1->2
The minimum distance from 0 to 3 = 19. 0->1->2->3
The minimum distance from 0 to 4 = 21. 0->7->6->5->4
The minimum distance from 0 to 5 = 11. 0->7->6->5
The minimum distance from 0 to 6 = 9. 0->7->6
The minimum distance from 0 to 7 = 8. 0->7
The minimum distance from 0 to 8 = 14. 0->1->2->8
There are two ways we can apply Dijkstra's Algorithm:

- **Using Prim's Algorithm**
- **Using Heap**

Dijkstra's algorithm is very similar to [Prim's algorithm for minimum spanning tree](#).

Like Prim's MST, generate a SPT (shortest path tree) with a given source as a root. Maintain two sets, one set contains vertices included in the shortest-path tree, other set includes vertices not yet included in the shortest-path tree. At every step of the algorithm, find a vertex that is in the other set (set not yet included) and has a minimum distance from the source.

Follow the steps below to solve the problem:

- Create a set **sptSet** (shortest path tree set) that keeps track of vertices included in the shortest path tree, i.e., whose minimum distance from the source is calculated and finalized. Initially, this set is empty.

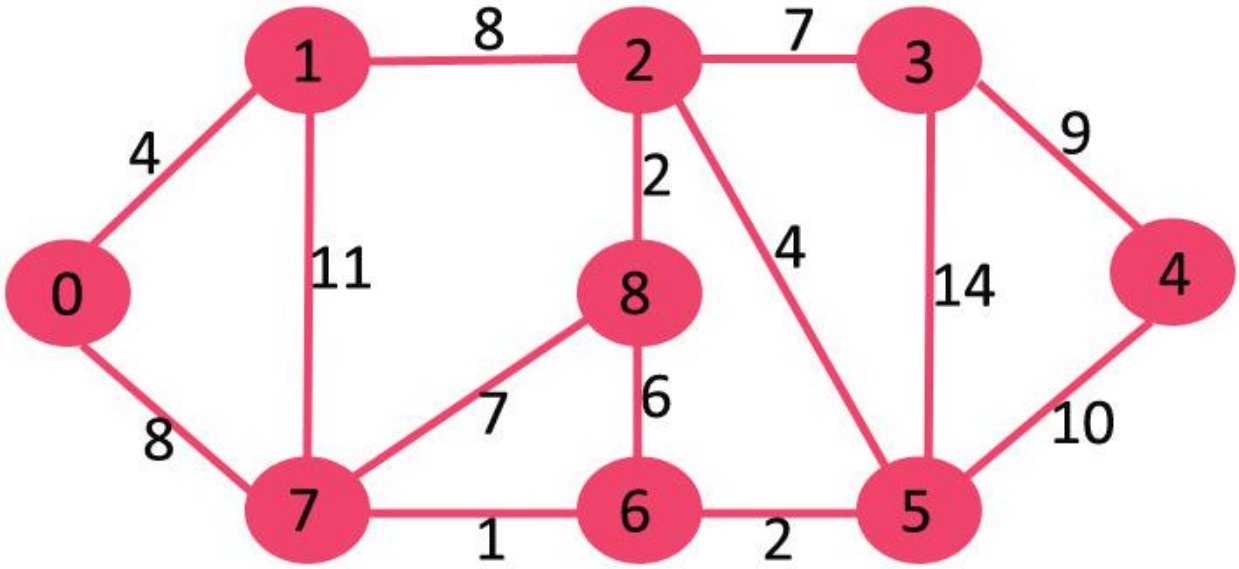
- Assign a distance value to all vertices in the input graph. Initialize all distance values as **INFINITE**. Assign the distance value as 0 for the source vertex so that it is picked first.
- While **sptSet** doesn't include all vertices
    - Pick a vertex **u** that is not there in **sptSet** and has a minimum distance value.
    - Include u to **sptSet**.
    - Then update the distance value of all adjacent vertices of u.
        - To update the distance values, iterate through all adjacent vertices.
        - For every adjacent vertex v, if the sum of the distance value of u (from source) and weight of edge u-v, is less than the distance value of v, then update the distance value of v.

Note: We use a boolean array sptSet[] to represent the set of vertices included in SPT. If a value sptSet[v] is true, then vertex v is included in SPT, otherwise not. Array dist[] is used to store the shortest distance values of all vertices.

Below is the illustration of the above approach:

Illustration:

To understand the Dijkstra's Algorithm lets take a graph and find the shortest path from source to all nodes.
Consider below graph and ***src = 0***

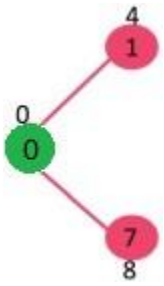


Step 1:

- *The set* ***sptSet*** *is initially empty and distances assigned to vertices are {0, INF, INF, INF, INF, INF, INF, INF} where* ***INF*** *indicates infinite.*

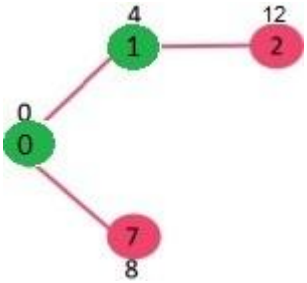
- *Now pick the vertex with a minimum distance value. The vertex 0 is picked, include it in sptSet. So sptSet becomes {0}. After including 0 to sptSet, update distance values of its adjacent vertices.*
- *Adjacent vertices of 0 are 1 and 7. The distance values of 1 and 7 are updated as 4 and 8.*

The following subgraph shows vertices and their distance values, only the vertices with finite distance values are shown. The vertices included in SPT are shown in ***green*** *colour.*



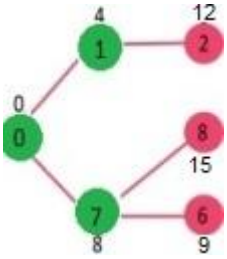
Step 2:

- *Pick the vertex with minimum distance value and not already included in SPT (not in sptSET). The vertex 1 is picked and added to sptSet.*
- *So sptSet now becomes {0, 1}. Update the distance values of adjacent vertices of 1.*
- *The distance value of vertex 2 becomes* ***12****.*



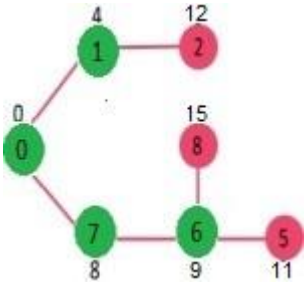
Step 3:

- *Pick the vertex with minimum distance value and not already included in SPT (not in sptSET). Vertex 7 is picked. So sptSet now becomes* ***{0, 1, 7}.***
- *Update the distance values of adjacent vertices of 7. The distance value of vertex 6 and 8 becomes finite (****15 and 9*** *respectively).*



Step 4:

- *Pick the vertex with minimum distance value and not already included in SPT (not in sptSET). Vertex 6 is picked. So sptSet now becomes* ***{0, 1, 7, 6}****.*
- *Update the distance values of adjacent vertices of 6. The distance value of vertex 5 and 8 are updated.*



We repeat the above steps until sptSet ***includes*** *all vertices of the given graph. Finally, we get the following Shortest Path Tree (SPT).*

C++

```
#include <iostream>
using namespace std;
#include <limits.h>

#define V 9

int minDistance(int dist[], bool sptSet[]){

        int min = INT_MAX, min_index;

        for (int v = 0; v < V; v++)
                if (sptSet[v] == false && dist[v] <= min)
                        min = dist[v], min_index = v;

        return min_index;
}

void printSolution(int dist[]){
```

```
        cout << "Vertex \t Distance from Source" << endl;
        for (int i = 0; i < V; i++)
                cout << i << " \t\t\t\t" << dist[i] << endl;
}

void dijkstra(int graph[V][V], int src){
        int dist[V];

        bool sptSet[V];
        for (int i = 0; i < V; i++)
                dist[i] = INT_MAX, sptSet[i] = false;

        dist[src] = 0;

        for (int count = 0; count < V - 1; count++) {
                int u = minDistance(dist, sptSet);

                sptSet[u] = true;

                for (int v = 0; v < V; v++)
                        if (!sptSet[v] && graph[u][v]
                                && dist[u] != INT_MAX
                                && dist[u] + graph[u][v] < dist[v])
                                dist[v] = dist[u] + graph[u][v];
        }

        printSolution(dist);
}

int main(){

        int graph[V][V] = { { 0, 4, 0, 0, 0, 0, 0, 8, 0 },
                                                { 4, 0, 8, 0, 0, 0, 0, 11, 0
},
                                                { 0, 8, 0, 7, 0, 4, 0, 0, 2 },
                                                { 0, 0, 7, 0, 9, 14, 0, 0, 0
},
                                                { 0, 0, 0, 9, 0, 10, 0, 0, 0
},
                                                { 0, 0, 4, 14, 10, 0, 2, 0, 0
},
                                                { 0, 0, 0, 0, 0, 2, 0, 1, 6 },
                                                { 8, 11, 0, 0, 0, 0, 1, 0, 7
},
                                                { 0, 0, 2, 0, 0, 0, 6, 7, 0 }
};
        dijkstra(graph, 0);

        return 0;
```

```
}
```

Time Complexity: $O(V^2)$
Auxiliary Space: $O(V)$

Notes:

- The code calculates the shortest distance but doesn't calculate the path information. Create a parent array, update the parent array when distance is updated (like prim's implementation), and use it to show the shortest path from the source to different vertices.
- The code is for undirected graphs, the same Dijkstra function can be used for directed graphs also.
- The code finds the shortest distances from the source to all vertices. If we are interested only in the shortest distance from the source to a single target, break them for a loop when the picked minimum distance vertex is equal to the target.
- The time Complexity of the implementation is **$O(V^2)$**. If the input graph is represented using an adjacency list, it can be reduced to O(E * log V) with the help of a binary heap.
- Dijkstra's algorithm doesn't work for graphs with negative weight cycles. It may give correct results for a graph with negative edges but you must allow a vertex can be visited multiple times and that version will lose its fast time complexity. For graphs with negative weight edges and cycles, the Bellman-Ford algorithm can be used.

Dijkstra's shortest path algorithm using Heap in O(E logV):

For Dijkstra's algorithm, it is always recommended to use **Heap** (or **priority queue**) as the required operations (extract minimum and decrease key) match with the specialty of the heap (or priority queue). However, the problem is, that priority_queue doesn't support the decrease key. To resolve this problem, do not update a key, but insert one more copy of it. So we allow multiple instances of the same vertex in the priority queue. This approach doesn't require decreasing key operations and has below-important properties.

- Whenever the distance of a vertex is reduced, we add one more instance of a vertex in priority_queue. Even if there are multiple instances, we only consider the instance with minimum distance and ignore other instances.
- The time complexity remains **O(E * LogV)** as there will be at most O(E) vertices in the priority queue and O(logE) is the same as O(logV)

C++

```
#include <bits/stdc++.h>
using namespace std;

#define INF 0x3f3f3f3f
typedef pair<int, int> iPair;

class Graph {
        int V;
        list<pair<int, int> >* adj;
```

```
public:
        Graph(int V);

        void addEdge(int u, int v, int w);

        void shortestPath(int s);
};

Graph::Graph(int V){
        this->V = V;
        adj = new list<iPair>[V];
}

void Graph::addEdge(int u, int v, int w){
        adj[u].push_back(make_pair(v, w));
        adj[v].push_back(make_pair(u, w));
}

void Graph::shortestPath(int src){
        priority_queue<iPair, vector<iPair>, greater<iPair>> pq;

        vector<int> dist(V, INF);

        pq.push(make_pair(0, src));
        dist[src] = 0;

        while (!pq.empty()) {
                int u = pq.top().second;
                pq.pop();
                list<pair<int, int> >::iterator i;
                for (i = adj[u].begin(); i != adj[u].end(); ++i) {
                        int v = (*i).first;
                        int weight = (*i).second;

                        if (dist[v] > dist[u] + weight) {
                                dist[v] = dist[u] + weight;
                                pq.push(make_pair(dist[v], v));
                        }
                }
        }

        printf("Vertex Distance from Source\n");
        for (int i = 0; i < V; ++i)
                printf("%d \t\t %d\n", i, dist[i]);
}

int main(){
        int V = 9;
```

```
        Graph g(V);

        g.addEdge(0, 1, 4);
        g.addEdge(0, 7, 8);
        g.addEdge(1, 2, 8);
        g.addEdge(1, 7, 11);
        g.addEdge(2, 3, 7);
        g.addEdge(2, 8, 2);
        g.addEdge(2, 5, 4);
        g.addEdge(3, 4, 9);
        g.addEdge(3, 5, 14);
        g.addEdge(4, 5, 10);
        g.addEdge(5, 6, 2);
        g.addEdge(6, 7, 1);
        g.addEdge(6, 8, 6);
        g.addEdge(7, 8, 7);

        g.shortestPath(0);

        return 0;
}
```

Time Complexity: O(E * logV), Where E is the number of edges and V is the number of vertices.
Auxiliary Space: O(V)

Bellman Ford Algorithm

Bellman-Ford Algoritm
Dijkstra's algorithm is a Greedy algorithm and the time complexity is O((V+E)LogV) (with the use of the Fibonacci heap). Dijkstra doesn't work for Graphs with negative weights, Bellman-Ford works for such graphs. Bellman-Ford is also simpler than Dijkstra and suites well for distributed systems. But the time complexity of Bellman-Ford is O(V * E), which is more than Dijkstra.

Steps for finding the shortest distance to all vertices from the source using the Bellman-Ford algorithm:

- *This step initializes distances from the source to all vertices as infinite and distance to the source itself as 0. Create an array dist[] of size |V| with all values as infinite except dist[src] where src is source vertex.*

- *This step calculates shortest distances. Do following |V|-1 times where |V| is the number of vertices in given graph. Do following for each edge u-v*
    - *If dist[v] > dist[u] + weight of edge uv, then update dist[v] to*
    - *dist[v] = dist[u] + weight of edge uv*
- *This step reports if there is a negative weight cycle in the graph. Again traverse every edge and do following for each edge u-v*
  ……If dist[v] > dist[u] + weight of edge uv, then "Graph contains negative weight cycle"
  The idea of step 3 is, step 2 guarantees the shortest distances if the graph doesn't contain a negative weight cycle. If we iterate through all edges one more time and get a shorter path for any vertex, then there is a negative weight cycle

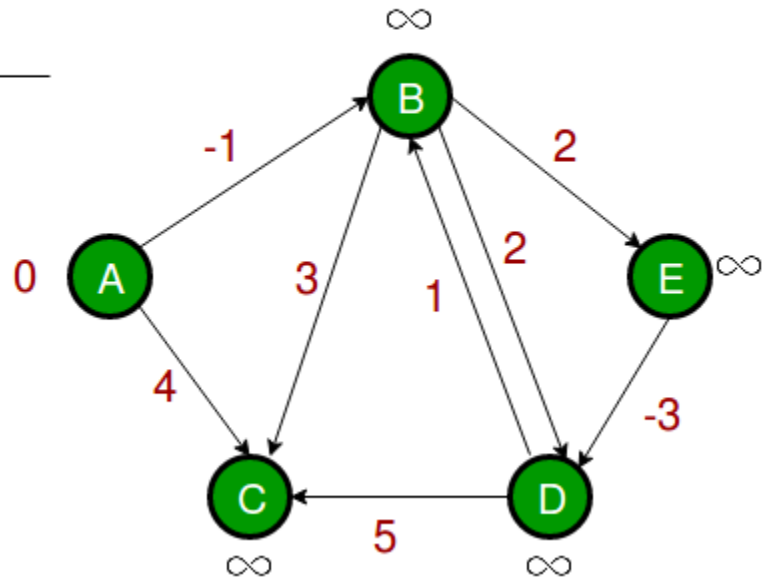
How does this work?

Like other Dynamic Programming Problems, the algorithm calculates the shortest paths in a bottom-up manner. It first calculates the shortest distances which have at most one edge in the path. Then, it calculates the shortest paths with at-most 2 edges, and so on. After the i-th iteration of the outer loop, the shortest paths with at most i edges are calculated. There can be maximum |V| – 1 edges in any simple path, that is why the outer loop runs |v| – 1 times. The idea is, assuming that there is no negative weight cycle if we have calculated shortest paths with at most i edges, then an iteration over all edges guarantees to give the shortest path with at-most (i+1) edges

Below is the illustration of the above algorithm:

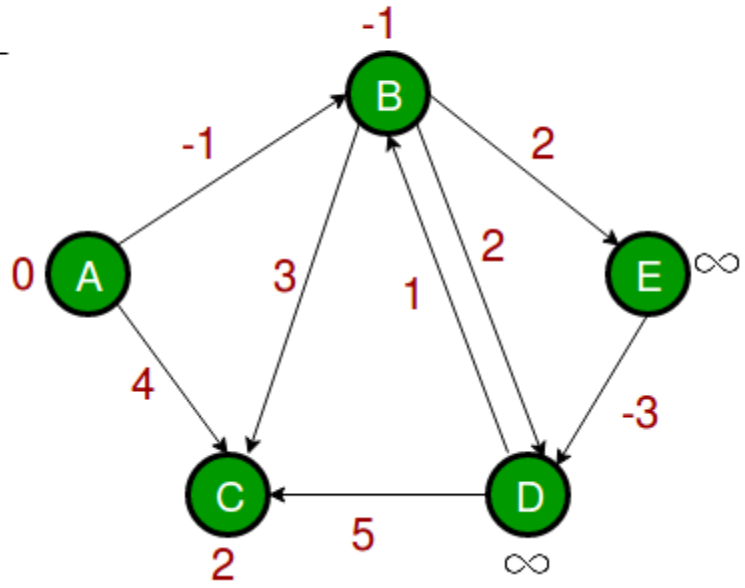
Step 1: *Let the given source vertex be 0. Initialize all distances as infinite, except the distance to the source itself. Total number of vertices in the graph is 5, so all edges must be processed 4 times.*

A	B	C	D	E
0	∞	∞	∞	∞



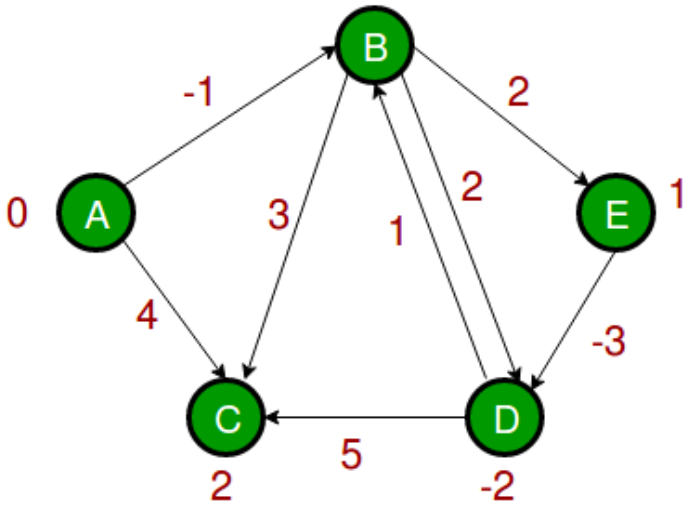
Step 2: *Let all edges are processed in the following order: (B, E), (D, B), (B, D), (A, B), (A, C), (D, C), (B, C), (E, D). We get the following distances when all edges are processed the first time. The first row shows initial distances. The second row shows distances when edges (B, E), (D, B), (B, D) and (A, B) are processed. The third row shows distances when (A, C) is processed. The fourth row shows when (D, C), (B, C) and (E, D) are processed.*

A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞



Step 3: *The first iteration guarantees to give all shortest paths which are at most 1 edge long. We get the following distances when all edges are processed second time (The last row shows final values).*

A	B	C	D	E
0	∞	∞	∞	∞
0	-1	∞	∞	∞
0	-1	4	∞	∞
0	-1	2	∞	∞
0	-1	2	∞	1
0	-1	2	1	1
0	-1	2	-2	1



Step 4: *The second iteration guarantees to give all shortest paths which are at most 2 edges long. The algorithm processes all edges 2 more times. The distances are minimized after the second iteration, so third and fourth iterations don't update the distances.*

C++

```
#include <bits/stdc++.h>
using namespace std;

struct Edge {
	int src, dest, weight;
};

struct Graph {
	int V, E;

	struct Edge* edge;
};

struct Graph* createGraph(int V, int E){
	struct Graph* graph = new Graph;
	graph->V = V;
	graph->E = E;
	graph->edge = new Edge[E];
	return graph;
}

void printArr(int dist[], int n){
	printf("Vertex Distance from Source\n");
	for (int i = 0; i < n; ++i)
		printf("%d \t\t %d\n", i, dist[i]);
}

void BellmanFord(struct Graph* graph, int src){
	int V = graph->V;
	int E = graph->E;
	int dist[V];
	for (int i = 0; i < V; i++)
		dist[i] = INT_MAX;
	dist[src] = 0;

	for (int i = 1; i <= V - 1; i++) {
		for (int j = 0; j < E; j++) {
			int u = graph->edge[j].src;
			int v = graph->edge[j].dest;
			int weight = graph->edge[j].weight;
			if (dist[u] != INT_MAX
				&& dist[u] + weight < dist[v])
```

```
                                dist[v] = dist[u] + weight;
                }
        }

        for (int i = 0; i < E; i++) {
                int u = graph->edge[i].src;
                int v = graph->edge[i].dest;
                int weight = graph->edge[i].weight;
                if (dist[u] != INT_MAX
                        && dist[u] + weight < dist[v]) {
                        printf("Graph contains negative weight cycle");
                        return;
                }
        }

        printArr(dist, V);

        return;
}

int main(){
        int V = 5;
        int E = 8;
        struct Graph* graph = createGraph(V, E);

        graph->edge[0].src = 0;
        graph->edge[0].dest = 1;
        graph->edge[0].weight = -1;

        graph->edge[1].src = 0;
        graph->edge[1].dest = 2;
        graph->edge[1].weight = 4;

        graph->edge[2].src = 1;
        graph->edge[2].dest = 2;
        graph->edge[2].weight = 3;

        graph->edge[3].src = 1;
        graph->edge[3].dest = 3;
        graph->edge[3].weight = 2;

        graph->edge[4].src = 1;
        graph->edge[4].dest = 4;
        graph->edge[4].weight = 2;

        graph->edge[5].src = 3;
        graph->edge[5].dest = 2;
        graph->edge[5].weight = 5;
```

```
        graph->edge[6].src = 3;
        graph->edge[6].dest = 1;
        graph->edge[6].weight = 1;

        graph->edge[7].src = 4;
        graph->edge[7].dest = 3;
        graph->edge[7].weight = -3;

        BellmanFord(graph, 0);

        return 0;
}
```

Time Complexity: O(V * E), where V is the number of vertices in the graph and E is the number of edges in the graph
Auxiliary Space: O(E)

Notes:

- Negative weights are found in various applications of graphs. For example, instead of paying the cost for a path, we may get some advantages if we follow the path.
- Bellman-Ford works better (better than Dijkstra's) for distributed systems. Unlike Dijkstra's where we need to find the minimum value of all vertices, in Bellman-Ford, edges are considered one by one.
- Bellman-Ford does not work with an undirected graph with negative edges as it will be declared as a negative cycle.

Floyd Warshall Algorithm

Floyd Warshall Algorithm

The **Floyd Warshall Algorithm** is for solving all pairs of shortest-path problems. The problem is to find the shortest distances between every pair of vertices in a given edge-weighted directed Graph.

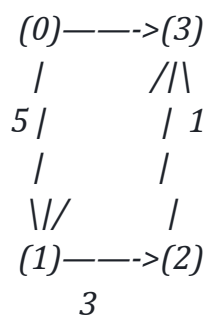
It is an algorithm for finding the shortest path between all the pairs of vertices in a weighted graph. This algorithm follows the dynamic programming approach to find the shortest path.

A C-function for a N x N graph is given below. The function stores the all-pair shortest path in the matrix cost [N][N]. The cost matrix of the given graph is available in cost Mat [N][N].

Example:

Input: *graph[][] = { {0, 5, INF, 10},*
{INF, 0, 3, INF},
{INF, INF, 0, 1},
{INF, INF, INF, 0} }

which represents the following graph



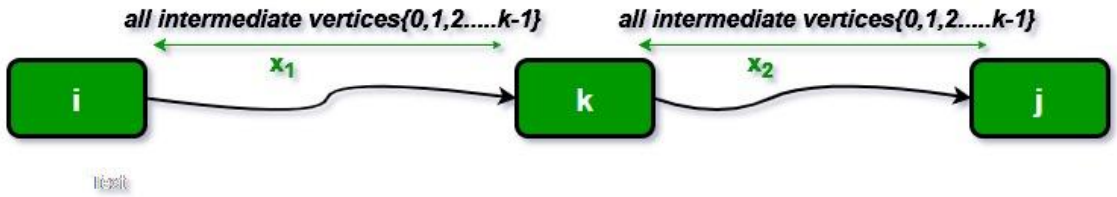
Output: *Shortest distance matrix*

0	5	8	9
INF	0	3	4
INF	INF	0	1
INF	INF	INF	0

Floyd Warshall Algorithm:

- ***Initialize the solution matrix same as the input graph matrix as a first step.***
- ***Then update the solution matrix by considering all vertices as an intermediate vertex.***
- ***The idea is to one by one pick all vertices and updates all shortest paths which include the picked vertex as an intermediate vertex in the shortest path.***
- ***When we pick vertex number k as an intermediate vertex, we already have considered vertices {0, 1, 2, .. k-1} as intermediate vertices.***
- ***For every pair (i, j) of the source and destination vertices respectively, there are two possible cases.***
    - ***k is not an intermediate vertex in shortest path from i to j. We keep the value of dist[i][j] as it is.***
    - ***k is an intermediate vertex in shortest path from i to j. We update the value of dist[i][j] as dist[i][k] + dist[k][j] if dist[i][j] > dist[i][k] + dist[k][j]***

The following figure shows the above optimal substructure property in the all-pairs shortest path problem:



C++

```
#include <bits/stdc++.h>
using namespace std;

#define V 4

#define INF 99999

void printSolution(int dist[][V]);

void floydWarshall(int dist[][V]){

	int i, j, k;

	for (k = 0; k < V; k++) {
		for (i = 0; i < V; i++) {
			for (j = 0; j < V; j++) {
				if (dist[i][j] > (dist[i][k] + dist[k][j])
					&& (dist[k][j] != INF
						&& dist[i][k] != INF))
				dist[i][j] = dist[i][k] + dist[k][j];
			}
		}
	}

	printSolution(dist);
}

void printSolution(int dist[][V]){
	cout << "The following matrix shows the shortest "
			"distances"
			" between every pair of vertices \n";
	for (int i = 0; i < V; i++) {
		for (int j = 0; j < V; j++) {
			if (dist[i][j] == INF)
				cout << "INF"
					<< " ";
			else
```

```
                                cout << dist[i][j] << " ";
                }
                cout << endl;
        }
}

int main(){
        /* Let us create the following weighted graph
                        10
        (0)------->(3)
                |        /|\
        5 |      |
                |        | 1
        \|/      |
        (1)------->(2)
                        3        */
        int graph[V][V] = { { 0, 5, INF, 10 },
                                                { INF, 0, 3, INF },
                                                { INF, INF, 0, 1 },
                                                { INF, INF, INF, 0 } };

        floydWarshall(graph);
        return 0;
}
```

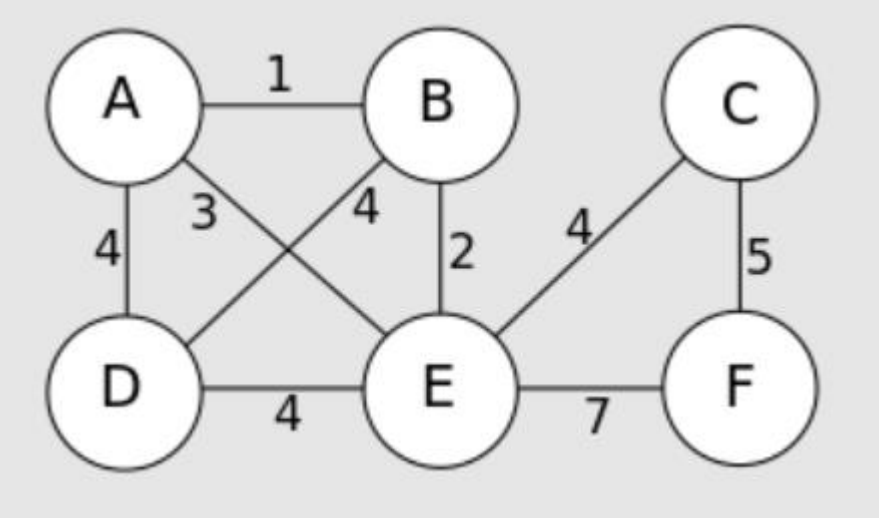
Time Complexity: $O(V^3)$
Auxiliary Space: $O(V^2)$

Spanning Tree

Minimum Spanning Tree - Introduction

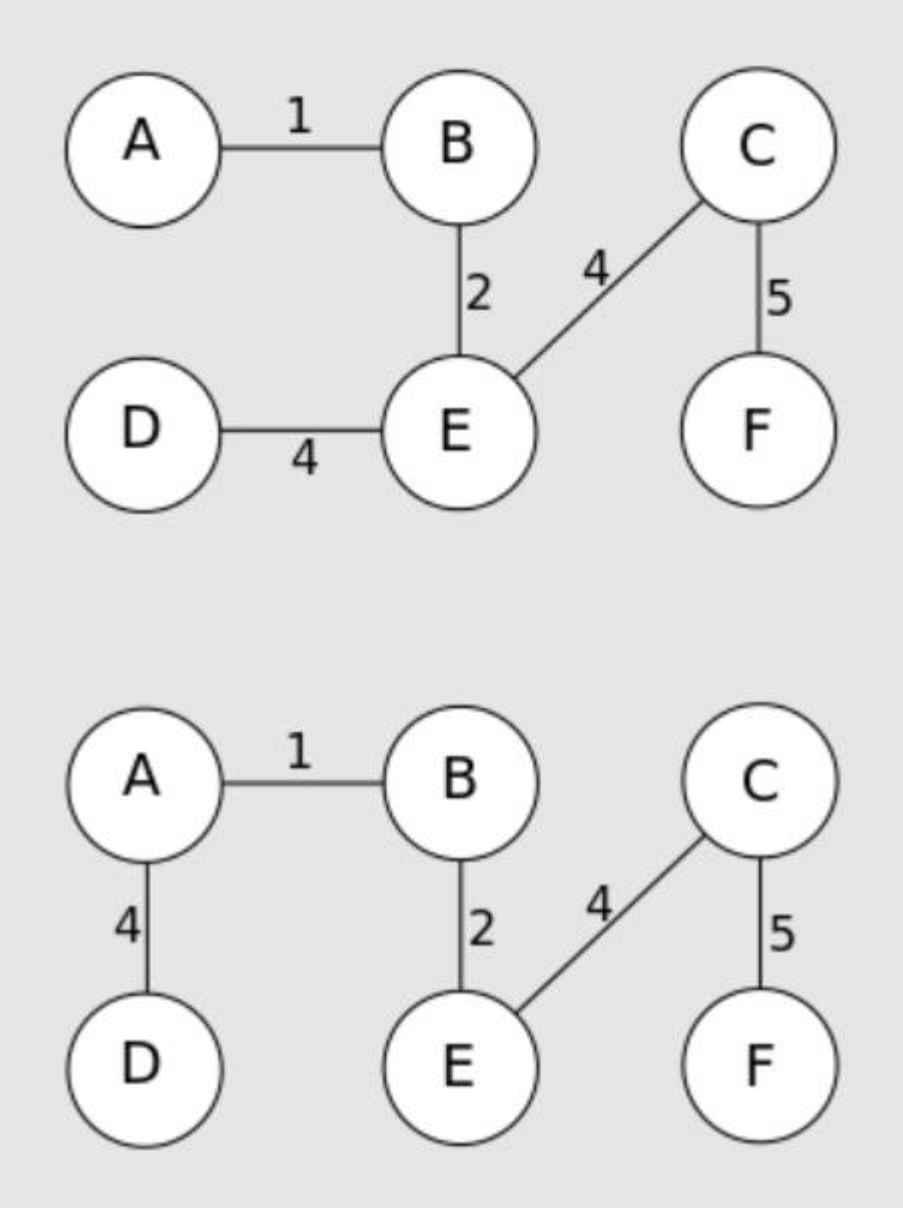
What is a Minimum Spanning Tree?
A minimum spanning tree (MST) or minimum weight spanning tree for a weighted, connected, undirected graph is a spanning tree with a weight less than or equal to the weight of every other spanning tree.

Suppose this is the graph given to you:



Weighted Directed Graph

These two can be the two possible MSTs for this:



MST formation for the given weighted directed graph

We have two different approaches to finding the MST of a given graph:

- Kruskal's Algorithm
- Prim's Algorithm

Minimum Spanning Tree - Prim's Algorithm

Prim's Algorithm

Like Kruskal's algorithm, Prim's algorithm is also a **Greedy algorithm**. This algorithm always starts with a single node and moves through several adjacent nodes, in order to explore all of the connected edges along the way.

The algorithm starts with an empty spanning tree. The idea is to maintain two sets of vertices. The first set contains the vertices already included in the MST, and the other set contains the vertices not yet included. At every step, it considers all the edges that connect the two sets and picks the minimum weight edge from these edges. After picking the edge, it moves the other endpoint of the edge to the set containing MST.

A group of edges that connects two sets of vertices in a graph is called a cut in graph theory. So, at every step of Prim's algorithm, find a cut, pick the minimum weight edge from the cut, and include this vertex in MST Set (the set that contains already included vertices).

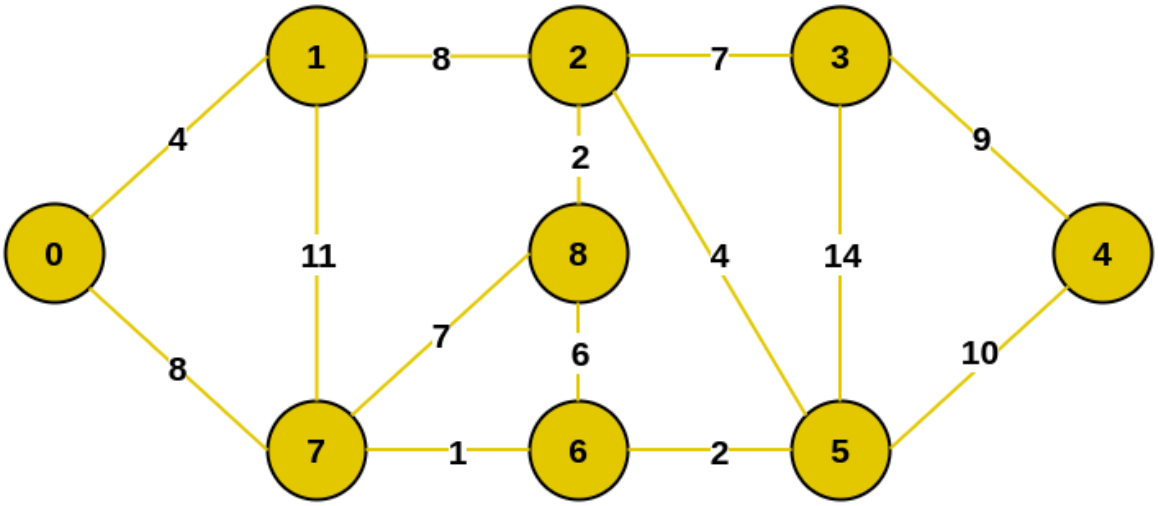
How does Prim's Algorithm Work?

The working of Prim's algorithm can be described by using the following steps:

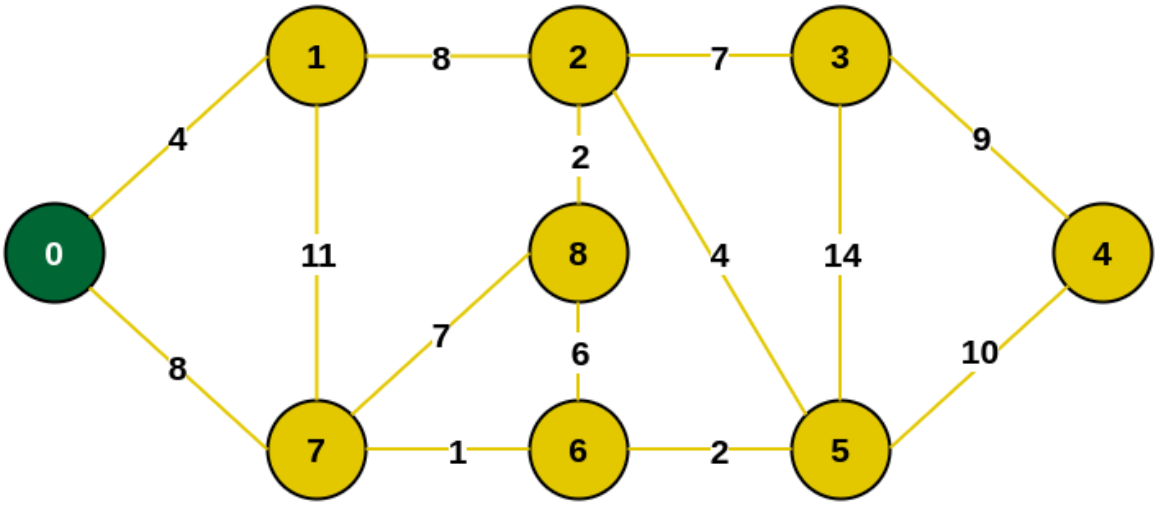
Step 1: *Determine an arbitrary vertex as the starting vertex of the MST.*
Step 2: *Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).*
Step 3: *Find edges connecting any tree vertex with the fringe vertices.*
Step 4: *Find the minimum among these edges.*
Step 5: *Add the chosen edge to the MST if it does not form any cycle.*
Step 6: *Return the MST and exit*

Note: For determining a cycle, we can divide the vertices into two sets [one set contains the vertices included in MST and the other contains the fringe vertices.]

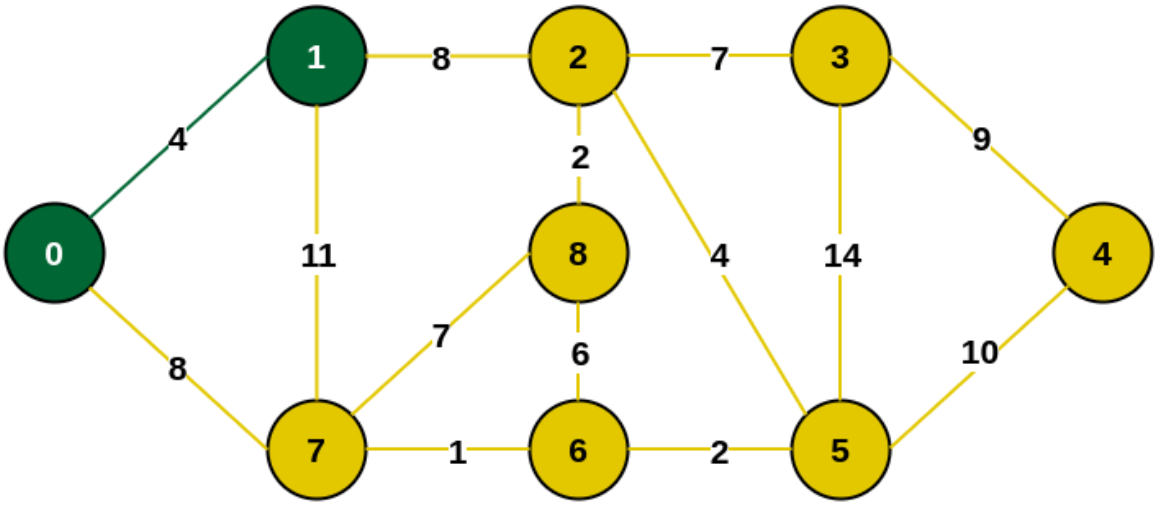
Illustration of Prim's Algorithm:



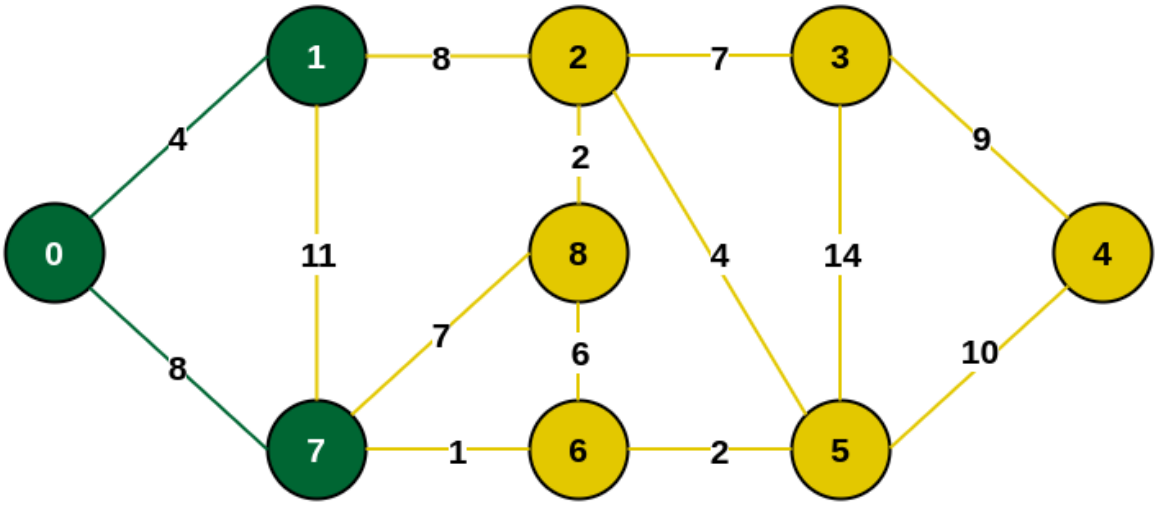
Example of a Graph



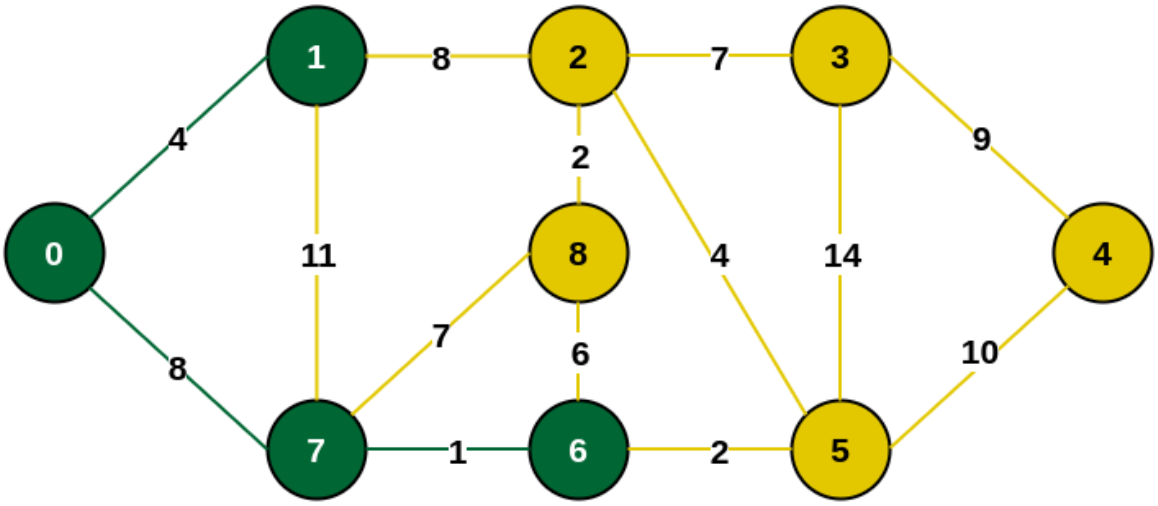
Select an arbitrary starting vertex. Here we have selected 0



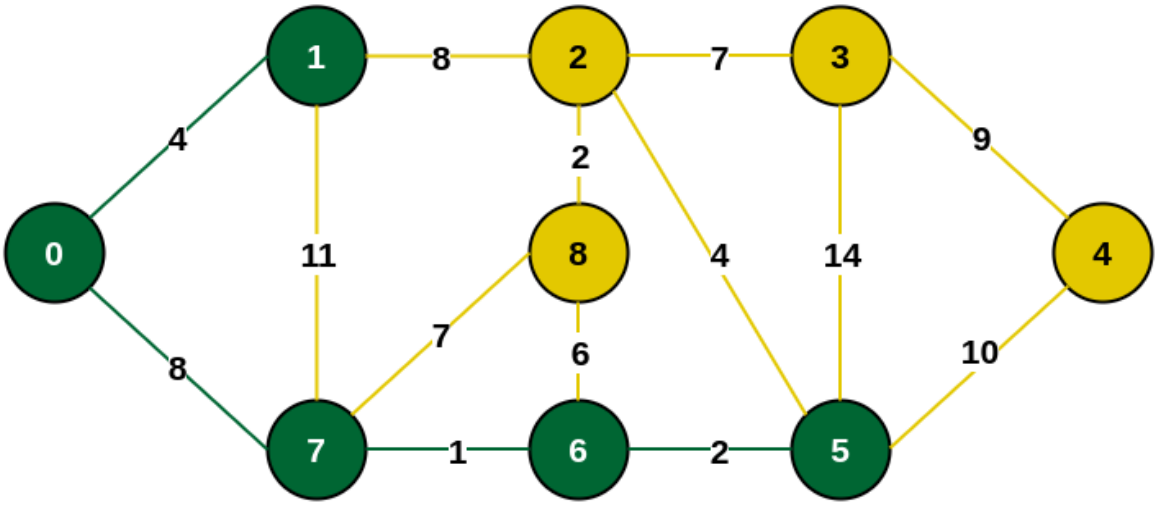
Minimum weighted edge from MST to other vertices is 0-1 with weight 4



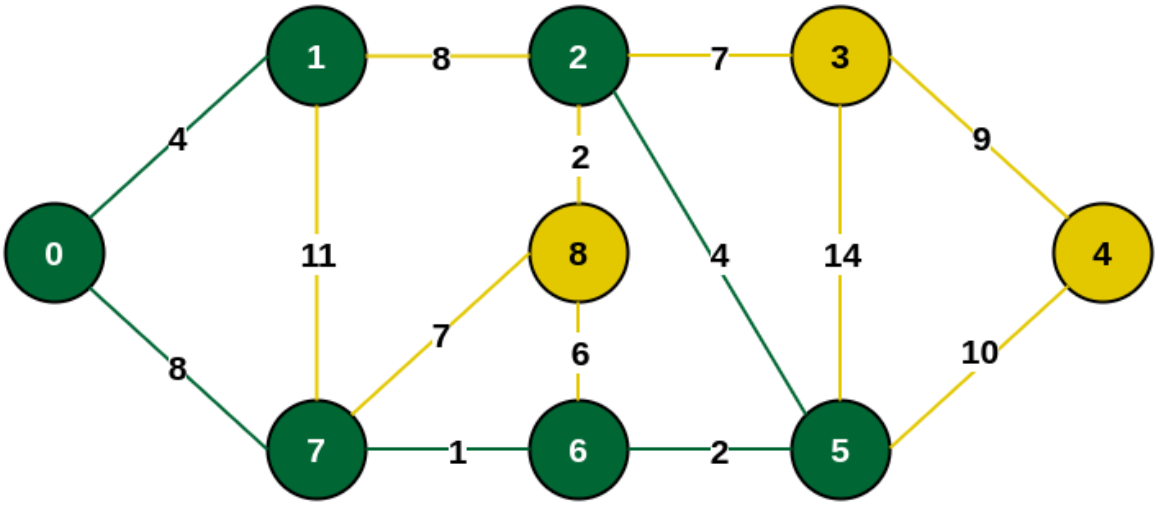
Minimum weighted edge from MST to other vertices is 0-7 with weight 8



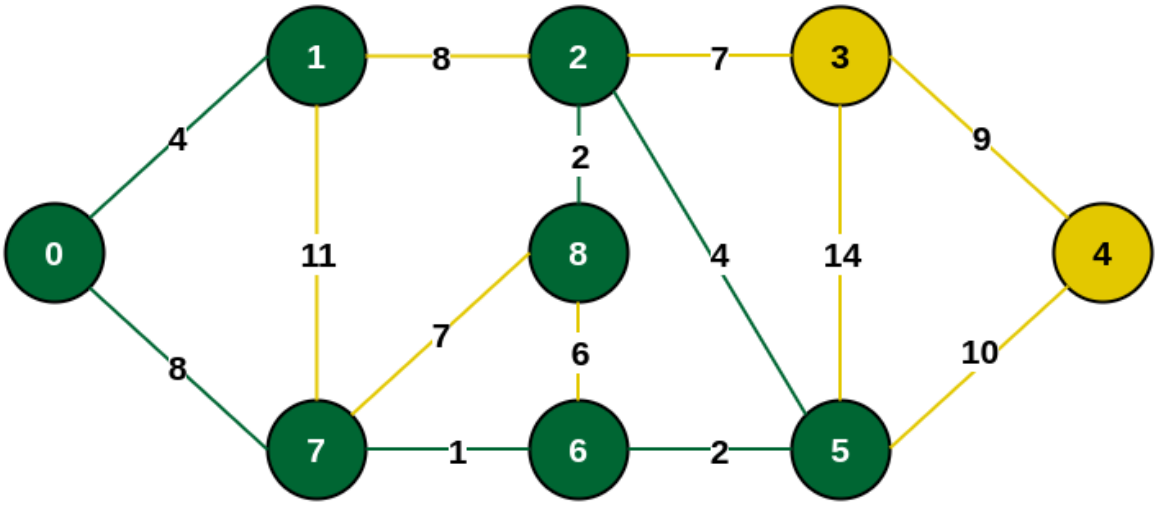
Minimum weighted edge from MST to other vertices is 7-6 with weight 1



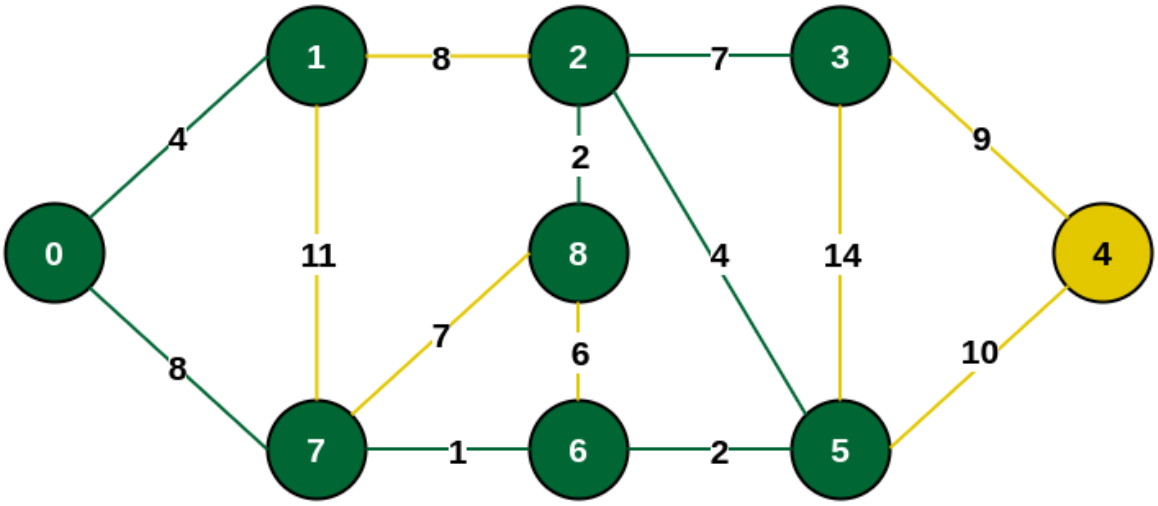
Minimum weighted edge from MST to other vertices is 6-5 with weight 2



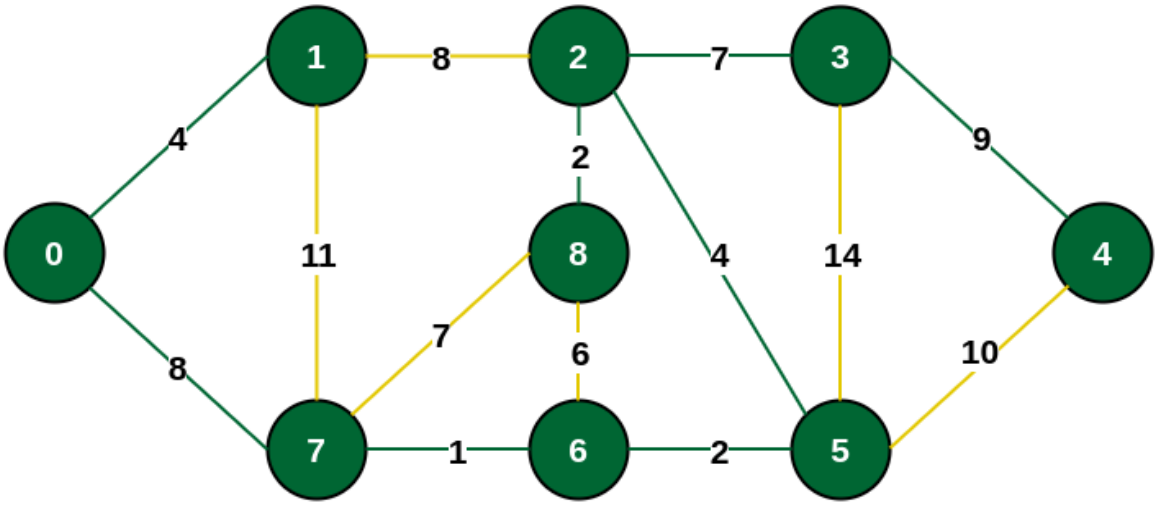
Minimum weighted edge from MST to other vertices is 5-2 with weight 4



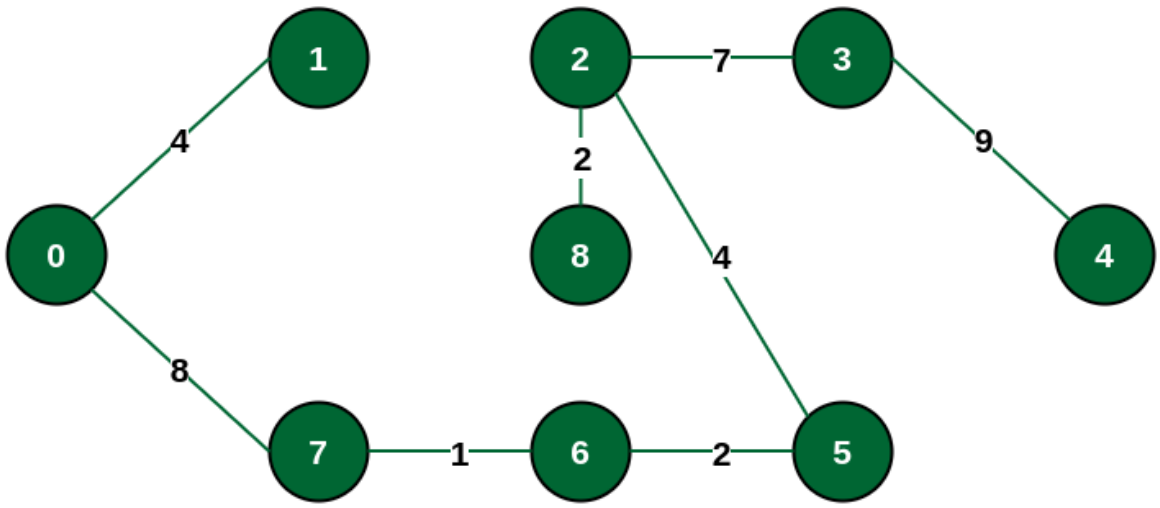
Minimum weighted edge from MST to other vertices is 2-8 with weight 2



Minimum weighted edge from MST to other vertices is 2-3 with weight 7



Minimum weighted edge from MST to other vertices is 3-4 with weight 9



The final structure of MST

Note: There can be multiple MST structures for a given graph.

How to implement Prim's Algorithm?

Follow the given steps to utilize the **Prim's Algorithm** mentioned above for finding MST of a graph:

- Create a set **mstSet** that keeps track of vertices already included in MST.
- Assign a key value to all vertices in the input graph. Initialize all key values as INFINITE. Assign the key value as 0 for the first vertex so that it is picked first.
- While **mstSet** doesn't include all vertices
    - Pick a vertex **u** that is not there in **mstSet** and has a minimum key value.
    - Include **u** in the **mstSet**.
    - Update the key value of all adjacent vertices of **u**. To update the key values, iterate through all adjacent vertices.
        - For every adjacent vertex **v**, if the weight of edge **u-v** is less than the previous key value of **v**, update the key value as the weight of **u-v**.

The idea of using key values is to pick the minimum weight edge from the cut. The key values are used only for vertices that are not yet included in MST, the key value for these vertices indicates the minimum weight edges connecting them to the set of vertices included in MST.

Below is the implementation of the approach:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

#define V 5

int minKey(int key[], bool mstSet[]){
	int min = INT_MAX, min_index;

	for (int v = 0; v < V; v++)
		if (mstSet[v] == false && key[v] < min)
			min = key[v], min_index = v;

	return min_index;
}

void printMST(int parent[], int graph[V][V]){
	cout << "Edge \tWeight\n";
	for (int i = 1; i < V; i++)
		cout << parent[i] << " - " << i << " \t"
			<< graph[i][parent[i]] << " \n";
}

void primMST(int graph[V][V]){
	int parent[V];

	int key[V];

	bool mstSet[V];

	for (int i = 0; i < V; i++)
		key[i] = INT_MAX, mstSet[i] = false;

	key[0] = 0;

	parent[0] = -1;
	for (int count = 0; count < V - 1; count++) {

		int u = minKey(key, mstSet);
		mstSet[u] = true;

		for (int v = 0; v < V; v++)

			if (graph[u][v] && mstSet[v] == false
				&& graph[u][v] < key[v])
				parent[v] = u, key[v] = graph[u][v];
	}
```

```
        printMST(parent, graph);
}

int main(){
        int graph[V][V] = { { 0, 2, 0, 6, 0 },
                            { 2, 0, 3, 8, 5 },
                            { 0, 3, 0, 0, 7 },
                            { 6, 8, 0, 0, 9 },
                            { 0, 5, 7, 9, 0 } };

        primMST(graph);

        return 0;
}
```

Time Complexity: O(V2), If the input graph is represented using an adjacency list, then the time complexity of Prim's algorithm can be reduced to O(E * logV) with the help of a binary heap. In this implementation, we are always considering the spanning tree to start from the root of the graph
Auxiliary Space: O(V)

Kruskal's Algorithm

Kruskal's Algorithm
Here we will discuss **Kruskal's algorithm** to find the MST of a given weighted graph.

In Kruskal's algorithm, sort all edges of the given graph in increasing order. Then it keeps on adding new edges and nodes in the MST if the newly added edge does not form a cycle. It picks the minimum weighted edge at first at the maximum weighted edge at last. Thus we can say that it makes a locally optimal choice in each step in order to find the optimal solution. Hence this is a **Greedy Algorithm**.

How to find MST using Kruskal's algorithm?

Below are the steps for finding MST using Kruskal's algorithm:

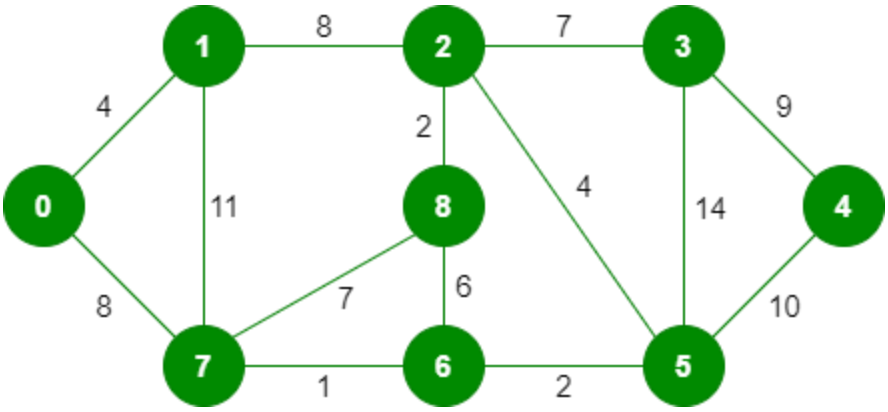
1. Sort all the edges in non-decreasing order of their weight.
2. Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
3. Repeat step#2 until there are (V-1) edges in the spanning tree.

Kruskal's algorithm to find the minimum cost-spanning tree uses the greedy approach. The Greedy Choice is to pick the smallest weight edge that does not cause a cycle in the MST constructed so far. Let us understand it with an example:

Illustration:

Below is the illustration of the above approach:

Input Graph:



The graph contains 9 vertices and 14 edges. So, the minimum spanning tree formed will be having (9 – 1) = 8 edges.
After sorting:

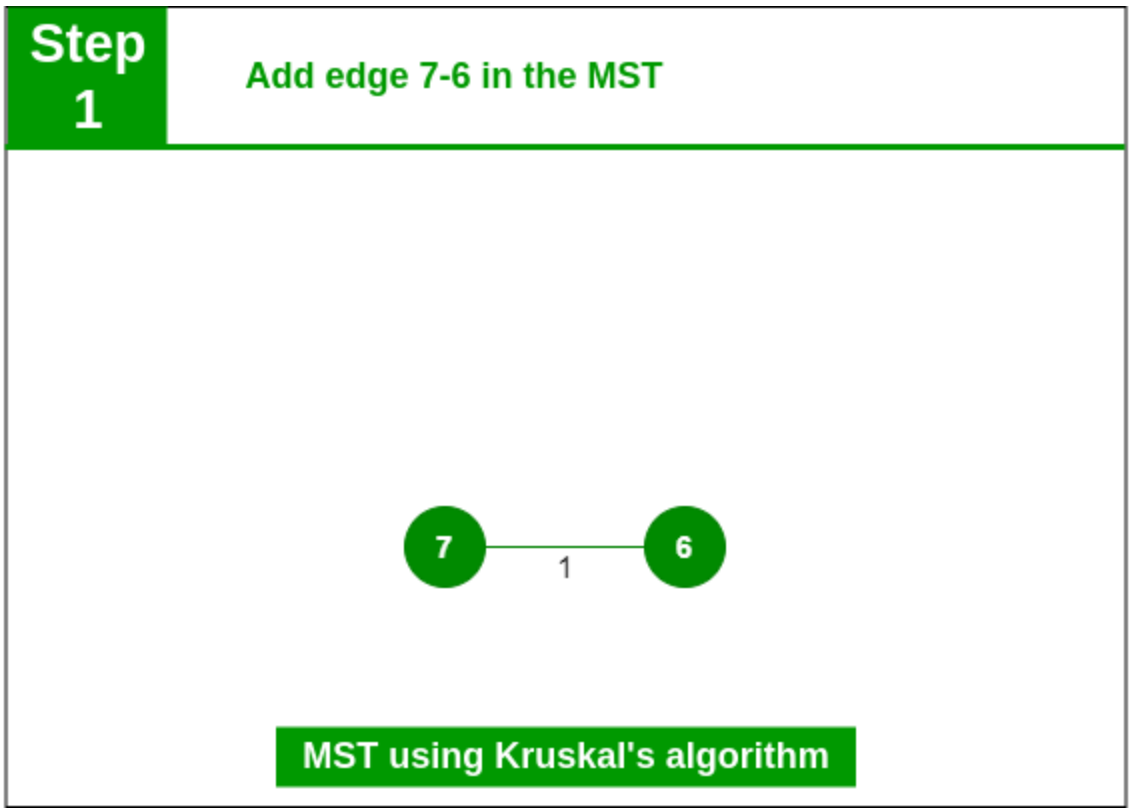
Weight	*Source*	*Destination*
1	*7*	*6*
2	*8*	*2*
2	*6*	*5*

4	*0*	*1*
4	*2*	*5*
6	*8*	*6*
7	*2*	*3*
7	*7*	*8*
8	*0*	*7*
8	*1*	*2*
9	*3*	*4*
10	*5*	*4*
11	*1*	*7*

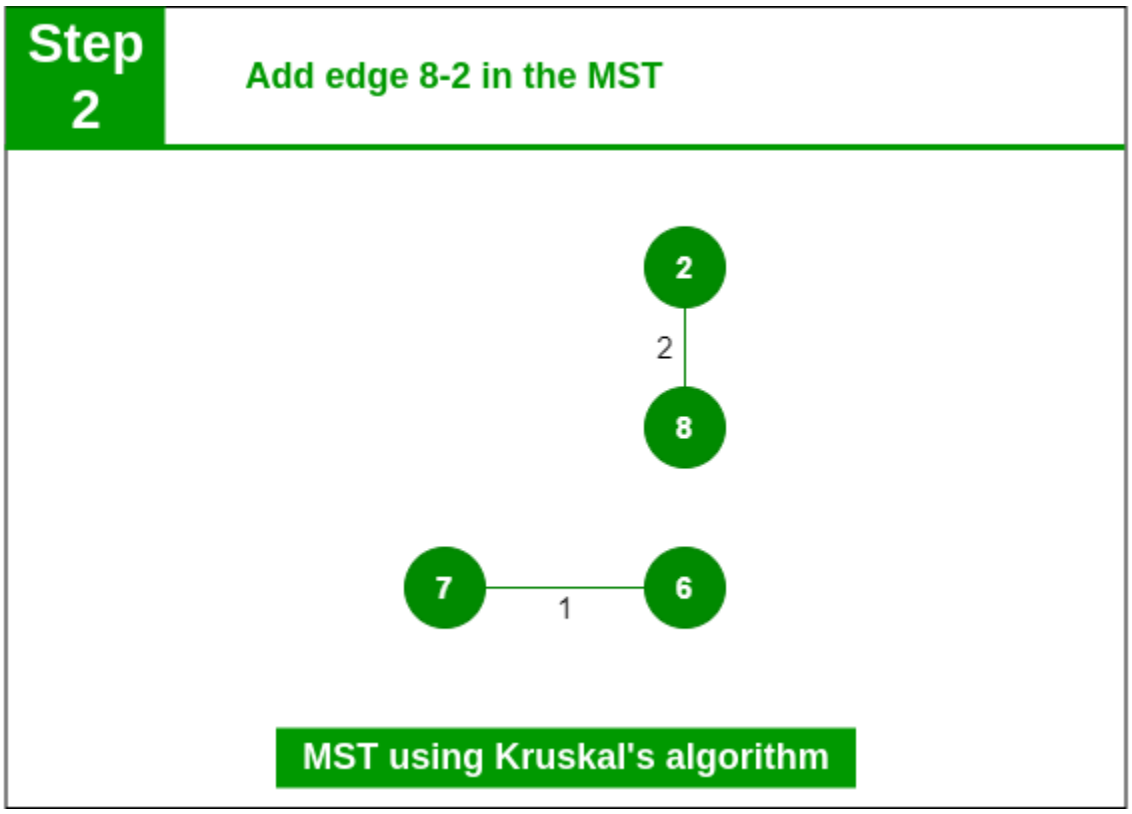
14	*3*	*5*

Now pick all edges one by one from the sorted list of edges

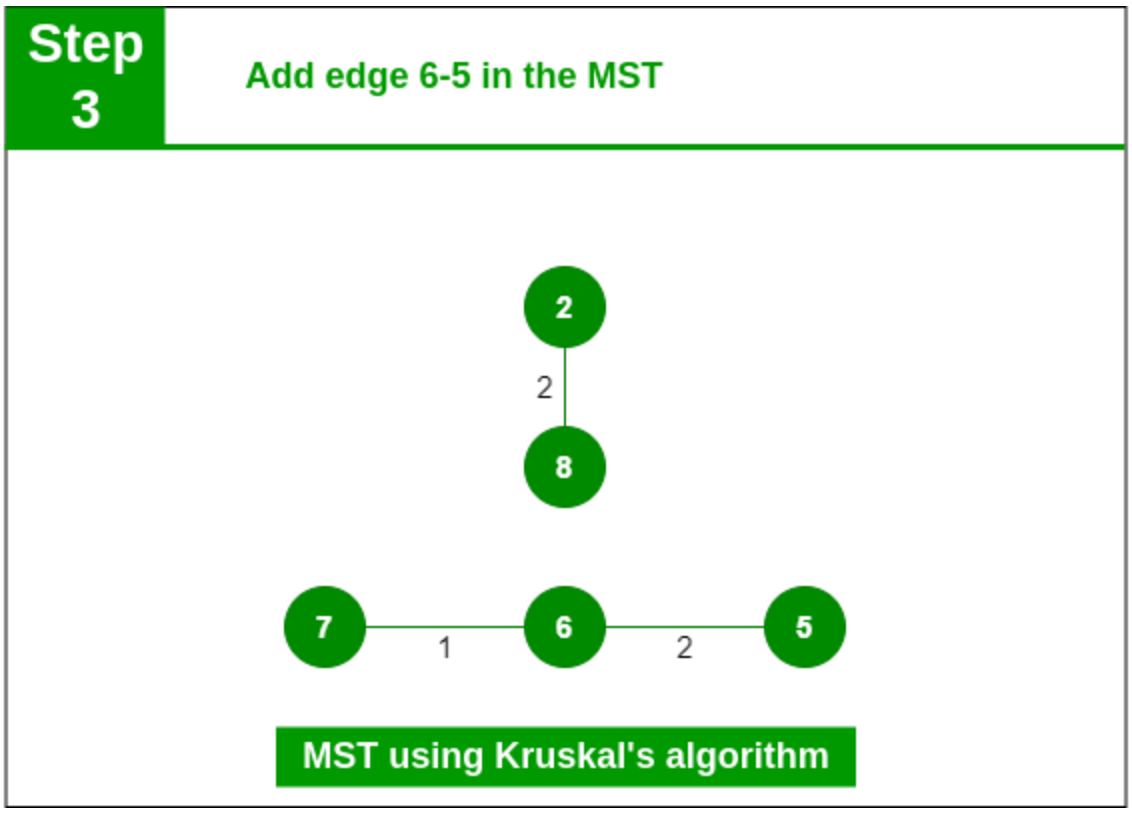
Step 1: Pick edge 7-6. No cycle is formed, include it.



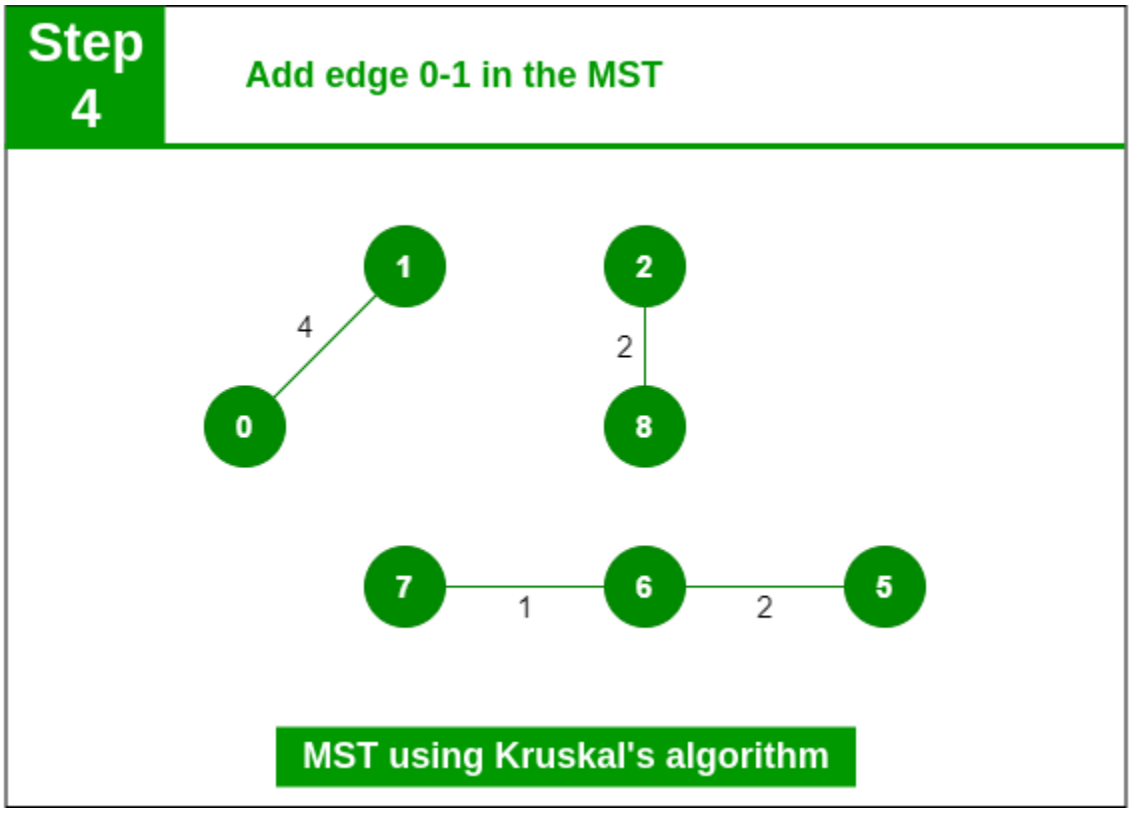
Step 2: Pick edge 8-2. No cycle is formed, include it.



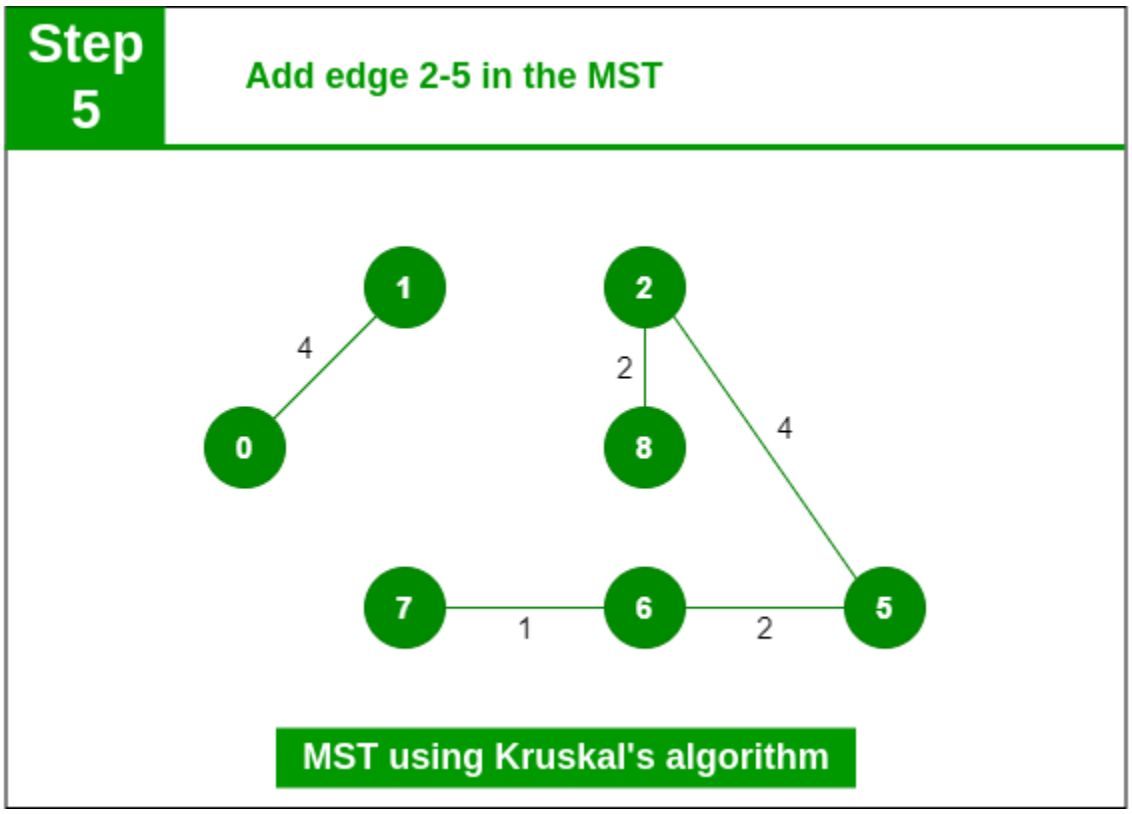
Step 3: Pick edge 6-5. No cycle is formed, include it.



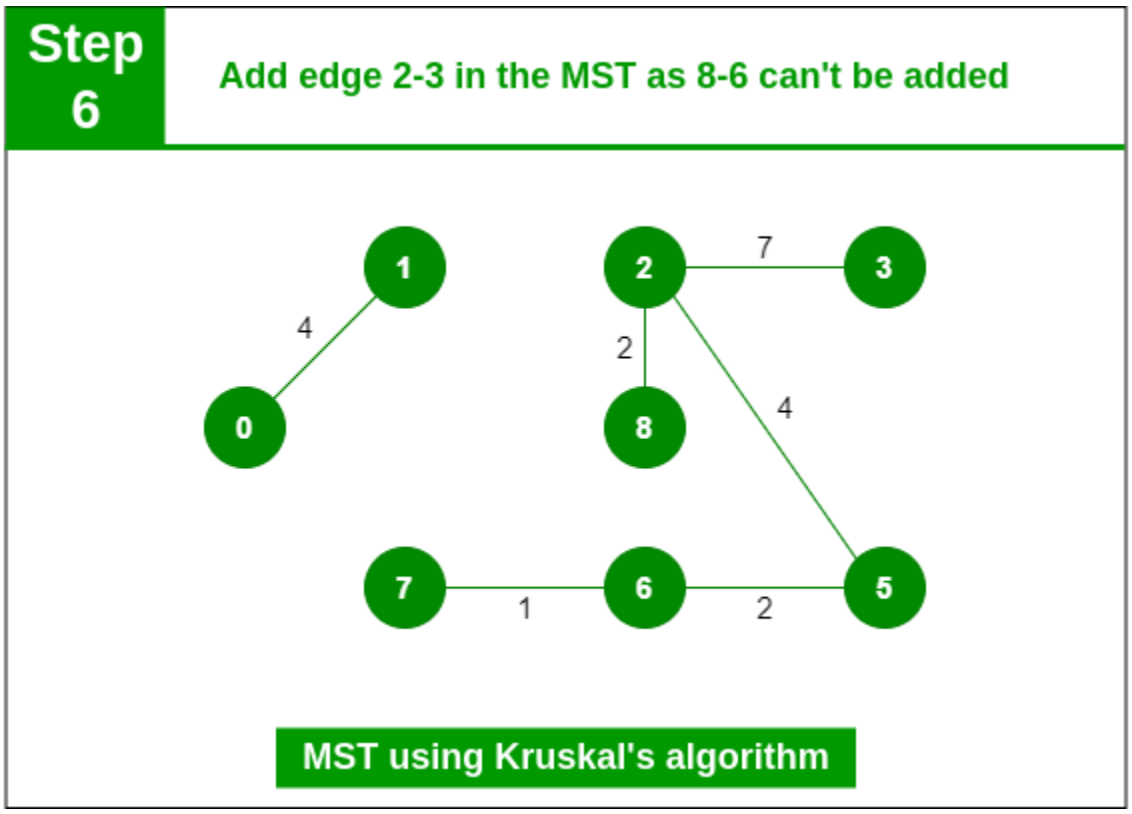
Step 4: Pick edge 0-1. No cycle is formed, include it.



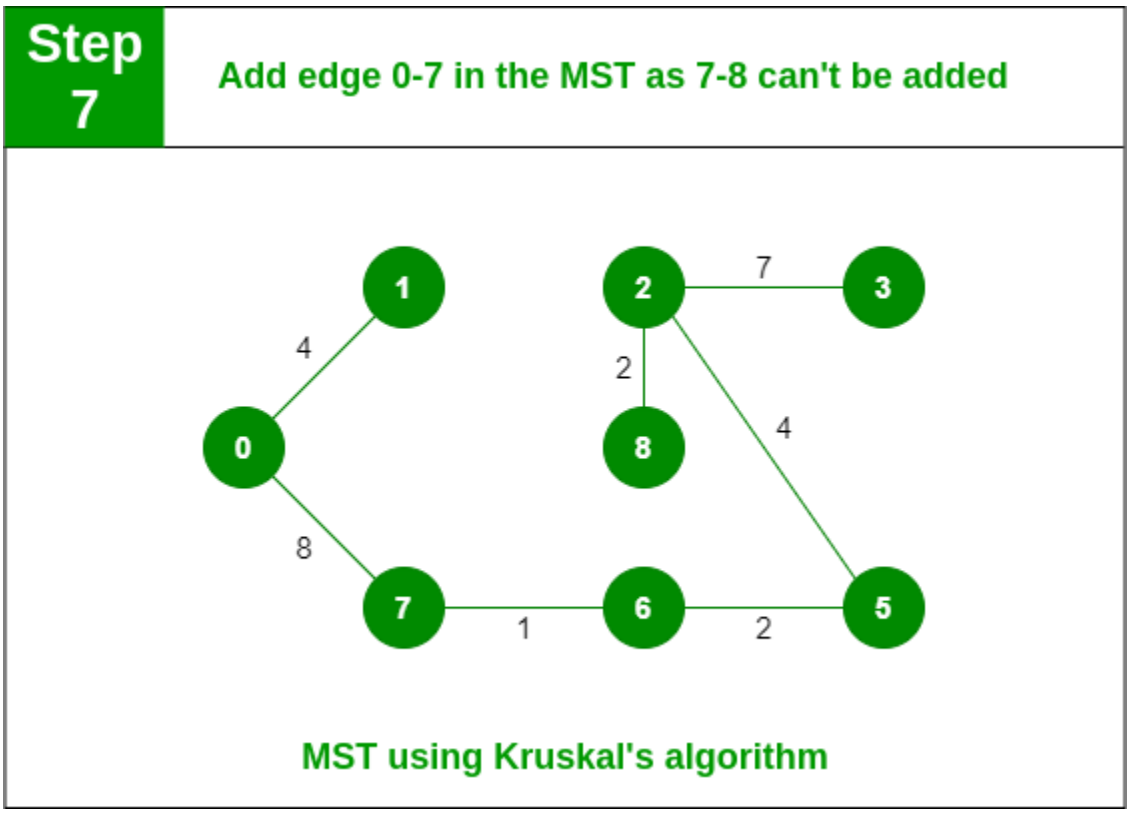
Step 5: Pick edge 2-5. No cycle is formed, include it.



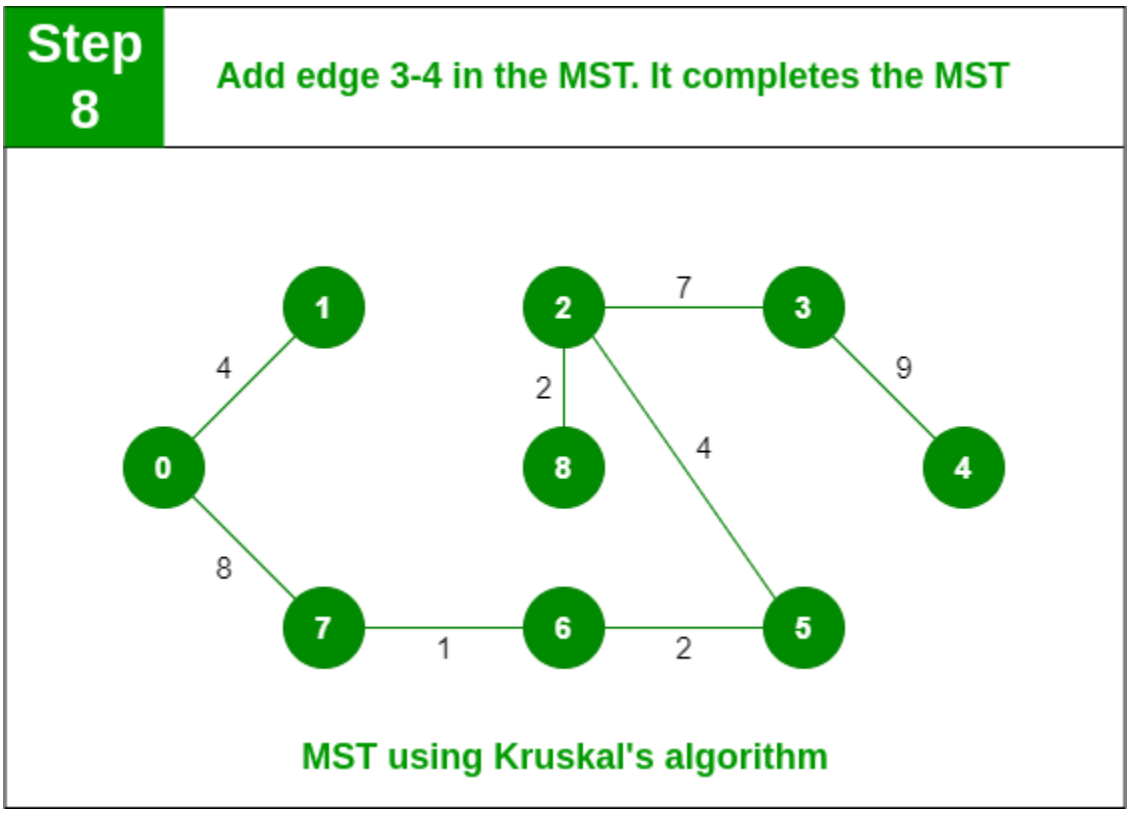
Step 6: Pick edge 8-6. Since including this edge results in the cycle, discard it. Pick edge 2-3: No cycle is formed, include it.



Step 7: Pick edge 7-8. Since including this edge results in the cycle, discard it. Pick edge 0-7. No cycle is formed, include it.



Step 8: Pick edge 1-2. Since including this edge results in the cycle, discard it. Pick edge 3-4. No cycle is formed, include it.



Note: Since the number of edges included in the MST equals to (V – 1), so the algorithm stops here

C++

```
#include <bits/stdc++.h>
using namespace std;

class DSU {
        int* parent;
        int* rank;

public:
        DSU(int n){
                parent = new int[n];
                rank = new int[n];

                for (int i = 0; i < n; i++) {
                        parent[i] = -1;
                        rank[i] = 1;
                }
        }

        int find(int i){
                if (parent[i] == -1)
```

```
                        return i;

                return parent[i] = find(parent[i]);
        }

        void unite(int x, int y)
        {
                int s1 = find(x);
                int s2 = find(y);

                if (s1 != s2) {
                        if (rank[s1] < rank[s2]) {
                                parent[s1] = s2;
                        }
                        else if (rank[s1] > rank[s2]) {
                                parent[s2] = s1;
                        }
                        else {
                                parent[s2] = s1;
                                rank[s1] += 1;
                        }
                }
        }
};

class Graph {
        vector<vector<int> > edgelist;
        int V;

public:
        Graph(int V) { this->V = V; }

        void addEdge(int x, int y, int w){
                edgelist.push_back({ w, x, y });
        }

        void kruskals_mst(){
                sort(edgelist.begin(), edgelist.end());

                DSU s(V);
                int ans = 0;
                cout << "Following are the edges in the "
                                "constructed MST"
                        << endl;
                for (auto edge : edgelist) {
                        int w = edge[0];
                        int x = edge[1];
                        int y = edge[2];
```

```
                        if (s.find(x) != s.find(y)) {
                                s.unite(x, y);
                                ans += w;
                                cout << x << " -- " << y << " == " << w
                                        << endl;
                        }
                }
                cout << "Minimum Cost Spanning Tree: " << ans;
        }
};

int main(){
        Graph g(4);
        g.addEdge(0, 1, 10);
        g.addEdge(1, 3, 15);
        g.addEdge(2, 3, 4);
        g.addEdge(2, 0, 6);
        g.addEdge(0, 3, 5);

        g.kruskals_mst();

        return 0;
}
```

Time Complexity: O(E * logE) or O(E * logV)

- Sorting of edges takes O(E * logE) time.
- After sorting, we iterate through all edges and apply the find-union algorithm. The find and union operations can take at most O(logV) time.
- So overall complexity is O(E * logE + E * logV) time.
- The value of E can be at most $O(V^2)$, so O(logV) and O(logE) are the same. Therefore, the overall time complexity is O(E * logE) or O(E*logV)

Auxiliary Space: O(V + E), where V is the number of vertices and E is the number of edges in the graph.

String Algorithms

Naive Pattern Searching

Naive Pattern Searching

i) It is the simplest method that uses a brute force approach.

ii) It is a straightforward approach to solving the problem.

iii) It compares the first character of the pattern with searchable text. If a match is found, pointers in both strings are advanced. If a match is not found, the pointer of the text is incremented and the pointer of the pattern is reset. This process is repeated until the end of the text.

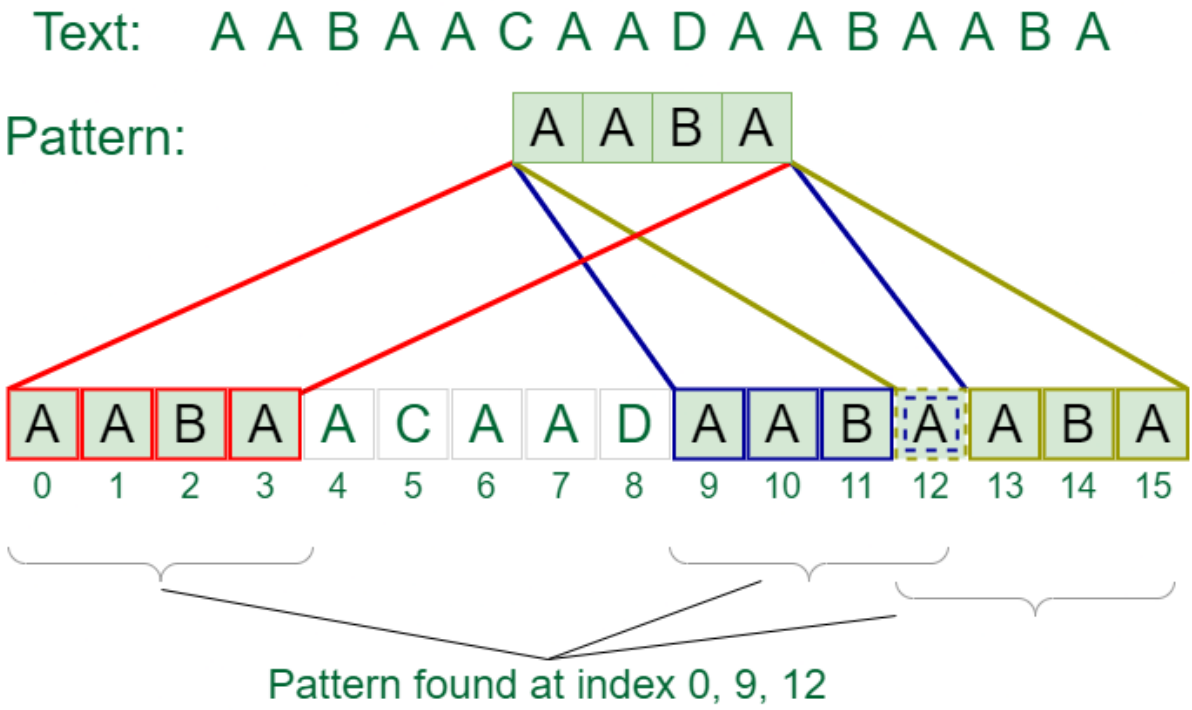
iv) It does not require any pre-processing. It directly starts comparing both strings character by character.

v) Time Complexity = **O(m* (n-m))**

Given a text of length **N** txt[0..N-1] and a pattern of length M pat[0..M-1], write a function search(char pat[], char txt[]) that prints all occurrences of pat[] in txt[]. You may assume that N > M.
Examples:

Input: *txt[] = "THIS IS A TEST TEXT", pat[] = "TEST"*
Output: *Pattern found at index 10*

Input: *txt[] = "AABAACAADAABAABA", pat[] = "AABA"*
Output: *Pattern found at index 0, Pattern found at index 9, Pattern found at index 12*



Pattern searching is an important problem in computer science. When we do search for a string in a notepad/word file or browser or database, pattern-searching algorithms are used to show the search results.

C++

```
#include <bits/stdc++.h>
using namespace std;

void search(char* pat, char* txt){
        int M = strlen(pat);
        int N = strlen(txt);
        for (int i = 0; i <= N - M; i++) {
                int j;

                for (j = 0; j < M; j++)
                        if (txt[i + j] != pat[j])
                                break;

                if (j == M)
                        cout << "Pattern found at index " << i << endl;
        }
}

int main(){
        char txt[] = "AABAACAADAABAAABAA";
        char pat[] = "AABA";

        // Function call
        search(pat, txt);
        return 0;
}
```

Improved Naive Pattern Searching for distinct

The following code shows the implementation of an Improved Naive pattern-searching algorithm that searches for occurrences of a distinct pattern in a given text string:

C++

```
#include <bits/stdc++.h>
using namespace std;
```

```
void patSearchinng(string &txt,string &pat){
    int m=pat.length();
    int n=txt.length();
    for(int i=0;i<=(n-m);  ){
        int j;
        for(j=0;j<m;j++)
        if(pat[j]!=txt[i+j])
        break;

        if(j==m)
        cout<<i<<" ";
        if(j==0){
        i++;}
        else{
        i=(i+j);}
    }
}

int main() {
    string txt = "ABCABCD";string pat="ABCD";
    cout<<"All index numbers where pattern found:"<<" ";
    patSearchinng(txt,pat);

    return 0;
}
```

In this implementation, the function patSearchinng takes two string arguments txt and pat, representing the text string and the pattern string respectively.

The algorithm works as follows:

1. Calculate the lengths of the text string and the pattern string.
2. Iterate over all possible starting positions of the pattern in the text (i.e., from 0 to n-m, where n is the length of the text and m is the length of the pattern).
3. For each starting position i, compare each character of the pattern with the corresponding character of the text starting from position i. If any character does not match, break out of the loop and move to the next starting position.
4. If all characters match, then the pattern has been found at position i in the text. Print the index i and continue searching for further occurrences.
5. After the loop completes, the function returns.

The time complexity of this algorithm is O((n-m+1)*m), where n is the length of the text and m is the length of the pattern. This can be improved using more advanced algorithms like the Knuth-Morris-Pratt (KMP) algorithm or the Boyer-Moore algorithm, which have a worst-case time complexity of O(n+m).

Rabin Karp Algorithm

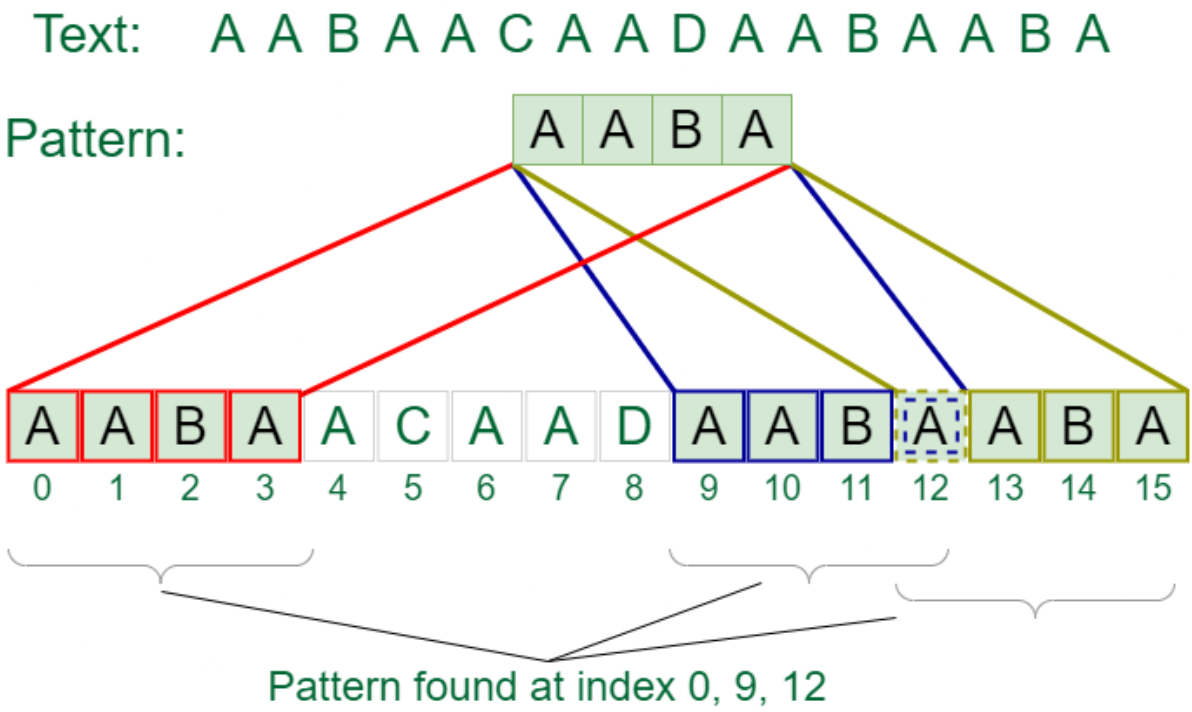
Rabin Karp Algorithm

Given a text **txt[0. . .n-1]** and a pattern **pat[0. . .m-1]**, write a function search(char pat[], char txt[]) that prints all occurrences of pat[] in txt[]. You may assume that **n > m**.

Examples:

Input: *txt[] = "THIS IS A TEST TEXT", pat[] = "TEST"*
Output: *Pattern found at index 10*

Input: *txt[] = "AABAACAADAABAABA", pat[] = "AABA"*
Output: *Pattern found at index 0*
Pattern found at index 9
Pattern found at index 12



Like the Naive Algorithm, the Rabin-Karp algorithm also slides the pattern one by one. But unlike the Naive algorithm, the Rabin Karp algorithm matches the hash value of the pattern with the hash value of the current substring of text, and if the hash values match then only it starts matching individual characters. So Rabin Karp's algorithm needs to calculate hash values for the following strings.

- Pattern itself
- All the substrings of the text of length m

Since we need to efficiently calculate hash values for all the substrings of size m of text, we must have a hash function that has the **following property**:

- *Hash at the next shift must be efficiently computable from the current hash value and next character in text or we can say hash(txt[s+1 .. s+m]) must be efficiently computable from hash(txt[s .. s+m-1]) and txt[s+m] i.e., hash(txt[s+1 .. s+m]) = rehash(txt[s+m], hash(txt[s .. s+m-1])) and*
- *Rehash must be O(1) operation.*

Note: The hash function suggested by Rabin and Karp calculates an integer value. The integer value for a string is the numeric value of a string.
For example, If all possible characters are from 1 to 10, the numeric value of "122" will be 122.

The number of possible characters is higher than 10 (256 in general) and the pattern length can be large. So the numeric values cannot be practically stored as an integer. Therefore, the numeric value is calculated using modular arithmetic to make sure that the hash values can be stored in an integer variable (can fit in memory words). To do rehashing, we need to take off the most significant digit and add the new least significant digit for in hash value. Rehashing is done using the following formula:

hash(txt[s+1 .. s+m]) = (d (hash(txt[s .. s+m-1]) – txt[s]*h) + txt[s + m]) mod q
hash(txt[s .. s+m-1]) : Hash value at shift s
hash(txt[s+1 .. s+m]): Hash value at next shift (or shift s+1)
d: Number of characters in the alphabet
q: A prime number
h: $d^{(m-1)}$

Follow the steps mentioned here to implement the idea:

- Initially calculate the hash value of the pattern.
- Start iterating from the start of the string:
    - Calculate the hash value of the current substring having length m.
    - If the hash value of the current substring and the pattern are the same checks if the substring is the same as the pattern.
    - If they are the same, store the starting index as a valid answer. Otherwise, continue for the next substrings.
- Return the starting indices as the required answer.

Below is the implementation of the above approach:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

#define d 256

void search(char pat[], char txt[], int q){
	int M = strlen(pat);
	int N = strlen(txt);
	int i, j;
	int p = 0;
	int t = 0;
	int h = 1;

	for (i = 0; i < M - 1; i++)
		h = (h * d) % q;

	for (i = 0; i < M; i++) {
		p = (d * p + pat[i]) % q;
		t = (d * t + txt[i]) % q;
	}

	for (i = 0; i <= N - M; i++) {
		if (p == t) {
			for (j = 0; j < M; j++) {
				if (txt[i + j] != pat[j]) {
					break;
				}
			}

			if (j == M)
				cout << "Pattern found at index " << i
					<< endl;
		}

		if (i < N - M) {
			t = (d * (t - txt[i] * h) + txt[i + M]) % q;

			if (t < 0)
				t = (t + q);
		}
	}
}

int main(){
	char txt[] = "GEEKS FOR GEEKS";
	char pat[] = "GEEK";

	int q = INT_MAX;
```

```
        search(pat, txt, q);
        return 0;
}
```

Time Complexity:

- The average and best-case running time of the Rabin-Karp algorithm is O(n+m), but its worst-case time is O(nm).
- The worst case of the Rabin-Karp algorithm occurs when all characters of the pattern and text are the same as the hash values of all the substrings of txt[] match with the hash value of pat[].

Auxiliary Space: O(1)

KMP Algorithm - Part 1

KMP Algorithm

Examples:

1) *txt[] = "AAAAAAAAAAAAAAAAAB", pat[] = "AAAAB"*

2) *txt[] = "ABABABCABABABCABABABC", pat[] = "ABABAC" (not a worst case, but a bad case for Naive)*

The KMP matching algorithm uses degenerating property (pattern having the same sub-patterns appearing more than once in the pattern) of the pattern and improves the worst-case complexity to O(n).

The basic idea behind KMP's algorithm is: whenever we detect a mismatch (after some matches), we already know some of the characters in the text of the next window. We take advantage of this information to avoid matching the characters that we know will anyway match.

Example:

Matching Overview

txt = "AAAAABAAABA"
pat = "AAAA"

We compare first window of ***txt*** *with* ***pat***

*txt = "**AAAA**ABAAABA"*
*pat = "**AAAA**" [Initial position]*
We find a match. This is same as Naive String Matching.

*In the next step, we compare next window of **txt** with **pat**.*

*txt = "**AAAAA**BAAABA"*
*pat = "**AAAA**" [Pattern shifted one position]*

This is where KMP does optimization over Naive. In this second window, we only compare fourth A of pattern
with fourth character of current window of text to decide whether current window matches or not. Since we know
first three characters will anyway match, we skipped matching first three characters.

Need of Preprocessing?

An important question arises from the above explanation, how to know how many characters to be skipped. To know this,
we pre-process pattern and prepare an integer array lps[] that tells us the count of characters to be skipped

Preprocessing Overview:

- KMP algorithm preprocesses pat[] and constructs an auxiliary lps[] of size m (same as the size of the pattern) which is used to skip characters while matching.
- name lps indicates the longest proper prefix which is also a suffix. A proper prefix is a prefix with a whole string not allowed. For example, prefixes of "ABC" are "", "A", "AB" and "ABC". Proper prefixes are "", "A" and "AB". Suffixes of the string are "", "C", "BC", and "ABC".
- We search for lps in sub-patterns. More clearly we focus on sub-strings of patterns that are both prefix and suffix.
- For each sub-pattern pat[0..i] where i = 0 to m-1, lps[i] stores the length of the maximum matching proper prefix which is also a suffix of the sub-pattern pat[0..i].

lps[i] = the longest proper prefix of pat[0..i] which is also a suffix of pat[0..i].

Note: lps[i] could also be defined as the longest prefix which is also a proper suffix. We need to use it properly in one place to make sure that the whole substring is not considered.

Examples of lps[] construction:

For the pattern "AAAA", lps[] is [0, 1, 2, 3]

For the pattern "ABCDE", lps[] is [0, 0, 0, 0, 0]

For the pattern "AABAACAABAA", lps[] is [0, 1, 0, 1, 2, 0, 1, 2, 3, 4, 5]

For the pattern "AAACAAAAAC", lps[] is [0, 1, 2, 0, 1, 2, 3, 3, 3, 4]

For the pattern "AAABAAA", lps[] is [0, 1, 2, 0, 1, 2, 3]

Preprocessing Algorithm:

In the preprocessing part,

- We calculate values in lps[]. To do that, we keep track of the length of the longest prefix suffix value (we use len variable for this purpose) for the previous index
- We initialize lps[0] and len as 0
- If pat[len] and pat[i] match, we increment len by 1 and assign the incremented value to lps[i].
- If pat[i] and pat[len] do not match and len is not 0, we update len to lps[len-1]
- See computeLPSArray () in the above code for details

Illustration of preprocessing (or construction of lps[]):

pat[] = "AAACAAAA"

len = 0, i = 0.
lps[0] is always 0, we move

to i = 1
len = 0, i = 1.

Since pat[len] and pat[i] match, do len++,
store it in lps[i] and do i++.
len = 1, lps[1] = 1, i = 2

len = 1, i = 2.

Since pat[len] and pat[i] match, do len++,
store it in lps[i] and do i++.
len = 2, lps[2] = 2, i = 3

len = 2, i = 3.

Since pat[len] and pat[i] do not match, and len > 0,
set len = lps[len-1] = lps[1] = 1

len = 1, i = 3.

Since pat[len] and pat[i] do not match and len > 0,
len = lps[len-1] = lps[0] = 0

len = 0, i = 3.

Since pat[len] and pat[i] do not match and len = 0,
Set lps[3] = 0 and i = 4.
We know that characters pat

len = 0, i = 4.

Since pat[len] and pat[i] match, do len++,
store it in lps[i] and do i++. len = 1, lps[4] = 1, i = 5

len = 1, i = 5.

Since pat[len] and pat[i] match, do len++,
store it in lps[i] and do i++.

len = 2, lps[5] = 2, i = 6
len = 2, i = 6.

Since pat[len] and pat[i] match, do len++,
store it in lps[i] and do i++.
len = 3, lps[6] = 3, i = 7

len = 3, i = 7.

Since pat[len] and pat[i] do not match and len > 0,
set len = lps[len-1] = lps[2] = 2

len = 2, i = 7.

Since pat[len] and pat[i] match, do len++,
store it in lps[i] and do i++.
len = 3, lps[7] = 3, i = 8

We stop here as we have constructed the whole lps[].

Implementation of KMP algorithm:

Unlike the Naive algorithm, where we slide the pattern by one and compare all characters at each shift, we use a value from lps[] to decide the next characters to be matched. The idea is to not match a character that we know will anyway match.

How to use lps[] to decide the next positions (or to know the number of characters to be skipped)?

- We start the comparison of pat[j] with j = 0 with characters of the current window of text.
- We keep matching characters txt[i] and pat[j] and keep incrementing i and j while pat[j] and txt[i] keep **matching**.
- When we see a **mismatch**
    - We know that characters pat[0..j-1] match with txt[i-j…i-1] (Note that j starts with 0 and increments it only when there is a match).
    - We also know (from the above definition) that lps[j-1] is the count of characters of pat[0…j-1] that are both proper prefixes and suffixes.
    - From the above two points, we can conclude that we do not need to match these lps[j-1] characters with txt[i-j…i-1] because we know that these characters will anyway match. Let us consider the above example to understand this.

Below is the illustration of the above algorithm:

txt[] = "AAAAABAAABA" , pat[] = "AAAA"
lps[] = {0, 1, 2, 3}

i = 0, j = 0

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++, j++

i = 1, j = 1

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++, j++

i = 2, j = 2

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++, j++

i = 3, j = 3

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++, j++

i = 4, j = 4

Since j == M, print pattern found and reset j,
j = lps[j-1] = lps[3] = 3

Here unlike Naive algorithm, we do not match first three
characters of this window. Value of lps[j-1] (in above step) gave us index of next character to match.

i = 4, j = 3

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++, j++

i = 5, j = 4

Since j == M, print pattern found and reset j, j = lps[j-1] = lps[3] = 3
Again unlike Naive algorithm, we do not match first three
characters of this window. Value of lps[j-1] (in above step) gave us index of next character to match.

i = 5, j = 3

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] do NOT match and j > 0, change only j
j = lps[j-1] = lps[2] = 2

i = 5, j = 2

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] do NOT match and j > 0, change only j
j = lps[j-1] = lps[1] = 1

i = 5, j = 1

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] do NOT match and j > 0, change only j
j = lps[j-1] = lps[0] = 0

i = 5, j = 0

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] do NOT match and j is 0, we do i++.

i = 6, j = 0

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++ and j++

i = 7, j = 1

txt[] = "AAAAABAAABA"
pat[] = "AAAA"
txt[i] and pat[j] match, do i++ and j++

We continue this way till there are sufficient characters in the text to be compared with the characters in the pattern...

C++

```
#include <bits/stdc++.h>

void computeLPSArray(char* pat, int M, int* lps);

void KMPSearch(char* pat, char* txt){
	int M = strlen(pat);
	int N = strlen(txt);

	int lps[M];

	computeLPSArray(pat, M, lps);

	int i = 0;
	int j = 0;
	while ((N - i) >= (M - j)) {
```

```
                if (pat[j] == txt[i]) {
                        j++;
                        i++;
                }

                if (j == M) {
                        printf("Found pattern at index %d ", i - j);
                        j = lps[j - 1];
                }

                else if (i < N && pat[j] != txt[i]) {
                        if (j != 0)
                                j = lps[j - 1];
                        else
                                i = i + 1;
                }
        }
}

void computeLPSArray(char* pat, int M, int* lps){

        int len = 0;

        lps[0] = 0;
        int i = 1;
        while (i < M) {
                if (pat[i] == pat[len]) {
                        len++;
                        lps[i] = len;
                        i++;
                }
                else{
                        if (len != 0) {
                                len = lps[len - 1];
                        }
                        else {
                                lps[i] = 0;
                                i++;
                        }
                }
        }
}

int main(){
        char txt[] = "ABABDABACDABABCABAB";
        char pat[] = "ABABCABAB";
        KMPSearch(pat, txt);
        return 0;
}
```

Minimum characters to be added at front to make a Palindrome

Minimum characters to be added at the front to make string palindrome

Given string **str** we need to tell minimum characters to be added in front of the string to make string palindrome.

Examples:

```
Input  : str = "ABC"
Output : 2
We can make above string palindrome as "CBABC"
by adding 'B' and 'C' at front.

Input  : str = "AACECAAAA";
Output : 2
We can make above string palindrome as AAAACECAAAA
by adding two A's at front of string.
```

Naive approach: Start checking the string each time if it is a palindrome and if not, then delete the last character and check again. When the string gets reduced to either a palindrome or an empty string then the number of characters deleted from the end till now will be the answer as those characters could have been inserted at the beginning of the original string in the order which will make the string a palindrome.

Below is the implementation of the above approach:

C++

```
#include<bits/stdc++.h>
using namespace std;

bool ispalindrome(string s){
        int l = s.length();
        int j;

        for(int i = 0, j = l - 1; i <= j; i++, j--){
                if(s[i] != s[j])
                        return false;
        }
        return true;
}

int main(){
```

```
        string s = "BABABAA";
        int cnt = 0;
        int flag = 0;

        while(s.length()>0){
                if(ispalindrome(s)){
                        flag = 1;
                        break;
                }
                else{
                        cnt++;
                        s.erase(s.begin() + s.length() - 1);
                }
        }

        if(flag)
                cout << cnt;
}
```

Time complexity: $O(n^2)$
Auxiliary Space: $O(1)$

Efficient approach: We can solve this problem **efficiently in O(n) time** using lps array of the KMP algorithm.
First, we concat the string by concatenating the given string, a special character, and the reverse of the given string then we will get lps array for this concatenated string, and recall that each index of lps array represents the longest proper prefix which is also a suffix. We can use this lps array to solve the problem.

```
For string = AACECAAAA
Concatenated String = AACECAAAA$AAAACECAA
LPS array will be {0, 1, 0, 0, 0, 1, 2, 2, 2,
                   0, 1, 2, 2, 2, 3, 4, 5, 6, 7}
```

Here we are only interested in the last value of this lps array because it shows us the largest suffix of the reversed string that matches the prefix of the original string i.e these many characters already satisfy the palindrome property. Finally the minimum number of characters needed to make the string a palindrome is the length of the input string minus the last entry of our lps array.

Below is the implementation of the above approach:

C++

```cpp
#include <bits/stdc++.h>
using namespace std;

vector<int> computeLPSArray(string str){
	int M = str.length();
	vector<int> lps(M);

	int len = 0;
	lps[0] = 0;

	int i = 1;
	while (i < M){
		if (str[i] == str[len]){
			len++;
			lps[i] = len;
			i++;
		}
		else {
			if (len != 0){
				len = lps[len-1];
			}
			else{
				lps[i] = 0;
				i++;
			}
		}
	}
	return lps;
}

int getMinCharToAddedToMakeStringPalin(string str){
	string revStr = str;
	reverse(revStr.begin(), revStr.end());

	string concat = str + "$" + revStr;

	vector<int> lps = computeLPSArray(concat);

	return (str.length() - lps.back());
}

int main(){
	string str = "AACECAAAA";
	cout << getMinCharToAddedToMakeStringPalin(str);
	return 0;
}
```

Time Complexity: O(n)
Auxiliary Space: O(n)
Z Function

Z Function

This algorithm finds all occurrences of a pattern in a text in linear time. Let length of text be n and of pattern be m, then total time taken is O(m + n) with linear space complexity. Now we can see that both time and space complexity is same as KMP algorithm but this algorithm is Simpler to understand.
In this algorithm, we construct a Z array.

What is Z Array?
For a string str[0..n-1], Z array is of same length as string. An element Z[i] of Z array stores length of the longest substring starting from str[i] which is also a prefix of str[0..n-1]. The first entry of Z array is meaning less as complete string is always prefix of itself.

Example:

Index	0	1	2	3	4	5	6	7	8	9	10	11
Text	a	a	b	c	a	a	b	x	a	a	a	z
Z values	X	1	0	0	3	1	0	0	2	2	1	0

More Examples:

```
str  = "aaaaaa"
Z[]  = {x, 5, 4, 3, 2, 1}

str = "aabaacd"
Z[] = {x, 1, 0, 2, 1, 0, 0}

str = "abababab"
Z[] = {x, 0, 6, 0, 4, 0, 2, 0}
```

How is Z array helpful in Searching Pattern in Linear time?
The idea is to concatenate pattern and text, and create a string “PT”wherePispattern, is a special character should not be present in pattern and text, and T is text. Build the Z array for concatenated string. In Z array, if Z value at any point is equal to pattern length, then pattern is present at that point.

```
Example:
Pattern P = "aab",  Text T = "baabaa"

The concatenated string is = "aab$baabaa"
```

```
Z array for above concatenated string is {x, 1, 0, 0, 0,
                                          3, 1, 0, 2, 1}.
Since length of pattern is 3, the value 3 in Z array
indicates presence of pattern.
```

How to construct a Z array?

A Simple Solution is to run two nested loops, the outer loop goes to every index and the inner loop finds length of the longest prefix that matches the substring starting at the current index. The time complexity of this solution is $O(n^2)$.

We can construct a Z array in linear time.

```
The idea is to maintain an interval [L, R] which is the interval
with max R
such that [L,R] is prefix substring (substring which is also
prefix).

Steps for maintaining this interval are as follows -

1) If i > R then there is no prefix substring that starts before
i and
   ends after i, so we reset L and R and compute new [L,R] by
comparing
   str[0..] to str[i..] and get Z[i] (= R-L+1).

2) If i <= R then let K = i-L,  now Z[i] >= min(Z[K], R-i+1)
because
   str[i..] matches with str[K..] for atleast R-i+1 characters
(they are in
   [L,R] interval which we know is a prefix substring).
   Now two sub cases arise -
      a) If Z[K] < R-i+1  then there is no prefix substring
starting at
         str[i] (otherwise Z[K] would be larger)  so  Z[i] =
Z[K]  and
         interval [L,R] remains same.
      b) If Z[K] >= R-i+1 then it is possible to extend the
[L,R] interval
         thus we will set L as i and start matching from str[R]
onwards  and
         get new R then we will update interval [L,R] and
calculate Z[i] (=R-L+1).
```

C++

```cpp
#include<iostream>
using namespace std;

void getZarr(string str, int Z[]);
```

```
void search(string text, string pattern){
	string concat = pattern + "$" + text;
	int l = concat.length();

	int Z[l];
	getZarr(concat, Z);

	for (int i = 0; i < l; ++i){
		if (Z[i] == pattern.length())
			cout << "Pattern found at index "
				<< i - pattern.length() -1 << endl;
	}
}

void getZarr(string str, int Z[]){
	int n = str.length();
	int L, R, k;

	L = R = 0;
	for (int i = 1; i < n; ++i){
		if (i > R){
			L = R = i;

			while (R<n && str[R-L] == str[R])
				R++;
			Z[i] = R-L;
			R--;
		}
		else{
			k = i-L;
			if (Z[k] < R-i+1)
				Z[i] = Z[k];

			else{
				L = i;
				while (R<n && str[R-L] == str[R])
					R++;
				Z[i] = R-L;
				R--;
			}
		}
	}
}

int main(){
	string text = "GEEKS FOR GEEKS";
	string pattern = "GEEK";
	search(text, pattern);
	return 0;
```

```
}
```

Range Query Problems

Sparse Table

LCA for n-ary Tree | Constant Query O(1)

We have seen various methods with different Time Complexities to calculate LCA in n-ary tree:-

ethod 1 : Naive Method (by calculating root to node path) | O(n) per query
Method 2 : Using Sqrt Decomposition | O(sqrt H)
Method 3 : Using Sparse Matrix DP approach | O(logn)

Lets study another method that has faster query time than all the above methods. So, our aim will be to calculate LCA in **constant time ~ O(1)**. Let's see how we can achieve it.

Method 4 : Using Range Minimum Query

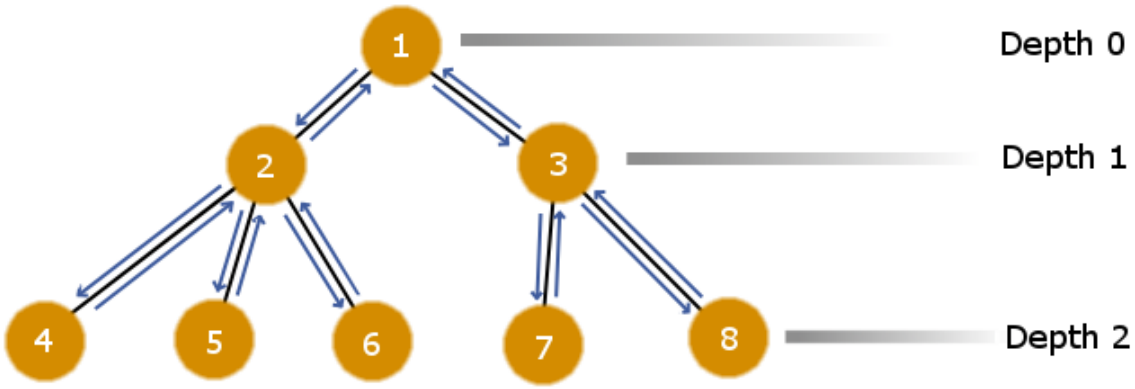
We have discussed LCA and RMQ for binary tree. Here we discuss LCA problem to RMQ problem conversion for n-ary tree.

Pre-requisites:- LCA in Binary Tree using RMQ
RMQ using sparse table

Key Concept : In this method, we will be reducing our LCA problem to RMQ(Range Minimum Query) problem over a static array. Once, we do that then we will relate the Range minimum queries to the required LCA queries.

The first step will be to decompose the tree into a flat linear array. To do this we can apply the Euler walk. The Euler walk will give the pre-order traversal of the graph. So we will perform a Euler Walk on the tree and store the nodes in an array as we visit them. This process reduces the tree data-structure to a simple linear array.

Consider the below tree and the euler walk over it:-



Euler Walk for the above tree

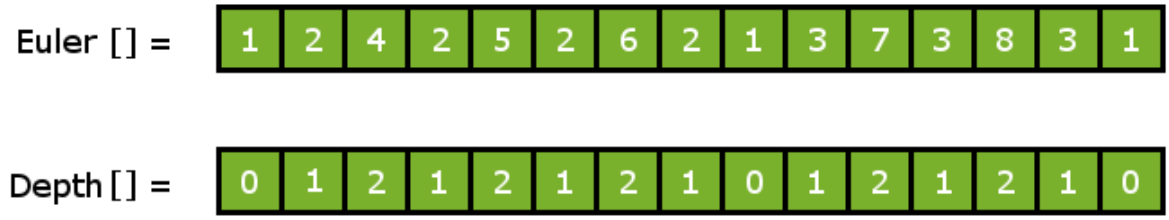
Euler [ ] =

1	2	4	2	5	2	6	2	1	3	7	3	8	3	1

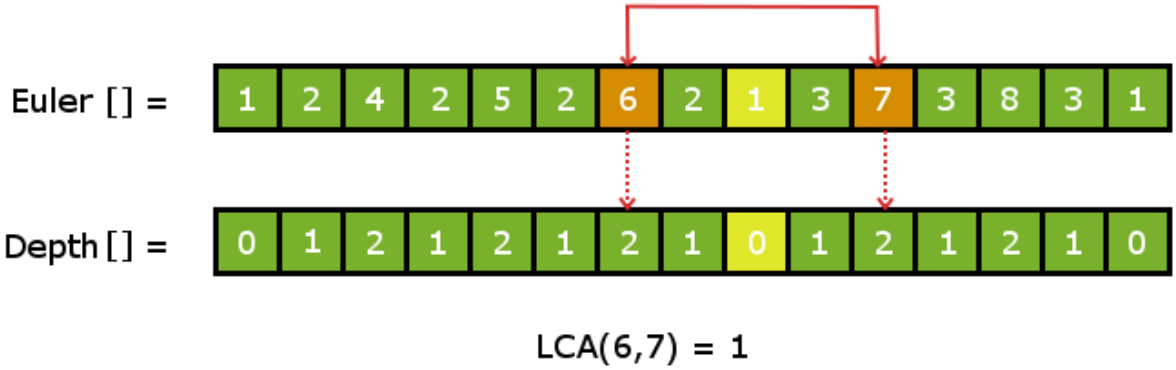
Now lets think in general terms : Consider any two nodes on the tree. There will be exactly one path connecting both the nodes and the node that has the smallest depth value in the path will be the LCA of the two given nodes.
Now take any two distinct node say **u** and **v** in the Euler walk array. Now all the elements in the path from u to v will lie in between the index of nodes u and v in the Euler walk array. Therefore, we just need to calculate the node with the minimum depth between the index of node u and node v in the euler array.

For this we will maintain another array that will contain the depth of all the nodes corresponding to their position in the Euler walk array so that we can Apply our RMQ algorithm on it.

Given below is the euler walk array parallel to its depth track array.



Example :- Consider two nodes **node 6** and **node 7** in the euler array. To calculate the LCA of node 6 and node 7 we look the smallest depth value for all the nodes in between node 6 and node 7 .
Therefore, **node 1** has the smallest *depth value = 0* and hence, it is the LCA for node 6 and node 7.



Implementation:-

We will be maintaining three arrays
1) Euler Path
2) Depth array
3) First Appearance Index

Euler Path and Depth array are the same as described above

First Appearance Index FAI[] : The First Appearance index Array will store the index for the first position of every node in the Euler Path array. FAI[i] = First appearance of ith node in Euler Walk array.

The Implementation for the above method is given below:-

Implementation:

C++Java

```cpp
// C++ program to demonstrate LCA of n-ary tree
// in constant time.
#include "bits/stdc++.h"
using namespace std;
#define sz 101

vector < int > adj[sz];     // stores the tree
vector < int > euler;       // tracks the eulerwalk
vector < int > depthArr;    // depth for each node corresponding
                            // to eulerwalk

int FAI[sz];      // stores first appearance index of every node
int level[sz];    // stores depth for all nodes in the tree
int ptr;          // pointer to euler walk
int dp[sz][18];   // sparse table
int logn[sz];     // stores log values
int p2[20];       // stores power of 2

void buildSparseTable(int n)
{
    // initializing sparse table
    memset(dp,-1,sizeof(dp));

    // filling base case values
    for (int i=1; i<n; i++)
        dp[i-1][0] = (depthArr[i]>depthArr[i-1])?i-1:i;

    // dp to fill sparse table
    for (int l=1; l<15; l++)
      for (int i=0; i<n; i++)
        if (dp[i][l-1]!=-1 and dp[i+p2[l-1]][l-1]!=-1)
          dp[i][l] =
            (depthArr[dp[i][l-1]]>depthArr[dp[i+p2[l-1]][l-1]])?
             dp[i+p2[l-1]][l-1] : dp[i][l-1];
        else
             break;
}

int query(int l,int r)
{
    int d = r-l;
    int dx = logn[d];
    if (l==r) return l;
    if (depthArr[dp[l][dx]] > depthArr[dp[r-p2[dx]][dx]])
        return dp[r-p2[dx]][dx];
    else
        return dp[l][dx];
}
```

```
void preprocess()
{
    // memorizing powers of 2
    p2[0] = 1;
    for (int i=1; i<18; i++)
        p2[i] = p2[i-1]*2;

    // memorizing all log(n) values
    int val = 1,ptr=0;
    for (int i=1; i<sz; i++)
    {
        logn[i] = ptr-1;
        if (val==i)
        {
            val*=2;
            logn[i] = ptr;
            ptr++;
        }
    }
}

/**
 * Euler Walk ( preorder traversal)
 * converting tree to linear depthArray
 * Time Complexity : O(n)
 * */
void dfs(int cur,int prev,int dep)
{
    // marking FAI for cur node
    if (FAI[cur]==-1)
        FAI[cur] = ptr;

    level[cur] = dep;

    // pushing root to euler walk
    euler.push_back(cur);

    // incrementing euler walk pointer
    ptr++;

    for (auto x:adj[cur])
    {
        if (x != prev)
        {
            dfs(x,cur,dep+1);

            // pushing cur again in backtrack
            // of euler walk
```

```
            euler.push_back(cur);

            // increment euler walk pointer
            ptr++;
        }
    }
}

// Create Level depthArray corresponding
// to the Euler walk Array
void makeArr()
{
    for (auto x : euler)
        depthArr.push_back(level[x]);
}

int LCA(int u,int v)
{
    // trivial case
    if (u==v)
       return u;

    if (FAI[u] > FAI[v])
       swap(u,v);

    // doing RMQ in the required range
    return euler[query(FAI[u], FAI[v])];
}

void addEdge(int u,int v)
{
    adj[u].push_back(v);
    adj[v].push_back(u);
}

int main(int argc, char const *argv[])
{
    // constructing the described tree
    int numberOfNodes = 8;
    addEdge(1,2);
    addEdge(1,3);
    addEdge(2,4);
    addEdge(2,5);
    addEdge(2,6);
    addEdge(3,7);
    addEdge(3,8);

    // performing required precalculations
    preprocess();
```

```
    // doing the Euler walk
    ptr = 0;
    memset(FAI,-1,sizeof(FAI));
    dfs(1,0,0);

    // creating depthArray corresponding to euler[]
    makeArr();

    // building sparse table
    buildSparseTable(depthArr.size());

    cout << "LCA(6,7) : " << LCA(6,7) << "\n";
    cout << "LCA(6,4) : " << LCA(6,4) << "\n";

    return 0;
}
```

Output

```
LCA(6,7) : 1
LCA(6,4) : 2
```

Note : We are precalculating all the required power of 2's and also precalculating the all the required log values to ensure constant time complexity per query. Else if we did log calculation for every query operation our Time complexity would have not been constant.

Time Complexity: The Conversion process from LCA to RMQ is done by Euler Walk that takes **O(n)** time.
Pre-processing for the sparse table in RMQ takes O(nlogn) time and answering each Query is a Constant time process. Therefore, overall Time Complexity is O(nlogn) - preprocessing and **O(1)** for each query.

This article is contributed by **Nitish Kumar**. If you like GeeksforGeeks and would like to contribute, you can also write an article using write.geeksforgeeks.org or mail your article to review-team@geeksforgeeks.org. See your article appearing on the GeeksforGeeks main page and help other Geeks.

Range Query - Module Walkthrough

Range Query Introduction

We have an array arr[0 . . . n-1]. We should be able to efficiently find the minimum value from index L (query start) to R (query end) where 0 <= **L** <= **R** <= **n-1**. Consider a situation when there are many range queries.

Example:

```
Input:  arr[]   = {7, 2, 3, 0, 5, 10, 3, 12, 18};
        query[] = [0, 4], [4, 7], [7, 8]

Output: Minimum of [0, 4] is 0
        Minimum of [4, 7] is 3
        Minimum of [7, 8] is 12
```

A **simple solution** is to run a loop from **L** to **R** and find the minimum element in the given range. This solution takes **O(n)** time to query in the worst case.

Another approach is to use a **Segment tree**. With the segment tree, preprocessing time is O(n) and the time to for range minimum query is **O(Logn)**. The extra space required is O(n) to store the segment tree. The segment tree allows updates also in **O(Log n)** time.

Sparse Tables - Introduction RMQ

Range Query Introduction

We have an array arr[0 . . . n-1]. We should be able to efficiently find the minimum value from index L (query start) to R (query end) where 0 <= **L** <= **R** <= **n-1**. Consider a situation when there are many range queries.

Example:

```
Input:  arr[]   = {7, 2, 3, 0, 5, 10, 3, 12, 18};
        query[] = [0, 4], [4, 7], [7, 8]

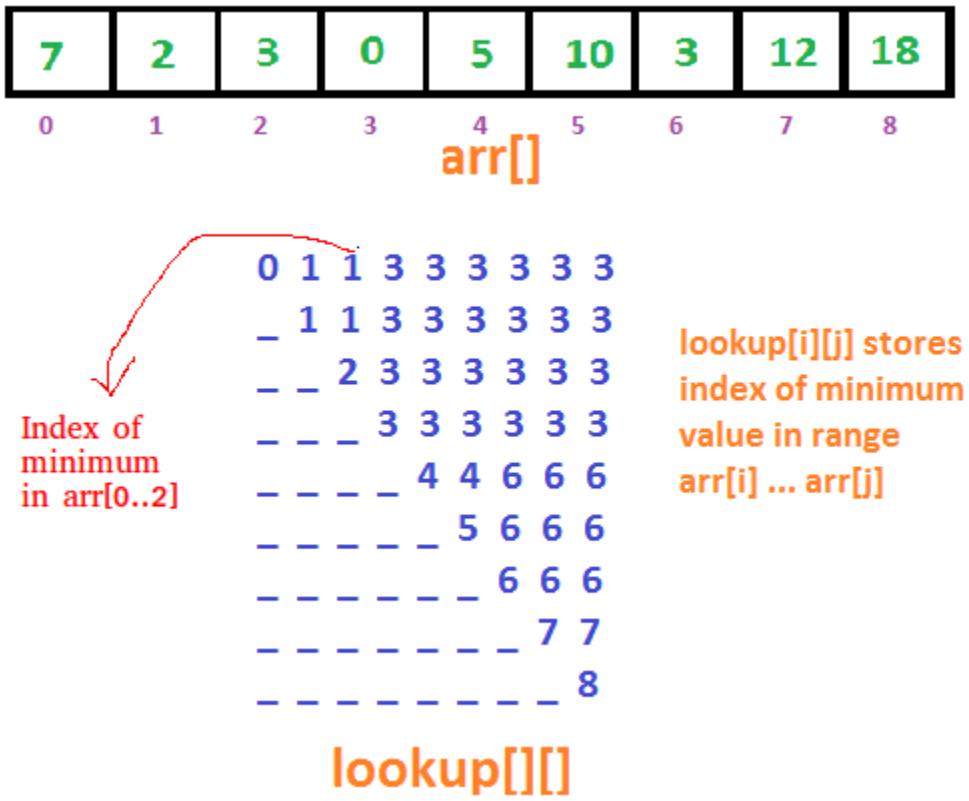
Output: Minimum of [0, 4] is 0
        Minimum of [4, 7] is 3
        Minimum of [7, 8] is 12
```

A **simple solution** is to run a loop from **L** to **R** and find the minimum element in the given range. This solution takes **O(n)** time to query in the worst case.

Another approach is to use a **Segment tree**. With the segment tree, preprocessing time is O(n) and the time to for range minimum query is **O(Logn)**. The extra space required is O(n) to store the segment tree. The segment tree allows updates also in **O(Log n)** time.

Can we do better if we know that the array is static?
How to optimize query time when there are no update operations and there are many range minimum queries?
Below are different methods.

Method 1 (Simple Solution)
A Simple Solution is to create a 2D array lookup[][] where an entry lookup[i][j] stores the minimum value in range arr[i..j]. The minimum of a given range can now be calculated in O(1) time.



C++

```
#include <bits/stdc++.h>
using namespace std;
#define MAX 500

int lookup[MAX][MAX];

struct Query {
        int L, R;
};
```

```
void preprocess(int arr[], int n){
	for (int i = 0; i < n; i++)
		lookup[i][i] = i;

	for (int i = 0; i < n; i++) {
		for (int j = i + 1; j < n; j++)
			if (arr[lookup[i][j - 1]] < arr[j])
				lookup[i][j] = lookup[i][j - 1];
			else
				lookup[i][j] = j;
	}
}

void RMQ(int arr[], int n, Query q[], int m){
	preprocess(arr, n);

	for (int i = 0; i < m; i++){
		int L = q[i].L, R = q[i].R;

		cout << "Minimum of [" << L
			<< ", " << R << "] is "
			<< arr[lookup[L][R]] << endl;
	}
}

int main(){
	int a[] = { 7, 2, 3, 0, 5, 10, 3, 12, 18 };
	int n = sizeof(a) / sizeof(a[0]);
	Query q[] = { { 0, 4 }, { 4, 7 }, { 7, 8 } };
	int m = sizeof(q) / sizeof(q[0]);
	RMQ(a, n, q, m);
	return 0;
}
```

This approach supports queries in **O(1)**, but preprocessing takes **$O(n^2)$** time. Also, this approach needs **$O(n^2)$** extra space which may become huge for large input arrays.

Method 2 (Square Root Decomposition)

We can use Square Root Decompositions to reduce space required in the above method.

Preprocessing:

1) Divide the range [0, n-1] into different blocks of $\sqrt{n}$ each.

2) Compute the minimum of every block of size $\sqrt{n}$ and store the results.

Preprocessing takes **$O(\sqrt{n} * \sqrt{n}) = O(n)$** time and **$O(\sqrt{n})$** space.





lookup[0] is index of minimum in arr[0..2],

lookup[1] is index minimum of arr[3..5]

lookup[2] is index minimum of arr[6..8]

Query:

1) To query a range [L, R], we take a minimum of all blocks that lie in this range. For left and right corner blocks that may partially overlap with the given range, we linearly scan them to find the minimum.

The time complexity of the query is $\mathbf{O(\sqrt{n})}$. Note that we have a minimum of the middle block directly accessible and there can be at most $\mathbf{O(\sqrt{n})}$ middle blocks. There can be at most two corner blocks that we may have to scan, so we may have to scan $\mathbf{2*O(\sqrt{n})}$ elements of corner blocks. Therefore, the overall time complexity is $\mathbf{O(\sqrt{n})}$.

Method 3 (Sparse Table Algorithm)

The above solution requires only O(√n) space but takes O(√n) time to query. The sparse table method supports query time **O(1)** with extra space **O(n Log n)**.

The idea is to precompute a minimum of all subarrays of size $\mathbf{2^j}$ where j varies from 0 to **Log n**. Like method 1, we make a lookup table. Here lookup[i][j] contains a minimum of range starting from i and of size 2^j. For example, lookup[0][3] contains a minimum of range [0, 7] (starting with 0 and of size 2^3)

Preprocessing:

How to fill this lookup table? The idea is simple, fill in a bottom-up manner using previously computed values.

For example, to find a minimum of range [0, 7], we can use a minimum of the following two.

a) Minimum of range [0, 3]

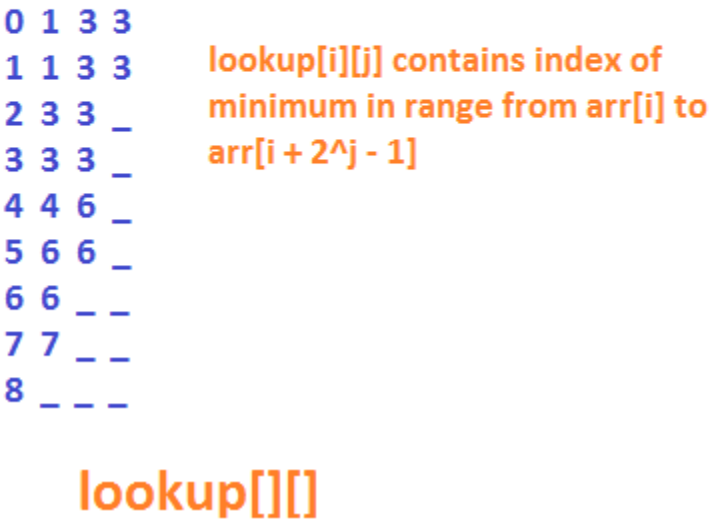
b) Minimum of range [4, 7]

Based on the above example, below is the formula,

```
// If arr[lookup[0][2]] <=  arr[lookup[4][2]],
// then lookup[0][3] = lookup[0][2]
If arr[lookup[i][j-1]] <= arr[lookup[i+2^(j-1)][j-1]]
   lookup[i][j] = lookup[i][j-1]

// If arr[lookup[0][2]] >  arr[lookup[4][2]],
// then lookup[0][3] = lookup[4][2]
Else
   lookup[i][j] = lookup[i+2^(j-1)][j-1]
```





Query:

For any arbitrary range [l, R], we need to use ranges that are in powers of 2. The idea is to use the closest power of 2. We always need to do at most one comparison (compare a minimum of two ranges which are powers of 2). One range starts with L and ends with "L + closest-power-of-2". The other range ends at R and starts with "R – same-closest-power-of-2 + 1". For example, if the given range is (2, 10), we compare a minimum of two ranges (2, 9) and (3, 10).

Based on the above example, below is the formula,

```
// For (2,10), j = floor(Log2(10-2+1)) = 3
j = floor(Log(R-L+1))

// If arr[lookup[0][3]] <=  arr[lookup[3][3]],
// then RMQ(2,10) = lookup[0][3]
If arr[lookup[L][j]] <= arr[lookup[R-(int)pow(2,j)+1][j]]
   RMQ(L, R) = lookup[L][j]

// If arr[lookup[0][3]] >  arr[lookup[3][3]],
// then RMQ(2,10) = lookup[3][3]
Else
   RMQ(L, R) = lookup[R-(int)pow(2,j)+1][j]
```

Since we do only one comparison, the time complexity of the query is O(1).

Below is the implementation of the above idea.

C++

```
#include <bits/stdc++.h>
using namespace std;
#define MAX 500

int lookup[MAX][MAX];

struct Query {
	int L, R;
};

void preprocess(int arr[], int n){
	for (int i = 0; i < n; i++)
		lookup[i][0] = i;

	for (int j = 1; (1 << j) <= n; j++){
		for (int i = 0; (i + (1 << j) - 1) < n; i++){
			if (arr[lookup[i][j - 1]]
				< arr[lookup[i + (1 << (j - 1))][j - 1]])
				lookup[i][j] = lookup[i][j - 1];
			else
				lookup[i][j]
					= lookup[i + (1 << (j - 1))][j - 1];
		}
	}
}

int query(int arr[], int L, int R){
	int j = (int)log2(R - L + 1);

	if (arr[lookup[L][j]]
```

```
                <= arr[lookup[R - (1 << j) + 1][j]])
                return arr[lookup[L][j]];

        else
                return arr[lookup[R - (1 << j) + 1][j]];
}

void RMQ(int arr[], int n, Query q[], int m){
        preprocess(arr, n);

        for (int i = 0; i < m; i++){
                int L = q[i].L, R = q[i].R;

                cout << "Minimum of [" << L << ", "
                        << R << "] is "
                        << query(arr, L, R) << endl;
        }
}

int main(){
        int a[] = { 7, 2, 3, 0, 5, 10, 3, 12, 18 };
        int n = sizeof(a) / sizeof(a[0]);
        Query q[] = { { 0, 4 }, { 4, 7 }, { 7, 8 } };
        int m = sizeof(q) / sizeof(q[0]);
        RMQ(a, n, q, m);
        return 0;
}
```

So sparse table method supports query operation in O(1) time with O(n Log n) preprocessing time and O(n Log n) space.

Sparse Tables

The sparse table concept is used for fast queries on a set of static data (elements do not change). It does preprocessing so that the queries can be answered efficiently.

Sparse Tables | Problem 1

Example Problem 1 : Range Minimum Query

We have an array arr[0 . . . n-1]. We need to efficiently find the minimum value from index L (query start) to R (query end) where 0 <= **L** <= **R** <= **n-1**. Consider a situation when there

are many range queries.

Example:

```
Input:  arr[]   = {7, 2, 3, 0, 5, 10, 3, 12, 18};
        query[] = [0, 4], [4, 7], [7, 8]

Output: Minimum of [0, 4] is 0
        Minimum of [4, 7] is 3
        Minimum of [7, 8] is 12
```

The idea is to precompute minimum of all subarrays of size 2^j where j varies from 0 to **Log n**. We make a table lookup[i][j] such that lookup[i][j] contains minimum of range starting from i and of size 2^j. For example lookup[0][3] contains minimum of range [0, 7] (starting with 0 and of size 2^3)

How to fill this lookup or sparse table?

The idea is simple, fill in a bottom-up manner using previously computed values. We compute ranges with current power of 2 using values of lower power of two. For example, to find a minimum of range [0, 7] (Range size is a power of 3), we can use the minimum of following two.

a) Minimum of range [0, 3] (Range size is a power of 2)

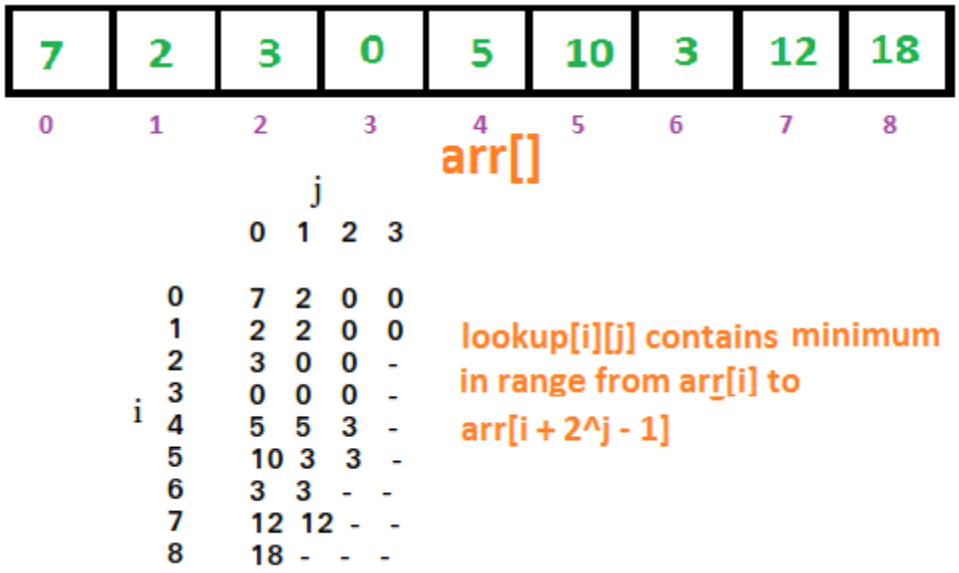
b) Minimum of range [4, 7] (Range size is a power of 2)

Based on above example, below is formula,

```
// Minimum of single element subarrays is same
// as the only element.
lookup[i][0] = arr[i]

// If lookup[0][2] <=  lookup[4][2],
// then lookup[0][3] = lookup[0][2]
If lookup[i][j-1] <= lookup[i+2^(j-1)][j-1]
   lookup[i][j] = lookup[i][j-1]

// If lookup[0][2] >  lookup[4][2],
// then lookup[0][3] = lookup[4][2]
Else
   lookup[i][j] = lookup[i+2^(j-1)][j-1]
```



For any arbitrary range [l, R], we need to use ranges which are in powers of 2. The idea is to use the closest power of 2. We always need to do at most one comparison (compare the minimum of two ranges which are powers of 2). One range starts with L and ends with "L + closest-power-of-2". The other range ends at R and starts with "R – same-closest-power-of-2 + 1". For example, if the given range is (2, 10), we compare the minimum of two ranges (2, 9) and (3, 10).
Based on above example, below is formula,

```
// For (2, 10), j = floor(Log2(10-2+1)) = 3
j = floor(Log(R-L+1))

// If lookup[2][3] <=  lookup[3][3],
// then RMQ(2, 10) = lookup[2][3]
If lookup[L][j] <= lookup[R-(int)pow(2, j)+1][j]
   RMQ(L, R) = lookup[L][j]

// If lookup[2][3] >  arr[lookup[3][3],
// then RMQ(2, 10) = lookup[3][3]
Else
   RMQ(L, R) = lookup[R-(int)pow(2, j)+1][j]
```

Since we do only one comparison, the time complexity of the query is O(1).
Below is the implementation of the above idea.

C++

```
#include <bits/stdc++.h>
using namespace std;
#define MAX 500
```

```
int lookup[MAX][MAX];

void buildSparseTable(int arr[], int n){
	for (int i = 0; i < n; i++)
		lookup[i][0] = arr[i];

	for (int j = 1; (1 << j) <= n; j++) {
		for (int i = 0; (i + (1 << j) - 1) < n; i++) {
			if (lookup[i][j - 1] <
					lookup[i + (1 << (j - 1))][j -
1])
				lookup[i][j] = lookup[i][j - 1];
			else
				lookup[i][j] =
					lookup[i + (1 << (j - 1))][j -
1];
		}
	}
}

int query(int L, int R){
	int j = (int)log2(R - L + 1);

	if (lookup[L][j] <= lookup[R - (1 << j) + 1][j])
		return lookup[L][j];

	else
		return lookup[R - (1 << j) + 1][j];
}

int main(){
	int a[] = { 7, 2, 3, 0, 5, 10, 3, 12, 18 };
	int n = sizeof(a) / sizeof(a[0]);
	buildSparseTable(a, n);
	cout << query(0, 4) << endl;
	cout << query(4, 7) << endl;
	cout << query(7, 8) << endl;
	return 0;
}
```

Sparse Tables | Problem 2

Example Problem 2: Range GCD Query

We have an array arr[0 . . . n-1]. We need to find the greatest common divisor in the range L and R where 0 <= L <= R <= n-1. Consider a situation when there are many range queries Examples:

```
Input : arr[] = {2, 3, 5, 4, 6, 8}
        queries[] = {(0, 2), (3, 5), (2, 3)}
Output : 1
         2
         1
```

We use below properties of GCD:

- GCD function is associative [GCD(a, b, c) = GCD(GCD(a, b), c) = GCD(a, GCD(b, c))], we can compute GCD of a range using GCDs of subranges.
- If we take the GCD of an overlapping range more than once, then it does not change the answer. For example GCD(a, b, c) = GCD(GCD(a, b), GCD(b, c)). Therefore like the minimum range query problem, we need to do only one comparison to find the GCD of the given range.

We build a sparse table using the same logic as above. After building the sparse table, we can find all GCDs by breaking the given range in powers of 2 and adding the GCD of every piece to the current answer.

C++

```
#include <bits/stdc++.h>
using namespace std;
#define MAX 500

int table[MAX][MAX];

// it builds sparse table.
void buildSparseTable(int arr[], int n){
	// GCD of single element is element itself
	for (int i = 0; i < n; i++)
		table[i][0] = arr[i];

	// Build sparse table
	for (int j = 1; j <= log2(n); j++)
		for (int i = 0; i <= n - (1 << j); i++)
			table[i][j] = __gcd(table[i][j - 1],
					table[i + (1 << (j - 1))][j - 1]);
}

int query(int L, int R){
	int j = (int)log2(R - L + 1);

	return __gcd(table[L][j], table[R - (1 << j) + 1][j]);
```

```
}

int main(){
        int a[] = { 7, 2, 3, 0, 5, 10, 3, 12, 18 };
        int n = sizeof(a) / sizeof(a[0]);
        buildSparseTable(a, n);
        cout << query(0, 2) << endl;
        cout << query(1, 3) << endl;
        cout << query(4, 5) << endl;
        return 0;
}
```

Time Complexity: O(n*Logn)

Auxiliary Space: O(n*Logn)

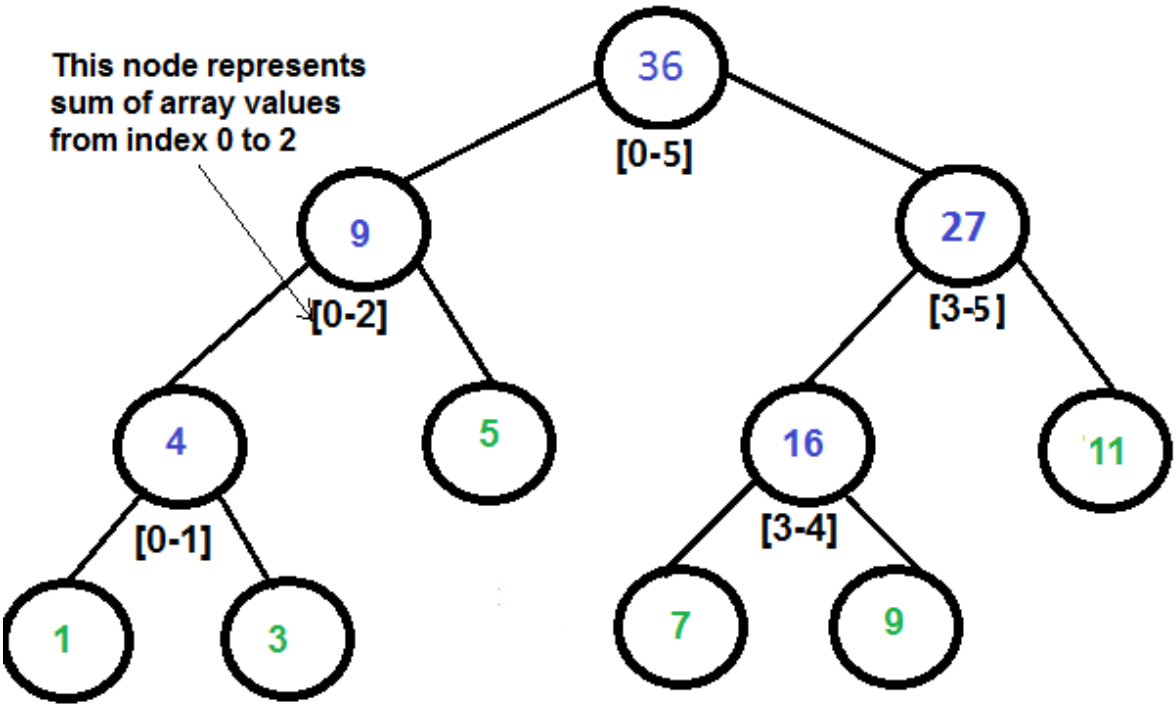
Segment Trees

Segment Trees | Introduction

Segment Trees Introduction

In computer science, a **Segment Tree**, also known as a statistic tree, is a tree data structure used for storing information about intervals, or segments. It allows querying which of the stored segments contain a given point. It is, in principle, a static structure; that is, it's a structure that cannot be modified once it's built. A similar data structure is the interval tree.

A **segment tree** for a set I of n intervals uses O(n log n) storage and can be built in O(n log n) time. Segment trees support searching for all the intervals that contain a query point in time O(log n + k), k being the number of retrieved intervals or segments.



Segment Tree for input array {1, 3, 5, 7, 9, 11}

Persistent Segment Tree

Segment Tree is itself a great data structure that comes into play in many cases. Persistency simply means retaining the changes. But obviously, retaining the changes cause extra memory consumption and hence affects the Time Complexity.

Our aim is to apply persistency in the segment tree and also to ensure that it does not take more than **O(log n) time and space** for each change.

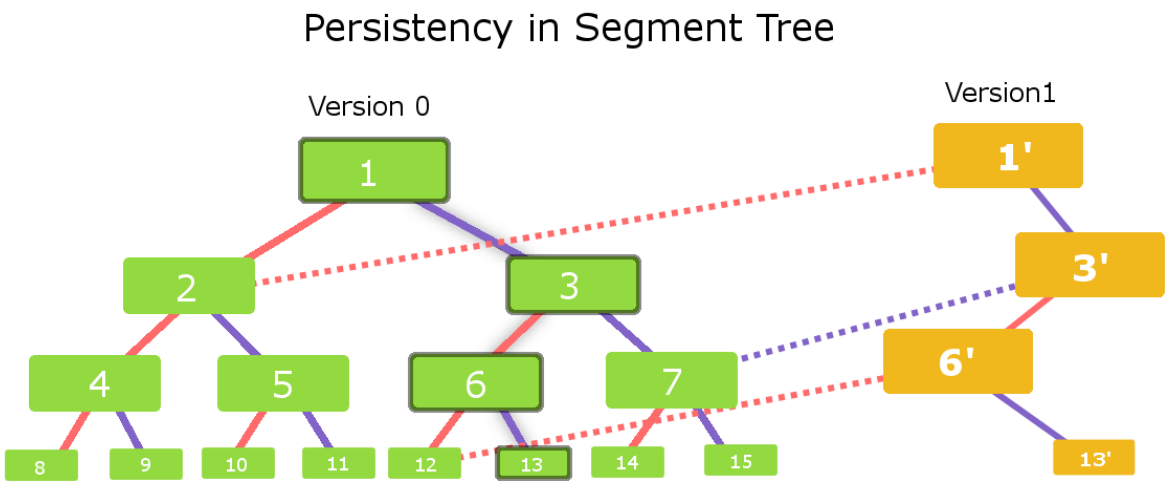
Let's think in terms of versions i.e. for each change in our segment tree we create a new version of it.
We will consider our initial version to be Version 0. Now, as we do any update in the segment tree we will create a new version for it and in a similar fashion track the record for all versions.

But creating the whole tree for every version will take O(n log n) extra space and O(n log n) time. So, this idea runs out of time and memory for a large number of versions.

Let's exploit the fact that for each new update(say point update for simplicity) in the segment tree, At max logn nodes will be modified. So, our new version will only contain

these log n new nodes and the rest nodes will be the same as the previous version. Therefore, it is quite clear that for each new version, we only need to create these log n new nodes whereas the rest of the nodes can be shared from the previous version.

Consider the below figure for better visualization(click on the image for a better view):-



Consider the segment tree with green nodes . Lets call this segment tree as **version-0**. The left child for each node is connected with solid red edge where as the right child for each node is connected with solid purple edge. Clearly, this segment tree consists of 15 nodes.

Now consider we need to make change in the leaf node 13 of version-0.
So, the affected nodes will be – **node 13 , node 6 , node 3 , node 1**.
Therefore, for the new version **(Version-1)** we need to create only these **4 new nodes**.

Now, lets construct version-1 for this change in segment tree. We need a new node 1 as it is affected by change done in node 13. So , we will first create a new **node 1′**(yellow color) . The left child for node 1′ will be the same for left child for node 1 in version-0. So, we connect the left child of node 1′ with node 2 of version-0(red dashed line in figure). Let's now examine the right child for node 1′ in version-1. We need to create a new node as it is affected . So we create a new node called node 3′ and make it the right child for node 1′(solid purple edge connection).

In the similar fashion we will now examine for **node 3′**. The left child is affected , So we create a new node called **node 6′** and connect it with solid red edge with node 3′ , where as the right child for node 3′ will be the same as right child of node 3 in version-0. So, we will

make the right child of node 3 in version-0 as the right child of node 3′ in version-1(see the purple dash edge.)

Same procedure is done for node 6′ and we see that the left child of node 6′ will be the left child of node 6 in version-0(red dashed connection) and right child is newly created node called **node 13′**(solid purple dashed edge).
Each **yellow color node** is a newly created node and dashed edges are the inter-connection between the different versions of the segment tree.

Now, the Question arises : **How to keep track of all the versions?**
– We only need to keep track the first root node for all the versions and this will serve the purpose to track all the newly created nodes in the different versions. For this purpose we can maintain an array of pointers to the first node of segment trees for all versions.

Let's consider a very basic problem to see how to implement persistence in segment tree

Problem : Given an array A[] and different point update operations.Considering
each point operation to create a new version of the array. We need to answer
the queries of type
Q v l r : output the sum of elements in range l to r just after the v-th update.
We will create all the versions of the segment tree and keep track of their root node.Then for each range sum query we will pass the required version's root node in our query function and output the required sum.

C++

```
#include "bits/stdc++.h"
using namespace std;

#define MAXN 100

struct node{
        int val;

        node* left, *right;

        node() {}
        node(node* l, node* r, int v)
        {
                left = l;
                right = r;
                val = v;
```

```
        }
};

int arr[MAXN];

node* version[MAXN];

void build(node* n,int low,int high){
        if (low==high){
                n->val = arr[low];
                return;
        }
        int mid = (low+high) / 2;
        n->left = new node(NULL, NULL, 0);
        n->right = new node(NULL, NULL, 0);
        build(n->left, low, mid);
        build(n->right, mid+1, high);
        n->val = n->left->val + n->right->val;
}

void upgrade(node* prev, node* cur, int low, int high, int idx, int
value){
        if (idx > high or idx < low or low > high)
                return;

        if (low == high){
                cur->val = value;
                return;
        }
        int mid = (low+high) / 2;
        if (idx <= mid){
                cur->right = prev->right;

                cur->left = new node(NULL, NULL, 0);

                upgrade(prev->left,cur->left, low, mid, idx, value);
        }
        else{
                cur->left = prev->left;

                cur->right = new node(NULL, NULL, 0);

                upgrade(prev->right, cur->right, mid+1, high, idx, value);
        }

        cur->val = cur->left->val + cur->right->val;
}

int query(node* n, int low, int high, int l, int r){
```

```
        if (l > high or r < low or low > high)
        return 0;
        if (l <= low and high <= r)
        return n->val;
        int mid = (low+high) / 2;
        int p1 = query(n->left,low,mid,l,r);
        int p2 = query(n->right,mid+1,high,l,r);
        return p1+p2;
}

int main(int argc, char const *argv[]){
        int A[] = {1,2,3,4,5};
        int n = sizeof(A)/sizeof(int);

        for (int i=0; i<n; i++)
        arr[i] = A[i];

        node* root = new node(NULL, NULL, 0);
        build(root, 0, n-1);

        version[0] = root;

        version[1] = new node(NULL, NULL, 0);
        upgrade(version[0], version[1], 0, n-1, 4, 1);

        version[2] = new node(NULL, NULL, 0);
        upgrade(version[1],version[2], 0, n-1, 2, 10);

        cout << "In version 1 , query(0,4) : ";
        cout << query(version[1], 0, n-1, 0, 4) << endl;

        cout << "In version 2 , query(3,4) : ";
        cout << query(version[2], 0, n-1, 3, 4) << endl;

        cout << "In version 0 , query(0,3) : ";
        cout << query(version[0], 0, n-1, 0, 3) << endl;
        return 0;
}
```

Note: The above problem can also be solved by processing the queries offline by sorting it with respect to the version and answering the queries just after the corresponding update.

Time Complexity: The time complexity will be the same as the query and point update operation in the segment tree as we can consider the extra node creation step to be done in O(1). Hence, the overall Time Complexity per query for new version creation and range sum query will be **O(log n)**.

Auxiliary Space: O(log n)

Segment Tree Problem Solving

Range Sum and Update in Array: Segment Tree using Stack

Given an array **arr[]** of **N** integers. The task is to do the following operations:

1. Add a value **X** to all the elements from index **A** to **B** where **0 ≤ A ≤ B ≤ N-1**.
2. Find the sum of the element from index **L** to **R** where **0 ≤ L ≤ R ≤ N-1** before and after the update given to the array above.

Example:

Input: *arr[] = {1, 3, 5, 7, 9, 11}, L = 1, R = 3, A = 1, B = 5, X = 10*
Output:
Sum of values in given range = 15
Updated sum of values in given range = 45
Explanation:
*Sum of values in the range **1 to 3** is 3 + 5 + 7 = 15.*
*arr[] after adding **10** from index **1 to 5** is arr[] = {1, 13, 15, 17, 19, 21}*
*Sum of values in the range **1 to 3** after update is 13 + 15 + 17 = 45.*
Input: *arr[] = { 11, 32, 5, 7, 19, 11, 8}, L = 2, R = 6, A = 1, B = 5, X = 16*
Output:
Sum of values in given range = 50
Updated sum of values in given range = 114
Explanation:
*Sum of values in the range **2 to 6** is 5 + 7 + 19 + 11 + 8 = 50.*
*arr[] after adding **16** from index **1 to 5** is arr[] = {11, 48, 21, 23, 35, 27, 8}*
*Sum of values in the range **2 to 6** after update is 21 + 23 + 35 + 27 + 8 = 114.*

Below are the steps to implement Segment Tree using Stack:

1. The idea is to use tuple to store the state which has Node number and range indexes in the Stack.
2. **For Building Segment Tree:**
    - Push the root Node to the stack as a tuple:

```
Stack S;
start = 0, end = arr_size - 1
S.emplace(1, start, end)
```

- Pop the element from the stack and do the following until stack becomes empty:
    1. If starting index equals to ending index, then we reach the leaf node and update the value at Segment tree array as value at current index in the given array.
    2. Else insert the flag tuple with the current Node as **S.emplace(current_node, INF, INF)** to inverse the order of evaluation and insert the tuple with value for left and right child as:

mid = (start + end) / 2
*st.emplace(curr_node * 2, start, mid)*
*st.emplace(curr_node * 2 + 1, mid + 1, end)*

1. If start index and end index is same as **INF**, then update the Segment Tree value at current index as:

Value at current index is updated as value at left child and right child:
*tree[curr_node] = tree[2*curr_node] + tree[2*curr_node + 1]*

1. **For Update Tree:**
    - Push the root node to the stack as done for building Segment Tree.
    - Pop the element from the stack and do the following until stack becomes empty:
        1. If the current node has any pending update then first update to the current node.
        2. If the current node ranges lies completely in the update query range, then update the current node with that value.
        3. If the current node ranges overlap with the update query range, then follow the above approach and push the tuple for left child and right child in the Stack.
        4. Update the query using the result of left and right child above.
2. **For Update Query:**
    - Push the root node to the stack as done for building Segment Tree.
    - Pop the element from the stack and do the following until the stack becomes empty:
        1. If the current node ranges lie outside the given query, then continue with the next iteration.
        2. If the current node ranges lies completely in the update query range, then update the result with the current node value.
        3. If the current node ranges overlap with the update query range, then follow the above approach and push the tuple for left child and right child in the Stack.
        4. Update the result using the value obtained from left and right child Node.

Below is the implementation of the above approach:

C++

```
#include"bits/stdc++.h"
```

```cpp
using namespace std;

constexpr static int MAXSIZE = 1000;
constexpr static int INF = numeric_limits<int>::max();

// Segment Tree array
int64_t tree[MAXSIZE];

// Lazy Update array
int64_t lazy[MAXSIZE];

using QueryAdaptor
	= tuple<int64_t, int64_t, int64_t>;

// Build our segment tree
void build_tree(int64_t* arr, int64_t arr_size){

	// Stack will use to update
	// the tree value
	stack<QueryAdaptor> st;

	// Emplace the root of the tree
	st.emplace(1, 0, arr_size - 1);

	// Repeat until empty
	while (!st.empty()) {

		// Take the indexes at the
		// top of the stack
		int64_t currnode, curra, currb;

		// value at the top of the
		// stack
		tie(currnode, curra, currb) = st.top();

		// Pop the value from the
		// stack
		st.pop();

		// Flag with INF ranges are merged
		if (curra == INF && currb == INF) {
			tree[currnode] = tree[currnode * 2]
					+ tree[currnode * 2 +
1];
		}

		// Leaf node
		else if (curra == currb) {
			tree[currnode] = arr[curra];
```

```
                }

                else {

                        // Insert flag node inverse
                        // order of evaluation
                        st.emplace(currnode, INF, INF);
                        int64_t mid = (curra + currb) / 2;

                        // Push children
                        st.emplace(currnode * 2,
                                        curra, mid);
                        st.emplace(currnode * 2 + 1,
                                        mid + 1, currb);
                }
        }
}

inline void push_down(int64_t node, int64_t a, int64_t b){
        if (lazy[node] != 0) {
                tree[node] += lazy[node] * (b - a + 1);

                if (a != b) {
                        lazy[2 * node] += lazy[node];
                        lazy[2 * node + 1] += lazy[node];
                }

                lazy[node] = 0;
        }
}

void update_tree(int64_t arr_size, int64_t i, int64_t j, int64_t val){
        stack<QueryAdaptor> st;

        st.emplace(1, 0, arr_size - 1);

        while (!st.empty()) {
                int64_t currnode, curra, currb;
                tie(currnode, curra, currb) = st.top();
                st.pop();

                if (curra == INF && currb == INF) {
                        tree[currnode] = tree[currnode * 2]
                                                + tree[currnode * 2 +
1];
                }

                else {
                        push_down(currnode, curra, currb);
```

```
                        // No overlap condition
                        if (curra > currb || curra > j
                                || currb < i) {
                                continue;
                        }

                        else if (curra >= i && currb <= j){

                                tree[currnode] += val * (currb - curra + 1);

                                if (curra != currb) {
                                        lazy[currnode * 2] += val;
                                        lazy[currnode * 2 + 1] += val;
                                }
                        }

                        // Partial Overlap
                        else{
                                st.emplace(currnode, INF, INF);

                                int64_t mid = (curra + currb) / 2;

                                // Push children
                                st.emplace(currnode * 2,
                                                curra, mid);
                                st.emplace(currnode * 2 + 1,
                                                mid + 1, currb);
                        }
                }
        }
}

int64_t query(int64_t arr_size, int64_t i, int64_t j){
        stack<QueryAdaptor> st;

        st.emplace(1, 0, arr_size - 1);

        int64_t result = 0;

        while (!st.empty()){
                int64_t currnode, curra, currb;
                tie(currnode, curra, currb) = st.top();
                st.pop();

                push_down(currnode, curra, currb);

                if (curra > currb || curra > j
                        || currb < i) {
```

```
                        continue;
                }

                else if (curra >= i && currb <= j) {
                        result += tree[currnode];
                }

                else {
                        std::int64_t mid = (curra + currb) / 2;

                        st.emplace(currnode * 2,
                                        curra, mid);
                        st.emplace(currnode * 2 + 1,
                                        mid + 1, currb);
                }
        }

        return result;
}

int main(){
        memset(tree, 0, sizeof(int64_t) * MAXSIZE);
        memset(lazy, 0, sizeof(int64_t) * MAXSIZE);

        int64_t arr[] = { 1, 3, 5, 7, 9, 11 };
        int n = sizeof(arr) / sizeof(arr[0]);

        build_tree(arr, n);

        cout << "Sum of values in given range = "
                << query(n, 1, 3)
                << endl;

        update_tree(n, 1, 5, 10);

        cout << "Updated sum of values in given range = "
                << query(n, 1, 3)
                << endl;

        return 0;
}
```

Time Complexity:

- **For Tree Construction:** O(N), There are (2n-1) nodes in the tree and value of every node is calculated once.

- **For Query:** O(log N), To query a sum we processed atmost four nodes at every level and number of level is log N.
- **For Update:** O(log N), To update the tree with lazy propagation is O(Log N) as we update the root of the tree and then update only that part of the tree whose ranges overlaps at each level.

Merge Sort Tree

Merge Sort Tree

Introduction

Given an array where each element is a vector containing integers in sorted order. The task is to answer the following queries:

```
count(start, end, k) : Count the numbers smaller than or equal
                       to k in range from array index 'start'
                       to 'end'.
```

For convenience, we consider an n * n 2-D array where each row corresponds to an integer vector. Examples:

```
Input : ar[][] = {{2, 4, 5},
                  {3, 4, 9},
                  {6, 8, 10}}

        Queries[] = (0, 1, 5)
                    (1, 2, 1)
                    (0, 2, 6)
Output : 5
         0
         6
Count of elements (smaller than or equal to 5) from 1st row
(index 0) to 2nd row (index 1) is 5.
Count of elements (smaller than or equal to 1) from 2nd row to
3nd row is 0
Count of elements (smaller than or equal to 6) from 1st row to
3nd row is 6.
```

The key idea is to build a Segment Tree with a vector at every node and the vector contains all the elements of the sub-range in a sorted order. And if we observe this segment tree structure this is somewhat similar to the tree formed during the merge sort algorithm(that is why it is called merge sort tree).

C++

```
#include <bits/stdc++.h>
using namespace std;

const int MAX = 1000;

void buildTree(int idx, int ss, int se, vector<int> a[], vector<int>
sTree[]){
	if (ss == se) {
		sTree[idx] = a[ss];
		return;
	}

	int mid = (ss + se) / 2;

	buildTree(2 * idx + 1, ss, mid, a, sTree);

	buildTree(2 * idx + 2, mid + 1, se, a, sTree);

	merge(sTree[2 * idx + 1].begin(),
		sTree[2 * idx + 1].end(),
		sTree[2 * idx + 2].begin(),
		sTree[2 * idx + 2].end(),
		back_inserter(sTree[idx]));
}

int queryRec(int node, int start, int end, int ss, int se,
			int k, vector<int> a[], vector<int> sTree[]){
	if (ss > end || start > se)
		return 0;

	if (ss <= start && se >= end) {
		return upper_bound(sTree[node].begin(), sTree[node].end(),
k) - sTree[node].begin();
	}

	int mid = (start + end) / 2;

	int p1 = queryRec(2 * node + 1, start, mid, ss, se, k, a, sTree);

	int p2 = queryRec(2 * node + 2, mid + 1, end, ss, se, k, a, sTree);

	return p1 + p2;
}

int query(int start, int end, int k, vector<int> a[], int n, vector<int>
sTree[]){
	return queryRec(0, 0, n - 1, start, end, k, a, sTree);
```

```
}

// Driver code
int main(){
	int n = 3;
	int arr[][3]
		= { { 2, 4, 5 }, { 3, 4, 9 }, { 6, 8, 10 } };

	// build an array of vectors from above input
	vector<int> a[n];
	for (int i = 0; i < n; i++)
		for (int j = 0; j < n; j++)
			a[i].push_back(arr[i][j]);

	// Construct segment tree
	vector<int> sTree[MAX];
	buildTree(0, 0, n - 1, a, sTree);

	/* un-comment to print merge sort tree*/
	/*for (int i=0;i<2*n-1;i++)
	{
			cout << i << " ";
			for (int j=0;j<sTree[i].size();j++)
			cout << sTree[i][j]<<" ";
			cout << endl;
	}*/

	// Answer queries
	cout << query(0, 1, 5, a, n, sTree) << endl;
	cout << query(1, 2, 1, a, n, sTree) << endl;
	cout << query(0, 2, 6, a, n, sTree) << endl;

	return 0;
}
```

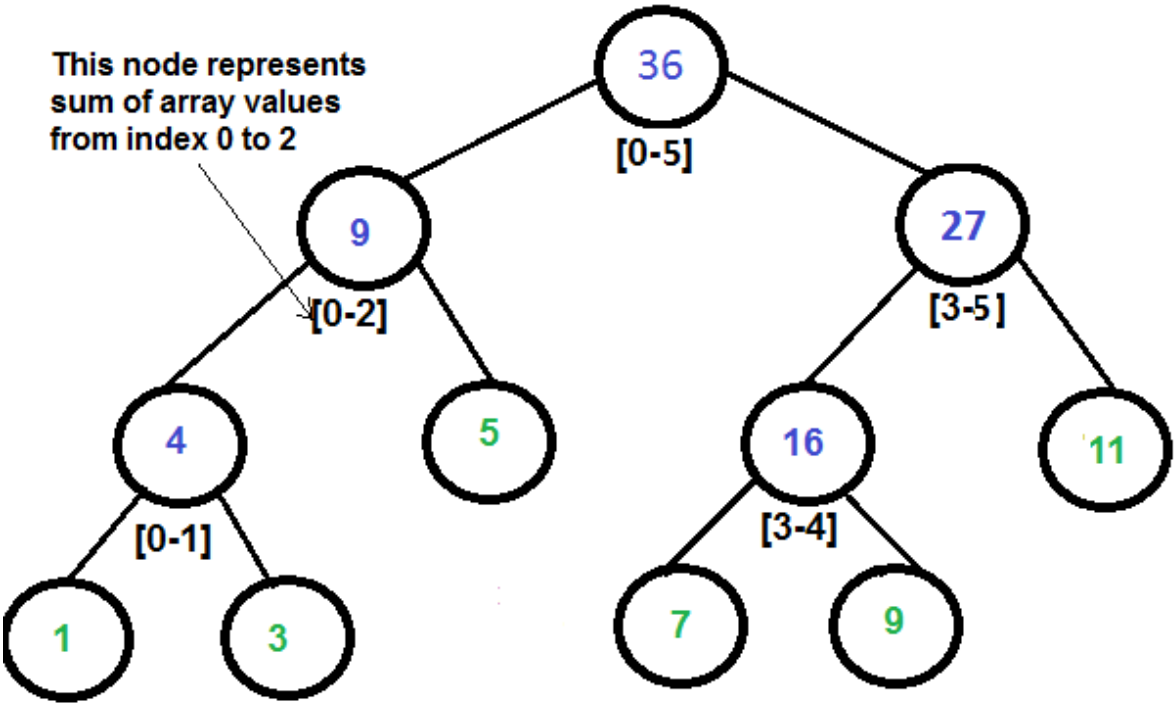
buildTree() analysis : Build a merge sort tree takes O(N log N) time which is same as Merge Sort Algorithm. It also takes O(n log n) extra space. **query() analysis :** A range 'start' to 'end' can divided into at most Log(n) parts, where we will perform binary search on each part . Binary search requires O(Log n). Therefore **total complexity** O(Log n * Log n).

The **space complexity** of the program is O(n log n) since the segment tree is constructed using an array of vectors with a maximum size of n log n.

Lazy Propagation

Lazy Propagation | Introduction

Lazy Propagation
We have used the same “Sum of given Range” problem to explain Lazy propagation



Segment Tree for input array {1, 3, 5, 7, 9, 11}

How does the update work in Simple Segment Tree?
Please note that a single value update in the array may cause multiple updates in Segment Tree as there may be many segment tree nodes that have a single array element in their ranges.
Below is the simple logic used:
1) Start with the root of the segment tree.
2) If the array index to be updated is not in the current node’s range, then return
3) Else update the current node and recur for children.

Segment Trees and Strings

Given a string S of length N, and M queries of the following type:

Type 1: *1 L x,*
Indicates update Lth index of string S by character 'x'.
Type 2: *2 L R str*
Find the number of subsets in range L to R
which is equal to the string str modulo 1000000007.
Constraints :
|S| <= 100000,
M <= 100000,
1 <= L, R <= |S|,
|str| <= 26,
All characters in str are unique,
S & str consisting of lowercase latin letters.

Examples:

Input : *N = 16, M = 3, S = "geeekkksgskeegks"*
type = 2: L = 1, R = 7, str = "gek"
type = 1: index = 2, character = 'x'
type = 2: L = 1, R = 7, str = "gek"
Output : *9 6 4*
query2 : No of subsets in string S i range [1…7] that is equal to ***gek*** *is 9.*
query1 : string S is changed to ***gxeekkksgskeegks*** *after second query.*
query2 : No of subsets in string S i range [1…7] that is equal to ***gek*** *is 6.*

Naive Approach :
Query type 1: We will calculate the frequency of each character of the query string in the range [L…R] and then multiply all the calculated frequencies to get the desired result.
Query type 2: We will replace the i'th character of the string with the given character.

Time complexity : 0(m*n)

Efficient Approach :

1. Using *Segment Tree* we can perform both operations in log(n) time. Every node of segment tree will contain frequency of characters in range [L..R].
2. The function **build** takes n*log(n) time to create a segment tree with every node containing the frequency of characters of some segment of the string.
3. The function **get** returns a vector containing frequency of all characters. Multiplication of all frequency of the given query string modulo 1e9+7 gives the desired result.
4. The function **update** decreases the frequency of character placed earlier and increases the frequency of new character present in the nodes of the segment tree by one.
5. The function **add_two_result** adds two vector and returns their result.

Below is the implementation of the above approach:

C++

```
// C++ implementation of the approach
#include<bits/stdc++.h>
using namespace std;

#define N 400005
#define mod 1000000007

vector <int> seg_tree[N];
string str;

// A recursive function that constructs
// Segment Tree for given string
void build(int pos, int st, int en)
{

        if (st == en)
        {
                // Increment the frequency of character
                // by one if st is equal to en
                seg_tree[pos][str[st - 1] - 'a']++;
                return ;
        }

        int mid = st + en >> 1;

        // Build the segment tree for range st to mid
        build(2 * pos, st, mid);

        // Build the segment tree for range mid+1 to en
        build(2 * pos + 1, mid + 1, en);

        // It stores addition of frequency of
        // characters of both of its child after
        // segment tree is build
```

```
        for (int i = 0; i < 26; i++)
                seg_tree[pos][i] = seg_tree[2 * pos + 1][i]
                                                + seg_tree[2 * pos][i];

}

// A utility function for
// addition of two vectors
vector <int> add_two_result(vector <int> v1,
                                                        vector <int> v2)
{
        vector <int> added_vec(26);

        // Adding two vector and storing
        // it in another vector
        for (int i = 0; i < 26; i++)
                added_vec[i] = v1[i] + v2[i];

        // Returning final vector
        return added_vec;

}

// A recursive function that return vector
// which contains frequency of every character
vector <int> get(int pos, int l, int r,
                                                int st, int en)
{
        // If segment of this node is
        // outside the given range
        if (l > en || r < st)
        {
                vector <int> empty(26, 0);
                return empty;
        }

        // If segment of this node is a part
        // of given range, then return the
        // frequency every character of the segment
        if (st >= l && en <= r)
        {
                return seg_tree[pos];
        }

        // getting mid of st and en
        int mid = st + en >> 1;

        //storing answer of left child n v1
        vector <int> v1 = get(2 * pos, l,
```

```
                                                                r, st,
mid);
        //storing answer of left child n v1
        vector <int> v2 = get(2 * pos + 1,
                                                        l, r, mid + 1, en);

        //returning the addition of both vectors
        return add_two_result(v1, v2);

}

// A recursive function that update
// frequency of new and old character
void update(int pos, int indx, int st,
                                        int en, char pre, char cur) {

        // If segment of this node is
        // outside the given range
        if (indx > en || indx < st) return;

        // Subtract frequency of previous character
        seg_tree[pos][pre - 'a']--;

        // Add frequency of new character
        seg_tree[pos][cur - 'a']++;

        if (st != en)
        {
                int mid = st + en >> 1;

                // update left child
                update(2 * pos, indx, st,
                                                        mid, pre, cur);

                // update right child
                update(2 * pos + 1, indx,
                                        mid + 1, en, pre, cur);
        }

}

// Utility function to
// initialize seg_tree vector
void initialize()
{
        for (int i = 0; i < N; i++)
                seg_tree[i].resize(26, 0);
}
```

```
int count_frequency(string s, vector <int> v)
{
	int ans = 1;
	// multiplying frequency of all
	// characters in string hard
	for (int i = 0; s[i]; i++)
		ans = (ans * v[s[i] - 'a']) % mod;

	return ans;
}

// Driver Code
int main()
{
	int n, q, ans;
	vector <int> v;

	n = 15;
	str = "haardhhardrddrd";

	//initialize and build the seg_tree vector
	initialize();
	build(1, 1, n);
	string s = "hard";

	// getting frequency of all
	// characters from 1 to 5
	v = get(1, 1, 5, 1, n);

	// Calling count_frequency
	ans = count_frequency(s, v);

	cout << ans << '\n';

	// Updating character at index 3
	update(1, 3, 1, n, str[2], 'x');
	str[2] = 'x';

	// getting frequency of all
	// characters from 1 to 5
	v = get(1, 1, 5, 1, n);

	//calling count_frequency
	ans = count_frequency(s, v);

	cout << ans << '\n';

	return 0;
```

```
}
```

Lazy Propagation continued

What if there are updates on a range of array indexes?
For example, add 10 to all values at indexes from 2 to 7 in the array. The above update has to be called for every index from 2 to 7. We can avoid multiple calls by writing a function updateRange() that updates nodes accordingly.

C++

```
/* Function to update segment tree for range update in input
array.
        si -> index of current node in segment tree
        ss and se -> Starting and ending indexes of elements for
                                which current nodes stores sum.
        us and ue -> starting and ending indexes of update query
        diff -> which we need to add in the range us to ue */
void updateRangeUtil(int si, int ss, int se, int us, int ue, int diff){
        if (ss > se || ss > ue || se < us)
                return;

        if (ss == se) {
                tree[si] += diff;
                return;
        }

        int mid = (ss + se) / 2;
        updateRangeUtil(si * 2 + 1, ss, mid, us, ue, diff);
        updateRangeUtil(si * 2 + 2, mid + 1, se, us, ue, diff);

        tree[si] = tree[si * 2 + 1] + tree[si * 2 + 2];
}
```

Lazy Propagation – An optimization to make range updates faster
When there are many updates and updates are done on a range, we can postpone some updates (avoid recursive calls in the update) and do those updates only when required. Please remember that a node in the segment tree stores or represents the result of a query for a range of indexes. And if this node's range lies within the update operation range, then all descendants of the node must also be updated. For example, consider the node with value 27 in the above diagram, this node stores the sum of values at indexes from 3 to 5. If our update query is for the range 2 to 5, then we need to update this node and all descendants of this node. With Lazy propagation, we update only nodes with value 27 and

postpone updates to their children by storing this update information in separate nodes called lazy nodes or values. We create an array lazy[] which represents a lazy node. The size of lazy[] is the same as the array that represents the segment tree, which is tree[] in the below code.
The idea is to initialize all elements of lazy[] as 0. A value 0 in lazy[i] indicates that there are no pending updates on node i in the segment tree. A non-zero value of lazy[i] means that this amount needs to be added to node i in the segment tree before making any query to the node.
Below is the modified update method.

```
// To update segment tree for change in array
// values at array indexes from us to ue.
updateRange(us, ue)
1) If current segment tree node has any pending
   update, then first add that pending update to
   current node.
2) If current node's range lies completely in
   update query range.
....a) Update current node
....b) Postpone updates to children by setting
       lazy value for children nodes.
3) If current node's range overlaps with update
   range, follow the same approach as above simple
   update.
...a) Recur for left and right children.
...b) Update current node using results of left
      and right calls.
```

Is there any change in Query Function also?
Since we have changed update to postpone its operations, there may be problems if a query is made to a node that is yet to be updated. So we need to update our query method also which is getSumUtil. The getSumUtil() now first checks if there is a pending update and if there is, then updates the node. Once it makes sure that the pending update is done, it works the same as the previous getSumUtil().
Below are programs to demonstrate the working of Lazy Propagation.

C++

```cpp
#include <iostream>
using namespace std;
#define MAX 1000

int tree[MAX] = {0};
int lazy[MAX] = {0};
```

```
/* si -> index of current node in segment tree
       ss and se -> Starting and ending indexes of elements for
                            which current nodes stores sum.
       us and ue -> starting and ending indexes of update query
       diff -> which we need to add in the range us to ue */
void updateRangeUtil(int si, int ss, int se, int us, int ue, int diff){
       if (lazy[si] != 0){
              tree[si] += (se-ss+1)*lazy[si];

              if (ss != se){
                     lazy[si*2 + 1] += lazy[si];
                     lazy[si*2 + 2] += lazy[si];
              }

              // Set the lazy value for current node as 0 as it
              // has been updated
              lazy[si] = 0;
       }

       // out of range
       if (ss>se || ss>ue || se<us)
              return ;

       // Current segment is fully in range
       if (ss>=us && se<=ue){
              // Add the difference to current node
              tree[si] += (se-ss+1)*diff;

              // same logic for checking leaf node or not
              if (ss != se){
                     lazy[si*2 + 1] += diff;
                     lazy[si*2 + 2] += diff;
              }
              return;
       }

       int mid = (ss+se)/2;
       updateRangeUtil(si*2+1, ss, mid, us, ue, diff);
       updateRangeUtil(si*2+2, mid+1, se, us, ue, diff);

       tree[si] = tree[si*2+1] + tree[si*2+2];
}

// Function to update a range of values in segment
// tree
/* us and eu -> starting and ending indexes of update query
       ue -> ending index of update query
       diff -> which we need to add in the range us to ue */
void updateRange(int n, int us, int ue, int diff){
```

```
        updateRangeUtil(0, 0, n-1, us, ue, diff);
}


/* A recursive function to get the sum of values in given
        range of the array. The following are parameters for
        this function.
        si --> Index of current node in the segment tree.
                Initially 0 is passed as root is always at'
                index 0
        ss & se --> Starting and ending indexes of the
                                segment represented by current node,
                                i.e., tree[si]
        qs & qe --> Starting and ending indexes of query
                                range */
int getSumUtil(int ss, int se, int qs, int qe, int si){
        if (lazy[si] != 0){
                tree[si] += (se-ss+1)*lazy[si];

                if (ss != se){
                        lazy[si*2+1] += lazy[si];
                        lazy[si*2+2] += lazy[si];
                }
                lazy[si] = 0;
        }

        if (ss>se || ss>qe || se<qs)
                return 0;
        if (ss>=qs && se<=qe)
                return tree[si];

        int mid = (ss + se)/2;
        return getSumUtil(ss, mid, qs, qe, 2*si+1) +
                getSumUtil(mid+1, se, qs, qe, 2*si+2);
}

// Return sum of elements in range from index qs (query
// start) to qe (query end). It mainly uses getSumUtil()
int getSum(int n, int qs, int qe){
        if (qs < 0 || qe > n-1 || qs > qe){
                cout <<"Invalid Input";
                return -1;
        }

        return getSumUtil(0, n-1, qs, qe, 0);
}

// A recursive function that constructs Segment Tree for
// array[ss..se]. si is index of current node in segment
```

```
// tree st.
void constructSTUtil(int arr[], int ss, int se, int si){
	if (ss > se)
		return ;

	if (ss == se){
		tree[si] = arr[ss];
		return;
	}
	int mid = (ss + se)/2;
	constructSTUtil(arr, ss, mid, si*2+1);
	constructSTUtil(arr, mid+1, se, si*2+2);

	tree[si] = tree[si*2 + 1] + tree[si*2 + 2];
}

/* Function to construct segment tree from given array.
This function allocates memory for segment tree and
calls constructSTUtil() to fill the allocated memory */
void constructST(int arr[], int n){
	constructSTUtil(arr, 0, n-1, 0);
}


int main(){
	int arr[] = {1, 3, 5, 7, 9, 11};
	int n = sizeof(arr)/sizeof(arr[0]);

	constructST(arr, n);

	cout <<"Sum of values in given range = "<< getSum(n, 1, 3);

	updateRange(n, 1, 5, 10);

	cout <<"\nUpdated sum of values in given range = "<<
			getSum( n, 1, 3);

	return 0;
}
```

Time Complexity: O(n)

Auxiliary Space: O(MAX)

Fenwick Tree

Fenwick Tree Introduction and Binary Indexed Tree

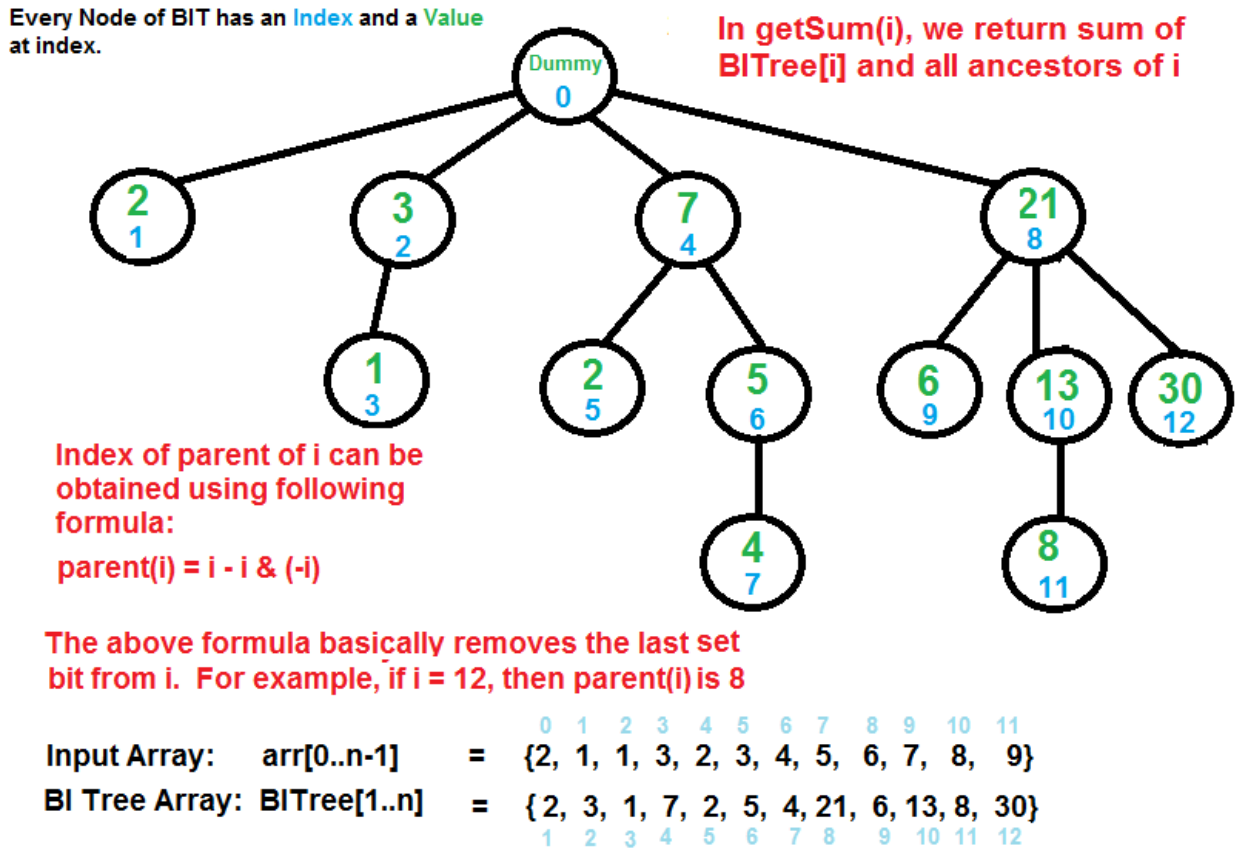
Fenwick Tree Introduction

Let us consider the following problem to understand Binary Indexed Tree.
We have an array arr[0 . . . n-1]. We would like to
1 Compute the sum of the first i elements.
2 Modify the value of a specified element of the array arr[i] = x where 0 <= i <= n-1.
A **simple solution** is to run a loop from 0 to i-1 and calculate the sum of the elements. To update a value, simply do arr[i] = x. The first operation takes O(n) time and the second operation takes O(1) time. Another simple solution is to create an extra array and store the sum of the first i-th elements at the i-th index in this new array. The sum of a given range can now be calculated in O(1) time, but the update operation takes O(n) time now. This works well if there are a large number of query operations but a very few number of update operations.
Could we perform both the query and update operations in O(log n) time?
One efficient solution is to use Segment Tree that performs both operations in O(Logn) time.
An alternative solution is Binary Indexed Tree, which also achieves O(Logn) time complexity for both operations. Compared with Segment Tree, Binary Indexed Tree requires less space and is easier to implement..
Representation
Binary Indexed Tree is represented as an array. Let the array be BITree[]. Each node of the Binary Indexed Tree stores the sum of some elements of the input array. The size of the Binary Indexed Tree is equal to the size of the input array, denoted as n. In the code below, we use a size of n+1 for ease of implementation.
Construction
We initialize all the values in BITree[] as 0. Then we call update() for all the indexes, the update() operation is discussed below.
Operations

getSum(x): Returns the sum of the sub-array arr[0,…,x]

// Returns the sum of the sub-array arr[0,…,x] using BITree[0..n], which is constructed from arr[0..n-1]
1) Initialize the output sum as 0, the current index as x+1.
2) Do following while the current index is greater than 0.
…a) Add BITree[index] to sum

...b) Go to the parent of BITree[index]. The parent can be obtained by removing the last set bit from the current index, i.e., index = index – (index & (-index))
3) Return sum.

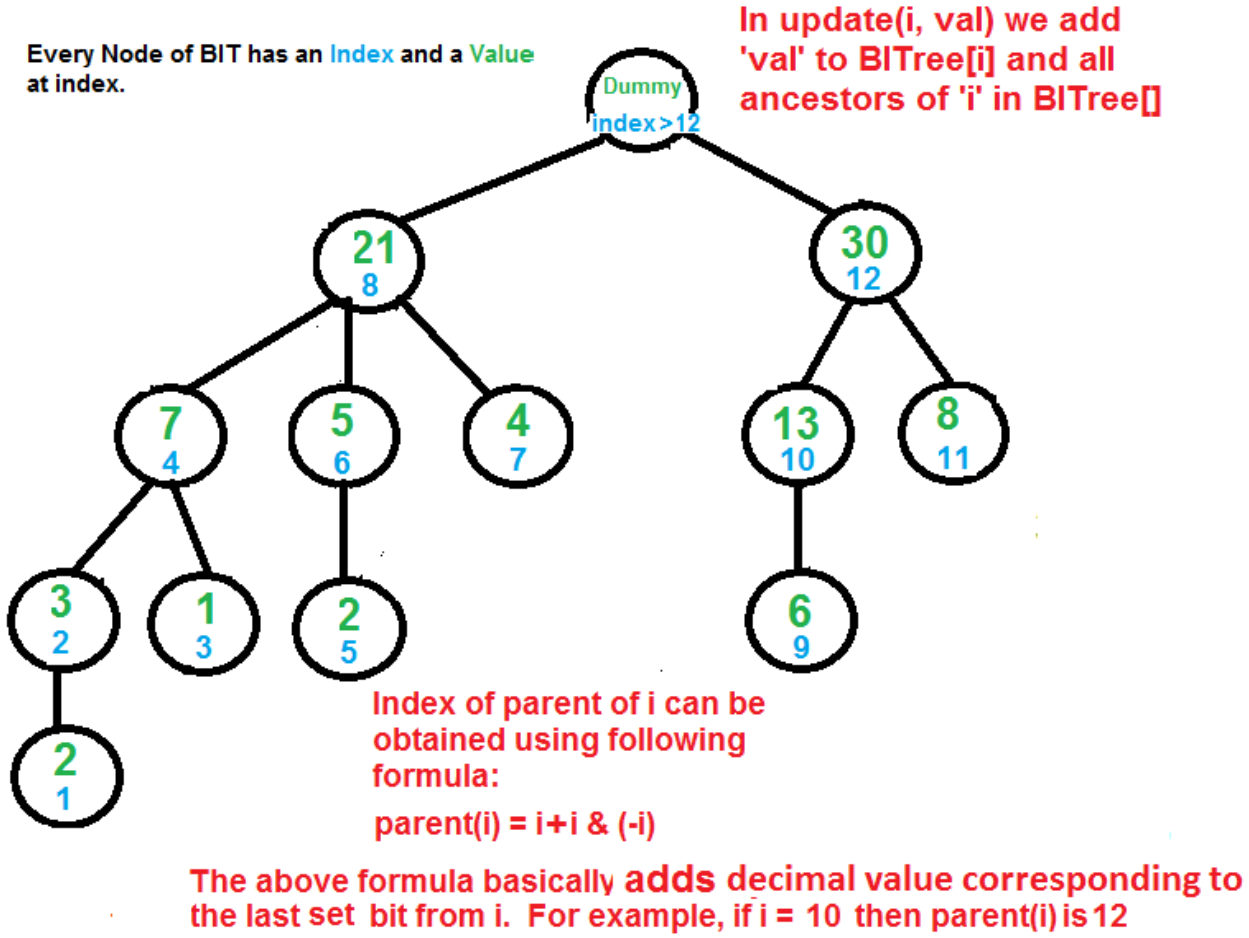


View of Binary Indexed Tree to unserstand getSum() operation

The diagram above provides an example of how getSum() is working. Here are some important observations.
BITree[0] is a dummy node.
BITree[y] is the parent of BITree[x], if and only if y can be obtained by removing the last set bit from the binary representation of x, that is y = x – (x & (-x)).
The child node BITree[x] of the node BITree[y] stores the sum of the elements between y(inclusive) and x(exclusive): arr[y,...,x).

update(x, val): Updates the Binary Indexed Tree (BIT) by performing arr[index] += val
// Note that the update(x, val) operation will not change arr[]. It only makes changes to BITree[]

1) Initialize the current index as x+1.
2) Do the following while the current index is smaller than or equal to n.
...a) Add the val to BITree[index]
...b) Go to next element of BITree[index]. The next element can be obtained by incrementing the last set bit of the current index, i.e., index = index + (index & (-index))



Contents of arr[] and BITree[] are same as above diagram for getSum()

View of Binary Indexed Tree to understand update() operation

The update function needs to make sure that all the BITree nodes which contain arr[i] within their ranges being updated. We loop over such nodes in the BITree by repeatedly adding the decimal number corresponding to the last set bit of the current index.

How does Binary Indexed Tree work?
The idea is based on the fact that all positive integers can be represented as the sum of

powers of 2. For example 19 can be represented as 16 + 2 + 1. Every node of the BITree stores the sum of n elements where n is a power of 2. For example, in the first diagram above (the diagram for getSum()), the sum of the first 12 elements can be obtained by the sum of the last 4 elements (from 9 to 12) plus the sum of 8 elements (from 1 to 8). The number of set bits in the binary representation of a number n is O(Logn). Therefore, we traverse at-most O(Logn) nodes in both getSum() and update() operations. The time complexity of the construction is O(nLogn) as it calls update() for all n elements.
Implementation:
Following are the implementations of Binary Indexed Tree.

C++

```
// C++ code to demonstrate operations of Binary Index Tree
#include <iostream>

using namespace std;

/*              n --> No. of elements present in input array.
        BITree[0..n] --> Array that represents Binary Indexed Tree.
        arr[0..n-1] --> Input array for which prefix sum is evaluated. */

// Returns sum of arr[0..index]. This function assumes
// that the array is preprocessed and partial sums of
// array elements are stored in BITree[].
int getSum(int BITree[], int index)
{
        int sum = 0; // Initialize result

        // index in BITree[] is 1 more than the index in arr[]
        index = index + 1;

        // Traverse ancestors of BITree[index]
        while (index>0)
        {
                // Add current element of BITree to sum
                sum += BITree[index];

                // Move index to parent node in getSum View
                index -= index & (-index);
        }
        return sum;
}

// Updates a node in Binary Index Tree (BITree) at given index
// in BITree. The given value 'val' is added to BITree[i] and
// all of its ancestors in tree.
```

```
void updateBIT(int BITree[], int n, int index, int val)
{
        // index in BITree[] is 1 more than the index in arr[]
        index = index + 1;

        // Traverse all ancestors and add 'val'
        while (index <= n)
        {
        // Add 'val' to current node of BI Tree
        BITree[index] += val;

        // Update index to that of parent in update View
        index += index & (-index);
        }
}

// Constructs and returns a Binary Indexed Tree for given
// array of size n.
int *constructBITree(int arr[], int n)
{
        // Create and initialize BITree[] as 0
        int *BITree = new int[n+1];
        for (int i=1; i<=n; i++)
                BITree[i] = 0;

        // Store the actual values in BITree[] using update()
        for (int i=0; i<n; i++)
                updateBIT(BITree, n, i, arr[i]);

        // Uncomment below lines to see contents of BITree[]
        //for (int i=1; i<=n; i++)
        //      cout << BITree[i] << " ";

        return BITree;
}


// Driver program to test above functions
int main()
{
        int freq[] = {2, 1, 1, 3, 2, 3, 4, 5, 6, 7, 8, 9};
        int n = sizeof(freq)/sizeof(freq[0]);
        int *BITree = constructBITree(freq, n);
        cout << "Sum of elements in arr[0..5] is "
                << getSum(BITree, 5);

        // Let use test the update operation
        freq[3] += 6;
        updateBIT(BITree, n, 3, 6); //Update BIT for above change in arr[]
```

```
        cout << "\nSum of elements in arr[0..5] after update is "
               << getSum(BITree, 5);

        return 0;
}
```

Binary Indexed Tree | Count Inversions

Inversion count in an Array using Binary Indexed Tree

Inversion Count for an array indicates – how far (or close) the array is from being sorted. If the array is already sorted then the inversion count is 0. If the array is sorted in the reverse order that inversion count is the maximum.

Two elements a[i] and a[j] form an inversion if a[i] > a[j] and i < j. For simplicity, we may assume that all elements are unique.

Example:

Input: *arr[] = {8, 4, 2, 1}*
Output: *6*
Explanation: *Given array has six inversions: (8,4), (4,2), (8,2), (8,1), (4,1), (2,1).*

Input: *arr[] = {3, 1, 2}*
Output: *2*
Explanation: *Given array has two inversions: (3,1), (3,2).*

Solution using BIT of size Θ(maxElement):

- **Approach:** Traverse through the array and for every index find the number of smaller elements on the right side of the array. This can be done using BIT. Sum up the counts for all indexes in the array and print the sum.
- **Background on BIT:**
    1. BIT basically supports two operations for an array arr[] of size n:
        1. Sum of elements till arr[i] in O(log n) time.
        2. Update an array element in O(log n) time.
    2. BIT is implemented using an array and works in form of trees. Note that there are two ways of looking at BIT as a tree.
        1. The sum operation where parent of index x is "x – (x & -x)".
        2. The update operation where parent of index x is "x + (x & -x)".
- **Algorithm:**
    1. Create a BIT, to find the count of the smaller elements in the BIT for a given number and also a variable *result = 0*.
    2. Traverse the array from end to start.

3. For every index check how many numbers less than the current element are present in BIT and add it to the result
4. To get the count of smaller elements, getSum() of BIT is used.
5. In his basic idea, BIT is represented as an array of size equal to maximum element plus one. So that elements can be used as an index.
6. After that we add the current element to the BIT[] by doing an update operation that updates the count of the current element from 0 to 1, and therefore updates ancestors of the current element in BIT.

Below is the implementation of the above approach:

C++

```
// C++ program to count inversions using Binary Indexed Tree
#include<bits/stdc++.h>
using namespace std;

// Returns sum of arr[0..index]. This function assumes
// that the array is preprocessed and partial sums of
// array elements are stored in BITree[].
int getSum(int BITree[], int index)
{
        int sum = 0; // Initialize result

        // Traverse ancestors of BITree[index]
        while (index > 0)
        {
                // Add current element of BITree to sum
                sum += BITree[index];

                // Move index to parent node in getSum View
                index -= index & (-index);
        }
        return sum;
}

// Updates a node in Binary Index Tree (BITree) at given index
// in BITree. The given value 'val' is added to BITree[i] and
// all of its ancestors in tree.
void updateBIT(int BITree[], int n, int index, int val)
{
        // Traverse all ancestors and add 'val'
        while (index <= n)
        {
        // Add 'val' to current node of BI Tree
        BITree[index] += val;

        // Update index to that of parent in update View
        index += index & (-index);
        }
```

```
}

// Returns inversion count arr[0..n-1]
int getInvCount(int arr[], int n)
{
	int invcount = 0; // Initialize result

	// Find maximum element in arr[]
	int maxElement = 0;
	for (int i=0; i<n; i++)
		if (maxElement < arr[i])
			maxElement = arr[i];

	// Create a BIT with size equal to maxElement+1 (Extra
	// one is used so that elements can be directly be
	// used as index)
	int BIT[maxElement+1];
	for (int i=1; i<=maxElement; i++)
		BIT[i] = 0;

	// Traverse all elements from right.
	for (int i=n-1; i>=0; i--){
		invcount += getSum(BIT, arr[i]-1);

		updateBIT(BIT, maxElement, arr[i], 1);
	}

	return invcount;
}

int main(){
	int arr[] = {8, 4, 2, 1};
	int n = sizeof(arr)/sizeof(int);
	cout << "Number of inversions are : " << getInvCount(arr,n);
	return 0;
}
```

Time Complexity :- The update function and getSum function runs for O(log(maximumelement)). The getSum function has to be run for every element in the array. So overall time complexity is : O(nlog(maximumelement)).
Auxiliary space : O(maxElement), space required for the BIT is an array of the size of the largest element.

Better solution using BIT of size Θ(n):

Approach: Traverse through the array and for every index find the number of smaller elements on its right side of the array. This can be done using BIT. Sum up the counts for all indexes in the array and print the sum. The approach remains the same but the problem

with the previous approach is that it doesn't work for negative numbers as the index cannot be negative. The idea is to convert the given array to an array with values from 1 to n and the relative order of smaller and greater elements remains the same.

Example:

```
arr[] = {7, -90, 100, 1}
It gets  converted to,
arr[] = {3, 1, 4 ,2 }
as -90 < 1 < 7 < 100.
```

Explanation: Make a BIT array of a number of elements instead of a maximum element. Changing element will not have any change in the answer as the greater elements remain greater and at the same position.

Algorithm:

1. Create a BIT, to find the count of the smaller elements in the BIT for a given number and also a variable *result = 0*.
2. The previous solution does not work for arrays containing negative elements. So, convert the array into an array containing relative numbering of elements,i.e make a copy of the original array and then sort the copy of the array and replace the elements in the original array with the indices of the same elements in the sorted array.
   For example, if the array is {-3, 2, 0} then the array gets converted to {1, 3, 2}
3. Traverse the array from end to start.
4. For every index check how many numbers less than the current element are present in BIT and add it to the result.

C++

```cpp
// C++ program to count inversions using Binary Indexed Tree
#include<bits/stdc++.h>
using namespace std;

// Returns sum of arr[0..index]. This function assumes
// that the array is preprocessed and partial sums of
// array elements are stored in BITree[].
int getSum(int BITree[], int index)
{
        int sum = 0; // Initialize result

        // Traverse ancestors of BITree[index]
        while (index > 0)
        {
                // Add current element of BITree to sum
                sum += BITree[index];

                // Move index to parent node in getSum View
```

```
                index -= index & (-index);
        }
        return sum;
}

// Updates a node in Binary Index Tree (BITree) at given index
// in BITree. The given value 'val' is added to BITree[i] and
// all of its ancestors in tree.
void updateBIT(int BITree[], int n, int index, int val)
{
        // Traverse all ancestors and add 'val'
        while (index <= n)
        {
        // Add 'val' to current node of BI Tree
        BITree[index] += val;

        // Update index to that of parent in update View
        index += index & (-index);
        }
}

// Converts an array to an array with values from 1 to n
// and relative order of smaller and greater elements remains
// same. For example, {7, -90, 100, 1} is converted to
// {3, 1, 4 ,2 }
void convert(int arr[], int n)
{
        // Create a copy of arrp[] in temp and sort the temp array
        // in increasing order
        int temp[n];
        for (int i=0; i<n; i++)
                temp[i] = arr[i];
        sort(temp, temp+n);

        // Traverse all array elements
        for (int i=0; i<n; i++)
        {
                // lower_bound() Returns pointer to the first element
                // greater than or equal to arr[i]
                arr[i] = lower_bound(temp, temp+n, arr[i]) - temp + 1;
        }
}

// Returns inversion count arr[0..n-1]
int getInvCount(int arr[], int n)
{
        int invcount = 0; // Initialize result

        // Convert arr[] to an array with values from 1 to n and
```

```
        // relative order of smaller and greater elements remains
        // same. For example, {7, -90, 100, 1} is converted to
        // {3, 1, 4 ,2 }
        convert(arr, n);

        // Create a BIT with size equal to maxElement+1 (Extra
        // one is used so that elements can be directly be
        // used as index)
        int BIT[n+1];
        for (int i=1; i<=n; i++)
                BIT[i] = 0;

        // Traverse all elements from right.
        for (int i=n-1; i>=0; i--)
        {
                // Get count of elements smaller than arr[i]
                invcount += getSum(BIT, arr[i]-1);

                // Add current element to BIT
                updateBIT(BIT, n, arr[i], 1);
        }

        return invcount;
}

// Driver program
int main()
{
        int arr[] = {8, 4, 2, 1};
        int n = sizeof(arr)/sizeof(int);
        cout << "Number of inversions are : " << getInvCount(arr,n);
        return 0;
}
```

Time Complexity: The update function and getSum function runs for O(log(n)). The getSum function has to be run for every element in the array. So overall time complexity is O(nlog(n)).
Auxiliary Space: O(n).
The space required for the BIT is an array of the size n.